

Essential Coding Theory

Venkatesan Guruswami Atri Rudra¹ Madhu Sudan

October 3, 2023

¹Department of Computer Science and Engineering, University at Buffalo, SUNY. Work supported by NSF CAREER grant CCF-0844796.

Foreword

This book is based on lecture notes from coding theory courses taught by Venkatesan Guruswami at University at Washington and CMU; by Atri Rudra at University at Buffalo, SUNY and by Madhu Sudan at Harvard and MIT.

This version is dated **October 3, 2023**. For the latest version, please go to

<http://www.cse.buffalo.edu/faculty/atri/courses/coding-theory/book/>

The material in this book is supported in part by the National Science Foundation under CAREER grant CCF-0844796. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation (NSF).



©Venkatesan Guruswami, Atri Rudra, Madhu Sudan, 2019.

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License. To view a copy of this license, visit

<http://creativecommons.org/licenses/by-nc-nd/3.0/> or send a letter to Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, USA.

Contents

1 The Fundamental Question	3
1.1 Overview	3
1.2 Some Definitions and Codes	5
1.3 Error Correction	7
1.4 Distance of a Code	12
1.5 Hamming Code	16
1.6 Hamming Bound	19
1.7 Generalized Hamming Bound	20
1.8 Family of codes	22
1.9 Exercises	24
1.10 Bibliographic Notes	26
I The Basics	27
2 A Look at Some Nicely Behaved Codes: Linear Codes	31
2.1 Groups and Finite Fields	31
2.2 Vector Spaces and Linear Subspaces	33
2.3 Linear Codes and Basic Properties	37
2.4 Hamming Codes	39
2.5 Efficient Decoding of Hamming codes	41
2.6 Dual of a Linear Code	43
2.7 Exercises	44
2.8 Bibliographic Notes	51
3 Probability as Fancy Counting and the q-ary Entropy Function	53
3.1 A Crash Course on Probability	53
3.2 The Probabilistic Method	59
3.3 The q -ary Entropy Function	60
3.4 Exercises	67
3.5 Bibliographic Notes	67

II The Combinatorics	69
4 What Can and Cannot Be Done-I	73
4.1 Asymptotic Version of the Hamming Bound	73
4.2 Gilbert-Varshamov Bound	74
4.3 Singleton Bound	79
4.4 Plotkin Bound	81
4.5 Exercises	88
4.6 Bibliographic Notes	92
5 The Greatest Code of Them All: Reed-Solomon Codes	93
5.1 Polynomials and Finite Fields	93
5.2 Reed-Solomon Codes	98
5.3 Maximum Distance Separable Codes and Properties	100
5.4 Exercises	102
5.5 Bibliographic Notes	111
6 What Happens When the Noise is Stochastic: Shannon's Theorem	113
6.1 Overview of Shannon's Result	113
6.2 Shannon's Noise Model	114
6.3 Shannon's Result for BSC _{p}	117
6.4 Hamming vs. Shannon	126
6.5 Exercises	126
6.6 Bibliographic Notes	130
7 Bridging the Gap Between Shannon and Hamming: List Decoding	131
7.1 Hamming versus Shannon: part II	131
7.2 List Decoding	133
7.3 The Johnson Bound	135
7.4 List-Decoding Capacity	139
7.5 List Decoding from Random Errors	143
7.6 Exercises	147
7.7 Bibliographic Notes	155
8 What Cannot be Done-II	157
8.1 Elias-Bassalygo bound	157
8.2 The MRRW bound: A better upper bound	159
8.3 A Breather	159
8.4 Exercises	161
8.5 Bibliographic Notes	162

III The Codes	163
9 When Polynomials Save the Day: Polynomial Based Codes	165
9.1 The generic construction	166
9.2 The low degree case	167
9.3 The case of the binary field	169
9.4 The general case	170
9.5 Exercises	177
9.6 Bibliographic Notes	178
10 From Large to Small Alphabets: Code Concatenation	179
10.1 Code Concatenation: The basic idea	180
10.2 Zyablov Bound	182
10.3 Advanced Concatenation and Strongly Explicit Constructions	184
10.4 Summary of concatenation	187
10.5 Exercises	187
10.6 Bibliographic Notes	190
11 When Graphs Come to the Party: Expander Codes	193
11.1 Graphs	194
11.2 Expander Graphs	195
11.3 Basic Expander Codes	203
11.4 Codes from weaker expanders	207
11.5 Distance Amplification	210
11.6 Existence of lossless expanders: Proof of Theorem 11.2.3	218
11.7 Exercises	220
11.8 Bibliographic notes	223
IV The Algorithms	225
12 Efficient Decoding of Reed-Solomon Codes	227
12.1 Unique decoding of Reed-Solomon codes	227
12.2 List Decoding Reed-Solomon Codes	233
12.3 Extensions	250
12.4 Exercises	251
12.5 Bibliographic Notes	256
13 Decoding Reed-Muller Codes	259
13.1 A natural decoding algorithm	259
13.2 Majority Logic Decoding	266
13.3 Decoding by reduction to Reed-Solomon decoding	268
13.4 Exercises	274
13.5 Bibliographic Notes	276

14 Decoding Concatenated Codes	279
14.1 A Natural Decoding Algorithm	279
14.2 Decoding From Errors and Erasures	282
14.3 Generalized Minimum Distance Decoding	283
14.4 Bibliographic Notes	287
15 Efficiently Achieving the Capacity of the BSC_p	289
15.1 Achieving capacity of BSC_p	289
15.2 Decoding Error Probability	292
15.3 The Inner Code	292
15.4 The Outer Code	293
15.5 Discussion and Bibliographic Notes	295
16 Information Theory Strikes Back: Polar Codes	297
16.1 Achieving Gap to Capacity	298
16.2 Reduction to Linear Compression	299
16.3 The Polarization Phenomenon	300
16.4 Polar codes, Encoder and Decoder	306
16.5 Analysis: Speed of Polarization	312
16.6 Entropic Calculations	324
16.7 Summary and additional information	327
16.8 Exercises	328
16.9 Bibliographic Notes	329
17 Efficiently Achieving List Decoding Capacity	331
17.1 Folded Reed-Solomon Codes	331
17.2 List Decoding Folded Reed-Solomon Codes: I	335
17.3 List Decoding Folded Reed-Solomon Codes: II	338
17.4 Bibliographic Notes and Discussion	349
17.5 Exercises	352
18 Fast encoding: linear time encodable codes	357
18.1 Overview of the construction	357
18.2 Low-density Error-Reduction Codes	358
18.3 The error-correcting code: Recursive construction	361
18.4 Analysis	362
18.5 Exercises	364
18.6 Bibliographic Notes	364
19 Recovering very locally: Locally Recoverable Codes	365
19.1 Context	365
19.2 Definition of Locally Recoverable Codes	366
19.3 A simple construction for message symbol LRCs	367
19.4 A Singleton-type bound	369

19.5 An LRC meeting the Singleton type bound	371
19.6 Exercises	373
19.7 Bibliographic notes	375
V The Applications	377
20 Cutting Data Down to Size: Hashing	379
20.1 Why Should You Care About Hashing?	379
20.2 Avoiding Hash Collisions	381
20.3 Almost Universal Hash Function Families and Codes	384
20.4 Data Possession Problem	385
20.5 Bibliographic Notes	389
21 Securing Your Fingerprints: Fuzzy Vaults	391
21.1 Some quick background on fingerprints	391
21.2 The Fuzzy Vault Problem	393
21.3 The Final Fuzzy Vault	396
21.4 Bibliographic Notes	398
22 Finding Defectives: Group Testing	399
22.1 Formalization of the problem	399
22.2 Bounds on $t^a(d, N)$	401
22.3 Bounds on $t(d, N)$	402
22.4 Coding Theory and Disjunct Matrices	406
22.5 An Application in Data Stream Algorithms	409
22.6 Summary of best known bounds	414
22.7 Exercises	415
22.8 Bibliographic Notes	417
23 Complexity of Coding Problems	419
23.1 Nearest Codeword Problem (NCP)	420
23.2 Decoding with Preprocessing	421
23.3 Approximate NCP	424
23.4 Distance bounded decoding	427
23.5 Minimum distance problem	431
23.6 Conclusions	432
23.7 Exercises	433
23.8 Bibliographic Notes	437
24 Giving Computational Complexity a helping hand	439
24.1 Communication Complexity	439
24.2 Derandomization and Pseudorandomness	444
24.3 Hardcore Predicates	454

24.4 Average case complexity	457
24.5 Exercises	460
24.6 Bibliographic Notes	467
A Notation Table	481
B Some Useful Facts	483
B.1 Some Useful Inequalities	483
B.2 Some Useful Identities and Bounds	485
C Background on Asymptotic notation, Algorithms and Complexity	487
C.1 Asymptotic Notation	487
C.2 Bounding Algorithm run time	489
C.3 Randomized Algorithms	493
C.4 Efficient Algorithms	496
C.5 More on intractability	500
C.6 Exercises	502
C.7 Bibliographic Notes	505
D Basic Algebraic Algorithms	507
D.1 Executive Summary	507
D.2 Groups, Rings, Fields	507
D.3 Polynomials	508
D.4 Vector Spaces	510
D.5 Finite Fields	512
D.6 Algorithmic aspects of Finite Fields	518
D.7 Algorithmic aspects of Polynomials	520
D.8 Exercises	526
E Some Information Theory Essentials	529
E.1 Entropy	529
E.2 Joint and conditional entropy	531
E.3 Mutual information	534

List of Figures

1.1	Decoding for Akash English, one gets “I need little little (trail)mix.”	3
1.2	Coding process	9
1.3	Bad example for unique decoding.	15
1.4	Illustration for proof of Hamming Bound	20
3.1	The q -ary Entropy Function	61
4.1	The Hamming and Gilbert-Varshamov (GV) bounds for binary codes	74
4.2	An illustration of Gilbert’s greedy algorithm (Algorithm 4.2.1) for the first five iterations.	76
4.3	Construction of a new code in the proof of the Singleton bound.	80
4.4	The Hamming, GV and Singleton bound for binary codes.	80
4.5	R vs δ tradeoffs for binary codes	83
6.1	The communication process	114
6.2	Binary Symmetric Channel BSC_p	115
6.3	Binary Erasure Channel BEC_α	116
6.4	The sets $D_{\mathbf{m}}$ partition the ambient space $\{0, 1\}^n$.	118
6.5	The shell $S_{\mathbf{m}}$ of inner radius $(1 - \gamma)pn$ and outer radius $(1 + \gamma)pn$.	119
6.6	Illustration of Proof of Shannon’s Theorem	121
7.1	Bad example of unique decoding revisited	133
7.2	Comparing the Johnson Bound with Unique decoding and Singleton bounds	139
7.3	An error pattern	144
7.4	Illustration of notation used in the proof of Theorem 7.5.1	145
7.5	An error pattern in the middle of the proof	146
8.1	Bounds on R vs δ for binary codes	158
10.1	Concatenated code $C_{\text{out}} \circ C_{\text{in}}$.	181
10.2	The Zyablov bound for binary codes	183
11.1	A bipartite graph G_H	195
11.2	The ‘triangle’ graph G on the left, and its double cover G' (see Definition 11.2.13) in the middle and H ,	
11.3	The C_{in} -Coded G -Amplification of C_{out}	215
12.1	A received word in 2-D space	228

12.2	The closest polynomial to a received word	229
12.3	The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 12.2.1.	238
12.4	A received word in 2-D space for the second Reed-Solomon	239
12.5	An interpolating polynomial $Q(X, Y)$ for the received word in Figure 12.4.	240
12.6	The two polynomials that need to be output are shown in blue.	240
12.7	The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 12.2.1 and A	
12.8	Multiplicity of 1	243
12.9	Multiplicity of 2	244
12.10	Multiplicity of 3	244
12.11	A received word in 2-D space for the third Reed-Solomon	245
12.12	An interpolating polynomial $Q(X, Y)$ for the received word in Figure 12.11.	245
12.13	The five polynomials that need to be output are shown in blue.	246
14.1	Encoding and Decoding of Concatenated Codes	280
14.2	All values of $\theta \in [q_i, q_{i+1})$ lead to the same outcome	287
15.1	Efficiently achieving capacity of BSC_p .	290
15.2	Error Correction cannot decrease during “folding”	294
16.1	The 2×2 Basic Polarizing Transform. Included in red are the conditional entropies of the variables, co	
16.2	The $n \times n$ Basic Polarizing Transform defined as $P_n(\mathbf{Z}) = P_n(\mathbf{U}, \mathbf{V}) = \left(P_{\frac{n}{2}}(\mathbf{U} + \mathbf{V}), P_{\frac{n}{2}}(\mathbf{V}) \right)$. Acknowledgement	
16.3	Block structure of the Basic Polarizing Transform. Circled are a block at the 2nd level and two 2nd level	
17.1	Encoding for Reed-Solomon Codes	332
17.2	Folded Reed-Solomon code for $m = 2$.	332
17.3	Folded Reed-Solomon code for general $m \geq 1$.	332
17.4	Error pattern under unfolding	333
17.5	Error pattern under folding	334
17.6	Performance of Algorithm 17.2.1	338
17.7	An agreement in position i .	339
17.8	More agreement with a sliding window of size 2.	339
17.9	Performance of Algorithm 17.3.1	342
17.10	An upper triangular system of linear equations	343
18.1	The recursive construction of C_k . The final code $\tilde{C}_k$ is also shown.	362
21.1	The minutiae are unordered and form a set, not a vector.	393
22.1	Pick a subset S (not necessarily contiguous). Then pick a column j that is not present in S . There will	
22.2	Construction of the final matrix M_{C^*} from $M_{C_{\text{out}}}$ and $M_{C_{\text{in}}}$ from Example 22.4.3. The rows in M_{C^*} that	
E.1	Relationship between entropy, joint entropy, conditional entropy, and mutual information for two ran	

List of Tables

3.1	Uniform distribution over $\mathbb{F}_2^{2 \times 2}$ along with values of four random variables.	54
8.1	High level summary of results seen so far. An asymptotically good code has $R > 0$ and $\delta > 0$.160	
10.1	Strongly explicit binary codes that we have seen so far.	180
15.1	An overview of the results seen so far	289
15.2	Summary of properties of C_{out} and C_{in}	291

List of Algorithms

1.3.1 Error Detector for Parity Code	12
1.4.1 Naive Maximum Likelihood Decoder	14
2.5.1 Naive Decoder for Hamming Code	41
2.5.2 Decoder for Any Linear Code	42
2.5.3 Efficient Decoder for Hamming Code	42
4.2.1 Gilbert's Greedy Code Construction	75
4.5.1 $q^{O(k)}$ time algorithm to compute a code on the GV bound	90
5.1.1 Generating Irreducible Polynomial	98
12.1. Welch-Berlekamp Algorithm	231
12.2. The Basic List-Decoder for Reed-Solomon Codes	236
12.2. The Second List Decoding Algorithm for Reed-Solomon Codes	241
12.2. The Third List Decoding Algorithm for Reed-Solomon Codes	246
13.1. SIMPLE REED-MULLER DECODER	263
13.2. Majority Logic Decoder	268
13.3. REED-SOLOMON-BASED DECODER	271
14.1. Natural Decoder for $C_{\text{out}} \circ C_{\text{in}}$	280
14.3. Generalized Minimum Decoder (ver 1)	284
14.3. Generalized Minimum Decoder (ver 2)	286
14.3. Deterministic Generalized Minimum Decoder'	287
15.1. Decoder for efficiently achieving BSC_p capacity	291
16.3. POLAR COMPRESSOR($\mathbf{Z}, S$)	302
16.3. S uccessive Cancellation Decompressor SCD($\mathbf{W}, \mathbf{P}, S$)	303
16.4. BASIC POLAR ENCODER($\mathbf{Z}; n, S$)	307
16.4. B ASIC POLAR DECODER: BPD($\mathbf{W}; n, p$)	309
17.1. Decoding Folded Reed-Solomon Codes by Unfolding	333
17.2. The First List Decoding Algorithm for Folded Reed-Solomon Codes	336
17.3. The Second List Decoding Algorithm for Folded Reed-Solomon Codes	340
17.3. The Root Finding Algorithm for Algorithm 17.3.1	347
18.2. GEN-FLIP	360
18.4. LINEAR-DECODE	363
19.4. Computing disjoint S and T	370
20.4. Pre-Processing for Data Possession Verification	385
20.4. V erification for Data Possession Verification	386

20.4. D ecompression Algorithm	386
20.4. D ecompression Algorithm Using List Decoding	388
21.2. UNLOCK ₂	395
21.3. LOCK ₃	396
21.3. U NLOCK ₂	397
22.3. Decoder for Separable Matrices	404
22.3. N aive Decoder for Disjunct Matrices	406
22.5. Initialization	411
22.5. U ppdate	411
22.5. R eport Heavy Items	412
24.2. RANDOM t -SAT	446
24.2. D e-randomized RANDOM t -SAT	449
24.3. INVERT f	456
24.5. $\mathcal{B}_p^{A^+}$	465
C.2.1 Simple Search	491
C.3.1 Sampling algorithm for GAPHAMMING	495
C.3.2 An average-case algorithm for GAPHAMMING	496
C.4.1 Exponential time algorithm for MAXLINEAREQ	497
C.4.2 Reduction from MAXCUT to MAXLINEAREQ	500
D.7.1 ROOT-FIND($\mathbb{F}_q, f$)	524
D.7.2 LINEAR-ROOT-FIND($\mathbb{F}_q, g$)	524

Chapter 1

The Fundamental Question

1.1 Overview

Communication is a fundamental need of our modern lives. In fact, communication is something that humans have been doing for a long time. For simplicity, let us restrict ourselves to English. It is quite remarkable that different people speaking English can be understood pretty well: even if e.g. the speaker has an accent. This is because English has some built-in redundancy, which allows for “errors” to be tolerated. We will pick an example from one of the author’s experiences conversing with his two-year-old son, Akash. When Akash started to speak his own version of English, which we will dub “Akash English,” we got examples such as the one illustrated below:

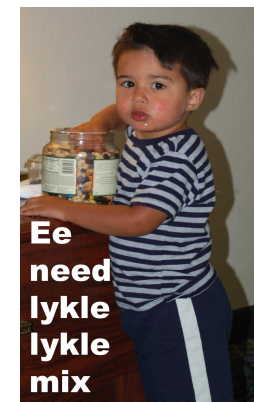


Figure 1.1: Decoding for Akash English, one gets “I need little little (trail)mix.”

With some practice Akash's parents were able to “decode” what Akash really meant. In fact, Akash could communicate even if he did not say an entire word properly and gobbled up part(s) of word(s).

The above example shows that having redundancy in a language allows for communication even in the presence of (small amounts of) differences and errors. Of course, in our modern digital world, all kinds of entities communicate (and most of the entities do not communicate in English, or any natural language for that matter). Errors are also present in the digital world, so these digital communications also use redundancy.

Error-correcting codes (henceforth, just codes) are clever ways of representing data so that one can recover the original information even if parts of it are corrupted. The basic idea is to judiciously introduce redundancy so that the original information can be recovered even when parts of the (redundant) data have been corrupted.

For example, when packets are transmitted over the Internet, some of the packets get corrupted or dropped. Packet drops are resolved by the TCP layer by a combination of sequence numbers and ACKs. To deal with data corruption, the TCP/IP protocol use a form of error correction called CRC Checksum [124]. From a theoretical point of view, the checksum is a terrible code since it does not have good error correction properties (for that matter so is English). However, on the Internet, the current dominant mode of operation is to detect errors and if errors have occurred, then ask for retransmission. This is the reason why the use of checksum has been hugely successful in the Internet. However, there are other communication applications where re-transmission is not an option. Codes are used when transmitting data over the telephone line or via cell phones. They are also used in deep space communication and in satellite broadcast (for example, TV signals are transmitted via satellite). Indeed, asking the Mars Rover to re-send an image just because it got corrupted during transmission is not an option—this is the reason that for such applications, the codes used have always been very sophisticated.

Codes also have applications in areas not directly related to communication. In particular, in the applications above, we want to communicate over space. Codes can also be used to communicate over time. For example, codes are used heavily in data storage. CDs and DVDs work fine even in presence of scratches precisely because they use codes. Codes are used in Redundant Array of Inexpensive Disks (RAID) [29] and error correcting memory [28]. Sometimes, in the Blue Screen of Death displayed by Microsoft Windows family of operating systems, you might see a line saying something along the lines of “parity check failed”—this happens when the code used in the error-correcting memory cannot recover from error(s). Also, certain consumers of memory, e.g. banks, do not want to suffer from even one bit flipping (this e.g. could mean someone's bank balance either got halved or doubled—neither of which are welcome¹). Codes are also deployed in other applications such as paper bar codes; for example, the bar code used by UPS called MaxiCode [27]. Unlike the Internet example, in all of these applications, there is no scope for “re-transmission.”

In this book, we will mainly think of codes in the communication scenario. In this framework, there is a sender who wants to send (say) k message symbols over a noisy channel. The

¹This is a bit tongue-in-cheek: in real life banks have more mechanisms to prevent one-bit flip from wreaking havoc.

sender first *encodes* the k message symbols into n symbols (called a *codeword*) and then sends it over the *channel*. The receiver gets a *received word* consisting of n symbols. The receiver then tries to *decode* and recover the original k message symbols. Thus, encoding is the process of adding redundancy and decoding is the process of removing errors.

Unless mentioned otherwise, in this book we will make the following assumption:

The sender and the receiver only communicate via the channel.^a In other words, other than some setup information about the code, the sender and the receiver do not have any other information exchange (other than of course what was transmitted over the channel). In particular, no message is more likely to be transmitted over another.

^aThe scenario where the sender and receiver have a “side-channel” is an interesting topic that has been studied but is outside the scope of this book.

The fundamental question that will occupy our attention for almost the entire book is the tradeoff between the amount of redundancy used and the number of errors that can be corrected by a code. In particular, we would like to understand:

Question 1.1.1. *How much redundancy do we need to correct a given amount of errors? (We would like to correct as many errors as possible with as little redundancy as possible.)*

Note that maximizing error correction and minimizing redundancy are contradictory goals: a code with higher redundancy should be able to tolerate a greater number of errors. By the end of this chapter, we will see a formalization of this question.

Once we determine the optimal tradeoff, we will be interested in achieving this optimal tradeoff with codes that come equipped with *efficient* encoding and decoding. (A DVD player that tells its consumer that it will recover from a scratch on a DVD by tomorrow is not exactly going to be a best-seller.) In this book, we will primarily define efficient algorithms to be ones that run in polynomial time.²

1.2 Some Definitions and Codes

To formalize Question 1.1.1, we begin with the definition of a code.

Definition 1.2.1 (Code). *A code of block length n over an alphabet Σ is a subset of Σ^n . Typically, we will use q to denote the alphabet size $|\Sigma|$.³*

²Readers unfamiliar with runtime analysis are referred to Appendix C. Coming back to the claim on efficiency—we are not claiming that this is the correct notion of efficiency in practice. However, we believe that it is a good definition as the “first cut”—quadratic or cubic time algorithms are definitely more desirable than exponential time algorithms: see Section C.4 for more on this.

³Note that q need not be a constant and can depend on n : we'll see codes in this book where this is true.

Remark 1.2.2. We note that the ambient space Σ^n can be viewed as a set of sequences, vectors or functions. In other words, we can think of a vector $(v_1, \dots, v_n) \in \Sigma^n$ as just the sequence $v_1, \dots, v_n$ (in order) or a vector tuple $(v_1, \dots, v_n)$ or as the function $f: [n] \rightarrow \Sigma$ such that $f(i) = v_i$. Sequences assume least structure on Σ and hence are most generic. Vectors work well when Σ has some structure (and in particular is what is known as a field, which we will see next chapter). Functional representation will be convenient when the set of coordinates has structure (e.g., $[n]$ may come from a finite field of size n). For now, however, the exact representation does not matter and the reader can work with representation as sequences.

We will also frequently use the following alternate way of looking at a code. Given a code $C \subseteq \Sigma^n$, with $|C| = M$, we will think of C as a mapping of the following form:

$$C: [M] \rightarrow \Sigma^n. \quad (1.1)$$

In the above equation (1.1), we have used the notation $[M]$ for any integer $M \geq 1$ to denote the set $\{1, 2, \dots, M\}$.

We will also need the notion of *dimension* of a code.

Definition 1.2.3 (Dimension of a code). Given a code $C \subseteq \Sigma^n$, its dimension is given by

$$k \stackrel{\text{def}}{=} \log_q |C|.$$

Let us begin by looking at two specific codes. Both codes are defined over $\Sigma = \{0, 1\}$ (also known as *binary codes*). In both cases $|C| = 2^4$ and we will think of each of the 16 messages as a 4 bit vector.

We first look at the so-called *parity code*, which we will denote by $C_\oplus$. Given a message $(x_1, x_2, x_3, x_4) \in \{0, 1\}^4$, its corresponding codeword is given by

$$C_\oplus(x_1, x_2, x_3, x_4) = (x_1, x_2, x_3, x_4, x_1 \oplus x_2 \oplus x_3 \oplus x_4), \quad (1.2)$$

where the $\oplus$ denotes the XOR (also known as the EXOR or Exclusive-OR) operator. In other words, the parity code appends the parity of the message bits (or takes the remainder of the sum of the message bits when divided by 2) at the end of the message. For example, the message $(1, 0, 0, 1)$ will have a 0 appended at the end while $(1, 0, 0, 0)$ will have a 1 appended at the end. Note that such a code uses the minimum amount of non-zero redundancy.

The second code we will look at is the so-called *repetition code*. This is a very natural code (and perhaps the first code one might think of). The idea is to repeat every message bit a fixed number of times. For example, we repeat each of the 4 message bits 3 times and we use $C_{3,rep}$ to denote this code. Given a message $(x_1, x_2, x_3, x_4) \in \{0, 1\}^4$, its corresponding codeword is given by

$$C_{3,rep}(x_1, x_2, x_3, x_4) = (x_1, x_1, x_1, x_2, x_2, x_2, x_3, x_3, x_3, x_4, x_4, x_4). \quad (1.3)$$

Let us now try to look at the tradeoff between the amount of redundancy and the number of errors each of these codes can correct. Even before we begin to answer the question, we need to define how we are going to measure the amount of redundancy. One natural way to define

redundancy for a code with dimension k and block length n is by their difference $n - k$. By this definition, the parity code uses the least amount of redundancy. However, one “pitfall” of such a definition is that it does not distinguish between a code with $k = 100$ and $n = 102$ and another code with dimension and block length 2 and 4, respectively. The first code uses 0.02 bits of redundancy per message bit while the second code uses 1 bit of redundancy per message bit. Thus, in the relative sense, the latter code is using more redundancy. This motivates the following notion of measuring redundancy.

Definition 1.2.4 (Rate of a code). The rate of a code with dimension k and block length n is given by

$$R \stackrel{\text{def}}{=} \frac{k}{n}.$$

Note that the higher the rate, the lesser the amount of redundancy in the code. Thus, when constructing or analyzing codes, we will be interested in lower bounding the rate of a code. (Occasionally we will also be sloppy and say that a code “has rate R ” when we really mean it “has rate at least R .”) Also note that as $k \leq n$,⁴

$$R \leq 1.$$

In other words, the rate of a code is the average amount of real information in each of the n symbols transmitted over the channel. So, in some sense, rate captures the complement of redundancy. However, for historical reasons, we will deal with the rate R (instead of the more natural $1 - R$) as our notion of redundancy. Given the above definition, $C_\oplus$ and $C_{3,rep}$ have rates of $\frac{4}{5}$ and $\frac{1}{3}$. As expected, the parity code has a higher rate than the repetition code.

We have formalized the notion of redundancy as the rate of a code as well as other parameters of a code. However, to formalize Question 1.1.1, we still need to formally define what it means to correct errors. We do so next.

1.3 Error Correction

Before we formally define error correction, we will first formally define the notion of *encoding*.

Definition 1.3.1 (Encoding function). Let $C \subseteq \Sigma^n$. An equivalent description of the code C is an injective mapping $E: [|C|] \rightarrow \Sigma^n$ called the encoding function.

Next we move to error correction. Informally, we can correct a received word if we can recover the transmitted codeword (or equivalently the corresponding message). This “reverse” process is called *decoding*.

Definition 1.3.2 (Decoding function). Let $C \subseteq \Sigma^n$ be a code. A mapping $D: \Sigma^n \rightarrow [|C|]$ is called a decoding function for C .

⁴Further, in this book, we will always consider the case $k > 0$ and $n < \infty$ and hence, we can also assume that $R > 0$.

The definition of a decoding function by itself does not give anything interesting. What we really need from a decoding function is for the function to recover the transmitted message. To understand this notion, we first need to understand the nature of errors that we aim to tackle. In particular, if a transmitter transmits $\mathbf{u} \in \Sigma^n$ and the receiver receives $\mathbf{v} \in \Sigma^n$, how do we quantify the amount of “error” that has happened during this transmission? While multiple notions are possible, the most central one, and the one we will focus on for most of this book, is based on “Hamming distance,” a notion of distance that captures how close are two given sequences $\mathbf{u}$ and $\mathbf{v}$.

Definition 1.3.3 (Hamming distance). *Given two vectors $\mathbf{u}, \mathbf{v} \in \Sigma^n$ the Hamming distance between $\mathbf{u}$ and $\mathbf{v}$, denoted by $\Delta(\mathbf{u}, \mathbf{v})$, is the number of positions in which $\mathbf{u}$ and $\mathbf{v}$ differ. We also define the relative Hamming distance, denoted $\delta(\mathbf{u}, \mathbf{v})$, to be the quantity $\delta(\mathbf{u}, \mathbf{v}) = \frac{1}{n} \Delta(\mathbf{u}, \mathbf{v})$.*

Note that the relative Hamming distance normalizes the distance so that $\delta(\mathbf{u}, \mathbf{v})$ always lies in the interval $[0, 1]$ (for every n, Σ and strings $\mathbf{u}, \mathbf{v} \in \Sigma^n$). This normalization will be useful when we study the asymptotic behavior of encoding and decoding functions, i.e., as $n \rightarrow \infty$. For now, though we will focus mostly on the (non-relative) Hamming distance.

The Hamming distance is a distance in a very formal mathematical sense: see Exercise 1.5. Note that the definition of Hamming distance depends only on the *number* of differences and not the nature of the difference. For example, consider the vectors $\mathbf{u} = 00000$ and $\mathbf{v} = 10001$. One can see that their Hamming distance is $\Delta(\mathbf{u}, \mathbf{v}) = 2$. Now consider the vector $\mathbf{w} = 01010$. Note that even though $\mathbf{v} \neq \mathbf{w}$, we again have a Hamming distance $\Delta(\mathbf{u}, \mathbf{w}) = 2$.

To return to the quantification of errors, from now on we will say that if $\mathbf{u}$ is transmitted and $\mathbf{v}$ is received then $\Delta(\mathbf{u}, \mathbf{v})$ errors occurred during transmission. This allows us to quantify the performance of an encoding/decoding function, or equivalently the underlying code as we do next.

Definition 1.3.4 (t -Error Channel). *An n -symbol t -Error Channel over the alphabet Σ is a function $\text{Ch} : \Sigma^n \rightarrow \Sigma^n$ that satisfies $\Delta(\mathbf{v}, \text{Ch}(\mathbf{v})) \leq t$ for every $\mathbf{v} \in \Sigma^n$.*

Definition 1.3.5 (Error Correcting Code). *Let $C \subseteq \Sigma^n$ be a code and let $t \geq 1$ be an integer. C is said to be a t -error-correcting code if there exists a decoding function D such that for every message $\mathbf{m} \in [|C|]$ every t -error channel Ch we have $D(\text{Ch}(C(\mathbf{m}))) = \mathbf{m}$.*

Thus, a t -error-correcting code is one where there is a decoding function that corrects any pattern of t errors. For example, consider the case when the codeword $(0, 0, 0, 0)$ is transmitted. Then a 1-error-correcting code (over the alphabet $\{0, 1\}$) should be able to decode from any of the following received words:

$$(0, 0, 0, 0), (1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1).$$

Figure 1.2 illustrates how the definitions we have examined so far interact.

We will also very briefly look at a weaker form of error recovery called *error detection*.

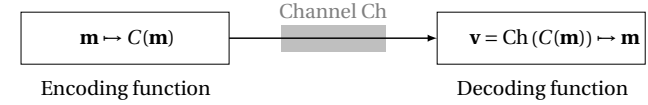


Figure 1.2: Coding process

Definition 1.3.6 (Error detection code). *Let $C \subseteq \Sigma^n$ be a code and let $t \geq 1$ be an integer. C is said to be a t -error-detecting code if there exists a detecting procedure D such that for every message $\mathbf{m}$ and every received vector $\mathbf{v} \in \Sigma^n$ satisfying $\Delta(C(\mathbf{m}), \mathbf{v}) \leq t$, it holds that D outputs a 1 if $\mathbf{v} = C(\mathbf{m})$ and 0 otherwise. In other words*

$$D(\mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v} = C(\mathbf{m}) \\ 0 & \text{otherwise} \end{cases}.$$

Thus, a t -error-detecting code is one where if the transmission has at least one error and at most t errors, then the decoding function detects the error (by outputting 0). Note that a t -error correcting code is also a t -error detecting code (but not necessarily the other way round): see Exercise 1.1. Although error detection might seem like a weak error recovery model, it is useful in settings where the receiver can ask the sender to re-send the message. For example, error detection is used quite heavily in the Internet.

Finally, we also consider a more benign model of errors referred to as “erasures,” where a symbol is merely (and explicitly) omitted from the transmission (as opposed to being replaced by some other symbol). More specifically, if a symbol is erased, then it is *replaced* by a special symbol “?” that is not a member of the alphabet Σ . For example, if $(0, 0, 0, 0)$ was transmitted and the second symbol was erased by the channel, then the vector $(0, ?, 0, 0)$ will be received.

Definition 1.3.7 (t -Erasure Channel). *An n -symbol t -Erasure Channel over the alphabet Σ is a function $\text{Ch} : \Sigma^n \rightarrow (\Sigma \cup \{?\})^n$ that satisfies $\Delta(\mathbf{v}, \text{Ch}(\mathbf{v})) \leq t$ for every $\mathbf{v} \in \Sigma^n$ (where both arguments to $\Delta(\cdot, \cdot)$ are viewed as elements of $(\Sigma \cup \{?\})^n$) and for every $i \in [n]$ such that $\mathbf{v}_i \neq \text{Ch}(\mathbf{v})_i$ we have $\text{Ch}(\mathbf{v})_i = ?$.*

A coordinate i such that $\text{Ch}(\mathbf{v})_i = ?$ is called an erasure. We may now define erasure correcting codes analogously to error-correcting codes.

Definition 1.3.8 (Erasure Correcting Code). *Let $C \subseteq \Sigma^n$ be a code and let $t \geq 1$ be an integer. C is said to be a t -erasure-correcting code if there exists a decoding function D such that for every message $\mathbf{m} \in [|C|]$ and for every t -erasure channel Ch we have $D(\text{Ch}(C(\mathbf{m}))) = \mathbf{m}$.*

With the above definitions in place, we are now ready to look at the error correcting capabilities of the codes we looked at in the previous section.

1.3.1 Error-Correcting Capabilities of Parity and Repetition Codes

In Section 1.2, we looked at examples of parity code and repetition code with the following properties:

$$C_{\oplus} : q = 2, k = 4, n = 5, R = 4/5.$$

$$C_{3,rep} : q = 2, k = 4, n = 12, R = 1/3.$$

We will start with the repetition code. To study its error-correcting capabilities, we will consider the following natural decoding function. Given a received word $\mathbf{y} \in \{0, 1\}^{12}$ (where recall the transmitted codeword is for the form $(x_1, x_1, x_1, x_2, x_2, x_2, x_3, x_3, x_3, x_4, x_4, x_4)$ for some $(x_1, x_2, x_3, x_4) \in \{0, 1\}^4$), divide it up into four consecutive blocks (y_1, y_2, y_3, y_4) where every block consists of three bits. Then, for every block y_i ($1 \leq i \leq 4$), output the majority bit as the message bit. We claim this decoding function can correct any error pattern with at most 1 error (see Exercise 1.2.) For example, if a block of 010 is received, since there are two 0's we know the original message bit was 0. In other words, we have argued the following error correcting capability of $C_{3,rep}$:

Proposition 1.3.9. $C_{3,rep}$ is a 1-error correcting code.

However, it is not too hard to see that $C_{3,rep}$ cannot correct two errors. For example, if both of the errors happen in the same block and a block in the received word is 010, then the original block in the codeword could have been either 111 or 000. Therefore in this case, no decoder can successfully recover the transmitted message.⁵

Thus, we have pin-pointed the error-correcting capabilities of the $C_{3,rep}$ code: it can correct one error, but not two or more. However, note that the argument assumed that the error positions can be located arbitrarily. In other words, we are assuming that the channel noise behaves arbitrarily (subject to a bound on the total number of errors). However, we can model the noise differently. We now briefly digress to look at this issue in slightly more detail.

Digression: Channel Noise. As was mentioned above, until now we have been assuming the following noise model, which was first studied by Hamming:

Any error pattern can occur during transmission as long as the total number of errors is bounded. Note that this means that the location as well as the nature⁶ of the errors is arbitrary.

We will frequently refer to Hamming's model as the *Adversarial Noise Model*. It is important to note that the atomic unit of error is a symbol from the alphabet. For example, if the error pattern⁷ is $(1, 0, 1, 0, 0, 0)$ and we consider the alphabet to be $\{0, 1\}$, then the pattern has two

⁵Recall we are assuming that the decoder has no side information about the transmitted message.

⁶For binary codes, there is only one kind of error: a bit flip. However, for codes over a larger alphabet, say $\{0, 1, 2\}$, 0 being converted to a 1 and 0 being converted into a 2 are both errors, but are different kinds of errors.

⁷If $\mathbf{v}$ is transmitted and $\text{Ch}(\mathbf{v})$ is received then the 'difference' between $\text{Ch}(\mathbf{v})$ and $\mathbf{v}$ is the error pattern. For binary alphabet the difference is the XOR operator.

errors (since the first and the third locations in the vector have a non-zero value, i.e. value of 1). However, if our alphabet is $\{0, 1\}^3$ (i.e. we think of the vector above as $((1, 0, 1), (0, 0, 0))$, with $(0, 0, 0)$ corresponding to the zero element in $\{0, 1\}^3$), then the pattern has only one error. Thus, by increasing the alphabet size we can also change the adversarial noise model. As the book progresses, we will see how error correction over a larger alphabet is easier than error correction over a smaller alphabet.

However, the above is not the only way to model noise. For example, we could also have following error model:

No more than 1 error can happen in any contiguous three-bit block.

First note that, for the error model above, no more than four errors can occur when a codeword in $C_{3,rep}$ is transmitted. (Recall that in $C_{3,rep}$, each of the four bits is repeated three times.) Second, note that the decoding function that takes the majority vote of each block can successfully recover the transmitted codeword for *any* error pattern, while in the worst-case noise model it could only correct at most one error. This channel model is admittedly contrived, but it illustrates the point that the error-correcting capabilities of a code (and a decoding function) are crucially dependent on the noise model.

A popular alternate noise model is to model the channel as a stochastic process. As a concrete example, let us briefly mention the *binary symmetric channel with crossover probability* $0 \leq p \leq 1$, denoted by BSC_p , which was first studied by Shannon. In this model, when a (binary) codeword is transferred through the channel, every bit flips independently with probability p .

Note that the two noise models proposed by Hamming and Shannon are in some sense two extremes: Hamming's model assumes *no* knowledge about the channel (except that a bound on the total number of errors is known⁸ while Shannon's noise model assumes *complete* knowledge about how noise is produced. In this book, we will consider only these two extreme noise models. In real life, the situation often is somewhere in between.

For real life applications, modeling the noise model correctly is an extremely important task, as we can tailor our codes to the noise model at hand. However, in this book we will not study this aspect of designing codes at all, and will instead mostly consider the worst-case noise model. Informally, if one can communicate over the worst-case noise model, then one could use the same code to communicate over nearly every other noise model with the same amount of noise.

We now return to $C_{\oplus}$ and examine its error-correcting capabilities in the worst-case noise model. We claim that $C_{\oplus}$ cannot correct even one error. Suppose $\mathbf{y} = 10000$ is the received word. Then we know that an error has occurred, but we do not know which bit was flipped. This is because the two codewords $\mathbf{u} = 00000$ and $\mathbf{v} = 10001$ differ from the received word $\mathbf{y}$ in exactly one bit. As we are assuming that the receiver has no side information about the transmitted codeword, no decoder can know what the transmitted codeword was.

Thus, from an error-correction point of view, $C_{\oplus}$ is a terrible code (as it cannot correct even 1 error). However, we will now see that $C_{\oplus}$ can *detect* one error. Consider Algorithm 1.3.1. Note

⁸A bound on the total number of errors is necessary; otherwise, error correction would be impossible: see Exercise 1.3.

Algorithm 1.3.1 Error Detector for Parity Code

INPUT: Received word $\mathbf{y} = (y_1, y_2, y_3, y_4, y_5)$ OUTPUT: 1 if $\mathbf{y} \in C_{\oplus}$ and 0 otherwise

```
1:  $b \leftarrow y_1 \oplus y_2 \oplus y_3 \oplus y_4 \oplus y_5$ 
2: RETURN  $1 \oplus b$     ▷ If there is no error, then  $b = 0$  and hence we need to "flip" the bit for the answer
```

that when no error has occurred during transmission, $y_i = x_i$ for $1 \leq i \leq 4$ and $y_5 = x_1 \oplus x_2 \oplus x_3 \oplus x_4$, in which case $b = 0$ and we output $1 \oplus 0 = 1$ as required. If there is a single error then either $y_i = x_i \oplus 1$ (for exactly one $1 \leq i \leq 4$) or $y_5 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus 1$. It can be checked that in this case, $b = 1$. In fact, one can extend this argument to obtain the following result (see Exercise 1.4).

Proposition 1.3.10. *The parity code $C_{\oplus}$ can detect an odd number of errors.*

Let us now revisit the example that showed that one cannot correct one error using $C_{\oplus}$. Recall, we considered two codewords in $C_{\oplus}$, $\mathbf{u} = 00000$ and $\mathbf{v} = 10001$ (which are codewords corresponding to messages 0000 and 1000, respectively). Now consider the scenarios in which $\mathbf{u}$ and $\mathbf{v}$ are each transmitted and a single error occurs resulting in the received word $\mathbf{r} = 10000$. Thus, given the received word $\mathbf{r}$ and the fact that at most one error can occur, the decoder has no way of knowing whether the original transmitted codeword was $\mathbf{u}$ or $\mathbf{v}$. Looking back at the example, it is clear that the decoder is "confused" because the two codewords $\mathbf{u}$ and $\mathbf{v}$ do not differ in many positions. This notion is formalized in the next section.

1.4 Distance of a Code

We now turn to a new parameter associated with a code that we call the minimum distance of a code. As we will see later, minimum distance is connected to the other parameters, including the error-correction and error-detection capacity of the code. However, due to the cleanliness of the definition, it will often be the first of the parameters we will explore when studying a new error-correcting code.

Definition 1.4.1 (Minimum distance). *Let $C \subseteq \Sigma^n$. The minimum distance (or just distance) of C , denoted $\Delta(C)$, is defined to be*

$$\Delta(C) = \min_{\mathbf{c}_1 \neq \mathbf{c}_2 \in C} \Delta(\mathbf{c}_1, \mathbf{c}_2).$$

We also define the relative minimum distance of C to be $\delta(C)$, is defined to be

$$\delta(C) = \min_{\mathbf{c}_1 \neq \mathbf{c}_2 \in C} \delta(\mathbf{c}_1, \mathbf{c}_2).$$

In other words, $\Delta(C)$ is the minimum distance between two distinct codewords in C . We note that the repetition code $C_{3,rep}$ has distance 3 (recall (1.3)). Indeed, any two distinct messages will differ in at least one of the message bits. After encoding, the difference in one message

bit will translate into a difference of three bits in the corresponding codewords. For example

$$C_{3,rep}(0,0,0,0) = (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0) \text{ and } C_{3,rep}(1,0,0,0) = (1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0).$$

We now claim that the distance of $C_{\oplus}$ is 2. This is a consequence of the following observations. If two messages $\mathbf{m}_1$ and $\mathbf{m}_2$ differ in at least two places then $\Delta(C_{\oplus}(\mathbf{m}_1), C_{\oplus}(\mathbf{m}_2)) \geq 2$ (even if we just ignored the parity bits). If two messages differ in exactly one place then the parity bits in the corresponding codewords are different, which implies a Hamming distance of 2 between the codewords. For example,

$$C_{\oplus}(1,0,0,0) = (1,0,0,0,1) \text{ and } C_{\oplus}(1,0,0,1) = (1,0,0,1,0).$$

Thus, $C_{\oplus}$ has a smaller distance than $C_{3,rep}$ and can correct less number of errors than $C_{3,rep}$. This suggests that a larger distance implies greater error-correcting capabilities. The next result formalizes this intuition. As we will see, minimum distance exactly captures both the ability to recover from errors as also the notion of erasures (Definition 1.3.8).

Proposition 1.4.2. *Given a code C , the following are equivalent:*

1. C has minimum distance $d \geq 2$,
2. If d is odd, C can correct $(d-1)/2$ errors.
3. C can detect $d-1$ errors.
4. C can correct $d-1$ erasures.

Remark 1.4.3. *Property (2) above for even d is slightly different. In this case, one can correct up to $\frac{d}{2} - 1$ errors but cannot correct $\frac{d}{2}$ errors. (See Exercise 1.6.)*

Before we prove Proposition 1.4.2, let us apply it to the codes $C_{\oplus}$ and $C_{3,rep}$ which have distances of 2 and 3, respectively. Proposition 1.4.2 implies the following facts that we have already proved:

- $C_{3,rep}$ can correct 1 error (Proposition 1.3.9).
- $C_{\oplus}$ can detect 1 error but cannot correct 1 error (Proposition 1.3.10).

The proof of Proposition 1.4.2 will need the following decoding function. *Maximum likelihood decoding* (MLD) is a well-studied decoding method for error correcting codes. The MLD function outputs the codeword $\mathbf{c} \in C$, which is as close as possible to the received word in Hamming distance (with ties broken arbitrarily). More formally, the MLD function denoted by $D_{MLD} : \Sigma^n \rightarrow C$ is defined as follows. For every $\mathbf{y} \in \Sigma^n$,

$$D_{MLD}(\mathbf{y}) = \arg \min_{\mathbf{c} \in C} \Delta(\mathbf{c}, \mathbf{y}).$$

Algorithm 1.4.1 is a naive implementation of the MLD.

Algorithm 1.4.1 Naive Maximum Likelihood Decoder

INPUT: Received word $\mathbf{y} \in \Sigma^n$ OUTPUT: $D_{MLD}(\mathbf{y})$

```
1: Pick an arbitrary  $\mathbf{c} \in C$  and assign  $\mathbf{z} \leftarrow \mathbf{c}$ 
2: FOR every  $\mathbf{c}' \in C$  such that  $\mathbf{c} \neq \mathbf{c}'$  DO
3:   IF  $\Delta(\mathbf{c}', \mathbf{y}) < \Delta(\mathbf{z}, \mathbf{y})$  THEN
4:      $\mathbf{z} \leftarrow \mathbf{c}'$ 
5: RETURN  $\mathbf{z}$ 
```

Proof of Proposition 1.4.2 We will complete the proof in two steps. First, we will show that if property 1 is satisfied then so are properties 2, 3 and 4 (we prove this via three implications (1) implies (2), (1) implies (3) and (1) implies (4)). Then we show that if property 1 is not satisfied then none of the properties 2, 3 or 4 hold (again via the corresponding three implications).

Item 1. implies 2. Assume C has distance d . We first prove 2 (for this case assume that $d = 2t + 1$). We now need to show that there exists a decoding function such that for all error patterns with at most t errors it always outputs the transmitted message. We claim that the MLD function has this property. Assume this is not so and let $\mathbf{c}_1$ be the transmitted codeword and let $\mathbf{y}$ be the received word. Note that

$$\Delta(\mathbf{y}, \mathbf{c}_1) \leq t. \quad (1.4)$$

As we have assumed that MLD does not work, $D_{MLD}(\mathbf{y}) = \mathbf{c}_2 \neq \mathbf{c}_1$. Note that by the definition of MLD,

$$\Delta(\mathbf{y}, \mathbf{c}_2) \leq \Delta(\mathbf{y}, \mathbf{c}_1). \quad (1.5)$$

Consider the following set of inequalities:

$$\Delta(\mathbf{c}_1, \mathbf{c}_2) \leq \Delta(\mathbf{c}_2, \mathbf{y}) + \Delta(\mathbf{c}_1, \mathbf{y}) \quad (1.6)$$

$$\leq 2\Delta(\mathbf{c}_1, \mathbf{y}) \quad (1.7)$$

$$\leq 2t \quad (1.8)$$

$$= d - 1, \quad (1.9)$$

where (1.6) follows from the triangle inequality (see Exercise 1.5), (1.7) follows from (1.5) and (1.8) follows from (1.4). (1.9) implies that the distance of C is at most $d - 1$, which is a contradiction.

Item 1. implies 3. We now show that property 3 holds. That is, we need to describe an algorithm that can successfully detect whether errors have occurred during transmission (as long as the total number of errors is bounded by $d - 1$). Consider the following error detection algorithm: check if the received word $\mathbf{y} = \mathbf{c}$ for some $\mathbf{c} \in C$ (this can be done via an exhaustive check). If no errors occurred during transmission, $\mathbf{y} = \mathbf{c}_1$, where $\mathbf{c}_1$ was the transmitted codeword and the algorithm above will accept (as it should). On the other hand if $1 \leq \Delta(\mathbf{y}, \mathbf{c}_1) \leq d - 1$, then by the fact that the distance of C is d , $\mathbf{y} \notin C$ and hence the algorithm rejects, as required.

Item 1. implies 4. Finally, we prove that property 4 holds. Let $\mathbf{y} \in (\Sigma \cup \{?\})^n$ be the received word. First we claim that there is a unique $\mathbf{c} = (c_1, \dots, c_n) \in C$ that agrees with $\mathbf{y}$ (i.e. $y_i = c_i$ for every i such that $y_i \neq ?$). Indeed, for the sake of contradiction, assume that this is not true, i.e. there exists two distinct codewords $\mathbf{c}_1, \mathbf{c}_2 \in C$ such that both $\mathbf{c}_1$ and $\mathbf{c}_2$ agree with $\mathbf{y}$ in the unerased positions. Note that this implies that $\mathbf{c}_1$ and $\mathbf{c}_2$ agree in the positions i such that $y_i \neq ?$. Thus, $\Delta(\mathbf{c}_1, \mathbf{c}_2) \leq |\{i | y_i \neq ?\}| \leq d - 1$, which contradicts the assumption that C has distance d .

Given the uniqueness of the codeword $\mathbf{c} \in C$ that agrees with $\mathbf{y}$ in the unerased position, an algorithm to find $\mathbf{c}$ is as follows: go through all the codewords in C and output the desired codeword.

Item $\neg 1$. implies $\neg 2$. For the other direction of the proof, assume that property 1 does not hold, that is, C has distance $d - 1$. We now show that property 2 cannot hold: i.e., for every decoding function there exists a transmitted codeword $\mathbf{c}_1$ and a received word $\mathbf{y}$ (where $\Delta(\mathbf{y}, \mathbf{c}_1) \leq (d - 1)/2$) such that the decoding function cannot output $\mathbf{c}_1$. Let $\mathbf{c}_1 \neq \mathbf{c}_2 \in C$ be codewords such that $\Delta(\mathbf{c}_1, \mathbf{c}_2) = d - 1$ (such a pair exists as C has distance $d - 1$). Now consider a vector $\mathbf{y}$ such that $\Delta(\mathbf{y}, \mathbf{c}_1) = \Delta(\mathbf{y}, \mathbf{c}_2) = (d - 1)/2$. Such a $\mathbf{y}$ exists as d is odd and by the choice of $\mathbf{c}_1$ and $\mathbf{c}_2$. Figure 1.3 gives an illustration of such a $\mathbf{y}$ (matching color implies that the vectors agree on those positions).

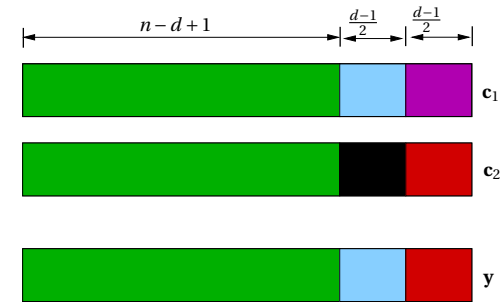


Figure 1.3: Bad example for unique decoding.

Now, since $\mathbf{y}$ could have been generated if *either* of $\mathbf{c}_1$ or $\mathbf{c}_2$ were the transmitted codeword, no decoding function can work in this case.⁹

Item $\neg 1$. implies $\neg 3$. For the remainder of the proof, assume that the transmitted word is $\mathbf{c}_1$ and there exists another codeword $\mathbf{c}_2$ such that $\Delta(\mathbf{c}_2, \mathbf{c}_1) = d - 1$. To see why property 3 is not true, let $\mathbf{y} = \mathbf{c}_2$. In this case, either the error detecting algorithm detects no error, or it declares an error when $\mathbf{c}_2$ is the transmitted codeword and no error takes place during transmission.

⁹Note that this argument is just a generalization of the argument that $C_{\oplus}$ cannot correct 1 error.

Item $\neg 1$. implies $\neg 4$. We finally argue that property 4 does not hold. Let $\mathbf{y}$ be the received word in which the positions that are erased are exactly those where $\mathbf{c}_1$ and $\mathbf{c}_2$ differ. Thus, given $\mathbf{y}$ both $\mathbf{c}_1$ and $\mathbf{c}_2$ could have been the transmitted codeword, and no algorithm for correcting (at most $d - 1$) erasures can work in this case. ■

Proposition 1.4.2 implies that Question 1.1.1 can be reframed as

Question 1.4.1. *What is the largest rate R that a code with distance d can have?*

We have seen that the repetition code $C_{3,rep}$ has distance 3 and rate $1/3$. A natural follow-up question (which is a special case of Question 1.4.1) is to ask

Question 1.4.2. *Can we have a code with distance 3 and rate $R > \frac{1}{3}$?*

1.5 Hamming Code

With the above question in mind, let us consider the so-called *Hamming code*, which we will denote by C_H . Given a message $(x_1, x_2, x_3, x_4) \in \{0, 1\}^4$, its corresponding codeword is given by

$$C_H(x_1, x_2, x_3, x_4) = (x_1, x_2, x_3, x_4, x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4).$$

It can be verified that this code has the following parameters:

$$C_H: q = 2, k = 4, n = 7, R = 4/7.$$

We will show shortly that C_H has a distance of 3. We would like to point out that we could have picked the three parities differently. The reason we mention the three particular parities above is due to historical reasons. We leave it as an exercise to define an alternate set of parities such that the resulting code still has a distance of 3: see Exercise 1.9.

Before we move on to determining the distance of C_H , we will need another definition.

Definition 1.5.1 (Hamming Weight). *Let $q \geq 2$. Given any vector $\mathbf{v} \in \{0, 1, 2, \dots, q - 1\}^n$, its Hamming weight, denoted by $wt(\mathbf{v})$ is the number of non-zero symbols in $\mathbf{v}$.*

For example, if $\mathbf{v} = 01203400$, then $wt(\mathbf{v}) = 4$.

We now look at the distance of C_H .

Proposition 1.5.2. *C_H has a distance of 3.*

Proof. We will prove the claimed distance by using two properties of C_H :

$$\min_{\mathbf{c} \in C_H, \mathbf{c} \neq \mathbf{0}} wt(\mathbf{c}) = 3, \quad (1.10)$$

and

$$\min_{\mathbf{c} \in C_H, \mathbf{c} \neq \mathbf{0}} wt(\mathbf{c}) = \min_{\mathbf{c}_1 \neq \mathbf{c}_2 \in C_H} \Delta(\mathbf{c}_1, \mathbf{c}_2) \quad (1.11)$$

The proof of (1.10) follows from a case analysis on the Hamming weight of the message bits. Let us use $\mathbf{x} = (x_1, x_2, x_3, x_4)$ to denote the message vector.

- Case 0: If $wt(\mathbf{x}) = 0$, then $C_H(\mathbf{x}) = \mathbf{0}$, which means we do not have to consider this codeword.
- Case 1: If $wt(\mathbf{x}) = 1$ then at least two parity check bits in $(x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4)$ are 1 (see Exercise 1.10). So in this case, $wt(C_H(\mathbf{x})) \geq 3$.
- Case 2: If $wt(\mathbf{x}) = 2$ then at least one parity check bit in $(x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4)$ is 1 (see Exercise 1.11). So in this case, $wt(C_H(\mathbf{x})) \geq 3$.
- Case 3: If $wt(\mathbf{x}) \geq 3$ then those message bits themselves imply that $wt(C_H(\mathbf{x})) \geq 3$.

Thus, we can conclude that $\min_{\mathbf{c} \in C_H, \mathbf{c} \neq \mathbf{0}} wt(\mathbf{c}) \geq 3$. Further, note that $wt(C_H(1, 0, 0, 0)) = 3$, which implies that $\min_{\mathbf{c} \in C_H, \mathbf{c} \neq \mathbf{0}} wt(\mathbf{c}) \leq 3$. This along with the lower bound that we just obtained proves (1.10).

We now turn to the proof of (1.11). For the rest of the proof, let $\mathbf{x} = (x_1, x_2, x_3, x_4)$ and $\mathbf{y} = (y_1, y_2, y_3, y_4)$ denote the two distinct messages. Using associativity and commutativity of the $\oplus$ operator, we obtain that

$$C_H(\mathbf{x}) + C_H(\mathbf{y}) = C_H(\mathbf{x} + \mathbf{y}),$$

where the “+” operator is just the bit-wise $\oplus$ of the operand vectors¹⁰. Further, it can be verified that for two vectors $\mathbf{u}, \mathbf{v} \in \{0, 1\}^n$, we have:

$$\Delta(\mathbf{u}, \mathbf{v}) = wt(\mathbf{u} + \mathbf{v})$$

(see Exercise 1.12). Thus, we have

$$\begin{aligned} \min_{\mathbf{x} \neq \mathbf{y} \in \{0, 1\}^4} \Delta(C_H(\mathbf{x}), C_H(\mathbf{y})) &= \min_{\mathbf{x} \neq \mathbf{y} \in \{0, 1\}^4} wt(C_H(\mathbf{x} + \mathbf{y})) \\ &= \min_{\mathbf{x} \neq \mathbf{0} \in \{0, 1\}^4} wt(C_H(\mathbf{x})), \end{aligned}$$

where the second equality follows from the observation that $\{\mathbf{x} + \mathbf{y} | \mathbf{x} \neq \mathbf{y} \in \{0, 1\}^n\} = \{\mathbf{x} \in \{0, 1\}^n | \mathbf{x} \neq \mathbf{0}\}$. Recall that $wt(C_H(\mathbf{x})) = 0$ if and only if $\mathbf{x} = \mathbf{0}$ and this completes the proof of (1.11). Combining (1.10) and (1.11), we conclude that C_H has a distance of 3. □

¹⁰E.g. $(0, 1, 1, 0) + (1, 1, 1, 0) = (1, 0, 0, 0)$.

The second part of the proof could also be shown in the following manner. It can be verified that the Hamming code is the set $\{\mathbf{x} \cdot G_H \mid \mathbf{x} \in \{0, 1\}^4\}$, where G_H is the following matrix (where we think $\mathbf{x}$ as a row vector).¹¹

$$G_H = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

For example, the first column in G_H gives the first codeword bit of x_1 and the fifth column of G_H gives the codeword bit $x_2 \oplus x_3 \oplus x_4$.

In fact, any binary code (of dimension k and block length n) that is generated¹² by a $k \times n$ matrix is called a *binary linear code*. (Both $C_\oplus$ and $C_{3,rep}$ are binary linear codes: see Exercise 1.13.) This implies the following simple fact.

Lemma 1.5.3. *For any binary linear code C and any two messages $\mathbf{x}$ and $\mathbf{y}$, $C(\mathbf{x}) + C(\mathbf{y}) = C(\mathbf{x} + \mathbf{y})$.*

Proof. For any binary linear code, we have a generator matrix G . The following sequence of equalities (which follow from the distributivity and associativity properties of the Boolean EXOR and AND operators) proves the lemma.

$$\begin{aligned} C(\mathbf{x}) + C(\mathbf{y}) &= \mathbf{x} \cdot G + \mathbf{y} \cdot G \\ &= (\mathbf{x} + \mathbf{y}) \cdot G \\ &= C(\mathbf{x} + \mathbf{y}) \end{aligned}$$

□

We stress that in the lemma above, $\mathbf{x}$ and $\mathbf{y}$ need *not* be distinct. Note that due to the fact that $b \oplus b = 0$ for every $b \in \{0, 1\}$, $\mathbf{x} + \mathbf{x} = \mathbf{0}$, which along with the lemma above implies that $C(\mathbf{0}) = \mathbf{0}$.¹³ We can infer the following result from the above lemma and the arguments used to prove (1.11) in the proof of Proposition 1.5.2.

Proposition 1.5.4. *For any binary linear code, its minimum distance is equal to the minimum Hamming weight of any non-zero codeword.*

Thus, we have seen that C_H has distance $d = 3$ and rate $R = \frac{4}{7}$ while $C_{3,rep}$ has distance $d = 3$ and rate $R = \frac{1}{3}$. Thus, the Hamming code is provably better than the repetition code (in terms of the tradeoff between rate and distance) and thus, answers Question 1.4.2 in the affirmative. The next natural question is

Question 1.5.1. *Can we have a distance 3 code with a rate higher than that of C_H ?*

We will address this question in the next section.

1.6 Hamming Bound

Now we switch gears to present our first tradeoff between redundancy (in the form of the dimension of a code) and its error-correction capability (in the form of its distance). In particular, we will first prove a special case of the so-called Hamming bound for a distance of 3.

We begin with another definition.

Definition 1.6.1 (Hamming Ball). *For any vector $\mathbf{x} \in [q]^n$,*

$$B(\mathbf{x}, e) = \{\mathbf{y} \in [q]^n \mid \Delta(\mathbf{x}, \mathbf{y}) \leq e\}.$$

In other words, a Hamming ball of *radius* e , centered at $\mathbf{x}$, contains all vectors within Hamming distance at most e of $\mathbf{x}$.

Next, we prove an upper bound on the dimension of *every* code with distance 3.

Theorem 1.6.2 (Hamming bound for $d = 3$). *Every binary code with block length n , dimension k , distance $d = 3$ satisfies*

$$k \leq n - \log_2(n + 1).$$

Proof. Given any two codewords, $\mathbf{c}_1 \neq \mathbf{c}_2 \in C$, the following is true (as C has distance¹⁴ 3):

$$B(\mathbf{c}_1, 1) \cap B(\mathbf{c}_2, 1) = \emptyset. \quad (1.12)$$

See Figure 1.4 for an illustration.

Note that for all $\mathbf{x} \in \{0, 1\}^n$ (see Exercise 1.16),

$$|B(\mathbf{x}, 1)| = n + 1. \quad (1.13)$$

Now consider the union of all Hamming balls centered around some codeword; their union is a subset of $\{0, 1\}^n$. In other words,

$$\left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, 1) \right| \leq 2^n. \quad (1.14)$$

As (1.12) holds for every pair of distinct codewords,

$$\begin{aligned} \left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, 1) \right| &= \sum_{\mathbf{c} \in C} |B(\mathbf{c}, 1)| \\ &= \sum_{\mathbf{c} \in C} (n + 1) \end{aligned} \quad (1.15)$$

$$= 2^k \cdot (n + 1), \quad (1.16)$$

¹⁴Assume that $\mathbf{y} \in B(\mathbf{c}_1, 1) \cap B(\mathbf{c}_2, 1)$, that is $\Delta(\mathbf{y}, \mathbf{c}_1) \leq 1$ and $\Delta(\mathbf{y}, \mathbf{c}_2) \leq 1$. Thus, by the triangle inequality $\Delta(\mathbf{c}_1, \mathbf{c}_2) \leq 2 < 3$, which is a contradiction.

¹¹Indeed $(x_1, x_2, x_3, x_4) \cdot G_H = (x_1, x_2, x_3, x_4, x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4)$, as desired.

¹²That is, $C = \{\mathbf{x} \cdot G \mid \mathbf{x} \in \{0, 1\}^k\}$, where addition is the $\oplus$ operation and multiplication is the AND operation.

¹³This of course should not be surprising as for any matrix G , we have $\mathbf{0} \cdot G = \mathbf{0}$.

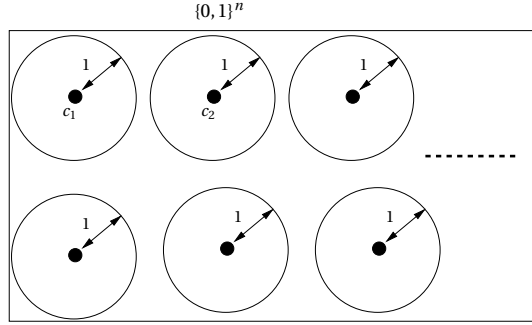


Figure 1.4: Hamming balls of radius 1 are disjoint. The figure is technically not correct: the balls above are actually balls in the Euclidean space, which is easier to visualize than the Hamming space.

where (1.15) follows from (1.13) and (1.16) follows from the fact that C has dimension k . Combining (1.16) and (1.14), we get

$$2^k(n+1) \leq 2^n,$$

or equivalently

$$2^k \leq \frac{2^n}{n+1}.$$

Taking $\log_2$ of both sides we get the desired bound:

$$k \leq n - \log_2(n+1).$$

□

Thus, Theorem 1.6.2 shows that for $n = 7$, C_H has the largest possible dimension for any binary code of block length 7 and distance 3 (as for $n = 7$, $n - \log_2(n+1) = 4$). In particular, it also answers Question 1.5.1 for $n = 7$ in the negative. Next, we will present the general form of Hamming bound.

1.7 Generalized Hamming Bound

We start with a new notation.

Definition 1.7.1. A code $C \subseteq \Sigma^n$ with dimension k and distance d will be called a $(n, k, d)_\Sigma$ code. We will also refer to it as a $(n, k, d)_{|\Sigma|}$ code.

We now proceed to generalize Theorem 1.6.2 to any distance d (from $d = 3$).

Theorem 1.7.2 (Hamming Bound for any d). For every $(n, k, d)_q$ code

$$k \leq n - \log_q \left(\sum_{i=0}^{\lfloor \frac{d-1}{2} \rfloor} \binom{n}{i} (q-1)^i \right).$$

Proof. The proof is a straightforward generalization of the proof of Theorem 1.6.2. For notational convenience, let $e = \lfloor \frac{d-1}{2} \rfloor$. Given any two codewords, $\mathbf{c}_1 \neq \mathbf{c}_2 \in C$, the following is true (as C has distance¹⁵ d):

$$B(\mathbf{c}_1, e) \cap B(\mathbf{c}_2, e) = \emptyset. \quad (1.17)$$

We claim that for all $\mathbf{x} \in [q]^n$,

$$|B(\mathbf{x}, e)| = \sum_{i=0}^e \binom{n}{i} (q-1)^i. \quad (1.18)$$

Indeed any vector in $B(\mathbf{x}, e)$ must differ from $\mathbf{x}$ in exactly $0 \leq i \leq e$ positions. In the summation, $\binom{n}{i}$ is the number of ways of choosing the differing i positions and in each such position, a vector can differ from $\mathbf{x}$ in $q-1$ ways.

Now consider the union of all Hamming balls centered around a codeword. Obviously, their union is a subset of $[q]^n$. In other words,

$$\left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, e) \right| \leq q^n. \quad (1.19)$$

As (1.17) holds for every pair of distinct codewords,

$$\begin{aligned} \left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, e) \right| &= \sum_{\mathbf{c} \in C} |B(\mathbf{c}, e)| \\ &= q^k \sum_{i=0}^e \binom{n}{i} (q-1)^i, \end{aligned} \quad (1.20)$$

where (1.20) follows from (1.18) and the fact that C has dimension k . Combining (1.20) and (1.19) and taking $\log_q$ of both sides we will get the desired bound:

$$k \leq n - \log_q \left(\sum_{i=0}^e \binom{n}{i} (q-1)^i \right).$$

□

Note that the Hamming bound gives a partial answer to Question 1.4.1. In particular, any code of distance d can have rate R at most

$$1 - \frac{\log_q \left(\sum_{i=0}^e \binom{n}{i} (q-1)^i \right)}{n}.$$

Further, the Hamming bound also leads to the following definition:

¹⁵Assume that $\mathbf{y} \in B(\mathbf{c}_1, e) \cap B(\mathbf{c}_2, e)$, that is $\Delta(\mathbf{y}, \mathbf{c}_1) \leq e$ and $\Delta(\mathbf{y}, \mathbf{c}_2) \leq e$. Thus, by the triangle inequality, $\Delta(\mathbf{c}_1, \mathbf{c}_2) \leq 2e \leq d-1$, which is a contradiction.

Definition 1.7.3. Codes that meet Hamming bound are called perfect codes.

In other words, a perfect code leads to the following perfect “packing”: if one constructs Hamming balls of radius $\left\lfloor \frac{d-1}{2} \right\rfloor$ around all the codewords, then we would cover the entire ambient space, i.e. every possible vector will lie in one of these Hamming balls.

One example of perfect code is the $(7, 4, 3)_2$ Hamming code that we have seen in this chapter (so is the family of general Hamming codes that we will see in the next chapter). A natural question to ask is if

Question 1.7.1. Other than the Hamming codes, are there any other perfect (binary) codes?

We will see the answer in Section 2.4.

1.8 Family of codes

Until now, we have mostly studied specific codes with *fixed* block lengths and dimensions. However, when we perform an asymptotic study of codes, it makes more sense to talk about a family of codes and study their asymptotic rate and distance. We define these notions next.

Definition 1.8.1 (Code families, Rate and Distance). Let $\{n_i\}_{i \geq 1}$ be an increasing sequence of block lengths and suppose there exists sequences $\{k_i\}_{i \geq 1}$, $\{d_i\}_{i \geq 1}$ and $\{q_i\}_{i \geq 1}$ such that for all $i \geq 1$ there exists an $(n_i, k_i, d_i)_{q_i}$ code C_i . Then the sequence $\mathcal{C} = \{C_i\}_{i \geq 1}$ is a family of codes. The rate of $\mathcal{C}$ is defined as

$$R(\mathcal{C}) = \lim_{i \rightarrow \infty} \left\{ \frac{k_i}{n_i} \right\},$$

when the limit exists. The relative distance of $\mathcal{C}$ is defined as

$$\delta(\mathcal{C}) = \lim_{i \rightarrow \infty} \left\{ \frac{d_i}{n_i} \right\},$$

when the limit exists. If for all $i \geq 1$, $q_i = q$ then $\mathcal{C}$ is referred to as a family of q -ary codes.^{16 17}

For instance, we will in Section 2.4 see that Hamming code of Section 1.5 can be extended to an entire family of codes. Specifically, $\mathcal{C}_H = \{C_i\}_{i \in \mathbb{Z}^+}$, with C_i being an (n_i, k_i, d_i) -code with $n_i = 2^i - 1$, $k_i = 2^i - i - 1$, $d_i = 3$ and thus,

$$R(\mathcal{C}_H) = \lim_{i \rightarrow \infty} 1 - \frac{i}{2^i - 1} = 1,$$

¹⁶In all codes we will study these limits will exist, but of course it is possible to construct families of codes where the limits do not exist.

¹⁷While a central goal is to understand q -ary families of codes, families over growing alphabets turn out to be useful both to illustrate ideas and to get interesting q -ary families.

and

$$\delta(\mathcal{C}_H) = \lim_{i \rightarrow \infty} \frac{3}{2^i - 1} = 0.$$

A significant focus of this text from now on will be on families of codes. This is necessary as we will be studying the asymptotic behavior of algorithms on codes, which does not make sense for a fixed code. For example, when we say that a decoding algorithm for a code $\mathcal{C}$ takes $O(n^2)$ time, we would be implicitly assuming that $\mathcal{C}$ is a family of codes and that the algorithm has an $O(n^2)$ running time when the block length is large enough. From now on, unless mentioned otherwise, whenever we talk about a code, we will be implicitly assuming that we are talking about a family of codes.

Given that we can only formally talk about asymptotic run time of algorithms, we now also state our formal notion of efficient algorithms:

We'll call an algorithm related to a code of block length n to be *efficient* if it runs in time polynomial in n .

For all the specific codes that we will study in this book, the corresponding family of codes will be a “family” in a more natural sense. By this we mean that all the specific codes in a family of codes will be the “same” code except with different parameters. A bit more formally, we will consider families $\{C_i\}_{i \geq 1}$, where given only the ‘index’ i , one can compute a sufficient description of C_i efficiently.¹⁸

Finally, the definition of a family of codes allows us to present the final version of the big motivating question for the book. The last formal version of the main question we considered was Question 1.4.1, where we were interested in the tradeoff of rate R and distance d . The comparison was somewhat unfair because R was a ratio while d was an integer. A more appropriate comparison should be between rate R and the relative distance δ . Further, we would be interested in tackling the main motivating question for families of codes, which results in the following final version:

Question 1.8.1. Given q , what is the optimal tradeoff between $R(\mathcal{C})$ and $\delta(\mathcal{C})$ that can be achieved by some family $\mathcal{C}$ of q -ary codes?

A natural special case of Question 1.8.1 is whether the rate and relative distance of a family of codes can be simultaneously positive. We formulate this special case as a separate question below.



¹⁸We stress that this is not *always* going to be the case. In particular, we will consider “random” codes where this efficient constructibility will not be true.

Question 1.8.2. Does there exist a constant q and a qq -ary family of codes $\mathcal{C}$ such that $R(\mathcal{C}) > 0$ and $\delta(\mathcal{C}) > 0$ hold simultaneously?

Codes that have the above property are called *asymptotically good*. For the curious reader, we will present many asymptotically good codes in the rest of this book, though a priori the existence of these is not immediate.

1.9 Exercises

Exercise 1.1. Show that every t -error correcting code is also t -error detecting but not necessarily the other way around.

Exercise 1.2. Prove Proposition 1.3.9.

Exercise 1.3. Show that for every integer n , there is no code with block length n that can handle arbitrary number of errors.

Exercise 1.4. Prove Proposition 1.3.10.

Exercise 1.5. A distance function on Σ^n (i.e. $d : \Sigma^n \times \Sigma^n \rightarrow \mathbb{R}$) is called a metric if the following conditions are satisfied for every $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \Sigma^n$:

1. $d(\mathbf{x}, \mathbf{y}) \geq 0$.
2. $d(\mathbf{x}, \mathbf{y}) = 0$ if and only if $\mathbf{x} = \mathbf{y}$.
3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$.
4. $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$. (This property is called the triangle inequality.)

Prove that the Hamming distance is a metric.

Exercise 1.6. Let C be a code with distance d for even d . Then argue that C can correct up to $d/2 - 1$ many errors but cannot correct $d/2$ errors. Using this or otherwise, argue that if a code C is t -error correctable then it either has a distance of $2t + 1$ or $2t + 2$.

Exercise 1.7. In this exercise, we will see that one can convert arbitrary codes into code with slightly different parameters:

1. Prove that if there exists an $(n, k, d)_\Sigma$ code then there also exists an $(n - 1, k, d - 1)_\Sigma$ code. Specifically, show how to convert an $(n, k, d)_\Sigma$ code C into an $(n - 1, k, d - 1)_\Sigma$ code.
2. For odd d , prove that if an $(n, k, d)_2$ code exists, then there also exists an $(n + 1, k, d + 1)_2$ code. Specifically, show how to convert an $(n, k, d)_2$ code C into an $(n + 1, k, d + 1)_2$ code.

Note: Your conversion should not assume anything else about the code other than the parameters of the code C . Also your conversion should work for every $n, k, d \geq 1$ and every Σ .

Exercise 1.8. In this problem we will consider a noise model that has both errors and erasures. In particular, let C be an $(n, k, d)_\Sigma$ code. As usual a codeword $\mathbf{c} \in C$ is transmitted over the channel and the received word is a vector $\mathbf{y} \in (\Sigma \cup \{?\})^n$, where as before $a?$ denotes an erasure. We will use s to denote the number of erasures in $\mathbf{y}$ and e to denote the number of (non-erasure) errors that occurred during transmission. To decode such a vector means to output a codeword $\mathbf{c} \in C$ such that the number of positions where $\mathbf{c}$ disagree with $\mathbf{y}$ in the $n - s$ non-erased positions is at most e . For the rest of the problem assume that

$$2e + s < d. \quad (1.21)$$

1. Argue that the output of the decoder for any C under (1.21) is unique.
2. Let C be a binary code (but not necessarily linear). Assume that there exists a decoder D that can correct from $< d/2$ many errors in $T(n)$ time. Then under (1.21) one can perform decoding in time $O(T(n))$.

Exercise 1.9. Define codes other than C_H with $k = 4, n = 7$ and $d = 3$.

Hint: Refer to the proof of Proposition 1.5.2 to figure out the properties needed from the three parities.

Exercise 1.10. Argue that if $wt(\mathbf{x}) = 1$ then at least two parity check bits in $(x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4, x_1 \oplus x_3 \oplus x_4)$ are 1.

Exercise 1.11. Argue that if $wt(\mathbf{x}) = 2$ then at least one parity check bit in $(x_2 \oplus x_3 \oplus x_4, x_1 \oplus x_2 \oplus x_4, x_1 \oplus x_3 \oplus x_4)$ is 1.

Exercise 1.12. Prove that for any $\mathbf{u}, \mathbf{v} \in \{0, 1\}^n$, $\Delta(\mathbf{u}, \mathbf{v}) = wt(\mathbf{u} + \mathbf{v})$.

Exercise 1.13. Argue that $C_\oplus$ and $C_{3,rep}$ are binary linear codes.

Exercise 1.14. Let G be a generator matrix of an $(n, k, d)_2$ binary linear code. Then G has at least kd ones in it.

Exercise 1.15. Argue that in any binary linear code, either all all codewords begin with a 0 of exactly half of the codewords begin with a 0.

Exercise 1.16. Prove (1.13).

Exercise 1.17. Show that there is no binary code with block length 4 that achieves the Hamming bound.

Exercise 1.18. (*) There are n people in a room, each of whom is given a black/white hat chosen uniformly at random (and independent of the choices of all other people). Each person can see the hat color of all other people, but not their own. Each person is asked if they wish to guess their own hat color. They can either guess, or abstain. Each person makes their choice without

knowledge of what the other people are doing. They either win collectively, or lose collectively. They win if at least one person does not abstain and all the people who don't abstain guess their hat color correctly. They lose if all people abstain, or if some person guesses their color incorrectly. Your goal below is to come up with a strategy that will allow the n people to win with pretty high probability. We begin with a simple warm-up:

- (a) Argue that the n people can win with probability at least $\frac{1}{2}$.

Next we will see how one can really bump up the probability of success with some careful modeling, and some knowledge of Hamming codes. (Below are assuming knowledge of the general Hamming code (see Section 2.4). If you do not want to skip ahead, you can assume that $n = 7$ in the last part of this problem.

- (b) Let's say that a directed graph G is a subgraph of the n -dimensional hypercube if its vertex set is $\{0, 1\}^n$ and if $u \rightarrow v$ is an edge in G , then u and v differ in at most one coordinate. Let $K(G)$ be the number of vertices of G with in-degree at least one, and out-degree zero. Show that the probability of winning the hat problem equals the maximum, over directed subgraphs G of the n -dimensional hypercube, of $K(G)/2^n$.
- (c) Using the fact that the out-degree of any vertex is at most n , show that $K(G)/2^n$ is at most $\frac{n}{n+1}$ for any directed subgraph G of the n -dimensional hypercube.
- (d) Show that if $n = 2^r - 1$, then there exists a directed subgraph G of the n -dimensional hypercube with $K(G)/2^n = \frac{n}{n+1}$.

Hint: This is where the Hamming code comes in.

1.10 Bibliographic Notes

Coding theory owes its origin to two remarkable papers: one by Shannon [138] and the other by Hamming [83] both of which were published within a couple of years of each other. Shannon's paper defined the BSC_p channel (among others) and defined codes in terms of its encoding function. Shannon's paper also explicitly defined the decoding function. Hamming's work defined the notion of codes as in Definition 1.2.1 as well as the notion of Hamming distance. Both the Hamming bound and the Hamming code are (not surprisingly) due to Hamming. The specific definition of Hamming code that we used in this book was the one proposed by Hamming and is also mentioned in Shannon's paper (which pre-dates Hamming's) with attribution to Hamming. The notion of erasures was defined by Elias [48]. Most exercises of this chapter are based on [83]. The hat problem in Exercise 1.18 is from Ebert, Merkle and Vollmer [45].

Part I

The Basics

Overview of the Basics

Chapter 1 introduced the fundamental quests of this book — namely error-correcting codes over some given alphabet and block length that achieve the best possible tradeoff between dimension and distance, and the asymptotics of these parameters. We refer to all the theory pertinent to this quest as *coding theory*.

To construct codes, it turns out that a little bit of algebra goes a long way. Specifically if the alphabet is viewed as a *finite field* and the code is required to be a vector space over this finite field, then we get the subclass of error-correcting codes called linear codes. Linear codes tend to be interesting in their own right, but also focussing on linear codes somehow allows us to come up with creative ways to construct codes. In Chapter 2 we review the basic algebra concepts needed to define these linear codes and embark on a preliminary study of these codes to illustrate their power.

A second subfield of mathematics that turns out to be central to coding theory is probability theory. The probability theory considered in this text is mostly over finite sets, and thus is just a fancy way to ‘count.’ Yet the basic tools of probability theory turn out to be very useful in eliciting proofs of basic facts in coding theory. So in Chapter 3 we review some basics of coding theory and also introduce the entropy function which also plays a prominent role in coding theory.

Future parts of this book return to our fundamental question and use the tools from Chapters 2 and 3 to make progress on them.

Chapter 2

A Look at Some Nicely Behaved Codes: Linear Codes

One motivation for the topic of this chapter is the following question: How we can represent a code? Or more specifically, how many bits does it take to describe a code $C : [q]^k \rightarrow [q]^n$? In general, a code $C : [q]^k \rightarrow [q]^n$ can be stored using nq^k symbols from $[q]$ (n symbols for each of the q^k codewords) or $nq^k \log q$ bits. For constant rate codes, this is exponential space, which is prohibitive even for modest values of k like $k = 100$. A natural question is whether we can do better. To have any hope of doing so, a succinct representation the code must have some extra structure. It turns out that one broad class of codes that do possess extra structure than general codes, is what are called *linear codes*. We have already seen binary linear codes in Section 1.5, that is: $C \subseteq \{0, 1\}^n$ is a linear code if for all $\mathbf{c}_1, \mathbf{c}_2 \in C$, $\mathbf{c}_1 + \mathbf{c}_2 \in C$, where the "+" denotes bit-wise XOR. In this chapter, we will see more general linear codes. We will see that they not only offer enough structure to get succinct representations, but they also possess several other nice properties.

To define general linear codes, we first need to introduce general finite fields and vector spaces over such fields and we do so first before returning to codes.

2.1 Groups and Finite Fields

To define linear subspaces, we will need to work with (finite) fields. At a high level, we need finite fields since when we talk about codes, we deal with finite symbols/numbers and we want to endow these symbols with the same math that makes arithmetic over real numbers work. Finite fields accomplish this precise task. We begin with a quick overview of fields. We start with the more elementary notion of a group.

Definition 2.1.1. A group $\mathbb{G}$ is given by a pair $(S, \circ)$, where S is the set of elements and $\circ$ is a function $S \times S \rightarrow S$ with the following properties:

- CLOSURE: For every $a, b \in S$, we have $a \circ b \in S$.
- ASSOCIATIVITY: $\circ$ is associative: that is, for every $a, b, c \in S$, $a \circ (b \circ c) = (a \circ b) \circ c$.

- **IDENTITY:** *There exists distinct a special elements $e \in S$ such that for every $a \in S$ we have $a \circ e = e \circ a = a$.*
- **INVERSE:** *For every $a \in S$, there exists its unique inverse a^{-1} such that $a \circ a^{-1} = a^{-1} \circ a = e$.*

If $\mathbb{G} = (S, \circ)$ satisfies all the properties except the existence of inverses then $\mathbb{G}$ is called a monoid. We say $\mathbb{G}$ is commutative if for every $a, b \in S$, $a \circ b = b \circ a$.

We often use the same letter to denote the group (or other algebraic structures) and the set of elements.

We now turn to the definition of a field. Informally speaking, a field is a set of elements on which one can do addition, subtraction, multiplication and division and still stay in the set.

Definition 2.1.2. A field $\mathbb{F}$ is given by a triple $(S, +, \cdot)$, where S is the set of elements and $+$, $\cdot$ are functions $S \times S \rightarrow S$ with the following properties:

- *Addition:* $(S, +)$ form a commutative group with identity element denoted $0 \in S$.
- *Multiplication:* $(S \setminus \{0\}, \cdot)$ form a commutative group with identity element $1 \in S \setminus \{0\}$.¹
- *Distributivity:* $\cdot$ distributes over $+$: that is, for every $a, b, c \in S$, $a \cdot (b + c) = a \cdot b + a \cdot c$.

Again we typically use the same letter to denote the field and its set of elements. We also use $-a$ to denote the additive inverse of $a \in \mathbb{F}$ and a^{-1} to denote the multiplicative inverse of $a \in \mathbb{F} \setminus \{0\}$.

We note that in the above definition we have not explicitly argued that $a \cdot 0 = 0 = 0 \cdot a$ for any $a \in S$. (Technically this means $(S, \cdot)$ is a *commutative monoid*.) This is because this property is implied by Definition 2.1.2—see Exercise 2.1.

With the usual semantics for $+$ and $\cdot$, $\mathbb{R}$ (set of real number) is a field, but $\mathbb{Z}$ (set of integers) is not a field as division of two integers results in a rational number that need not be an integer (the set of rational numbers itself is a field though: see Exercise 2.2). In this course, we will exclusively deal with *finite fields*. As the name suggests these are fields with a finite set of elements. (We will overload notation and denote the size of a field $|\mathbb{F}| = |S|$.) The following is a well known result.

Theorem 2.1.3 (Size of Finite Fields). *Every finite field has size p^s for some prime p and integer $s \geq 1$. Conversely for every prime p and integer $s \geq 1$ there exists a field $\mathbb{F}$ of size p^s .*

One example of a finite field that we have seen is the field with $S = \{0, 1\}$, which we will denote by $\mathbb{F}_2$ (we have seen this field in the context of binary linear codes). For $\mathbb{F}_2$, addition is the XOR operation, while multiplication is the AND operation. The additive inverse of an element in $\mathbb{F}_2$ is the number itself while the multiplicative inverse of 1 is 1 itself.

Let p be a prime number. Then the integers modulo p form a field, denoted by $\mathbb{F}_p$ (and also by $\mathbb{Z}_p$), where the addition and multiplication are carried out modulo p . For example, consider $\mathbb{F}_7$, where the elements are $\{0, 1, 2, 3, 4, 5, 6\}$. We have $(4 + 3) \bmod 7 = 0$ and $4 \cdot 4 \bmod 7 = 2$.

¹Note that we do not include 0 since it does not have a multiplicative inverse.

Further, the additive inverse of 4 is 3 as $(3 + 4) \bmod 7 = 0$ and the multiplicative inverse of 4 is 2 as $4 \cdot 2 \bmod 7 = 1$.

More formally, we prove the following result.

Lemma 2.1.4. *Let p be a prime. Then $\mathbb{F}_p = (\{0, 1, \dots, p-1\}, +_p, \cdot_p)$ is a field, where $+_p$ and $\cdot_p$ are addition and multiplication modulo p .*

Proof. The properties of associativity, commutativity, distributivity and identities hold for integers and hence, they hold for $\mathbb{F}_p$. The closure property follows since both the “addition” and “multiplication” are done modulo p , which implies that for any $a, b \in \{0, \dots, p-1\}$, $a +_p b, a \cdot_p b \in \{0, \dots, p-1\}$. Thus, to complete the proof, we need to prove the existence of unique additive and multiplicative inverses.

Fix an arbitrary $a \in \{0, \dots, p-1\}$. Then we claim that its additive inverse is $p - a \bmod p$. It can be verified that $a + p - a = 0 \bmod p$. Next we argue that this is the unique additive inverse. To see this note that the sequence $a, a + 1, a + 2, \dots, a + p - 1$ are p consecutive numbers and thus, exactly one of them is a multiple of p , which happens for $b = p - a \bmod p$, as desired.

Now fix an $a \in \{1, \dots, p-1\}$. Next we argue for the existence of a unique multiplicative inverse a^{-1} . Consider the set of numbers $T = \{a \cdot_p b \mid b \in \{1, \dots, p-1\}\}$. We claim that all these numbers are unique. To see this, note that if this is not the case, then there exist $b_1 \neq b_2 \in \{0, 1, \dots, p-1\}$ such that $a \cdot b_1 = a \cdot b_2 \bmod p$, which in turn implies that $a \cdot (b_1 - b_2) = 0 \bmod p$. Since a and $b_1 - b_2$ are non-zero numbers, this implies that p divides $a \cdot (b_1 - b_2)$. Further, since a and $|b_1 - b_2|$ are both at most $p-1$, this implies that multiplying a and $(b_1 - b_2) \bmod p$ results in p , which is a contradiction since p is prime. Thus, we have argued that $|T| = p - 1$ and since each number in T is in $[p - 1]$, we have that $T = [p - 1]$. Thus, we can conclude that there exists a unique element b such that $a \cdot b = 1 \bmod p$ and thus, b is the required a^{-1} . $\square$

One might think that there could be different finite fields with the same number of elements. However, this is not the case:

Theorem 2.1.5. *For every prime power q there is a unique finite field with q elements (up to isomorphism²).*

Thus, we are justified in just using $\mathbb{F}_q$ to denote a finite field on q elements.

2.2 Vector Spaces and Linear Subspaces

Definition 2.2.1 (Vector Space). *A vector space V over a field $\mathbb{F}$ is given by a triple $(T, +, \cdot)$ such that $(T, +)$ form a commutative group and $\cdot$, referred to as the scalar product, is a function $\mathbb{F} \times T \rightarrow T$ such that for every $a, b \in \mathbb{F}$ and $\mathbf{u}, \mathbf{v} \in T$ we have $(a + b) \cdot \mathbf{u} = a \cdot \mathbf{u} + b \cdot \mathbf{u}$ and $a \cdot (\mathbf{u} + \mathbf{v}) = a \cdot \mathbf{u} + a \cdot \mathbf{v}$.*

²An isomorphism $\phi: S \rightarrow S'$ is a bijective map (such that $\mathbb{F} = (S, +, \cdot)$ and $\mathbb{F}' = (S', \oplus, \odot)$ are fields) where for every $a_1, a_2 \in S$, we have $\phi(a_1 + a_2) = \phi(a_1) \oplus \phi(a_2)$ and $\phi(a_1 \cdot a_2) = \phi(a_1) \odot \phi(a_2)$. In other words, an isomorphism is a map between representations that ‘preserves’ the effect of operators on elements.

The most common vector space we will focus on is $\mathbb{F}^n$ with $+$ representing coordinatewise addition in $\mathbb{F}$ and $a \cdot \mathbf{u}$ representing the coordinatewise scaling of $\mathbf{u}$ by a .

We are finally ready to define the notion of linear subspaces of $\mathbb{F}^n$.

Definition 2.2.2 (Linear Subspace). A non-empty subset $S \subseteq \mathbb{F}^n$ is a linear subspace if the following properties hold:

1. For every $\mathbf{x}, \mathbf{y} \in S$, $\mathbf{x} + \mathbf{y} \in S$, where the addition is vector addition over $\mathbb{F}$ (that is, do addition componentwise over $\mathbb{F}$).
2. For every $a \in \mathbb{F}$ and $\mathbf{x} \in S$, $a \cdot \mathbf{x} \in S$, where the multiplication is done componentwise over $\mathbb{F}$.

Here is a (trivial) example of a linear subspace of $\mathbb{F}_5^3$:

$$S_1 = \{(0, 0, 0), (1, 1, 1), (2, 2, 2), (3, 3, 3), (4, 4, 4)\}. \quad (2.1)$$

Note that for example $(1, 1, 1) + (3, 3, 3) = (4, 4, 4) \in S_1$ and $2 \cdot (4, 4, 4) = (3, 3, 3) \in S_1$ as required by the definition. Here is another somewhat less trivial example of a linear subspace over $\mathbb{F}_3^3$:

$$S_2 = \{(0, 0, 0), (1, 0, 1), (2, 0, 2), (0, 1, 1), (0, 2, 2), (1, 1, 2), (1, 2, 0), (2, 1, 0), (2, 2, 1)\}. \quad (2.2)$$

Note that $(1, 0, 1) + (0, 2, 2) = (1, 2, 0) \in S_2$ and $2 \cdot (2, 0, 2) = (1, 0, 1) \in S_2$ as required.

Remark 2.2.3. Note that the second property implies that $\mathbf{0}$ is contained in every linear subspace. Further for any subspace over $\mathbb{F}_2$, the second property is redundant: see Exercise 2.5.

Before we state some properties of linear subspaces, we state some relevant definitions.

Definition 2.2.4 (Span). Given a set $B = \{\mathbf{v}_1, \dots, \mathbf{v}_\ell\}$. The span of B is the set of vectors

$$\left\{ \sum_{i=1}^{\ell} a_i \cdot \mathbf{v}_i \mid a_i \in \mathbb{F}_q \text{ for every } i \in [\ell] \right\}.$$

Definition 2.2.5 (Linear (in)dependence of vectors). We say that $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ are linearly independent if for every $1 \leq i \leq k$ and for every $(k-1)$ -tuple $(a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_k) \in \mathbb{F}_q^{k-1}$,

$$\mathbf{v}_i \neq a_1 \mathbf{v}_1 + \dots + a_{i-1} \mathbf{v}_{i-1} + a_{i+1} \mathbf{v}_{i+1} + \dots + a_k \mathbf{v}_k.$$

In other words, $\mathbf{v}_i$ is not in the span of the set $\{\mathbf{v}_1, \dots, \mathbf{v}_{i-1}, \mathbf{v}_{i+1}, \dots, \mathbf{v}_k\}$ for every $1 \leq i \leq k$. We say that $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ are linearly dependent if they are not linearly independent.

For example the vectors $(1, 0, 1), (1, 1, 1) \in S_2$ are linearly independent since

- $a_1 \cdot (1, 0, 1) = (a_1, 0, a_1) \neq (1, 1, 1)$ for any $a_1 \in \{0, 1\}$.
- $a_2 \cdot (1, 0, 1) = (a_2, a_2, a_2) \neq (1, 0, 1)$ for any $a_2 \in \{0, 1\}$.

Definition 2.2.6 (Rank of a matrix). The rank of matrix in $\mathbb{F}_q^{k \times k}$ is the maximum number of linearly independent rows (or columns). A matrix in $\mathbb{F}_q^{k \times n}$ with rank $\min(k, n)$ is said to have full rank.

One can define the row (column) rank of a matrix as the maximum number of linearly independent rows (columns). However, it is a well-known theorem that the row rank of a matrix is the same as its column rank. For example, the matrix below over $\mathbb{F}_3$ has full rank (see Exercise 2.6):

$$G_2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}. \quad (2.3)$$

Any linear subspace satisfies the following properties (the full proof can be found in any standard linear algebra textbook).

Theorem 2.2.7. If $S \subseteq \mathbb{F}_q^n$ is a linear subspace then

1. $|S| = q^k$ for some $k \geq 0$. The parameter k is called the dimension of S .
2. There exists at least one set of linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_k \in S$ called basis elements such that every $\mathbf{x} \in S$ can be expressed as $\mathbf{x} = a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_k \mathbf{v}_k$ where $a_i \in \mathbb{F}_q$ for $1 \leq i \leq k$. In other words, there exists a full rank $k \times n$ matrix G (also known as a generator matrix) with entries from $\mathbb{F}_q$ such that every $\mathbf{x} \in S$, $\mathbf{x} = (a_1, a_2, \dots, a_k) \cdot G$ where

$$G = \begin{pmatrix} \leftarrow \mathbf{v}_1 \longrightarrow \\ \leftarrow \mathbf{v}_2 \longrightarrow \\ \vdots \\ \leftarrow \mathbf{v}_k \longrightarrow \end{pmatrix}.$$

3. There exists a full rank $(n-k) \times n$ matrix H (called a parity check matrix) such that for every $\mathbf{x} \in S$, $H\mathbf{x}^T = \mathbf{0}$.
4. G and H are orthogonal, that is, $G \cdot H^T = \mathbf{0}$.

Proof Sketch.

Property 1. We begin with the proof of the first property. For the sake of contradiction, let us assume that $q^k < |S| < q^{k+1}$, for some $k \geq 0$. Iteratively, we will construct a set of linearly independent vectors $B \subseteq S$ such that $|B| \geq k+1$. Note that by the definition of a linear subspace the span of B should be contained in S . However, this is a contradiction as the size of the span of B is at least $q^{k+1} > |S|$.

To complete the proof, we show how to construct the set B in a greedy fashion. In the first step pick $\mathbf{v}_1$ to be any non-zero vector in S and set $B \leftarrow \{\mathbf{v}_1\}$ (we can find such a vector as $|S| > q^k \geq 1$). Now say after the step t (for some $t \leq k$), $|B| = t$. Now the size of the span of the current B is $q^t \leq q^k < |S|$. Thus there exists a vector $\mathbf{v}_{t+1} \in S \setminus B$ that is linearly independent of vectors in B . Set $B \leftarrow B \cup \{\mathbf{v}_{t+1}\}$. Thus, we can continue building B until $|B| = k+1$, as desired.

³See Exercise 2.8.

Property 2. We first note that we can pick $B = \{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ to be any set of k linearly independent vectors— this just follows from the argument above for **Property 1.1**. This is because the span of B is contained in S . However, since $|S| = q^k$ and the span of B has q^k vectors, the two have to be the same.

Property 3. Property 3 above follows from another fact that every linear subspace S has a null space $N \subseteq \mathbb{F}_q^n$ such that for every $\mathbf{x} \in S$ and $\mathbf{y} \in N$, $\langle \mathbf{x}, \mathbf{y} \rangle = 0$. Further, it is known that N itself is a linear subspace of dimension $n - k$. (The claim that N is also a linear subspace follows from the following two facts: for every $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}_q^n$, (i) $\langle \mathbf{x}, \mathbf{y} + \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{z} \rangle$ and (ii) for any $a \in \mathbb{F}_q$, $\langle \mathbf{x}, a\mathbf{y} \rangle = a \cdot \langle \mathbf{x}, \mathbf{y} \rangle$.) In other words, there exists a generator matrix H for it. This matrix H is called the parity check matrix of S .

Property 4. See Exercise 2.9. □

As examples, the linear subspace S_1 in (2.1) has as one of its generator matrices

$$G_1 = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}$$

and as one of its parity check matrices

$$H_1 = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 2 & 1 \end{pmatrix}.$$

Further, the linear subspace S_2 in (2.2) has G_2 as one of its generator matrices and has the following as one of its parity check matrices

$$H_2 = \begin{pmatrix} 1 & 1 & 2 \end{pmatrix}.$$

Finally, we state another property of linear subspaces that is useful.

Lemma 2.2.8. *Given matrix G of dimension $k \times n$ that is a generator matrix of subspace S_1 and matrix H of dimension $(n - k) \times n$ that is a parity check matrix of subspace S_2 such that $GH^T = \mathbf{0}$, then $S_1 = S_2$.*

Proof. We first prove that $S_1 \subseteq S_2$. Given any $\mathbf{c} \in S_1$, there exists $\mathbf{x} \in \mathbb{F}_q^k$ such that $\mathbf{c} = \mathbf{x}G$. Then,

$$H \cdot \mathbf{c}^T = H \cdot (\mathbf{x}G)^T = HG^T \mathbf{x}^T = (GH^T)^T \mathbf{x}^T = \mathbf{0},$$

which implies that $\mathbf{c} \in S_2$, as desired.

To complete the proof note that as H has full rank, its null space (or S_2) has dimension $n - (n - k) = k$ (this follows from a well known fact from linear algebra called the *rank-nullity theorem*). Now as G has full rank, the dimension of S_1 is also k . Thus, as $S_1 \subseteq S_2$, it has to be the case that $S_1 = S_2$.⁴ □

⁴If not, $S_1 \subset S_2$ which implies that that $|S_2| \geq |S_1| + 1$. The latter is not possible if both S_1 and S_2 have the same dimension.

2.3 Linear Codes and Basic Properties

We now return to the topic of codes and introduce the central concept for this chapter as well as much of this text.

Definition 2.3.1 (Linear Codes). *Let q be a prime power (i.e. $q = p^s$ for some prime p and integer $s \geq 1$). $C \subseteq \mathbb{F}_q^n$ is a linear code if it is a linear subspace of $\mathbb{F}_q^n$. If C has dimension k and distance d then it will be referred to as an $[n, k, d]_q$ or just an $[n, k]_q$ code.*

Theorem 2.2.7 now gives two alternate characterizations of an $[n, k]_q$ linear code C : first, C is generated by a $k \times n$ generator matrix G . Second, C is defined by a $(n - k) \times n$ parity check matrix H . Since these are important concepts for us, we define these formally below before giving examples and consequences.

Definition 2.3.2 (Generator and Parity Check Matrices). *If C is an $[n, k]_q$ linear code then there exists a matrix $G \in \mathbb{F}_q^{k \times n}$ of rank k satisfying*

$$C = \{\mathbf{x} \cdot G \mid \mathbf{x} \in \mathbb{F}_q^k\}.$$

G is referred to as a generator matrix of C . In other words, the code C is the set of all possible linear combinations of rows of G .

If C is an $[n, k]_q$ linear code then there exists a matrix $H \in \mathbb{F}_q^{(n-k) \times n}$ of rank $n - k$ satisfying

$$C = \{\mathbf{y} \in \mathbb{F}_q^n \mid H \cdot \mathbf{y}^T = \mathbf{0}\}.$$

H is referred to as a parity check matrix of C .

Note that we require G and H to have full row rank (i.e., the rows of G are linearly independent and the same holds for H). Sometimes we will consider matrices $M \in \mathbb{F}_q^{m \times n}$ that are not of full row rank. These can still be used to generate a code $C = \{\mathbf{x} \cdot G \mid \mathbf{x} \in \mathbb{F}_q^m\}$ though the code C will not be an $[n, m]_q$ code. We will still refer to C as the code generated by M in such a case, though the phrase “generator matrix” will be reserved for full rank matrices.

Note that neither the generator matrix nor the parity check matrix are unique for a given code. However, all generator matrices (and parity check matrices) have the same dimensions, i.e. all are $k \times n$ (and $(n - k) \times n$ respectively) matrices. We give examples of these matrices for the case of the $[7, 4, 3]_2$ Hamming code below.

- The $[7, 4, 3]_2$ Hamming code has the following generator matrix:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- The following matrix is a parity check matrix of the $[7, 4, 3]_2$ Hamming code:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

Indeed, it can be easily verified that $G \cdot H^T = \mathbf{0}$. Then Lemma 2.2.8 proves that H is a parity check matrix of the $[7, 4, 3]_2$ Hamming code.

We now look at some consequences of the above characterizations of an $[n, k]_q$ linear code C . We started this chapter with a quest for succinct representation of a code. Note that both the generator matrix and the parity check matrix can be represented using $O(n^2)$ symbols from $\mathbb{F}_q$. Note that this is much smaller than the exponential representation of a general code. More precisely we have the following result on succinct representations of a linear code (see also Exercise 2.11):

Proposition 2.3.3. *Any $[n, k]_q$ linear code can be represented with $\min(nk, n(n-k))$ symbols from $\mathbb{F}_q$.*

There is an encoding algorithm for C that runs in $O(n^2)$ (in particular $O(kn)$) time— given a message $\mathbf{m} \in \mathbb{F}_q^k$, the corresponding codeword $C(\mathbf{m}) = \mathbf{m} \cdot G$, where G is the generator matrix of C . (See Exercise 2.12.)

Proposition 2.3.4. *For any $[n, k]_q$ linear code, given its generator matrix, encoding can be done with $O(nk)$ operations over $\mathbb{F}_q$.*

There is an error-detecting algorithm for C that runs in $O(n^2)$. This is a big improvement over the naive brute force exponential time algorithm (that goes through all possible codewords $\mathbf{c} \in C$ and checks if $\mathbf{y} = \mathbf{c}$). (See Exercise 2.13.)

Proposition 2.3.5. *For any $[n, k]_q$ linear code, given its parity check matrix, error detection can be performed in $O(n(n-k))$ operations over $\mathbb{F}_q$.*

Next, we look at some alternate characterizations of the distance of a linear code.

2.3.1 On the Distance of a Linear Code

Linear codes admit a nice characterization of minimum distance in terms of the Hamming weight of non-zero codewords, which we have seen for the special case of binary linear codes (Proposition 1.5.4). Recall that we use $\text{wt}(x)$ to denote the Hamming weight of a vector $x \in \Sigma^n$, i.e., the number of non-zero coordinates in x .

Proposition 2.3.6. *For every $[n, k, d]_q$ code C , we have*

$$d = \min_{\substack{\mathbf{c} \in C, \\ \mathbf{c} \neq \mathbf{0}}} \text{wt}(\mathbf{c}).$$

Proof. To show that d is the same as the minimum weight we show that d is no more than the minimum weight and d is no less than the minimum weight.

First, we show that d is no more than the minimum weight. We can see this by considering $\Delta(\mathbf{0}, \mathbf{c}')$ where $\mathbf{c}'$ is the non-zero codeword in C with minimum weight; its distance from $\mathbf{0}$ is equal to its weight. Thus, we have $d \leq \text{wt}(\mathbf{c}')$, as desired.

Now, to show that d is no less than the minimum weight, consider $\mathbf{c}_1 \neq \mathbf{c}_2 \in C$ such that $\Delta(\mathbf{c}_1, \mathbf{c}_2) = d$. Note that $\mathbf{c}_1 - \mathbf{c}_2 \in C$ (this is because $-\mathbf{c}_2 = -1 \cdot \mathbf{c}_2 \in C$, where -1 is the additive inverse of 1 in $\mathbb{F}_q$ and $\mathbf{c}_1 - \mathbf{c}_2 = \mathbf{c}_1 + (-\mathbf{c}_2)$, which is in C by the definition of linear codes). Now note that $\text{wt}(\mathbf{c}_1 - \mathbf{c}_2) = \Delta(\mathbf{c}_1, \mathbf{c}_2) = d$, since the non-zero symbols in $\mathbf{c}_1 - \mathbf{c}_2$ occur exactly in the positions where the two codewords differ. Further, since $\mathbf{c}_1 \neq \mathbf{c}_2$, $\mathbf{c}_1 - \mathbf{c}_2 \neq \mathbf{0}$, which implies that the minimum Hamming weight of any non-zero codeword in C is at most d . $\square$

Next, we look at another property implied by the parity check matrix of a linear code.

Proposition 2.3.7. *For every $[n, k, d]_q$ code C with parity check matrix H , d equals the size of the smallest subset of columns of H that are linearly dependent.*

Proof. By Proposition 2.3.6, we need to show that the minimum weight of a non-zero codeword in C is the minimum number of linearly dependent columns. Let t be the minimum number of linearly dependent columns in H . To prove the claim we will show that $t \leq d$ and $t \geq d$.

For the first direction, Let $\mathbf{c} \neq \mathbf{0} \in C$ be a codeword with $\text{wt}(\mathbf{c}) = d$. Now note that, by the definition of the parity check matrix, $H \cdot \mathbf{c}^T = \mathbf{0}$. Working through the matrix multiplication, this gives us that $\sum_{i=1}^n c_i H^i = \mathbf{0}$, where

$$H = \begin{pmatrix} \uparrow & \uparrow & \dots & \uparrow & \dots & \uparrow \\ H^1 & H^2 & \dots & H^i & \dots & H^n \\ \downarrow & \downarrow & \dots & \downarrow & \dots & \downarrow \end{pmatrix}$$

and $\mathbf{c} = (c_1, \dots, c_n)$. Note that we can skip multiplication for those columns for which the corresponding bit c_i is zero, so for $H \cdot \mathbf{c}^T$ to be zero, those H^i with $c_i \neq 0$ are linearly dependent. This means that $d \geq t$, as the columns corresponding to non-zero entries in $\mathbf{c}$ are one instance of linearly dependent columns.

For the other direction, consider the minimum set of columns from $H, H^{i_1}, H^{i_2}, \dots, H^{i_t}$ that are linearly dependent. This implies that there exists non-zero elements $c'_{i_1}, \dots, c'_{i_t} \in \mathbb{F}_q$ such that $c'_{i_1} H^{i_1} + \dots + c'_{i_t} H^{i_t} = \mathbf{0}$. (Note that all the c'_{i_j} are non-zero as no set of less than t columns are linearly dependent.) Now extend $c'_{i_1}, \dots, c'_{i_t}$ to the vector $\mathbf{c}'$ such that $c'_j = 0$ for $j \notin \{i_1, \dots, i_t\}$. Note that we have $H \cdot (\mathbf{c}')^T = \mathbf{0}$ and thus, we have $\mathbf{c}' \in C$. This in turn implies that $d \leq \text{wt}(\mathbf{c}') = t$ (where recall t is the minimum number of linearly independent columns in H). $\square$

2.4 Hamming Codes

We now change gears and look at the general family of linear codes, which were discovered by Hamming. So far, we have seen the $[7, 4, 3]_2$ Hamming code (in Section 1.5). In fact, for any $r \geq 2$

there is a $[2^r - 1, 2^r - r - 1, 3]_2$ Hamming code. Thus in Section 1.5, we have seen this code for $r = 3$.

Definition 2.4.1 (Binary Hamming Codes). *For any positive integer r , define the matrix $\mathbf{H}_r \in \mathbb{F}_2^{r \times (2^r - 1)}$ to be the $r \times (2^r - 1)$ matrix whose i th column $\mathbf{H}_r^i$ is the binary representation of i , for $1 \leq i \leq 2^r - 1$. (Note that such a representation is a vector in $\{0, 1\}^r$.)*

The $[2^r - 1, 2^r - r - 1]_2$ Hamming code, denoted by $C_{H,r}$, is the code with parity check matrix $\mathbf{H}_r$.

In other words, the general $[2^r - 1, 2^r - r - 1]_2$ Hamming code is the code

$$\{\mathbf{c} \in \{0, 1\}^{2^r - 1} \mid \mathbf{H}_r \cdot \mathbf{c}^T = \mathbf{0}\}.$$

For example, for the case we have seen ($r = 3$),

$$\mathbf{H}_3 = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix},$$

and the resulting code was a $[7, 4, 3]_2$ code.

Next we argue that the above Hamming code has distance 3 (in Proposition 1.5.2, we argued this for $r = 3$).

Proposition 2.4.2. *The Hamming code $[2^r - 1, 2^r - r - 1, 3]_2$ has distance 3.*

Proof. No two columns in $\mathbf{H}_r$ are linearly dependent. If they were, we would have $\mathbf{H}_r^i + \mathbf{H}_r^j = \mathbf{0}$, but this is impossible since they differ in at least one bit (being binary representations of integers, $i \neq j$). Thus, by Proposition 2.3.7, the distance is at least 3. It is at most 3, since (e.g.) $\mathbf{H}_r^1 + \mathbf{H}_r^2 + \mathbf{H}_r^3 = \mathbf{0}$. $\square$

Now note that under the Hamming bound for $d = 3$ (Theorem 1.6.2), $k \leq n - \log_2(n + 1)$, so for $n = 2^r - 1$, $k \leq 2^r - r - 1$. Hence, the Hamming code is a perfect code. (See Definition 1.7.3.)

In Question 1.7.1, we asked which codes are perfect codes. Interestingly, the only perfect binary codes are the following:

- The Hamming codes which we just studied.
- The trivial $[n, 1, n]_2$ codes for odd n (which have 0^n and 1^n as the only codewords): see Exercise 2.24.
- Two codes due to Golay [63].

2.5 Efficient Decoding of Hamming codes

We have shown that the Hamming code has a distance of 3 and thus, by Proposition 1.4.2, can correct one error. However, this is a *combinatorial* result and does not give us an efficient algorithm. One obvious candidate for decoding is the MLD function (Algorithm 1.4.1). Unfortunately, the only implementation of MLD that we know is the one in Algorithm 1.4.1, which will take time $2^{\Theta(n)}$, where n is the block length of the Hamming code.

However, we can do much better. Consider the following simple algorithm: given the received word $\mathbf{y}$, first check if it is indeed a valid codeword. If it is, we are done. Otherwise, flip each of the n bits and check if the resulting vector is a valid codeword. If so, we have successfully decoded from one error. If none of the checks are successful, then we declare a decoding failure. Algorithm 2.5.1 formally presents this algorithm (where $C_{H,r}$ is the $[2^r - 1, 2^r - r - 1, 3]_2$ Hamming code).⁵

Algorithm 2.5.1 Naive Decoder for Hamming Code

INPUT: Received word $\mathbf{y}$

OUTPUT: $\mathbf{c}$ if $\Delta(\mathbf{y}, \mathbf{c}) \leq 1$ else Fail

```

1: IF  $\mathbf{y} \in C_{H,r}$  THEN
2:   RETURN  $\mathbf{y}$ 
3: FOR  $i = 1 \dots n$  DO
4:    $\mathbf{y}' \leftarrow \mathbf{y} + \mathbf{e}_i$                                  $\triangleright \mathbf{e}_i$  is the  $i$ th standard basis vector
5:   IF  $\mathbf{y}' \in C_{H,r}$  THEN
6:     RETURN  $\mathbf{y}'$ 
7: RETURN Fail

```

It can be verified that Algorithm 2.5.1 can correct up to 1 error. If each of the checks $\mathbf{y}' \in C_{H,r}$ can be done in $T(n)$ time, then the time complexity of the proposed algorithm will be $O(nT(n))$. Note that since $C_{H,r}$ is a linear code (and dimension $k = n - O(\log n)$) by Proposition 2.3.5, we have $T(n) = O(n \log n)$. Thus, the proposed algorithm has running time $O(n^2 \log n)$.

Note that Algorithm 2.5.1 can be generalized to work for any linear code C with distance $2t + 1$ (and hence, can correct up to t errors): go through all possible error vectors $\mathbf{z} \in [q]^n$ (with $wt(\mathbf{z}) \leq t$) and check if $\mathbf{y} - \mathbf{z}$ is in the code or not. Algorithm 2.5.2 presents the formal algorithm (where C is an $[n, k, 2t + 1]_q$ code).

The number of error patterns $\mathbf{z}$ considered by Algorithm 2.5.2 is $\sum_{i=0}^t \binom{n}{i} (q-1)^i \leq O((nq)^t)$. Furthermore by Proposition 2.3.5, Step 4 can be performed with $O(n^2)$ operations over $\mathbb{F}_q$. Thus, Algorithm 2.5.2 runs with $O(n^{t+2} q^t)$ operations over $\mathbb{F}_q$, which for q being a small polynomial in n , is $n^{O(t)}$ operations. In other words, the algorithm will have polynomial running time for codes

⁵Formally speaking, a decoding algorithm should return the transmitted message $\mathbf{x}$ but Algorithm 2.5.1 actually returns $C_{H,r}(\mathbf{x})$. However, since $C_{H,r}$ is a linear code, it is not too hard to see that one can obtain $\mathbf{x}$ from $C_{H,r}(\mathbf{x})$ in $O(n^3)$ time: see Exercise 2.25. Further, for $C_{H,r}$ one can do this in $O(n)$ time: see Exercise 2.26.

⁶Recall (1.18).

Algorithm 2.5.2 Decoder for Any Linear Code

INPUT: Received word $\mathbf{y}$ OUTPUT: $\mathbf{c} \in C$ if $\Delta(\mathbf{y}, \mathbf{c}) \leq t$ else Fail

```

1: FOR  $i = 0 \dots t$  DO
2:   FOR  $S \subseteq [n]$  such that  $|S| = i$  DO
3:     FOR  $\mathbf{z} \in \mathbb{F}_q^n$  such that  $wt(\mathbf{z}_S) = wt(\mathbf{z}) = i$  DO
4:       IF  $\mathbf{y} - \mathbf{z} \in C$  THEN
5:         RETURN  $\mathbf{y} - \mathbf{z}$ 
6: RETURN Fail

```

with a constant distance (though the running time would not be practical even for moderate values of t).

However, it turns out that for Hamming codes there exists a decoding algorithm with an $O(n^2)$ running time. To see this, first note that if the received word $\mathbf{y}$ has no errors, then $\mathbf{H}_r \cdot \mathbf{y}^T = \mathbf{0}$. If not, then $\mathbf{y} = \mathbf{c} + \mathbf{e}_i$, where $\mathbf{c} \in C$ and $\mathbf{e}_i$ is the unit vector with the only nonzero element at the i -th position. Thus, if $\mathbf{H}_r^i$ stands for the i -th column of $\mathbf{H}_r$,

$$\mathbf{H}_r \cdot \mathbf{y}^T = \mathbf{H}_r \cdot \mathbf{c}^T + \mathbf{H}_r \cdot (\mathbf{e}_i)^T = \mathbf{H}_r \cdot (\mathbf{e}_i)^T = \mathbf{H}_r^i,$$

where the second equality follows as $\mathbf{H}_r \cdot \mathbf{c}^T = \mathbf{0}$, which in turn follows from the fact that $\mathbf{c} \in C$. In other words, $\mathbf{H}_r \cdot \mathbf{y}^T$ gives the *location* of the error. This leads to Algorithm 2.5.3.

Algorithm 2.5.3 Efficient Decoder for Hamming Code

INPUT: Received word $\mathbf{y}$ OUTPUT: $\mathbf{c}$ if $\Delta(\mathbf{y}, \mathbf{c}) \leq 1$ else Fail

```

1:  $\mathbf{b} \leftarrow \mathbf{H}_r \cdot \mathbf{y}^T$ .
2: Let  $i \in [n]$  be the number whose binary representation is  $\mathbf{b}$ 
3: IF  $\mathbf{y} - \mathbf{e}_i \in C$  THEN
4:   RETURN  $\mathbf{y} - \mathbf{e}_i$ 
5: RETURN Fail

```

Note that $\mathbf{H}_r$ is an $r \times n$ matrix where $n = 2^r - 1$ and thus, $r = \Theta(\log n)$. This implies Step 1 in Algorithm 2.5.3, which is a matrix vector multiplication can be done in time $O(n \log n)$. By a similar argument and by Proposition 2.3.5 Step 3 can be performed in $O(n \log n)$ time, and therefore Algorithm 2.5.3 overall runs in $O(n \log n)$ time. Thus,

Theorem 2.5.1. *The $[n = 2^r - 1, 2^r - r - 1, 3]_2$ Hamming code is 1-error correctable. Furthermore, decoding can be performed in time $O(n \log n)$.*

2.6 Dual of a Linear Code

Until now, we have thought of parity check matrix as defining a code via its null space. However, we are not beholden to think of the parity check matrix in this way. A natural alternative is to use the parity check matrix as a generator matrix. The following definition addresses this question.

Definition 2.6.1 (Dual of a code). *Let H be a parity check matrix of a code C , then the code generated by H is called the dual of C . The dual of a code C is denoted by $C^\perp$.*

It is obvious from the definition that if C is an $[n, k]_q$ code, then $C^\perp$ is an $[n, n - k]_q$ code. Applying duality to the Hamming codes and a close relative, we get two families of codes described below.

Definition 2.6.2 (Simplex and Hadamard Codes). *For positive integer r the Simplex Code $C_{Sim,r}$ is the code generated by H_r . (Equivalently $C_{Sim,r} = C_{H,r}^\perp$.) For positive integer r the Hadamard Code $C_{Had,r}$ is the $[2^r, r]_2$ code generated by the $r \times 2^r$ matrix H_r^1 obtained by adding the all zero column to (say in front of columns in) H_r .*

We claim that $C_{Sim,r}$ and $C_{Had,r}$ are $[2^r - 1, r, 2^{r-1}]_2$ and $[2^r, r, 2^{r-1}]_2$ codes respectively. The claimed block length and dimension follow from the definition of the codes, while the distance follows from the following result.

Proposition 2.6.3. *$C_{Sim,r}$ and $C_{Had,r}$ both have distances of 2^{r-1} .*

Proof. We first show the result for $C_{Had,r}$. In fact, we will show something stronger: every non-zero codeword in $C_{Had,r}$ has weight exactly equal to 2^{r-1} (the claimed distance follows from Proposition 2.3.6). Consider a message $\mathbf{x} \neq \mathbf{0}$. Let its i th entry be $x_i = 1$. $\mathbf{x}$ is encoded as

$$\mathbf{c} = (x_1, x_2, \dots, x_r)(H_r^0, H_r^1, \dots, H_r^{2^r-1}),$$

where H_r^j is the binary representation of $0 \leq j \leq 2^r - 1$ (that is, the set of vector H_r^j is exactly the set of all the vectors in $\{0, 1\}^r$). Further note that the j th bit of the codeword $\mathbf{c}$ is $\langle \mathbf{x}, H_r^j \rangle$. Group all the columns of the generator matrix into pairs $(\mathbf{u}, \mathbf{v})$ such that $\mathbf{v} = \mathbf{u} + \mathbf{e}_i$ (i.e. $\mathbf{v}$ and $\mathbf{u}$ are the same except in the i th position). For example for $r = 3$ and $i = 2$, the paired up columns are marked with the same color below:

$$\begin{pmatrix} 0 & \mathbf{0} & 0 & \mathbf{0} & 1 & \mathbf{1} & 1 & \mathbf{1} \\ 0 & \mathbf{0} & 1 & \mathbf{1} & 0 & \mathbf{0} & 1 & \mathbf{1} \\ 0 & \mathbf{1} & 0 & \mathbf{1} & 0 & \mathbf{1} & 0 & \mathbf{1} \end{pmatrix}$$

Notice that this partitions all the columns into 2^{r-1} disjoint pairs. Then,

$$\langle \mathbf{x}, \mathbf{v} \rangle = \langle \mathbf{x}, \mathbf{u} + \mathbf{e}_i \rangle = \langle \mathbf{x}, \mathbf{u} \rangle + \langle \mathbf{x}, \mathbf{e}_i \rangle = \langle \mathbf{x}, \mathbf{u} \rangle + x_i = \langle \mathbf{x}, \mathbf{u} \rangle + 1.$$

Thus we have that $\langle \mathbf{x}, \mathbf{v} \rangle$ is the negation of $\langle \mathbf{x}, \mathbf{u} \rangle$, i.e. exactly one of $\langle \mathbf{x}, \mathbf{v} \rangle$ and $\langle \mathbf{x}, \mathbf{u} \rangle$ is 1. As the choice of the pair $(\mathbf{u}, \mathbf{v})$ was arbitrary, we have proved that for any non-zero codeword $\mathbf{c}$ such that $\mathbf{c} \in C_{Had,r}$, $wt(\mathbf{c}) = 2^{r-1}$.

For the simplex code, we observe that all codewords of $C_{Had,r}$ are obtained by padding a 0 to the beginning of the codewords in $C_{Sim,r}$, which implies that all non-zero codewords in $C_{Sim,r}$ also have a weight of 2^{r-1} , which completes the proof. $\square$

We remark that the family of Hamming code has a rate of 1 and a (relative) distance of 0 while the families of Simplex/Hadamard codes have a rate of 0 and a relative distance of 1/2. Thus neither gives a positive answer to Question 1.8.2 and so the quest for an asymptotically good code remains ongoing for now (and we will get to these in future chapters).

2.7 Exercises

Exercise 2.1. Let $(S, +, \cdot)$ be a field (as per Definition 2.1.2). Then argue that $a \cdot 0 = 0 \cdot a = 0$ for every $a \in S$.

Exercise 2.2. Prove that the set of rationals (i.e. the set of reals of the form $\frac{a}{b}$, where both a and $b \neq 0$ are integers), denoted by $\mathbb{Q}$, is a field.

Exercise 2.3. Let q be a prime power. Let $x \in \mathbb{F}_q$ such that $x \notin \{0, 1\}$. Then prove that for any $n \leq q - 1$:

$$\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}.$$

Exercise 2.4. The main aim of this exercise is to prove the following identity that is true for any $\alpha \in \mathbb{F}_q$:

$$\alpha^q = \alpha \quad (2.4)$$

To make progress towards the above we will prove a sequence of properties of groups. A group G is a pair $(S, \circ)$ where the operator $\circ : G \times G \rightarrow G$ such that $\circ$ is commutative⁷ and the elements of S are closed under $\circ$. Further, there is a special element $\iota \in S$ that is the identity element and every element $a \in S$ has an inverse element $b \in S$ such that $a \circ b = \iota$. Note that a finite field $\mathbb{F}_q$ consists of an additive group with the $+$ operator (and 0 as additive identity) and a multiplicative group on the non-zero elements of $\mathbb{F}_q$ (which is also denoted by $\mathbb{F}_q^*$) with the $\cdot$ operator (and 1 as the multiplicative identity).⁸

For the rest of the problem let $G = (S, \cdot)$ be a multiplicative group with $|G| = m$. Prove the following statements.

1. For any $\beta \in G$, let $o(\beta)$ be the smallest integer o such that $\beta^o = 1$. Prove that such an $o \leq m$ always exists. Further, argue that $T = \{1, \beta, \dots, \beta^{o-1}\}$ also forms a group. $(T, \cdot)$ is called a sub-group of G and $o(\beta)$ is called the order of β .
2. For any $g \in G$, define the coset (w.r.t. T) as

$$gT = \{g \cdot \beta \mid \beta \in T\}.$$

Prove that if $h^{-1} \cdot g \in T$ then $gT = hT$ and $gT \cap hT = \emptyset$ otherwise. Further argue that these cosets partition the group G into disjoint sets.

⁷Technically, G is an abelian group.

⁸Recall Definition 2.1.2.

3. Argue that for any $g \in G$, we have $|gT| = |T|$.

4. Using the above results or otherwise, argue that for any $\beta \in G$, we have

$$\beta^m = 1.$$

5. Prove (2.4).

Exercise 2.5. Prove that for $q = 2$, the second condition in Definition 2.2.2 is implied by the first condition.

Exercise 2.6. Prove that G_2 from (2.3) has full rank.

Exercise 2.7. In this problem we will look at the problem of solving a system of linear equations over $\mathbb{F}_q$. That is, one needs to solve for unknowns $x_1, \dots, x_n$ given the following m linear equations (where $a_{i,j}, b_i \in \mathbb{F}_q$ for $1 \leq i \leq m$ and $1 \leq j \leq n$):

$$a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n}x_n = b_1.$$

$$a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,n}x_n = b_2.$$

$$\vdots$$

$$a_{m,1}x_1 + a_{m,2}x_2 + \dots + a_{m,n}x_n = b_m.$$

1. (Warm-up) Convince yourself that the above problem can be stated as $A \cdot \mathbf{x}^T = \mathbf{b}^T$, where A is an $m \times n$ matrix over $\mathbb{F}_q$, $\mathbf{x} \in \mathbb{F}_q^n$ and $\mathbf{b} \in \mathbb{F}_q^m$.
2. (Upper Triangular Matrix) Assume $n = m$ and that A is upper triangular, i.e. all diagonal elements $(a_{i,i})$ are non-zero and all lower triangular elements $(a_{i,j}, i > j)$ are 0. Then present an $O(n^2)$ time⁹ algorithm to compute the unknown vector $\mathbf{x}$.
3. (Gaussian Elimination) Assume that A has full rank (or equivalently a rank of n .)
 - (a) Prove that the following algorithm due to Gauss converts A into an upper triangular matrix. By permuting the columns if necessary make sure that $a_{1,1} \neq 0$. (Why can one assume w.l.o.g. that this can be done?) Multiply all rows $1 < i \leq n$ with $\frac{a_{1,1}}{a_{i,1}}$ and then subtract $a_{1,j}$ from the (i, j) th entry $1 \leq j \leq n$. Recurse with the same algorithm on the $(n-1) \times (n-1)$ matrix A' obtained by removing the first row and column from A . (Stop when $n = 1$.)
 - (b) What happens if A does not have full rank? Show how one can modify the algorithm above to either upper triangulate a matrix or report that it does not have full rank. (Convince yourself that your modification works.)

⁹For this problem, any basic operation over $\mathbb{F}_q$ takes unit time.

(c) Call a system of equations $A \cdot \mathbf{x}^T = \mathbf{b}^T$ consistent if there exists a solution to $\mathbf{x} \in \mathbb{F}_q^n$. Show that there exists an $O(n^3)$ algorithm that finds the solution if the system of equations is consistent and A has full rank (and report “fail” otherwise).

4. ($m < n$ case) Assume that A has full rank, i.e. has a rank of m . In this scenario either the system of equations is inconsistent or there are q^{n-m} solutions to $\mathbf{x}$. Modify the algorithm from above to design an $O(m^2 n)$ time algorithm to output the solutions (or report that the system is inconsistent).

- Note that in case the system is consistent there will be q^{n-m} solutions, which might be much bigger than $O(m^2 n)$. Show that this is not a problem as one can represent the solutions as system of linear equations. (I.e. one can have $n - m$ “free” variables and m “bound” variables.)

5. ($m > n$ case) Assume that A has full rank, i.e. a rank of n . In this scenario either the system of equations is inconsistent or there is a unique solution to $\mathbf{x}$. Modify the algorithm from above to design an $O(m^2 n)$ time algorithm to output the solution (or report that the system is inconsistent).

6. (Non-full rank case) Give an $O(m^2 n)$ algorithm for the general case, i.e. the $m \times n$ matrix A need not have full rank. (The algorithm should either report that the system of equations is inconsistent or output the solution(s) to $\mathbf{x}$.)

Exercise 2.8. Prove that the span of k linearly independent vectors over $\mathbb{F}_q$ has size exactly q^k .

Exercise 2.9. Let G and H be a generator and parity check matrix of the same linear code of dimension k and block length n . Then $G \cdot H^T = \mathbf{0}$.

Exercise 2.10. Let C be an $[n, k]_q$ linear code with a generator matrix with no all zeros columns. Then for every position $i \in [n]$ and $\alpha \in \mathbb{F}_q$, the number of codewords $\mathbf{c} \in C$ such that $c_i = \alpha$ is exactly q^{k-1} .

Exercise 2.11. Prove Proposition 2.3.3.

Exercise 2.12. Prove Proposition 2.3.4.

Exercise 2.13. Prove Proposition 2.3.5.

Exercise 2.14. A set of vector $S \subseteq \mathbb{F}_q^n$ is called t -wise independent if for every set of positions I with $|I| = t$, the set S projected to I has each of the vectors in $\mathbb{F}_q^t$ appear the same number of times. (In other words, if one picks a vector $(s_1, \dots, s_n)$ from S at random then any of the t random variables are uniformly and independently random over $\mathbb{F}_q$).

Prove that any linear code C whose dual $C^\perp$ has distance $d^\perp$ is $(d^\perp - 1)$ -wise independent.

Exercise 2.15. A set of vectors $S \subseteq \mathbb{F}_2^k$ is called ϵ -biased sample space if the following property holds. Pick a vector $X = (x_1, \dots, x_k)$ uniformly at random from S . Then X has bias at most ϵ , that is, for every $I \subseteq [k]$,

$$\left| \Pr \left(\sum_{i \in I} x_i = 0 \right) - \Pr \left(\sum_{i \in I} x_i = 1 \right) \right| \leq \epsilon.$$

We will look at some connections of such sets to codes.

1. Let C be an $[n, k]_2$ code such that all non-zero codewords have Hamming weight in the range $[(\frac{1-\epsilon}{2})n, (\frac{1+\epsilon}{2})n]$. Then there exists an ϵ -biased space of size n in $\mathbb{F}_2^k$.
2. Let C be an $[n, k]_2$ code such that all non-zero codewords have Hamming weight in the range $[(\frac{1}{2} - \gamma)n, (\frac{1}{2} + \gamma)n]$ for some constant $0 < \gamma < 1/2$. Then there exists an ϵ -biased space in $\mathbb{F}_2^k$ of size $n^{O(\gamma^{-1} \cdot \log(1/\epsilon))}$.

Exercise 2.16. Let C be an $[n, k, d]_q$ code. Let $\mathbf{y} = (y_1, \dots, y_n) \in (\mathbb{F}_q \cup \{?\})^n$ be a received word¹⁰ such that $y_i = ?$ for at most $d - 1$ values of i . Present an $O(n^3)$ time algorithm that outputs a codeword $\mathbf{c} = (c_1, \dots, c_n) \in C$ that agrees with $\mathbf{y}$ in all un-erased positions (i.e., $c_i = y_i$ if $y_i \neq ?$) or states that no such $\mathbf{c}$ exists. (Recall that if such a $\mathbf{c}$ exists then it is unique.)

Exercise 2.17. In the chapter, we did not talk about how to obtain the parity check matrix of a linear code from its generator matrix. In this problem, we will look at this “conversion” procedure.

- (a) Prove that any generator matrix $\mathbf{G}$ of an $[n, k]_q$ code C (recall that $\mathbf{G}$ is a $k \times n$ matrix) can be converted into another equivalent generator matrix of the form $\mathbf{G}' = [\mathbf{I}_k | \mathbf{A}]$, where $\mathbf{I}_k$ is the $k \times k$ identity matrix and $\mathbf{A}$ is some $k \times (n - k)$ matrix. By “equivalent,” we mean that the code generated by $\mathbf{G}'$ has a linear bijective map to C .

Note that the code generated by $\mathbf{G}'$ has the message symbols as its first k symbols in the corresponding codeword. Such codes are called systematic codes. In other words, every linear code can be converted into a systematic code. Systematic codes are popular in practice as they allow for immediate access to the message symbols.

- (b) Given an $k \times n$ generator matrix of the form $[\mathbf{I}_k | \mathbf{A}]$, give a corresponding $(n - k) \times n$ parity check matrix. Briefly justify why your construction of the parity check matrix is correct.

Hint: Try to think of a parity check matrix that can be decomposed into two submatrices: one will be closely related to $\mathbf{A}$ and the other will be an identity matrix, though the latter might not be a $k \times k$ matrix).

- (c) Use part (b) to present a generator matrix for the $[2^r - 1, 2^r - r - 1, 3]_2$ Hamming code.

Exercise 2.18. So far in this book we have seen that one can modify one code to get another code with interesting properties (for example, the construction of the Hadamard code from the Simplex code from Section 2.6 and Exercise 1.7). In this problem you will need to come up with more ways of constructing new codes from existing ones.

¹⁰A ? denotes an erasure.

Prove the following statements (recall that the notation $(n, k, d)_q$ code is used for general codes with q^k codewords where k need not be an integer, whereas the notation $[n, k, d]_q$ code stands for a linear code of dimension k):

1. If there exists an $(n, k, d)_{2^m}$ code, then there also exists an $(nm, km, d' \geq d)_2$ code.
2. If there exists an $[n, k, d]_{2^m}$ code, then there also exists an $[nm, km, d' \geq d]_2$ code.
3. If there exists an $[n, k, d]_q$ code, then there also exists an $[n - d, k - 1, d' \geq \lceil d/q \rceil]_q$ code.
4. If there exists an $[n, k, \delta n]_q$ code, then for every $m \geq 1$, there also exists an $(n^m, k/m, (1 - (1 - \delta)^m) \cdot n^m)_{q^m}$ code.
5. If there exists an $[n, k, \delta n]_2$ code, then for every odd $m \geq 1$, there also exists an $[n^m, k, \frac{1}{2} \cdot (1 - (1 - 2\delta)^m) \cdot n^m]_2$ code.

Note: In all the parts, the only things that you can assume about the original code are only the parameters given by its definition— nothing else!

Exercise 2.19. Let C_1 be an $[n, k_1, d_1]_q$ code and C_2 be an $[n, k_2, d_2]_q$ code. Then define a new code as follows:

$$C_1 \ominus C_2 = \{(\mathbf{c}_1, \mathbf{c}_1 + \mathbf{c}_2) | \mathbf{c}_1 \in C_1, \mathbf{c}_2 \in C_2\}.$$

Next we will prove interesting properties of this operations on codes:

1. If G_i is the generator matrix for C_i for $i \in [2]$, what is a generator matrix for $C_1 \ominus C_2$?
2. Argue that $C_1 \ominus C_2$ is an $[2n, k_1 + k_2, d \stackrel{\text{def}}{=} \min(2d_1, d_2)]_q$ code.
3. Assume there exists algorithms $\mathcal{A}_i$ for code C_i for $i \in [2]$ such that: (i) $\mathcal{A}_1$ can decode from e errors and s erasures such that $2e + s < d_1$ and (ii) $\mathcal{A}_2$ can decode from $\lfloor (d_2 - 1)/2 \rfloor$ errors. Then argue that one can correct $\lfloor (d - 1)/2 \rfloor$ errors for $C_1 \ominus C_2$.

Hint: Given a received word $(\mathbf{y}_1, \mathbf{y}_2) \in \mathbb{F}_q^n \times \mathbb{F}_q^n$, first apply $\mathcal{A}_2$ on $\mathbf{y}_2 - \mathbf{y}_1$. Then create an intermediate received word for $\mathcal{A}_1$.

4. We will now consider a recursive construction of a binary linear code that uses the $\ominus$ operator. For integers $0 \leq r \leq m$, we define the code $C(r, m)$ as follows:

- $C(r, r) = \mathbb{F}_2^r$ and $C(0, r)$ is the code with only two codewords: the all ones and all zeroes vector in $\mathbb{F}_2^r$.
- For $1 < r < m$, $C(r, m) = C(r, m - 1) \ominus C(r - 1, m - 1)$.

Determine the parameters of the code $C(r, m)$.

Exercise 2.20. Let C_1 be an $[n_1, k_1, d_1]_2$ binary linear code, and C_2 an $[n_2, k_2, d_2]_2$ binary linear code. Let $C \subseteq \mathbb{F}_2^{n_1 \times n_2}$ be the subset of $n_2 \times n_1$ matrices whose rows belong to C_1 and whose columns belong to C_2 . C is called the tensor of C_1 and C_2 and is denoted by $C_1 \otimes C_2$.

Prove that C is an $[n_1 n_2, k_1 k_2, d_1 d_2]_2$ binary linear code.

Further, if $\mathbf{G}_1$ and $\mathbf{G}_2$ are generator matrices of C_1 and C_2 , construct a generator matrix of $C_1 \otimes C_2$ from $\mathbf{G}_1$ and $\mathbf{G}_2$. In particular, argue that given $\mathbf{G}_1$ and $\mathbf{G}_2$, a generator matrix of $C_1 \otimes C_2$ can be computed in polynomial time.

Hint: For the latter problem, it might be useful to think of the codewords and messages as vectors instead of matrices.

Exercise 2.21. In Section 2.4 we considered the binary Hamming code. In this problem we will consider the more general q -ary Hamming code. In particular, let q be a prime power and $r \geq 1$ be an integer. Define the following $r \times n$ matrix $H_{q,r}$, where each column is a non-zero vector from $\mathbb{F}_q^r$ such that the first non-zero entry is 1. For example,

$$H_{3,2} = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 2 \end{pmatrix}$$

In this problem we will derive the parameters of the code. Define the generalized Hamming code $C_{H,q,r}$ to be the linear code whose parity check matrix is $H_{q,r}$. Argue that

1. The block length of $C_{H,q,r}$ is $n = \frac{q^r - 1}{q - 1}$.
2. $C_{H,q,r}$ has dimension $n - r$.
3. $C_{H,q,r}$ has distance 3.

Exercise 2.22. In Section 2.6, we considered the binary Hadamard code. In this problem we will consider the more general q -ary Hadamard code. In particular, let q be a prime power and $r \geq 1$ be an integer. Define the following $r \times q^r$ matrix $\bar{H}_{q,r}$, where each column is a vector in $\mathbb{F}_q^r$. In this problem we will derive the parameters of the code. Define the generalized Hadamard code $C_{Had,r,q}$ to be the linear code whose parity check matrix is $\bar{H}_{q,r}$. Argue that

1. The block length of $C_{Had,r,q}$ is $n = q^r$.
2. $C_{Had,q,r}$ has dimension r .
3. $C_{Had,q,r}$ has distance $\left(1 - \frac{1}{q}\right) \cdot n$.

Exercise 2.23. Design the best 6-ary code (family) with distance 3 that you can.

Hint: Start with a 7-ary Hamming code.

Exercise 2.24. Prove that the $[n, 1, n]_2$ code for odd n (i.e. the code with the all zeros and all ones vector as it only two codewords) attains the Hamming bound (Theorem 1.7.2).

Exercise 2.25. Let C be an $[n, k]_q$ code with generator matrix G . Then given a codeword $\mathbf{c} \in C$ one can compute the corresponding message in time $O(kn^2)$.

Exercise 2.26. Given a $\mathbf{c} \in C_{H,r}$, one can compute the corresponding message in time $O(n)$.

Exercise 2.27. Let C be an $(n, k)_q$ code. Prove that if C can be decoded from e errors in time $T(n)$, then it can be decoded from $n + c$ errors in time $O((nq)^c \cdot T(n))$.

Exercise 2.28. Show that the bound of kd of the number of ones in the generator matrix of any binary linear code (see Exercise 1.14) cannot be improved for every code.

Exercise 2.29. Let C be a linear code. Then prove that $(C^\perp)^\perp = C$.

Exercise 2.30. Note that for any linear code C , the codewords $\mathbf{0}$ is in both C and $C^\perp$. Show that there exists a linear code C such that it shares a non-zero codeword with $C^\perp$.

Exercise 2.31. We go into a bit of diversion and look at how finite fields are different from infinite fields (e.g. $\mathbb{R}$). Most of the properties of linear subspaces that we have used for linear codes (e.g. notion of dimension, the existence of generator and parity check matrices, notion of duals) also hold for linear subspaces over $\mathbb{R}$.¹¹ One trivial property that holds for linear subspaces over finite fields that does not hold over $\mathbb{R}$ is that linear subspaces over $\mathbb{F}_q$ with dimension k has size q^k (though this is a trivial consequence that $\mathbb{F}_q$ is a finite field while $\mathbb{R}$ is an infinite field). Next, we consider a more subtle distinction.

Let $S \subseteq \mathbb{R}^n$ be a linear subspace over $\mathbb{R}$ and let $S^\perp$ is the dual of S . Then show that

$$S \cap S^\perp = \{\mathbf{0}\}.$$

By contrast, linear subspaces over finite fields can have non-trivial intersection with their duals (see e.g. Exercise 2.30).

Exercise 2.32. A linear code C is called self-orthogonal if $C \subseteq C^\perp$. Show that

1. The binary repetition code with even number of repetitions is self-orthogonal.
2. The Hadamard code $C_{Had,r}$ is self-orthogonal.

Exercise 2.33. A linear code C is called self dual if $C = C^\perp$. Show that for

1. Any self dual code has dimension $n/2$.
2. Prove that the following code is self-dual

$$\{(\mathbf{x}, \mathbf{x}) | \mathbf{x} \in \mathbb{F}_2^k\}.$$

Exercise 2.34. Given a code C a puncturing of C is another code C' where the same set of positions are dropped in all codewords of C . More precisely, if $C \subseteq \Sigma^n$ and the set of punctured positions is $P \subseteq [n]$, then the punctured code is $\{(c_i)_{i \notin P} | (c_1, \dots, c_n) \in C\}$.

Prove that a linear code with no repetitions (i.e. there are no two positions $i \neq j$ such that for every codeword $\mathbf{c} \in C$, $c_i = c_j$) is a puncturing of the Hadamard code. Hence, Hadamard code is the “longest” linear code that does not repeat.

¹¹ A linear subspace $S \subseteq \mathbb{R}^n$ is the same as in Definition 2.2.2 where all occurrences of the finite field $\mathbb{F}_q$ is replaced by $\mathbb{R}$.

Exercise 2.35. In this problem we will consider the long code. For the definition, we will use the functional way of looking at the ambient space as mentioned in Remark 1.2.2. A long code of dimension k is a binary code such that the codeword corresponding to $\mathbf{x} = \mathbb{F}_2^k$, is the function $f : \{0, 1\}^{2^k} \rightarrow \{0, 1\}$ defined as follows. For any $\mathbf{m} \in \{0, 1\}^{\mathbb{F}_2^k}$, we have $f((m_\alpha)_{\alpha \in \mathbb{F}_2^k}) = m_{\mathbf{x}}$. Derive the parameters of the long code.

Finally, argue that the long code is the code with the longest block length such that the codewords do not have a repeated coordinate (i.e. there does not exist $i \neq j$ such that for every codeword $\mathbf{c}$, $c_i = c_j$). (Contrast this with the property of Hadamard code above.)

Exercise 2.36. Given a linear code $C \subseteq \mathbb{F}_2^n$, define its generating function to be a $2n$ -variate polynomial over variables $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ given by $G_C(\mathbf{x}, \mathbf{y}) = \sum_{\mathbf{w} \in C} P_{\mathbf{w}}(\mathbf{x}, \mathbf{y})$ where $P_{\mathbf{w}}(\mathbf{x}, \mathbf{y}) = \left(\prod_{\{i \in [n] | w_i = 0\}} x_i \right) \cdot \left(\prod_{\{i \in [n] | w_i = 1\}} y_i \right)$. For $w \in \{0, \dots, n\}$, let A_C^w denote the number of codewords of weight w and let $A_C(z) = \sum_{w=0}^n A_C^w z^w$ be the “weight enumerator” polynomial of C .

1. For every $\mathbf{w} \in \mathbb{F}_2^n$, prove that $P_{\mathbf{w}}(\mathbf{x} + \mathbf{y}, \mathbf{x} - \mathbf{y}) = \sum_{\mathbf{v} \in \mathbb{F}_2^n} (-1)^{\langle \mathbf{v}, \mathbf{w} \rangle} P_{\mathbf{v}}(\mathbf{x}, \mathbf{y})$.
2. Prove that $G_{C^\perp}(\mathbf{x}, \mathbf{y}) = \frac{1}{|C|} G_C(\mathbf{x} + \mathbf{y}, \mathbf{x} - \mathbf{y})$.
3. Prove that $A_C(z) = G_C(1, \dots, 1, z, \dots, z)$.
4. Prove that $A_{C^\perp}(z) = \frac{(1+z)^n}{|C|} A_C\left(\frac{1-z}{1+z}\right)$.
5. Conclude that $A_{C^\perp}^w = \frac{1}{|C|} \sum_{u=0}^n A_C^u \left(\sum_{i=0}^u (-1)^i \binom{u}{i} \binom{n-u}{w-i} \right)$. In other words, the distributions of the weights $(A_C^0, \dots, A_C^n)$ of the primal code completely determine the distributions of weights $(A_{C^\perp}^0, \dots, A_{C^\perp}^n)$ of the dual code!

2.8 Bibliographic Notes

The background material on algebra is essentially folklore. Readers interested in a more extensive treatment are referred to classical texts such as by Artin [10]. For a perspective focussing more on finite fields, see the text by Lidl and Niederreiter [106]. Linear codes arose already in the paper of Hamming [83] and were systematically studied by Slepian [145]. The answer to Question 1.7.1 was given by van Lint [158] and Tietavainen [156]. Hadamard codes (Definition 2.6.2) are named after the work of mathematician Jacques Hadamard and in particular the notion of Hadamard matrices which are self-orthogonal matrices with $+1/-1$ entries.

Exercises 2.14 and 2.15 come from the theory of pseudorandomness, which we will cover more extensively in Chapter 24. The long codes in Exercise 2.35 were introduced by Bellare, Goldreich and Sudan [16]. Exercise 2.36 is based on the MacWilliams Identity proved by MacWilliams [110].

Chapter 3

Probability as Fancy Counting and the q -ary Entropy Function

In the chapters to come we will explore questions of the form: “Given n, k, d and q does an $(n, k, d)_q$ code exist?” To answer such questions, we will apply the “probabilistic method” — the method that demonstrates the existence of an object with a given property by showing that a randomly chosen object has the property with positive probability. To elaborate on this sentence, we need to introduce the basic language and tools of probability theory which we do in Section 3.1.

We then introduce the probabilistic method in Section 3.2. We even apply the method to answer a very simple question:

Question 3.0.1. *Does there exist a $[2, 2, 1]_2$ code?*

We note that the answer to the above question is trivially yes: just pick the generator matrix to be the 2×2 identity matrix. But our proof will have the advantage of generalizing to broader settings, though we save the generalizations for later chapters.

Finally in Section 3.3 we introduce the “entropy function” which turns out to be central in the understanding of limits of codes (both existence and non-existence).

3.1 A Crash Course on Probability

In this section we review basic concepts in probability theory, specialized to the needs of this book. Specifically, we introduce distributions, events and random variables, and give some tools to analyze them.

In this book, we will only consider probability distributions defined over finite spaces. In particular, given a finite domain $\mathbb{D}$, a probability *distribution* is defined as a function

$$p : \mathbb{D} \rightarrow [0, 1] \text{ such that } \sum_{x \in \mathbb{D}} p(x) = 1,$$

G	$\mathcal{U}(G)$	V_{00}	V_{01}	V_{10}	V_{11}
$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	0	0	0
$\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	1	0	1
$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	1	0	1
$\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	2	0	2
$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	0	1	1
$\begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	1	1	0
$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	1	1	2
$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	2	1	1

G	$\mathcal{U}(G)$	V_{00}	V_{01}	V_{10}	V_{11}
$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	0	1	1
$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	1	1	2
$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	1	1	0
$\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	2	1	1
$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	0	2	2
$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	1	2	1
$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$	$\frac{1}{16}$	0	1	2	1
$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$	$\frac{1}{16}$	0	2	2	0

Table 3.1: Uniform distribution over $\mathbb{F}_2^{2 \times 2}$ along with values of four random variables.

where $[0, 1]$ is shorthand for the interval of all real numbers between 0 and 1.

An *event* $\mathbb{E}$ is a predicate over the domain $\mathbb{D}$, i.e. it maps every element of $\mathbb{D}$ to “true” or “false”. Equivalently an event is a subset of the domain $\mathbb{D}$, i.e., those elements that are mapped to true. We switch between “logical” or “set-theoretic” notation to denote combinations of events. So the disjunction of events $\mathbb{E}_1$ and $\mathbb{E}_2$ may be denoted $\mathbb{E}_1 \vee \mathbb{E}_2$ or $\mathbb{E}_1 \cup \mathbb{E}_2$. Similarly, the conjunction of $\mathbb{E}_1$ and $\mathbb{E}_2$ may be denoted $\mathbb{E}_1 \wedge \mathbb{E}_2$ or $\mathbb{E}_1 \cap \mathbb{E}_2$; and the negation of $\mathbb{E}_1$ may be denote $\neg \mathbb{E}_1$ or $\overline{\mathbb{E}_1}$.

In this book, we will primarily deal with the following special distribution:

Definition 3.1.1 (Uniform Distribution). *The uniform distribution over $\mathbb{D}$, denoted by $\mathcal{U}_{\mathbb{D}}$, is given by*

$$\Pr_{\mathcal{U}_{\mathbb{D}}}(x) = \frac{1}{|\mathbb{D}|} \text{ for every } x \in \mathbb{D}.$$

Typically, we will drop the subscript when the domain $\mathbb{D}$ is clear from the context.

For example, consider the domain $\mathbb{D} = \mathbb{F}_2^{2 \times 2}$, i.e. the set of all 2×2 matrices over $\mathbb{F}_2$. (Note that each such matrix is a generator matrix of some $[2, 2]_2$ code.) The first two columns of Table 3.1 list the elements of this $\mathbb{D}$ along with the corresponding probabilities for the uniform distribution.

Typically, we will be interested in a real-valued function defined on $\mathbb{D}$ and how it behaves under a probability distribution defined over $\mathbb{D}$. This is captured by the notion of a random variable¹:

¹We note that the literature on probability theory allows for more general random variables, but for our purposes we restrict only to real-valued ones.

Definition 3.1.2 (Random Variable). *Let $\mathbb{D}$ be a finite domain and $I \subset \mathbb{R}$ be a finite² subset. Let p be a probability distribution defined over $\mathbb{D}$. A random variable is a function:*

$$V : \mathbb{D} \rightarrow I.$$

The expectation of V is defined as

$$\mathbb{E}[V] = \sum_{x \in \mathbb{D}} p(x) \cdot V(x).$$

For example, given $(i, j) \in \{0, 1\}^2$, let V_{ij} denote the random variable $V_{ij}(G) = wt((i, j) \cdot G)$, for any $G \in \mathbb{F}_2^{2 \times 2}$. The last four columns of Table 3.1 list the values of these four random variables.

Of particular interest in this book will be binary random variables, i.e., with $I = \{0, 1\}$. In particular, given an *event* E over $\mathbb{D}$, we will define its *indicator variable* to be a function $\mathbb{1}_E : \mathbb{D} \rightarrow \{0, 1\}$ such that for any $x \in \mathbb{D}$:

$$\mathbb{1}_E(x) = \begin{cases} 1 & \text{if } x \in E \\ 0 & \text{otherwise.} \end{cases}$$

For example,

$$\mathbb{1}_{V_{01}=0}\left(\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}\right) = 1 \text{ and } \mathbb{1}_{V_{01}=0}\left(\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}\right) = 0.$$

In most cases we will shorten this notation to $\mathbb{1}_{E(x)}$ or simply $\mathbb{1}_E$. Finally, sometimes we will abuse notation and use E instead of $\mathbb{1}_E$.

As a further use of indicator variables, consider the expectations of the four indicator variables:

$$\mathbb{E}[\mathbb{1}_{V_{00}=0}] = 16 \cdot \frac{1}{16} = 1.$$

$$\mathbb{E}[\mathbb{1}_{V_{01}=0}] = 4 \cdot \frac{1}{16} = \frac{1}{4}. \tag{3.1}$$

$$\mathbb{E}[\mathbb{1}_{V_{10}=0}] = 4 \cdot \frac{1}{16} = \frac{1}{4}. \tag{3.2}$$

$$\mathbb{E}[\mathbb{1}_{V_{11}=0}] = 4 \cdot \frac{1}{16} = \frac{1}{4}. \tag{3.3}$$

3.1.1 Some Useful Results

Before we proceed, we record a simple property of indicator variables that will be useful. (See Exercise 3.1.)

Lemma 3.1.3. *Let E be any event. Then*

$$\mathbb{E}[\mathbb{1}_E] = \Pr[E \text{ is true}].$$

Next, we state a simple yet useful property of expectation of a sum of random variables:

²In general, I need not be finite. However, for this book this definition suffices.

Proposition 3.1.4 (Linearity of Expectation). *Given random variables $V_1, \dots, V_m$ defined over the same domain $\mathbb{D}$ and with the same probability distribution p , we have*

$$\mathbb{E} \left[\sum_{i=1}^m V_i \right] = \sum_{i=1}^m \mathbb{E}[V_i].$$

Proof. For notational convenience, define $V = V_1 + \dots + V_m$. Thus, we have

$$\mathbb{E}[V] = \sum_{x \in \mathbb{D}} V(x) \cdot p(x) \quad (3.4)$$

$$= \sum_{x \in \mathbb{D}} \left(\sum_{i=1}^m V_i(x) \right) \cdot p(x) \quad (3.5)$$

$$= \sum_{i=1}^m \sum_{x \in \mathbb{D}} V_i(x) \cdot p(x) \quad (3.6)$$

$$= \sum_{i=1}^m \mathbb{E}[V_i]. \quad (3.7)$$

In the equalities above, (3.4) and (3.7) follow from the definition of expectation of a random variable. (3.5) follows from the definition of V and (3.6) follows by switching the order of the two summations. $\square$

As an example, we have

$$\mathbb{E} [\mathbb{1}_{V_{01}=0} + \mathbb{1}_{V_{10}=0} + \mathbb{1}_{V_{11}=0}] = \frac{3}{4} \quad (3.8)$$

Frequently, we will need to deal with the probability of the union of events. We will use the following result to upper bound such probabilities:

Proposition 3.1.5 (Union Bound). *Given m binary random variables $A_1, \dots, A_m$, we have*

$$\Pr \left[\left(\bigvee_{i=1}^m A_i \right) = 1 \right] \leq \sum_{i=1}^m \Pr[A_i = 1].$$

Proof. For every $i \in [m]$, define

$$S_i = \{x \in \mathbb{D} \mid A_i(x) = 1\}.$$

Then we have

$$\Pr \left[\left(\bigvee_{i=1}^m A_i \right) = 1 \right] = \sum_{x \in \bigcup_{i=1}^m S_i} p(x) \quad (3.9)$$

$$\leq \sum_{i=1}^m \sum_{x \in S_i} p(x) \quad (3.10)$$

$$= \sum_{i=1}^m \Pr[A_i = 1]. \quad (3.11)$$

In the above, (3.9) and (3.11) follow from the definition of S_i . (3.10) follows from the fact that some of the $x \in \bigcup_i S_i$ get counted more than once. $\square$

We remark that the union bound is tight when the events are *disjoint*. (In other words, using the notation in the proof above, when $S_i \cap S_j = \emptyset$ for every $i \neq j$.)

As an example, let $A_1 = \mathbb{1}_{V_{01}=0}$, $A_2 = \mathbb{1}_{V_{10}=0}$ and $A_3 = \mathbb{1}_{V_{11}=0}$. Note that in this case the event $A_1 \vee A_2 \vee A_3$ is the same as the event that there exists a non-zero $\mathbf{m} \in \{0, 1\}^2$ such that $w t(\mathbf{m} \cdot G) = 0$. Thus, the union bound implies (that under the uniform distribution over $\mathbb{F}_2^{2 \times 2}$)

$$\Pr [\text{There exists an } \mathbf{m} \in \{0, 1\}^2 \setminus \{(0, 0)\}, \text{ such that } w t(\mathbf{m}G) = 0] \leq \frac{3}{4}. \quad (3.12)$$

Finally, we present two bounds on the probability of a random variable deviating significantly from its expectation. The first bound holds for any random variable:

Lemma 3.1.6 (Markov Bound). *Let V be a non-negative random variable. Then for any $t > 0$,*

$$\Pr[V \geq t] \leq \frac{\mathbb{E}[V]}{t}.$$

In particular, for any $a \geq 1$,

$$\Pr[V \geq a \cdot \mathbb{E}[V]] \leq \frac{1}{a}.$$

Proof. The second bound follows from the first bound by substituting $t = a \cdot \mathbb{E}[V]$. Thus, to complete the proof, we argue the first bound. Consider the following sequence of relations:

$$\mathbb{E}[V] = \sum_{i \in [0, t)} i \cdot \Pr[V = i] + \sum_{i \in [t, \infty)} i \cdot \Pr[V = i] \quad (3.13)$$

$$\geq \sum_{i \geq t} i \cdot \Pr[V = i] \quad (3.14)$$

$$\geq t \cdot \sum_{i \geq t} \Pr[V = i] \quad (3.15)$$

$$= t \cdot \Pr[V \geq t]. \quad (3.16)$$

In the above relations, (3.13) follows from the definition of expectation of a random variable and the fact that V is non-negative. (3.14) follows as we have dropped some non-negative terms. (3.15) follows by noting that in the summands $i \geq t$. (3.16) follows from the definition of $\Pr[V \geq t]$.

The proof is complete by noting that (3.16) implies the claimed bound. $\square$

The second bound works only for sums of *independent* random variables. We begin by defining independent random variables:

Definition 3.1.7 (Independence). *Two random variables A and B are called independent if for every a and b in the ranges of A and B respectively, we have*

$$\Pr[A = a \wedge B = b] = \Pr[A = a] \cdot \Pr[B = b].$$

For example, for the uniform distribution in Table 3.1, let A denote the bit $G_{0,0}$ and B denote the bit $G_{0,1}$. It can be verified that these two random variables are independent. In fact, it can be verified all the random variables corresponding to the four bits in G are independent random variables. (We'll come to a related comment shortly.)

Another related concept that we will use is that of probability of an event happening conditioned on another event happening:

Definition 3.1.8 (Conditional Probability). *Given two events A and B defined over the same domain and probability distribution, we define the probability of A conditioned on B as*

$$\Pr[A|B] = \frac{\Pr[A \text{ and } B]}{\Pr[B]}.$$

For example, note that

$$\Pr[\mathbb{1}_{v_{01}=1} | G_{0,0} = 0] = \frac{4/16}{1/2} = \frac{1}{2}.$$

The above definition implies that two events A and B are independent if and only if $\Pr[A] = \Pr[A|B]$. We will also use the following result later on in the book (see Exercise 3.2):

Lemma 3.1.9. *For any two events A and B defined on the same domain and the probability distribution:*

$$\Pr[A] = \Pr[A|B] \cdot \Pr[B] + \Pr[A|\neg B] \cdot \Pr[\neg B].$$

Next, we state a deviation bound that asserts that the sum of independent random variables takes values close to its expectation with high probability. We only state it for sums of binary random variables, which is the form that will be needed in the book. We refer to this bound as the “Chernoff bound” though we note that this is part of a larger body of work and the bibliographic notes give more details.

Theorem 3.1.10 (Chernoff Bound). *Let $X_1, \dots, X_m$ be independent binary random variables and define $X = \sum X_i$. Then the multiplicative Chernoff bound states that for $0 < \varepsilon \leq 1$,*

$$\Pr[|X - \mathbb{E}(X)| > \varepsilon \mathbb{E}(X)] < 2e^{-\varepsilon^2 \mathbb{E}(X)/3},$$

and the additive Chernoff bound states that

$$\Pr[|X - \mathbb{E}(X)| > \varepsilon m] < 2e^{-\varepsilon^2 m/2}.$$

We omit the proof, which can be found in any standard textbook on randomized algorithms.

Finally, we present an alternate view of uniform distribution over product spaces and then use that view to prove a result that we will use later in the book. Given probability distributions p_1 and p_2 over domains $\mathbb{D}_1$ and $\mathbb{D}_2$ respectively, we define the product distribution $p_1 \times p_2$ over $\mathbb{D}_1 \times \mathbb{D}_2$ as follows: every element $(x, y) \in \mathbb{D}_1 \times \mathbb{D}_2$ under $p_1 \times p_2$ is picked by choosing x from $\mathbb{D}_1$ according to p_1 and y is picked *independently* from $\mathbb{D}_2$ under p_2 . This leads to the following observation (see Exercise 3.4).

Lemma 3.1.11. *For any $m \geq 1$, the distribution $\mathcal{U}_{\mathbb{D}_1 \times \mathbb{D}_2 \times \dots \times \mathbb{D}_m}$ is identical³ to the distribution $\mathcal{U}_{\mathbb{D}_1} \times \mathcal{U}_{\mathbb{D}_2} \times \dots \times \mathcal{U}_{\mathbb{D}_m}$.*

For example, the uniform distribution in Table 3.1 can be described equivalently as follows: pick each of the four bits in G independently and uniformly at random from $\{0, 1\}$.

We conclude this section by proving the following result:

Lemma 3.1.12. *Given a non-zero vector $\mathbf{m} \in \mathbb{F}_q^k$ and a uniformly random $k \times n$ matrix G over $\mathbb{F}_q$, the vector $\mathbf{m} \cdot G$ is uniformly distributed over $\mathbb{F}_q^n$.*

Proof. Let the (j, i) th entry in G ($1 \leq j \leq k, 1 \leq i \leq n$) be denoted by g_{ji} . Note that as G is a random $k \times n$ matrix over $\mathbb{F}_q$, by Lemma 3.1.11, each of the g_{ji} is an independent uniformly random element from $\mathbb{F}_q$. Now, note that we would be done if we can show that for every $1 \leq i \leq n$, the i th entry in $\mathbf{m} \cdot G$ (call it b_i) is an independent uniformly random element from $\mathbb{F}_q$. To finish the proof, we prove this latter fact. If we denote $\mathbf{m} = (m_1, \dots, m_k)$, then $b_i = \sum_{j=1}^k m_j g_{ji}$. Note that the disjoint entries of G participate in the sums for b_i and b_j for $i \neq j$. Given our choice of G , this implies that the random variables b_i and b_j are independent. Hence, to complete the proof we need to prove that b_i is a uniformly independent element of $\mathbb{F}_q$. The rest of the proof is a generalization of the argument we used in the proof of Proposition 2.6.3.

Note that to show that b_i is uniformly distributed over $\mathbb{F}_q$, it is sufficient to prove that b_i takes every value in $\mathbb{F}_q$ equally often over all the choices of values that can be assigned to $g_{1i}, g_{2i}, \dots, g_{ki}$. Now, as $\mathbf{m}$ is non-zero, at least one of the m_j is non-zero. Without loss of generality assume that $m_1 \neq 0$. Thus, we can write $b_i = m_1 g_{1i} + \sum_{j=2}^k m_j g_{ji}$. Now, for every fixed assignment of values to $g_{2i}, g_{3i}, \dots, g_{ki}$ (note that there are q^{k-1} such assignments), b_i takes a different value for each of the q distinct possible assignments to g_{1i} (this is where we use the assumption that $m_1 \neq 0$). Thus, over all the possible assignments of $g_{1i}, \dots, g_{ki}$, b_i takes each of the values in $\mathbb{F}_q$ exactly q^{k-1} times, which proves our claim. $\square$

3.2 The Probabilistic Method

The *probabilistic method* is a very powerful method in combinatorics which can be used to show the existence of objects that satisfy certain properties. In this course, we will use the probabilistic method to prove existence of a code $\mathcal{C}$ with certain property $\mathcal{P}$. Towards that end, we define a distribution $\mathcal{D}$ over all possible codes and prove that when $\mathcal{C}$ is chosen according to $\mathcal{D}$:

$$\Pr[\mathcal{C} \text{ has property } \mathcal{P}] > 0 \text{ or equivalently } \Pr[\mathcal{C} \text{ doesn't have property } \mathcal{P}] < 1.$$

Note that the above inequality proves the existence of $\mathcal{C}$ with property $\mathcal{P}$.

As an example consider Question 3.0.1. To answer this in the affirmative, we note that the set of all $[2, 2]_2$ linear codes is covered by the set of all 2×2 matrices over $\mathbb{F}_2$. Then, we let $\mathcal{D}$ be the uniform distribution over $\mathbb{F}_2^{2 \times 2}$. Then by Proposition 2.3.6 and (3.12), we get that

$$\Pr_{\mathcal{U}_{\mathbb{F}_2^{2 \times 2}}} [\text{There is no } [2, 2, 1]_2 \text{ code}] \leq \frac{3}{4} < 1,$$

³We say two distributions p_1 and p_2 on $\mathbb{D}$ are identical if for every $x \in \mathbb{D}$, $p_1(x) = p_2(x)$.

which by the probabilistic method answers the Question 3.0.1 in the affirmative.

For the more general case, when we apply the probabilistic method, the typical approach will be to define (sub-)properties $P_1, \dots, P_m$ such that $\mathcal{P} = P_1 \wedge P_2 \wedge P_3 \dots \wedge P_m$ and show that for every $1 \leq i \leq m$:

$$\Pr[\mathcal{C} \text{ doesn't have property } P_i] = \Pr[\overline{P_i}] < \frac{1}{m}.$$

Finally, by the union bound, the above will prove that⁴ $\Pr[\mathcal{C} \text{ doesn't have property } \mathcal{P}] < 1$, as desired.

As an example, an alternate way to answer Question 3.0.1 in the affirmative is the following. Define $P_1 = \mathbb{1}_{V_{01} \geq 1}$, $P_2 = \mathbb{1}_{V_{10} \geq 1}$ and $P_3 = \mathbb{1}_{V_{11} \geq 1}$. (Note that we want a $[2, 2]_2$ code that satisfies $P_1 \wedge P_2 \wedge P_3$.) Then, by (3.1), (3.2) and (3.3), we have for $i \in [3]$,

$$\Pr[\mathcal{C} \text{ doesn't have property } P_i] = \Pr[\overline{P_i}] = \frac{1}{4} < \frac{1}{3},$$

as desired.

Finally, we mention a special case of the general probabilistic method that we outlined above. In particular, let $\mathcal{P}$ denote the property that the randomly chosen $\mathcal{C}$ satisfies $f(\mathcal{C}) \leq b$. Then we claim (see Exercise 3.5) that $\mathbb{E}[f(\mathcal{C})] \leq b$ implies that $\Pr[\mathcal{C} \text{ has property } \mathcal{P}] > 0$. Note that this implies that $\mathbb{E}[f(\mathcal{C})] \leq b$ implies that there exists a code $\mathcal{C}$ such that $f(\mathcal{C}) \leq b$.

3.3 The q -ary Entropy Function

Finally, in this chapter we introduce a fundamental function — the “entropy” function — that plays a central role in the analysis of the limits of codes. For example, in Section 4.1 of Chapter 4 we will show how this function captures an upper bound on the rate of codes as a function of the relative distance. Later in Section 4.2 of Chapter 4 we will see that this function captures a lower bound on the rate of codes obtained by the probabilistic method.

We begin with the definition of the entropy function.

Definition 3.3.1 (q -ary Entropy Function). *Let q be an integer and x be a real number such that $q \geq 2$ and $0 \leq x \leq 1$. Then the q -ary entropy function is defined as follows:*

$$H_q(x) = x \log_q(q-1) - x \log_q(x) - (1-x) \log_q(1-x).$$

Figure 3.1 presents a pictorial representation of the H_q function for the first few values of q . For the special case of $q = 2$, we will drop the subscript from the entropy function and denote $H_2(x)$ by just $H(x)$, that is, $H(x) = -x \log x - (1-x) \log(1-x)$, where $\log x$ is defined as $\log_2(x)$ (we are going to follow this convention for the rest of the book).

Under the lens of Shannon's entropy function, $H(x)$ denotes the entropy of the distribution over $\{0, 1\}$ that selects 1 with probability x and 0 with probability $1-x$. However, there is no similar analogue for the more general $H_q(x)$. The reason why this quantity will turn out to be so central in this book is that it is very closely related to the “volume” of a Hamming ball. We make this connection precise in the next subsection.

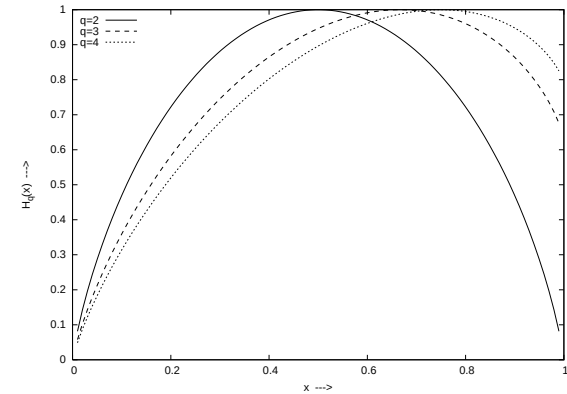


Figure 3.1: A plot of $H_q(x)$ for $q = 2, 3$ and 4 . The maximum value of 1 is achieved at $x = 1 - 1/q$.

3.3.1 Volume of Hamming Balls

It turns out that in many of our combinatorial results, we will need good upper and lower bounds on the volume of a Hamming ball. Next we formalize the notion of the volume of a Hamming ball:

Definition 3.3.2 (Volume of a Hamming Ball). *Let $q \geq 2$ and $n \geq r \geq 1$ be integers. Then the volume of a Hamming ball of radius r is given by*

$$\text{Vol}_q(r, n) = |B_q(\mathbf{0}, r)| = \sum_{i=0}^r \binom{n}{i} (q-1)^i.$$

The choice of $\mathbf{0}$ as the center for the Hamming ball above was arbitrary: since the volume of a Hamming ball is independent of its center (as is evident from the last equality above), we could have picked any point as the center.

We will prove the following result:

Proposition 3.3.3. *Let $q \geq 2$ be an integer and $0 \leq p \leq 1 - \frac{1}{q}$ be a real number. Then:*

- (i) $\text{Vol}_q(pn, n) \leq q^{H_q(p)n}$; and
- (ii) for large enough n , $\text{Vol}_q(pn, n) \geq q^{H_q(p)n - o(n)}$.

Proof. We start with the proof of (i). Consider the following sequence of relations:

$$1 = (p + (1-p))^n$$

⁴Note that $\overline{P} = \overline{P_1 \vee P_2 \vee \dots \vee P_m}$.

$$= \sum_{i=0}^n \binom{n}{i} p^i (1-p)^{n-i} \quad (3.17)$$

$$= \sum_{i=0}^{pn} \binom{n}{i} p^i (1-p)^{n-i} + \sum_{i=pn+1}^n \binom{n}{i} p^i (1-p)^{n-i} \quad (3.18)$$

$$\begin{aligned} &\geq \sum_{i=0}^{pn} \binom{n}{i} p^i (1-p)^{n-i} \\ &= \sum_{i=0}^{pn} \binom{n}{i} (q-1)^i \left(\frac{p}{q-1}\right)^i (1-p)^{n-i} \\ &= \sum_{i=0}^{pn} \binom{n}{i} (q-1)^i (1-p)^n \left(\frac{p}{(q-1)(1-p)}\right)^i \\ &\geq \sum_{i=0}^{pn} \binom{n}{i} (q-1)^i (1-p)^n \left(\frac{p}{(q-1)(1-p)}\right)^{pn} \end{aligned} \quad (3.19)$$

$$\begin{aligned} &= \sum_{i=0}^{pn} \binom{n}{i} (q-1)^i \left(\frac{p}{q-1}\right)^{pn} (1-p)^{(1-p)n} \\ &\geq Vol_q(pn, n) q^{-H_q(p)n}. \end{aligned} \quad (3.20) \quad (3.21)$$

In the above, (3.17) follows from the binomial expansion. (3.18) follows by dropping the second sum and (3.19) follows from the facts that $\frac{p}{(q-1)(1-p)} \leq 1$ (as⁵ $p \leq 1-1/q$). Rest of the steps except (3.21) follow from rearranging the terms. (3.21) follows as $q^{-H_q(p)n} = \left(\frac{p}{q-1}\right)^{pn} (1-p)^{(1-p)n}$. (3.21) implies that

$$1 \geq Vol_q(pn, n) q^{-H_q(p)n},$$

which proves (i).

We now turn to the proof of part (ii). For this part, we will need Stirling's approximation for $n!$ (Lemma B.1.2).

By the Stirling's approximation, we have the following inequality:

$$\begin{aligned} \binom{n}{pn} &= \frac{n!}{(pn)!((1-p)n)!} \\ &> \frac{(n/e)^n}{(pn/e)^{pn}((1-p)n/e)^{(1-p)n}} \cdot \frac{1}{\sqrt{2\pi p(1-p)n}} \cdot e^{\lambda_1(n) - \lambda_2(pn) - \lambda_2((1-p)n)} \\ &= \frac{1}{p^{pn}(1-p)^{(1-p)n}} \cdot \ell(n), \end{aligned} \quad (3.22)$$

$$\text{where } \ell(n) = \frac{e^{\lambda_1(n) - \lambda_2(pn) - \lambda_2((1-p)n)}}{\sqrt{2\pi p(1-p)n}}.$$

⁵Indeed, note that $\frac{p}{(q-1)(1-p)} \leq 1$ is true if $\frac{p}{1-p} \leq \frac{q-1}{1}$, which in turn is true if $p \leq \frac{q-1}{q}$, where the last step follows from Lemma B.2.1.

Now consider the following sequence of relations that complete the proof:

$$Vol_q(pn, n) \geq \binom{n}{pn} (q-1)^{pn} \quad (3.23)$$

$$> \frac{(q-1)^{pn}}{p^{pn}(1-p)^{(1-p)n}} \cdot \ell(n) \quad (3.24)$$

$$\geq q^{H_q(p)n - o(n)}. \quad (3.25)$$

In the above (3.23) follows by only looking at the last term in the sum that defined $Vol_q(pn, n)$. (3.24) follows from (3.22) while (3.25) follows from the definition of $H_q(\cdot)$ and the fact that for large enough n , $\ell(n)$ is $q^{-o(n)}$. $\square$

Next, we consider how the q -ary entropy function behaves for various ranges of its parameters.

3.3.2 Other Properties of the q -ary Entropy function

This section uses asymptotic analysis in few places. Reader who wish to brush up their knowledge of asymptotic analysis are referred to Appendix C.1.

We begin by recording the behavior of the q -ary entropy function for large q .

Proposition 3.3.4. *For small enough ε , $1 - H_q(\rho) \geq 1 - \rho - \varepsilon$ for every $0 < \rho \leq 1 - 1/q$ if and only if q is $2^{\Omega(1/\varepsilon)}$.*

Proof. We first note that by definition of $H_q(\rho)$ and $H(\rho)$,

$$\begin{aligned} H_q(\rho) &= \rho \log_q(q-1) - \rho \log_q \rho - (1-\rho) \log_q(1-\rho) \\ &= \rho \log_q(q-1) + H(\rho) / \log_2 q. \end{aligned}$$

Now if $q \geq 2^{1/\varepsilon}$, we get that

$$H_q(\rho) \leq \rho + \varepsilon$$

as $\log_q(q-1) \leq 1$ and $H(\rho) \leq 1$. Thus, we have argued that for $q \geq 2^{1/\varepsilon}$, we have $1 - H_q(\rho) \geq 1 - \rho - \varepsilon$, as desired.

Next, we consider the case when $q = 2^{o(1/\varepsilon)}$. We begin by claiming that for small enough ε ,

$$\text{if } q \geq 1/\varepsilon^2 \text{ then } \log_q(q-1) \geq 1 - \varepsilon.$$

Indeed, $\log_q(q-1) = 1 + (1/\ln q) \ln(1-1/q) = 1 - O\left(\frac{1}{q \ln q}\right)$ ⁶ which is at least $1 - \varepsilon$ for $q \geq 1/\varepsilon^2$ (and small enough ε).

Finally, if $q = 2^{o(\frac{1}{\varepsilon})}$, then for fixed ρ ,

$$H(\rho) / \log q = \varepsilon \cdot \omega(1).$$

⁶The last equality follows from the fact that by Lemma B.2.2, for $0 < x < 1$, $\ln(1-x) = -O(x)$.

Then for $q = 2^{o(\frac{1}{\varepsilon})}$ (but $q \geq 1/\varepsilon^2$) we have

$$\rho \log_q(q-1) + H(\rho)/\log q \geq \rho - \varepsilon + \varepsilon \cdot \omega(1) > \rho + \varepsilon,$$

which implies that

$$1 - H_q(\rho) < 1 - \rho - \varepsilon,$$

as desired. For $q \leq 1/\varepsilon^2$, Lemma 3.3.5 shows that $1 - H_q(\rho) \leq 1 - H_{1/\varepsilon^2}(\rho) < 1 - \rho - \varepsilon$, as desired. $\square$

We will also be interested in how $H_q(x)$ behaves for fixed x and increasing q :

Lemma 3.3.5. *Let $q \geq 2$ be an integer and let $0 \leq \rho \leq 1 - 1/q$, then for any real $m \geq 1$ such that*

$$q^{m-1} \geq \left(1 + \frac{1}{q-1}\right)^{q-1}, \quad (3.26)$$

we have

$$H_q(\rho) \geq H_{q^m}(\rho).$$

Proof. Note that $H_q(0) = H_{q^m}(0) = 0$. Thus, for the rest of the proof we will assume that $\rho \in (0, 1 - 1/q]$.

As observed in the proof of Proposition 3.3.4, we have

$$H_q(\rho) = \rho \cdot \frac{\log(q-1)}{\log q} + H(\rho) \cdot \frac{1}{\log q}.$$

Using this, we obtain

$$H_q(\rho) - H_{q^m}(\rho) = \rho \left(\frac{\log(q-1)}{\log q} - \frac{\log(q^m-1)}{m \log q} \right) + H(\rho) \left(\frac{1}{\log q} - \frac{1}{m \log q} \right).$$

The above in turn implies that

$$\begin{aligned} \frac{1}{\rho} \cdot m \log q \cdot (H_q(\rho) - H_{q^m}(\rho)) &= \log(q-1)^m - \log(q^m-1) + \frac{H(\rho)}{\rho} (m-1) \\ &\geq \log(q-1)^m - \log(q^m-1) + \frac{H(1-1/q)}{1-1/q} (m-1) \quad (3.27) \\ &= \log(q-1)^m - \log(q^m-1) + (m-1) \left(\log \frac{q}{q-1} + \frac{\log q}{q-1} \right) \\ &= \log \left(\frac{(q-1)^m}{q^m-1} \cdot \left(\frac{q}{q-1} \right)^{m-1} \cdot q^{\frac{m-1}{q-1}} \right) \\ &= \log \left(\frac{(q-1) \cdot q^{m-1} \cdot q^{\frac{m-1}{q-1}}}{q^m-1} \right) \\ &\geq 0 \quad (3.28) \end{aligned}$$

In the above (3.27) follows from the fact that $H(\rho)/\rho$ is decreasing⁷ in ρ and that $\rho \leq 1 - 1/q$. (3.28) follows from the claim that

$$(q-1) \cdot q^{\frac{m-1}{q-1}} \geq q.$$

Indeed the above follows from (3.26). $\square$

Finally, note that (3.28) completes the proof. $\square$

Since $(1 + 1/x)^x \leq e$ (by Lemma B.2.5), we also have that (3.26) is also satisfied for $m \geq 1 + \frac{1}{\ln q}$. Further, we note that (3.26) is satisfied for every $m \geq 2$ (for any $q \geq 3$), which leads to the following (also see Exercise 3.6):

Corollary 3.3.6. *Let $q \geq 3$ be an integer and let $0 \leq \rho \leq 1 - 1/q$, then for any $m \geq 2$, we have*

$$H_q(\rho) \geq H_{q^m}(\rho).$$

Next, we look at the entropy function when its input is very close to 1.

Proposition 3.3.7. *For small enough $\varepsilon > 0$,*

$$H_q \left(1 - \frac{1}{q} - \varepsilon \right) \leq 1 - c_q \varepsilon^2,$$

where c_q is a constant that only depends on q .

Proof. The intuition behind the proof is the following. Since the derivative of $H_q(x)$ is zero at $x = 1 - 1/q$, in the Taylor expansion of $H_q(1 - 1/q - \varepsilon)$ the ε term will vanish. We will now make this intuition more concrete. We will think of q as fixed and $1/\varepsilon$ as growing. In particular, we will assume that $\varepsilon < 1/q$. Consider the following equalities:

$$\begin{aligned} H_q(1 - 1/q - \varepsilon) &= - \left(1 - \frac{1}{q} - \varepsilon \right) \log_q \left(\frac{1 - 1/q - \varepsilon}{q-1} \right) - \left(\frac{1}{q} + \varepsilon \right) \log_q \left(\frac{1}{q} + \varepsilon \right) \\ &= - \log_q \left(\frac{1}{q} \left(1 - \frac{\varepsilon q}{q-1} \right) \right) + \left(\frac{1}{q} + \varepsilon \right) \log_q \left(\frac{1 - (\varepsilon q)/(q-1)}{1 + \varepsilon q} \right) \\ &= 1 - \frac{1}{\ln q} \left[\ln \left(1 - \frac{\varepsilon q}{q-1} \right) - \left(\frac{1}{q} + \varepsilon \right) \ln \left(\frac{1 - (\varepsilon q)/(q-1)}{1 + \varepsilon q} \right) \right] \\ &= 1 + o(\varepsilon^2) - \frac{1}{\ln q} \left[- \frac{\varepsilon q}{q-1} - \frac{\varepsilon^2 q^2}{2(q-1)^2} - \left(\frac{1}{q} + \varepsilon \right) \left(- \frac{\varepsilon q}{q-1} \right. \right. \\ &\quad \left. \left. - \frac{\varepsilon^2 q^2}{2(q-1)^2} - \varepsilon q + \frac{\varepsilon^2 q^2}{2} \right) \right] \\ &= 1 + o(\varepsilon^2) - \frac{1}{\ln q} \left[- \frac{\varepsilon q}{q-1} - \frac{\varepsilon^2 q^2}{2(q-1)^2} \right] \quad (3.29) \end{aligned}$$

⁷Indeed, $H(\rho)/\rho = \log(1/\rho) - (1/\rho - 1)\log(1 - \rho)$. Note that the first term is decreasing in ρ . We claim that the second term is also decreasing in ρ – this e.g. follows from the observation that $-(1/\rho - 1)\ln(1 - \rho) = (1 - \rho)(1 + \rho/2! + \rho^2/3! + \dots) = 1 - \rho/2 - \rho^2(1/2 - 1/3!) - \dots$ is also decreasing in ρ .

$$\begin{aligned}
& - \left(\frac{1}{q} + \varepsilon \right) \left(- \frac{\varepsilon q^2}{q-1} + \frac{\varepsilon^2 q^3 (q-2)}{2(q-1)^2} \right) \\
& = 1 + o(\varepsilon^2) - \frac{1}{\ln q} \left[- \frac{\varepsilon^2 q^2}{2(q-1)^2} + \frac{\varepsilon^2 q^2}{q-1} - \frac{\varepsilon^2 q^2 (q-2)}{2(q-1)^2} \right] \quad (3.30) \\
& = 1 - \frac{\varepsilon^2 q^2}{2 \ln q (q-1)} + o(\varepsilon^2) \\
& \leq 1 - \frac{\varepsilon^2 q^2}{4 \ln q (q-1)} \quad (3.31)
\end{aligned}$$

(3.29) follows from the fact that for $|x| < 1$, $\ln(1+x) = x - x^2/2 + x^3/3 - \dots$ (Lemma B.2.2) and by collecting the ε^3 and smaller terms in $o(\varepsilon^2)$. (3.30) follows by rearranging the terms and by absorbing the ε^3 terms in $o(\varepsilon^2)$. The last step is true assuming ε is small enough. $\square$

Next, we look at the entropy function when its input is very close to 0.

Proposition 3.3.8. *For small enough $\varepsilon > 0$,*

$$H_q(\varepsilon) = \Theta \left(\frac{1}{\log q} \cdot \varepsilon \log \left(\frac{1}{\varepsilon} \right) \right).$$

Proof. By definition

$$H_q(\varepsilon) = \varepsilon \log_q(q-1) + \varepsilon \log_q(1/\varepsilon) + (1-\varepsilon) \log_q(1/(1-\varepsilon)).$$

Since all the terms in the RHS are positive we have

$$H_q(\varepsilon) \geq \varepsilon \log(1/\varepsilon) / \log q. \quad (3.32)$$

Further, by Lemma B.2.2, $(1-\varepsilon) \log_q(1/(1-\varepsilon)) \leq 2\varepsilon / \ln q$ for small enough ε . Thus, this implies that

$$H_q(\varepsilon) \leq \frac{2 + \ln(q-1)}{\ln q} \cdot \varepsilon + \frac{1}{\ln q} \cdot \varepsilon \ln \left(\frac{1}{\varepsilon} \right). \quad (3.33)$$

(3.32) and (3.33) proves the claimed bound. $\square$

We will also work with the inverse of the q -ary entropy function. Note that $H_q(\cdot)$ on the domain $[0, 1 - 1/q]$ is a bijective map into $[0, 1]$. Thus, we define $H_q^{-1}(y) = x$ such that $H_q(x) = y$ and $0 \leq x \leq 1 - 1/q$. Finally, we will need the following lower bound:

Lemma 3.3.9. *For every $0 < y \leq 1 - 1/q$ and for every small enough $\varepsilon > 0$,*

$$H_q^{-1}(y - \varepsilon^2 / c'_q) \geq H_q^{-1}(y) - \varepsilon,$$

where $c'_q \geq 1$ is a constant that depends only on q .

Proof. It is easy to check that $H_q^{-1}(y)$ is a strictly increasing convex function when $y \in [0, 1]$. This implies that the derivative of $H_q^{-1}(y)$ increases with y . In particular, $(H_q^{-1})'(1) \geq (H_q^{-1})'(y)$ for every $0 \leq y \leq 1$. In other words, for every $0 < y \leq 1$, and (small enough) $\delta > 0$,

$$\frac{H_q^{-1}(y) - H_q^{-1}(y - \delta)}{\delta} \leq \frac{H_q^{-1}(1) - H_q^{-1}(1 - \delta)}{\delta}.$$

Proposition 3.3.7 along with the facts that $H_q^{-1}(1) = 1 - 1/q$ and H_q^{-1} is increasing completes the proof if one picks $c'_q = \max(1, 1/c_q)$ and $\delta = \varepsilon^2 / c'_q$. $\square$

3.4 Exercises

Exercise 3.1. *Prove Lemma 3.1.3.*

Exercise 3.2. *Prove Lemma 3.1.9.*

Exercise 3.3. *In this exercise, we will see a common use of the Chernoff bound (Theorem 3.1.10). Say we are trying to determine an (unknown) value $x \in \mathbb{F}$ to which we have access to via a randomized algorithm $\mathcal{A}$ that on input (random) input $\mathbf{r} \in \{0, 1\}^m$ outputs an estimate $\mathcal{A}(\mathbf{r})$ of x such that*

$$\Pr[\mathbf{A}(\mathbf{r}) = x] \geq \frac{1}{2} + \gamma,$$

for some $0 < \gamma < \frac{1}{2}$. Then show that for any $t \geq 1$ with $O\left(\frac{t}{\gamma^2}\right)$ calls to $\mathcal{A}$ one can determine x with probability at least $1 - e^{-t}$.

Hint: Call $\mathcal{A}$ with independent random bits and take majority of the answer and then use the Chernoff bound.

Exercise 3.4. *Prove Lemma 3.1.11.*

Exercise 3.5. *Let $\mathcal{P}$ denote the property that the randomly chosen $\mathcal{C}$ satisfies $f(\mathcal{C}) \leq b$. Then $\mathbb{E}[f(\mathcal{C})] \leq b$ implies that $\Pr[\mathcal{C} \text{ has property } \mathcal{P}] > 0$.*

Exercise 3.6. *Prove that for any $Q \geq q \geq 2$ and $\rho \leq 1 - 1/q$, we have $H_Q(\rho) \leq H_q(\rho)$.*

Exercise 3.7. *Prove that for $p < \frac{1}{2}$, we have $H_2(p) \leq O(p \log p)$.*

3.5 Bibliographic Notes

The Chernoff bounds of this chapter come from a family of bounds on the concentration of sums of random variables around their expectation. They originate with the work of Chernoff [31] though Chernoff himself attributes the bound to personal communication with Rubin [14, Page 340]. These bounds and variations are ubiquitous in information theory and computer science — see for instance [35, 117, 115]. Proofs of various concentration bounds can e.g. be found in [40].

The use of the probabilistic method in combinatorics seems to have originated in the early 40s and became especially well known after works of Erdős, notably [51]. Shannon's adoption of the method in [138] is one of the first applications in a broader setting. For more on the probabilistic method, see the book by Alon and Spencer [6].

The entropy function also dates back to Shannon [138]. Shannon's definition is more general and applies to discrete random variables. Our specialization to a two parameter function (namely a function of q and p) is a special case derived from applying the original definition to some special random variables.

Part II

The Combinatorics

Overview of the Combinatorics

Now that we have described the overarching goal of this book, and reviewed some background mathematics, we are now ready to explore our first quest, namely to understand the ‘limits of error-correcting codes.’ More specifically we will ask, given an alphabet parameter q and block length n , what values of dimension k and distance d are simultaneously achievable. Alternately we may study the relative versions, namely the rate $\frac{k}{n}$ and the relative distance $\frac{d}{n}$ that can be achieved in the asymptotic limit where $n \rightarrow \infty$.

Chapter 4 will mostly describe simple ‘negative results,’ i.e., the impossibility of the existence of codes with certain choices of parameters. The chapter will also include some existential results, which shows that some choices of parameters are achievable, without necessarily showing how to ‘construct’ such codes. (As an aside, Chapter 6 includes formal mathematical definitions of *explicit* constructions (Definition 6.3.4) and even *strongly explicit* constructions (Definition 6.3.5).) The results in Chapter 4 are chosen mostly due to the simplicity and versatility of the proof techniques which could be applied in broad contexts beyond coding theory. But these are not necessarily tight results, nor are they even the best known results. Some of the best-known asymptotic results appear without proof in Chapter 8, which also includes some other results with slightly more sophisticated proofs. One negative result from Chapter 4 with a pretty simple proof turns out to be tight over large alphabets. The codes showing this tightness are the famed Reed-Solomon codes whose analysis is also extremely elegant. These codes are introduced in Chapter 5.

Finally in this part we also introduce two ‘weaker’ models of errors and correction that are related to the Hamming model. Whereas in the Hamming model our focus was on correcting *all* patterns of errors subject to a limit on the total number of errors, Chapter 6 considers errors that are generated randomly and explores the limits when the goal is to correct these errors in *most* cases (i.e., with high probability over the randomness of the errors). This leads us to the ground breaking work of Shannon and this chapter reviews his work. It turns out the tradeoffs between the rate and the (typical) number of errors that can be corrected in this setting is tightly understood; and this model does achieve better tradeoffs for many settings of the parameters. We note that despite the difference in objectives, error-correcting codes (as studied in the rest of this book) continue to be the primary tool to correct random errors.

Finally in Chapter 7, we relax the notion of recovery in a different way. Instead of weakening the model only to consider random error patterns, we return to the notion of worst-case error patterns. But we relax the notion of recovery to allow the decoding algorithm to output a small list of codewords and deem the recovery to be successful if this list includes the transmitted

codeword. This notion, called *list-decoding*, turns out to be a very useful bridge between the Hamming and Shannon models of recovery; and also leads to new insights on the combinatorial limits of error-correcting codes.

Chapter 4

What Can and Cannot Be Done-I

In this chapter, we will try to tackle Question 1.8.1. We will approach this trade-off in the following way:

If we fix the relative distance of the code to be δ , what is the best rate R that we can achieve?

While we will not be able to pin down the exact optimal relationship between R and δ , we will start establishing some limits. Note that an upper bound on R is a *negative* result in that it establishes that codes with certain parameters do not exist. Similarly, a lower bound on R is a *positive* result.

In this chapter, we will consider only one positive result, i.e. a lower bound on R called the Gilbert-Varshamov bound in Section 4.2. In Section 4.1, we recall a negative result that we have already seen– Hamming bound and state its asymptotic version to obtain an upper bound on R . We will consider two other upper bounds: the Singleton bound (Section 4.3), which gives a tight upper bound for large enough alphabets (but not binary codes) and the Plotkin bound (Section 4.4), which gives a stronger upper bound than Singleton bound for binary codes.

4.1 Asymptotic Version of the Hamming Bound

We have already seen an upper bound in Section 1.7 due to Hamming. However, we had stated this as an upper bound on the dimension k in terms of n, q and d . In this section we convert this into a relation on R versus δ .

Consider any $(n, k, d)_q$ code with rate $R = k/n$ and relative distance $\delta = d/n$. Recall that Theorem 1.7.2 implies the following:

$$R = \frac{k}{n} \leq 1 - \frac{\log_q \text{Vol}_q\left(\left\lfloor \frac{d-1}{2} \right\rfloor, n\right)}{n}$$

Recall further that Proposition 3.3.3 states the following lower bound on the volume of a Hamming ball:

$$\text{Vol}_q\left(\left\lfloor \frac{d-1}{2} \right\rfloor, n\right) \geq q^{H_q\left(\frac{\delta}{2}\right)n - o(n)}.$$

Taking logarithms to base q of both sides above, and dividing by n yields that the second term in the right hand side of the inequality above is lower bounded by $H_q(\delta/2) - o(1)$, where the $o(1)$ term tends to 0 as $n \rightarrow \infty$. Thus Theorem 1.7.2 implies that for a q -ary code C of rate R , relative distance δ and block length n , we have:

$$R \leq 1 - H_q\left(\frac{\delta}{2}\right) + o(1), \quad (4.1)$$

where the $o(1)$ term tends to 0 as $n \rightarrow \infty$. Thus for an infinite family of q -ary codes $\mathcal{C}$, by taking limits as $n \rightarrow \infty$, we get the following asymptotic Hamming bound (see Exercise 4.1).

Proposition 4.1.1 (Asymptotic Hamming Bound). *Let $\mathcal{C}$ be an infinite family of q -ary codes with rate $R = R(\mathcal{C})$ and relative distance $\delta = \delta(\mathcal{C})$. Then we have:*

$$R \leq 1 - H_q\left(\frac{\delta}{2}\right).$$

Figure 4.1 gives a pictorial description of the asymptotic Hamming bound for binary codes.

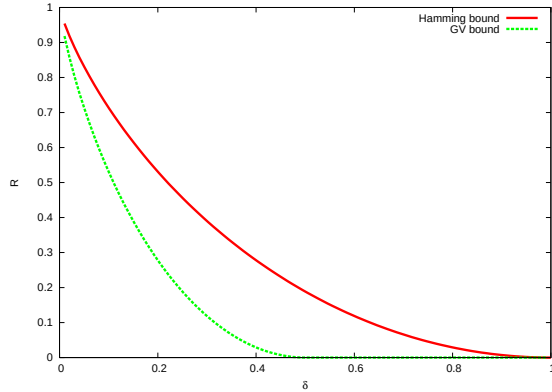


Figure 4.1: The Hamming and GV bounds for binary codes. Note that any point below the GV bound is achievable by some code while no point above the Hamming bound is achievable by any code. In this part of the book we would like to push the GV bound as much up as possible while at the same time try and push down the Hamming bound as much as possible.

4.2 Gilbert-Varshamov Bound

Next, we will switch gears by proving our first non-trivial lower bound on R in terms of δ . (In fact, this is the only positive result on the R vs δ tradeoff question that we will see in this book.) In particular, we will prove the following result:

Theorem 4.2.1 (Gilbert-Varshamov Bound). *Let $q \geq 2$. For every $0 \leq \delta < 1 - \frac{1}{q}$ there exists a family of q -ary codes $\mathcal{C}$ with rate $R(\mathcal{C}) \geq 1 - H_q(\delta)$ and relative distance $\delta(\mathcal{C}) \geq \delta$. If q is a prime power then there exists such a q -ary family of linear codes. Furthermore, for every $0 \leq \varepsilon \leq 1 - H_q(\delta)$ and integer n , if a matrix G is picked uniformly from $\mathbb{F}_q^{k \times n}$ for $k = n(1 - H_q(\delta) - \varepsilon)$, then G generates a code of rate $1 - H_q(\delta) - \varepsilon$ and relative distance at least δ with probability strictly greater than $1 - q^{-\varepsilon n}$.*

The bound of the theorem is referred to as the GV bound. For a pictorial description of the GV bound for binary codes, see Figure 4.1. We will present the proofs for general codes and linear codes in Sections 4.2.1 and 4.2.2 respectively.

In what follows we first prove the existence of a non-linear code of rate $1 - H_q(\delta)$ and relative distance at least δ . Later we show how to get a linear code, and with high probability (when $\varepsilon > 0$). (Note that the existence of a linear code is implied by the final part using $\varepsilon = 0$.)

4.2.1 Greedy Construction

We will prove Theorem 4.2.1 for general codes by a greedy construction described next: Fix an integer n and let $d = \delta n$. Start with the empty code $C \subseteq [q]^n$ and then keep on adding strings to C that are at Hamming distance at least d from all the existing words in C . Algorithm 4.2.1 presents a formal description of the algorithm and Figure 4.2 illustrates the first few executions of this algorithm.

Algorithm 4.2.1 Gilbert's Greedy Code Construction

INPUT: n, q, d

OUTPUT: A code $C \subseteq [q]^n$ of distance $d \geq 1$

- 1: $C \leftarrow \emptyset$
 - 2: WHILE there exists a $\mathbf{v} \in [q]^n$ such that $\Delta(\mathbf{v}, \mathbf{c}) \geq d$ for every $\mathbf{c} \in C$ DO
 - 3: Add $\mathbf{v}$ to C
 - 4: RETURN C
-

We claim that Algorithm 4.2.1 terminates and the C that it outputs has distance d . The latter is true by step 2, which makes sure that in Step 3 we never add a vector $\mathbf{c}$ that will make the distance of C fall below d . For the former claim, note that, if we cannot add $\mathbf{v}$ at some point, we cannot add it later. Indeed, since we only add vectors to C , if a vector $\mathbf{v} \in [q]^n$ is ruled out in a certain iteration of Step 2 because $\Delta(\mathbf{c}, \mathbf{v}) < d$, then in all future iterations, we have $\Delta(\mathbf{v}, \mathbf{c}) < d$ and thus, this $\mathbf{v}$ will never be added in Step 3 in any future iteration.

The running time of Algorithm 4.2.1 is $q^{O(n)}$. To see this, note that Step 2 in the worst-case could be repeated for every vector in $[q]^n$, that is at most q^n times. In a naive implementation, for each iteration, we cycle through all vectors in $[q]^n$ and for each vector $\mathbf{v} \in [q]^n$, iterate through all (at most q^n) vectors $\mathbf{c} \in C$ to check whether $\Delta(\mathbf{c}, \mathbf{v}) < d$. If no such $\mathbf{c}$ exists, then we add $\mathbf{v}$ to C . Otherwise, we move to the next $\mathbf{v}$. However, note that we can do slightly better—since we know that once a $\mathbf{v}$ is “rejected” in an iteration, it’ll keep on being rejected in the future

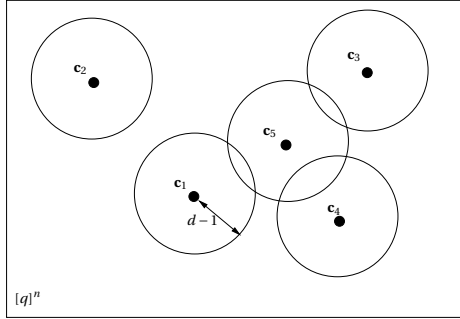


Figure 4.2: An illustration of Gilbert's greedy algorithm (Algorithm 4.2.1) for the first five iterations.

iterations, we can fix up an ordering of vectors in $[q]^n$ and for each vector $\mathbf{v}$ in this order, check whether it can be added to C or not. If so, we add $\mathbf{v}$ to C , else we move to the next vector in the order. This algorithm has time complexity $O(nq^{2n})$, which is still $q^{O(n)}$.

Further, we claim that after termination of Algorithm 4.2.1

$$\bigcup_{\mathbf{c} \in C} B(\mathbf{c}, d-1) = [q]^n.$$

This is because if the above is not true, then there exists a vector $\mathbf{v} \in [q]^n \setminus C$, such that $\Delta(\mathbf{v}, \mathbf{c}) \geq d$ and hence $\mathbf{v}$ can be added to C . However, this contradicts the fact that Algorithm 4.2.1 has terminated. Therefore,

$$\left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, d-1) \right| = q^n. \quad (4.2)$$

It is not too hard to see that

$$\sum_{\mathbf{c} \in C} |B(\mathbf{c}, d-1)| \geq \left| \bigcup_{\mathbf{c} \in C} B(\mathbf{c}, d-1) \right|,$$

which by (4.2) implies that

$$\sum_{\mathbf{c} \in C} |B(\mathbf{c}, d-1)| \geq q^n$$

or since the volume of a Hamming ball is translation invariant,

$$\sum_{\mathbf{c} \in C} \text{Vol}_q(d-1, n) \geq q^n.$$

Since $\sum_{\mathbf{c} \in C} \text{Vol}_q(d-1, n) = \text{Vol}_q(d-1, n) \cdot |C|$, we have

$$\begin{aligned} |C| &\geq \frac{q^n}{\text{Vol}_q(d-1, n)} \\ &\geq \frac{q^n}{q^{nH_q(\delta)}} \\ &= q^{n(1-H_q(\delta))}, \end{aligned} \quad (4.3)$$

as desired. In the above, (4.3) follows from the fact that

$$\begin{aligned} \text{Vol}_q(d-1, n) &\leq \text{Vol}_q(\delta n, n) \\ &\leq q^{nH_q(\delta)}, \end{aligned} \quad (4.4)$$

where the second inequality follows from the upper bound on the volume of a Hamming ball in Proposition 3.3.3.

We thus conclude that for every q, n and δ there exists a code of rate at least $n(1 - H_q(\delta))$. We state this formally as a lemma below.

Lemma 4.2.2. *For every pair of positive integers n, q and real $\delta \in [0, 1]$ there exists an $(n, k, \delta n)_q$ code satisfying $q^k \geq \frac{q^n}{\text{Vol}_q(d-1, n)}$.*

In particular, for every positive integer q and real $\delta \in [0, 1 - 1/q]$ there exists an infinite family of q -ary codes C of rate R and distance δ satisfying $R \geq 1 - H_q(\delta)$.

It is worth noting that the code from Algorithm 4.2.1 is not guaranteed to have any special structure. In particular, even storing the code can take exponential space. We have seen in Proposition 2.3.3 that linear codes have a much more succinct representation. Thus, a natural question is:

Question 4.2.1. *Do linear codes achieve the $R \geq 1 - H_q(\delta)$ tradeoff that the greedy construction achieves?*

Next, we will answer the question in the affirmative.

4.2.2 Linear Code Construction

Now we will show that a random linear code, with high probability, lies on the GV bound. The construction is a use of the probabilistic method (Section 3.2).

Proof of Theorem 4.2.1. By Proposition 2.3.6, we are done if we can show that there exists a $k \times n$ matrix $\mathbf{G}$ of full rank (for $k = (1 - H_q(\delta) - \epsilon)n$) such that

$$\text{For every } \mathbf{m} \in \mathbb{F}_q^k \setminus \{\mathbf{0}\}, \text{wt}(\mathbf{m}\mathbf{G}) \geq d.$$

We will prove the existence of such a $\mathbf{G}$ by the probabilistic method. Pick a random linear code by picking a random $k \times n$ matrix $\mathbf{G}$ where each of kn entries is chosen uniformly and independently at random from $\mathbb{F}_q$. Fix $\mathbf{m} \in \mathbb{F}_q^k \setminus \{\mathbf{0}\}$. Recall that by Lemma 3.1.12, for a random $\mathbf{G}$, $\mathbf{mG}$ is a uniformly random vector from $\mathbb{F}_q^n$. Thus, for every non-zero vector $\mathbf{m}$, we have

$$\begin{aligned} \Pr_{\mathbf{G}}[\text{wt}(\mathbf{mG}) < d] &= \frac{\text{Vol}_q(d-1, n)}{q^n} \\ &\leq \frac{q^{nH_q(\delta)}}{q^n} \end{aligned} \quad (4.5)$$

$$\leq q^{-k} \cdot q^{-\varepsilon n}, \quad (4.6)$$

where (4.5) follows from the fact that the condition $\text{wt}(\mathbf{mG}) < d$ is equivalent to the condition that $\mathbf{mG} \in B(\mathbf{0}, d-1)$ and the fact that $\mathbf{mG}$ is uniformly random in $\mathbb{F}_q^n$, (4.5) follows from (4.4) and (4.6) uses $k \leq n(1 - H_q(\delta) - \varepsilon)$. There are $q^k - 1$ non-zero vectors $\mathbf{m}$ and taking the union over all such vectors and applying the union bound (Lemma 3.1.5) we have

$$\begin{aligned} \Pr_{\mathbf{G}}[\text{There exists a non-zero } \mathbf{m} \text{ s.t. } \text{wt}(\mathbf{mG}) < d] &\leq (q^k - 1) \cdot q^{-k} \cdot q^{-\varepsilon n} \\ &< q^{-\varepsilon n}. \end{aligned}$$

Fix a matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ such that for every non-zero $\mathbf{M}$ we have $\text{wt}(\mathbf{mG}) \geq d$. The argument above has shown that a random matrix has this property with probability strictly greater than $1 - q^{-\varepsilon n}$. By Proposition 2.3.6 this implies that the code generated by $\mathbf{G}$ has distance at least d . To conclude the theorem we only need to argue that the code has dimension k , i.e., that $\mathbf{G}$ has full rank. But this also follows immediately from the property that for every $\varepsilon \geq 0$, we have that the probability that the code generated by a uniformly random matrix has distance less than or equal to d is strictly less than 1. Thus using the probabilistic method we conclude there exists a matrix $\mathbf{G}$ such that the code it generates is an $[n, k, d]_q$ code. Furthermore if $\varepsilon > 0$ then the probability that the code does not have distance d is exponentially small, specifically at most $q^{-\varepsilon n}$.

To conclude we need to verify that the code generated by $\mathbf{G}$ has dimension k , i.e., that $\mathbf{G}$ has full rank. But note that an equivalent definition of $\mathbf{G}$ not having full rank is that there exists a non-zero vector $\mathbf{M}$ such that $\mathbf{mG} = \mathbf{0}$. But the existence of such a vector $\mathbf{m}$ would imply $\text{wt}(\mathbf{mG}) = 0 < d$ contradicting the property that for every non-zero $\mathbf{M}$ we have $\text{wt}(\mathbf{mG}) \geq d$. We thus conclude that $\mathbf{G}$ generates a code of rate $k/n = 1 - H_q(\delta) - \varepsilon$ and relative distance δ . The theorem follows. $\square$

Discussion. We now digress a bit to stress some aspects of the GV bound and its proof. First, note that that proof by the probabilistic method shows something stronger than just the existence of a code, but rather gives a high probability result. Furthermore, as pointed out explicitly for the non-linear setting in Lemma 4.2.2, the result gives a lower bound not only in the asymptotic case but also one for every choice of n and k . The proof of the GV bound in the non-linear case gives a similar non-asymptotic bound in the linear setting also.

Note that we can also pick a random linear code by picking a random $(n-k) \times n$ parity check matrix. This also leads to an alternate proof of the GV bound: see Exercise 4.2.

Finally, we note that Theorem 4.2.1 requires $\delta < 1 - \frac{1}{q}$. An inspection of Gilbert and Varshamov's proofs shows that the only reason the proof required that $\delta \leq 1 - \frac{1}{q}$ is because it is needed for the volume bound (recall the bound in Proposition 3.3.3)– $\text{Vol}_q(\delta n, n) \leq q^{H_q(\delta)n}$ – to hold. It is natural to wonder if the above is just an artifact of the proof or if better codes exist. This leads to the following question:

Question 4.2.2. Does there exist a code with $R > 0$ and $\delta > 1 - \frac{1}{q}$?

We will return to this question in Section 4.4.

4.3 Singleton Bound

We will now change gears again and prove an upper bound on R (for fixed δ). We start by proving the Singleton bound.

Theorem 4.3.1 (Singleton Bound). *For every $(n, k, d)_q$ code,*

$$k \leq n - d + 1.$$

Consequently, if C is an infinite family of codes of rate R and relative distance δ then $R \leq 1 - \delta$.

Note that the asymptotic bound holds for any family of codes, even those where the alphabet may grow (arbitrarily) with the length of the code.

Proof. We start by proving the non-asymptotic bound first. The asymptotic version follows easily and is shown at the end.

Let $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_M$ be the codewords of an $(n, k, d)_q$ code C . Note that we need to show $M \leq q^{n-d+1}$. To this end, we define $\mathbf{c}'_i$ to be the prefix of the codeword $\mathbf{c}_i$ of length $n - d + 1$ for every $i \in [M]$. See Figure 4.3 for a pictorial description.

We now claim that for every $i \neq j$, $\mathbf{c}'_i \neq \mathbf{c}'_j$. For the sake of contradiction, assume that there exists an $i \neq j$ such that $\mathbf{c}'_i = \mathbf{c}'_j$. Notice this implies that $\mathbf{c}_i$ and $\mathbf{c}_j$ agree in all the first $n - d + 1$ positions, which in turn implies that $\Delta(\mathbf{c}_i, \mathbf{c}_j) \leq d - 1$. This contradicts the fact that C has distance d . Thus, M is the number of prefixes of codewords in C of length $n - d + 1$, which implies that $M \leq q^{n-d+1}$ as desired.

To get the asymptotic bound, assume some infinite family of codes $\mathcal{C}$ has rate $R = R(\mathcal{C}) = 1 - \delta + \varepsilon$ for some $\varepsilon > 0$. Then there must exist an $n > 2/\varepsilon$ and a code $C_n \in \mathcal{C}$ that is an $(n, k, d)_q$ code with $k \geq n(1 - \delta + \varepsilon)$ and $d \geq \delta n$. By our choice of n we thus have $k \geq n - d + 2$ contradicting the non-asymptotic bound proved above. $\square$

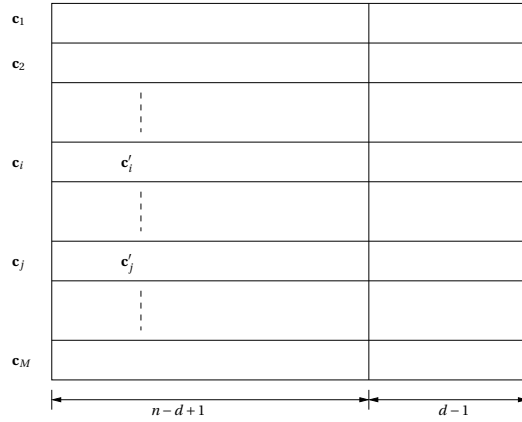


Figure 4.3: Construction of a new code in the proof of the Singleton bound.

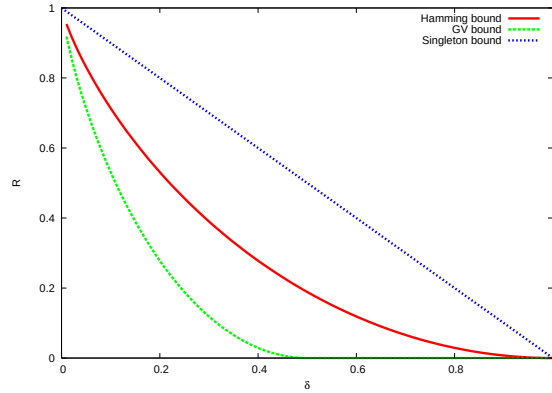


Figure 4.4: The Hamming, GV and Singleton bound for binary codes.

Figure 4.4 presents a pictorial description of the asymptotic version of the Singleton bound. It is worth noting that the bound is *independent* of the alphabet size. As is evident from Figure 4.4, the Singleton bound is worse than the Hamming bound for binary codes. However, this bound is better for larger alphabet sizes. In fact, we will look at a family of codes called Reed-Solomon codes in Chapter 5 that meets the Singleton bound. However, the alphabet size of the Reed-Solomon codes increases with the block length n . Thus, a natural follow-up question is the following:

Question 4.3.1. Given a fixed $q \geq 2$, does there exist a q -ary code that meets the Singleton bound?

We'll see an answer to this question in the next section.

4.4 Plotkin Bound

In this section, we will study the Plotkin bound, which will answer Questions 4.2.2 and 4.3.1. We start by stating the bound.

Theorem 4.4.1 (Plotkin bound). *The following hold for any code $C \subseteq [q]^n$ with distance at least d :*

1. If $d = \left(1 - \frac{1}{q}\right)n$, $|C| \leq 2qn$.
2. If $d > \left(1 - \frac{1}{q}\right)n$, $|C| \leq \frac{qd}{qd - (q-1)n}$.

Note that the Plotkin bound (Theorem 4.4.1) implies that a code with relative distance $\delta \geq 1 - \frac{1}{q}$, must necessarily have $R = 0$, which answers Question 4.2.2 in the negative.

Before proving Theorem 4.4.1, we make a few remarks. We first note that the upper bound in the first part of Theorem 4.4.1 can be improved to $2(q-1)n$ for $q \geq 2$. (See Exercise 4.13.) Second, it can be shown that this bound is tight for $q = 2$. (See Exercise 4.14.) Third, the statement of Theorem 4.4.1 gives a trade-off only for relative distance greater than $1 - 1/q$. However, as the following corollary shows, the result can be extended to work for $0 \leq \delta \leq 1 - 1/q$. (See Figure 4.5 for an illustration for binary codes.)

Corollary 4.4.2. *Let C be an infinite family of q -ary codes with relative distance $0 \leq \delta \leq 1 - \frac{1}{q}$ and rate R . Then*

$$R \leq 1 - \left(\frac{q}{q-1}\right)\delta.$$

Proof. Assume for contradiction that $\mathcal{C}$ is an infinite family of q -ary codes with rate $R = 1 - \left(\frac{q}{q-1}\right)\delta + \varepsilon$ for some $\varepsilon > 0$. Let $C \in \mathcal{C}$ be a code of block length $n \geq 2/\varepsilon$ with distance $d \leq \delta n$

and message length $k \geq Rn$. We argue now that an appropriate “shortening” of C yields a code contradicting Theorem 4.4.1.

Partition the codewords of C so that codewords within a partition agree on the first $n - n'$ symbols, where $n' = \left\lfloor \frac{qd}{q-1} \right\rfloor - 1$. (We will see later why this choice of n' makes sense.) In particular, for every $\mathbf{x} \in [q]^{n-n'}$, define the ‘prefix code’

$$C_{\mathbf{x}} = \{(c_{n-n'+1}, \dots, c_n) \mid (c_1 \dots c_N) \in C, (c_1 \dots c_{n-n'}) = \mathbf{x}\}.$$

In other words $C_{\mathbf{x}}$ consists of the n' -length suffixes of all codewords of C that start with the string $\mathbf{x}$.)

By definition $C_{\mathbf{x}}$ is a q -ary code of block length $n' = \left\lfloor \frac{qd}{q-1} \right\rfloor - 1$. We claim that it also has distance at least d for every $\mathbf{x}$: To see this, suppose for some $\mathbf{x}$, $\mathbf{c}_1 \neq \mathbf{c}_2 \in C_{\mathbf{x}}$, $\Delta(\mathbf{c}_1, \mathbf{c}_2) < d$. But this yields two codewords of C , namely $(\mathbf{x}, \mathbf{c}_1)$ and $(\mathbf{x}, \mathbf{c}_2)$, a Hamming distance is less than d from each other, contradicting the assumption that $\Delta(C) \geq d$.

Since $n' < \left(\frac{q}{q-1}\right)d$ (by definition of n') and thus, $d > \left(1 - \frac{1}{q}\right)n'$. Applying Theorem 4.4.1 to $C_{\mathbf{x}}$ we get that

$$|C_{\mathbf{x}}| \leq \frac{qd}{qd - (q-1)n'} \leq qd \leq q^2, \quad (4.7)$$

where the second inequality follows from the fact that $qd - (q-1)n'$ is a positive integer and the third is immediate from $d \leq q$.

We now use the bound on $|C_{\mathbf{x}}|$ for all $\mathbf{x}$ to get a bound on $|C|$. Note that by the definition of $C_{\mathbf{x}}$:

$$|C| = \sum_{\mathbf{x} \in [q]^{n-n'}} |C_{\mathbf{x}}|,$$

which by (4.7) implies that

$$|C| \leq \sum_{\mathbf{x} \in [q]^{n-n'}} q^2 = q^{n-n'+2} \leq q^{n-\frac{q}{q-1}d+2} \leq q^{n(1-\delta \cdot \frac{q}{q-1} + \epsilon)},$$

where the first inequality uses the definition of n' and the final inequality uses the fact that $n \geq 2/\epsilon$. We conclude that $R \leq 1 - \left(\frac{q}{q-1}\right)\delta + \epsilon$. Since this holds for every $\epsilon > 0$ the corollary follows. $\square$

Note that Corollary 4.4.2 implies that for any q -ary code of rate R and relative distance δ (where q is a constant independent of the block length of the code), $R < 1 - \delta$. In other words, this answers Question 4.3.1 in the negative.

Let us pause for a bit at this point and recollect the bounds on R versus δ that we have proved till now, which are all depicted in Figure 4.5 (for $q = 2$). The GV bound is the best known lower bound at the time of writing of this book. Better upper bounds are known and we will see one such trade-off (called the Elias-Bassalygo bound) in Section 8.1.

Now, we turn to the proof of Theorem 4.4.1, for which we will need two more lemmas. The first lemma deals with vectors over real spaces. We quickly recap the necessary definitions.

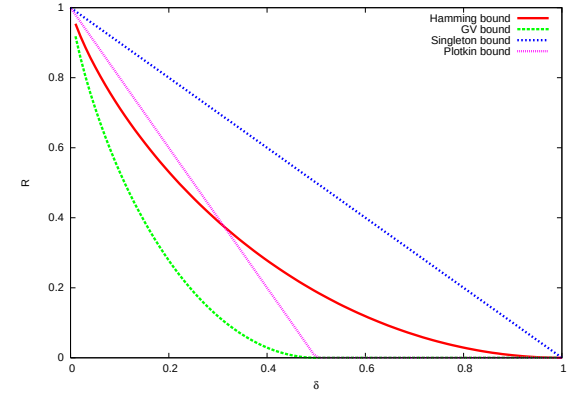


Figure 4.5: The current bounds on the rate R vs. relative distance δ for binary codes. The GV bound is a lower bound on R while the other three bounds are upper bounds on R .

Consider a vector $\mathbf{v}$ in $\mathbb{R}^n$, that is, a tuple of n real numbers. This vector has (Euclidean) norm $\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$, and is a unit vector if and only if its norm is 1. The inner product of two vectors, $\mathbf{u}$ and $\mathbf{v}$, is $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i u_i \cdot v_i$. The following lemma gives a bound on the number of vectors that can exist such that every pair is at an obtuse angle with each other.

Lemma 4.4.3 (Geometric Lemma). *Let $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m \in \mathbb{R}^N$ be non-zero vectors.*

1. *If $\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0$ for all $i \neq j$, then $m \leq 2N$.*
2. *Let $\mathbf{v}_i$ be unit vectors for $1 \leq i \leq m$. Further, if $\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq -\epsilon < 0$ for all $i \neq j$, then $m \leq 1 + \frac{1}{\epsilon}$.*¹

(Both items 1 and 2 are tight: see Exercises 4.15 and 4.16.) The proof of the Plotkin bound will need the existence of a map from codewords to real vectors with certain properties, which the next lemma guarantees.

Lemma 4.4.4 (Mapping Lemma). *For every q and n , there exists a function $f: [q]^n \rightarrow \mathbb{R}^{nq}$ such that for every $\mathbf{c}_1, \mathbf{c}_2 \in [q]^n$ we have*

$$\langle f(\mathbf{c}_1), f(\mathbf{c}_2) \rangle = 1 - \left(\frac{q}{q-1}\right) \left(\frac{\Delta(\mathbf{c}_1, \mathbf{c}_2)}{n}\right).$$

Consequently we get:

1. *For every $\mathbf{c} \in [q]^n$, $\|f(\mathbf{c})\| = 1$.*

¹Note that since $\mathbf{v}_i$ and $\mathbf{v}_j$ are both unit vectors, $\langle \mathbf{v}_i, \mathbf{v}_j \rangle$ is the cosine of the angle between them.

2. If $\Delta(\mathbf{c}_1, \mathbf{c}_2) \geq d$ then we have $\langle f(\mathbf{c}_1), f(\mathbf{c}_2) \rangle \leq 1 - \left(\frac{q}{q-1}\right) \left(\frac{d}{n}\right)$.

We defer the proofs of the Geometric Lemma and the Mapping Lemma to the end of the section and turn instead to proving Theorem 4.4.1 using the lemmas.

Proof of Theorem 4.4.1. Let $C = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m\}$ be a q -ary code of block length n and distance d . Let $f: [q]^n \rightarrow \mathbb{R}^{nq}$ be the function from Lemma 4.4.4. Then for all i we have that $f(\mathbf{c}_i)$ is a unit length vector in $\mathbb{R}^{nq}$. Furthermore for all $i \neq j$, we have

$$\langle f(\mathbf{c}_i), f(\mathbf{c}_j) \rangle \leq 1 - \left(\frac{q}{q-1}\right) \frac{d}{n}.$$

Thus $f(\mathbf{c}_1), \dots, f(\mathbf{c}_m)$ give us unit vectors in $\mathbb{R}^{nq}$ to which we can apply Lemma 4.4.3 and this will yield the upper bounds claimed on $m = |C|$ in the theorem statement.

For part 1 of the theorem, if $d = \left(1 - \frac{1}{q}\right)n = \frac{(q-1)n}{q}$, then for all $i \neq j$, we have

$$\langle f(\mathbf{c}_i), f(\mathbf{c}_j) \rangle \leq 0.$$

So by the first part of Lemma 4.4.3, $m \leq 2nq$, as desired.

For part 2, if $d > \left(\frac{q-1}{q}\right)n$ then for all $i \neq j$ we have

$$\langle f(\mathbf{c}_i), f(\mathbf{c}_j) \rangle \leq 1 - \left(\frac{q}{q-1}\right) \frac{d}{n} = -\left(\frac{qd - (q-1)n}{(q-1)n}\right).$$

Let $\varepsilon \stackrel{\text{def}}{=} \left(\frac{qd - (q-1)n}{(q-1)n}\right) > 0$. We can apply the second part of Lemma 4.4.3 to $f(\mathbf{c}_1), \dots, f(\mathbf{c}_m)$ and ε to get $m \leq 1 + \frac{(q-1)n}{qd - (q-1)n} = \frac{qd}{qd - (q-1)n}$, as desired. $\square$

4.4.1 Proof of Geometric and Mapping Lemmas

We now prove Lemmas 4.4.3 and 4.4.4. We start with Lemma 4.4.3, namely the Geometric Lemma.

Proof of Lemma 4.4.3. We prove both parts using linear algebra over the reals.

We start by proving the first part of the lemma. This part is also linear algebraic but involves a few more steps.

We first focus on a subset of the m vectors that has a positive inner product with some fixed vector $\mathbf{u}$. Specifically we pick $\mathbf{u}$ to be a generic vector in $\mathbb{R}^N$ so that $\langle \mathbf{u}, \mathbf{v}_i \rangle \neq 0$ for every i . Such vector exists since the set of vectors satisfying $\langle \mathbf{u}, \mathbf{v}_i \rangle = 0$ is a dimension $N - 1$ linear subspace of $\mathbb{R}^N$ (since $\mathbf{v}_i \neq \mathbf{0}$). And the union of N such linear subspaces (one for each $i \in [N]$) cannot cover all of $\mathbb{R}^N$.

Assume w.l.o.g. that at least half of the $\mathbf{v}_i$'s have a positive inner product with $\mathbf{u}$ (if not we can work with $-\mathbf{u}$ instead) and assume further that these are the first $\ell \geq m/2$ vectors by renumbering the vectors. We now show that $\mathbf{v}_1, \dots, \mathbf{v}_\ell$ are linearly independent. This suffices to prove the first part, since linear independence implies $\ell \leq N$ and thus $m \leq 2\ell \leq 2N$.

Assume for contradiction that there is a linear dependency among the vectors $\mathbf{v}_1, \dots, \mathbf{v}_\ell$, i.e., there exist $\alpha_1, \dots, \alpha_\ell$ with at least one $\alpha_i \neq 0$ such that $\sum_{i \in [\ell]} \alpha_i \mathbf{v}_i = \mathbf{0}$. Note we can assume that at least one α_i is positive since if all are non-negative we can negate all α_i 's to get a positive α_i . Further, by renumbering the indices we can assume that there exists $k \geq 1$ such that $\alpha_1, \dots, \alpha_k > 0$ and $\alpha_{k+1}, \dots, \alpha_\ell \leq 0$.

Let $\mathbf{w} = \sum_{i=1}^k \alpha_i \mathbf{v}_i$. By the definition of α_i 's we have that $\mathbf{w} = -\sum_{j=k+1}^\ell \alpha_j \mathbf{v}_j$. We first argue that $\mathbf{w} \neq \mathbf{0}$ by using the vector $\mathbf{u}$. Note that we have

$$\langle \mathbf{u}, \mathbf{w} \rangle = \langle \mathbf{u}, \sum_{i=1}^k \alpha_i \mathbf{v}_i \rangle = \sum_{i=1}^k \alpha_i \langle \mathbf{u}, \mathbf{v}_i \rangle \geq \alpha_1 \langle \mathbf{u}, \mathbf{v}_1 \rangle > 0.$$

We thus conclude $\mathbf{w}$ has a non-zero inner product with some vector and hence can not be the zero vector.

But now we have the following contradiction:

$$0 < \langle \mathbf{w}, \mathbf{w} \rangle = \left\langle \sum_{i=1}^k \alpha_i \mathbf{v}_i, -\sum_{j=k+1}^\ell \alpha_j \mathbf{v}_j \right\rangle = -\sum_{i=1, j=k+1}^{k, \ell} \alpha_i \alpha_j \langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0,$$

where the first inequality uses $\mathbf{w} \neq \mathbf{0}$, the first equality uses the two definitions of $\mathbf{w}$ namely $\mathbf{w} = \sum_{i=1}^k \alpha_i \mathbf{v}_i = -\sum_{j=k+1}^\ell \alpha_j \mathbf{v}_j$, and the final inequality holds for every term in the summation. Specifically for every $0 \leq i \leq k$ and $k+1 \leq j \leq \ell$ we have $\alpha_i \geq 0$, $\alpha_j \leq 0$ and $\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0$ and so $-\alpha_i \alpha_j \langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0$. We conclude that $\mathbf{v}_1, \dots, \mathbf{v}_\ell$ must be linearly independent and this proves the first part of the lemma.

We now move on to the proof of the second part. Define $\mathbf{z} = \mathbf{v}_1 + \dots + \mathbf{v}_m$. Now consider the following sequence of relationships:

$$\|\mathbf{z}\|^2 = \sum_{i=1}^m \|\mathbf{v}_i\|^2 + 2 \sum_{i < j} \langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq m + 2 \cdot \binom{m}{2} \cdot (-\varepsilon) = m(1 - \varepsilon m + \varepsilon).$$

The inequality follows from the facts that each $\mathbf{v}_i$ is a unit vector and the assumption that for every $i \neq j$, $\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq -\varepsilon$. As $\|\mathbf{z}\|^2 \geq 0$,

$$m(1 - \varepsilon m + \varepsilon) \geq 0.$$

Since $m \geq 1$, we have that

$$1 - \varepsilon m + \varepsilon \geq 0$$

or

$$\varepsilon m \leq 1 + \varepsilon.$$

Thus, we have $m \leq 1 + \frac{1}{\varepsilon}$, as desired.

Alternate proof of first part. We now present an alternate proof of the first result, which we do by induction on n . Note that in the base case of $N = 0$, we have $m = 0$, which satisfies the claimed inequality $m \leq 2N$.

In the general case, we have $m \geq 1$ non-zero vectors $\mathbf{v}_1, \dots, \mathbf{v}_m \in \mathbb{R}^N$ such that for every $i \neq j$,

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0. \quad (4.8)$$

Since rotating all the vectors by the same amount does not change the sign of the inner product (nor does scaling any of the vectors), w.l.o.g. we can assume that $\mathbf{v}_m = \langle 1, 0, \dots, 0 \rangle$. For $1 \leq i \leq m-1$, denote the vectors as $\mathbf{v}_i = \langle \alpha_i, \mathbf{y}_i \rangle$, for some $\alpha_i \in \mathbb{R}$ and $\mathbf{y}_i \in \mathbb{R}^{N-1}$. Now, for any $i \neq 1$, $\langle \mathbf{v}_1, \mathbf{v}_i \rangle = 1 \cdot \alpha_i + \sum_{j=2}^m 0 = \alpha_i$. However, we know from (4.8) that $\langle \mathbf{v}_1, \mathbf{v}_i \rangle \leq 0$, which in turn implies that

$$\alpha_i \leq 0. \quad (4.9)$$

Next, we claim that at most one of $\mathbf{y}_1, \dots, \mathbf{y}_{m-1}$ can be the all zeroes vector, $\mathbf{0}$. If not, assume w.l.o.g., that $\mathbf{y}_1 = \mathbf{y}_2 = \mathbf{0}$. This in turn implies that

$$\begin{aligned} \langle \mathbf{v}_1, \mathbf{v}_2 \rangle &= \alpha_1 \cdot \alpha_2 + \langle \mathbf{y}_1, \mathbf{y}_2 \rangle \\ &= \alpha_1 \cdot \alpha_2 + 0 \\ &= \alpha_1 \cdot \alpha_2 \\ &> 0, \end{aligned}$$

where the last inequality follows from the subsequent argument. As $\mathbf{v}_1 = \langle \alpha_1, \mathbf{0} \rangle$ and $\mathbf{v}_2 = \langle \alpha_2, \mathbf{0} \rangle$ are non-zero, we have that $\alpha_1, \alpha_2 \neq 0$. (4.9) then implies that $\alpha_1, \alpha_2 < 0$. However, $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle > 0$ contradicts (4.8).

Thus, w.l.o.g., assume that $\mathbf{y}_1, \dots, \mathbf{y}_{m-2}$ are all non-zero vectors. Further, note that for every $i \neq j \in [m-2]$, $\langle \mathbf{y}_i, \mathbf{y}_j \rangle = \langle \mathbf{v}_i, \mathbf{v}_j \rangle - \alpha_i \cdot \alpha_j \leq \langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq 0$. Thus, we have reduced problem on m vectors with dimension N to an equivalent problem on $m-2$ vectors with dimension $N-1$. By induction we have $m-2 \leq 2(N-1)$ and thus implying $m \leq 2N$. $\square$

Finally, we prove the Mapping Lemma, i.e., Lemma 4.4.4.

Proof of Lemma 4.4.4. We begin by defining a map $\phi : [q] \rightarrow \mathbb{R}^q$ which essentially satisfies the requirements of the lemma statement for the case $n = 1$ (up to some normalization constant). Then, we essentially apply ϕ separately to each coordinate of a word to get the map $f : [q]^n \rightarrow \mathbb{R}^{nq}$ that satisfies the claimed properties. We now fill in the details.

Let $\mathbf{e}_i$ denote the unit vector along the i th direction in $\mathbb{R}^q$, i.e.,

$$\mathbf{e}_i = \left\langle 0, 0, \dots, \underbrace{1}_{i^{\text{th}} \text{ position}}, \dots, 0 \right\rangle.$$

Let $\bar{\mathbf{e}} = \frac{1}{q} \sum_{i \in [q]} \mathbf{e}_i = \langle 1/q, 1/q, \dots, 1/q \rangle$. Note that we have $\langle \mathbf{e}_i, \mathbf{e}_j \rangle = 1$ if $i = j$ and 0 otherwise. Note also $\langle \bar{\mathbf{e}}, \mathbf{e}_i \rangle = \langle \bar{\mathbf{e}}, \bar{\mathbf{e}} \rangle = 1/q$ for every i .

Now we define $\phi : [q] \rightarrow \mathbb{R}^q$ to be $\phi(i) = \mathbf{e}_i - \bar{\mathbf{e}}$. For every pair $i, j \in [q]$ we have

$$\langle \phi(i), \phi(j) \rangle = \langle \mathbf{e}_i - \bar{\mathbf{e}}, \mathbf{e}_j - \bar{\mathbf{e}} \rangle = \langle \mathbf{e}_i, \mathbf{e}_j \rangle - \langle \mathbf{e}_i, \bar{\mathbf{e}} \rangle - \langle \bar{\mathbf{e}}, \mathbf{e}_j \rangle + \langle \bar{\mathbf{e}}, \bar{\mathbf{e}} \rangle = \langle \mathbf{e}_i, \mathbf{e}_j \rangle - 1/q.$$

Thus, for every $i \in [q]$, we get:

$$\|\phi(i)\|^2 = \langle \mathbf{e}_i, \mathbf{e}_i \rangle - 1/q = \frac{(q-1)}{q}. \quad (4.10)$$

Also for every $i \neq j \in [q]$, we have:

$$\langle \phi(i), \phi(j) \rangle = -\frac{1}{q}. \quad (4.11)$$

We are now ready to define our final map $f : [q]^n \rightarrow \mathbb{R}^{nq}$. For every $\mathbf{c} = (c_1, \dots, c_n) \in [q]^n$, define

$$f(\mathbf{c}) = \sqrt{\frac{q}{n(q-1)}} \cdot (\phi(c_1), \phi(c_2), \dots, \phi(c_n)).$$

(The multiplicative factor $\sqrt{\frac{q}{n(q-1)}}$ will be used to ensure below that $f(\mathbf{c})$ for every $\mathbf{c} \in [q]^n$ is a unit vector.)

To complete the proof, we will show that f satisfies the claimed properties. We begin with condition 1. Note that

$$\|f(\mathbf{c})\|^2 = \frac{q}{(q-1)n} \cdot \sum_{i=1}^n \|\phi(i)\|^2 = 1,$$

where the first equality follows from the definition of f and the second equality follows from (4.10).

We now turn to the second condition. For notational convenience, define $\mathbf{c}_1 = (x_1, \dots, x_n)$ and $\mathbf{c}_2 = (y_1, \dots, y_n)$. Consider the following sequence of relations:

$$\begin{aligned} \langle f(\mathbf{c}_1), f(\mathbf{c}_2) \rangle &= \sum_{\ell=1}^n \langle f(x_\ell), f(y_\ell) \rangle \\ &= \left[\sum_{\ell: x_\ell \neq y_\ell} \langle \phi(x_\ell), \phi(y_\ell) \rangle + \sum_{\ell: x_\ell = y_\ell} \langle \phi(x_\ell), \phi(y_\ell) \rangle \right] \cdot \left(\frac{q}{n(q-1)} \right) \\ &= \left[\sum_{\ell: x_\ell \neq y_\ell} \left(\frac{-1}{q} \right) + \sum_{\ell: x_\ell = y_\ell} \left(\frac{q-1}{q} \right) \right] \cdot \left(\frac{q}{n(q-1)} \right) \end{aligned} \quad (4.12)$$

$$\begin{aligned} &= \left[\Delta(\mathbf{c}_1, \mathbf{c}_2) \left(\frac{-1}{q} \right) + (n - \Delta(\mathbf{c}_1, \mathbf{c}_2)) \left(\frac{q-1}{q} \right) \right] \cdot \left(\frac{q}{n(q-1)} \right) \\ &= 1 - \Delta(\mathbf{c}_1, \mathbf{c}_2) \left(\frac{q}{n(q-1)} \right) \left[\frac{1}{q} + \frac{q-1}{q} \right] \\ &= 1 - \left(\frac{q}{q-1} \right) \left(\frac{\Delta(\mathbf{c}_1, \mathbf{c}_2)}{n} \right), \end{aligned} \quad (4.13)$$

as desired. In the above, (4.12) is obtained using (4.11) and (4.10) while (4.13) follows from the definition of the Hamming distance. $\square$

4.5 Exercises

Exercise 4.1. Given an infinite family of q -ary codes C of relative distance δ , and $\epsilon > 0$ prove that there exists an n_0 such that for all $n \geq n_0$, if $C_n \in C$ is an $[n, k]_q$ code, then $k/n < 1 - H_q(\delta/2) + \epsilon$. Use this to conclude Proposition 4.1.1.

Exercise 4.2. Pick a $(n - k) \times n$ matrix H over $\mathbb{F}_q$ at random. Show that with high probability the code whose parity check matrix is H achieves the GV bound.

Exercise 4.3. Recall the definition of an ϵ -biased space from Exercise 2.15. Show that there exists an ϵ -biased space of size $O(k/\epsilon^2)$.

Hint: Recall part 1 of Exercise 2.15.

Exercise 4.4. Argue that a random linear code as well as its dual both lie on the corresponding GV bound.

Exercise 4.5. In Section 4.2.2, we saw that random linear code meets the GV bound. It is natural to ask the question for general random codes. (By a random $(n, k)_q$ code, we mean the following: for each of the q^k messages, pick a random vector from $[q]^n$. Further, the choices for each codeword is independent.) We will do so in this problem.

1. Prove that a random q -ary code with rate $R > 0$ with high probability has relative distance $\delta \geq H_q^{-1}(1 - 2R - \epsilon)$. Note that this is worse than the bound for random linear codes in Theorem 4.2.1.
2. Prove that with high probability the relative distance of a random q -ary code of rate R is at most $H_q^{-1}(1 - 2R) + \epsilon$. In other words, general random codes are worse than random linear codes in terms of their distance.

Hint: Use Chebyshev's inequality.

Exercise 4.6. We saw that Algorithm 4.2.1 can compute an $(n, k)_q$ code on the GV bound in time $q^{O(n)}$. Now the construction for linear codes is a randomized construction and it is natural to ask how quickly can we compute an $[n, k]_q$ code that meets the GV bound. In this problem, we will see that this can also be done in $q^{O(n)}$ deterministic time, though the deterministic algorithm is not that straight-forward anymore.

1. Argue that Theorem 4.2.1 gives a $q^{O(kn)}$ time algorithm that constructs an $[n, k]_q$ code on the GV bound. (Thus, the goal of this problem is to "shave" off a factor of k from the exponent.)
2. A $k \times n$ Toeplitz Matrix $A = \{A_{i,j}\}_{i=1}^k, j=1}^n$ satisfies the property that $A_{i,j} = A_{i-1,j-1}$. In other words, any diagonal has the same value. For example, the following is a 4×6 Toeplitz matrix:

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 7 & 1 & 2 & 3 & 4 & 5 \\ 8 & 7 & 1 & 2 & 3 & 4 \\ 9 & 8 & 7 & 1 & 2 & 3 \end{pmatrix}$$

A random $k \times n$ Toeplitz matrix $T \in \mathbb{F}_q^{k \times n}$ is chosen by picking the entries in the first row and column uniformly (and independently) at random.

Prove the following claim: For any non-zero $\mathbf{m} \in \mathbb{F}_q^k$, the vector $\mathbf{m} \cdot T$ is uniformly distributed over $\mathbb{F}_q^n$, that is for every $\mathbf{y} \in \mathbb{F}_q^n$, $\Pr[\mathbf{m} \cdot T = \mathbf{y}] = q^{-n}$.

Hint: Write down the expression for the value at each of the n positions in the vector $\mathbf{m} \cdot T$ in terms of the values in the first row and column of T . Think of the values in the first row and column as variables. Then divide these variables into two sets (this "division" will depend on $\mathbf{m}$) say S and $\bar{S}$. Then argue the following: for every fixed $\mathbf{y} \in \mathbb{F}_q^n$ and for every fixed assignment to variables in S , there is a unique assignment to variables in $\bar{S}$ such that $\mathbf{m} \cdot T = \mathbf{y}$.

3. Briefly argue why the claim in part 2 implies that a random code defined by picking its generator matrix as a random Toeplitz matrix with high probability lies on the GV bound.
4. Conclude that an $[n, k]_q$ code on the GV bound can be constructed in time $q^{O(k+n)}$.

Exercise 4.7. Show that one can construct the parity check matrix of an $[n, k]_q$ code that lies on the GV bound in time $q^{O(n)}$.

Exercise 4.8. So far in Exercises 4.6 and 4.7, we have seen two constructions of $[n, k]_q$ code on the GV bound that can be constructed in $q^{O(n)}$ time. For constant rate codes, at the time of writing of this book, this is fastest known construction of any code that meets the GV bound. For $k = o(n)$, there is a better construction known, which we explore in this exercise.

We begin with some notation. For the rest of the exercise we will target a distance of $d = \delta n$. Given a message $\mathbf{m} \in \mathbb{F}_q^k$ and an $[n, k]_q$ code C , define the indicator variable:

$$W_{\mathbf{m}}(C) = \begin{cases} 1 & \text{if } \text{wt}(C(\mathbf{m})) < d \\ 0 & \text{otherwise.} \end{cases}$$

Further, define

$$D(C) = \sum_{\mathbf{m} \in \mathbb{F}_q^k \setminus \{0\}} W_{\mathbf{m}}(C).$$

We will also use $D(G)$ and $W_{\mathbf{m}}(G)$ to denote the variables above for the code C generated by G .

Given an $k \times n$ matrix M , we will use M^i to denote the i th column of M and $M^{\leq i}$ to denote the column submatrix of M that contains the first i columns. Finally below we will use $\mathcal{G}$ to denote a uniformly random $k \times n$ generator matrix and G to denote a specific instantiation of the generator matrix. We will arrive at the final construction in a sequence of steps. In what follows define $k < (1 - H_q(\delta))n$ for large enough n .

1. Argue that C has a distance d if and only if $D(C) < 1$.
2. Argue that $\mathbb{E}[D(\mathcal{G})] < 1$.

3. Argue that for any $1 \leq i < n$ and fixed $k \times n$ matrix G ,

$$\min_{\mathbf{v} \in \mathbb{F}_q^k} \mathbb{E} \left[D(\mathcal{G}) | \mathcal{G}^{\leq i} = G^{\leq i}, \mathcal{G}^{i+1} = \mathbf{v} \right] \leq \mathbb{E} \left[D(\mathcal{G}) | \mathcal{G}^{\leq i} = G^{\leq i} \right].$$

4. We are now ready to define the algorithm to compute the final generator matrix G : see Algorithm 4.5.1. Prove that Algorithm 4.5.1 outputs a matrix G such that the linear code

Algorithm 4.5.1 $q^{O(k)}$ time algorithm to compute a code on the GV bound

INPUT: Integer parameters $1 \leq k \neq n$ such that $k < (1 - H_q(\delta))n$

OUTPUT: An $k \times n$ generator matrix G for a code with distance δn

- 1: Initialize G to be the all 0s matrix ▷ This initialization is arbitrary
 - 2: FOR every $1 \leq i \leq n$ DO
 - 3: $G^i \leftarrow \arg \min_{\mathbf{v} \in \mathbb{F}_q^k} \mathbb{E} \left[D(\mathcal{G}) | \mathcal{G}^{\leq i} = G^{\leq i}, \mathcal{G}^{i+1} = \mathbf{v} \right]$
 - 4: RETURN G
-

generated by G is an $[n, k, \delta n]_q$ code. Conclude that this code lies on the GV bound.

5. Finally, we will analyze the run time of Algorithm 4.5.1. Argue that Step 2 can be implemented in $\text{poly}(n, q^k)$ time. Conclude Algorithm 4.5.1 can be implemented in time $\text{poly}(n, q^k)$.

Hint: It might be useful to maintain a data structure that keeps track of one number for every non-zero $\mathbf{m} \in \mathbb{F}_q^k$ throughout the run of Algorithm 4.5.1.

Exercise 4.9. In this problem we will derive the GV bound using a graph-theoretic proof, which is actually equivalent to the greedy proof we saw in Section 4.2.1. Let $1 \leq d \leq n$ and $q \geq 1$ be integers. Now consider the graph $G_{n,d,q} = (V, E)$, where the vertex set is the set of all vectors in $[q]^n$. Given two vertices $\mathbf{u} \neq \mathbf{v} \in [q]^n$, we have the edge $(u, v) \in E$ if and only if $\Delta(\mathbf{u}, \mathbf{v}) < d$. An independent set of a graph $G = (V, E)$ is a subset $I \subseteq V$ such that for every $u \neq v \in I$, we have that (u, v) is not an edge. We now consider the following sub-problems:

1. Argue that any independent set C of $G_{n,d,q}$ is a q -ary code of distance d .
2. The degree of a vertex in a graph G is the number of edges incident on that vertex. Let Δ be the maximum degree of any vertex in $G = (V, E)$. Then argue that G has an independent set of size at least $\frac{|V|}{\Delta+1}$.
3. Using parts 1 and 2 argue the GV bound.

Exercise 4.10. In this problem we will improve slightly on the GV bound using a more sophisticated graph-theoretic proof. Let $G_{n,d,q}$ and N and Δ be as in the previous exercise (Exercise 4.9).

So far we used the fact that $G_{n,d,q}$ has many vertices and small degree to prove it has a large independent set, and thus to prove there is a large code of minimum distance d . In this exercise we will see how a better result can be obtained by counting the number of “triangles” in the graph. A triangle in a graph $G = (V, E)$ is a set $\{u, v, w\} \subset V$ of three vertices such that all three vertices are adjacent, i.e., $(u, v), (v, w), (w, u) \in E$. For simplicity we will focus on the case where $q = 2$ and $d = n/5$, and consider the limit as $n \rightarrow \infty$.

1. Prove that a graph on N vertices of maximum degree Δ has at most $O(N\Delta^2)$ triangles.
2. Prove that the number of triangle in graph $G_{n,d,2}$ is at most

$$2^n \cdot \sum_{0 \leq e \leq 3d/2} \binom{n}{e} \cdot 3^e.$$

Hint: Fix u and let e count the number of coordinates where at least one of v or w disagree with u . Prove that e is at most $3d/2$.

3. Simplify the expression in the case where $d = n/5$ to show that the number of triangles in $G_{n,n/5,2}$ is $O(N \cdot \Delta^{2-\eta})$ for some $\eta > 0$.
4. A famous result in the “probabilistic method” shows (and you don’t have to prove this), that if a graph on N vertices of maximum degree Δ has at most $O(N \cdot \Delta^{2-\eta})$ triangles, then it has an independent set of size $\Omega(\frac{N}{\Delta} \log \Delta)$. Use this result to conclude that there is a binary code of block length n and distance $n/5$ of size $\Omega(n 2^n / \binom{n}{n/5})$. (Note that this improves over the GV-bound by an $\Omega(n)$ factor.)

Exercise 4.11. Use part 1 from Exercise 1.7 to prove the Singleton bound.

Exercise 4.12. Let C be an $(n, k, d)_q$ code. Then prove that fixing any $n - d + 1$ positions uniquely determines the corresponding codeword.

Exercise 4.13. Our goal in this problem is to improve the bound in part 1 in Theorem 4.4.1. Towards that end,

1. Prove that the following holds for every $k \geq 1$. There exists $k+1$ vectors $\mathbf{v}_i^k \in \mathbb{R}^k$ for $i \in [k+1]$ such that (1) $\|\mathbf{v}_i^k\|_2^2 = 1$ for every $i \in [k+1]$ and (2) $\langle \mathbf{v}_i^k, \mathbf{v}_j^k \rangle = -\frac{1}{k}$ for every $i \neq j \in [k+1]$.
2. Using the above part, or otherwise, prove the following result. Let C be a q code of block length n and distance $(1 - \frac{1}{q})n$. Then $|C| \leq 2(q-1)n$. (Note that this is a factor $q/(q-1)$ better than part 1 in Theorem 4.4.1.)

Exercise 4.14. Prove that the bound in Exercise 4.13 is tight for $q = 2$ —i.e. there exists binary codes C with block length n and distance $n/2$ such that $|C| = 2n$.

Exercise 4.15. Prove that part 1 of Lemma 4.4.3 is tight.

Exercise 4.16. Prove that part 2 of Lemma 4.4.3 is tight.

Exercise 4.17. In this exercise we will prove the Plotkin bound (at least part 2 of Theorem 4.4.1) via a purely combinatorial proof.

Given an $(n, k, d)_q$ code C with $d > \left(1 - \frac{1}{q}\right)n$ define

$$S = \sum_{\mathbf{c}_1 \neq \mathbf{c}_2 \in C} \Delta(\mathbf{c}_1, \mathbf{c}_2).$$

For the rest of the problem think of C has an $|C| \times n$ matrix where each row corresponds to a codeword in C . Now consider the following:

1. Looking at the contribution of each column in the matrix above, argue that

$$S \leq \left(1 - \frac{1}{q}\right) \cdot n|C|^2.$$

2. Look at the contribution of the rows in the matrix above, argue that

$$S \geq |C|(|C| - 1) \cdot d.$$

3. Conclude part 2 of Theorem 4.4.1.

Exercise 4.18. In this exercise, we will prove the so called Griesmer Bound. For any $[n, k, d]_q$, prove that

$$n \geq \sum_{i=0}^{k-1} \left\lceil \frac{d}{q^i} \right\rceil.$$

Hint: Recall Exercise 2.18.

Exercise 4.19. Use Exercise 4.18 to prove part 2 of Theorem 4.4.1 for linear codes.

Exercise 4.20. Use Exercise 4.18 to prove Theorem 4.3.1 for linear codes.

4.6 Bibliographic Notes

Theorem 4.2.1 was proved for general codes by Gilbert ([62]) and for linear codes by Varshamov ([160]). Hence, the bound is called the Gilbert-Varshamov bound. The Singleton bound (Theorem 4.3.1) is due to Singleton [142], though versions of this result with the same simple proof seem to have appeared earlier in the work of Joshi [93] who only states the bound for the case $q = 2$. For larger (but still constant) values of q , better lower bounds than the GV bound (i.e., results on the existence of codes) are known. In particular, for every prime power $q \geq 49$, there exist linear codes, called *algebraic geometric* (or AG) codes that outperform the corresponding GV bound². AG codes are out of the scope of this book. An introduction to this class of codes can be found, for instance, in a survey by Høholdt, van Lint, and Pellikaan [88]. Exercise 4.10 is from the work of Jiang and Vardy [92].

²AG codes are only defined for q being a square or a prime and achieve a rate $R \geq 1 - \delta - \frac{1}{\sqrt{q}-1}$. The lower bound of 49 comes from the fact that it is the smallest square of a prime for which this bound improves on the q -ary GV bound.

Chapter 5

The Greatest Code of Them All: Reed-Solomon Codes

Reed-Solomon codes have been studied a lot in coding theory, and are ubiquitous in practice. These codes are basic and based only very elementary algebra. Yet they are optimal in the sense that they exactly meet the Singleton bound (Theorem 4.3.1). For every choice of n and k satisfying $k \leq n$ there is a Reed-Solomon code of dimension k , block length n and distance $n - k + 1$. As if this were not enough, Reed-Solomon codes turn out to be more versatile: they are fully explicit and they have many applications outside of coding theory. (We will see some applications later in the book.)

These codes are defined in terms of univariate polynomials (i.e. polynomials in one unknown/variable) with coefficients from a finite field $\mathbb{F}_q$. It turns out that polynomials over $\mathbb{F}_p$, for prime p , also help us describe finite fields $\mathbb{F}_{p^s}$, for $s > 1$. We start with a quick review of polynomials over finite fields (for a more careful review, please see Appendix D). This will allow us to define Reed-Solomon codes over every field $\mathbb{F}_q$, which we do in the second part of this chapter. Finally in the third part of this chapter we discuss “Maximum Distance Separable” (MDS) codes, which are codes that meet the Singleton bound. We discuss their properties (which in turn are also properties of the Reed-Solomon codes, since they are MDS codes).

5.1 Polynomials and Finite Fields

We start by reviewing the notion of a (univariate) polynomial over a field and define basic notions such as degree, evaluation and root of a polynomial. We conclude with the “degree mantra” that relates the degree to the number of roots.

We begin with the formal definition of a (univariate) polynomial.

Definition 5.1.1. A polynomial over a variable X and a finite field $\mathbb{F}_q$ is given by a finite sequence $(f_0, f_1, \dots, f_d)$ with $f_i \in \mathbb{F}_q$ and is denoted by $F(X) = \sum_{i=0}^d f_i X^i$. The degree of $F(X)$, denoted $\deg(F)$, is the largest index i such that $f_i \neq 0$.

For example, $2X^3 + X^2 + 5X + 6$ is a polynomial over $\mathbb{F}_7$ of degree 3. We ignore leading zeroes

in the definition of a polynomial. For example $0X^4 + 2X^3 + X^2 + 5X + 6$ is the same polynomial as $2X^3 + X^2 + 5X + 6$.

Next, we define some useful notions related to polynomials. We begin with the notion of degree of a polynomial.

We let $\mathbb{F}_q[X]$ denote the set of polynomials over $\mathbb{F}_q$, that is, with coefficients from $\mathbb{F}_q$. Let $F(X), G(X) \in \mathbb{F}_q[X]$ be polynomials. Then $\mathbb{F}_q[X]$ has the following natural operations defined on it:

Addition:

$$F(X) + G(X) = \sum_{i=0}^{\max(\deg(F), \deg(G))} (f_i + g_i) X^i,$$

where the addition on the coefficients is done over $\mathbb{F}_q$. For example, over $\mathbb{F}_2$,

$$X + (1 + X) = X \cdot (1 + 1) + 1 \cdot (0 + 1) = 1$$

(recall that over $\mathbb{F}_2$, $1 + 1 = 0$).¹

Multiplication:

$$F(X) \cdot G(X) = \sum_{i=0}^{\deg(F) + \deg(G)} \left(\sum_{j=0}^{\min(i, \deg(F))} f_j \cdot g_{i-j} \right) X^i,$$

where all the operations on the coefficients are over $\mathbb{F}_q$. For example, over $\mathbb{F}_2$, $X(1 + X) = X + X^2$; $(1 + X)^2 = 1 + 2X + X^2 = 1 + X^2$, where the latter equality follows since $2 \equiv 0 \pmod{2}$.

Next, we define evaluations of a polynomial.

Definition 5.1.2. Given a polynomial $F(X) \in \mathbb{F}_q[X]$ and $\alpha \in \mathbb{F}_q$, the evaluation of $F(X)$ at α , denoted $F(\alpha)$ is $\sum_{i=0}^{\deg F} f_i \alpha^i$. Note that $F(\alpha) \in \mathbb{F}_q$.²

Finally, polynomials don't have multiplicative inverses, but one can divide polynomials by each other and get quotients and residues. The following proposition defines this notion and states some basic properties.

Proposition 5.1.3 (Polynomial Division). Given polynomial $f(X), g(X) \in \mathbb{F}_q[X]$ there exist unique polynomials $q(X)$, the quotient, and $r(X)$, the remainder, with $\deg(r) < \deg(g)$ such that $f(X) = q(X)g(X) + r(X)$. If $g(X) = X - \alpha$ for $\alpha \in \mathbb{F}_q$, then $r(X)$ is the degree 0 polynomial $f(\alpha)$, i.e., the evaluation of f at α .

Definition 5.1.4. $\alpha \in \mathbb{F}_q$ is a root of a polynomial $F(X)$ if $F(\alpha) = 0$.

¹This will be a good time to remember that operations over a finite field are much different from operations over integers/reals. For example, over reals/integers $X + (X + 1) = 2X + 1$.

²While this definition requires the coefficients of F and α to come from the same field, it also extends naturally to the case where one of these is from a field $\mathbb{F}_Q$ extending $\mathbb{F}_q$. Since $\mathbb{F}_q \subseteq \mathbb{F}_Q$, if $\alpha \in \mathbb{F}_q$ and $F(X) \in \mathbb{F}_Q[X]$ then the evaluation is well-defined since $\alpha \in \mathbb{F}_Q$. If $F(X) \in \mathbb{F}_q[X]$ then we use the fact that $\mathbb{F}_q[X] \subseteq \mathbb{F}_Q[X]$ to get a definition of $F(\alpha)$. In both cases $F(\alpha) \in \mathbb{F}_Q$.

For instance, 1 is a root of $1 + X^2$ over $\mathbb{F}_2$.

We now state a basic property of polynomials, the “Degree Mantra”, that will be crucial to our use of polynomials to build error-correcting codes. We also introduce the notion of irreducible polynomials whose existence is closely related to the existence of finite fields of prime power size. Finally, motivated by the need to make fields and field operations fully constructive, we briefly remark on the construction of irreducible polynomials.

Proposition 5.1.5 (“Degree Mantra”). A nonzero polynomial $f(X)$ of degree t over a field $\mathbb{F}_q$ has at most t distinct roots in $\mathbb{F}_q$.

Proof. We will prove the theorem by induction on t . If $t = 0$, we are done. Now, consider $f(X)$ of degree $t > 0$. If f has no roots then we are done, else let $\alpha \in \mathbb{F}_q$ be a root of f . Let $g(X) = X - \alpha$. By the fundamental rule of division of polynomials (Proposition 5.1.3) we have that $f(X) = (X - \alpha)q(X) + f(\alpha) = (X - \alpha)q(X)$. It follows that the degree of $q(X)$ satisfies $\deg(f) = 1 + \deg(q)$, and thus $\deg(q) = t - 1$. Note further that if $\beta \neq \alpha$ is a root of f then we have that $q(\alpha) = f(\beta) \cdot (\beta - \alpha)^{-1}$ and so β is also a root of q . By induction we have that q has at most $t - 1$ roots, and this f has at most t distinct roots (the at most $t - 1$ roots of q plus the root at α). $\square$

The codes we will construct in this chapter do not need any more algebra, except to describe the finite fields that they work over. To understand finite fields beyond those of prime size, we now describe some more basic properties of polynomials.

5.1.1 Irreducibility and Field Extensions

We will start with a special class of polynomials, called irreducible polynomials, which are analogous to how prime numbers are special for natural numbers.

Definition 5.1.6. A polynomial $F(X)$ is irreducible if for every $G_1(X), G_2(X)$ such that $F(X) = G_1(X)G_2(X)$, we have $\min(\deg(G_1), \deg(G_2)) = 0$

For example, $1 + X^2$ is not irreducible over $\mathbb{F}_2$, as

$$(1 + X)(1 + X) = 1 + X^2.$$

However, $1 + X + X^2$ is irreducible, since its non-trivial factors have to be from the linear terms X or $X + 1$. However, it can be checked that neither is a factor of $1 + X + X^2$. (In fact, one can show that $1 + X + X^2$ is the only irreducible polynomial of degree 2 over $\mathbb{F}_2$ — see Exercise 5.4.) A word of caution: if a polynomial $E(X) \in \mathbb{F}_q[X]$ has no root in $\mathbb{F}_q$, it does *not* mean that $E(X)$ is irreducible. For example consider the polynomial $(1 + X + X^2)^2$ over $\mathbb{F}_2$ — it does not have any root in $\mathbb{F}_2$ but it obviously is not irreducible.

The main reason we consider irreducibility of polynomials in this book is that irreducible polynomials lead us to non-prime fields. Just as the set of integers modulo a prime is a field, so is the set of polynomials modulo an irreducible polynomial, and these fields can have non-prime size. We start by first asserting that they form a field; and then turn to properties such as size later.

Theorem 5.1.7. Let $E(X)$ be an irreducible polynomial of degree $s \geq 2$ over $\mathbb{F}_p$, p prime. Then the set of polynomials in $\mathbb{F}_p[X]$ modulo $E(X)$, denoted by $\mathbb{F}_p[X]/E(X)$, is a field.

The proof of the theorem above is similar to the proof of Lemma 2.1.4, so we only sketch the proof here. In particular, we will explicitly state the basic tenets of $\mathbb{F}_p[X]/E(X)$.

- Elements are polynomials in $\mathbb{F}_p[X]$ of degree at most $s-1$. Note that there are p^s such polynomials.
- Addition: $(F(X) + G(X)) \bmod E(X) = F(X) \bmod E(X) + G(X) \bmod E(X) = F(X) + G(X)$. (Since $F(X)$ and $G(X)$ are of degree at most $s-1$, addition modulo $E(X)$ is just plain polynomial addition.)
- Multiplication: $(F(X) \cdot G(X)) \bmod E(X)$ is the unique polynomial $R(X)$ with degree at most $s-1$ such that for some $A(X)$, $R(X) + A(X)E(X) = F(X) \cdot G(X)$
- The additive identity is the zero polynomial, and the additive inverse of any element $F(X)$ is $-F(X)$.
- The multiplicative identity is the constant polynomial 1. It can be shown that for every element $F(X)$, there exists a unique multiplicative inverse $(F(X))^{-1}$.

For example, for $p = 2$ and $E(X) = 1 + X + X^2$, $\mathbb{F}_2[X]/(1 + X + X^2)$ has as its elements

$$\{0, 1, X, 1 + X\}.$$

The additive inverse of any element in $\mathbb{F}_2[X]/(1 + X + X^2)$ is the element itself while the multiplicative inverses of

$$1, X \text{ and } 1 + X$$

in $\mathbb{F}_2[X]/(1 + X + X^2)$ are

$$1, 1 + X \text{ and } X$$

respectively.

Next we turn to the size of the field $\mathbb{F}_q[X]/E(X)$ for an irreducible polynomial E .

Lemma 5.1.8. Let $E(x) \in \mathbb{F}_q[x]$ be an irreducible polynomial of degree s . Then $\mathbb{F}_q[x]/E(x)$ is a field of size q^s .

Proof. This follows from the fact that the elements of $\mathbb{F}_q[x]/E(x)$ are in one to one correspondence with set of remainders of all polynomials in $\mathbb{F}_q[X]$ when divided by $E(X)$ which in turn is simply the set of all polynomials of degree less than s . The number of such polynomials equals q^s (there are q possibilities for the coefficient of X^i for every $0 \leq i < s$). $\square$

Thus a natural question to ask is if irreducible polynomials exist for every degree. Indeed, they do. The following theorem asserts this and the reader may find a proof in Appendix D.

Theorem 5.1.9. For all $s \geq 2$ and $\mathbb{F}_p$, there exists an irreducible polynomial of degree s over $\mathbb{F}_p$. In fact, the number of such monic irreducible polynomials is $\Theta\left(\frac{p^s}{s}\right)$.

The result is true even for general finite fields $\mathbb{F}_q$ and not just prime fields but we stated the version over prime fields for simplicity.

Now recall that Theorem 2.1.5 states that for every prime power p^s , there is a unique field $\mathbb{F}_{p^s}$. This along with Theorems 5.1.7, Lemma 5.1.8 and 5.1.9 imply that:

Corollary 5.1.10. The field $\mathbb{F}_{p^s}$ is $\mathbb{F}_p[X]/E(X)$, where $E(X)$ is an irreducible polynomial of degree s .

The facts about irreducible polynomials listed above give sufficient information not only to determine when finite fields exist, but also how to represent them so as to be able to add, multiply or invert elements, given an irreducible polynomial of degree s over $\mathbb{F}_p$. To make our ability to work with fields completely algorithmic we need one more ingredient — one that allows us to find an irreducible polynomial of degree s in $\mathbb{F}_p$ fast. We now turn to this question.

5.1.2 Finding Irreducible Polynomials

Given any monic³ polynomial $E(X)$ of degree s , it can be verified whether it is an irreducible polynomial by checking if $\gcd(E(X), X^{q^s} - X) = E(X)$, where $\gcd(F(X), G(X))$ is the greatest common denominator (or factor) of polynomials $F(X)$ and $G(X)$. This is true as every irreducible polynomial in $\mathbb{F}_q[X]$ of degree exactly s divides the polynomial $X^{q^s} - X$ (see Proposition D.5.14). Since Euclid's algorithm for computing the $\gcd(F(X), G(X))$ can be implemented in time polynomial in the minimum of $\deg(F)$ and $\deg(G)$ and $\log q$ (see Section D.7.2), this implies that checking whether a given polynomial of degree s over $\mathbb{F}_q[X]$ is irreducible can be done in time $\text{poly}(s, \log q)$.

We now turn to the question of finding an irreducible polynomial, given q and s . A brute force algorithm can simply enumerate all monic polynomials of degree s over $\mathbb{F}_q$ and test each one for irreducibility. This takes $\text{poly}(q^s)$ time. To get a more efficient algorithm we use randomness and Theorem 5.1.9, while will give us a Las Vegas algorithm⁴ to generate an irreducible polynomial of degree s over $\mathbb{F}_q$. We give the code below, but note that the idea of the algorithm is to keep on generating random polynomials until it comes across an irreducible polynomial (Theorem 5.1.9 implies that the algorithm will check $O(p^s)$ polynomials in expectation). Algorithm 5.1.1 presents the formal algorithm.

The above discussion implies the following:

Corollary 5.1.11. There is a Las Vegas algorithm to generate an irreducible polynomial of degree s over any $\mathbb{F}_q$ in expected time $\text{poly}(s, \log q)$.

³I.e. the coefficient of the highest degree term is 1. It can be checked that if $E(X) = e_s X^s + e_{s-1} X^{s-1} + \dots + 1$ is irreducible, then $e_s^{-1} \cdot E(X)$ is also an irreducible polynomial.

⁴A Las Vegas algorithm is a randomized algorithm which always succeeds and we consider its time complexity to be its expected worst-case run time.

Algorithm 5.1.1 Generating Irreducible Polynomial

INPUT: Prime power q and an integer $s > 1$ OUTPUT: A monic irreducible polynomial of degree s over $\mathbb{F}_q$

```
1:  $b \leftarrow 0$ 
2: WHILE  $b = 0$  DO
3:    $F(X) \leftarrow X^s + \sum_{i=0}^{s-1} f_i X^i$ , where each  $f_i$  is chosen uniformly at random from  $\mathbb{F}_q$ .
4:   IF  $\gcd(F(X), X^{q^s} - X) = F(X)$  THEN
5:      $b \leftarrow 1$ .
6: RETURN  $F(X)$ 
```

The above implies that we can ‘construct’ a finite field $\mathbb{F}_q$ in randomized $\text{poly}(\log q)$ time. (See Exercise 5.5 for more including details on what it means to ‘construct’ a finite field.)

This concludes our discussion of polynomials, polynomial arithmetic and properties of polynomials. We now turn to using them to building codes.

5.2 Reed-Solomon Codes

Recall that the Singleton bound (Theorem 4.3.1) states that for every $(n, k, d)_q$ code, $k \leq n - d + 1$. Next, we will study Reed-Solomon codes, which meet the Singleton bound (i.e. satisfy $k = n - d + 1$) but have the unfortunate property that $q \geq n$. Note that this implies that the Singleton bound is tight, at least for $q \geq n$.

We begin with the definition of Reed-Solomon codes.

Definition 5.2.1 (Reed-Solomon code). Let $\mathbb{F}_q$ be a finite field, and choose n and k satisfying $k \leq n \leq q$. Fix a sequence $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)$ of n distinct elements (also called evaluation points) from $\mathbb{F}_q$. We define an encoding function for Reed-Solomon code $\text{RS}_q[\alpha, k] : \mathbb{F}_q^k \rightarrow \mathbb{F}_q^n$ as follows. Map a message $\mathbf{m} = (m_0, m_1, \dots, m_{k-1})$ with $m_i \in \mathbb{F}_q$ to the degree $k-1$ polynomial.

$$\mathbf{m} \mapsto f_{\mathbf{m}}(X),$$

where

$$f_{\mathbf{m}}(X) = \sum_{i=0}^{k-1} m_i X^i. \quad (5.1)$$

Note that $f_{\mathbf{m}}(X) \in \mathbb{F}_q[X]$ is a polynomial of degree at most $k-1$. The encoding of $\mathbf{m}$ is the evaluation of $f_{\mathbf{m}}(X)$ at all the α_i 's:

$$\text{RS}_q[\alpha, k](\mathbf{m}) = (f_{\mathbf{m}}(\alpha_1), f_{\mathbf{m}}(\alpha_2), \dots, f_{\mathbf{m}}(\alpha_n)).$$

When q , α and k are known from context, we suppress them in the notation and simply refer to the map as RS. We call the image of this map, i.e., the set $\{\text{RS}[\mathbf{m}] \mid \mathbf{m} \in \mathbb{F}_q^k\}$, the Reed-Solomon code or RS code. A common special case is $n = q-1$ with the set of evaluation points being $\mathbb{F}^* \stackrel{\text{def}}{=} \mathbb{F} \setminus \{0\}$.

For example, the first row below are all the codewords in the $[3, 2]_3$ Reed-Solomon codes where the evaluation points are $\mathbb{F}_3$ (and the codewords are ordered by the corresponding messages from $\mathbb{F}_3^2$ in lexicographic order where for clarity the second row shows the polynomial $f_{\mathbf{m}}(X)$ for the corresponding $\mathbf{m} \in \mathbb{F}_3^2$ in gray):

(0,0,0),	(1,1,1),	(2,2,2),	(0,1,2),	(1,2,0),	(2,0,1),	(0,2,1),	(1,0,2),	(2,1,0)
0,	1,	2,	X ,	$X+1$,	$X+2$,	$2X$,	$2X+1$,	$2X+2$

Notice that by definition, the entries in $\{\alpha_1, \dots, \alpha_n\}$ are distinct and thus, must have $n \leq q$.

In what follows we will describe the basic properties of Reed-Solomon codes. In principle we should refer to the codes as $\text{RS}_q[\alpha, k]$ since all the parameters are needed to specify the code. However for notation simplicity we will assume k, n, q , and $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ are fixed and satisfy $k \leq n \leq q$ allowing us to refer to the resulting code as simply RS. (Thus all results below hold for every such choice of k, n, q and $\alpha_1, \dots, \alpha_n$.)

Claim 5.2.2. RS codes are linear codes.

Proof. The proof follows from the fact that if $a \in \mathbb{F}_q$ and $f(X), g(X) \in \mathbb{F}_q[X]$ are polynomials of degree $\leq k-1$, then $af(X)$ and $f(X) + g(X)$ are also polynomials of degree $\leq k-1$. In particular, let messages $\mathbf{m}_1$ and $\mathbf{m}_2$ be mapped to $f_{\mathbf{m}_1}(X)$ and $f_{\mathbf{m}_2}(X)$ where $f_{\mathbf{m}_1}(X), f_{\mathbf{m}_2}(X) \in \mathbb{F}_q[X]$ are polynomials of degree at most $k-1$ and because of the mapping defined in (5.1), it can be verified that:

$$f_{\mathbf{m}_1}(X) + f_{\mathbf{m}_2}(X) = f_{\mathbf{m}_1 + \mathbf{m}_2}(X),$$

and

$$af_{\mathbf{m}_1}(X) = f_{a\mathbf{m}_1}(X).$$

In other words,

$$\text{RS}(\mathbf{m}_1) + \text{RS}(\mathbf{m}_2) = \text{RS}(\mathbf{m}_1 + \mathbf{m}_2)$$

$$a\text{RS}(\mathbf{m}_1) = \text{RS}(a\mathbf{m}_1).$$

Therefore RS is a $[n, k]_q$ linear code. □

The second and more interesting claim is the following:

Claim 5.2.3. The minimum distance of RS is $n - k + 1$.

The claim on the distance follows from Proposition 5.1.5 which asserted that every non-zero polynomial of degree $k-1$ over $\mathbb{F}_q[X]$ has at most $k-1$ roots. The proof below uses this to prove a lower bound on the distance. The upper bound follows from the Singleton Bound (Theorem 4.3.1). Details below.

Proof of Claim 5.2.3. Fix arbitrary $\mathbf{m}_1 \neq \mathbf{m}_2 \in \mathbb{F}_q^k$. Note that $f_{\mathbf{m}_1}(X), f_{\mathbf{m}_2}(X) \in \mathbb{F}_q[X]$ are distinct polynomials of degree at most $k-1$ since $\mathbf{m}_1 \neq \mathbf{m}_2 \in \mathbb{F}_q^k$. Then $f_{\mathbf{m}_1}(X) - f_{\mathbf{m}_2}(X) \neq 0$ also has degree at most $k-1$. Note that $\text{wt}(\text{RS}(\mathbf{m}_2) - \text{RS}(\mathbf{m}_1)) = \Delta(\text{RS}(\mathbf{m}_1), \text{RS}(\mathbf{m}_2))$. The weight of $\text{RS}(\mathbf{m}_2) - \text{RS}(\mathbf{m}_1)$ is n minus the number of zeroes in $\text{RS}(\mathbf{m}_2) - \text{RS}(\mathbf{m}_1)$, which is equal to n minus the number of roots that $f_{\mathbf{m}_1}(X) - f_{\mathbf{m}_2}(X)$ has among $\{\alpha_1, \dots, \alpha_n\}$. That is,

$$\Delta(\text{RS}(\mathbf{m}_1), \text{RS}(\mathbf{m}_2)) = n - |\{\alpha_i \mid f_{\mathbf{m}_1}(\alpha_i) = f_{\mathbf{m}_2}(\alpha_i)\}|.$$

By Proposition 5.1.5, $f_{\mathbf{m}_1}(X) - f_{\mathbf{m}_2}(X)$ has at most $k-1$ roots. Thus, the weight of $\text{RS}(\mathbf{m}_2) - \text{RS}(\mathbf{m}_1)$ is at least $n - (k-1) = n - k + 1$. Therefore $d \geq n - k + 1$, and since the Singleton bound (Theorem 4.3.1) implies that $d \leq n - k + 1$, we have $d = n - k + 1$.⁵ The argument above also shows that distinct polynomials $f_{\mathbf{m}_1}(X), f_{\mathbf{m}_2}(X) \in \mathbb{F}_q[X]$ are mapped to distinct codewords. (This is because the Hamming distance between any two codewords is at least $n - k + 1 \geq 1$, where the last inequality follows as $k \leq n$.) Therefore, the code contains q^k codewords and has dimension k . The claim on linearity of the code follows from Claim 5.2.2. $\square$

We thus have an exact understanding of the dimension and distance of the Reed-Solomon codes, which we summarize in the theorem below. The theorem also notes that the parameters match those of the Singleton Bound. Recall that the Plotkin bound (Corollary 4.4.2) implies that to achieve the Singleton bound, the alphabet size cannot be a constant. Thus, some growth of q with n is unavoidable to match the Singleton bound, and the Reed-Solomon codes match it with $q \geq n$.

Theorem 5.2.4. *RS is a $[n, k, n - k + 1]_q$ code. That is, RS codes match the Singleton bound.*

Finally, we describe a generator matrix for RS codes. Such a matrix is guaranteed to exist by Claim 5.2.2, but now we give an explicit one. By Definition 5.2.1, any basis $f_{\mathbf{m}_1}, \dots, f_{\mathbf{m}_k}$ of polynomial of degree at most $k-1$ gives rise to a basis $\text{RS}(\mathbf{m}_1), \dots, \text{RS}(\mathbf{m}_k)$ of the code. A particularly nice polynomial basis is the set of monomials $1, X, \dots, X^i, \dots, X^{k-1}$. The corresponding generator matrix, whose i th row (numbering rows from 0 to $k-1$) is

$$(\alpha_1^i, \alpha_2^i, \dots, \alpha_j^i, \dots, \alpha_n^i)$$

and this generator matrix is called the *Vandermonde* matrix of size $k \times n$:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_j & \cdots & \alpha_n \\ \alpha_1^2 & \alpha_2^2 & \cdots & \alpha_j^2 & \cdots & \alpha_n^2 \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ \alpha_1^i & \alpha_2^i & \cdots & \alpha_j^i & \cdots & \alpha_n^i \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ \alpha_1^{k-1} & \alpha_2^{k-1} & \cdots & \alpha_j^{k-1} & \cdots & \alpha_n^{k-1} \end{pmatrix}$$

The class of codes that match the Singleton bound have their own name, which we define and study next.

5.3 Maximum Distance Separable Codes and Properties

Definition 5.3.1 (MDS codes). *An $(n, k, d)_q$ code is called Maximum Distance Separable (MDS) if $d = n - k + 1$.*

⁵See Exercise 5.2 for an alternate direct argument.

Thus, Reed-Solomon codes are MDS codes.

Next, we prove an interesting property of an MDS code $C \subseteq \Sigma^n$ with integral dimension k . We begin with the following notation.

Definition 5.3.2. *For every subset of indices $S \subseteq [n]$ of size exactly k and a code $C \subseteq \Sigma^n$, C_S is the set of all codewords in C projected onto the indices in S .*

MDS codes have the following nice property that we shall prove for the special case of Reed-Solomon codes first and subsequently for the general case as well.

Proposition 5.3.3. *Let $C \subseteq \Sigma^n$ of integral dimension k be an MDS code, then for all $S \subseteq [n]$ such that $|S| = k$, we have $|C_S| = \Sigma^k$.*

Before proving Proposition 5.3.3 in its full generality, we present its proof for the special case of Reed-Solomon codes.

Consider any $S \subseteq [n]$ of size k and fix an arbitrary $\mathbf{v} = (v_1, \dots, v_k) \in \mathbb{F}_q^k$, we need to show that there exists a codeword $\mathbf{c} \in \text{RS}$ (assume that the RS code evaluates polynomials of degree at most $k-1$ over $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$) such that $\mathbf{c}_S = \mathbf{v}$. Consider a generic degree $k-1$ polynomial $F(X) = \sum_{i=0}^{k-1} f_i X^i$. Thus, we need to show that there exists $F(X)$ such that $F(\alpha_i) = v_i$ for all $i \in S$, where $|S| = k$.

For notational simplicity, assume that $S = [k]$. We think of f_i 's as unknowns in the equations that arise out of the relations $F(\alpha_i) = v_i$. Thus, we need to show that there is a solution to the following system of linear equations:

$$\begin{pmatrix} p_0 & p_1 & \cdots & p_{k-1} \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ \alpha_1 & \alpha_i & \alpha_k \\ \alpha_1^2 & \alpha_i^2 & \alpha_k^2 \\ \vdots & \vdots & \vdots \\ \alpha_1^{k-1} & \alpha_i^{k-1} & \alpha_k^{k-1} \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_k \end{pmatrix}$$

The above constraint matrix is a Vandermonde matrix and is known to have full rank (see Exercise 5.3). Hence, by Exercise 2.7, there always exists a unique solution for $(p_0, \dots, p_{k-1})$. This completes the proof for Reed-Solomon codes.

Next, we prove the property for the general case which is presented below

Proof of Proposition 5.3.3. Consider a $|C| \times n$ matrix where each row represents a codeword in C . Hence, there are $|C| = |\Sigma|^k$ rows in the matrix. The number of columns is equal to the block length n of the code. Since C is Maximum Distance Separable, its distance $d = n - k + 1$.

Let $S \subseteq [n]$ be of size exactly k . It can be verified that for every $\mathbf{c}^i \neq \mathbf{c}^j \in C$, the corresponding projections $\mathbf{c}_S^i$ and $\mathbf{c}_S^j \in C_S$ are not the same. As otherwise $\Delta(\mathbf{c}^i, \mathbf{c}^j) \leq d - 1$, which is not possible as the minimum distance of the code C is d . Therefore, every codeword in C gets mapped to a

distinct codeword in C_S . As a result, $|C_S| = |C| = |\Sigma|^k$. As $C_S \subseteq \Sigma^k$, this implies that $C_S = \Sigma^k$, as desired. $\square$

Proposition 5.3.3 implies an important property in pseudorandomness: see Exercise 5.13 for more.

5.4 Exercises

Exercise 5.1. Prove that every function $f: \mathbb{F}_q \rightarrow \mathbb{F}_q$ is equivalent to a polynomial $P(X) \in \mathbb{F}_q[X]$ of degree at most $q-1$: that is, for every $\alpha \in \mathbb{F}_q$

$$f(\alpha) = P(\alpha).$$

Furthermore, prove the choice of this polynomial P is unique.

Exercise 5.2. For every $[n, k]_q$ Reed-Solomon code, i.e., for every $RS_q[\alpha, k]$ for every choice of $k \leq n \leq q$ and $\alpha = (\alpha_1, \dots, \alpha_n)$, exhibit two codewords that are at Hamming distance exactly $n - k + 1$.

Exercise 5.3. Let $\alpha_1, \dots, \alpha_k$ be distinct elements in a field $\mathbb{F}$. Consider the $k \times k$ Vandermonde matrix $V(\alpha_1, \dots, \alpha_k)$ whose (i, j) 'th entry is α_i^{j-1} for $i, j \in \{1, 2, \dots, k\}$. Prove that $V(\alpha_1, \dots, \alpha_k)$ has full rank. Use this property to prove that a Reed-Solomon code of dimension k can efficiently correct $n - k$ erasures.

Exercise 5.4. Prove that $X^2 + X + 1$ is the unique irreducible polynomial of degree two over $\mathbb{F}_2$.

Exercise 5.5. In this problem we will consider what it means to 'construct' a finite field. For simplicity, assume that $q = p^s$ for some $s \geq 1$. A representation of a finite field $\mathbb{F}_q$ is a triple (S, θ, f) where $S \subset \{0, 1\}^*$ with $|S| = p^s$ is set of representations of elements on $\mathbb{F}_q$, θ is some 'auxiliary' representation and a bijection $f: \mathbb{F}_{p^s} \rightarrow S$. For every $\alpha \in \mathbb{F}_{p^s}$, $f(\alpha)$ is the representation. Also implicit in this definition is given $\alpha, \beta \in \mathbb{F}_p^s$ how one computes $f(\alpha) + f(\beta)$, $-f(\alpha)$, $f(\alpha) \cdot f(\beta)$. Further, one needs to identify the additive and multiplicative identities in S . Finally, given a non-zero element $\alpha \in \mathbb{F}_{p^s}$, compute $f(\alpha)^{-1}$. The auxiliary representation θ can be used to implement these operations.

We call a representation efficient if all of the operations can be supported in $\text{poly}(\log q)$ time. In this problem we will explore the problem of constructing an efficient representation of a finite field in $\text{poly}(\log q)$ (randomized) time.

1. Let $E(X)$ be an irreducible polynomial of degree s . Given $E(X)$, prove that the representation $\mathbb{F}_p[X]/E(X)$ (i.e. $\theta = E(X)$ and for every $\mathbf{u} \in \mathbb{F}_p^s$, $f(\mathbf{u}) = f_{\mathbf{u}}(X)$ as per (5.1) and the additive and multiplicative identities are the 0 and 1 polynomials) is an efficient representation.

Hint: The following fact might be useful: for every $\alpha \in \mathbb{F}_q^*$, $\alpha^{q-2} = \alpha^{-1}$.

⁶Note that there is a bijection between $\mathbb{F}_{p^s}$ to $\mathbb{F}_p^s$ and hence we can define f on $\mathbb{F}_p^s$ instead of $\mathbb{F}_{p^s}$.

2. Using the above part or otherwise prove that for every prime p and integer $s \geq 1$, an efficient representation of $\mathbb{F}_{p^s}$ can be computed in (randomized) $\text{poly}(s \log p)$ time.

Exercise 5.6. In Exercise 2.17, we saw that any linear code can be converted into a systematic code. In other words, there is a map to convert Reed-Solomon codes into a systematic one. In this exercise the goal is to come up with an explicit encoding function that results in a systematic Reed-Solomon code.

In particular, given the set of evaluation points $\alpha_1, \dots, \alpha_n$, design an explicit map f from $\mathbb{F}_q^k$ to a polynomial of degree at most $k-1$ such that the following holds. For every message $\mathbf{m} \in \mathbb{F}_q^k$, if the corresponding polynomial is $f_{\mathbf{m}}(X)$, then the vector $(f_{\mathbf{m}}(\alpha_i))_{i \in [n]}$ has the message $\mathbf{m}$ appear in the corresponding codeword (say in its first k positions). Further, prove that this map results in an $[n, k, n - k + 1]_q$ code.

Exercise 5.7. We have defined Reed-Solomon codes as evaluation codes. They are sometimes also defined in a dual sense, as polynomials with pre-specified roots, and this exercise will demonstrate that.

Let $\mathbb{F}_q$ be a field, and $\mathbb{F}_q^*$ be the multiplicative group of its nonzero elements. Let $n = q - 1$ and let α be a generator of $\mathbb{F}_q^*$ so that the vector $\alpha = (1, \alpha, \dots, \alpha^{n-1})$ has all distinct elements and $\alpha^n = 1$. Consider the Reed-Solomon code over a field $\mathbb{F}_q$ with evaluation points being α :

$$RS_q[\alpha, k] = \{(p(1), p(\alpha), \dots, p(\alpha^{n-1})) \mid p(X) \in \mathbb{F}[X] \text{ has degree} \leq k-1\}.$$

1. Prove that

$$RS_q[\alpha, k] = \{(c_0, c_1, \dots, c_{n-1}) \in \mathbb{F}^n \mid C(\alpha^\ell) = 0 \text{ for } 1 \leq \ell \leq n - k, \text{ where } C(X) = c_0 + c_1 X + \dots + c_{n-1} X^{n-1}\}. \quad (5.2)$$

Hint: Exercise 2.3 might be useful.

2. Consider the "twisted" variant of Reed-Solomon code that maps a message polynomial $m(X) = m_0 + m_1 X + \dots + m_{\ell-1} X^{\ell-1}$ to the evaluations of the polynomial $X \cdot m(X)$ on the sequence α , where $\ell = n - k$. Prove that this code equals the dual $(RS_q[\alpha, k])^\perp$ of the above Reed-Solomon code.

Exercise 5.8 (Generalized Reed-Solomon Codes). For a field $\mathbb{F}$ with $|\mathbb{F}| \geq n$, an n -tuple $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)$ of n distinct elements of $\mathbb{F}$, and a vector $\mathbf{v} = (v_1, v_2, \dots, v_n) \in (\mathbb{F}^*)^n$ of n (not necessarily distinct) nonzero elements from $\mathbb{F}$, the Generalized Reed-Solomon code $GRS_{\mathbb{F}}[\alpha, k, \mathbf{v}, k]$ is defined as follows:

$$GRS_{\mathbb{F}}[\alpha, k, \mathbf{v}] = \{(v_1 \cdot p(\alpha_1), v_2 \cdot p(\alpha_2), \dots, v_n \cdot p(\alpha_n)) \mid p(X) \in \mathbb{F}[X] \text{ has degree} < k\}. \quad (5.3)$$

(In particular, note that $RS_q[\alpha, k] = GRS_{\mathbb{F}_q}[\alpha, k, (1, \dots, 1)]$.)

1. Prove that $GRS_{\mathbb{F}}[\alpha, k, \mathbf{v}, k]$ is an $[n, k, n - k + 1]_{\mathbb{F}}$ linear code.

2. Prove that the dual code of $\text{GRS}_{\mathbb{F}}[\alpha, k, \mathbf{v}, k]$ is

$$\text{GRS}_{\mathbb{F}}[\alpha, k, \mathbf{v}, k]^{\perp} = \text{GRS}_{\mathbb{F}}[\alpha, k, \mathbf{u}, n - k]$$

for $\mathbf{u} = (u_1, u_2, \dots, u_n) \in (\mathbb{F}^*)^n$ where for $i = 1, 2, \dots, n$,

$$u_i = \frac{1}{v_i \prod_{j \neq i} (\alpha_i - \alpha_j)}.$$

Hint: Express a polynomial of degree $< n$ in terms of the Lagrange polynomials L_i that satisfy $L_i(\alpha_j) = 1$ if $i = j$ and 0 otherwise. For polynomials p, q of degree $< k$ and $< n - k$ respectively, the product $p \cdot q$ has degree $< n - 1$.

3. Deduce that statement about the dual of $\text{RS}_{\mathbb{F}_q}^*[n, k]$ from Exercise 5.7, Part 2 as a corollary of the above.

4. If we take the full Reed-Solomon code consisting of the evaluations of polynomials of degree less than k at all field elements (in some fixed order), i.e., $\text{RS}_q[\alpha, k]$ for $\alpha = (\eta_1, \dots, \eta_q)$ where $\mathbb{F}_q = \{\eta_1, \dots, \eta_q\}$, prove that its dual $\text{RS}_q[\alpha, k]^{\perp}$ equals $\text{RS}_q[\alpha, q - k]$ (with evaluation points in the same order).

Exercise 5.9. In this problem we will look at a very important class of codes called BCH codes⁷.

Fix an integer m and let $q = 2^m$ and $n = q - 1$. Let non-zero elements of the field $\mathbb{F}_{2^m}$ be $\{\eta_1, \dots, \eta_n\}$ and let $\alpha = (\eta_1, \dots, \eta_n)$. Given non-negative integer $k \leq n$, the binary BCH code, denoted $C_{\text{BCH}} = C_{\text{BCH}}(m, k)$, is defined as $\text{RS}_{2^m}[\alpha, k] \cap \mathbb{F}_2^n$. In other words C_{BCH} consists of those codewords in the Reed-Solomon code $\text{RS}_{2^m}[\alpha, k]$ all of whose coordinates lie in the subfield $\mathbb{F}_2 \subseteq \mathbb{F}_{2^m}$.

1. Let $d = n - k + 1$. Prove that C_{BCH} is a binary linear code of distance at least d and dimension at least $n - (d - 1) \log_2(n + 1)$.

Hint: Use the characterization (5.2) of the Reed-Solomon code from Exercise 5.7.

2. Prove a better lower bound of $n - \left\lceil \frac{d-1}{2} \right\rceil \log_2(n + 1)$ on the dimension of C_{BCH} .

Hint: There are redundant checks among the parity checks (5.2) defining C_{BCH} , using the fact that the coefficients are in $\mathbb{F}_2$.

3. For $d = 3$, C_{BCH} is the same as another code we have seen. What is that code?

4. Define the subcode of C_{BCH} with a global parity check, i.e., the condition $c_1 + c_2 + \dots + c_n = 0$ (over $\mathbb{F}_2$). Let d be an even integer. Show how to use the BCH code with a global parity check to construct a binary linear code of distance at least d and dimension at least $n - (d/2 - 1) \log_2(n + 1) - 1$.

⁷The acronym BCH stands for Bose-Chaudhuri-Hocquenghem, the discoverers of this family of codes.

5. Conclude that for all n of the form $2^m - 1$ and integers d , $2 \leq d < n / \log_2(n_1)$, one can construct an $[n, k', d']_2$ binary linear code with $d' \geq d$ and $k' \geq n - \left\lceil \frac{d-1}{2} \right\rceil \log_2(n + 1) - 1$.

6. Prove that the $\left\lceil \frac{d-1}{2} \right\rceil$ factor cannot be any smaller.

Hint: What does the Hamming bound say?

Exercise 5.10. In this exercise, we will consider BCH-like codes in the theme of Exercise 5.9, but applied to the GRS codes of Exercise 5.8. Consider the Generalized Reed-Solomon code $C_{\text{GRS}} = \text{GRS}_{\mathbb{F}}[\alpha, k, \mathbf{v}]$ defined in (5.3) of dimension k and block length n over a field $\mathbb{F} = \mathbb{F}_{2^m}$. Now, define its binary intersection code $C^* := C_{\text{GRS}} \cap \mathbb{F}_2^n$, which will be the object of study in this exercise.

1. Prove that C^* is a binary linear code of rate at least $1 - \frac{(n-k)m}{n}$.

Hint: How many parity checks are needed to define this code?

2. Let $\mathbf{c} \in \mathbb{F}_2^n$ be a nonzero binary vector. Prove that for every choice of the evaluation points sequence α there are at most $(2^m - 1)^k$ choices of the vector $\mathbf{v}$ for which $\mathbf{c} \in C_{\text{GRS}}$.

3. Using the above, prove that if the integer D satisfies $\text{Vol}_2(n, D - 1) < (2^m - 1)^{n-k}$ (where $\text{Vol}_2(n, D - 1) = \sum_{i=0}^{D-1} \binom{n}{i}$), then there exists a vector $\mathbf{v} \in (\mathbb{F}^*)^n$ such that the minimum distance of the binary code C^* is at least D .

4. Using parts 1 and 3 above, prove that the family of codes $\text{GRS}_{\mathbb{F}}[\alpha, k, \mathbf{v}] \cap \mathbb{F}_2^n$ contains binary linear codes that meet the Gilbert-Varshamov bound.

Exercise 5.11. Since Reed-Solomon codes are linear codes, by Proposition 2.3.5, one can do error detection for Reed-Solomon codes in quadratic time. In this problem, we will see that one can design even more efficient error detection algorithm for Reed-Solomon codes. In particular, we will consider data streaming algorithms (see Section 22.5 for more motivation on this class of algorithms). A data stream algorithm makes a sequential pass on the input, uses poly-logarithmic space and spend only poly-logarithmic time on each location in the input. In this problem we show that there exists a randomized data stream algorithm to solve the error detection problem for Reed-Solomon codes.

1. Give a randomized data stream algorithm that given as input a sequence $(i_1, \alpha_1), \dots, (i_n, \alpha_n) \in [m] \times \mathbb{F}_q$ that implicitly defines $\mathbf{y} \in \mathbb{F}_q^m$, where for every $\ell \in [m]$, $y_{\ell} = \sum_{j \in [n] | i_j = \ell} \alpha_{\ell}$, decides whether $\mathbf{y} = \mathbf{0}$ with probability at least $2/3$. Your algorithm should use $O(\log q(m + n))$ space and $\text{polylog}(q(m + n))$ time per position of $\mathbf{y}$. For simplicity, you can assume that given an integer $t \geq 1$ and prime power q , the algorithm has oracle access to an irreducible polynomial of degree t over $\mathbb{F}_q$.

Hint: Use Reed-Solomon codes.

2. Given $[q, k]_q$ Reed-Solomon code C (i.e. with the evaluation points being $\mathbb{F}_q$), present a data stream algorithm for error detection of C with $O(\log q)$ space and $\text{polylog} q$ time per position of the received word. The algorithm should work correctly with probability at least

2/3. You should assume that the data stream algorithm has access to the values of k and q (and knows that C has $\mathbb{F}_q$ as its evaluation points).

Hint: Part 1 and Exercise ?? should be helpful.

Exercise 5.12. Recall the definition of Hadamard codes from Section 2.6: the $[2^r, r, 2^{r-1}]_2$ Hadamard code is generated by the $r \times 2^r$ matrix whose i th (for $0 \leq i \leq 2^r - 1$) column is the binary representation of i . This exercise gives a polynomial view of Hadamard codes.

Specifically, prove that the Hadamard codeword for the message $(m_1, m_2, \dots, m_r) \in \{0, 1\}^r$ is the evaluation of the (multivariate) polynomial $m_1 X_1 + m_2 X_2 + \dots + m_r X_r$ (where $X_1, \dots, X_r$ are the r variables) over all the possible assignments to the variables $(X_1, \dots, X_r)$ from $\{0, 1\}^r$.

Using the definition of Hadamard codes above (re)prove the fact that the code has distance 2^{r-1} .

Exercise 5.13. A set $S \subseteq \mathbb{F}_q^n$ is said to be a t -wise independent source (for some $1 \leq t \leq n$) if given a uniformly random sample $(X_1, \dots, X_n)$ from S , the n random variables are t -wise independent: i.e. any subset of t variables are uniformly independent random variables over $\mathbb{F}_q$. We will explore properties of these objects in this exercise.

1. Prove that the definition of t -wise independent source is equivalent to the definition in Exercise 2.14.
2. Prove that for every linear code C which does not have a codeword position that is 0 in all codewords is a 1-wise independent source.
3. Prove that any $[n, k]_q$ MDS code is a k -wise independent source but is not a $k+1$ -wise independent source.
4. Using part 3 or otherwise prove that there exists a k -wise independent source $S \subseteq \mathbb{F}_2^n$ of size at most $(2n)^k$. Conclude that $k(\log_2 n + 1)$ uniformly and independent random bits are enough to compute n random bits that are k -wise independent. Improve the bound slightly to show that $k(\log_2 n - \log_2 \log_2 n + O(1))$ -random bits are enough to generate k -wise independent source over $\mathbb{F}_2$.
5. For $0 < p \leq 1/2$, we say the n binary random variables $X_1, \dots, X_n$ are p -biased and t -wise independent if any of the t random variables are independent and $\Pr[X_i = 1] = p$ for every $i \in [n]$. For the rest of the problem, let p be a power of $1/2$. Then show that any $t \cdot \log_2(1/p)$ -wise independent random variables can be converted into t -wise independent p -biased random variables. Conclude that one can construct such sources with $t \log_2(1/p)(1 + \log_2(n \log_2(1/p)))$ uniformly random bits. Then improve this bound to $t(1 + \max(\log_2(1/p), \log_2 n))$ uniformly random bits.

Exercise 5.14. Use Exercises 2.14 and 5.9 part 5 to prove the following: for every integers n, k with $1 \leq k \leq n$, at most $\lfloor \frac{k}{2} \rfloor \log_2(2n)$ random bits are enough to compute n -bits that are k -wise independent.

Exercise 5.15. In many applications, errors occur in “bursts”—i.e., all the error locations are contained in a contiguous region (think of a scratch on a DVD or disk). In this problem we will use how one can use Reed-Solomon codes to correct bursty errors.

An error vector $\mathbf{e} \in \{0, 1\}^n$ is called a t -single burst error pattern if all the non-zero bits in $\mathbf{e}$ occur in the range $[i, i + t - 1]$ for some $1 \leq i \leq n - t + 1$. Further, a vector $\mathbf{e} \in \{0, 1\}^n$ is called a (s, t) -burst error pattern if it is the union of at most s t -single burst error pattern (i.e. all non-zero bits in $\mathbf{e}$ are contained in one of at most s contiguous ranges in $[n]$).

We call a binary code $C \subseteq \{0, 1\}^n$ to be (s, t) -burst error correcting if one can uniquely decode from any (s, t) -burst error pattern. More precisely, given an (s, t) -burst error pattern $\mathbf{e}$ and any codeword $\mathbf{c} \in C$, the only codeword $\mathbf{c}' \in C$ such that $(\mathbf{c} + \mathbf{e}) - \mathbf{c}'$ is an (s, t) -burst error pattern satisfies $\mathbf{c}' = \mathbf{c}$.

1. Prove that if C is (st) -error correcting (in the sense of Definition 1.3.5), then it is also (s, t) -burst error correcting. Conclude that for every $\epsilon > 0$, there exists code with rate $\Omega(\epsilon^2)$ and block length n that is (s, t) -burst error correcting for every s, t such that $s \cdot t \leq (\frac{1}{4} - \epsilon) \cdot n$.
2. Prove that for every rate $R > 0$ and for large enough n , there exist (s, t) -burst error correcting as long as $s \cdot t \leq (\frac{1-R-\epsilon}{2}) \cdot n$ and $t \geq \Omega(\frac{\log n}{\epsilon})$. In particular, one can correct from $\frac{1}{2} - \epsilon$ fraction of burst-errors (as long as each burst is “long enough”) with rate $\Omega(\epsilon)$ (compare this with item 1).

Hint: Use Reed-Solomon codes.

Exercise 5.16. In this problem, we will consider the number-theoretic counterpart of Reed-Solomon codes. Let $1 \leq k < n$ be integers and let $p_1 < p_2 < \dots < p_n$ be n distinct primes. Denote $K = \prod_{i=1}^k p_i$ and $N = \prod_{i=1}^n p_i$. The notation $\mathbb{Z}_M$ stands for integers modulo M , i.e., the set $\{0, 1, \dots, M-1\}$. Consider the Chinese Remainder code defined by the encoding map $E : \mathbb{Z}_K \rightarrow \mathbb{Z}_{p_1} \times \mathbb{Z}_{p_2} \times \dots \times \mathbb{Z}_{p_n}$ defined by:

$$E(m) = (m \bmod p_1, m \bmod p_2, \dots, m \bmod p_n).$$

(Note that this is not a code in the usual sense we have been studying since the symbols at different positions belong to different alphabets. Still notions such as distance of this code make sense and are studied in the question below.)

Suppose that $m_1 \neq m_2$. For $1 \leq i \leq n$, define the indicator variable $b_i = 1$ if $E(m_1)_i \neq E(m_2)_i$ and $b_i = 0$ otherwise. Prove that $\prod_{i=1}^n p_i^{b_i} > N/K$.

Use the above to deduce that when $m_1 \neq m_2$, the encodings $E(m_1)$ and $E(m_2)$ differ in at least $n - k + 1$ locations.

Exercise 5.17. In this problem, we will consider derivatives over a finite field $\mathbb{F}_q$. Unlike the case of derivatives over reals, derivatives over finite fields do not have any physical interpretation but as we shall see shortly, the notion of derivatives over finite fields is still a useful concept. In particular, given a polynomial $f(X) = \sum_{i=0}^t f_i X^i$ over $\mathbb{F}_q$, we define its derivative as

$$f'(X) = \sum_{i=0}^{t-1} (i+1) \cdot f_{i+1} \cdot X^i.$$

Further, we will denote by $f^{(i)}(X)$, the result of applying the derivative on f i times. In this problem, we record some useful facts about derivatives.

1. Define $R(X, Z) = f(X + Z) = \sum_{i=0}^t r_i(X) \cdot Z^i$. Then for every $j \geq 1$,

$$f^{(j)}(X) = j! \cdot r_j(X).$$

2. Using part 1 or otherwise, show that for every $j \geq \text{char}(\mathbb{F}_q)$,⁸ $f^{(j)}(X) \equiv 0$.
3. Let $j < \text{char}(\mathbb{F}_q)$. Further, assume that for every $0 \leq i < j$, $f^{(i)}(\alpha) = 0$ for some $\alpha \in \mathbb{F}_q$. Then prove that $(X - \alpha)^j$ divides $f(X)$.
4. Finally, prove the following generalization of the degree mantra (Proposition 5.1.5). Let $f(X)$ be a non-zero polynomial of degree t and $m \leq \text{char}(\mathbb{F}_q)$. Then there exists at most $\lfloor \frac{t}{m} \rfloor$ distinct elements $\alpha \in \mathbb{F}_q$ such that $f^{(j)}(\alpha) = 0$ for every $0 \leq j < m$.

Exercise 5.18. In this exercise, we will consider a code that is related to Reed-Solomon codes and uses derivatives from Exercise 5.17. These codes are called derivative codes.

Let $m \geq 1$ be an integer parameter and consider parameters $k < \text{char}(\mathbb{F}_q)$ and n such that $m < k < nm$. Then the derivative code with parameters (n, k, m) is defined as follow. Consider any message $\mathbf{m} \in \mathbb{F}_q^k$ and let $f_{\mathbf{m}}(X)$ be the message polynomial as defined for the Reed-Solomon code. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ be distinct elements. Then the codeword for $\mathbf{m}$ is given by

$$\begin{pmatrix} f_{\mathbf{m}}(\alpha_1) & f_{\mathbf{m}}(\alpha_2) & \cdots & f_{\mathbf{m}}(\alpha_n) \\ f_{\mathbf{m}}^{(1)}(\alpha_1) & f_{\mathbf{m}}^{(1)}(\alpha_2) & \cdots & f_{\mathbf{m}}^{(1)}(\alpha_n) \\ \vdots & \vdots & \ddots & \vdots \\ f_{\mathbf{m}}^{(m-1)}(\alpha_1) & f_{\mathbf{m}}^{(m-1)}(\alpha_2) & \cdots & f_{\mathbf{m}}^{(m-1)}(\alpha_n) \end{pmatrix}.$$

1. Prove that the above code is linear over $\mathbb{F}_q$, meaning that if $c_1, c_2 \in (\mathbb{F}_q^m)^n$ are codewords, then so is $\alpha c_1 + \beta c_2$ for all $\alpha, \beta \in \mathbb{F}_q$. Here we define $\alpha \mathbf{v}$ for $\alpha \in \mathbb{F}_q$ and $\mathbf{v} \in \mathbb{F}_q^m$ as multiplication of coordinates of $\mathbf{v}$ by α , and as usual αc_1 is componentwise multiplication of symbols of c_1 by α .

2. Prove that the above code has rate $k/(nm)$ and distance at least $n - \lfloor \frac{k-1}{m} \rfloor$.

Exercise 5.19. In this exercise, we will consider another code related to Reed-Solomon codes that are called Folded Reed-Solomon codes. We will see a lot more of these codes in Chapter 17.

Let $m \geq 1$ be an integer parameter and let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ are distinct elements such that for some element $\gamma \in \mathbb{F}_q^*$, the sets

$$\{\alpha_i, \alpha_i \gamma, \alpha_i \gamma^2, \dots, \alpha_i \gamma^{m-1}\}, \quad (5.4)$$

⁸ $\text{char}(\mathbb{F}_q)$ denotes the characteristic of $\mathbb{F}_q$. That is, if $q = p^s$ for some prime p , then $\text{char}(\mathbb{F}_q) = p$. Any natural number i in $\mathbb{F}_q$ is equivalent to $i \bmod \text{char}(\mathbb{F}_q)$.

are pair-wise disjoint for different $i \in [n]$. Then the folded Reed-Solomon code with parameters $(m, k, n, \gamma, \alpha_1, \dots, \alpha_n)$ is defined as follows. Consider any message $\mathbf{m} \in \mathbb{F}_q^k$ and let $f_{\mathbf{m}}(X)$ be the message polynomial as defined for the Reed-Solomon code. Then the codeword for $\mathbf{m}$ is given by:

$$\begin{pmatrix} f_{\mathbf{m}}(\alpha_1) & f_{\mathbf{m}}(\alpha_2) & \cdots & f_{\mathbf{m}}(\alpha_n) \\ f_{\mathbf{m}}(\alpha_1 \cdot \gamma) & f_{\mathbf{m}}(\alpha_2 \cdot \gamma) & \cdots & f_{\mathbf{m}}(\alpha_n \cdot \gamma) \\ \vdots & \vdots & \ddots & \vdots \\ f_{\mathbf{m}}(\alpha_1 \cdot \gamma^{m-1}) & f_{\mathbf{m}}(\alpha_2 \cdot \gamma^{m-1}) & \cdots & f_{\mathbf{m}}(\alpha_n \cdot \gamma^{m-1}) \end{pmatrix}.$$

Prove that the above code has rate $k/(nm)$ and distance at least $n - \lfloor \frac{k-1}{m} \rfloor$.

Exercise 5.20. In this problem we will see that Reed-Solomon codes, derivative codes (Exercise 5.18) and folded Reed-Solomon codes (Exercise 5.19) are all essentially special cases of a large family of codes that are based on polynomials. We begin with the definition of these codes.

Let $m \geq 1$ be an integer parameter and define $m < k \leq n$. Further, let $E_1(X), \dots, E_n(X)$ be n polynomials over $\mathbb{F}_q$, each of degree m . Further, these polynomials pair-wise do not have any non-trivial factors (i.e. $\gcd(E_i(X), E_j(X))$ has degree 0 for every $i \neq j \in [n]$.) Consider any message $\mathbf{m} \in \mathbb{F}_q^k$ and let $f_{\mathbf{m}}(X)$ be the message polynomial as defined for the Reed-Solomon code. Then the codeword for $\mathbf{m}$ is given by:

$$(f_{\mathbf{m}}(X) \bmod E_1(X), f_{\mathbf{m}}(X) \bmod E_2(X), \dots, f_{\mathbf{m}}(X) \bmod E_n(X)).$$

In the above we think of $f_{\mathbf{m}}(X) \bmod E_i(X)$ as an element of $\mathbb{F}_q^m$. In particular, given a polynomial of degree at most $m-1$, we will consider any bijection between the q^m such polynomials and $\mathbb{F}_q^m$. We will first see that this code is MDS and then we will see why it contains Reed-Solomon and related codes as special cases.

1. Prove that the above code has rate $k/(nm)$ and distance at least $n - \lfloor \frac{k-1}{m} \rfloor$.
2. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ be distinct elements. Define $E_i(X) = X - \alpha_i$. Prove that for this special case the above code (with $m = 1$) is the Reed-Solomon code.
3. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ be distinct elements. Define $E_i(X) = (X - \alpha_i)^m$. Prove that for this special case the above code is the derivative code (with an appropriate mapping from polynomials of degree at most $m-1$ and $\mathbb{F}_q^m$, where the mapping could be different for each $i \in [n]$ and can depend on $E_i(X)$).
4. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ be distinct elements and $\gamma \in \mathbb{F}_q^*$ such that (5.4) is satisfied. Define $E_i(X) = \prod_{j=0}^{m-1} (X - \alpha_i \cdot \gamma^j)$. Prove that for this special case the above code is the folded Reed-Solomon code (with an appropriate mapping from polynomials of degree at most $m-1$ and $\mathbb{F}_q^m$, where the mapping could be different for each $i \in [n]$ and can depend on $E_i(X)$).

Exercise 5.21. In this exercise we will develop a sufficient condition to determine the irreducibility of certain polynomials called the Eisenstein's criterion.

Let $F(X, Y)$ be a polynomial of $\mathbb{F}_q$. Think of this polynomial as over X with coefficients as polynomials in Y over $\mathbb{F}_q$. Technically, we think of the coefficients as coming from the ring of polynomials in Y over $\mathbb{F}_q$. We will denote the ring of polynomials in Y over $\mathbb{F}_q$ as $\mathbb{F}_q(Y)$ and we will denote the polynomials in X with coefficients from $\mathbb{F}_q(Y)$ as $\mathbb{F}_q(Y)[X]$.

In particular, let

$$F(X, Y) = X^t + f_{t-1}(Y) \cdot X^{t-1} + \cdots + f_0(Y),$$

where each $f_i(Y) \in \mathbb{F}_q(Y)$. Let $P(Y)$ be a prime for $\mathbb{F}_q(Y)$ (i.e. $P(Y)$ has degree at least one and if $P(Y)$ divides $A(Y) \cdot B(Y)$ then $P(Y)$ divides at least one of $A(Y)$ or $B(Y)$). If the following conditions hold:

- (i) $P(Y)$ divides $f_i(Y)$ for every $0 \leq i < t$; but
- (ii) $P^2(Y)$ does not divide $f_0(Y)$

then $F(X, Y)$ does not have any non-trivial factors over $\mathbb{F}_q(Y)[X]$ (i.e. all factors have either degree t or 0 in X).

In the rest of the problem, we will prove this result in a sequence of steps:

1. For the sake of contradiction assume that $F(X, Y) = G(X, Y) \cdot H(X, Y)$ where

$$G(X, Y) = \sum_{i=0}^{t_1} g_i(Y) \cdot X^i \text{ and } H(X, Y) = \sum_{i=0}^{t_2} h_i(Y) \cdot X^i,$$

where $0 < t_1, t_2 < t$. Then prove that $P(Y)$ does not divide both of $g_0(Y)$ and $h_0(Y)$.

For the rest of the problem WLOG assume that $P(Y)$ divides $g_0(Y)$ (and hence does not divide $h_0(Y)$).

2. Prove that there exists an i^* such that $P(Y)$ divide $g_i(Y)$ for every $0 \leq i < i^*$ but $P(Y)$ does not divide $g_{i^*}(Y)$ (define $g_t(Y) = 1$).
3. Prove that $P(Y)$ does not divide $f_i(Y)$. Conclude that $F(X, Y)$ does not have any non-trivial factors, as desired.

Exercise 5.22. We have mentioned objects called algebraic-geometric (AG) codes, that generalize Reed-Solomon codes and have some amazing properties: see for example, Section 4.6. The objective of this exercise is to construct one such AG code, and establish its rate vs distance trade-off.

Let p be a prime and $q = p^2$. Consider the equation

$$Y^p + Y = X^{p+1} \tag{5.5}$$

over $\mathbb{F}_q$.

1. Prove that there are exactly p^3 solutions in $\mathbb{F}_q \times \mathbb{F}_q$ to (5.5). That is, if $S \subseteq \mathbb{F}_q^2$ is defined as

$$S = \{(\alpha, \beta) \in \mathbb{F}_q^2 \mid \beta^p + \beta = \alpha^{p+1}\}$$

then $|S| = p^3$.

2. Prove that the polynomial $F(X, Y) = Y^p + Y - X^{p+1}$ is irreducible over $\mathbb{F}_q$.

Hint: Exercise 5.21 could be useful.

3. Let $n = p^3$. Consider the evaluation map $\text{ev} : \mathbb{F}_q[X, Y] \rightarrow \mathbb{F}_q^n$ defined by

$$\text{ev}(f) = (f(\alpha, \beta) : (\alpha, \beta) \in S).$$

Prove that if $f \neq 0$ and is not divisible by $Y^p + Y - X^{p+1}$, then $\text{ev}(f)$ has Hamming weight at least $n - \deg(f)(p+1)$, where $\deg(f)$ denotes the total degree of f .

Hint: You are allowed to make use of Bézout's theorem, which states that if $f, g \in \mathbb{F}_q[X, Y]$ are nonzero polynomials with no common factors, then they have at most $\deg(f)\deg(g)$ common zeroes.

4. For an integer parameter $\ell \geq 1$, consider the set $\mathcal{F}_\ell$ of bivariate polynomials

$$\mathcal{F}_\ell = \{f \in \mathbb{F}_q[X, Y] \mid \deg(f) \leq \ell, \deg_X(f) \leq p\}$$

where $\deg_X(f)$ denotes the degree of f in X .

Prove that $\mathcal{F}_\ell$ is an $\mathbb{F}_q$ -linear space of dimension $(\ell+1)(p+1) - \frac{p(p+1)}{2}$.

5. Consider the code $C \subseteq \mathbb{F}_q^n$ for $n = p^3$ defined by

$$C = \{\text{ev}(f) \mid f \in \mathcal{F}_\ell\}.$$

Prove that C is a linear code with minimum distance at least $n - \ell(p+1)$.

6. Deduce a construction of an $[n, k]_q$ code with distance $d \geq n - k + 1 - p(p-1)/2$.

(Note that Reed-Solomon codes have $d = n - k + 1$, whereas these codes are off by $p(p-1)/2$ from the Singleton bound. However they are much longer than Reed-Solomon codes, with a block length of $n = q^{3/2}$, and the deficiency from the Singleton bound is only $o(n)$.)

5.5 Bibliographic Notes

Reed-Solomon codes were invented by Reed and Solomon [131] in the form described in Definition 5.2.1, i.e., as evaluations of polynomials. Later, Gorenstein and Zierler [?] showed that for specific choices of α , the resulting Reed-Solomon code is actually a “BCH code”. (This is the connection explored in Exercise 5.7.) BCH codes were themselves discovered slightly earlier in the independent works of Bose and Ray-Chaudhuri [21] and Hocquenghem [87]. We note that the original definitions of BCH codes used the coefficients of polynomials to represent messages (as in Exercise 5.7). The equivalent definition of these codes used in Exercise 5.7 as subcodes of Reed-Solomon codes, again uses the above mentioned connection in [?].

Chapter 6

What Happens When the Noise is Stochastic: Shannon's Theorem

Shannon was the first to present a rigorous mathematical framework for communication, which (as we have already seen) is the problem of reproducing at one point (typically called the *receiver* of the channel) a message selected at another point (called the *sender* to the channel). Unlike Hamming, Shannon modeled the noise stochastically, i.e., as a well defined random process. He proved a result that pin-pointed the best possible rate of transmission of information over a very wide range of stochastic channels. In fact, Shannon looked at the communication problem at a higher level, where he allowed for compressing the data first (before applying any error-correcting code), so as to minimize the amount of symbols transmitted over the channel.

In this chapter, we will study some stochastic noise models most of which were proposed by Shannon. We then prove an optimal tradeoff between the rate and fraction of errors that are correctable for a specific stochastic noise model called the Binary Symmetric Channel.

6.1 Overview of Shannon's Result

Shannon introduced the notion of reliable communication¹ over (potentially) noisy channels. Broadly, there are two types of channels that were studied by Shannon:

- (Noisy Channel) This type of channel introduces errors during transmission, which result in an incorrect reception of the transmitted signal by the receiver. Redundancy is added at the transmitter to increase reliability of the transmitted data. The redundancy is taken off at the receiver. This process is termed as *Channel Coding*.
- (Noise-free Channel) As the name suggests, this channel does not introduce any type of error in transmission. Redundancy in source data is used to compress the source data at the transmitter. The data is decompressed at the receiver. The process is popularly known as *Source Coding*.

¹That is, the ability to successfully send the required information over a channel that can lose or corrupt data.

Figure 6.1 presents a generic model of a communication system, which combines the two concepts we discussed above.

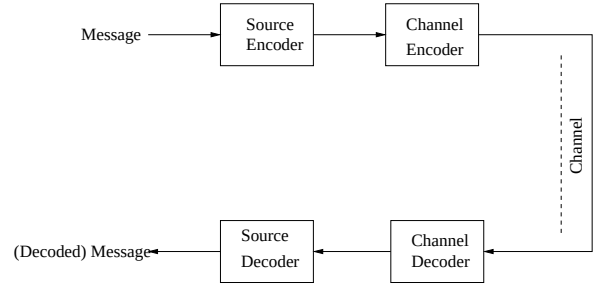


Figure 6.1: The communication process

In Figure 6.1, source coding and channel coding are coupled. In general, to get the optimal performance, it makes sense to design both the source and channel coding schemes simultaneously. However, Shannon's source coding theorem allows us to decouple both these parts of the communication setup and study each of these parts separately. Intuitively, this makes sense: if one can have reliable communication over the channel using channel coding, then for the source coding the resulting channel effectively has no noise.

For source coding, Shannon proved a theorem that precisely identifies the amount by which the message can be compressed: this amount is related to the *entropy* of the message. We will not talk much more about source coding in this book. (However, see Exercises 6.10, 6.11, 6.12 and Chapter 16.) From now on, we will exclusively focus on the channel coding part of the communication setup. Note that one aspect of channel coding is how we model the channel noise. So far we have seen Hamming's worst case noise model in some detail. Next, we will study some specific stochastic channels.

6.2 Shannon's Noise Model

Shannon proposed a stochastic way of modeling noise. The input symbols to the channel are assumed to belong to some *input alphabet* $\mathcal{X}$, while the channel outputs symbols from its *output alphabet* $\mathcal{Y}$. The following diagram shows this relationship:

$$\mathcal{X} \ni x \rightarrow \boxed{\text{channel}} \rightarrow y \in \mathcal{Y}$$

The channels considered by Shannon are also *memoryless*, that is, noise acts independently on each transmitted symbol. In this book, we will only study *discrete* channels where both the alphabets $\mathcal{X}$ and $\mathcal{Y}$ are finite. For the sake of variety, we will define one channel that is continuous, though we will not study it in any detail later on.

The final piece in specification of a channel is the *transition matrix* $\mathbf{M}$ that governs the process of how the channel introduces error. In particular, the channel is described in the form of a matrix with entries as the crossover probability over all combination of the input and output alphabets. For every pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$, let $\Pr(y|x)$ denote the probability that y is output by the channel when x is input to the channel. Then the transition matrix is given by $\mathbf{M}(x, y) = \Pr(y|x)$. The specific structure of the matrix is shown below.

$$\mathbf{M} = \begin{pmatrix} & \vdots & \\ \cdots & \Pr(y|x) & \cdots \\ & \vdots & \end{pmatrix}$$

Next, we look at some specific instances of channels.

Binary Symmetric Channel (BSC). Let $0 \leq p \leq 1$. The Binary Symmetric Channel with *crossover probability* p or BSC_p is defined as follows. $\mathcal{X} = \mathcal{Y} = \{0, 1\}$. The 2×2 transition matrix can naturally be represented as a bipartite graph where the left vertices correspond to the rows and the right vertices correspond to the columns of the matrix, where $\mathbf{M}(x, y)$ is represented as the weight of the corresponding (x, y) edge. For BSC_p , the graph is illustrated in Figure 6.2.

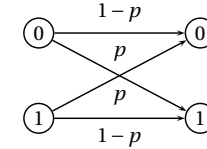


Figure 6.2: Binary Symmetric Channel BSC_p

The corresponding transition matrix would look like this:

$$\begin{matrix} & 0 & 1 \\ \begin{matrix} 0 \\ 1 \end{matrix} & \begin{pmatrix} 1-p & p \\ p & 1-p \end{pmatrix} \end{matrix}$$

In other words, every bit is flipped with probability p . We claim that we need to only consider the case when $p \leq \frac{1}{2}$, i.e. if we know how to ensure reliable communication over BSC_p for $p \leq \frac{1}{2}$, then we can also handle the case of $p > \frac{1}{2}$. (See Exercise 6.1.)

q -ary Symmetric Channel ($q\text{SC}$). We now look at the generalization of BSC_p to alphabets of size $q \geq 2$. Let $0 \leq p \leq 1 - \frac{1}{q}$. (As with the case of BSC_p , we can assume that $p \leq 1 - \frac{1}{q}$ —see Exercise 6.2.) The q -ary Symmetric Channel with crossover probability p , or $q\text{SC}_p$, is defined as follows. $\mathcal{X} = \mathcal{Y} = [q]$. The transition matrix $\mathbf{M}$ for $q\text{SC}_p$ is defined as follows:

$$M(x, y) = \begin{cases} 1-p & \text{if } y = x \\ \frac{p}{q-1} & \text{if } y \neq x \end{cases}$$

In other words, every symbol is retained as is at the output with probability $1 - p$ and is distorted to each of the $q - 1$ possible different symbols with equal probability of $\frac{p}{q-1}$.

Binary Erasure Channel (BEC) In the previous two examples that we saw, $\mathcal{X} = \mathcal{Y}$. However, this might not always be the case.

Let $0 \leq \alpha \leq 1$. The Binary Erasure Channel with *erasure probability* α (denoted by BEC_α) is defined as follows. $\mathcal{X} = \{0, 1\}$ and $\mathcal{Y} = \{0, 1, ?\}$, where $?$ denotes an *erasure*. The transition matrix is as follows:

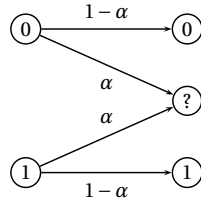


Figure 6.3: Binary Erasure Channel BEC_α

In Figure 6.3 any missing edge represents a transition that occurs with 0 probability. In other words, every bit in BEC_α is erased with probability α (and is left unchanged with probability $1 - \alpha$).

Binary Input Additive Gaussian White Noise Channel (BIAGWN). We now look at a channel that is continuous. Let $\sigma \geq 0$. The Binary Input Additive Gaussian White Noise Channel with standard deviation σ or BIAGWN_σ is defined as follows. $\mathcal{X} = \{-1, 1\}$ and $\mathcal{Y} = \mathbb{R}$. The noise is modeled by the continuous Gaussian probability distribution function. The Gaussian distribution has lots of nice properties and is a popular choice for modeling noise continuous in nature. Given $(x, y) \in \{-1, 1\} \times \mathbb{R}$, the noise $y - x$ is distributed according to the Gaussian distribution of mean of zero and standard deviation of σ . In other words,

$$\Pr(y | x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\left(\frac{y-x}{2\sigma^2}\right)^2\right).$$

6.2.1 Error Correction in Stochastic Noise Models

We now need to revisit the notion of error correction from Section 1.3. Note that unlike Hamming's noise model, we cannot hope to *always* recover the transmitted codeword. As an example, in BSC_p there is always some positive probability that a codeword can be distorted into another codeword during transmission. In such a scenario no decoding algorithm can hope to recover the transmitted codeword. Thus, in some stochastic channels there is always some *decoding error probability* (where the randomness is from the channel noise): see Exercise 6.14 for example channels where one can have zero decoding error probability. However, we would

like this error probability to be small for every possible transmitted codeword. More precisely, for every message, we would like the decoding algorithm to recover the transmitted message with probability $1 - f(n)$, where $\lim_{n \rightarrow \infty} f(n) \rightarrow 0$; that is, $f(n)$ is $o(1)$. Ideally, we would like to have $f(n) = 2^{-\Omega(n)}$. We will refer to $f(n)$ as the decoding error probability.

6.2.2 Shannon's General Theorem

Recall that the big question of interest in this book is the tradeoff between the rate of the code and the fraction of errors that can be corrected. For stochastic noise models that we have seen, it is natural to think of the fraction of errors to be the parameter that governs the amount of error that is introduced by the channel. For example, for BSC_p , we will think of p as the fraction of errors.

Shannon's remarkable theorem on channel coding was to *precisely* identify when reliable transmission is possible over the stochastic noise models that he considered. In particular, for the general framework of noise models, Shannon defined the notion of *capacity*, which is a real number such that reliable communication is possible if and only if the rate is less than the capacity of the channel. In other words, given a noisy channel with capacity C , if information is transmitted at rate R for every $R < C$, then there exists a coding scheme that guarantees negligible probability of miscommunication. On the other hand if $R > C$, then regardless of the chosen coding scheme there will be some message for which the decoding error probability is bounded from below by some constant.

In this chapter, we are going to state (and prove) Shannon's general result for the special case of BSC_p .

6.3 Shannon's Result for BSC_p

For the rest of the chapter, we will use the notation $\mathbf{e} \sim \text{BSC}_p$ to denote an error pattern $\mathbf{e} \in \{0, 1\}^n$ that is drawn according to the error distribution induced by BSC_p . We are now ready to state the theorem.

Theorem 6.3.1 (Shannon's Capacity Theorem for BSC). *For real numbers p, ϵ such that $0 \leq p < \frac{1}{2}$ and $0 \leq \epsilon \leq \frac{1}{2} - p$, the following statements are true for large enough n :*

1. *There exists a real $\delta > 0$, an encoding function $E : \{0, 1\}^k \rightarrow \{0, 1\}^n$ and a decoding function $D : \{0, 1\}^n \rightarrow \{0, 1\}^k$ where $k \leq \lfloor (1 - H(p + \epsilon))n \rfloor$, such that the following holds for every $\mathbf{m} \in \{0, 1\}^k$:*

$$\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \leq 2^{-\delta n}.$$

2. *If $k \geq \lceil (1 - H(p) + \epsilon)n \rceil$ then for every pair of encoding and decoding functions, $E : \{0, 1\}^k \rightarrow \{0, 1\}^n$ and $D : \{0, 1\}^n \rightarrow \{0, 1\}^k$, there exists $\mathbf{m} \in \{0, 1\}^k$ such that*

$$\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] > \frac{1}{2}.$$

Note that Theorem 6.3.1 implies that the capacity of BSC_p is $1 - H(p)$. It can also be shown that the capacity of $q\text{SC}_p$ and BEC_α are $1 - H_q(p)$ and $1 - \alpha$ respectively. (See Exercises 6.6 and 6.7.)

The entropy function appears in Theorem 6.3.1 due to the same technical reason that it appears in the GV bound: the entropy function allows us to use sufficiently tight bounds on the volume of a Hamming ball (Proposition 3.3.3).

6.3.1 Proof of Converse of Shannon's Capacity Theorem for BSC

We start with the proof of part (2) of Theorem 6.3.1. (Proof of part (1) follows in the next section.)

For the proof we will assume that $p > 0$ (since when $p = 0$, $1 - H(p) + \varepsilon > 1$ and so we have nothing to prove). For the sake of contradiction, assume that the following holds for every $\mathbf{m} \in \{0, 1\}^k$:

$$\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \leq \frac{1}{2}.$$

Define $D_{\mathbf{m}}$ to be the set of received words $\mathbf{y}$ that are decoded to $\mathbf{m}$ by D , that is,

$$D_{\mathbf{m}} = \{\mathbf{y} | D(\mathbf{y}) = \mathbf{m}\}.$$

The main idea behind the proof is the following: first note that the sets $D_{\mathbf{m}}$ partition the entire space of received words $\{0, 1\}^n$ (see Figure 6.4 for an illustration) since D is a function.

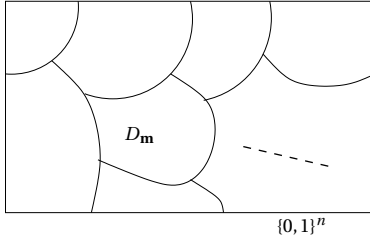


Figure 6.4: The sets $D_{\mathbf{m}}$ partition the ambient space $\{0, 1\}^n$.

Next, we will argue that since the decoding error probability is at most a $1/2$, then $D_{\mathbf{m}}$ for every $\mathbf{m} \in \{0, 1\}^k$ is "large." Then by a simple packing argument, it follows that we cannot have too many distinct $\mathbf{m}$, which we will show implies that $k < (1 - H(p) + \varepsilon)n$: a contradiction. Before we present the details, we outline how we will argue that $D_{\mathbf{m}}$ is large. Let $S_{\mathbf{m}}$ be the shell of radius $[(1 - \gamma)pn, (1 + \gamma)pn]$ around $E(\mathbf{m})$, that is,

$$S_{\mathbf{m}} = B(E(\mathbf{m}), (1 + \gamma)pn) \setminus B(E(\mathbf{m}), (1 - \gamma)pn).$$

We will set $\gamma > 0$ in terms of ε and p at the end of the proof. (See Figure 6.5 for an illustration.) Then we argue that because the decoding error probability is bounded by $1/2$, most of

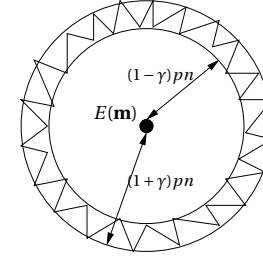


Figure 6.5: The shell $S_{\mathbf{m}}$ of inner radius $(1 - \gamma)pn$ and outer radius $(1 + \gamma)pn$.

the received words in the shell $S_{\mathbf{m}}$ are decoded correctly, i.e. they fall in $D_{\mathbf{m}}$. To complete the argument, we show that the number of such received words is indeed large enough.

Fix an arbitrary message $\mathbf{m} \in \{0, 1\}^k$. Note that by our assumption, the following is true (where from now on we omit the explicit dependence of the probability on the BSC_p noise for clarity):

$$\Pr [E(\mathbf{m}) + \mathbf{e} \notin D_{\mathbf{m}}] \leq \frac{1}{2}. \quad (6.1)$$

Further, by the (multiplicative) Chernoff bound (Theorem 3.1.10),

$$\Pr [E(\mathbf{m}) + \mathbf{e} \notin S_{\mathbf{m}}] \leq 2^{-\Omega(\gamma^2 n)}. \quad (6.2)$$

(6.1) and (6.2) along with the union bound (Proposition 3.1.5) imply the following:

$$\Pr [E(\mathbf{m}) + \mathbf{e} \notin D_{\mathbf{m}} \cap S_{\mathbf{m}}] \leq \frac{1}{2} + 2^{-\Omega(\gamma^2 n)}.$$

The above in turn implies that

$$\Pr [E(\mathbf{m}) + \mathbf{e} \in D_{\mathbf{m}} \cap S_{\mathbf{m}}] \geq \frac{1}{2} - 2^{-\Omega(\gamma^2 n)} \geq \frac{1}{4}, \quad (6.3)$$

where the last inequality holds for large enough n . Next we upper bound the probability above to obtain a lower bound on $|D_{\mathbf{m}} \cap S_{\mathbf{m}}|$.

It is easy to see that

$$\Pr [E(\mathbf{m}) + \mathbf{e} \in D_{\mathbf{m}} \cap S_{\mathbf{m}}] \leq |D_{\mathbf{m}} \cap S_{\mathbf{m}}| \cdot p_{\max},$$

where

$$p_{\max} = \max_{\mathbf{y} \in S_{\mathbf{m}}} \Pr [E(\mathbf{m}) + \mathbf{e} = \mathbf{y}] = \max_{d \in [(1 - \gamma)pn, (1 + \gamma)pn]} p^d (1 - p)^{n - d}.$$

In the above, the second equality follows from the fact that all error patterns with the same Hamming weight appear with the same probability when chosen according to BSC_p . Next, note

that $p^d(1-p)^{n-d}$ is decreasing in d for $p \leq \frac{1}{2}$.² Thus, we have

$$p_{\max} = p^{(1-\gamma)pn}(1-p)^{n-(1-\gamma)pn} = \left(\frac{1-p}{p}\right)^{\gamma pn} \cdot p^{pn}(1-p)^{(1-p)n} = \left(\frac{1-p}{p}\right)^{\gamma pn} 2^{-nH(p)}.$$

Thus, we have shown that

$$\Pr[E(\mathbf{m}) + \mathbf{e} \in D_{\mathbf{m}} \cap S_{\mathbf{m}}] \leq |D_{\mathbf{m}} \cap S_{\mathbf{m}}| \cdot \left(\frac{1-p}{p}\right)^{\gamma pn} 2^{-nH(p)},$$

which, by (6.3), implies that

$$|D_{\mathbf{m}} \cap S| \geq \frac{1}{4} \cdot \left(\frac{1-p}{p}\right)^{-\gamma pn} 2^{nH(p)}. \quad (6.4)$$

Next, we consider the following sequence of relations:

$$2^n = \sum_{\mathbf{m} \in \{0,1\}^k} |D_{\mathbf{m}}| \quad (6.5)$$

$$\begin{aligned} &\geq \sum_{\mathbf{m} \in \{0,1\}^k} |D_{\mathbf{m}} \cap S_{\mathbf{m}}| \\ &\geq \frac{1}{4} \left(\frac{1}{p} - 1\right)^{-\gamma pn} \sum_{\mathbf{m} \in \{0,1\}^k} 2^{H(p)n} \end{aligned} \quad (6.6)$$

$$\begin{aligned} &= 2^{k-2} \cdot 2^{H(p)n - \gamma p \log(1/p-1)n} \\ &> 2^{k+H(p)n - \varepsilon n}. \end{aligned} \quad (6.7)$$

In the above, (6.5) follows from the fact that for $\mathbf{m}_1 \neq \mathbf{m}_2$, $D_{\mathbf{m}_1}$ and $D_{\mathbf{m}_2}$ are disjoint. (6.6) follows from (6.4). (6.7) follows for large enough n and if we pick $\gamma = \frac{\varepsilon}{2p \log(\frac{1}{p}-1)}$. (Note that as $0 < p < \frac{1}{2}$, $\gamma = \Theta(\varepsilon)$.)

(6.7) implies that $k < (1 - H(p) + \varepsilon)n$, which is a contradiction. The proof of part (2) of Theorem 6.3.1 is complete.

Remark 6.3.2. It can be verified that the proof above can also work if the decoding error probability is bounded by $1 - 2^{-\beta n}$ (instead of the $1/2$ in part (2) of Theorem 6.3.1) for small enough $\beta = \beta(\varepsilon) > 0$.

Next, we will prove part (1) of Theorem 6.3.1.

6.3.2 Proof of Positive Part of Shannon's Theorem

Proof Overview. The proof of part (1) of Theorem 6.3.1 will be done by the probabilistic method (Section 3.2). In particular, we randomly select an encoding function $E : \{0,1\}^k \rightarrow \{0,1\}^n$. That is, for every $\mathbf{m} \in \{0,1\}^k$ pick $E(\mathbf{m})$ uniformly and independently at random from $\{0,1\}^n$. D will be the maximum likelihood decoding (MLD) function. The proof will have the following two steps:

- (Step 1) For every arbitrary $\mathbf{m} \in \{0,1\}^k$, we will show that for a random choice of E , the probability of failure, over BSC_p noise, is small. This implies the existence of a good encoding function for every message.
- (Step 2) We will show a similar result for *all* $\mathbf{m}$. This involves dropping half of the code words.

Note that there are two sources of randomness in the proof:

1. Randomness in the choice of encoding function E and
2. Randomness in the noise introduced by the BSC_p channel.

We stress that the first kind of randomness is for the probabilistic method while the second kind of randomness will contribute to the decoding error probability.

“Proof by picture” of Step 1. Before proving part (1) of Theorem 6.3.1, we will provide a pictorial proof of Step 1. We begin by fixing $\mathbf{m} \in \{0,1\}^k$. In Step 1, we need to estimate the following quantity:

$$\mathbb{E}_E \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right].$$

By the additive Chernoff bound (Theorem 3.1.10), with all but an exponentially small probability, the received word will be contained in a Hamming ball of radius $(p + \varepsilon')n$ (for some $\varepsilon' > 0$ that we will choose appropriately). So one can assume that the received word $\mathbf{y}$ with high probability satisfies $\Delta(E(\mathbf{m}), \mathbf{y}) \leq (p + \varepsilon')n$. Given this, pretty much the only thing to do is to estimate the decoding error probability for such a $\mathbf{y}$. Note that by the fact that D is MLD, an error can happen only if there exists another message $\mathbf{m}'$ such that $\Delta(E(\mathbf{m}'), \mathbf{y}) \leq \Delta(E(\mathbf{m}), \mathbf{y})$. The latter event implies that $\Delta(E(\mathbf{m}'), \mathbf{y}) \leq (p + \varepsilon')n$ (see Figure 6.6).

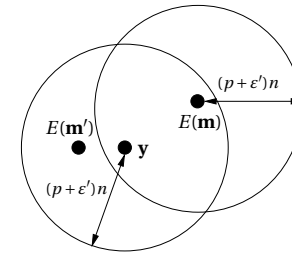


Figure 6.6: Hamming balls of radius $(p + \varepsilon')n$ and centers $E(\mathbf{m})$ and $\mathbf{y}$ illustrates Step 1 in the proof of part (1) of Shannon's capacity theorem for the BSC.

²Indeed $p^d(1-p)^{n-d} = (p/(1-p))^d(1-p)^n$ and the bound $p \leq \frac{1}{2}$ implies that the first exponent is at most 1, which implies that the expression is decreasing in d .

Thus, the decoding error probability is upper bounded by

$$\Pr_{\mathbf{e} \sim \text{BSC}_p} [E(\mathbf{m}') \in B(\mathbf{y}, (p + \varepsilon')n)] = \frac{\text{Vol}_2((p + \varepsilon')n, n)}{2^n} \approx \frac{2^{H(p)n}}{2^n},$$

where the last step follows from Proposition 3.3.3 (and we ignore the term dependent on ε' by using the $\approx$ notation). Finally, by the union bound (Proposition 3.1.5), the existence of such a “bad” $\mathbf{m}'$ is upper bounded by $\approx \frac{2^k 2^{nH(p)}}{2^n}$, which by our choice of k is $2^{-\Omega(n)}$, as desired.

The Details of Step 1. For notational convenience, we will use $\mathbf{y}$ and $E(\mathbf{m}) + \mathbf{e}$ interchangeably:

$$\mathbf{y} = E(\mathbf{m}) + \mathbf{e}.$$

That is, $\mathbf{y}$ is the received word when $E(\mathbf{m})$ is transmitted and $\mathbf{e}$ is the error pattern.

We start the proof by restating the decoding error probability in part (1) of Shannon's capacity theorem for BSC_p (Theorem 6.3.1) by breaking up the quantity into two sums:

$$\begin{aligned} \Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] &= \sum_{\mathbf{y} \in B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \cdot \mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}} \\ &+ \sum_{\mathbf{y} \notin B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \cdot \mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}, \end{aligned} \quad (6.8)$$

where $\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}$ is the indicator function for the event that $D(\mathbf{y}) \neq \mathbf{m}$ given that $E(\mathbf{m})$ was the transmitted codeword and we use $\mathbf{y}|E(\mathbf{m})$ as a shorthand for “ $\mathbf{y}$ is the received word given that $E(\mathbf{m})$ was the transmitted codeword.” As $\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}} \leq 1$ (since it takes a value in $\{0, 1\}$), we have

$$\begin{aligned} \sum_{\mathbf{y} \notin B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \cdot \mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}} &\leq \sum_{\mathbf{y} \notin B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \\ &= \Pr_{\mathbf{e} \sim \text{BSC}_p} [wt(\mathbf{e}) > (p + \varepsilon')n] \\ &\leq e^{-(\varepsilon')^2 n/2}. \end{aligned} \quad (6.9) \quad (6.10)$$

In the above the equality follows from recalling that $\mathbf{y} = E(\mathbf{m}) + \mathbf{e}$ and (6.10) follows from the (additive) Chernoff bound (Theorem 3.1.10).³ Combining (6.8) and (6.10), we have

$$\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \leq \sum_{\mathbf{y} \in B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \cdot \mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}} + e^{-(\varepsilon')^2 n/2}.$$

In order to apply the probabilistic method (Section 3.2), we will analyze the expectation (over the random choice of E) of the decoding error probability, which by the upper bound above satisfies

³Let $\mathbf{e} = (X_1, \dots, X_n)$. By definition of BSC_p, each X_i is an independent binary r.v. with $\mathbb{E}[X_i] = p$. The bound then follows from Theorem 3.1.10 and noting that $wt(\mathbf{e}) = \sum_{i=1}^n X_i$.

$$\mathbb{E}_E \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right] \leq e^{-(\varepsilon')^2 n/2} + \sum_{\mathbf{y} \in B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr_{\mathbf{e} \sim \text{BSC}_p} [\mathbf{y}|E(\mathbf{m})] \cdot \mathbb{E}_E [\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}]. \quad (6.11)$$

In the above, we used linearity of expectation (Proposition 3.1.4) and the fact that the distributions on $\mathbf{e}$ and E are independent (and hence the term $\Pr_{\mathbf{e} \sim \text{BSC}_p} [\mathbf{y}|E(\mathbf{m})]$ is a fixed quantity for a random E).

Next, for a fixed received word $\mathbf{y}$ and the transmitted codeword $E(\mathbf{m})$ such that $\Delta(\mathbf{y}, E(\mathbf{m})) \leq (p + \varepsilon')n$ we estimate $\mathbb{E}_E [\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}]$. Since D is MLD, we have

$$\mathbb{E}_E [\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}] = \Pr_E [\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}} | E(\mathbf{m})] \leq \sum_{\mathbf{m}' \neq \mathbf{m}} \Pr [\Delta(E(\mathbf{m}'), \mathbf{y}) \leq \Delta(E(\mathbf{m}), \mathbf{y}) | E(\mathbf{m})], \quad (6.12)$$

where, in the above, “ $|E(\mathbf{m})$ ” is short for “being conditioned on $E(\mathbf{m})$ being transmitted” and the inequality follows from the union bound (Proposition 3.1.5) and the fact that D is MLD.

Noting that $\Delta(E(\mathbf{m}'), \mathbf{y}) \leq \Delta(E(\mathbf{m}), \mathbf{y}) \leq (p + \varepsilon')n$ (see Figure 6.6), by (6.12) we have

$$\begin{aligned} \mathbb{E}_E [\mathbb{1}_{D(\mathbf{y}) \neq \mathbf{m}}] &\leq \sum_{\mathbf{m}' \neq \mathbf{m}} \Pr [E(\mathbf{m}') \in B(\mathbf{y}, (p + \varepsilon')n) | E(\mathbf{m})] \\ &= \sum_{\mathbf{m}' \neq \mathbf{m}} \frac{|B(\mathbf{y}, (p + \varepsilon')n)|}{2^n} \end{aligned} \quad (6.13)$$

$$\leq \sum_{\mathbf{m}' \neq \mathbf{m}} \frac{2^{H(p + \varepsilon')n}}{2^n} \quad (6.14)$$

$$< 2^k \cdot 2^{-n(1 - H(p + \varepsilon'))} \leq 2^{n(1 - H(p + \varepsilon)) - n(1 - H(p + \varepsilon'))} \quad (6.15)$$

$$= 2^{-n(H(p + \varepsilon) - H(p + \varepsilon'))}. \quad (6.16)$$

In the above, (6.13) follows from the fact that the choice for $E(\mathbf{m}')$ is independent of $E(\mathbf{m})$. (6.14) follows from the upper bound on the volume of a Hamming ball (Proposition 3.3.3), while (6.15) follows from our choice of k .

Using (6.16) in (6.11), we get

$$\begin{aligned} \mathbb{E}_E \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right] &\leq e^{-(\varepsilon')^2 n/2} + 2^{-n(H(p + \varepsilon) - H(p + \varepsilon'))} \sum_{\mathbf{y} \in B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \\ &\leq e^{-(\varepsilon')^2 n/2} + 2^{-n(H(p + \varepsilon) - H(p + \varepsilon'))} \leq 2^{-\delta' n}, \end{aligned} \quad (6.17)$$

where the second inequality follows from the fact that

$$\sum_{\mathbf{y} \in B(E(\mathbf{m}), (p + \varepsilon')n)} \Pr[\mathbf{y}|E(\mathbf{m})] \leq \sum_{\mathbf{y} \in \{0,1\}^n} \Pr[\mathbf{y}|E(\mathbf{m})] = 1$$

and the last inequality follows for large enough n , say $\varepsilon' = \varepsilon/2$ and by picking $\delta' > 0$ to be small enough. (See Exercise 6.3.)

Thus, we have shown that for every $\mathbf{m}$ the average (over the choices of E) decoding error probability is small. However, we still need to show that the decoding error probability is exponentially small for *all* messages *simultaneously*. Towards this end, as the bound holds for each $\mathbf{m}$, we have

$$\mathbb{E}_{\mathbf{m}} \left[\mathbb{E}_E \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right] \right] \leq 2^{-\delta' n}.$$

The order of the summation in the expectation with respect to $\mathbf{m}$ and the summation in the expectation with respect to the choice of E can be switched (as the probability distributions are defined over different domains), resulting in the following expression:

$$\mathbb{E}_E \left[\mathbb{E}_{\mathbf{m}} \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right] \right] \leq 2^{-\delta' n}.$$

By the probabilistic method, there exists an encoding function E^* (and a corresponding decoding function D^*) such that

$$\mathbb{E}_{\mathbf{m}} \left[\Pr_{\mathbf{e} \sim \text{BSC}_p} [D^*(E^*(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \right] \leq 2^{-\delta' n}. \quad (6.18)$$

(6.18) implies that the *average* decoding error probability is exponentially small. However, recall we need to show that the *maximum* decoding error probability is small. To achieve such a result, we will throw away half of the messages, i.e. *expurgate* the code. In particular, we will order the messages in decreasing order of their decoding error probability and then drop the top half. We claim that the maximum decoding error probability for the remaining messages is $2 \cdot 2^{-\delta' n}$. Next, we present the details.

From Average to Worst-Case Decoding Error Probability. We begin with the following “averaging” argument.

Claim 6.3.3. *Let the messages be ordered as $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_{2^k}$ and define*

$$P_i = \Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}_i) + \mathbf{e}) \neq \mathbf{m}_i].$$

Assume that $P_1 \leq P_2 \leq \dots \leq P_{2^k}$ and (6.18) holds, then $P_{2^{k-1}} \leq 2 \cdot 2^{-\delta' n}$

Proof. By the definition of P_i ,

$$\begin{aligned} \frac{1}{2^k} \sum_{i=1}^{2^k} P_i &= \mathbb{E}_{\mathbf{m}} \Pr_{\mathbf{e} \sim \text{BSC}_p} [D(E(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \\ &\leq 2^{-\delta' n}, \end{aligned} \quad (6.19)$$

where (6.19) follows from (6.18). For the sake of contradiction assume that

$$P_{2^{k-1}} > 2 \cdot 2^{-\delta' n}. \quad (6.20)$$

So,

$$\frac{1}{2^k} \sum_{i=1}^{2^k} P_i \geq \frac{1}{2^k} \sum_{i=2^{k-1}+1}^{2^k} P_i \quad (6.21)$$

$$\begin{aligned} &> \frac{2 \cdot 2^{-\delta' n} \cdot 2^{k-1}}{2^k} \\ &> 2^{-\delta' n}, \end{aligned} \quad (6.22)$$

where (6.21) follows by dropping half the summands from the sum. (6.22) follows from (6.20) and the assumption on the sortedness of P_i . The proof is now complete by noting that (6.23) contradicts (6.19). $\square$

Thus, our final code will have $\mathbf{m}_1, \dots, \mathbf{m}_{2^{k-1}}$ as its messages and hence, has dimension $k' = k - 1$. Define $\delta = \delta' - \frac{1}{n}$. In the new code, maximum error probability is at most $2^{-\delta n}$. Also if we picked $k \leq \lfloor (1 - H(p + \epsilon)) n \rfloor + 1$, then $k' \leq \lfloor (1 - H(p + \epsilon)) n \rfloor$, as required. This completes the proof of Theorem 6.3.1.

We have shown that a random code can achieve capacity. However, we do not know of even a succinct representation of general codes. A natural question to ask is if random linear codes can achieve the capacity of BSC_p . The answer is yes: see Exercise 6.4.

For linear code, representation and encoding are efficient. But the proof does not give an explicit construction. Intuitively, it is clear that since Shannon's proof uses a random code, it does not present an ‘explicit’ construction. Below, we formally define what we mean by an explicit construction.

Definition 6.3.4. *A code C of block length n is called explicit if there exists a poly(n)-time algorithm that computes a succinct description of C given n . For linear codes, such a succinct description could be a generator matrix or a parity check matrix.*

We will also need the following stronger notion of an explicitness:

Definition 6.3.5. *A linear $[n, k]$ code C is called strongly explicit, if given any index pair $(i, j) \in [k] \times [n]$, there is a poly($\log n$) time algorithm that outputs $G_{i,j}$, where G is a generator matrix of C .*

Further, Shannon's proof uses MLD for which only exponential time implementations are known. Thus, the biggest question left unsolved by Shannon's work is the following.

Question 6.3.1. *Can we come up with an explicit construction of a code of rate $1 - H(p + \epsilon)$ with efficient decoding and encoding algorithms that achieves reliable communication over BSC_p ?*

As a baby step towards the resolution of the above question, one can ask the following question:

Question 6.3.2. Can we come up with an explicit construction with $R > 0$ and $p > 0$?

Note that the question above is similar to Question 1.8.2 in Hamming's world. See Exercise 6.13 for an affirmative answer.

6.4 Hamming vs. Shannon

As a brief interlude, let us compare the salient features of the works of Hamming and Shannon that we have seen so far:

QUALITATIVE COMPARISON	
HAMMING	SHANNON
Focus on codewords itself	Directly deals with encoding and decoding functions
Looked at explicit codes	Not explicit at all
Fundamental trade off: rate vs. distance (easier to get a handle on this)	Fundamental trade off: rate vs. error
Worst case errors	Stochastic errors

Intuitively achieving positive results in Hamming's world is harder than achieving positive results in Shannon's world. The reason is that the adversary in Shannon's world (e.g. BSC_p) is much weaker than the worst-case adversary in Hamming's world (for binary alphabet). We make this intuition precise as follows:

Proposition 6.4.1. Let $0 \leq p < \frac{1}{2}$ and $0 < \varepsilon \leq \frac{1}{2} - p$. If an algorithm A can handle $p + \varepsilon$ fraction of worst case errors, then it can be used for reliable communication over BSC_p .

Proof. By the additive Chernoff bound (Theorem 3.1.10), with probability $\geq 1 - e^{-\frac{\varepsilon^2 n}{2}}$, the fraction of errors in BSC_p is $\leq p + \varepsilon$. Then by assumption on A , it can be used to recover the transmitted message. $\square$

Note that the above result implies that one can have reliable transmission over BSC_p with every code of relative distance $2p + \varepsilon$ (for every $\varepsilon > 0$).

A much weaker converse of Proposition 6.4.1 is also true. More precisely, if the decoding error probability is exponentially small for the BSC, then the corresponding code must have constant relative distance (though this distance does not come even close to achieving the Gilbert-Varshamov bound). For more see Exercise 6.5.

6.5 Exercises

Exercise 6.1. Let (E, D) be a pair of encoder and decoder that allows for successful transmission over BSC_p for every $p \leq \frac{1}{2}$. Then there exists a pair (E', D') that allows for successful transmission

over $\text{BSC}_{p'}$ for every $p' > 1/2$. If D is (deterministic) polynomial time algorithm, then D' also has to be a (deterministic) polynomial time algorithm.

Exercise 6.2. Let (E, D) be a pair of encoder and decoder that allows for successful transmission over qSC_p for every $p \leq 1 - \frac{1}{q}$. Then there exists a pair (E', D') that allows for successful transmission over $\text{qSC}_{p'}$ for every $p' > 1 - \frac{1}{2}$. If D is polynomial time algorithm, then D' also has to be a polynomial time algorithm though D' can be a randomized algorithm even if D is deterministic.⁴

Exercise 6.3. Argue that in the positive part of Theorem 6.3.1, one can pick $\delta = \Theta(\varepsilon^2)$. That is, for $0 \leq p < 1/2$ and small enough ε , there exist codes of rate $1 - H(p) - \varepsilon$ and block length n that can be decoded with error probability at most $2^{-\Theta(\varepsilon^2)n}$ over BSC_p .

Exercise 6.4. Prove that there exists linear codes that achieve the BSC_p capacity. (Note that in Section 6.3 we argued that there exists not necessarily a linear code that achieves the capacity.)

Hint: Modify the argument in Section 6.3: in some sense the proof is easier.

Exercise 6.5. Prove that for communication on BSC_p , if an encoding function E achieves a maximum decoding error probability (taken over all messages) that is exponentially small, i.e., at most $2^{-\gamma n}$ for some $\gamma > 0$, then there exists a $\delta = \delta(\gamma, p) > 0$ such that the code defined by E has relative distance at least δ . In other words, good distance is necessary for exponentially small maximum decoding error probability.

Exercise 6.6. Prove that the capacity of the qSC_p is $1 - H_q(p)$.

Exercise 6.7. The binary erasure channel with erasure probability α has capacity $1 - \alpha$. In this problem, you will prove this result (and its generalization to larger alphabets) via a sequence of smaller results.

1. For positive integers $k \leq n$, show that less than a fraction q^{k-n} of the $k \times n$ matrices G over $\mathbb{F}_q$ fail to generate a linear code of block length n and dimension k . (Or equivalently, except with probability less than q^{k-n} , the rank of a random $k \times n$ matrix G over $\mathbb{F}_q$ is k .)

Hint: Try out the obvious greedy algorithm to construct a $k \times n$ matrix of rank k . You will see that you will have many choices every step: from this compute (a lower bound on) the number of full rank matrices that can be generated by this algorithm.

2. Consider the q -ary erasure channel with erasure probability α (qEC_α , for some α , $0 \leq \alpha \leq 1$): the input to this channel is a field element $x \in \mathbb{F}_q$, and the output is x with probability $1 - \alpha$, and an erasure '?' with probability α . For a linear code C generated by an $k \times n$ matrix G over $\mathbb{F}_q$, let $D : (\mathbb{F}_q \cup \{?\})^n \rightarrow C \cup \{\text{fail}\}$ be the following decoder:

$$D(\mathbf{y}) = \begin{cases} \mathbf{c} & \text{if } \mathbf{y} \text{ agrees with exactly one } \mathbf{c} \in C \text{ on the unerased entries in } \mathbb{F}_q \\ \text{fail} & \text{otherwise} \end{cases}$$

⁴A randomized D' means that given a received word $\mathbf{y}$ the algorithm can use random coins and the decoding error probability is over both the randomness from its internal coin tosses as well as the randomness from the channel.

For a set $J \subseteq \{1, 2, \dots, n\}$, let $P_{\text{err}}(G|J)$ be the probability (over the channel noise and choice of a random message) that D outputs fail conditioned on the erasures being indexed by J . Prove that the average value of $P_{\text{err}}(G|J)$ taken over all $G \in \mathbb{F}_q^{k \times n}$ is less than $q^{k-n+|J|}$.

- Let $P_{\text{err}}(G)$ be the decoding error probability of the decoder D for communication using the code generated by G on the $q\text{EC}_\alpha$. Show that when $k = Rn$ for $R < 1 - \alpha$, the average value of $P_{\text{err}}(G)$ over all $k \times n$ matrices G over $\mathbb{F}_q$ is exponentially small in n .
- Conclude that one can reliably communicate on the $q\text{EC}_\alpha$ at any rate less than $1 - \alpha$ using a linear code.

Exercise 6.8. Consider a binary channel whose input/output alphabet is $\{0, 1\}$, where a 0 is transmitted faithfully as a 0 (with probability 1), but a 1 is transmitted as a 0 with probability $\frac{1}{2}$ and a 1 with probability $1/2$. Compute the capacity of this channel.

Hint: This can be proved from scratch using only simple probabilistic facts already stated/used in the book.

Exercise 6.9. Argue that Reed-Solomon codes from Chapter 5 are strongly explicit codes (as in Definition 6.3.5).

Exercise 6.10. In this problem we will prove a special case of the source coding theorem. For every $0 \leq p \leq 1/2$, let $\mathcal{D}(p)$ be the distribution on $\{0, 1\}^n$, where each of the n bits are picked independently to be 1 with probability p and 0 otherwise. Argue that for every $\epsilon > 0$, strings from $\mathcal{D}(p)$ can be compressed with $H(p + \epsilon) \cdot n$ bits for large enough n .

More precisely show that for every constant $0 \leq p \leq 1/2$ and every $\epsilon > 0$, for large enough n there exists an encoding (or compression) function $E: \{0, 1\}^n \rightarrow \{0, 1\}^*$ and a decoding (or decompression) function $D: \{0, 1\}^* \rightarrow \{0, 1\}^n$ such that⁵

- For every $\mathbf{x} \in \{0, 1\}^n$, $D(E(\mathbf{x})) = \mathbf{x}$, and
- $\mathbb{E}_{\mathbf{x} \sim \mathcal{D}(p)}[|E(\mathbf{x})|] \leq H(p + \epsilon) \cdot n$, where we use $|E(\mathbf{x})|$ to denote the length of the string $E(\mathbf{x})$. In other words, the compression rate is $H(p + \epsilon)$.

Hint: Handle the “typical” strings from $\mathcal{D}$ and non-typical strings separately.

Exercise 6.11. Show that if there is a constructive solution to Shannon’s channel coding theorem with E being a linear map, then there is a constructive solution to Shannon’s source coding theorem in the case where the source produces a sequence of independent bits of bias p .

More precisely, let (E, D) be an encoding and decoding pairs that allows for reliable communication over BSC_p with exponentially small decoding error and E is a linear map with rate $1 - H(p) - \epsilon$. Then there exists a compressing and decompressing pair (E', D') that allows for compression rate $H(p) + \epsilon$ (where compression rate is as defined in part 2 in Exercise 6.10). The decompression algorithm D' can be randomized and is allowed exponentially small error probability (where the probability can be taken over both the internal randomness of D' and $\mathcal{D}(p)$).

⁵We use $\{0, 1\}^*$ to denote the set of all binary strings.

Finally if (E, D) are both polynomial time algorithms, then (E', D') have to be polynomial time algorithms too.

Exercise 6.12. Consider a Markovian source of bits, where the source consists of a 6-cycle with three successive vertices outputting 0, and three successive vertices outputting 1, with the probability of either going left (or right) from any vertex is exactly $1/2$. More precisely, consider a graph with six vertices $v_0, v_1, \dots, v_5$ such that there exists an edge $(v_i, v_{(i+1) \bmod 6})$ for every $0 \leq i \leq 5$. Further the vertices v_i for $0 \leq i < 3$ are labeled $\ell(v_i) = 0$ and vertices v_j for $3 \leq j < 6$ are labeled $\ell(v_j) = 1$. Strings are generated from this source as follows: one starts with some start vertex u_0 (which is one of the v_i ’s): i.e. the start state is u_0 . Any any point of time if the current state is u , then the source outputs $\ell(u)$. Then with probability $1/2$ the states moves to each of the two neighbors of u .

Compute the optimal compression rate of this source.

Hint: Compress “state diagram” to a minimum and then make some basic observations to compress the source information.

Exercise 6.13. Given codes C_1 and C_2 with encoding functions $E_1: \{0, 1\}^{k_1} \rightarrow \{0, 1\}^{n_1}$ and $E_2: \{0, 1\}^{k_2} \rightarrow \{0, 1\}^{n_2}$ let $E_1 \otimes E_2: \{0, 1\}^{k_1 \times k_2} \rightarrow \{0, 1\}^{n_1 \times n_2}$ be the encoding function obtained as follows: view a message $\mathbf{m}$ as a $k_1 \times k_2$ matrix. Encode the columns of $\mathbf{m}$ individually using the function E_1 to get an $n_1 \times k_2$ matrix $\mathbf{m}'$. Now encode the rows of $\mathbf{m}'$ individually using E_2 to get an $n_1 \times n_2$ matrix that is the final encoding under $E_1 \otimes E_2$ of $\mathbf{m}$. Let $C_1 \otimes C_2$ be the code associated with $E_1 \otimes E_2$ (recall Exercise 2.20).

For $i \geq 3$, let H_i denote the $[2^i - 1, 2^i - i - 1, 3]_2$ -Hamming code. Let $C_i = H_i \otimes C_{i-1}$ with $C_3 = H_3$ be a new family of codes.

- Give a lower bound on the relative minimum distance of C_i . Does it go to zero as $i \rightarrow \infty$?
- Give a lower bound on the rate of C_i . Does it go to zero as $i \rightarrow \infty$?
- Consider the following simple decoding algorithm for C_i : Decode the rows of the rec’d vector recursively using the decoding algorithm for C_{i-1} . Then decode each column according to the Hamming decoding algorithm (e.g. Algorithm 2.5.3). Let δ_i denote the probability of decoding error of this algorithm on the BSC_p . Show that there exists a $p > 0$ such that $\delta_i \rightarrow 0$ as $i \rightarrow \infty$.

Hint: First show that $\delta_i \leq 4^i \delta_{i-1}^2$.

Exercise 6.14. We consider the problem of determining the best possible rate of transmission on a stochastic memoryless channel with zero decoding error probability. Recall that a memoryless stochastic channel is specified by a transition matrix $\mathbf{M}$ s.t. $\mathbf{M}(x, y)$ denotes the probability of y being received if x was transmitted over the channel. Further, the noise acts independently on each transmitted symbol. Let $\mathbb{D}$ denote the input alphabet. Let $R(\mathbf{M})$ denote the best possible rate for a code C such that there exists a decoder D such that for every $\mathbf{c} \in C$, $\Pr[D(\mathbf{y}) \neq \mathbf{c}] = 0$, where $\mathbf{y}$

is picked according to the distribution induced by $\mathbf{M}$ when $\mathbf{c}$ is transmitted over the channel (i.e. the probability that $\mathbf{y}$ is a received word is exactly $\prod_{i=1}^n \mathbf{M}(c_i, y_i)$ where C has block length n). In this exercise we will derive an alternate characterization of $R(\mathbf{M})$.

We begin with some definitions related to graphs $\mathcal{G} = (V, E)$. An independent set S of $\mathcal{G}$ is a subset $S \subseteq V$ such that there is no edge contained in S , i.e. for every $u \neq v \in S$, $(u, v) \notin E$. For a given graph $\mathcal{G}$, we use $\alpha(\mathcal{G})$ to denote the size of largest independent set in $\mathcal{G}$. Further, given an integer $n \geq 1$, the n -fold product of $\mathcal{G}$, which we will denote by $\mathcal{G}^n$, is defined as follows: $\mathcal{G}^n = (V^n, E')$, where $((u_1, \dots, u_n), (v_1, \dots, v_n)) \in E'$ if and only if for every $i \in [n]$ either $u_i = v_i$ or $(u_i, v_i) \in E$.

Finally, define a confusion graph $\mathcal{G}_{\mathbf{M}} = (V, E)$ as follows. The set of vertices $V = \mathbb{D}$ and for every $x_1 \neq x_2 \in \mathbb{D}$, $(x, y) \in E$ if and only if there exists a y such that $\mathbf{M}(x_1, y) \neq 0$ and $\mathbf{M}(x_2, y) \neq 0$.

1. Prove that

$$R(\mathbf{M}) = \lim_{n \rightarrow \infty} \frac{1}{n} \cdot \log_{|\mathbb{D}|} (\alpha(\mathcal{G}_{\mathbf{M}}^n)).^6 \quad (6.24)$$

2. A clique cover for a graph $\mathcal{G} = (V, E)$ is a partition of the vertices $V = \{V_1, \dots, V_c\}$ (i.e. V_i and V_j are disjoint for every $i \neq j \in [c]$ and $\cup_i V_i = V$) such that the graph induced on V_i is a complete graph (i.e. for every $i \in [c]$ and $x \neq y \in V_i$, we have $(x, y) \in E$). We call c to be the size of the clique cover $V_1, \dots, V_c$. Finally, define $\nu(\mathcal{G})$ to be the size of the smallest clique cover for $\mathcal{G}$. Argue that

$$\alpha(\mathcal{G})^n \leq \alpha(\mathcal{G}^n) \leq \nu(\mathcal{G})^n.$$

Conclude that

$$\log_{|\mathbb{D}|} \alpha(\mathcal{G}) \leq R(\mathbf{M}) \leq \log_{|\mathbb{D}|} \nu(\mathcal{G}). \quad (6.25)$$

3. Consider any transition matrix $\mathbf{M}$ such that the corresponding graph $\mathcal{G}_4 = \mathcal{G}_{\mathbf{M}}$ is a 4-cycle (i.e. the graph $(\{0, 1, 2, 3\}, E)$ where $(i, i+1 \bmod 4) \in E$ for every $0 \leq i \leq 3$). Using part 2 or otherwise, argue that $R(\mathbf{M}) = \frac{1}{2}$.

4. Consider any transition matrix $\mathbf{M}$ such that the corresponding graph $\mathcal{G}_5 = \mathcal{G}_{\mathbf{M}}$ is a 5-cycle (i.e. the graph $(\{0, 1, 2, 3, 4\}, E)$ where $(i, i+1 \bmod 5) \in E$ for every $0 \leq i \leq 4$). Using part 2 or otherwise, argue that $R(\mathbf{M}) \geq \frac{1}{2} \cdot \log_5 5$. (This lower bound is known to be tight: see Section 6.6 for more.)

6.6 Bibliographic Notes

Shannon's results that were discussed in this chapter appeared in his seminal 1948 paper [138]. All the channels mentioned in this chapter were considered by Shannon except for the BEC channel, which was introduced by Elias.

The proof method used to prove Shannon's result for BSC_p has its own name—“random coding with expurgation.”

Elias [47] answered Question 6.3.2 (the argument in Exercise 6.13 is due to him).

⁶In literature, $R(\mathbf{M})$ is defined with $\log_{|\mathbb{D}|}$ replaced by $\log_2$. We used the definition in (6.24) to be consistent with our definition of capacity of a noisy channel. See Section 6.6 for more.

Chapter 7

Bridging the Gap Between Shannon and Hamming: List Decoding

In Section 6.4, we made a qualitative comparison between Hamming's and Shannon's world. We start this chapter by making a more quantitative comparison between the two threads of coding theory. In Section 7.2 we introduce the notion of list decoding, which potentially allows us to go beyond the (quantitative) results of Hamming and approach those of Shannon's. Then in Section 7.3, we show how list decoding allows us to go beyond half the distance bound for every code. Section 7.4 proves the optimal trade-off between rate and fraction of correctable errors via list decoding. Finally, in Section 7.5, we formalize why list decoding could be a useful primitive in practical communication setups.

7.1 Hamming versus Shannon: part II

Let us compare Hamming and Shannon theories in terms of the asymptotic bounds we have seen so far (recall rate $R = \frac{k}{n}$ and relative distance $\delta = \frac{d}{n}$).

- Hamming theory: Can correct $\leq \frac{\delta}{2}$ fraction of worse case errors for codes of relative distance δ . By the Singleton bound (Theorem 4.3.1),

$$\delta \leq 1 - R,$$

which by Proposition 1.4.2 implies that p , the fraction of errors that can be corrected, has to satisfy

$$p \leq \frac{1 - R}{2}.$$

If we don't care about efficient algorithms, the above can be achieved by any code approaching the Singleton Bound, and in particular the Reed-Solomon codes. In fact the above limit on p can even be achieved via efficient decoding algorithms, e.g., for Reed-Solomon codes (we will see this in Chapter 12). However, the focus of this chapter is the unachievable regime, i.e., when $p > (1 - R)/2$, which is lower than in the Shannon theory setting, as we explain below.

- Shannon theory: Here the main result shows that for the qSC_p channel, for every $0 \leq p < 1 - 1/q$ we can have reliable communication with $R < 1 - H_q(p)$. It can be shown that

1. $1 - H_q(p) \leq 1 - p$ (this is left as an exercise); and
2. $1 - H_q(p) \geq 1 - p - \epsilon$, for large enough q — in particular for $q = 2^{\Omega(1/\epsilon)}$ (Proposition 3.3.4).

Thus, for every $p < 1 - R$, we can have reliable communication with on qSC_p for large enough q (in particular for $q = 2^{\Omega(1/(1-R-p))}$).

Thus the limits on the fraction of errors that one can correct in Shannon's world vs. in Hamming's world have a large gap between them: One can correct twice as many errors in Shannon's world. One natural question to ask is whether we can somehow “bridge” this gap without completely assuming a benign stochastic error model.

In this chapter we will introduce a relaxed notion of error-recovery that will allow us to bridge the models. On the one hand error-correction limits with this notion of recovery will match in Shannon's and Hamming's world. On the other hand we will show that with stochastic error, the notion of recovery does not need any relaxations, so when applied in the Shannon world this recovers Shannon's limits, thus giving the right bridge between Shannon's world and Hamming's.

We now turn to this relaxed notion of decoding, which is called “list-decoding”. Recall that the standard notion of decoding we have worked with so far expects the decoder to output a single candidate for the message that the sender sent. We refer to this notion as “unique” decoding in this chapter. To motivate the notion of list-decoding, we re-visit the bad example for the unique decoding from Figure 1.3 and consider an extension as shown in Figure 7.1.

Recall that y and the codewords c_1 and c_4 form the bad example for unique decoding that we have already seen before. Recall that for this particular received word we cannot do error recovery by unique decoding since there are two codewords c_1 and c_4 that have the same distance $\frac{\delta}{2}$ from vector y . On the other hand, the received word z has a unique codeword c_4 with distance $p > \frac{\delta}{2}$. However, unique decoding does not allow for error recovery from z . This is because by definition of unique decoding, the decoder must be able to recover from *every* error pattern (with a given Hamming weight bound). Thus, by Proposition 1.4.2, the decoded codeword cannot have relative Hamming distance larger than $\delta/2$ from the received word. In this example, because of the received word y , unique decoding gives up on the opportunity to decode z .

Let us consider the example in Figure 7.1 for the binary case. It can be shown that the number of vectors in dotted lines is insignificant compared to the volume of shaded area (for large enough block length of the code). The volume of all Hamming balls of radius $\frac{\delta}{2}$ around all the 2^k codewords is roughly equal to:

$$2^k 2^{nH(\frac{\delta}{2})},$$

which implies that the volume of the shaded area (without the dotted lines) is approximately equal to:

$$2^n - 2^k 2^{nH(\frac{\delta}{2})}.$$

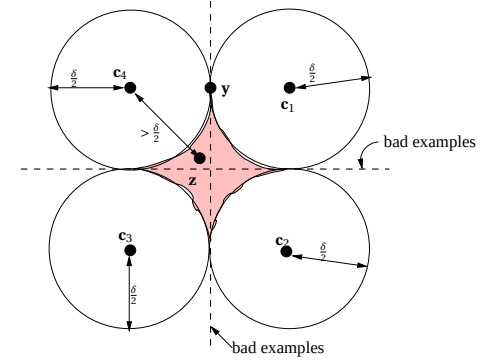


Figure 7.1: In this example, vectors are embedded into Euclidean space such that the Euclidean distance between two mapped points is the same as the Hamming distance between vectors. c_1, c_2, c_3, c_4 are codewords. The dotted lines contain the “bad examples,” that is, the received words for which unique decoding is not possible.

In other words, the volume when expressed as a fraction of the volume of the ambient space is roughly:

$$1 - 2^{-n(1-H(\frac{\delta}{2})-R)}, \quad (7.1)$$

where $k = Rn$. Recall that by the Hamming bound (Theorem 1.3) $R \leq 1 - H(\frac{\delta}{2})$. Now, if $R < 1 - H(\frac{\delta}{2})$ then second term of (7.1) is very small. Therefore, the number of vectors in the shaded area (without the bad examples) is almost all of the ambient space. Note that by the stringent condition on unique decoding, none of these received words can be decoded (even though for such received words there is a unique closest codeword). Thus, in order to be able to decode such received vectors, we need to relax the notion of unique decoding. This leads us to the notion of *list decoding* which we define next.

7.2 List Decoding

In notion of decoding called *list decoding* the decoder is allowed to output a short list of answers. The notion is parametrized by two parameters: ρ , the fraction of errors we wish to correct, and L , an upper bound on the size of the list output by the decoder. We now formally define (the combinatorial version of) list decoding:

Definition 7.2.1. Given $0 \leq \rho \leq 1, L \geq 1$, a code $C \subseteq \Sigma^n$ is (ρ, L) -list decodable if for every received word $y \in \Sigma^n$,

$$|\{c \in C | \Delta(y, c) \leq \rho n\}| \leq L$$

As opposed to unique decoding, a list decoding algorithm works as follows. Given an error parameter ρ , a code C and a received word $\mathbf{y}$, a list-decoding algorithm should output *all* codewords in C that are within (relative) Hamming distance ρ from $\mathbf{y}$. Note that if the fraction of errors that occurred during transmission is at most ρ then the transmitted codeword is *guaranteed* to be in the output list. Further, note that if C is (ρ, L) -list decodable, then the algorithm will always output at most L codewords for a given received word.

We now turn to the question of what is the right choice of L . Setting $L = 1$ would return us to the notion of unique decoding, so we clearly want to allow some $L > 1$. On the other hand allowing L to be exponential in the length of the code, e.g., $L = q^n$, makes the notion trivial. To get meaningful answers we need to restrict L and we. One way to restrict L is to set it to be an arbitrarily large *constant*. So here, given an infinite family of codes $\mathcal{C}$, we ask the question: What is the limit of ρ as $L \rightarrow \infty$ such that all sufficiently long codes $C \in \mathcal{C}$ are (ρ, L) -list-decodable. We refer to this as the list-decodability of $\mathcal{C}$ with constant-sized lists.

Keeping our algorithmic goals in mind, we could also settle for a weaker limit. Note that for an efficient list-decoding algorithm, L should be a polynomial in the block length n (as otherwise, the algorithm will have to output a super-polynomial number of codewords and hence, cannot have a polynomial running time). Thus we can also study the limit of ρ over the space of all polynomials L such that a given code is (ρ, L) -list-decodable. This would be the list-decodability of $\mathcal{C}$ with polynomial-sized lists.

For most natural families of codes, the two notions coincide (though it is certainly possible to create codes where the notions disagree— see Exercises 7.1 and 7.2). In this text we typically focus on the list-decodability with polynomial-sized lists.

Utility of List-decoding. Note that in the communication setup, we need to recover the transmitted message. In such a scenario, outputting a list might not be useful. There are two ways to get around this “problem”:

1. Declare a decoding error if list size > 1 . Note that this generalizes unique decoding (as when the number of errors is at most half the distance of the code then there is a unique codeword and hence, the list size will be at most one). However, the gain over unique decoding would be substantial only if for most error patterns (of weight significantly more than half the distance of the code) the output list size is at most one. Fortunately, it can be shown that:
 - For random codes, for most error patterns, the list size is at most one with high probability. In other words, for *most* codes, we can hope to see a gain over unique decoding. The proof of this fact follows from Shannon’s proof for the capacity for q SC (see Exercise 7.13).
 - In Section 7.5, we show that the above behavior is in fact general: i.e. for *any* code (over a large enough alphabet) it is true that with high probability, for most error patterns, the list size is at most one.

Thus, using this option to deal with multiple answers, we still deal with worse case errors but can correct more error patterns than unique decoding.

2. If the decoder has access to some side information, then it can use that to prune the list. Informally, if the worst-case list size is L , then the amount of extra information one needs is $O(\log L)$. This will effectively decrease¹ the dimension of the code by $O(\log L)$, so if L is small enough, this will have a negligible effect on the rate of the code. There are also applications (especially in complexity theory) where one does not really care about the rate being the best possible.

Limits of List-decoding Recall that Proposition 1.4.2 implies that $\delta/2$ is the maximum fraction of errors correctable with unique decoding. Since list decoding is a relaxation of unique decoding, it is natural to wonder

Question 7.2.1. *Given a code of distance δ , can we correct more than $\delta/2$ fraction of errors using list decoding?*

We stress that we study this question in the context of infinite families of codes of distance δ . And the goal is to determine if there is a constant $\rho > \delta/2$ and L such that *every sufficiently long code* in the family is (ρ, L) -list-decodable. In particular, note that the intuition from Figure 7.1 states that the answer to Question 7.2.1 should be yes.

Turning to the other direction one can ask if list-decoding can correct many more errors than unique decoding. But trying to relate list-decoding radius to the distance of a code does not make sense. (See Exercise 7.3.) The correct way to ask this question is by fixing the rate of the code (rather than the distance) and then asking about the fraction of errors that can be corrected using list-decoding.

Question 7.2.2. *Given a code of rate R , what is the maximum fraction of errors correctable using list decoding?*

We study these questions in the rest of this chapter.

7.3 The Johnson Bound

In this section, we will answer Question 7.2.1 in the affirmative by giving a bound due to S. Johnson. To set the context up again, recall that Proposition 1.4.2 implies that any code with relative distance δ is $(\frac{\delta}{2}, 1)$ -list decodable.

Notice that if we can show a code for some $e > \lfloor \frac{d-1}{2} \rfloor$ is $(\frac{e}{n}, n^{O(1)})$ -list decodable, then it is *potentially* possible to list decode the code up to e errors in polynomial time. By proving the

¹Note that side information effectively means that not all possible vectors are valid messages.

Johnson bound, we will show that this is indeed the case for every family of codes of relative distance $\delta > 0$ (and thus, answer Question 7.2.1 in the affirmative).

Theorem 7.3.1 (Johnson Bound). *Let $C \subseteq [q]^n$ be a code of distance d . If $\rho < J_q\left(\frac{d}{n}\right)$, then C is a (ρ, qdn) -list decodable code, where the function $J_q(\delta)$ is defined as*

$$J_q(\delta) = \left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{q\delta}{q-1}}\right).$$

We prove the theorem here only for the binary case, i.e., $q = 2$. The proof for the general case is left as an exercise — see Exercises 7.7 and 7.5. We also note that the bound on the list-size given here is a polynomial in n thus satisfying our weaker notion of list-decodability. However Exercise 7.8 also shows that the same limit is achieved for constant list-sizes.

Proof of Theorem 7.3.1 (for $q = 2$). We will prove the theorem using a proof technique called double counting. The main idea is to get both an upper and lower bound on the same quantity by counting it in two different ways. These bounds then imply an inequality, and we will derive our desired bound from this inequality.

For notational convenience, define

$$e = \rho n.$$

We have to show that for every binary code $C \subseteq \{0, 1\}^n$ with distance d (i.e. for every $\mathbf{c}_1 \neq \mathbf{c}_2 \in C$, $\Delta(\mathbf{c}_1, \mathbf{c}_2) \geq d$) and every $\mathbf{y} \in \{0, 1\}^n$,

$$|B(\mathbf{y}, e) \cap C| \leq 2dn.$$

Fix arbitrary C and $\mathbf{y}$. Let $\mathbf{c}_1, \dots, \mathbf{c}_M \in B(\mathbf{y}, e)$. We need to show that $M \leq 2dn$. Define $\mathbf{c}'_i = \mathbf{c}_i - \mathbf{y}$ for $1 \leq i \leq M$. Further, define

$$S = \sum_{i < j} \Delta(\mathbf{c}'_i, \mathbf{c}'_j).$$

We will prove both an upper and a lower bound on S , from which we will extract the required upper bound on M . Towards that end, note that we have the following:

- (i) $wt(\mathbf{c}'_i) \leq e$ for $1 \leq i \leq M$ because $\mathbf{c}_i \in B(\mathbf{y}, e)$.
- (ii) $\Delta(\mathbf{c}'_i, \mathbf{c}'_j) \geq d$ for every $i \neq j$ because $\Delta(\mathbf{c}_i, \mathbf{c}_j) \geq d$.

From (ii) we have

$$S \geq \binom{M}{2} d \quad (7.2)$$

Consider the $n \times M$ matrix $(\mathbf{c}'_1^T, \dots, \mathbf{c}'_M^T)$. Define m_i as the number of 1's in the i -th row for $1 \leq i \leq n$. Then the i -th row of the matrix contributes the value $m_i(M - m_i)$ to S because this is the number of 0-1 pairs in that row. (Note that each such pair contributes one to S .) This implies that

$$S = \sum_{i=1}^n m_i(M - m_i). \quad (7.3)$$

Define $\bar{e}$ such that

$$\bar{e}M = \sum_{i=1}^n m_i.$$

Note that

$$\sum_{i=1}^n m_i = \sum_{j=1}^M wt(\mathbf{c}_j) \leq eM,$$

where the inequality follows from (i) above. Thus, we have

$$\bar{e} \leq e.$$

Using the ('square' of) Cauchy-Schwarz inequality (Lemma B.1.6) i.e.,

$$\langle \mathbf{x}, \mathbf{z} \rangle^2 \leq \|\mathbf{x}\|^2 \cdot \|\mathbf{z}\|^2$$

for $\mathbf{x}, \mathbf{z} \in \mathbb{R}^n$ and by taking $\mathbf{x} = (m_1, \dots, m_n)$, $\mathbf{z} = (1/n, \dots, 1/n)$, we have

$$\left(\frac{\sum_{i=1}^n m_i}{n}\right)^2 \leq \left(\sum_{i=1}^n m_i^2\right) \frac{1}{n}. \quad (7.4)$$

Thus, from (7.3)

$$S = \sum_{i=1}^n m_i(M - m_i) = M^2 \bar{e} - \sum_{i=1}^n m_i^2 \leq M^2 \bar{e} - \frac{(M\bar{e})^2}{n} = M^2 \left(\bar{e} - \frac{\bar{e}^2}{n}\right), \quad (7.5)$$

where the inequality follows from (7.4). By (7.2) and (7.5),

$$M^2 \left(\bar{e} - \frac{\bar{e}^2}{n}\right) \geq \frac{M(M-1)}{2} d,$$

which implies that

$$M \leq \frac{dn}{dn - 2n\bar{e} + 2\bar{e}^2} = \frac{2dn}{2dn - n^2 + n^2 - 4n\bar{e} + 4\bar{e}^2} = \frac{2dn}{(n - 2\bar{e})^2 - n(n - 2d)} \leq \frac{2dn}{(n - 2e)^2 - n(n - 2d)}, \quad (7.6)$$

where the last inequality follows from the fact that $\bar{e} \leq e$. Then from definition of $J_2(\cdot)$, we get

$$\frac{e}{n} < \frac{1}{2} \left(1 - \sqrt{1 - \frac{2d}{n}}\right),$$

we get

$$n - 2e > \sqrt{n(n - 2d)}.$$

In other words

$$(n - 2e)^2 > n(n - 2d).$$

Thus, $(n - 2e)^2 - n(n - 2d) \geq 1$ because n, e are all integers and therefore, from (7.6), we have $M \leq 2dn$ as desired. $\square$

Next, we prove the following property of the function $J_q(\cdot)$, which along with the Johnson bound answers Question 7.2.1 in the affirmative.

Lemma 7.3.2. *Let $q \geq 2$ be an integer and let $0 \leq x \leq 1 - \frac{1}{q}$. Then the following inequalities hold:*

$$J_q(x) \geq 1 - \sqrt{1-x} \geq \frac{x}{2},$$

where the second inequality is strict for $x > 0$.

Proof. We start with by proving the inequality

$$\left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{xq}{q-1}}\right) \geq 1 - \sqrt{1-x}.$$

Indeed, both the LHS and RHS of the inequality are zero at $x = 0$. Further, it can be verified that the derivatives of the LHS and RHS are $\frac{1}{2\sqrt{1-\frac{xq}{q-1}}}$ and $\frac{1}{2\sqrt{1-x}}$ respectively. The former is always larger than the latter quantity. This implies that the LHS increases more rapidly than the RHS, which in turn proves the required inequality.

The second inequality follows from the subsequent relations. As $x \geq 0$,

$$1 - x + \frac{x^2}{4} \geq 1 - x,$$

which implies that

$$\left(1 - \frac{x}{2}\right)^2 \geq 1 - x,$$

which in turn implies the required inequality. (Note that the two inequalities above are strict for $x > 0$, which implies that $1 - \sqrt{1-x} > x/2$ for every $x > 0$, as desired.) $\square$

Theorem 7.3.1 and Lemma 7.3.2 imply that for every $\delta > 0$ list decoding can correct a strictly larger fraction of errors than unique decoding in polynomial time, as long as q is at most some polynomial in n (which will be true of all the codes that we discuss in this book). This answers Question 7.2.1 in the affirmative. See Figure 7.2 for an illustration of the gap between the Johnson bound and the unique decoding bound.

Theorem 7.3.1 and Lemma 7.3.2 also implies the following “alphabet-free” version of the Johnson bound.

Theorem 7.3.3 (Alphabet-Free Johnson Bound). *For every q -ary code with block length n and distance d , if $e \leq n - \sqrt{n(n-d)}$, then the code is $(e/n, qnd)$ -list decodable.*

See Exercise 7.5 for an alternate proof of the above result.

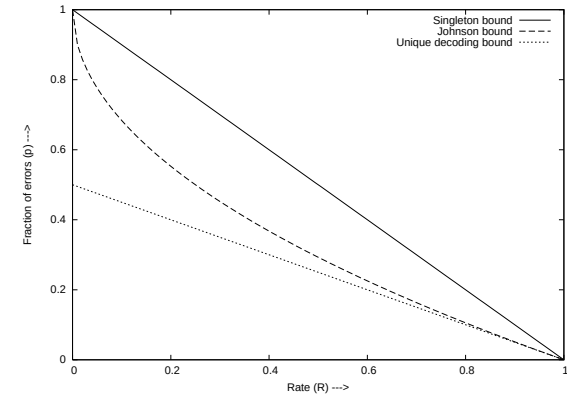


Figure 7.2: The trade-off between rate R and the fraction of errors that can be corrected. $1 - \sqrt{R}$ is the trade-off implied by the Johnson bound. The bound for unique decoding is $(1 - R)/2$ while $1 - R$ is the Singleton bound (and the list decoding capacity for codes over large alphabets).

The answer is yes in the sense that there exist codes, even linear codes, with relative distance δ such that we can find Hamming ball of radius larger than $J_q(\delta)$ with super-polynomially many codewords. (See Exercise 7.9.) On the other hand, in the next section, we will show that, in some sense, it is not tight.

7.4 List-Decoding Capacity

In the previous section, we saw what one can achieve with list decoding in terms of distance of a code. In this section, we return to Question 7.2.2. In particular, we will consider the trade-off between rate and the fraction of errors correctable by list decoding. Unlike the case of unique decoding (but like the case of BSC_p), we will be able to prove an optimal trade-off.

Next, we will prove the following result regarding the optimal trade-off between rate of a code and the fraction of errors that can be corrected via list decoding.

Theorem 7.4.1. *Let $q \geq 2$, $0 \leq \rho < 1 - \frac{1}{q}$, and $\varepsilon > 0$ be a small enough real. Then the following holds for codes of large enough block length n :*

- (i) *If $R \leq 1 - H_q(\rho) - \varepsilon$, then there exists a $(\rho, O(\frac{1}{\varepsilon}))$ -list decodable code.*
- (ii) *If $R \geq 1 - H_q(\rho) + \varepsilon$, every (ρ, L) -list decodable code has $L \geq q^{\Omega(\varepsilon n)}$.*

Question 7.3.1. *Is the Johnson bound tight?*

Thus, the *List-decoding capacity*² is $1 - H_q(\rho)$ (where ρ is the fraction of errors). Further, this fully answers Question 7.2.2. Finally, note that this exactly matches capacity for qSC_ρ and hence, list decoding can be seen as a bridge between Shannon's world and Hamming's world. The remarkable aspect of this result is that we bridge the gap between these worlds by allowing the decoder to output at most $O(1/\epsilon)$ many codewords.

7.4.1 Proof of Theorem 7.4.1

We begin with the basic idea behind the proof of part (i) of the theorem.

As in Shannon's proof for capacity of BSC, we will use the probabilistic method (Section 3.2). In particular, we will pick a random code and show that it satisfies the required property with non-zero probability. In fact, we will show that a random code is (ρ, L) -list decodable with high probability as long as:

$$R \leq 1 - H_q(\rho) - \frac{1}{L}$$

The analysis will proceed by proving that probability of a "bad event" is small. "Bad event" means there exist messages $\mathbf{m}_0, \mathbf{m}_1, \dots, \mathbf{m}_L \in [q]^{Rn}$ and a received code $\mathbf{y} \in [q]^n$ such that:

$$\Delta(C(\mathbf{m}_i), \mathbf{y}) \leq \rho n, \text{ for every } 0 \leq i \leq L.$$

Note that if a bad event occurs, then the code is not (ρ, L) -list decodable. The probability of the occurrence of any bad event will then be calculated by an application of the union bound.

Next, we restate Theorem 7.4.1 and prove a stronger version of part (i). (Note that $L = \lceil \frac{1}{\epsilon} \rceil$ in Theorem 7.4.2 implies Theorem 7.4.1.)

Theorem 7.4.2 (List-Decoding Capacity). *Let $q \geq 2$ be an integer, and $0 < \rho < 1 - \frac{1}{q}$ be a real number.*

(i) *Let $L \geq 1$ be an integer, then there exists an (ρ, L) -list decodable code with rate*

$$R \leq 1 - H_q(\rho) - \frac{1}{L}$$

(ii) *For every (ρ, L) code of rate $1 - H_q(\rho) + \epsilon$, it is necessary that $L \geq 2^{\Omega(\epsilon n)}$.*

Proof. We start with the proof of (i). Pick a code C at random where

$$|C| = q^k, \text{ and } k \leq \left(1 - H_q(\rho) - \frac{1}{L}\right)n.$$

That is, as in Shannon's proof, for every message $\mathbf{m}$, pick $C(\mathbf{m})$ uniformly and independently at random from $[q]^n$.

²Actually the phrase should be something like "list-decoding capacity of the q -ary worst-case error channel" as the capacity is a property of the channel. However, in the interest of brevity we will only use the term list-decoding capacity.

Given $\mathbf{y} \in [q]^n$, and $\mathbf{m}_0, \dots, \mathbf{m}_L \in [q]^k$, the tuple $(\mathbf{y}, \mathbf{m}_0, \dots, \mathbf{m}_L)$ defines a *bad event* if

$$C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n), \text{ for all } 0 \leq i \leq L.$$

Note that a code is (ρ, L) -list decodable if and only if there does not exist any bad event.

Fix $\mathbf{y} \in [q]^n$ and $\mathbf{m}_0, \dots, \mathbf{m}_L \in [q]^k$. Note that for fixed i , by the choice of C , we have:

$$\Pr[C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n)] = \frac{\text{Vol}_q(\rho n, n)}{q^n} \leq q^{-n(1-H_q(\rho))}, \quad (7.7)$$

where the inequality follows from the upper bound on the volume of a Hamming ball (Proposition 3.3.3). Now the probability of a bad event given $(\mathbf{y}, \mathbf{m}_0, \dots, \mathbf{m}_L)$ is

$$\Pr\left[\bigwedge_{i=0}^L C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n)\right] = \prod_{i=0}^L \Pr[C(\mathbf{m}_i) \in B(\mathbf{y}, \rho n)] \leq \left(q^{-n(1-H_q(\rho))}\right)^{L+1} = q^{-n(L+1)(1-H_q(\rho))}, \quad (7.8)$$

where the first equality follows from the fact that the random choices of codewords for distinct messages are independent (and thus, the events $C(\mathbf{m}_i) \in B(\mathbf{y})$ are independent for each $0 \leq i \leq L$) and the inequality follows from (7.7). Then,

$$\Pr[\text{There is a bad event}] \leq q^n \binom{q^k}{L+1} q^{-n(L+1)(1-H_q(\rho))} \quad (7.9)$$

$$\leq q^n q^{Rn(L+1)} q^{-n(L+1)(1-H_q(\rho))} \quad (7.10)$$

$$= q^{-n(L+1)[1-H_q(\rho) - \frac{1}{L+1} - R]}$$

$$\leq q^{-n(L+1)[1-H_q(\rho) - \frac{1}{L+1} - 1 + H_q(\rho) + \frac{1}{L}]} \quad (7.11)$$

$$= q^{-\frac{n}{L}}$$

$$< 1$$

In the above, (7.9) follows by the union bound (Lemma 3.1.5) with (7.8) and by counting the number of $\mathbf{y}$'s (which is q^n), and the number of $L+1$ tuples (which is $\binom{q^k}{L+1}$). (7.10) follows from the fact that $\binom{a}{b} \leq a^b$ and $k = Rn$. (7.11) follows by assumption $R \leq 1 - H_q(\rho) - \frac{1}{L}$. The rest of the steps follow from rearranging and canceling the terms. Therefore, by the probabilistic method, there exists C such that it is (ρ, L) -list decodable. In fact the argument shows that the probability of a random code not being (ρ, L) goes (exponentially fast) to 0 as $n \rightarrow \infty$, i.e. the argument shows that *most* codes have the list decodability property.

Now we turn to the proof of part (ii). For this part, we need to show the existence of a $\mathbf{y} \in [q]^n$ such that $|C \cap B(\mathbf{y}, \rho n)|$ is exponentially large for every C of rate $R \geq 1 - H_q(\rho) + \epsilon$. We will again use the probabilistic method to prove this result.

Pick $\mathbf{y} \in [q]^n$ uniformly at random. Fix $\mathbf{c} \in C$. (Note that unlike the proof of the part (i), here $\mathbf{c}$ is fixed and $\mathbf{y}$ is random.) Then

$$\begin{aligned} \Pr[\mathbf{c} \in B(\mathbf{y}, \rho n)] &= \Pr[\mathbf{y} \in B(\mathbf{c}, \rho n)] \\ &= \frac{\text{Vol}_q(\rho n, n)}{q^n} \end{aligned} \quad (7.12)$$

$$\geq q^{-n(1-H_q(\rho)) - o(n)}, \quad (7.13)$$

where (7.12) follows from the fact that $\mathbf{y}$ is chosen uniformly at random from $[q]^n$ and (7.13) follows by the lower bound on the volume of the Hamming ball (Proposition 3.3.3).

We have

$$E[|C \cap B(\mathbf{y}, \rho n)|] = \sum_{\mathbf{c} \in C} E[\mathbb{1}_{\mathbf{c} \in B(\mathbf{y}, \rho n)}] \quad (7.14)$$

$$= \sum_{\mathbf{c} \in C} \Pr[\mathbf{c} \in B(\mathbf{y}, \rho n)] \geq \sum_{\mathbf{c} \in C} q^{-n(1-H_q(\rho)+o(n))} \quad (7.15)$$

$$= |C| \cdot q^{-n(1-H_q(\rho)+o(n))} \quad (7.16)$$

$$= q^{n(R-1+H_q(\rho)-o(1))} \geq q^{\Omega(\epsilon n)}. \quad (7.17)$$

In the above, (7.14) follows by the linearity of expectation (Proposition 3.1.4), (7.15) follows from (7.13), and (7.17) follows by choice of R . Hence, by the probabilistic method, there exists $\mathbf{y}$ such that $|B(\mathbf{y}, \rho n) \cap C|$ is $q^{\Omega(n)}$, as desired. $\square$

The above proof can be modified to work for random linear codes. (See Exercise 7.10.)

We now return to Question 7.3.1. Note that by the Singleton bound, the Johnson bound implies that for every code one can hope to list decode from about $\rho \leq 1 - \sqrt{R}$ fraction of errors. However, this trade-off between ρ and R is not tight. Note that Lemma 3.3.4 along with Theorem 7.4.1 imply that for large q , the list decoding capacity is $1 - R > 1 - \sqrt{R}$. Figure 7.2 plots and compares the relevant trade-offs.

Finally, we have shown that the list decoding capacity is $1 - H_q(\rho)$. However, we showed the existence of a code that achieves the capacity by the probabilistic method. This then raises the following question:

Question 7.4.1. *Do there exist explicit codes that achieve list decoding capacity?*

Also the only list decoding algorithm that we have seen so far is the brute force algorithm that checks every codeword to see if they need to be output. This also leads to the follow-up question

Question 7.4.2. *Can we achieve list decoding capacity with efficient list decoding algorithms?*

A more modest goal related to the above would be the following:

Question 7.4.3. *Can we design an efficient list decoding algorithm that can achieve the Johnson bound? In particular, can we efficiently list decode a code of rate R from $1 - \sqrt{R}$ fraction of errors?*

We will tackle the above questions later on in the book (see Chapters 12 and 17. Roughly, Questions 7.4.1 and 7.4.2 above are open for small alphabets (e.g., $q = 2, 3$ etc.) However if we allow q to be polynomially large in n we will show in Chapter 17 explicit codes with efficient list-decoding algorithms getting arbitrarily close to the list-decoding capacity. We will also indicate how to get explicit codes over “large” constant sized alphabets getting ϵ close to capacity with the caveat that the alphabet size will grow as $\epsilon \rightarrow 0$.

7.5 List Decoding from Random Errors

Finally we turn to the question of the utility of the notion of list-decoding. In particular we study the the typical list-size one can expect in a ball of radius ρn around a received word when the channel injects roughly ρ random errors and the code used in the transmission has distance $\delta > \rho$.

We study this by formalizing the intuition we developed from Figure 7.1. In particular, recall that we had informally argued that for most error patterns we can correct from $\rho > \delta/2$ fraction of errors (Proposition 1.4.2). The Johnson bound (Theorem 7.3.1) tells us that one can indeed correct beyond $\delta/2$ fraction of errors. However, there are two shortcomings. The first is that the Johnson bounds tells us that the output list size is qdn but it does not necessarily imply that for most error patterns, there is a unique nearby codeword (i.e. one can uniquely recover the transmitted codeword). In other words, Johnson bound is a “true” list decoding result and tells us nothing about the behavior of codes on the “average.” The second aspect is that the Johnson bound holds for up to $\rho \leq 1 - \sqrt{1 - \delta}$ fraction of errors. Even though ρ is more than $\delta/2$ for every $\delta > 0$, it is not twice the unique decoding bound for every $\delta > 0$.

In what follows we remedy these two shortcomings by utilizing the fact that the errors are random. We show that for *every* code with relative distance δ (over a large enough alphabet size) for most error patterns with fraction of errors ρ being arbitrarily close to (but less than) δ , the output of the list decoder will have size one. In fact, the result is somewhat stronger: it shows that even if one fixes the error locations *arbitrarily*, for most error patterns the output list size is one.

Theorem 7.5.1. *Let $\epsilon > 0$ be a real and $q \geq 2^{\Omega(1/\epsilon)}$ be an integer. Then the following is true for every $0 < \delta < 1 - 1/q$ and large enough n . Let $C \subseteq \{0, 1, \dots, q-1\}^n$ be a code with relative distance δ and let $S \subseteq [n]$ such that $|S| = (1 - \rho)n$, where $(0 < \rho \leq \delta - \epsilon)$.*

Then, for all $\mathbf{c} \in C$ and all but a $q^{-\Omega(\epsilon n)}$ fraction of error patterns, $\mathbf{e} \in \{0, 1, \dots, q-1\}^n$ such that

$$\mathbf{e}_S = \mathbf{0} \text{ and } wt(\mathbf{e}) = \rho n \quad (7.18)$$

the only codeword within Hamming distance ρn of $\mathbf{c} + \mathbf{e}$ is $\mathbf{c}$ itself.

For illustration of the kinds of error pattern we will deal with, see Figure 7.3.

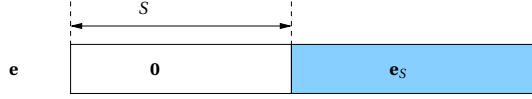


Figure 7.3: Illustration of the kind of error patterns we are trying to count.

Before we present the proof, we present certain corollaries (see Exercise 7.21). First the result above implies a similar result of the output list size being one for the following two random noise models: (i) uniform distribution over *all* error patterns of weight ρn and (ii) qSC_p . More specifically for the latter we claim that the above result also implies that any code with distance at least $p + \varepsilon$ allows for reliable communication over qSC_p . (Contrast the $2p + \varepsilon$ distance that was needed for a similar result that was implied by Proposition 6.4.1.)

Finally, we present a lemma (see Exercise 7.22) that will be crucial to the proof of Theorem 7.5.1.

Lemma 7.5.2. *Let C be an $(n, k, d)_q$ code. If we fix the values in $n - d + 1$ out of the n positions in a possible codeword, then at most one codeword in C can agree with the fixed values.*

Proof of Theorem 7.5.1. For the rest of the proof, fix a $\mathbf{c} \in C$. For notational convenience define $\mathcal{E}_S$ to be the set of all error patterns $\mathbf{e}$ such that $\mathbf{e}_S = 0$ and $wt(\mathbf{e}) = \rho n$. Note that as every error position has $(q - 1)$ non-zero choices and there are ρn such positions in $[n] \setminus S$, we have

$$|\mathcal{E}_S| = (q - 1)^{\rho n}. \quad (7.19)$$

Call an error pattern $\mathbf{e} \in \mathcal{E}_S$ as *bad* if there exists another codeword $\mathbf{c}' \neq \mathbf{c}$ such that

$$\Delta(\mathbf{c}', \mathbf{c} + \mathbf{e}) \leq \rho n.$$

Now, we need to show that the number of bad error patterns is at most

$$q^{-\Omega(\varepsilon n)} |\mathcal{E}_S|.$$

We will prove this by a somewhat careful counting argument.

We begin with a definition.

Definition 7.5.3. *Every error pattern $\mathbf{e}$ is associated with a codeword $c(\mathbf{e})$, which is the closest codeword which lies within Hamming distance ρn from it.*

For a bad error pattern we insist on having $c(\mathbf{e}) \neq \mathbf{c}$ —note that for a bad error pattern such a codeword always exists. Let A be the set of positions where $c(\mathbf{e})$ agrees with $\mathbf{c} + \mathbf{e}$.

The rest of the argument will proceed as follows. For each possible A , we count how many bad patterns $\mathbf{e}$ are associated with it (i.e. $\mathbf{c} + \mathbf{e}$ and $c(\mathbf{e})$ agree exactly in the positions in A). To bound this count non-trivially, we will use Lemma 7.5.2.

Define a real number α such that $|A| = \alpha n$. Note that since $c(\mathbf{e})$ and $\mathbf{c} + \mathbf{e}$ agree in at least $1 - \rho$ positions,

$$\alpha \geq 1 - \rho \geq 1 - \delta + \varepsilon. \quad (7.20)$$

For now let us fix A with $|A| = \alpha n$ and to expedite the counting of the number of bad error patterns, let us define two more sets:

$$A_1 = A \cap S,$$

and

$$A_2 = A \setminus A_1.$$

See Figure 7.4 for an illustration of the notation that we have fixed so far.

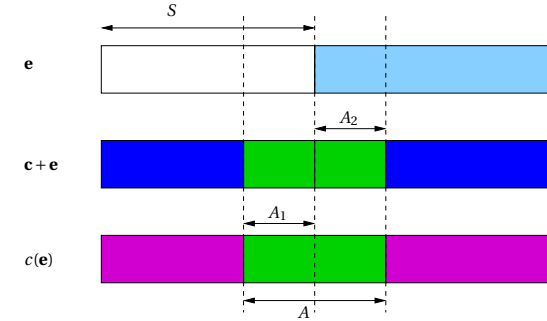


Figure 7.4: Illustration of notation used in the proof of Theorem 7.5.1. Positions in two different vectors that agree have the same color.

Define β such that

$$|A_1| = \beta n. \quad (7.21)$$

Note that this implies that

$$|A_2| = (\alpha - \beta)n. \quad (7.22)$$

Further, since $A_1 \subseteq A$, we have

$$\beta \leq \alpha.$$

To recap, we have argued that every bad error pattern $\mathbf{e}$ corresponds to a codeword $c(\mathbf{e}) \neq \mathbf{c}$ and is associated with a pair of subsets (A_1, A_2) . So, we fix (A_1, A_2) and then count the number of bad $\mathbf{e}$'s that map to (A_1, A_2) . (Later on we will aggregate this count over all possible choices of (A_1, A_2) .)

Towards this end, first we overestimate the number of error patterns $\mathbf{e}$ that map to (A_1, A_2) by allowing such $\mathbf{e}$ to have arbitrary values in $[n] \setminus (S \cup A_2)$. Note that all such values have to be non-zero (because of (7.18)). This implies that the number of possible distinct $\mathbf{e}_{[n] \setminus (S \cup A_2)}$ is at most

$$(q-1)^{n-|S|-|A_2|} = q^{n-(1-\rho)n-(\alpha-\beta)n}, \quad (7.23)$$

where the equality follows from the given size of S and (7.22). Next fix a non-zero $\mathbf{x}$ and let us only consider error patterns $\mathbf{e}$ such that

$$\mathbf{e}_{[n] \setminus (S \cup A_2)} = \mathbf{x}.$$

Note that at this stage we have an error pattern $\mathbf{e}$ as depicted in Figure 7.5.

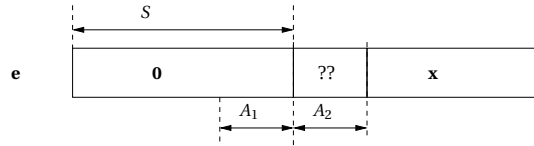


Figure 7.5: Illustration of the kind of error patterns we are trying to count now. The ? denote values that have not been fixed yet.

Now note that if we fix $c(\mathbf{e})_{A_2}$, then we would also fix $\mathbf{e}_{A_2}$ (as $(\mathbf{c} + \mathbf{e})_{A_2} = c(\mathbf{e})_{A_2}$). Recall that $\mathbf{c}$ is already fixed and hence, this would fix $\mathbf{e}$ as well. Further, note that

$$c(\mathbf{e})_{A_1} = (\mathbf{c} + \mathbf{e})_{A_1} = \mathbf{c}_{A_1}.$$

This implies that $c(\mathbf{e})_{A_1}$ is already fixed and hence, by Lemma 7.5.2 we would fix $c(\mathbf{e})$ if we fix (say the first) $(1-\delta)n+1-|A_1|$ positions in $c(\mathbf{e})_{A_2}$. Or in other words, by fixing the first $(1-\delta)n+1-|A_1|$ positions in $\mathbf{e}_{A_2}$, $\mathbf{e}$ would be completely determined. Thus, the number of choices for $\mathbf{e}$ that have the pattern in Figure 7.5 is upper bounded by

$$q^{(1-\delta)n+1-|A_1|} = (q-1)^{(1-\delta)n+1-\beta n}, \quad (7.24)$$

where the equality follows from (7.21).

Thus, by (7.23) and (7.24) the number of possible bad error patterns $\mathbf{e}$ that map to (A_1, A_2) is upper bounded by

$$(q-1)^{n-(1-\rho)n-\alpha n+\beta n+(1-\delta)n+1-\beta n} \leq (q-1)^{\rho n-\epsilon n+1} = (q-1)^{-\epsilon n+1} |\mathcal{E}_S|,$$

where the inequality follows from (7.20) and the equality follows from (7.19).

Finally, summing up over all choices of $A = (A_1, A_2)$ (of which there are at most 2^n), we get that the total number of bad patterns is upper bounded by

$$2^n \cdot (q-1)^{-\epsilon n+1} \cdot |\mathcal{E}_S| \leq q^{\frac{n}{\log_2 q} - \frac{\epsilon n}{2} + \frac{1}{2}} \cdot |\mathcal{E}_A| \leq q^{-\epsilon n/4} \cdot |\mathcal{E}_S|,$$

where the first inequality follows from $q-1 \geq \sqrt{q}$ (which in turn is true for $q \geq 3$) while the last inequality follows from the fact that for $q \geq \Omega(1/\epsilon)$ and large enough n , $\frac{n+1/2}{\log_2 q} < \frac{\epsilon n}{4}$. This completes the proof. $\square$

It can be shown that Theorem 7.5.1 is not true for $q = 2^{o(1/\epsilon)}$ (see Exercise 7.24).

7.6 Exercises

Exercise 7.1. In this exercise we consider the list decodability of the Hamming code C_H (of block length n). Prove that

1. C_H is $(\frac{\alpha}{n}, 1)$ -list decodable for any $\alpha < 2$.
2. C_H is not $(\frac{2}{n}, L)$ -list decodable for any $L = o(n)$.
3. C_H is $(\frac{2}{n}, O(n))$ -list decodable.

Exercise 7.2. Prove that if a code $C \subseteq \Sigma^n$ is uniquely decodable from τ fraction errors, then it is $(\tau + \frac{\epsilon}{n}, (qn)^\epsilon)$ -list-decodable for every integer $c \geq 1$, where $q = |\Sigma|$.

Exercise 7.3. In this problem, we show that the fraction of errors that a code can be list decoded from need not be related to its distance:

1. Let $0 < \rho < 1$. Then for every n such that ρn is an integer and any $(n, k, d)_q$ code C that is (ρ, L) -list-decodable prove there is a $(n+1, k+1, 1)_q$ code C' that is (ρ, qL) -list-decodable.
2. Argue that the above holds for linear codes (i.e. if C is a linear code then so is C').
3. Show that there exists a constant $0 < \rho < 1$, there exists a code that is $(\rho, O(q))$ -list decodable but has relative distance $\delta = o(1)$ (and indeed $\delta = \frac{1}{n}$).

Exercise 7.4. In this problem we will prove an extremal graph theory result, which will use to (re)prove the alphabet free Johnson bound in Exercise 7.5. This is a special case of the so called Zarankiewicz problem.

Let $G = (L, R, E)$ be a bipartite graph with $|L| = \ell$ and $|R| = r \geq 2$. For any $s \leq \ell$, we say G is $K_{s,2}$ free if there is **no** subset $L' \subseteq L$ and $R' \subseteq R$ with $|L'| = s$ and $|R'| = 2$ such that $L' \times R' \subseteq E$.³

In this problem we will prove that if G is $K_{s,2}$ free then

$$|E| \leq \ell + r \sqrt{(s-1)\ell}. \quad (7.25)$$

³I.e. $K_{s,2}$ is the complete $s \times 2$ bipartite graph.

Towards that end, define an $\ell \times r$ matrix M that is the adjacency matrix of G . I.e., each row and column of M is indexed by a vertex in L and R respectively and for any $(u, w) \in L \times R$, $M_{u,w} = 1$ iff $(u, w) \in E$. Define

$$\mathbf{v} = \sum_{w \in R} M^w,$$

where recall that M^w is the w th column of M .

Now consider the following sub-problems

1. Prove that

$$\|\mathbf{v}\|_2^2 \leq |E| + r(r-1)(s-1).$$

2. Prove that

$$|E|^2 \leq \ell \cdot \|\mathbf{v}\|_2^2.$$

Hint: Use Cauchy-Schwarz inequality.

3. Using the above two parts, or otherwise, prove (7.25).

Exercise 7.5. In this problem we will provide an alternate proof of Theorem 7.3.3 (with a better bound on the list size). Specifically we will work through a graph theoretic proof based on Exercise 7.4.

Towards that end let $C \subseteq \Sigma^n$ be a code of distance d , $\mathbf{y} \in \Sigma^n$ and $\mathbf{c}^1, \dots, \mathbf{c}^L$ be distinct codewords in C such that $\Delta(\mathbf{y}, \mathbf{c}^i) \leq n - \sqrt{n(n-d)} - 1$ for every $i \in [L]$. Define a graph $G = ([n], [L], E)$ to be a bipartite graph such that $(i, j) \in [n] \times [L]$ is an edge iff $\mathbf{y}_i = (\mathbf{c}^j)_i$. Consider the following sub-problems:

1. Prove that $|E| \geq L \cdot (\sqrt{n(n-d)} + 1)$.
2. Prove that G is $K_{n-d+1,2}$ free (recall the definition from Exercise 7.4).
3. Using the above parts, or otherwise, prove that $L \leq n$.

Hint: Use Exercise 7.4

4. Conclude that Theorem 7.3.3 (with a list size bound of n instead of nqd).

Exercise 7.6. In this exercise we will prove a technical result that will allow us to prove the general q -ary Johnson bound in Exercise 7.7 and bounded list size q -ary Johnson bound in Exercise 7.8.

Let $C \subseteq [q]^n$ be a q -ary code of relative distance

$$\delta = \left(1 - \frac{1}{q}\right)(1 - \beta).$$

Let $\gamma > \sqrt{\beta}$ and define

$$\rho = \left(1 - \frac{1}{q}\right)(1 - \gamma).$$

Let C be (ρ, L) -list decodable.

Let $\mathbf{c}_1, \dots, \mathbf{c}_L \in C$ be distinct codewords and let $\mathbf{y} \in [q]^n$ such that for every $i \in [L]$, $\Delta(\mathbf{c}_i, \mathbf{y}) \leq \rho n$.

Now consider the following subproblems:

1. For any $0 \leq \alpha \leq 1$, show there exists L vectors $\mathbf{v}_1, \dots, \mathbf{v}_L \in \mathbb{R}^{nq}$ such that for every $i, j \in [L]$:

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \Delta(\mathbf{y}, \mathbf{c}_i) + \Delta(\mathbf{y}, \mathbf{c}_j) - \Delta(\mathbf{c}_i, \mathbf{c}_j) + n \left(1 - \frac{1}{q}\right) (1 - \alpha)^2.$$

Hint: Map each symbol in $i \in [q]$ to $\mathbf{e}_i \in \mathbb{R}^q$. Then shift the 'origin' to a point defined by $\mathbf{y}$ and α .

2. Using the above part, or otherwise, prove that for all $i \in [L]$

$$\langle \mathbf{v}_i, \mathbf{v}_i \rangle \leq n \left(1 - \frac{1}{q}\right) (2\alpha(1 - \gamma) + (1 - \alpha)^2), \quad (7.26)$$

and for every $i \neq j \in [L]$,

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle \leq n \left(1 - \frac{1}{q}\right) (\beta + \alpha^2 - 2\alpha\gamma). \quad (7.27)$$

3. Argue that

$$L \leq \min \left\{ 2nq, \frac{1 - \beta}{\gamma^2 - \beta} \right\}.$$

Hint: Use Lemma 4.4.3 and pick α differently to get the two upper bounds.

Exercise 7.7. Let C be a q -ary code with distance d . Let

$$e < \left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{q}{q-1} \cdot \frac{d}{n}}\right) n.$$

Then prove that C is $\left(\frac{e}{n}, 2qn\right)$ -list decodable.

Exercise 7.8. Let C be a q -ary code with distance d , $L \geq 1$ and

$$e < \left(1 - \frac{1}{q}\right) \left(1 - \sqrt{1 - \frac{q}{q-1} \cdot \frac{L-1}{L} \cdot \frac{d}{n}}\right) n.$$

Then prove that C is $\left(\frac{e}{n}, L\right)$ -list decodable.

Exercise 7.9. In this problem we will show that the Johnson bound is tight (at least for non-linear codes), i.e. there exists codes that have exponential list size (just) beyond the Johnson bound. Consider the following sub-problems:

1. Call a code $C \subseteq [q]^n$ to be ρ -bounded (for $0 < \rho < 1$) if all codewords have Hamming weight at most ρn . Prove that C is (ρ, L) -list decodable for $L = |C|$.
2. Prove that there exists a ρ -bounded code with rate $\Omega\left(\frac{\epsilon^2}{\log q}\right)$ and relative distance

$$\delta \geq 2\rho - \frac{q}{q-1} \cdot \rho^2 - \epsilon.$$

3. Using the above two parts, or otherwise, prove that there exists a code C with relative distance δ such that for any $\varepsilon > 0$, it is $\left(J_q(\delta) + \varepsilon, 2^{\Omega(\varepsilon^4 n / \log q)}\right)$ -list decodable.

Exercise 7.10. Show that with high probability, a random linear code is (ρ, L) -list decodable code as long as

$$R \leq 1 - H_q(\rho) - \frac{1}{\lceil \log_q(L+1) \rceil}. \quad (7.28)$$

Hint: Think how to fix (7.8) for random linear code.

Exercise 7.11. In this exercise we will see how we can "fix" the dependence on L is the rate of random linear codes from Exercise 7.10. In particular, we will consider the following family of codes that are somewhere between linear and general codes and are called pseudolinear codes, which are defined as follows.

Let q be a prime power and let $1 \leq k \leq n$ and $L \geq 1$ be integers. Then an (n, k, L, r, q) -family of pseudolinear codes is defined as follows. Let $\mathbf{H}$ be the parity check matrix of an $[q^k - 1, q^k - 1 - r, L + 1]_q$ linear code and $\mathbf{H}'$ be an extension of $\mathbf{H}$ with the first column being $\mathbf{0}$ (and the rest being $\mathbf{H}$). Every code in the family is indexed by a matrix $\mathbf{A} \in \mathbb{F}_q^{n \times r}$. Fix such a $\mathbf{A}$. Then the corresponding code $C_{\mathbf{A}}$ is defined as follows. For every $\mathbf{x} \in \mathbb{F}_q^k$, we have

$$C_{\mathbf{A}}(\mathbf{x}) = \mathbf{A} \cdot \mathbf{H}'_{\mathbf{x}},$$

where $\mathbf{H}'_{\mathbf{x}}$ is the column corresponding to $\mathbf{x}$, when thought of as an integer between 0 and $q^k - 1$.

Next, we will argue that random pseudolinear codes have near optimal list decodability:

- Fix non-zero messages $\mathbf{m}_1, \dots, \mathbf{m}_L$. Then for a random code $C_{\mathbf{A}}$ from an (n, k, L, r, q) -family of pseudolinear code family, the codewords $C_{\mathbf{A}}(\mathbf{m}_1), \dots, C_{\mathbf{A}}(\mathbf{m}_L)$ are independent random vectors in $\mathbb{F}_q^n$.
- Define (n, k, L, q) -family of pseudolinear codes to be $(n, k, L, O(kL), q)$ -family of pseudolinear codes. Argue that (n, k, L, q) -family of pseudolinear codes exist.

Hint: Exercise 5.9 might be helpful.

- Let $\varepsilon > 0$ and $q \geq 2$ be a prime power. Further let $0 \leq \rho < 1 - 1/q$. Then for a large enough n and k such that

$$\frac{k}{n} \geq 1 - H_q(\rho) - \frac{1}{L} - \varepsilon,$$

a random (n, k, L, q) -pseudolinear code is (ρ, L) -list decodable.

- Show that one can construct a (ρ, L) -list decodable pseudolinear code with rate at least $1 - H_q(\rho) - \frac{1}{L} - \varepsilon$ in $q^{O(kL+n)}$ time.

Hint: Use method of conditional expectations.

Exercise 7.12. In this exercise we will consider a notion of "average" list decoding that is closely related to our usual notion of list decoding. As we will see in some subsequent exercises, sometimes it is easier to work with this average list decoding notion.

- We begin with an equivalent definition of our usual notion of list decoding. Argue that a code C is (ρ, L) list decodable if and only if for every $\mathbf{y} \in [q]^n$ and every subset of $L + 1$ codewords $\mathbf{c}_0, \dots, \mathbf{c}_L$ we have that

$$\max_{0 \leq i \leq L} \Delta(\mathbf{y}, \mathbf{c}_i) > \rho n.$$

- We define a code C to be (ρ, L) -average list decodable if for every $\mathbf{y} \in [q]^n$ and $L + 1$ codewords $\mathbf{c}_0, \dots, \mathbf{c}_L$ we have

$$\frac{1}{L} \cdot \sum_{i=0}^L \Delta(\mathbf{y}, \mathbf{c}_i) > \rho n.$$

Argue that if C is (ρ, L) -average list decodable then it is also (ρ, L) -list decodable.

- Argue that if C is (ρ, L) -list decodable then it is also $(\rho(1 - \gamma), \lceil L/\gamma \rceil)$ -average list decodable (for every $0 < \gamma < \rho$).

Exercise 7.13. In Section 7.5 we saw that for every code one can correct arbitrarily close to relative distance fraction of random errors. In this exercise we will see that one can prove a weaker result. In particular let $\mathcal{D}$ be an arbitrary distribution on $B_q(\mathbf{0}, \rho n)$. Then argue that for most codes, the list size with high probability is 1. In other words, show that for $1 - o(1)$, fraction of codes C we have that for every codeword $\mathbf{c} \in C$

$$\Pr_{\mathbf{e} \sim \mathcal{D}} [|\mathcal{B}_q(\mathbf{c} + \mathbf{e}, \rho n) \cap C| > 1] = o(1).$$

Hint: Adapt the proof of Theorem 6.3.1 from Section 6.3.2.

Exercise 7.14. We call a code (ρ, L) -erasure list-decodable is informally for every received word with at most p fraction of erasures at most L codewords agree with it in the unerased positions. More formally, an $(n, k)_q$ -code C is (ρ, L) -erasure list-decodable if for every $\mathbf{y} \in [q]^{(1-\rho)n}$ and every subset $T \subseteq [n]$ with $|T| = (1 - \rho)n$, we have that

$$|\{\mathbf{c} \in C \mid \mathbf{c}_T = \mathbf{y}\}| \leq L.$$

In this exercise you will prove some simple bounds on the best possible rate for erasure-list decodable code.

- Argue that if C has distance d then it is $\left(\frac{d-1}{n}, 1\right)$ -erasure list decodable.
- Show that there exists a (ρ, L) -erasure list decodable code of rate

$$\frac{L}{L+1} \cdot (1 - \rho) - \frac{H_q(\rho)}{L} - \gamma,$$

for every $\gamma > 0$.

3. Argue that there exists a linear (ρ, L) -erasure list decodable code with rate

$$\frac{J-1}{J} \cdot (1-\rho) - \frac{H_q(\rho)}{J-1} - \gamma,$$

where $J = \lceil \log_q(L+1) \rceil$ and $\gamma > 0$.

4. Argue that the bound in item 2 is tight for large enough L by showing that if a code of rate $1-\rho+\varepsilon$ is (ρ, L) -erasure list decodable then L is $2^{\Omega_\varepsilon(n)}$.

Exercise 7.15. In this exercise we will see an alternate characterization of erasure list-decodable code for linear codes, which we will use to show separation between linear and non-linear code in the next exercise.

Given a linear code $C \subseteq \mathbb{F}_q^n$ and an integer $1 \leq r \leq n$, define the r 'th generalized Hamming distance, denoted by $d_r(C)$, as follows. First given a set $D \subseteq \mathbb{F}_q^N$, we define the support of D as the union of the supports of vectors in D . More precisely

$$\text{supp}(D) = \{i \mid \text{there exists } (u_1, \dots, u_n) \in D \text{ such that } u_i \neq 0\}.$$

Then $d_r(C)$ is size of the smallest support of all r -dimensional subcodes of C .

Argue the following:

1. (Warmup) Convince yourself that $d_1(C)$ is the usual Hamming distance of C .
2. Prove that C is $(\rho n, L)$ -erasure list-decodable if and only if $d_{1+\lfloor \log_q L \rfloor}(C) > \rho n$.

Exercise 7.16. In this exercise we use the connection between generalized Hamming distance and erasure list decodability from Exercise 7.15 to show an "exponential separation" between linear and non-linear codes when it comes to list decoding from erasure.

Argue the following:

1. Let C be an $[n, k]_q$ code. Then show that the average support size of r -dimensional subcodes of C is exactly

$$\frac{q^r - 1}{q^r} \cdot \frac{|C|}{|C| - 1} \cdot n.$$

2. From previous part or otherwise, conclude that if for an $[n, k]_q$ code C we have $d_r(C) > n(1 - q^{-r})$, then we have

$$|C| \leq \frac{d_r(C)}{d_r(C) - n(1 - q^{-r})},$$

Note that the above bound for $r = 1$ recovers the Plotkin bound (second part of Theorem 4.4.1).

3. Argue that any (family) of code C with $d_r(C) = \delta_r \cdot n$, its rate satisfies:

$$R(C) \leq 1 - \frac{q^r}{q^r - 1} \cdot \delta_r + o(1).$$

Hint: Use the result from previous part on a code related to C .

4. Argue that for small enough $\varepsilon > 0$, any linear $(1 - \varepsilon, L)$ -erasure list decodable code with positive rate must have $L \geq \Omega(1/\varepsilon)$.

5. Argue that there exist $(1 - \varepsilon, O(\log(1/\varepsilon)))$ -erasure list decodable code with positive (in fact $\Omega(\varepsilon)$) rate. Conclude that there exists non-linear codes that have the same erasure list decodability but with exponentially smaller list sizes than linear codes.

Exercise 7.17. In this exercise we will prove an analog of the Johnson bound (Theorem 7.3.1) but for erasure list-decodable codes. In particular, let C be an $(n, k, \delta n)_q$ code. Then show that for every $\varepsilon > 0$, C is an $\left(\left(\frac{q}{q-1} - \varepsilon\right)\delta, \frac{q}{(q-1)\varepsilon}\right)$ -erasure list decodable.

Hint: The Plotkin bound (Theorem 4.4.1) might be useful.

Exercise 7.18. Let C be a q -ary (ρ, L) -(average) list decodable of rate R , then show that there exists another (ρ, L) -(average) list decodable code with rate at least

$$R + H_q(\lambda) - 1 - o(1),$$

for every $\lambda \in (\rho, 1 - 1/q]$ such that all codewords in C' have Hamming weight exactly λn .

Hint: Try to translate C .

Exercise 7.19. In this exercise, we will prove a lower bound on the list size of list decodable codes that have rate approaching list decoding capacity. We do this via a sequence of following steps:

1. Let $C \subseteq [q]^n$ be a $(\rho, L - 1)$ -list decodable code such that all codewords have Hamming weight exactly λn for

$$\lambda = \rho + \frac{1}{2L} \cdot \rho^L.$$

Then prove that

$$|C| < \frac{2L^2}{\lambda^L}.$$

Hint: It might be useful to use the following result due to Erdős [50] (where we choose the variables to match the relevant ones in the problem). Let $\mathcal{A}$ be a family of subsets of $[n]$. Then if every $A \in \mathcal{A}$ has size at least $2L^2/\lambda^L$, then there exist distinct $A_1, \dots, A_L \in \mathcal{A}$ such that $\cap_{i=1}^L A_i$ has size at least $\frac{n\lambda^L}{2}$.

2. Argue that any q -ary $(\rho, L - 1)$ -list decodable code C (for large enough block length) has rate at most $1 - H_q(\rho) - b_{\rho,q} \cdot \frac{\rho^L}{L}$ for some constant $b_{\rho,q}$ that only depends on ρ and q .

Hint: Use the previous part and Exercise 7.18.

3. Argue that any q -ary (ρ, L) -list decodable C with rate $1 - H_q(\rho) - \varepsilon$ must satisfy $L \geq \Omega_{\rho,q}(\log(1/\varepsilon))$.

Exercise 7.20. It follows from Theorem 7.4.1 that a random code of rate $1 - H_q(\rho) - \varepsilon$ with high probability is $(\rho, O(1/\varepsilon))$ -list decodable. On the other hand, the best lower bound on the list size for codes of rate $1 - H_q(\rho) - \varepsilon$ (for constant p, q) is $\Omega(\log(1/\varepsilon))$ (as we just showed in Exercise 7.19). It is natural to wonder if one can perhaps do a better argument for random codes. In this exercise, we will show that our argument for random codes is the best possible (for random codes). We will show this via the following sequence of steps:

1. Let C be a random $(n, k)_q$ code of rate $1 - H_q(\rho) - \varepsilon$. For every $\mathbf{y} \in [q]^n$ and any subset $S \subseteq [q]^k$ of size $L + 1$, define the random event $\mathcal{E}(\mathbf{y}, S)$ that for every $\mathbf{m} \in S$, $C(\mathbf{m})$ is at Hamming distance at most ρn from $\mathbf{y}$. Define

$$W = \sum_{\mathbf{y}, S} \mathcal{E}(\mathbf{y}, S).$$

Argue that C is (ρ, L) -list decodable if and only if $W = 0$.

2. Define

$$\mu = q^{-n} \cdot \text{Vol}_q(\rho n, n).$$

Argue that

$$\mathbb{E}[W] \geq \frac{1}{(L+1)^{L+1}} \cdot \mu^{L+1} \cdot q^n \cdot q^{k(L+1)}.$$

3. Argue that

$$\sigma^2(Z) \leq q^{2n} \cdot \sum_{\ell=1}^{L+1} (L+1)^{2(L+1)} \cdot q^{k(2L+2-\ell)} \cdot \mu^{2L-\ell+3}.$$

Hint: Analyze the probability of both events $\mathcal{E}(\mathbf{y}, S)$ and $\mathcal{E}(\mathbf{z}, T)$ happening together for various intersection sizes $\ell = |S \cap T|$.

4. Argue that C is $(\rho, \frac{1-H_q(\rho)}{2\varepsilon})$ -list decodable with probability at most $q^{-\Omega_{\rho,\varepsilon}(n)}$.

Hint: Use Chebyshev's inequality.

Exercise 7.21. In this problem we will show the following corollaries of Theorem 7.5.1. Let ρ, C and δ be as defined in Theorem 7.5.1. Prove the following:

1. Consider the channel that has the uniform distribution over all error patterns of weight ρn . Prove that for all codewords $\mathbf{c}$ with all but an exponentially small probability (over the distribution over the error pattern $\mathbf{e}$) $\mathbf{c}$ is the only codeword within Hamming distance ρn of $\mathbf{c}$.
2. Using the first part, or otherwise, prove that any code with distance at least $p + \varepsilon$ allows for reliable communication over $q\text{SC}_p$.

Exercise 7.22. Prove Lemma 7.5.2.

Exercise 7.23 (Inverse Markov Inequality). Recall that the Markov inequality allow us to bound the probability that a random variable X is larger than its expectation $\mathbb{E}[X]$. However, sometimes (and specifically in Exercise 7.24) we need to bound the probability that X is smaller than $\mathbb{E}[X]$. While obtaining probability bounds below the expectation is not possible in general in this exercise we will see a specific setting where we can indeed get such a bound (which will be useful in Exercise 7.24).

Let $G = (L, R, E)$ be a bipartite graph and let $\overline{d}_R$ be the average right degree of G . For any vertex $u \in L \cup R$, we use $d(u)$ to denote its degree in G .

Prove the following:

1. Pick an edge $(u, v) \in E$ uniformly at random. Then the probability that $d(v) \leq \varepsilon \overline{d}_R$ is at most ε .
2. Let G be d -left regular and consider the following process. Pick $u \in L$ uniformly at random and then pick $v \in R$ uniformly at random from the d right neighbors of u . Then the probability that $d(v) \leq \varepsilon \frac{d(L)}{|R|}$ is at most ε .

Exercise 7.24. In this exercise we will show that Theorem 7.5.1 (or more precisely its implication in Exercise 7.21) is not true for $q \leq 2^{o(1/\varepsilon)}$.

1. Let $q \geq 2$ and $0 \leq \rho < 1 - \frac{1}{q}$. Let $C \subseteq \{0, 1, \dots, q-1\}^n$ be a code with rate $1 - H_q(\rho) + \gamma$ for some $\gamma > 0$. Then prove that there exists a codeword $\mathbf{c} \in C$ such that for at least $1 - e^{-\Omega(\gamma n)}$ fraction of error patterns $\mathbf{e}$, $|B(\mathbf{c} + \mathbf{e}, \rho n) \cap C| \geq 2$.

Hint: Use Exercise 7.23.

2. Fix $\varepsilon > 0$ and let q be a prime power satisfying $49 \leq q \leq 2^{o(1/\varepsilon)}$. Show that there exists a code with relative distance δ that contains a codeword $\mathbf{c}$ with the following property: for $1 - e^{-\Omega(\varepsilon n)}$ fraction of error patterns $\mathbf{e}$, $|B(\mathbf{c} + \mathbf{e}, (\delta - \varepsilon)n) \cap C| \geq 2$.

Hint: For this part assume the existence of certain code (algebraic-geometry or AG code) that for $q \geq 49$ have relative distance δ and rate $1 - \delta - O(\frac{1}{\sqrt{q}})$.

7.7 Bibliographic Notes

List decoding was defined by Elias [49] and Wozencraft [163].

The result showing that for random error patterns, the list size with high probability is one for the special case of Reed-Solomon codes was shown by McEliece [113]. The result for all codes was proved by Rudra and Uurtamo [134].

In applications of list decoding in complexity theory (see for example [150], [66, Chap. 12]), side information is used crucially to prune the output of a list decoding algorithm to compute a unique answer.

Guruswami [65] showed that the answer to Question 7.3.1 is yes in the sense that there exist linear codes with relative distance δ such that we can find Hamming ball of radius larger than $J_q(\delta)$ with super-polynomially many codewords. This result was proven under a number-theoretic assumption, which was later removed by [77].

(7.28) implies that there exist linear codes with rate $1 - H_q(\rho) - \varepsilon$ that are $(\rho, q^{O(1/\varepsilon)})$ -list decodable. (This is also true for most linear codes with the appropriate parameters.) However, for a while just for $q = 2$, we knew the existence of $(\rho, O(1/\varepsilon))$ -list decodable codes [70] (though it was not a high probability result). Guruswami, Håstad and Kopparty resolved this “gap” by showing that random linear codes of rate $1 - H_q(\rho) - \varepsilon$ are $(\rho, O(1/\varepsilon))$ -list decodable (with high probability) [69].

Exercise 7.4 is from [91] while Exercise 7.5 is due to Jaikumar Radhakrishnan.

Chapter 8

What Cannot be Done-II

In this brief interlude of a chapter, we revisit the trade-offs between rate and relative distance for codes. Recall that the best (and only) lower bound on R that we have seen is the GV bound and the best upper bound on R that we have seen so far is a combination of the Plotkin and Hamming bounds (see Figure 4.5). In this chapter, we will prove one more upper bound on R (the last one in this book) due to Elias and Bassalygo. Then we will mention the best-known upper bound on rate (but without proving it or even stating it completely). Finally, we will conclude by summarizing what we have seen so far and laying down the course for the rest of the book.

8.1 Elias-Bassalygo bound

We begin with the statement of a new upper bound on the rate called the Elias-Bassalygo bound. Recall the q -ary Johnson bound function $J_q(\cdot)$ from Theorem 7.3.1. The Elias-Bassalygo bound uses the same function, and the q -ary entropy function, to give an upper bound on the distance of a code, as follows.

Theorem 8.1.1 (Elias-Bassalygo bound). *Every q -ary code of rate R , distance δ , and large enough block length n , satisfies the following:*

$$R \leq 1 - H_q(J_q(\delta)) + o(1)$$

See Figure 8.1 for an illustration of the Elias-Bassalygo bound for binary codes. Note that this bound is tighter than all the previous upper bounds on rate that we have seen so far.

The proof of Theorem 8.1.1 uses the following lemma:

Lemma 8.1.2. *Given a q -ary code, $C \subseteq [q]^n$, and $0 \leq e \leq n$, there exists a Hamming ball of radius e with at least $\frac{|C| \text{Vol}_q(e, n)}{q^n}$ codewords in it.*

Proof. We will prove the existence of the required Hamming ball by the probabilistic method. Pick a received word $\mathbf{y} \in [q]^n$ at random. It can be checked that the expected value of $|B(\mathbf{y}, e) \cap C|$ is $\frac{|C| \text{Vol}_q(e, n)}{q^n}$. (We have seen this argument earlier in the proof of part (ii) of Theorem 7.4.2.)

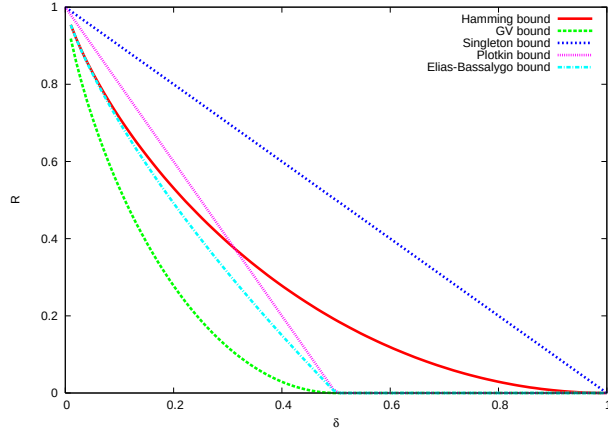


Figure 8.1: Singleton, Hamming, Plotkin, GV and Elias-Bassalygo bounds on rate versus distance for binary codes.

By the probabilistic method, this implies the existence of a $\mathbf{y} \in [q]^n$ such that

$$|B(\mathbf{y}, e) \cap C| \geq \frac{|C| \text{Vol}_q(e, n)}{q^n},$$

as desired. $\square$

Proof of Theorem 8.1.1. Let $C \subseteq [q]^n$ be any code with relative distance δ . Define $e = nJ_q(\delta) - 1$. By Lemma 8.1.2, there exists a Hamming ball with $\mathcal{B}$ codewords such that the following inequality is true:

$$|\mathcal{B}| \geq \frac{|C| \text{Vol}_q(e, n)}{q^n}. \quad (8.1)$$

By our choice of e and the Johnson bound (Theorem 7.3.1), we have

$$|\mathcal{B}| \leq qdn. \quad (8.2)$$

Combining the lower and upper bounds in (8.1) and (8.2) imply the following

$$|C| \leq qdn \cdot \frac{q^n}{\text{Vol}_q(e, n)} \leq q^{n(1-H_q(J_q(\delta))+o(1))},$$

where the second inequality follows from our good old lower bound on the volume of a Hamming ball (Proposition 3.3.3) and the fact that $qdn \leq qn^2 \leq q^{o(n)}$ for large enough n . This implies that the rate R of C satisfies:

$$R \leq 1 - H_q(J_q(\delta)) + o(1),$$

as desired. $\square$

8.2 The MRRW bound: A better upper bound

We now turn to the strongest known method to upper bound the rate of codes, that dates back to 1977. We first state this bound, called the first linear programming (LP) bound (and sometimes the first MRRW bound, after its discoverers McEliece, Rodemich, Rumsey and Welch). We will restrict ourselves to binary codes. The first LP bound states that the rate of a binary code family of relative distance δ is at most

$$R_{\text{MRRW}}(\delta) := h\left(\frac{1}{2} - \sqrt{\delta(1-\delta)}\right).$$

The MRRW bound is a better upper bound than the Elias-Bassalygo bound (though we will not state or prove the bound in this book). To give one data point on the MRRW bound, consider $\delta = \frac{1}{2} - \varepsilon$ (think of $\varepsilon \rightarrow 0$). The GV bound gives a lower bound on R of $\Omega(\varepsilon^2)$ (see Proposition 3.3.7). The Elias-Bassalygo bound gives an upper bound of $O(\varepsilon)$ on R , which is quite far from the Gilbert-Varshamov (GV) bound. The MRRW bound gets much closer to the GV bound and gives an upper bound on R of $O(\varepsilon^2 \log(\frac{1}{\varepsilon}))$. We note that there is still a gap between the GV lower bound and the MRRW upper bound, namely the $O(\log(\frac{1}{\varepsilon}))$ factor, and this gap is still unresolved to this day.

8.3 A Breather

Let us now recap the combinatorial results that we have seen so far. Table 8.1 summarizes what we have seen so far for binary codes in Shannon's world and Hamming's world (under both unique and list decoding settings).

For the rest of this section, we remind the reader about the definition of explicit codes (Definition 6.3.4) and strongly explicit codes (Definition 6.3.5).

We begin with BSC_p . We have seen that the capacity of BSC_p is $1 - H(p)$. The most natural open question is to obtain the capacity result but with explicit codes along with efficient decoding (and encoding) algorithms (Question 6.3.1).

Next, we consider Hamming's world under unique decoding. For large enough alphabets, we have seen that Reed-Solomon codes (Chapter 5) meet the Singleton bound (Theorem 4.3.1). Further, the Reed-Solomon codes are strongly explicit¹. The natural question then is

¹The proof is left as an exercise.

Shannon	Hamming	
BSC_p	Unique	List
$1 - H(p)$ is capacity	$R \geq 1 - H(\delta)$	$1 - H(p)$ is list decoding capacity
	$R \leq MRRW$	
Explicit codes at capacity?	Explicit Asymptotically good codes?	Explicit codes at capacity?
Efficient decoding algorithm?	Efficient decoding algorithms?	Efficient decoding algorithms?

Table 8.1: High level summary of results seen so far. An asymptotically good code has $R > 0$ and $\delta > 0$.

Question 8.3.1. Can we decode Reed-Solomon codes up to half its distance?

For smaller alphabets, especially binary codes, as we have seen in the last section, there is a gap between the best-known lower and upper bounds on the rate of a code with a given relative distance. Further, we do not know of an explicit construction of a binary code that lies on the GV bound. These lead to the following questions that are still wide open:

Open Question 8.3.1. What is the optimal trade-off between R and δ ?

Open Question 8.3.2.

Does there exist an explicit construction of (binary) codes on the GV bound?

If we scale down our ambitions, the following is a natural weaker version of the second question above:

Question 8.3.2. Do there exist explicit asymptotically good binary codes?

We also have the following algorithmic counterpart to the above question:

Question 8.3.3. If one can answer Question 8.3.2, then can we decode such codes efficiently

from a non-zero fraction of errors?

For list decoding, we have seen that the list decoding capacity is $1 - H_q(p)$. The natural open questions are whether we can achieve the capacity with explicit codes (Question 7.4.1) along with efficient list decoding algorithms (Question 7.4.2).

For the remainder of the book, we will primarily focus on the questions mentioned above (and summarized in the last two rows of Table 8.1).

8.4 Exercises

Exercise 8.1. Justify the linear program also for general codes.

Exercise 8.2. Connect the linear program to the Lovász theta function (SDP relaxation for the independent set).

Exercise 8.3. We mentioned the MRRW bound that the rate of binary codes of relative distance $1/2 - \epsilon$ is at most $O(\epsilon^2 \log(1/\epsilon))$. In this problem, you will prove this for the special case of ϵ -biased codes, which are binary codes in which every pair of distinct codewords have relative Hamming distance in the range $[1/2 - \epsilon, 1/2 + \epsilon]$ (note such codes have relative distance at least $1/2 - \epsilon$, but in addition no two codewords differ in more than $(1/2 + \epsilon)$ fraction of positions).

Consider the following claim:

Claim: Let A be an $m \times m$ real symmetric matrix with 1's on the diagonal, and all off-diagonal elements at most ϵ in absolute value. Assume $\epsilon \in (0, 1/4)$ and m is sufficiently large. Then, $\text{rank}(A) \geq \Omega\left(\frac{\log m}{\epsilon^2 \log(1/\epsilon)}\right)$.

1. Show why the claim implies that an ϵ -biased code of block length n can have rate at most $O(\epsilon^2 \log(1/\epsilon))$.
2. Towards proving the above claim, first prove the following fact when the off-diagonal entries are really small:

Let B be a real symmetric $m \times m$ matrix with 1's on the diagonal. If all off-diagonal entries of B are at most $1/\sqrt{m}$ in absolute value, then $\text{rank}(B) \geq m/2$.

Hint: Use the fact that $\text{Trace}(B) = \lambda_1 + \dots + \lambda_m$ if λ_i are the eigenvalues of B , and relate $\text{Trace}(B^2)$ and $\text{Trace}(B)$ via Cauchy-Schwarz.

3. Prove the claim above using the fact from Part 2.

Hint: Given A , consider the matrix B whose entries are t 'th powers of the entries of A for some large t , and argue that $\text{rank}(B)$ is at most $\binom{\text{rank}(A)+t}{t}$.

8.5 Bibliographic Notes

The Elias-Bassalygo bound appeared independently in two different articles. The first by Bassalygo [13] in 1965 gives a bound for general q in Russian. The second appearance is in the 1967 article of Shannon, Gallager and Berkekamp [139] where the bound for $q = 2$ appears as Lemma 7 and is attributed to an unpublished work of Elias from 1960. The McEliece-Rodemich-Rumsey-Welch (MRRW) bound was introduced in 1977 in the paper [114], and builds on the Linear Programming method introduced by Delsarte [37].

Part III

The Codes

Chapter 9

When Polynomials Save the Day: Polynomial Based Codes

As we saw in Chapter 5, the Reed-Solomon codes give a remarkable family of codes with optimal dimension vs. distance tradeoff. They even match the Singleton bound (recall Theorem 4.3.1), get $k = n - d + 1$ for a code of block length n , distance d and dimension k . However they achieve this remarkable performance only over large alphabets, namely when the alphabet size $q \geq n$. In fact, so far in this book, we have not seen *any explicit* asymptotically good code other than a Reed-Solomon code. This naturally leads to the following question (which is a weaker form for Question 8.3.2):

Question 9.0.1. *Do there exist explicit asymptotically good codes for small alphabets $q \ll n$?*

In this chapter we study an extension of Reed-Solomon codes, called the (generalized) Reed-Muller codes, that lead to codes over smaller alphabets while losing in the dimension-distance tradeoff (but under certain settings do answer Question 9.0.1 in the affirmative).

The main idea is to extend the notion of functions we work with, to multivariate functions. (See Exercise 5.1 for equivalence between certain Reed-Solomon codes and univariate functions.) Just working with bivariate functions (functions on two variables), allows us to get codes of block length $n = q^2$, and more variables can increase the length further for the same alphabet size. We look at functions of *total degree* at most r . Analysis of the dimension of the code reduces to simple combinatorics. Analysis of the distance follows from “polynomial-distance” lemmas (see Lemmas 9.2.2, 9.3.1 and 9.4.1), whose use is ubiquitous in algebra, coding theory and computer science, and we describe these in the sections below. We start with the generic construction.

9.1 The generic construction

Recall that for a monomial $\mathbf{X}^{\mathbf{d}} = X_1^{d_1} \cdot X_2^{d_2} \cdots X_m^{d_m}$ its total degree is $d_1 + d_2 + \cdots + d_m$. We next extend this to the definition of the degree of a polynomial:

Definition 9.1.1. *The total degree of a polynomial $P(\mathbf{X}) = \sum_{\mathbf{d}} c_{\mathbf{d}} \mathbf{X}^{\mathbf{d}}$ over $\mathbb{F}_q$ (i.e. every $c_{\mathbf{d}} \in \mathbb{F}_q$) is the maximum over $\mathbf{d}$ such that $c_{\mathbf{d}} \neq 0$, of the total degree of $\mathbf{X}^{\mathbf{d}}$. We denote the total degree of P by $\deg(P)$.*

For example, the degree of the polynomial $3X^3Y^4 + X^5 + Y^6$ is 7.

It turns out that when talking about Reed-Muller codes, it is convenient to switch back and forth between multivariate functions and multivariate polynomials. We can extend the notion above to functions from $\mathbb{F}_q^m \rightarrow \mathbb{F}_q$. For $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ let $\deg(f)$ be the minimal degree of a polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$ (where $\mathbb{F}_q[X_1, \dots, X_m]$ denotes the set of all m -variate polynomials with coefficients from $\mathbb{F}_q$) such that $f(\alpha) = P(\alpha)$ for every $\alpha \in \mathbb{F}_q^m$. Note that since (by Exercise 2.4) for every $a \in \mathbb{F}_q$ we have $a^q - a = 0$, it follows that a minimal degree polynomial does not contain monomials with degree more than $q - 1$ in any single variable. In what follows,

Definition 9.1.2. *We use $\deg_{X_i}(p)$ to denote the degree of polynomial p in variable X_i and $\deg_{X_i}(f)$ to denote the degree of (the minimal polynomial corresponding to) a function f in variable X_i .*

For example $\deg_X(3X^3Y^4 + X^5 + Y^6) = 5$ and $\deg_Y(3X^3Y^4 + X^5 + Y^6) = 6$. Further, in this notation we have for every function $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$, $\deg_{X_i}(f) \leq q - 1$ for every $i \in [m]$.

Reed-Muller codes are given by three parameters: a prime power q and positive integers m and r , and consist of the evaluations of m -variate polynomials of degree at most r over all of the domain $\mathbb{F}_q^m$.

Definition 9.1.3 (Reed-Muller Codes). *The Reed-Muller code with parameters q, m, r , denoted $\text{RM}(q, m, r)$, is the set of evaluations of all m -variate polynomials in $\mathbb{F}_q[X_1, \dots, X_m]$ of total degree at most r and individual degree at most $q - 1$ over all points in $\mathbb{F}_q^m$. Formally*

$$\text{RM}(q, m, r) \stackrel{\text{def}}{=} \{f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q \mid \deg(f) \leq r\}.$$

For example consider the case of $m = q = 2$ and $r = 1$. Note that all bivariate polynomials over $\mathbb{F}_2$ of degree at most 1 are 0, 1, X_1 , X_2 , $1 + X_1$, $1 + X_2$, $X_1 + X_2$ and $1 + X_1 + X_2$. Thus, we have that (where the evaluation points for (X_1, X_2) are ordered as $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$):

$$\text{RM}(2, 2, 1) = \{(0, 0, 0, 0), (1, 1, 1, 1), (0, 0, 1, 1), (0, 1, 0, 1), (1, 1, 0, 0), (1, 0, 1, 0), (0, 1, 1, 0), (1, 0, 0, 1)\}.$$

Also note that $\text{RM}(q, m, 1)$ is almost the Hadamard code (see Exercise 5.7).

The Reed-Muller code with parameters (q, m, r) clearly has alphabet $\mathbb{F}_q$ and block length $n = q^m$. Also it can be verified that $\text{RM}(q, m, r)$ is a linear code (see Exercise 9.1.). This leads to the following question, which will be the primary focus of this chapter:

Question 9.1.1. *What are the dimension and distance of an $\text{RM}(q, m, r)$ code?*

The dimension of the code is the number of m -variate monomials of degree at most r , with the condition that degree in each variable is at most $q - 1$. No simple closed form expression for this that works for all choices of q, m and r is known, so we will describe the effects only in some cases. The distance analysis of these codes takes a little bit more effort and we will start with two simple settings before describing the general result.

9.2 The low degree case

We start by considering $\text{RM}(q, m, r)$ when $r < q$, i.e., the degree is smaller than the field size. We refer to this setting as the “low-degree” setting.

Dimension. The dimension of $\text{RM}(q, m, r)$ in the low-degree case turns out to have a nice closed form, since we do not have to worry about the constraint that each variable has degree at most $q - 1$: this is already imposed by restricting the total degree to at most $r \leq q - 1$. This leads to a nice expression for the dimension:

Proposition 9.2.1. *The dimension of the Reed Muller code $\text{RM}(q, m, r)$ equals $\binom{m+r}{r}$ when $r < q$.*

Proof. The dimension equals the size of the set

$$D = \left\{ (d_1, \dots, d_m) \in \mathbb{Z}^m \mid d_i \geq 0 \text{ for all } i \in [m], \sum_{i=1}^m d_i \leq r \right\}, \quad (9.1)$$

since for every $(d_1, \dots, d_m) \in D$, the monomial $X_1^{d_1} \cdots X_m^{d_m}$ is a monomial of degree at most r and these are all such monomials. The closed form expression for the dimension follows by a simple counting argument. (See Exercise 9.2). $\square$

Distance. Next we turn to the analysis of the distance of the code. To understand the distance we will first state and prove a simple fact about the number of zeroes a multivariate polynomial can have. (We will have three versions of this in this chapter - with the third subsuming the first (Lemma 9.2.2) and second (Lemma 9.3.1), but the first two will be slightly simpler to state and remember.)

Lemma 9.2.2 (Polynomial Distance Lemma (low-degree case)). *Let $f \in \mathbb{F}_q[X_1, \dots, X_m]$ be a non-zero polynomial with $\deg(f) \leq r$. Then the fraction of zeroes of f is at most $\frac{r}{q}$, i.e.,*

$$\frac{|\{\mathbf{a} \in \mathbb{F}_q^m \mid f(\mathbf{a}) = 0\}|}{q^m} \leq \frac{r}{q}.$$

We make couple of remarks. First note that the above lemma for $m = 1$ is the degree mantra (Proposition 5.1.5). We note that for every $m \geq 1$ the above lemma is tight (see Exercise 9.3). However, there exists polynomials for which the lemma is not tight (see Exercise 9.4).

Proof of Lemma 9.2.2. Note that the lemma statement is equivalent to saying that the probability that $f(\mathbf{a}) = 0$ is at most $\frac{\deg(f)}{q}$ when $\mathbf{a} = (a_1, \dots, a_m)$ is chosen uniformly at random from $\mathbb{F}_q^m$. We claim that this holds by induction on m .

We will prove the lemma by induction on $m \geq 1$. Note that the base case follows from the degree mantra (Proposition 5.1.5). Now consider the case of $m > 1$ (and we assume that the lemma is true for $m - 1$). To apply inductive hypothesis we first write f as a polynomial in X_m with coefficients that are themselves polynomials in $X_1, \dots, X_{m-1}$. So let

$$f = f_0 X_m^0 + f_1 X_m^1 + \dots + f_t X_m^t,$$

where each $f_i(X_1, \dots, X_{m-1})$ is a polynomial from $\mathbb{F}_q[X_1, \dots, X_{m-1}]$ and $\deg(f_i) \leq r - i$. Furthermore let t be the largest index such that f_t is not zero. Now we consider picking $\mathbf{a} \in \mathbb{F}_q^m$ in two steps: We first pick $(a_1, \dots, a_{m-1})$ uniformly at random from $\mathbb{F}_q^{m-1}$, and then we pick a_m uniformly from $\mathbb{F}_q$. Let

$$f^{(a_1, \dots, a_{m-1})}(X_m) = f_0(a_1, \dots, a_{m-1})X_m^0 + \dots + f_t(a_1, \dots, a_{m-1})X_m^t.$$

We consider two possible events:

$$\mathcal{E}_1 = \{(a_1, \dots, a_m) \mid f_t(a_1, \dots, a_{m-1}) = 0\}$$

and

$$\mathcal{E}_2 = \{((a_1, \dots, a_m) \mid f_t(a_1, \dots, a_{m-1}) \neq 0 \text{ and } f^{(a_1, \dots, a_{m-1})}(a_m) = 0\}.$$

By the inductive hypothesis, we have that

$$\Pr[\mathcal{E}_1] \leq \frac{r-t}{q}, \quad (9.2)$$

since $\deg(f_t) \leq r - t$ and $f_t \neq 0$.

For every $(a_1, \dots, a_{m-1}) \in \mathbb{F}_q^{m-1}$ such that $f_t(a_1, \dots, a_{m-1}) \neq 0$ we also have that the univariate polynomial $f^{(a_1, \dots, a_{m-1})}(X_m)$ is non-zero and of degree at most t , and so by the degree mantra it has at most t roots. It follows that for every such $(a_1, \dots, a_{m-1})$ the probability, over a_m , that $f^{(a_1, \dots, a_{m-1})}(a_m) = 0$ is at most $\frac{t}{q}$. In turn, it now immediately follows that

$$\Pr[\mathcal{E}_2] \leq \frac{t}{q}. \quad (9.3)$$

Finally, we claim that if neither $\mathcal{E}_1$ nor $\mathcal{E}_2$ occur, then $f(\mathbf{a}) \neq 0$. This is immediate from the definitions of $\mathcal{E}_1$ and $\mathcal{E}_2$, since if $f(a_1, \dots, a_m) = 0$, it must either be the case that $f_t(a_1, \dots, a_{m-1}) = 0$ (corresponding to $\mathcal{E}_1$) or it must be that $f_t(a_1, \dots, a_{m-1}) \neq 0$ and $f^{(a_1, \dots, a_{m-1})}(a_m) = 0$ (covered by $\mathcal{E}_2$). Note that this implies that $\Pr_{\mathbf{a}}[f(\mathbf{a}) = 0] \leq \Pr[\mathcal{E}_1 \cup \mathcal{E}_2]$. The lemma now follows from the fact that

$$\Pr_{\mathbf{a}}[f(\mathbf{a}) = 0] \leq \Pr[\mathcal{E}_1 \cup \mathcal{E}_2] \leq \Pr[\mathcal{E}_1] + \Pr[\mathcal{E}_2] \leq \frac{r}{q},$$

where the second inequality follows from the union bound (Proposition 3.1.5) and the final inequality follows from (9.2) and (9.3). $\square$

Comparison with other codes

The lemmas above, while quite precise may not be fully transparent in explaining the asymptotics of the performance of the Reed-Muller codes, or contrast them with other codes we have seen. We mention a few basic facts here to get a clearer comparison.

If we set $m = 1$ and $r = k - 1$, then we get the Reed-Solomon codes evaluated on all of F_q (see Chapter 5). If we set $m = k - 1$, $r = 1$ and $q = 2$, then we get family of extended Hadamard codes (extended by including all Hadamard codewords and their complements). For more on this, see Exercise 5.7.

Thus Reed-Muller codes generalize some previously known codes - some with large alphabets and some with small alphabets. Indeed if we wish the alphabet to be small compared to the block length, then we can pick m to be a constant. For instance if we choose $m = 2$, we get codes of length n over an alphabets of size $\sqrt{n}$, while for a given choice of relative distance δ , the code has rate at least $\frac{(1-\delta)^2}{2}$. In general for larger values of m , the code has alphabet size $n^{1/m}$ and rate at least $\frac{(1-\delta)^m}{m!}$. (See Exercise 9.5.) Thus for small values of m and fixed positive distance $\delta < 1$ there is a rate $R > 0$ such that, by choosing q appropriately large, one get codes on infinitely long block length n and alphabet $n^{1/m}$ with rate R and distance δ , which answers Question 9.0.1 in the affirmative.

This is one of the simplest such families of codes with this feature. We will do better in later in the book (e.g. Chapter 10), and indeed get alphabet size q independent of n with $R > 0$ and $\delta > 0$. But for now this is best we have.

9.3 The case of the binary field

Next we turn to a different extreme of parameter choices for the Reed-Muller codes. Here we fix the alphabet size $q = 2$ and see what varying m and r gets us.

Since we will prove a stronger statement later in Lemma 9.4.1, we only state the distance of the code $\text{RM}(2, m, r)$ below, leaving the proof to Exercise 9.6.

Lemma 9.3.1 (Polynomial distance (binary case)). *Let f be a non-zero polynomial from $\mathbb{F}_2[X_1, \dots, X_m]$ with $\deg_{X_i}(f) \leq 1$ for every $i \in [m]$. Then $|\{\mathbf{a} \in \mathbb{F}_2^m \mid f(\mathbf{a}) \neq 0\}| \geq 2^{m-\deg(f)}$.*

Further, it can be established that the bound in Lemma 9.3.1 is tight (see Exercise 9.7).

The dimension of the code is relatively straightforward to analyze. The dimension is again given by the number of monomials of degree at most r . Since the degree in each variable is either zero or one, this just equals the number of subsets of $[m]$ of size at most r . Thus we have:

Proposition 9.3.2. *For any $r \leq m$, the dimension of the Reed-Muller code $\text{RM}(2, m, r)$ is exactly $\sum_{i=0}^r \binom{m}{i}$.*

Lemma 9.3.1 and Proposition 9.3.2 imply the following result:

Theorem 9.3.3. *For every $r \leq m$, the Reed-Muller code $\text{RM}(2, m, r)$ is a code of block length 2^m , dimension $\sum_{i=0}^r \binom{m}{i}$ and distance 2^{m-r} .*

Again, to get a sense of the asymptotics of this code, we can fix $\tau > 0$ and set $r = \tau \cdot m$ and let $m \rightarrow \infty$. In this case we get a code of block length n (for infinitely many n) with rate roughly $n^{H(\tau)-1}$ and distance $n^{-\tau}$ (see Exercise 9.8). So both the rate and the distance tend to zero at a rate that is a small polynomial in the block length but the code has a constant sized alphabet. (Note that this implies that we have made some progress towards answering Question 8.3.2.)

9.4 The general case

We now turn to the general case, where q is general and r is allowed to be larger than $q-1$. We will try to analyze the dimension and distance of this code. The distance turns out to still have a clean expression, so we will do that first. The dimension does not have a simple expression describing it exactly, so we will give a few lower bounds that may be generally useful (and are often asymptotically tight).

9.4.1 The general case: Distance

Lemma 9.4.1 (Polynomial distance (general case)). *Let f be a non-zero polynomial from $\mathbb{F}_q[X_1, \dots, X_m]$ with $\deg_{X_i}(f) \leq q-1$ for every $i \in [m]$ and $\deg(f) \leq r$. Furthermore, let s, t be the unique non-negative integers such that $t \leq q-2$ and*

$$s(q-1) + t = r.$$

Then

$$|\{\mathbf{a} \in \mathbb{F}_q^m \mid f(\mathbf{a}) \neq 0\}| \geq (q-t) \cdot q^{m-s-1} \geq q^{m-\frac{r}{q-1}}.$$

Hence, $\text{RM}(q, m, r)$ has distance at least $q^{m-\frac{r}{q-1}}$.

Before proving the lemma we make a few observations: The above lemma clearly generalizes both Lemma 9.2.2 (which corresponds to the case $s = 0$) and Lemma 9.3.1 (where $q = 2$, $s = r-1$ and $t = 1$). In the general case the second lower bound is a little simpler to apply and it shows that the probability that a polynomial is non-zero at a uniformly chosen point in $\mathbb{F}_q^m$ is at least $q^{-r/(q-1)}$. Finally, we note that Lemma 9.4.1 is tight for all settings of parameters (see Exercise 9.9).

Proof of Lemma 9.4.1. The proof is similar to the proof of Lemma 9.2.2 except we take advantage of the fact that the degree in a single variable is at most $q-1$. We also need to prove some simple inequalities.

As in the proof of Lemma 9.2.2 we prove that for a random choice of $\mathbf{a} = (a_1, \dots, a_m) \in \mathbb{F}_q^m$, the probability that $f(\mathbf{a}) \neq 0$ is at least

$$(q-t) \cdot q^{-(s+1)}. \quad (9.4)$$

Note that in contrast to the proof of Lemma 9.2.2 we focus on the good events — the polynomial being non-zero — rather than on the bad events.

We prove the lemma by induction on m . In the case of $m = 1$ we have by the degree mantra (Proposition 5.1.5) that the probability that $f(a_1) \neq 0$ is at least $\frac{q-r}{q}$. If $r < q-1$ we have $s = 0$ and $t = r$ and so the expression in (9.4) satisfies

$$(q-t) \cdot q^{-1} = \frac{q-r}{q} \leq \Pr[f(a_1) \neq 0].$$

If $r = q-1$ we have $s = 1$ and $t = 0$, but then again we have that (9.4) equals

$$q \cdot q^{-2} = \frac{q-(q-1)}{q} \leq \Pr[f(a_1) \neq 0],$$

where the inequality follows from the degree mantra.

Now we turn to the inductive step. Assume the hypothesis is true for $(m-1)$ -variate polynomials and let $f = \sum_{i=0}^b f_i X_m^i$ where $f_i \in \mathbb{F}_q[X_1, \dots, X_{m-1}]$ with $f_b \neq 0$. Note $0 \leq b \leq q-1$ and $\deg(f_b) \leq r-b$. Let $\mathcal{E}$ be the event of interest to us, i.e.,

$$\mathcal{E} = \{(a_1, \dots, a_m) \mid f(a_1, \dots, a_m) \neq 0\}.$$

Let

$$\mathcal{E}_1 = \{(a_1, \dots, a_{m-1}) \mid f_b(a_1, \dots, a_{m-1}) \neq 0\}.$$

We first bound $\Pr[\mathcal{E} \mid \mathcal{E}_1]$. Fix $a_1, \dots, a_{m-1}$ such that $f_b(a_1, \dots, a_{m-1}) \neq 0$ and let

$$P(Z) = \sum_{i=0}^b f_i(a_1, \dots, a_{m-1}) Z^i.$$

Note P is a non-zero polynomial of degree b and we have

$$\Pr[f(a_1, \dots, a_m) = 0 \mid a_1, \dots, a_{m-1}] = \Pr_{a_m}[P(a_m) = 0].$$

Since by the degree mantra, a univariate polynomial of degree b has at most b roots, we have

$$\Pr_{a_m}[P(a_m) \neq 0] \geq \frac{q-b}{q}.$$

We conclude

$$\Pr[\mathcal{E} \mid \mathcal{E}_1] \geq 1 - \frac{b}{q}.$$

Next we will bound $\Pr[\mathcal{E}_1]$. This will allow us to lower bound the probability of $\mathcal{E}$ since

$$\Pr[\mathcal{E}] \geq \Pr[\mathcal{E} \text{ and } \mathcal{E}_1] = \Pr[\mathcal{E}_1] \cdot \Pr[\mathcal{E} \mid \mathcal{E}_1].$$

Recall that $\deg(f_b) \leq r-b$. Write $r-b = s'(q-1) + t'$ where $s', t' \geq 0$ and $t' \leq q-2$. By induction we have

$$\Pr[\mathcal{E}_1] = \Pr[f_b(a_1, \dots, a_{m-1}) \neq 0] \geq (q-t') \cdot q^{-(s'+1)}.$$

Putting the two bounds together, we get

$$\Pr[\mathcal{E}] \geq \Pr[\mathcal{E}|\mathcal{E}_1] \cdot \Pr[\mathcal{E}_1] \geq \frac{q-b}{q} \cdot (q-t') \cdot q^{-(s'+1)}.$$

We are now left with a calculation to verify that the bound above is indeed lower bounded by $(q-t) \cdot q^{-(s+1)}$ and we do so in Claim 9.4.2 using the facts that $t, t' \leq q-2$, $b \leq q-1$, $r = s(q-1) + t$, and $r-b = s'(q-1) + t'$. In the claim further below (Claim 9.4.3), we also prove $(q-t) \cdot q^{-(s+1)} \geq q^{-r/(q-1)}$ and this concludes the proof of the lemma. $\square$

Claim 9.4.2. *If q, r, s, t, s', t', b are non-negative integers such that $r = s(q-1) + t$, $r-b = s'(q-1) + t'$, $t, t' \leq q-2$ and $b \leq q-1$ then we have*

$$\frac{q-b}{q} \cdot (q-t') \cdot q^{-(s'+1)} \geq (q-t) \cdot q^{-(s+1)}.$$

Proof. The proof breaks up into two cases depending on $s-s'$. Note that an equivalent definition of s and s' are that these are the quotients when we divide r and $r-b$ respectively by $q-1$. Since $0 \leq b \leq q-1$, it follows that either $s' = s$ or $s' = s-1$. We consider the two cases separately.

If $s = s'$ we have $t = t' + b$ and then it suffices to show that

$$\frac{q-b}{q} \cdot (q-t') \geq q - (t' + b).$$

In turn this is equivalent to showing

$$(q-b)(q-t') \geq q(q - (t' + b)).$$

But this is immediate since the expression on the left is

$$(q-b)(q-t') = q^2 - (b+t')q + bt' = q(q - (b+t')) + bt' \geq q(q - (b+t')),$$

where the final inequality uses $bt' \geq 0$.

If $s = s' + 1$ we have a bit more work. Here we have $t + q - 1 = t' + b$ and it suffices to show that

$$\frac{q-b}{q} \cdot (q-t') \cdot q \geq (q-t) = (2q - (t' + b + 1)).$$

Write $q-b = \alpha$ and $q-t' = \beta$. The expression on the left above simplifies to $\alpha\beta$ and on the right to $\alpha + \beta - 1$. Since $b, t' \leq q-1$, we also have $\alpha, \beta \geq 1$. So it suffices to show that $\alpha\beta \geq \alpha + \beta - 1$. This is true since $\alpha\beta = \alpha + \alpha(\beta-1)$ and we have $\alpha(\beta-1) \geq \beta-1$ since $\alpha \geq 1$ and $\beta-1 \geq 0$.

We thus conclude that the inequality holds for both $s = s'$ and $s = s' + 1$ and this yields the claim. $\square$

Claim 9.4.3. *Let q, r, s, t be non-negative real numbers such that $q \geq 2$, $r = s(q-1) + t$ and $t \leq q-2$. Then*

$$(q-t) \cdot q^{-(s+1)} \geq q^{-r/(q-1)}.$$

We remark that while the inequality is quite useful, the proof below is not particularly insightful. We include it for completeness, but we recommend that the reader skip it unless necessary.

Proof of Claim 9.4.3. We have four parameters in the inequality above. We will simplify it in steps removing parameters one at a time. First we get rid of r by substituting $r = s(q-1) + t$. So it suffices to prove:

$$(q-t) \cdot q^{-(s+1)} \geq q^{-(s(q-1)+t)/(q-1)} = q^{-s} \cdot q^{-t/(q-1)}.$$

We can get rid of q^{-s} from both sides (since the remaining terms are non-negative) and so it suffices to prove:

$$\frac{q-t}{q} \geq q^{-t/(q-1)}.$$

Let $f_q(t) = \frac{t}{q} + q^{-t/(q-1)} - 1$. The inequality above is equivalent to proving $f_q(t) \leq 0$ for $0 \leq t \leq q-2$. We use some basic calculus to prove the above. Note that the first and second derivatives of f_q with respect to t are given by $f'_q(t) = \frac{1}{q} - \frac{\ln q}{q-1} q^{-t/(q-1)}$ and $f''_q(t) = (\ln(q)/(q-1))^2 q^{-t/(q-1)}$. In particular the second derivative is always positive which means $f_q(t)$ is maximized at one of the two end points of the interval $t \in [0, q-2]$. We have $f_q(0) = 0 \leq 0$ as desired and so it suffices to prove that

$$f_q(q-2) = q^{-(q-2)/(q-1)} - \frac{2}{q} \leq 0.$$

Multiplying the expression above by q we have that it suffices to show $q^{1/(q-1)} \leq 2$ which in turn is equivalent to proving $q \leq 2^{q-1}$ for every $q \geq 2$. The final inequality follows easily from Bernoulli's inequality (Lemma B.1.4) $1 + kx \leq (1+x)^k$ which holds for every $x \geq -1$ and $k \geq 1$. In our case we substitute $x = 1$ and $k = q-1$ to conclude $q \leq 2^{q-1}$ as desired. $\square$

9.4.2 The general case: Dimension

For integers q, m, r let

$$S_{q,m,r} = \left\{ \mathbf{d} = (d_1, \dots, d_m) \in \mathbb{Z}^m \mid 0 \leq d_i \leq q-1 \text{ for all } i \in [m] \text{ and } \sum_{i=1}^m d_i \leq r \right\} \quad (9.5)$$

and let

$$K_{q,m,r} = |S_{q,m,r}|.$$

We start with the following, almost tautological, proposition.

Proposition 9.4.4. *For every prime power q and integers $m \geq 1$ and $r \geq 0$, the dimension of the code $\text{RM}(q, m, r)$ is $K_{q,m,r}$.*

Proof. Follows from the fact that for every $\mathbf{d} = (d_1, \dots, d_m) \in S_{q,m,r}$ the associated monomial $\mathbf{X}^{\mathbf{d}} = X_1^{d_1} \dots X_m^{d_m}$ is a monomial of degree at most r and individual degree at most $q-1$. Thus these monomials (i.e., their evaluations) form a basis for the Reed-Muller code $\text{RM}(q, m, r)$. (See Exercise 9.10.) $\square$

The definition of $K_{q,m,r}$ does not give a good hint about its growth so below we give a few bounds on $K_{q,m,r}$ that help estimate its growth. Specifically the proposition below gives a lower bound $K_{q,m,r}^-$ and an upper bound $K_{q,m,r}^+$ on $K_{q,m,r}$ that are (1) given by simple expressions and (2) within polynomial factors of each other for every setting of q, m , and r .

Proposition 9.4.5. *For integers $q \geq 2, m \geq 1$ and $r \geq 0$, let*

$$K_{q,m,r}^+ \triangleq \min \left\{ q^m, \binom{m+r}{r} \right\}$$

and let

$$K_{q,m,r}^- \triangleq \begin{cases} \max \left\{ q^m/2, q^m - K_{q,m,(q-1)m-r}^+ \right\} & \text{if } r \geq (q-1)m/2 \\ \max \left\{ \binom{m}{r}, \frac{1}{2} \left(\left\lfloor \frac{2r+m}{m} \right\rfloor \right)^m \right\} & \text{if } r < (q-1)m/2 \end{cases}$$

Then there are universal constants c_1, c_2 ($c_1 < 3.1$ and $c_2 < 8.2$ suffice) such that

$$K_{q,m,r}^- \leq K_{q,m,r} \leq K_{q,m,r}^+ \leq c_1 \cdot (K_{q,m,r}^-)^{c_2}$$

Proof. We tackle the inequalities in order of growing complexity of the proof. In our bounds we use the fact that $K_{q,m,r}$ is monotone non-decreasing in q as well as r (when other parameters are fixed)– see Exercise 9.11.

First we prove $K_{q,m,r} \leq K_{q,m,r}^+$. On the one hand we have

$$K_{q,m,r} \leq K_{q,m,(q-1)m} = q^m,$$

which follows by ignoring the total degree restriction and on the other hand we have

$$K_{q,m,r} \leq K_{r,m,r} = \binom{m+r}{r},$$

whereas here we ignored the individual degree restriction.

Next we show $K_{q,m,r}^- \leq K_{q,m,r}$. First we consider the case $r \geq (q-1)m/2$. Here we argue via symmetry. Consider a map that maps vectors $\mathbf{d} = (d_1, \dots, d_m) \in \mathbb{Z}^m$ with $0 \leq d_i < q$ to $\bar{\mathbf{d}} = (q-1-d_1, \dots, q-1-d_m)$. The map $\mathbf{d} \mapsto \bar{\mathbf{d}}$ is a one-to-one map which maps vectors with $\sum_i d_i > r$ to vectors with $\sum_i \bar{d}_i < (q-1)m - r$. In other words either $\mathbf{d} \in \{0, \dots, q-1\}^m$ is in $S_{q,m,r}$ or $\bar{\mathbf{d}} \in S_{q,m,(q-1)m-r}$, thus establishing

$$K_{q,m,r} = q^m - K_{q,m,(q-1)m-r}.$$

Since $r \geq (q-1)m/2$ we have $(q-1)m - r \leq r$ and so

$$K_{q,m,r} \geq K_{q,m,(q-1)m-r},$$

which in turn implies

$$K_{q,m,r} \geq q^m/2.$$

This establishes $K_{q,m,r} \geq K_{q,m,r}^-$ when $r \geq (q-1)m/2$. Next, turning to the case $r < (q-1)m/2$, first let $q' = \lfloor \frac{2r+m}{m} \rfloor$. We have

$$K_{q,m,r} \geq K_{q',m,r} \geq (q')^m/2$$

since $r \geq (q'-1)m/2$, and this yields

$$K_{q,m,r} \geq (q')^m/2 = \frac{1}{2} \left(\left\lfloor \frac{2r+m}{m} \right\rfloor \right)^m.$$

Finally we also have

$$K_{q,m,r} \geq K_{2,m,r} = \sum_{i=0}^r \binom{m}{i} \geq \binom{m}{r},$$

thus establishing $K_{q,m,r} \geq K_{q,m,r}^-$ when $r < (q-1)m/2$.

Finally we turn to the inequalities showing $K_{q,m,r}^+ \leq c_1 \cdot (K_{q,m,r}^-)^{c_2}$. If $r \geq (q-1)m/2$ we have

$$\frac{q^m}{2} \leq K_{q,m,r}^- \leq K_{q,m,r}^+ \leq q^m$$

establishing $K_{q,m,r}^+ \leq 2K_{q,m,r}^-$. Next we consider the case $r < m/2$. In this case we have

$$K_{q,m,r}^- \geq \binom{m}{r} \geq (m/r)^r \geq 2^r.$$

On the other hand we also have

$$\binom{m+r}{r} \leq \left(\frac{e(m+r)}{r} \right)^r \leq \left(\frac{e \cdot (3/2) \cdot m}{r} \right)^r = \left(\frac{3e}{2} \right)^r \cdot \left(\frac{m}{r} \right)^r.$$

From $2^r \leq K_{q,m,r}^-$ we get $\left(\frac{3e}{2} \right)^r \leq (K_{q,m,r}^-)^{\log_2(3e/2)}$. Combining with $\left(\frac{m}{r} \right)^r \leq K_{q,m,r}^-$ and $K_{q,m,r}^+ \leq \binom{m+r}{r}$ we get

$$K_{q,m,r}^+ \leq \left(\frac{3e}{2} \right)^r \cdot \left(\frac{m}{r} \right)^r \leq (K_{q,m,r}^-)^{1+\log_2(3e/2)}.$$

Finally, we consider the case $m/2 \leq r < (q-1)m/2$. In this range we have

$$\left\lfloor \frac{2r+m}{m} \right\rfloor = 1 + \left\lfloor \frac{2r}{m} \right\rfloor \geq 1 + \frac{r}{m} = \frac{m+r}{m}.$$

Thus

$$K_{q,m,r}^- \geq \frac{1}{2} \left(\left\lfloor \frac{2r+m}{m} \right\rfloor \right)^m \geq \frac{1}{2} \left(\frac{m+r}{m} \right)^m \geq \frac{1}{2} \left(\frac{3}{2} \right)^m.$$

On the other hand we have

$$K_{q,m,r}^+ \leq \binom{m+r}{m} \leq \left(\frac{e(m+r)}{m} \right)^m = e^m \cdot \left(\frac{m+r}{m} \right)^m.$$

Again we have $\left(\frac{m+r}{m} \right)^m \leq 2K_{q,m,r}^-$ and $e^m \leq (2K_{q,m,r}^-)^{\log_2(3e/2)}$ and so $K_{q,m,r}^+ \leq (2K_{q,m,r}^-)^{1+\log_2(3e/2)}$. Thus in all cases we have $K_{q,m,r}^+ \leq c_1 \cdot (K_{q,m,r}^-)^{c_2}$ for $c_2 = 1 + \log_2(3e/2) < 3.1$ and $c_1 = 2^{c_2} < 8.2$, as desired. $\square$

We now give a few examples of codes that can be derived from the bounds above, to illustrate the variety offered by Reed-Muller codes. In each of the cases we set one or more of the parameters among alphabet size, rate, (relative) distance or absolute distance to a constant and explore the behavior in the other parameters. In all cases we use Lemma 9.4.1 to lower bound the distance and Proposition 9.4.5 to lower bound the dimension.

Example 9.4.6 (RM Codes of constant alphabet size and (relative) distance.). Fix q and $r < q-1$ and consider $m \rightarrow \infty$. Then the Reed-Muller codes $\text{RM}(q, m, r)$ are $[N, K, D]_q$ codes with block length $N = q^m$, distance $D = \delta \cdot N$ for $\delta = 1 - r/q$, with dimension

$$K \geq \binom{m}{r} \geq \left(\frac{\log_q N}{r} \right)^r.$$

In other words Reed-Muller codes yield codes of constant alphabet size and relative distance with dimension growing as an arbitrary polynomial in the logarithm of the block length.

Example 9.4.7 (Binary RM Codes of rate close to 1 with constant (absolute) distance.). Fix $q = 2$ and d and let $m \rightarrow \infty$. Then the Reed-Muller codes $\text{RM}(2, m, m-d)$ are $[N, K, D]_2$ codes with $N = 2^m$, $D = 2^d$ and

$$K \geq N - \binom{\log_2 N + d}{d} \geq N - (\log_2 N)^d.$$

(See Exercise 9.12 for bound on K .) Note that the rate $\rightarrow 1$ as $N \rightarrow \infty$.

Example 9.4.8 (RM codes of constant rate and relative distance over polynomially small alphabets.). Given any $\varepsilon > 0$ and let $m = \lceil \frac{1}{\varepsilon} \rceil$ and now consider $q \rightarrow \infty$ with $r = q/2$. Then the Reed-Muller codes $\text{RM}(q, m, r)$ are $[N, K, D]_q$ codes with $N = q^m$, $D = \frac{N}{2}$ and

$$K \geq \frac{1}{2} \left(\frac{q+m}{m} \right)^m \geq \frac{1}{2m^m} \cdot N.$$

Expressed in terms of N and ε , the codes have length N , dimension $\Omega(\varepsilon^{1/\varepsilon}) \cdot N$ and relative distance $1/2$ over an alphabet of size N^ε .

Another natural regime is to consider the case of constant rate $1/2$: see Exercise 9.13 for more.

Finally we mention a range of parameters that has been very useful in the theory of computer science. Here the alphabet size is growing with N , but very slowly. But the code has a fixed relative distance and dimension that is polynomially related to the block length.

Example 9.4.9 (RM Codes over polylogarithmic alphabets with polynomial dimension.). Given $0 < \varepsilon < 1$, let $q \rightarrow \infty$ and let $r = q/2$ and $m = q^\varepsilon$. Then the Reed-Muller codes $\text{RM}(q, m, r)$ are $[N, K, D]_q$ codes with $N = q^m$, $D = \frac{N}{2}$ and

$$K \geq \frac{1}{2} \left(\frac{q+m}{m} \right)^m \geq \frac{1}{2} (q^{1-\varepsilon})^m = \frac{1}{2} \cdot N^{1-\varepsilon}.$$

Expressed in terms of N and ε , the codes have length N , dimension $\Omega(N^{1-\varepsilon})$ and relative distance $1/2$ over an alphabet of size $(\log N)^{1/\varepsilon}$. (See Exercise 9.14 for claim on the bound on q .)

9.5 Exercises

Exercise 9.1. Argue that any $\text{RM}(q, m, r)$ is a linear code.

Exercise 9.2. Argue that for D as defined in (9.1), we have

$$|D| = \binom{m+r}{r}.$$

Exercise 9.3. Show that Lemma 9.2.2 is tight in the sense that for every prime power q and integers $m \geq 1$ and $1 \leq r \leq q-1$, there exists a polynomial with exactly $r \cdot q^{m-1}$ roots.

Exercise 9.4. Show that Lemma 9.2.2 is not tight for most polynomials. In particular show that for every prime power q and integers $m \geq 1$ and $1 \leq r \leq q-1$, a random polynomial in $\mathbb{F}_q[X_1, \dots, X_m]$ of degree r has q^{m-1} expected number of roots.

Exercise 9.5. Show that the Reed-Muller codes of Section 9.2 give rise to codes of relative distance δ (for any $0 < \delta < 1$) and block length n such that they have alphabet size of $\sqrt[n]{n}$ and rate at least $\frac{(1-\delta)^m}{m!}$.

Exercise 9.6. Prove Lemma 9.3.1.

Exercise 9.7. Prove that the lower bound in Lemma 9.3.1 is tight.

Exercise 9.8. Show that there exists a binary RM code with block length n , rate $n^{H(\tau)-1}$ and relative distance $n^{-\tau}$ for any $0 < \tau < 1/2$.

Exercise 9.9. Prove that the (first) lower bound in Lemma 9.4.1 is tight for all settings of the parameters.

Exercise 9.10. Prove that the evaluations of $\mathbf{X}^{\mathbf{d}}$ for every $\mathbf{d} \in S_{q,m,r}$ (as in (9.5)) form a basis for $\text{RM}(q, m, r)$.

Exercise 9.11. Argue that $K_{q,m,r}$ is monotone non-decreasing in q as well as r (when other parameters are fixed).

Exercise 9.12. Argue the claimed bound on K in Example 9.4.7.

Exercise 9.13. Figure out a RM code that has rate $\frac{1}{2}$ and has as large a distance as possible and as small an alphabet as possible.

Exercise 9.14. Prove the claimed bound on q in Example 9.4.9.

Exercise 9.15. In this problem we will talk about the dual of Reed-Muller codes, which turn out to be Reed-Muller codes (with a different degree) themselves. We do so in a sequence of sub-problems:

1. Show that for $1 \leq j \leq q-1$

$$\sum_{\alpha \in \mathbb{F}_q} \alpha^j \neq 0$$

if and only if $j = q-1$.

Hint: Use Exercise 2.3.

2. Argue that for any $m \geq 1$ and $1 \leq j_1, \dots, j_m \leq q-1$,

$$\sum_{(c_1, \dots, c_m) \in \mathbb{F}_q^m} \prod_{i=1}^m c_i^{j_i} = 0$$

if and only if $j_1 = j_2 = \dots = j_m = q-1$.

3. Using the above or otherwise, show that for any $0 \leq r < (q-1) - s$, we have

$$\text{RM}(q, m, r)^\perp = \text{RM}(q, m, m(q-1) - r - 1).$$

9.6 Bibliographic Notes

The name Reed-Muller codes goes back to the first two papers introducing this code. The binary version of these codes were invented by Muller in [118], and Reed gave a non-trivially fast decoder for these codes in [128]. The latter work is especially significant in the algorithmic context in that it is the first setting allowing a polynomial time decoder correcting errors (up to half the distance) where the brute force decoders (enumerating all possible codeword or all possible error locations) take super polynomial time in the length of the code.

The polynomial distance lemmas (Lemmas 9.3.1, 9.2.2, and 9.4.1) date back at least to Ore [120]. Versions of this lemma appear in Muller [118] and Schwartz [137], Zippel [164] and DeMillo and Lipton [38].

Chapter 10

From Large to Small Alphabets: Code Concatenation

Recall Question 8.3.2: Is there an explicit asymptotically good binary code (that is, rate $R > 0$ and relative distance $\delta > 0$)? Recall that the word “explicit code” was defined in Definition 6.3.4 and then strengthened in Definition 6.3.5. Specifically Definition 6.3.4 deems a linear code to be explicit if we can construct its generator matrix in polynomial time. And Definition 6.3.5 defines a linear code to be strongly explicit if we can compute every entry of the generator matrix in poly-logarithmic time. In this chapter we will seek asymptotically good binary codes for each of these definitions of explicitness. To do we will introduce a new way of combining codes called “code concatenation”.

Before we get into the main tool introduced in this chapter, let us briefly recall all the explicit binary codes we have seen so far. In Section 2.4 we introduced the Hamming code, which has rate $R = 1 - O(\log n/n)$ and relative distance $\delta = O(1/n)$ for block length n . In Section 2.6 we introduced the Hadamard code, which has rate $R = O(\log n/n)$ and relative distance $1/2$. Both of these codes have extremely good values of one of two parameters, rate or relative distance, at the expense of the other parameter, which has an extremely poor value. It turns out we can turn Reed-Solomon codes, which are naturally codes over a large alphabet, into binary codes with some loss in relative distance. Specifically given an $[n, k, n-k+1]_q$ Reed-Solomon code we can write elements of $\mathbb{F}_q$ as $\lceil \log q \rceil$ bit strings and this gives a code that maps messages in $\mathbb{F}_q^k$ to $\{0, 1\}^{n \lceil \log q \rceil}$. The distance of this code is at least $n-k+1$. And if $q = n = 2^s$ then we can actually make this a linear code over $\mathbb{F}_2$ giving a $[n \log n, k \log n, n-k+1]_2$ code. (See Exercise ??.) We refer to these codes as binary-RS codes. Thus, if we set $k = n/2$ for instance, then we get a code of rate $1/2$ with relative distance $\Omega(1/\log n)$. This yields a code which seems to get closer to the target of being asymptotically good, but still falls short.¹ Table 10.1 summarizes these codes.

In this chapter we will try to improve the performance of the binary-RS codes by getting a better (and general) technique to convert large alphabet codes into binary codes. Note that

¹One way to capture the requirement that a code of rate R and distance δ is asymptotically good is to require $R\delta = \Omega(1)$. The Hamming code achieved $R\delta = \Theta(1/n)$ and the Hadamard code was mildly better with product $\Theta(\log n/n)$. The binary RS codes achieve $R\delta = \Theta(1/\log n)$ which is significantly better.

Code	R	δ	$R \cdot \delta$
Hamming	$1 - O\left(\frac{\log n}{n}\right)$	$O\left(\frac{1}{n}\right)$	$O\left(\frac{1}{n}\right)$
Hadamard	$O\left(\frac{\log n}{n}\right)$	$\frac{1}{2}$	$O\left(\frac{\log n}{n}\right)$
Binary-RS	$\frac{1}{2}$	$O\left(\frac{1}{\log n}\right)$	$O\left(\frac{1}{\log n}\right)$

Table 10.1: Strongly explicit binary codes that we have seen so far.

the reason for the (relatively) poor distance of the binary-RS codes is that the bit vectors corresponding to two different symbols of $\mathbb{F}_{2^s}$ may only differ by one bit. Thus if $x, y \in \mathbb{F}_{2^s}^n$ differ in d positions, their binary representations as element of $\mathbb{F}_2^{ns}$ may still differ in only d positions.

Is there a way to do better? What would be ideal is a way to represent elements of $\mathbb{F}_{2^s}$ as $O(s)$ bit strings, but with the property that representations of two different elements of $\mathbb{F}_{2^s}$ differ in many, specifically $\Omega(s)$, coordinates. But this is exactly the notion of an error-correcting code! And the requirements on the length and distance correspond to requiring the code to be asymptotically good. So all we need, to get an explicit asymptotically good binary error-correcting code, is an asymptotically good binary error-correcting code.

The last sentence above may seem to have landed us in the same spot where we started, but closer examination reveals we have made progress. Specifically we may not need the code being used to represent elements of $\mathbb{F}_{2^s}$ (we will call this the “inner code” from now) to be explicit. Specifically our eventual goal is to build an “outer code” of length 2^s and we have time polynomial in 2^s to build the generator matrix to get an explicit outer code. Our inner code on the other hand has messages of length s and we have time polynomial in 2^s to build its generator matrix. This is a much weaker requirement than the requirement of polynomial time construction and this is what leads to the codes of this section.

In what follows we introduce the notion of code concatenation that formalize the steps outlined above, and use them to build explicit codes. We start with the most basic form of this notion in Section 10.1 and use them to get explicit asymptotically-good binary codes. Then, in Section 10.3, we use a more involved form of code concatenation and analysis to get strongly explicit asymptotically-good binary codes.

10.1 Code Concatenation: The basic idea

A (basic) concatenated code is constructed from two codes: an *outer code* (which we will denote C_{out}) and an *inner code* (which we will denote C_{in}). We first use C_{out} to encode the message to get $(c_0, \dots, c_{N-1})$ and then use the C_{in} to encode each symbol c_i in the codeword in C_{out} .² This construction is also illustrated in Figure 10.1.

²Note that unlike the usual meaning of concatenation (e.g. string concatenation in computing), code concatenation does not concatenate codewords as strings. Rather, this is a recursive construction where the outer code is used to reduce the block length we need from the inner code.

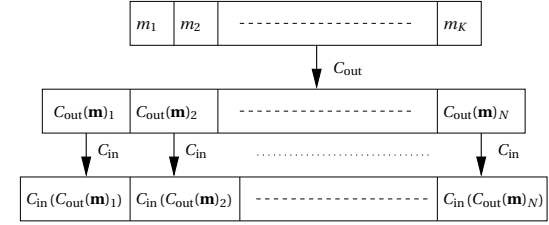


Figure 10.1: Concatenated code $C_{\text{out}} \circ C_{\text{in}}$.

We now formally define a concatenated code. For $q \geq 2$, $k \geq 1$ and $Q = q^k$, consider two codes which we call *outer code* and *inner code*:

$$C_{\text{out}} : [Q]^K \rightarrow [Q]^N,$$

$$C_{\text{in}} : [q]^k \rightarrow [q]^n.$$

Note that the alphabet size of C_{out} exactly matches the number of messages for C_{in} , which means that we can have a bijection between $[Q] = [q^k]$ and $[q]^k$ (this means we can use C_{in} to encode any symbol in a codeword in C_{out}). Then given $\mathbf{m} = (m_1, \dots, m_K) \in [Q]^K$, we have the code $C_{\text{out}} \circ C_{\text{in}} : [q]^{kK} \rightarrow [q]^{nN}$ defined as

$$C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}) = (C_{\text{in}}(C_{\text{out}}(\mathbf{m})_1), \dots, C_{\text{in}}(C_{\text{out}}(\mathbf{m})_N)),$$

where

$$C_{\text{out}}(\mathbf{m}) = (C_{\text{out}}(\mathbf{m})_1, \dots, C_{\text{out}}(\mathbf{m})_N).$$

We now look at some properties of a concatenated code.

Theorem 10.1.1. *If C_{out} is an $(N, K, D)_{q^k}$ code and C_{in} is an $(n, k, d)_q$ code, then $C_{\text{out}} \circ C_{\text{in}}$ is an $(nN, kK, dD)_q$ code. In particular, if C_{out} (C_{in} resp.) has rate R (r resp.) and relative distance δ_{out} (δ_{in} resp.) then $C_{\text{out}} \circ C_{\text{in}}$ has rate Rr and relative distance $\delta_{\text{out}} \cdot \delta_{\text{in}}$.*

Proof. The first claim immediately implies the second claim on the rate and relative distance of $C_{\text{out}} \circ C_{\text{in}}$. The claims on the block length, dimension and alphabet of $C_{\text{out}} \circ C_{\text{in}}$ follow from the definition.³ Next, we show that the distance is at least dD . Consider arbitrary $\mathbf{m}_1 \neq \mathbf{m}_2 \in [Q]^K$. Then by the fact that C_{out} has distance D , we have

$$\Delta(C_{\text{out}}(\mathbf{m}_1), C_{\text{out}}(\mathbf{m}_2)) \geq D.$$

Define

$$S = \{i \in [N] \mid C_{\text{out}}(\mathbf{m}_1)_i \neq C_{\text{out}}(\mathbf{m}_2)_i\},$$

³Technically, we need to argue that the q^{kK} messages map to distinct codewords to get the dimension of kK . However, this follows from the fact, which we will prove soon, that $C_{\text{out}} \circ C_{\text{in}}$ has distance $dD \geq 1$, where the inequality follows for $d, D \geq 1$.

which along with the lower bound on distance above implies

$$|S| \geq D. \quad (10.1)$$

Then for each position $i \in S$, we have

$$\Delta(C_{\text{in}}(C_{\text{out}}(\mathbf{m}_1)_i), C_{\text{in}}(C_{\text{out}}(\mathbf{m}_2)_i)) \geq d, \quad (10.2)$$

as C_{in} has distance d . Since there are at least D such positions (from (10.1)), (10.2) implies

$$\Delta(C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}_1), C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}_2)) \geq dD.$$

The proof is complete as the choices of $\mathbf{m}_1$ and $\mathbf{m}_2$ were arbitrary. $\square$

If C_{in} and C_{out} are linear codes, then so is $C_{\text{out}} \circ C_{\text{in}}$. Indeed, this can be proved for example, by defining a generator matrix for $C_{\text{out}} \circ C_{\text{in}}$ in terms of the generator matrices of C_{in} and C_{out} . The proof is left as an exercise.

10.2 Zyablov Bound

We now instantiate outer and inner codes in Theorem 10.1.1 to obtain a new lower bound on the rate given a relative distance. We'll initially just state the lower bound (which is called the Zyablov bound) and then we will consider the explicitness of such codes.

We begin with the instantiation of C_{out} . Note that this is a code over a large alphabet, and we have seen an optimal code over large enough alphabet: Reed-Solomon codes (Chapter 5). Recall that the Reed-Solomon codes are optimal because they meet the Singleton bound 4.3.1. Hence, let us assume that C_{out} meets the Singleton bound with rate of R , i.e. C_{out} has relative distance $\delta_{\text{out}} \geq 1 - R$. For $C_{\text{out}} \circ C_{\text{in}}$ to be an asymptotically good code, C_{in} needs to have rate $r > 0$ and relative distance $\delta_{\text{in}} > 0$ (i.e. C_{in} also needs to be an asymptotically good code). As noted earlier, this is precisely the kind of code we are looking for to answer Question 8.3.2! However, the saving grace will be that k can be much smaller than the block length of the concatenated code and hence, we can spend “more” time searching for such an inner code. But we set aside the question of explicitness for now and ask — what kind of parameters a concatenated code can give.

Fix $\varepsilon > 0$. Suppose C_{in} meets the GV bound (Theorem 4.2.1) with rate of r and thus has relative distance $\delta_{\text{in}} \geq H_q^{-1}(1 - r) - \varepsilon$. Then by Theorem 10.1.1, $C_{\text{out}} \circ C_{\text{in}}$ has rate of rR and $\delta = (1 - R)(H_q^{-1}(1 - r) - \varepsilon)$. Expressing R as a function of δ and r , we get the following:

$$R = 1 - \frac{\delta}{H_q^{-1}(1 - r) - \varepsilon}.$$

Then optimizing over the choice of r , we get that the rate of the concatenated code satisfies

$$\mathcal{R} \geq \lim_{\varepsilon \rightarrow 0} \left\{ \max_{0 < r < 1 - H_q(\delta + \varepsilon)} r \left(1 - \frac{\delta}{H_q^{-1}(1 - r) - \varepsilon} \right) \right\},$$

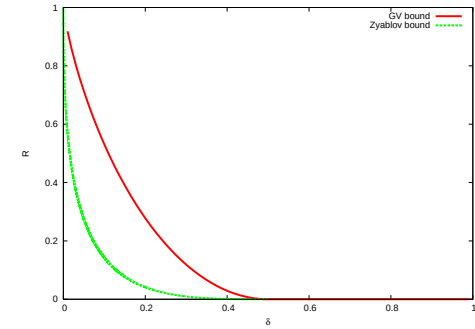


Figure 10.2: The Zyablov bound for binary codes. For comparison, the GV bound is also plotted.

where the bound of $r < 1 - H_q(\delta + \varepsilon)$ is necessary to ensure that $\mathcal{R} > 0$. This lower bound on the rate is called the Zyablov bound. See Figure 10.2 for a plot of this bound for binary codes.

To get a feel for how the bound behaves, consider the case when $\delta = \frac{1}{2} - \gamma$ where $\gamma \rightarrow 0$. We claim that the Zyablov bound states that $\mathcal{R} \geq \Omega(\gamma^3)$. (Recall that the GV bound for the same δ has a rate of $\Omega(\gamma^2)$.) The proof of this claim is left as an exercise (see Exercise 10.3).

Note that the Zyablov bound implies that for every $\delta > 0$, there exists a (concatenated) code with rate $R > 0$. However, we already knew about the existence of an asymptotically good code by the GV bound (Theorem 4.2.1). Thus, a natural question to ask is the following:

Question 10.2.1. *Does there exist an explicit code on the Zyablov bound?*

We will focus on linear codes in seeking an answer to the question above because linear codes have polynomial size representation. Let C_{out} be an $[N, K]_Q$ Reed-Solomon code where $N = Q - 1$ (evaluation points being $\mathbb{F}_Q^*$ with $Q = q^k$). This implies that $k = \Theta(\log N)$. However, we still need an efficient construction of an inner code that lies on the GV bound. We do not expect to construct such a C_{in} in time $\text{poly}(k)$ as that would answer Open Question 8.3.2! However, since $k = O(\log N)$, note that an exponential time (in k) algorithm is still a polynomial (in N) time algorithm.

There are two options for this exponential (in k) time construction algorithm for C_{in} :

- Perform an exhaustive search among all generator matrices for one satisfying the required property for C_{in} . One can do this because the Varshamov bound (Theorem 4.2.1) states that there exists a linear code which lies on the GV bound. This will take $q^{O(kn)}$ time. Using $k = rn$ (or $n = O(k)$), we get $q^{O(kn)} = q^{O(k^2)} = N^{O(\log N)}$, which is upper bounded by

$(nN)^{O(\log(nN))}$, a quasi-polynomial time bound.

- The second option is to construct C_{in} in $q^{O(n)}$ time and thus use $(nN)^{O(1)}$ time overall. See Exercise 4.6 for one way to construct codes on the GV bound in time $q^{O(n)}$.

Using the latter construction we thus get an explicit family of codes on the Zyablov bound.

Theorem 10.2.1. *For every prime power q , there exists an explicit q -ary code that achieves the Zyablov bound. Specifically for every $\epsilon > 0$ there exists an algorithm that, given $\delta \in [0, 1/2)$ and n , outputs in time polynomial in n , the generator matrix of a code of block length n and rate*

$$\mathcal{R} \geq \max_{0 < r < 1 - H_q(\delta + \epsilon)} r \left(1 - \frac{\delta}{H_q^{-1}(1 - r) - \epsilon} \right).$$

This answers Question 10.2.1 in the affirmative.

A somewhat unsatisfactory aspect of this construction (in the proof of Theorem 10.2.1) is that one needs a brute force search for a suitable inner code (which sufficed though for the polynomial construction time). A natural followup question is

Question 10.2.2. *Does there exist a strongly explicit asymptotically good code?*

We tackle this question in the next section using a more sophisticated form of concatenation.

10.3 Advanced Concatenation and Strongly Explicit Constructions

In this section we describe strongly explicit codes that are built using the idea of concatenation, but with some extra twists. The specific family of codes we describe are due to Justesen and we refer to the codes as the *Justesen codes*. The key insight behind these codes is that the arguments in the previous section can be generalized while still preserving the parameters of the Zyablov bound. Specifically we may:

1. Pick N different inner codes, one for each of the N coordinates of the outer codeword.
2. It suffices if most (but not necessarily all) of these inner codes lie on the GV bound.

The reason that these options are useful are that we already know how to design families of codes “explicitly” with the property that most of the codes are very good. As a concrete example of such an ensemble, the ensemble of all linear codes have this property (this is exactly what Varshamov proved). It is only when we need to pick one that is good that we have a problem. But by now allowing an entire ensemble of “inner codes” we have managed to reduce the task of code construction to a task we know how to solve.

The one catch with implementing the above idea is that the desired ensemble of inner codes must have exactly N such codes, each containing N codewords (of length $O(\log N)$). This is much smaller than, say, the number of linear codes of length $O(\log N)$ — the number of such codes is $2^{\Theta(\log^2 N)}$ which is in fact greater than any polynomial in N .

In what follows we will describe a new construction of an ensemble and this will lead us to the Justesen code. We now turn to the details.

The Justesen code is specified by an $(N, K, D)_{q^k}$ outer code (C_{out}) and N different inner codes $(C_{\text{in}}^i : 1 \leq i \leq N)$. Formally, the concatenation of these codes, denoted by $C_{\text{out}} \circ (C_{\text{in}}^1, \dots, C_{\text{in}}^N)$, is defined as follows: given a message $\mathbf{m} \in [q^k]^K$, let the outer codeword be denoted by $(c_1, \dots, c_N) \stackrel{\text{def}}{=} C_{\text{out}}(\mathbf{m})$. Then $C_{\text{out}} \circ (C_{\text{in}}^1, \dots, C_{\text{in}}^N)(\mathbf{m}) = (C_{\text{in}}^1(c_1), C_{\text{in}}^2(c_2), \dots, C_{\text{in}}^N(c_N))$.

For the inner family of codes we will use the following result, which shows that there is a set or *ensemble* of codes most of which lie on the Gilbert-Varshamov bound.

Theorem 10.3.1. *Let $\epsilon > 0$. There exists an ensemble of inner codes $C_{\text{in}}^1, C_{\text{in}}^2, \dots, C_{\text{in}}^N$ of rate $\frac{1}{2}$, where $N = q^k - 1$, such that for at least $(1 - \epsilon)N$ values of i , C_{in}^i has relative distance $\geq H_q^{-1}(\frac{1}{2} - \epsilon)$.*

In fact, this ensemble is the following: for $\alpha \in \mathbb{F}_{q^k}^*$, the inner code $C_{\text{in}}^\alpha : \mathbb{F}_q^k \rightarrow \mathbb{F}_{q^k}^{2k}$ is defined as $C_{\text{in}}^\alpha(x) = (x, \alpha x)$. This ensemble is called the *Wozencraft ensemble*. We claim that C_{in}^α for every $\alpha \in \mathbb{F}_{q^k}^*$ is linear and is strongly explicit. (The proof is left as an exercise.)

10.3.1 Justesen code

For the Justesen code, the outer code C_{out} is a Reed-Solomon code evaluated over $\mathbb{F}_{q^k}^*$ of rate R , with $0 < R < 1$. The outer code C_{out} has relative distance $\delta_{\text{out}} = 1 - R$ and block length of $N = q^k - 1$. The set of inner codes is the Wozencraft ensemble $\{C_{\text{in}}^\alpha\}_{\alpha \in \mathbb{F}_{q^k}^*}$ from Theorem 10.3.1.

So the Justesen code is the concatenated code $C^* \stackrel{\text{def}}{=} C_{\text{out}} \circ (C_{\text{in}}^1, C_{\text{in}}^2, \dots, C_{\text{in}}^N)$ with the rate $\frac{R}{2}$. The following proposition estimates the distance of C^* .

Proposition 10.3.2. *Let $\epsilon > 0$. C^* has relative distance at least $(1 - R - \epsilon) \cdot H_q^{-1}(\frac{1}{2} - \epsilon)$*

Proof. Consider $\mathbf{m}^1 \neq \mathbf{m}^2 \in (\mathbb{F}_{q^k})^K$. By the distance of the outer code $|S| \geq (1 - R)N$, where

$$S = \{i | C_{\text{out}}(\mathbf{m}^1)_i \neq C_{\text{out}}(\mathbf{m}^2)_i\}.$$

Call the i th inner code *good* if C_{in}^i has distance at least $d \stackrel{\text{def}}{=} H_q^{-1}(\frac{1}{2} - \epsilon) \cdot 2k$. Otherwise, the inner code is considered bad. Note that by Theorem 10.3.1, there are at most ϵN bad inner codes. Let S_g be the set of all good inner codes in S , while S_b is the set of all bad inner codes in S . Since $S_b \leq \epsilon N$,

$$|S_g| = |S| - |S_b| \geq (1 - R - \epsilon)N. \quad (10.3)$$

For each good $i \in S$, by definition we have

$$\Delta(C_{\text{in}}^i(C_{\text{out}}(\mathbf{m}^1)_i), C_{\text{in}}^i(C_{\text{out}}(\mathbf{m}^2)_i)) \geq d. \quad (10.4)$$

Finally, from (10.3) and (10.4), we obtain that the distance of C^* is at least

$$(1 - R - \varepsilon) \cdot Nd = (1 - R - \varepsilon) H_q^{-1} \left(\frac{1}{2} - \varepsilon \right) N \cdot 2k,$$

as desired. $\square$

Since the Reed-Solomon codes as well as the Wozencraft ensemble are strongly explicit (see Exercise 10.4), the above result implies the following:

Corollary 10.3.3. *The concatenated code C^* from Proposition 10.3.2 is an asymptotically good code and is strongly explicit.*

Thus, we have now satisfactorily answered Question 10.2.2 modulo Theorem 10.3.1, which we prove next.

Proof of Theorem 10.3.1. Fix $\mathbf{y} = (\mathbf{y}_1, \mathbf{y}_2) \in \mathbb{F}_q^{2k} \setminus \{\mathbf{0}\}$. Note that this implies that $\mathbf{y}_1 \neq \mathbf{0}$ or $\mathbf{y}_2 \neq \mathbf{0}$ are not possible. We claim that $\mathbf{y} \in C_{\text{in}}^\alpha$ for at most one $\alpha \in \mathbb{F}_{2^k}^*$. The proof is by a simple case analysis. First, note that if $\mathbf{y} \in C_{\text{in}}^\alpha$, then it has to be the case that $\mathbf{y}_2 = \alpha \cdot \mathbf{y}_1$.

- Case 1: $\mathbf{y}_1 \neq \mathbf{0}$ and $\mathbf{y}_2 \neq \mathbf{0}$, then $\mathbf{y} \in C_{\text{in}}^\alpha$, where $\alpha = \frac{\mathbf{y}_2}{\mathbf{y}_1}$.
- Case 2: $\mathbf{y}_1 \neq \mathbf{0}$ and $\mathbf{y}_2 = \mathbf{0}$, then $\mathbf{y} \notin C_{\text{in}}^\alpha$ for every $\alpha \in \mathbb{F}_{2^k}^*$ (as $\alpha \mathbf{y}_1 \neq \mathbf{0}$ since the product of two elements in $\mathbb{F}_{2^k}^*$ also belongs to $\mathbb{F}_{2^k}^*$).
- Case 3: $\mathbf{y}_1 = \mathbf{0}$ and $\mathbf{y}_2 \neq \mathbf{0}$, then $\mathbf{y} \notin C_{\text{in}}^\alpha$ for every $\alpha \in \mathbb{F}_{2^k}^*$ (as $\alpha \mathbf{y}_1 = \mathbf{0}$).

Now assume that $wt(\mathbf{y}) < H_q^{-1}(1 - \varepsilon)n$. Note that if $\mathbf{y} \in C_{\text{in}}^\alpha$, then C_{in}^α is “bad” (i.e. has relative distance $< H_q^{-1}(\frac{1}{2} - \varepsilon)$). Since $\mathbf{y} \in C_{\text{in}}^\alpha$ for at most one value of α , the total number of bad codes is at most

$$\left| \left\{ \mathbf{y} \mid wt(\mathbf{y}) < H_q^{-1} \left(\frac{1}{2} - \varepsilon \right) \cdot 2k \right\} \right| \leq Vol_q \left(H_q^{-1} \left(\frac{1}{2} - \varepsilon \right) \cdot 2k, 2k \right) \quad (10.5)$$

$$\leq q^{H_q(H_q^{-1}(\frac{1}{2} - \varepsilon) \cdot 2k)} \quad (10.5)$$

$$= q^{(\frac{1}{2} - \varepsilon) \cdot 2k}$$

$$= \frac{q^k}{q^{2\varepsilon k}} \quad (10.6)$$

$$< \varepsilon(q^k - 1) \quad (10.6)$$

$$= \varepsilon N. \quad (10.7)$$

In the above, (10.5) follows from our good old upper bound on the volume of a Hamming ball (Proposition 3.3.3) while (10.6) is true for large enough k . Thus, for at least $(1 - \varepsilon)N$ values of α , C_{in}^α has relative distance at least $H_q^{-1}(\frac{1}{2} - \varepsilon)$, as desired. $\square$

10.4 Summary of concatenation

In this chapter we saw how to use a simple (in retrospect) idea to build codes over small alphabets explicitly from explicit codes over large alphabets. Specifically, by concatenating an outer code of distance D and an inner code of distance d , we can obtain a code of distance at least $\geq Dd$ (Theorem 10.1.1). Dd is called the concatenated code's *designed distance*. The combinatorial performance of these codes is captured by the Zyablov bound — see Theorem 10.2.1 and Figure 10.2. The Zyablov bound may be a bit complex to understand on its own so we elaborate a bit here, by focusing on the two extreme cases of maximal distance and maximal rate. When the distance of a binary code is $1/2 - \varepsilon$ where $\varepsilon \rightarrow 0$, the GV bound assure a code of rate $\Omega(\varepsilon^2)$, whereas the Zyablov bound achieves rate that is roughly $\Omega(\varepsilon^3)$ (ignoring $\text{polylog } 1/\varepsilon$ factors). (See Exercise 10.3 for details.) On the other extreme when the distance of the code is $\delta \rightarrow 0$, the GV bound assures us of codes of rate $1 - O(\delta \log 1/\delta)$, while the Zyablov bound only achieves a rate of roughly $1 - O(\sqrt{\delta})$ (again ignoring $\text{polylog } 1/\delta$ factors) — see Part (4) of Exercise 10.12. The Zyablov bound is not known to be improvable in the high-distance regime using variations of concatenation. However at the other extreme of low-rate, a variation of concatenation due to Blokh and Zyablov does improve significantly on the Zyablov bound. The resulting Blokh-Zyablov bound approaches the GV bound upto polylogarithmic factors in δ . Exercise 10.12 describes the variation and illustrates how it improves on the rate in the low-distance regime.

Turning to the algorithm aspects results presented in this chapter, for asymptotically good codes, we have obtained polynomial time construction of such codes (Theorem 10.2.1) thereby making them “explicit”. We also showed how to get strongly explicit construction of such codes (Corollary 10.3.3), i.e., constructions where the entries of the generator matrix can be constructed in polynomial time. Further, since these codes were linear, we also get polynomial time encoding. However, the following natural question about decoding still remains unanswered.

Question 10.4.1. *Can we decode concatenated codes up to half their designed distance in polynomial time?*

We will answer this question in Chapter 14.

10.5 Exercises

Exercise 10.1. Let us define a code $C \subseteq \mathbb{F}_2^n$ to be a binary RS code if there exists $q = 2^t$, n' , a Reed-Solomon code $C' \subseteq \mathbb{F}_q^{n'}$ and a bijective map $\phi : \mathbb{F}_q \rightarrow \mathbb{F}_2^t$ such that $C = \{(\phi(x_1), \dots, \phi(x_{n'})) \mid (x_1, \dots, x_{n'}) \in C'\}$. (In other words C is obtained by taking codewords of C' and writing elements of $\mathbb{F}_q$ as t bit binary strings.) For every $0 \leq R \leq 1$ and integer t , prove that there exist linear binary RS codes of block length $n = t2^t$, rate R and relative distance at least $\frac{1-R}{\log n}$.

Exercise 10.2. Prove that the concatenation of two linear codes is linear. Specifically let $C_{\text{out}} \subseteq \mathbb{F}_{q^k}^N$

be an $\mathbb{F}_{q^k}$ -linear code and let $C_{\text{in}} \subseteq \mathbb{F}_q^n$ be an $\mathbb{F}_q$ linear code of dimension k . Then prove that $C_{\text{out}} \circ C_{\text{in}}$ is an $\mathbb{F}_q$ -linear code. Describe the generator matrix of $C_{\text{out}} \circ C_{\text{in}}$.

Exercise 10.3. Prove that Theorem 10.2.1 yields explicit binary codes of distance $\delta = 1/2 - \gamma$ and rate $R = \Omega(\gamma^3)$ for every $\gamma > 0$. Specifically show that there exists a constant $\eta > 0$ such that for all $\gamma > 0$ there exists an explicit family of binary codes of relative distance at least $1/2 - \gamma$ and rate at least $\eta \cdot \gamma^3$.

Exercise 10.4. Given positive integer t , let $\phi : \mathbb{F}_{2^t} \rightarrow \mathbb{F}_2^t$ be an $\mathbb{F}_2$ -linear bijection. Recall that the Wozencraft ensemble of codes is the collection of codes $\{C_\alpha \subseteq \mathbb{F}_2^{2t} \mid \alpha \in \mathbb{F}_{2^t}^*\}$ where C_α is given by the encoding map $E_\alpha : \beta \mapsto (\phi(\beta) \circ \phi(\alpha\beta))$ for every $\beta \in \mathbb{F}_{2^t}$. (Thus $E_\alpha : \mathbb{F}_{2^t} \rightarrow \mathbb{F}_2^{2t}$. Recall further that the Justesen code of rate $1/4$ is given by the encoding map $E_{\text{JUSTESEN}} : \mathbb{F}_2^{tk} \rightarrow \mathbb{F}_2^{2t2^t}$ given by $(\phi(c_0), \dots, \phi(c_{k-1})) \mapsto (E_\alpha(P(\alpha))_{\alpha \in \mathbb{F}_{2^t}})$ where $k = 2^{t-1}$ and $C(x) = \sum_{i=0}^{k-1} c_i x^i$ for every $(c_0, \dots, c_{k-1}) \in \mathbb{F}_2^k$.

1. Prove that for every $\alpha \in \mathbb{F}_{2^t}^*$, the code C_α is a linear code.
2. Prove that the Justesen code is a linear code.
3. Prove that the Justesen code is strongly explicit, i.e., give a generator matrix $G \in \mathbb{F}_2^{kt \times 2t2^t}$ that generates the Justesen code such that there is a $\text{poly}(t)$ time algorithm computing the (i, j) th entry of the generator matrix, given $(i, j) \in [kt] \times [2t2^t]$.

Exercise 10.5. Prove that a random code in the Wozencraft ensemble achieves capacity on BSC_p for every $p < H^{-1}(1/2)$. Specifically, given $\varepsilon > 0$ and $p = H^{-1}(1/2) - \varepsilon$, prove that there exists $\gamma > 0$ such that for every t there exists $\alpha \in \mathbb{F}_2^t$ and a decoding map $D : \mathbb{F}_2^{2t} \rightarrow \mathbb{F}_2^t$ such that for every $\mathbf{m} \in \mathbb{F}_2^t$ we have

$$\Pr_{\mathbf{e} \sim \text{BSC}(p)} [D(E_\alpha(\mathbf{m}) + \mathbf{e}) \neq \mathbf{m}] \leq 2^{-\gamma^t}.$$

Exercise 10.6. Let us say that linear codes $C_1, \dots, C_M \subseteq \mathbb{F}_2^n$ form a packing in $\mathbb{F}_2^n$ if (i) All codes are of the same size (so $|C_i| = |C_j|$ for all $i, j \in [M]$); and (ii) They have the minimal possible intersection i.e., $C_i \cap C_j = \{0^n\}$ for all $i \neq j$.

1. Prove that if $C_1, \dots, C_M$ form a packing, and d is such that $\sum_{i=1}^{d-1} \binom{n}{i} < M$ then there exists $i \in [M]$ such that $\Delta(C_i) \geq d$.
2. Extend the Wozencraft ensemble to get codes of rate $1/\ell$ and distance approaching $H^{-1}(1 - 1/\ell)$ for (constant) every positive integer ℓ .
3. Extend the notion of a ‘packing’ to a notion of ‘uniform cover’ so as to build codes of rate $1 - 1/\ell$ and distance $H^{-1}(1/\ell)$.

Exercise 10.7. Let $C_{\text{RS}} \subseteq \mathbb{F}_2^{2^t}$ be a Reed-Solomon code of rate ε . Let $C_{\text{Had}, t} \subseteq \mathbb{F}_2^{2^t}$ be the Hadamard code of dimension t and block length 2^t (from Definition 2.6.2). Prove that their concatenation yields an $[n = 4^t, k = t2^t]_2$ linear code where every non-zero codeword has Hamming weight in the interval $[(1 - \varepsilon)\frac{n}{2}, (1 + \varepsilon)\frac{n}{2}]$. Conclude that there exists an explicit ε -biased space in $\mathbb{F}_2^k$ of size $O\left(\frac{k^2}{\varepsilon^2 \log^2 k}\right)$.

Exercise 10.8. In this exercise we prove that (generalized) concatenated codes with the inner code having logarithmic length can achieve the asymptotic Gilbert-Varshamov bound. Specifically, let C_{RS} be an $[N, K, N - K + 1]_N$ code where $N = 2^t$. Let $C^1, \dots, C^N \subseteq \mathbb{F}_2^t$ be random independent linear codes of rate 1 (i.e., each is given by an random linear map (possibly with a non-trivial kernel) from $\mathbb{F}_N \rightarrow \mathbb{F}_2^t$ chosen uniformly and independently from the set of all such mappings). Prove that the concatenated code $C = C_{\text{RS}} \circ (C^1, \dots, C^N)$ has, with high probability, rate $R = K/N$ and relative distance approaching $H^{-1}(1 - R)$.

Exercise 10.9. This exercise explores the relative distance of duals of concatenated codes. Part (1) shows that most concatenated codes have very poor dual distance. Part (2) shows that with some care we can get a dual code of modest relative distance.

1. Let $C = C_{\text{out}} \circ C_{\text{in}}$ be a linear concatenated code where C_{in} is an $[n, k]_q$ code for some $k < n$. Prove that the minimum distance of $C^\perp$ is at most $k + 1$.
2. Prove that if $C = C_{\text{out}} \circ C_{\text{in}}$ is a linear concatenated code where C_{in} is an $[n, n]_q$ code, then the relative minimum distance of $C^\perp$ is at least $\delta(C_{\text{out}}^\perp)/n$.

Exercise 10.10. Given prime power q and integer $r \leq q^2$, the Hermitian code $H_{q,r}$ is an explicit $[n, k, d]_{q^2}$ code for $n = q^3$, $k = \binom{r+1}{2}$ and $d = n - rq$. Given $\varepsilon > 0$ and K_0 , give a choice of parameters q, r, t such that the concatenation of the Hermitian code $H_{q,r}$ with the Hadamard code $C_{\text{Had}, t}$ yields an $[N, K]_2$ binary code with $K \geq K_0$, $N = O\left(\max\left\{K^{5/2}, \left(\frac{\sqrt{K}}{\varepsilon}\right)^{5/2}\right\}\right)$ with the weight of every non-zero codeword lying in the interval $[(1 - \varepsilon)\frac{N}{2}, (1 + \varepsilon)\frac{N}{2}]$. Conclude there exists an explicit ε -biased space in $\mathbb{F}_2^K$ of size at most $O\left(\max\left\{K^{5/2}, \left(\frac{\sqrt{K}}{\varepsilon}\right)^{5/2}\right\}\right)$. (In particular when $\varepsilon = 1/K$ this improves up the concatenation of Reed-Solomon codes with Hadamard codes from Exercise 10.7, by achieving $N = O(K^{15/4})$ as opposed to $N = O(K^4)$.)

Exercise 10.11. Show that one can get a fully explicit asymptotically good code using a two-stage concatenation with two outer layers of Reed-Solomon codes and an inner code that is selected from an exponentially large ensemble of codes. Specifically show that there exist Reed-Solomon codes C_1 and C_2 and a code C_3 chosen from, say, a Wozencraft ensemble, such that $C_1 \circ C_2 \circ C_3$ is fully explicit and asymptotically good.

Exercise 10.12. In this exercise we will describe a more careful and advanced concatenation scheme that significantly improves upon the Zyablov bound for low rates. Given positive integers c , and $t_1, \dots, t_c$ with $t_1 + \dots + t_c = t$, a c -level concatenation of type $(t_1, \dots, t_c)$ is given by $c + 1$ $\mathbb{F}_q$ -linear codes $C_{\text{out}}^1, \dots, C_{\text{out}}^c$ and C_{in} with $C_{\text{out}}^i : \mathbb{F}_q^{k_i} \rightarrow (\mathbb{F}_q^{t_i})^n$ for $i \in [c]$ and $C_{\text{in}} : \mathbb{F}_q^t \rightarrow \mathbb{F}_q^N$. The concatenated code, denoted $C = (C_{\text{out}}^1 \times \dots \times C_{\text{out}}^c) \circ C_{\text{in}}$ is a code mapping $\mathbb{F}_q^k \rightarrow \mathbb{F}_q^N$ for $k = k_1 + \dots + k_c$ given as follows. Given $\mathbf{m} = (\mathbf{m}^1, \dots, \mathbf{m}^c)$ with $\mathbf{m}^i \in \mathbb{F}_q^{k_i}$, let $\mathbf{x}^i = (x_1^i, \dots, x_n^i) = C_{\text{out}}(\mathbf{m}^i) \in (\mathbb{F}_q^{t_i})^n$ and let $\mathbf{y}_j = C_{\text{in}}(x_1^1, \dots, x_n^c)$. Then $C(\mathbf{m}) = (\mathbf{y}_j)_{j \in [N]}$.

The power of the multilevel concatenation described above comes from analyzing the performance of C_{in} carefully as follows: We say that C_{in} has $(t_1, \dots, t_c)$ -type distance $(\delta_1, \dots, \delta_c)$ if for

every $i \in [c]$ the code C_{in}^i defined below has distance $\delta(C_{\text{in}}^i) \geq \delta_i$:

$$C_{\text{in}}^i = \left\{ C_{\text{in}}(0^{t_1+\dots+t_{i-1}} \mathbf{m}^i) \mid \mathbf{m}^i \in \mathbb{F}_q^{t_i+\dots+t_c} \right\}.$$

In other words the distance type measures not only the distance of the code C_{in} but also sub-codes obtained by restricting messages whose prefixes are zeroes. Every code C_{in} of distance δ has $(t_1, \dots, t_c)$ -type distance $(\delta, \dots, \delta)$, but for most codes we can get better values of $\delta_2, \delta_3, \dots$. Part (2) of the exercise below proves this, while Part (3) shows how this allows us to use codes of $C_{\text{out}}^2, C_{\text{out}}^3, \dots$ of higher rate than may be allowed by a vanilla use of the Zyablov bound. In turn this bumps up the rate of our final code.

1. Let $R_i = \frac{k_i}{t_i n}$ denote the rate of the code C_{out}^i and let $\tau_i = \frac{t_i}{T}$. Let $\bar{R} := 1 - R$ denote the redundancy of a code of rate R . Verify that the rate of C is $R = \frac{1}{T} \sum_i \tau_i R_i$ and its redundancy $\bar{R} \leq (1 - t/T) + \sum_{i=1}^c \tau_i \bar{R}_i$. Verify that C is $\mathbb{F}_q$ -linear.
2. Fix $\varepsilon > 0$. Prove that there exists $\delta > 0$ such that a random linear code C_{in} has $(t_1, \dots, t_c)$ -type distance $(\delta_1, \dots, \delta_c)$, where $\delta_i = H_q^{-1}(1 - r_i) - \varepsilon$ and $r_i = \frac{1}{T}(t_i + \dots + t_c)$, with probability at least $1 - \exp(-\delta T)$.
3. Prove that the minimum distance of C is $\min_i \{\delta_i \cdot \delta(C_{\text{out}}^i)\}$.
Hint: Given $\mathbf{m} = (\mathbf{m}^1, \dots, \mathbf{m}^c)$ consider the smallest i such that $\mathbf{m}^i \neq \mathbf{0}$ and prove that $C(\mathbf{m})$ has weight at least $\delta_i C_{\text{out}} \delta(C_{\text{out}}^i) n T$.
4. Fix $\varepsilon > 0$. In the following parts, let q be large enough so that $H_q^{-1}(1 - R) \geq 1 - R - \varepsilon$ for every $R \in [0, 1]$. Prove that Zyablov bound (see Theorem 10.2.1) on the rate R_Z of codes of minimum distance δ satisfies $1 - 2\sqrt{\delta} - \varepsilon \leq R_Z \leq 1 - \sqrt{\delta}$.
5. For every $\delta, \varepsilon > 0$ for sufficiently large q , prove that for every sufficiently large N there exists a q -ary two level concatenated code $C = (C_{\text{out}}^1 \times C_{\text{out}}^2) \circ C_{\text{in}}$ of length N , distance $\delta - O(\varepsilon)$ and rate $1 - \delta^{2/3} - O(\varepsilon)$.
Hint: Let $\gamma = \frac{t_1}{t_1 + t_2}$ and $\rho = \delta(C_{\text{in}})$. Ignoring ε and assuming all codes (including C_{in}^2 !) are on the Singleton bound, prove that the redundancy of C can be upper bounded by $\rho + \gamma\delta + \rho + \delta/\gamma$. Optimize over δ and γ .
6. Extend the idea above so that for every $\delta > 0$, positive integer c , and $\varepsilon > 0$, we get concatenated codes of distance $\delta - \varepsilon$ and rate at least $1 - O_c(\delta^{1-1/c}) - \varepsilon$.

Remark 10.5.1. We note that the exercise above can be carried out with $q = 2$ to get codes of distance δ and rate $1 - O_c(H_2(\delta)^{1-1/c}) - \varepsilon$ also, at the price of some slightly more complex expressions in the calculations.

10.6 Bibliographic Notes

Code concatenation was first proposed by Forney [52]. In addition to the introduction of the technique, Forney also gave decoding algorithms for concatenated codes that were strong enough

to achieve convergence to Shannon capacity with polynomial time algorithms. These algorithms, and the convergence to capacity will be covered in later chapters (in particular in Chapter 14). The tradeoff between rate and distance of these codes were explored by Zyablov [165] and in particular the Zyablov bound (see Theorem 10.2.1) is from there.

The Justesen codes were constructed by Justesen [96]. The Wozencraft ensembles were first reported by Massey [112] who attributes them to a personal communication with Wozencraft. The low-rate variant of these ensembles from Part (2) of Exercise 10.6 is due to Weldon [94].

The multilevel concatenation from Exercise 10.12 is due to Blokh and Zyablov [20], who also give a closed form expression for optimized versions of these codes, called the Blokh-Zyablov bound. The exact expression is somewhat complex, involving some integrals, and omitted from this book. Instead Exercise 10.12 gives a flavor of the implications of this bound focussing on the high-rate regime. We refer the reader to the survey article by Dumer [43] for further details on the Blokh-Zyablov bound as well as other aspects of concatenated codes.

Chapter 11

When Graphs Come to the Party: Expander Codes

In this chapter we will explore the use of graph theoretic methods to build codes either from scratch, or from much weaker codes. Specifically the class of graphs that we will look at are what are called *expanders* — informally, graphs in which all small enough sets have ‘large neighborhoods’ (i.e., many vertices outside the set are adjacent to some vertex in the set). Research over past few decades has shown remarkable constructions of such graphs, and in this chapter we will leverage the existence and construction of such graphs to help design new/better codes.

Our motivation to study these graph based codes from the fact that the only way we have seen so far to construct asymptotically good codes (over smaller alphabets) is via code concatenation. Recall that we answered Question 10.2.2 in the affirmative in Corollary 10.3.3 using Justesen codes. However, while concatenated codes do provide a nice generic way of constructing codes over smaller alphabets from codes over larger alphabets, it is not clear how we can decode such codes very efficiently (say in linear time). This motivates the need to look for alternatives:

Question 11.0.1. *Can we construct explicit asymptotically good binary codes without code concatenation?*

In this chapter we will get multiple affirmative answers to the above question. (In Chapter ?? we address the question of how to decode such codes in linear time.) The chapter roughly consists of four parts (not counting a preliminary section that reviews basic notions in graph theory). First, in Section 11.2, we will review two notions of expansion and also show a connection between the two notions. Then in Section 11.3 we will build codes using a strong notion of expanding graphs — the advantage here will be that the description of the code and analysis of the parameters will be extremely simple, given the description of the graphs. Next, in Section 11.4, we will generalize the construction from the previous section to be applicable to a broader class of graphs with weaker expansion. Finally, in Section 11.5 we will introduce a com-

pletely different way to use graph theory to construct codes. The methods of this section will start with codes of low minimum distance and amplify them using expanding graphs to have much higher minimum distance.

11.1 Graphs

In this section we review some basic material on graphs, including notions of bipartite graphs and regular graphs.

We start by recalling that a graph G is given by a pair (V, E) where V is a finite set called the vertex set and E is a family of unordered pairs of elements of V called the set of edges. Given a vertex $u \in V$, the set of vertices $N(u) = \{v \in V \mid (u, v) \in E\}$ is referred to as the neighbors of u , and $\deg(u) = |N(u)|$ is called the degree of u . While in most study of graphs it is customary to simply view $N(u)$ as an unordered set, in many of our applications it will sometimes be convenient to assume these elements are ordered in some arbitrary way, i.e., $N(u) = (v_1, \dots, v_{\deg(u)})$ is an ordered tuple of elements of V . When the ordering is important to us, we will stress this by referring to G as an *ordered graph*. (We will make no assumptions about these orderings and in particular the orderings of $N(u)$ and $N(v)$ may be totally unrelated. Our analysis will not rely on the exact ordering, but our codes will depend on the ordering.)

11.1.1 Basic terms and notions

We start with the definition of a bipartite graph, which places restrictions on the edges.

Definition 11.1.1 (Bipartite Graphs). A bipartite graph is a triple $G = (L, R, E)$, where L is the set of ‘left’ vertices and R is the set of ‘right’ vertices and the set $E \subseteq L \times R$ is the set of edges.

(Note that a bipartite graph (L, R, E) is a special case of a graph $(V = L \cup R, E)$.) Figure 11.1 gives an example of a bipartite graph G_H with seven left vertices, three right vertices, twelve edges and with every right vertex having degree 4.

One way to represent such graphs is via its *adjacency matrix*:

Definition 11.1.2 (Adjacency matrix). Given a bipartite graph $G = (L, R, E)$, its adjacency matrix, denoted by A_G , is an $|R| \times |L|$ binary matrix, where we index each row by an element of R and every column by an element of L such that for every $(r, \ell) \in R \times L$, the (r, ℓ) ’th entry in A_G is 1 if $(\ell, r) \in E$ and 0 otherwise. Conversely every matrix $A \in \{0, 1\}^{n \times m}$ corresponds to a bipartite graph $G_H = (L, R, E)$ with $|L| = n$ and $|R| = m$ with $(i, j) \in E$ if and only if $A_{ij} = 1$.

For example, the adjacency matrix of the graph in Figure 11.1 is given by

$$\begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

We remark that the matrix above is the parity check matrix of the $[7, 4, 3]_2$ Hamming code (see Section 2.3). This is not a coincidence but rather an intentional choice on the authors’ part.

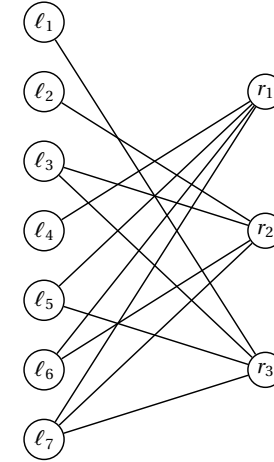


Figure 11.1: A bipartite graph G_H

It is always possible to view the adjacency matrix of a bipartite graph as a parity matrix of a code and vice versa. When the graph G is sparse, i.e., where each vertex in L has at most a constant number of neighbors in R , the associated parity check matrix has sparse rows with few 1’s and this property (along with more global properties of the graph) turn out to be very useful in the construction of nice error-correcting codes. With this end in mind we define (degree-)boundedness and regularity of bipartite graphs.

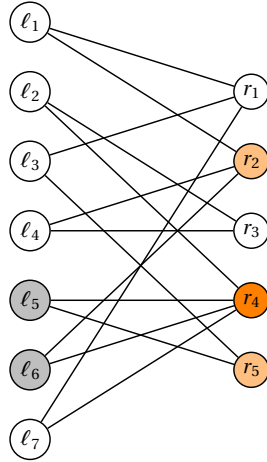
Definition 11.1.3 ((Left/Right) Regularity, Boundedness). A graph $G = (V, E)$ is said to be c -bounded if every vertex has degree at most c . It is said to be c -regular if every vertex has degree exactly c .

Similarly, a bipartite graph $G = (L, R, E)$ is said to be c -left regular (bounded) if every vertex in L has degree exactly (resp., at most) c . It is said to be d -right regular (bounded) if every vertex in R has degree exactly (resp. at most) d .

For example, the graph in Figure 11.1.1 is 2-left regular. (In the next few sections we will only restrict the graphs to be left regular, and only require right regularity in later sections.)

11.2 Expander Graphs

In the following sections we will describe multiple constructions of error-correcting codes from graphs. In all cases the efficacy of the construction will depend in one-way or another on the ‘expansion’ of the underlying graph, i.e., the property that small subsets of vertices in the graph



see many edges leaving the set, or many neighboring vertices not in the set. There are multiple ways of quantifying this notion of expansion. In this section we introduce several of these notions. In later sections we explain how these notions sometime relate to each other; and also explain what kinds of expanders exist, and what is known about their construction. We start with (arguably) the simplest notion — of bipartite expansion.

11.2.1 Bipartite Vertex Expanders

We start by defining the neighborhood of a set of vertices.

Definition 11.2.1 (Neighbor Set). For a left vertex set $S \subseteq L$, a vertex $u \in R$ is called a neighbor of S if it is adjacent to some vertex in S . We denote by $N(S)$ the set of neighbors of S . We analogously define $N(T)$ for any $T \subseteq R$.

Note that our notion of the neighbors $N(u)$ of a vertex $u \in L \cup R$ can be viewed as shorthand for $N(\{u\})$. For example, in the graph in Figure 11.1.1, if $S = \{\ell_5, \ell_6\}$ (set of gray left vertices), then $N(S) = \{r_2, r_4, r_5\}$ (the set of orange right vertices).

We are ready to define bipartite expander graphs.

Definition 11.2.2 (Bipartite Expander Graphs). An $(n, m, c, \gamma, \alpha)$ bipartite expander is a c -left regular bipartite graph $G = (L, R, E)$ where $|L| = n$ and $|R| = m$ such that for every $S \subseteq L$ with $|S| \leq \gamma n$, we have

$$|N(S)| \geq \alpha |S|.$$

An infinite family of bipartite graphs $\{G_i\}_{i \in \mathbb{Z}_+}$ is termed a (c, γ, α) -bipartite expander family if for every $i \in \mathbb{Z}_+$ we have G_i is an $(n_i, m_i, c, \gamma, \alpha)$ expander for some $n_i, m_i \in \mathbb{Z}_+$. We say $\{G_i\}_{i \in \mathbb{Z}_+}$ is

is a bipartite expander family if there exists $c \in \mathbb{Z}_+$ and $\alpha, \gamma > 0$ such that $\{G_i\}_{i \in \mathbb{Z}_+}$ is a (c, γ, α) -bipartite expander family

In words the definition requires that all ‘small’ sets of left vertices have comparatively large neighborhoods. For example, the graph in Figure 11.1.1 is a $(7, 5, 2, \frac{2}{7}, \frac{3}{5})$ bipartite expander (see Exercise 11.1). The parameter γ quantifies this notion of ‘smallness.’ Since the largest that $N(S)$ can be is $|R|$, it follows that $\gamma \leq |R|/(\alpha|L|)$. So to get a family of expanders with $\gamma > 0$ we need $|R| = \Omega(|L|)$ and this will be the setting of parameter we will be interested in. Turning to the second new parameter, α gives a measure of the expansion and is called the *expansion factor*. Note that we always have $\alpha \leq c$.

Of the two parameters we will focus a lot more on α . In our simplest of code constructions we will work with bipartite expanders with $\alpha > c/2$. Of course this would be of interest only if such graphs exists and we discuss this next.

Existence of Bipartite Expander graphs. The following result shows that exists expanders with an expansion factor that can get arbitrarily close to the upper bound of c :

Theorem 11.2.3. For every $\epsilon > 0$ and large enough $m \leq n$, there exists an $(n, m, c, \gamma, c(1 - \epsilon))$ bipartite expander where $c = \Theta\left(\frac{\log(n/m) + \log(1/\epsilon)}{\epsilon}\right)$ and $\gamma = \Theta\left(\frac{\epsilon m}{cn}\right)$.

The above can be proven with the probabilistic method (see Section 11.6 for a proof). For fixed $\epsilon > 0$ and large $m \leq n$, note that the left degree c scales as $O(\log(n/m)/\epsilon)$.

Remark 11.2.4. We present few remarks on the above result:

1. Note that we only focus on the case $m \leq n$ (since this is the setting of interest in constructing codes), so we want expansion from the larger side to the smaller side. This is the harder direction since is less room to expand to. (For example, if we could have $m = cn$, then we would have a trivial $(n, m, c, 1, c)$ bipartite expander by giving each node on the left its own private neighborhood of c nodes on the right.)
2. The ratio of the expansion factor to the degree c can be brought arbitrarily close to its maximum value of 1 (by letting $\epsilon \rightarrow 0$) at the cost of increasing the value of c .
3. By definition, $\gamma nc(1 - \epsilon)$ is a trivial lower bound on m since sets of size up to (and including) γn expand by a factor of $c(1 - \epsilon)$. The above result achieves a value of m that is $1/\epsilon$ times larger than this trivial bound. It turns out that this is necessary: see Section 11.8 for more on this.

Theorem 11.2.3 guarantees the *existence* of excellent expanding graphs but for our basic construction of explicit expander codes (to be described in Section 11.3), we will need an *explicit* construction of bipartite expanders with $\alpha > c/2$. The following theorem (which we will not prove) asserts the existence of explicit constructions of such graphs.

Theorem 11.2.5. For every constant $\epsilon > 0$ and every desired ratio $0 < \beta < 1$, there exist explicit $(n, m, c, \gamma, c(1 - \epsilon))$ bipartite expanders for any large enough n (and $m = \beta n$) with c and $\gamma > 0$ being constants (that only depend on ϵ and β).

The construction of the expanders asserted above is complicated and hence the is omitted from this book. We will simply take the construction as a black-box here and build explicit codes based on these in future sections of this chapter. Before turning to other definitions of expansion, we take a brief aside to note that we can assume that the right degree of the graphs we construct is bounded.

Bounding right degree. Note that the expander code that we have considered so far are *left regular* but in general there is no restriction on the degree of the right vertices. The following lemma shows, however, that any left-regular expander can be converted into another one that in addition has ‘bounded’ right degree (without changing the expansion factor)– we will need this fact later on in the book.

Lemma 11.2.6. *Let G be an $(n, m, c, \gamma, \alpha)$ bipartite expander. Then there exists another bipartite graph G' that is an $(n, m', c, \gamma, \alpha)$ bipartite expander such that*

- $m \leq m' \leq 2m$
- Every right vertex in G' has degree at most $\lceil \frac{nc}{m} \rceil$.

Proof. Let $G = (L, R, E)$ and we will construct $G' = (L, R', E')$ with the required properties. Define

$$d = \lceil \frac{nc}{m} \rceil.$$

For each vertex $r \in R$, let d_r be its degree. Then for each $r \in R$, add $\lceil \frac{d_r}{d} \rceil$ vertices to R' . Further the d_r edges incident to vertex r are divided evenly among the corresponding $\lceil \frac{d_r}{d} \rceil$ vertices in R' such that all but potentially one such vertex has degree exactly d (and at most one vertex has degree $< d$).

The claim on the right degree bound in G' follows by construction. The claim on the expansion follows since splitting the vertices can only increase vertex expansion. Recall that $m' = |R'|$ and $m = |R|$. Then again by construction, we have $m' \geq m$. To complete the proof we will argue that

$$m' - m \leq m.$$

To see this note that

$$m' - m = \sum_{r \in R} \left(\left\lceil \frac{d_r}{d} \right\rceil - 1 \right) \leq \sum_{r \in R} \frac{d_r}{d} = \frac{nc}{d} \leq m,$$

where the last inequality follows from the definition of d . $\square$

To summarize, in this section we defined a basic notion of expansion, namely bipartite expansion (Definition 11.2.2). We define this notion for left-regular bipartite graphs and without loss of generality their right degree can be assumed to be bounded. Expansion in these graphs is quantified (mainly) by a parameter α which lies between 0 and c , the left-degree of the graph, and larger α s imply better expansion. Expander families with $\alpha = (1 - \epsilon)c$ exist for every $\epsilon > 0$ for sufficiently large c (Theorem 11.2.3) and can even be explicitly constructed (Theorem 11.2.5), albeit with a construction that is complex (and out of scope for this book).

11.2.2 Spectral Expanders

Our next definition of expansion is not based on combinatorial properties of the graph, such as neighborhoods of vertex sets, but rather defined based on linear-algebraic properties of the adjacency matrix of a graph. The basic objects of interest in this section will be non-bipartite d -regular graphs. (We will use these to construct some bipartite expanders in Section 11.2.3.)

We begin by recalling the definition of an *eigenvalue* of a matrix (over $\mathbb{R}$):

Definition 11.2.7. *Let $M \in \mathbb{R}^{n \times n}$ be a matrix over the reals. Then, λ_i is an eigenvalue of M if exists a vector $\mathbf{v}_i \in \mathbb{R}^n$ such that $M \cdot \mathbf{v}_i = \lambda_i \cdot \mathbf{v}_i$.*

The spectral theorem in linear algebra asserts that any real symmetric matrix has n real eigenvalues (we will take this result as a given). Since the adjacency matrix of a graph is real and symmetric, this implies that we can associate to a graph the n eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n$ of its adjacency matrix. For a d regular graph G , it is easy to show that $\lambda_1 = d$ and $\lambda_n \geq -d$ (see Exercise 11.6). The spectral study of expansion focusses on the quantity $\max\{|\lambda_2|, |\lambda_n|\}$.

Definition 11.2.8. *A graph $G = (V, E)$ is said to be a (n, d, λ) -spectral expander if G is a d -regular (not necessarily bipartite) graph on n vertices and $\lambda \geq \max\{|\lambda_2|, |\lambda_n|\}$.*

In other words, λ is (an upper bound on) the second largest eigenvalue of the adjacency matrix of G , in absolute value. We will sometimes denote this value λ by $\lambda(G)$. In order to obtain a family of Tanner codes (see Section 11.4.1 for a formal definition), we will be interested in a family of d -regular graphs for a fixed d with the number of vertices n growing while maintaining λ bounded away from d . We define families of spectral expanders next.

Definition 11.2.9. *For positive integer d and $0 \leq \lambda < d$, we say that an infinite family of graphs $\{G_n\}_{n \in \mathbb{Z}^+}$ with G_n having n vertices and being d -regular; is a (d, λ) -spectral expander family if $\lambda(G_n) \leq \lambda$ for every n .*

We refer to a family as a spectral expander family if there exists $\lambda < d$ such that the family is a (d, λ) -spectral expander family. The smaller the λ the better the expansion. The key ingredient used to connect spectral expansion to more combinatorial notions of expansion is a well-known lemma about spectral expanders called the Expander Mixing Lemma, stated below. The lemma roughly says that, in a good spectral expander (one where λ is small compared to d), the number of edges between every pair of sizable subsets of vertices is approximately equal to what one would expect in a random d -regular graph. We start with a precise definition of the set of edges crossing from a set A to a set B in a graph.

Definition 11.2.10. *Given an undirected graph $G = (V, E)$ and sets $A, B \subseteq V$ we define $E(A, B)$ to be the set of pairs (a, b) with $a \in A$ and $b \in B$ such that $(a, b) \in E$. I.e., $E(A, B) = \{(a, b) \in A \times B \mid (a, b) \in E\}$. (Note that if $a, b \in A \cap B$ then both (a, b) and (b, a) belong to $E(A, B)$.)*

We are now ready to state the expander mixing lemma.

Lemma 11.2.11 (Expander mixing lemma). *Let $G = (V, E)$ be a (n, d, λ) -graph. Then for every $A, B \subseteq V$ we have¹*

$$\left| |E(A, B)| - \frac{d|A||B|}{n} \right| \leq \lambda \sqrt{|A| \cdot \left(1 - \frac{|A|}{n}\right) \cdot |B| \cdot \left(1 - \frac{|B|}{n}\right)}. \quad (11.1)$$

In particular, the above implies

$$\frac{d|A||B|}{n} - \lambda \sqrt{|A||B|} \leq |E(A, B)| \leq \frac{d|A||B|}{n} + \lambda \sqrt{|A||B|}. \quad (11.2)$$

We do not prove this lemma here. (Section 11.8 gives pointers to proofs). The expander mixing lemma is a power tool for the analysis of combinatorial properties of a spectral expander. For instance, it implies an upper bound of $\frac{n\lambda}{d}$ on the size of an independent set in an expander (see Exercise 11.8).

Good expanders and Ramanujan graphs. We now turn to the key questions: Do spectral expanders (ones with $\lambda \ll d$) exist? Can they be constructed explicitly? If so what is the best relationship between λ and d that can be achieved.

It turns out expanders with $\lambda \ll d$ do exist and can be constructed explicitly. And while analysis of spectral expansion can be quite complex, constructions can be surprisingly simple. For instance the 8-regular graph $G_n = (V, E)$ with $V = \mathbb{Z}_n \times \mathbb{Z}_n$ and $(x, y) \in V$ having neighborhood

$$N((x, y)) = \{(x \pm 1, y), (x, y \pm 1), (x, y \pm 2x), (x \pm 2y, y)\}, \quad (11.3)$$

(where $a \pm b$ is short hand for the values $a + b$ and $a - b$ and so each pair above actually are two vertices and all arithmetic performed mod n) is known to be a $(n, 8, \lambda)$ -spectral expander for some $\lambda < 8$. Furthermore if G is a (n, d, λ) -spectral-expander then G^k , where edges correspond to walks of length k in G , is an (n, d^k, λ^k) spectral expander. (See Exercise 11.2).

This already suggests that the relationship between λ and d may be polynomial. It turns out that the dependence is roughly of the form $\lambda \approx \sqrt{d}$. It is not too hard to show that $\lambda \geq \Omega(\sqrt{d})$ (see Exercise 11.10). In fact, one can show that for an infinite family of d -regular graphs, we must have $\lambda \geq 2\sqrt{d-1} - o(1)$. (This claim is harder to prove and not an exercise in this book.) Rather miraculously, this bound can be achieved; such (family of) graphs with $\lambda \leq 2\sqrt{d-1}$ are called Ramanujan graphs. Such strong spectral expansion implies that small sets have neighborhoods that are larger by a factor of nearly $d/4$ (see exercise 11.9).

Ramanujan graphs can be constructed explicitly for a dense enough sequence of degrees, for instance for $d = q + 1$ where q is a prime with $q \equiv 1 \pmod{4}$, and almost-Ramanujan graphs (which have $\lambda \approx 2\sqrt{d-1}$) can be constructed for any desired degree. For our purposes though, it is not important that we have exactly Ramanujan or even near-Ramanujan graphs. It suffices that the ratio λ/d can be made arbitrarily small by picking d large enough, and in this regime some simpler constructions are available. Let us record the following result on the availability of explicit expanders. See the bibliographic notes for relevant references.

¹Note that $A = B$ is allowed.

Theorem 11.2.12. *For every $\varepsilon > 0$, for all large enough d , there exists an infinite family of d -regular (n, d, λ) -graphs with $\lambda \leq \varepsilon d$. Here by explicit we mean that the adjacency matrix of the graph can be constructed in polynomial time. Furthermore, we can take $d = O(1/\varepsilon^2)$, which in turns implies that for large enough d , we can in polynomial time construct an $(n, d, O(\sqrt{d}))$ -graph.*

To summarize, in this section we defined a second notion of expansion, namely spectral expansion (Definition 11.2.8). We define this notion for d -regular (non-bipartite) graphs. Expansion in these graphs is quantified by a parameter λ which lies between $2\sqrt{d-1}$ and d , with smaller λ implying better expansion. Expander families with $\lambda = o(d)$ and even with $\lambda = O(\sqrt{d})$ can be constructed explicitly (with the former being somewhat easier to analyze than the latter).

11.2.3 Bipartite Expanders from Spectral Expanders

We now show how to convert a spectral expander (with say $\lambda = o(d)$) into a bipartite expander (with $\alpha > 0$). This conversion involves two simple operations that transform graphs into other graphs, called the *double cover* and the *edge-vertex incidence graph*. We introduce these next before describing the full transform. The final result of this section is summarized in Theorem 11.2.15.

We start with the definition of the double cover of a graph.

Definition 11.2.13 (Double cover). *The Double Cover of a graph $G = (V_G, E_G)$ is defined as the bipartite graph $H = (L_H, R_H, E_H)$ with vertices $L_H = \{u' | u \in V_G\}$ and $R_H = \{u'' | u \in V_G\}$, and edges $E_H = \{(u', v''), (v', u'') | (u, v) \in E_G\}$.*

Thus, the double cover H of graph G has two copies of each vertex of G , one in the left partition and the other in the right partition, and two copies of each edge (u, v) of G . See Figure 11.2 for an example of a double-cover of a graph.

Next we define the *edge vertex incidence graph* of a graph.

Definition 11.2.14. *The Edge Vertex Incidence Graph of a graph $G = (V, E)$ is defined as the bipartite graph $H_0 = (L, R, E')$ where $L = E$, $R = V$ and $E' = \{(e, v) | v \in e \in E\}$.*

In words, the edge vertex incidence graph H_0 has a left vertex corresponding to each edge in E , a right vertex corresponding to each vertex in V and every edge $e = \{u, v\} \in E$ turns into two edges (e, u) and (e, v) in E' . Figure 11.2 illustrates the above definition.

We are now ready to describe the transformation from a spectral expander to a bipartite one: Given a (n, d, λ) -spectral expander G , we consider the bipartite graph H obtained by taking the double cover G' of G and then taking H to be the edge-vertex incidence graph of G' . The following theorem shows that this leads to a bipartite expander in the sense of Definition 11.2.2.

Theorem 11.2.15. *Let G be an (n, d, λ) -spectral expander. Let G' be the double cover of G and H be the edge-vertex incidence graph of G' . Then, for every β such that $\lambda \leq \beta \leq d$, we have that H is a d -right regular $\left(dn, 2n, 2, \frac{\beta(\beta-\lambda)}{d^2}, \frac{2}{\beta}\right)$ -expander.*

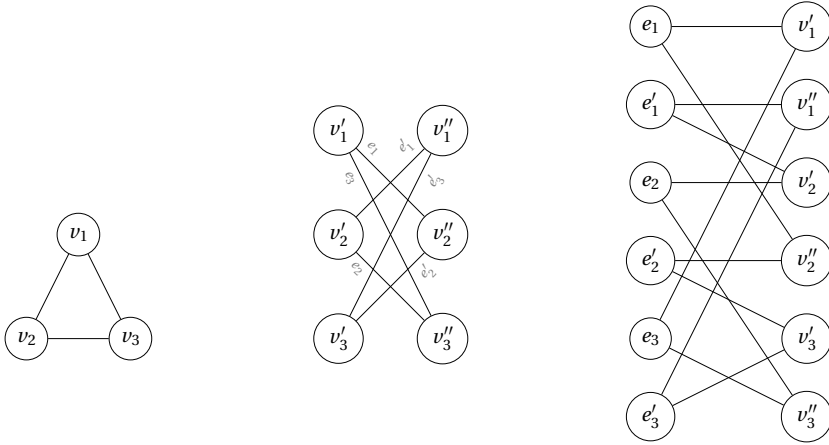


Figure 11.2: The ‘triangle’ graph G on the left, and its double cover G' (see Definition 11.2.13) in the middle and H , the edge vertex incidence graph of G' (see Definition 11.2.14) on the right. Note that H is a 12-cycle.

Proof. We start by inspecting H , the ‘edge-vertex incidence graph of the double cover of a graph G ’, and state the expansion conditions of H in terms of the structure of $G = (V, E)$. Let E' denote the edges of the double cover G' of G , and thus the left vertices of H . Note $|E'| = dn$. Note that the right vertices of H correspond to the vertices of G' . Let $V_1 \cup V_2$ denote these vertices so that $|V_1| = |V_2| = n$ such that every left vertex of H has exactly one neighbor in each of V_1 and V_2 . Thus H is indeed a graph with dn left vertices and $2n$ right vertices that is 2-left regular and d -right regular. The only property to be shown is that sets containing at most $\frac{\beta(\beta-\lambda)}{d^2}$ fraction of left vertices expand by a factor of at least $(2/\beta)$. Specifically, let $S \subseteq E'$ be an arbitrary set of left vertices of size at most $\frac{\beta(\beta-\lambda)}{d^2} dn = \frac{\beta(\beta-\lambda)}{d} n$. Let $N(S)$ denote the set of neighbors of S and let $N(S) = A \cup B$ where $A \subseteq V_1$ and $B \subseteq V_2$. Our goal is to show that

$$|A| + |B| \geq (2/\beta)|S|.$$

By the construction of H we have that every vertex of S is an edge in G from the set A to the set B . In particular we have $|S| \leq |E(A, B)|$ and so by the expander mixing lemma (Lemma 11.2.11) we have:

$$|S| \leq |E(A, B)| \leq \frac{d|A||B|}{n} + \lambda \sqrt{|A||B|}.$$

In the rest of the proof we carry out manipulations to show that the above, combined with the upper bound $|S| \leq \frac{\beta(\beta-\lambda)}{d} n$ imply $|S| \leq \beta \sqrt{|A||B|}$. Combining this with the AM-GM inequality we

get that

$$|S| \leq \beta \sqrt{|A||B|} \leq \frac{\beta}{2} \cdot (|A| + |B|)$$

and thus yielding

$$|A| + |B| \geq \frac{2}{\beta} |S|$$

as desired. We thus turn to proving $|S| \leq \beta \sqrt{|A||B|}$.

Assume for contradiction that $|S| > \beta \sqrt{|A||B|}$. Then we have:

$$\beta \sqrt{|A||B|} < |S| \leq |E(A, B)| \leq \frac{d|A||B|}{n} + \lambda \sqrt{|A||B|}.$$

Dividing LHS and RHS by $\sqrt{|A||B|}$ and collecting and rearranging terms we thus have:

$$\sqrt{|A||B|} > \frac{\beta - \lambda}{d} n.$$

Using our assumption $|S| > \beta \sqrt{|A||B|}$ again, we now get $|S| > \frac{\beta(\beta-\lambda)}{d} n$ contradicting the given upper bound on $|S|$. We thus conclude $|S| \leq \beta \sqrt{|A||B|}$. Summarizing the inequalities obtained above, we now have:

$$|S| \leq \beta \sqrt{|A||B|} \leq \frac{\beta}{2} \cdot (|A| + |B|) = \frac{\beta}{2} \cdot |N(S)|.$$

In other words every set S with $|S| \leq \frac{\beta(\beta-\lambda)}{d} n$ satisfies $|N(S)| \geq \frac{2}{\beta} \cdot |S|$ and so H is a d -right regular $(dn, 2n, 2, \frac{\beta(\beta-\lambda)}{d^2}, \frac{2}{\beta})$ -expander. $\square$

To summarize, spectral expansion with $\lambda = o(d)$ yields bipartite expansion (in related graphs) with $\alpha > 0$. In what follows in Section 11.3, we will see codes that build on bipartite expanders with $\alpha > c/2$ (something that we can not get from spectral expansion). In Section 11.4.1 we will weaken our expansion requirement so that even the bipartite expanders constructed in this section from spectral ones will suffice to get asymptotically good codes. Later yet, in Section 11.5 we will use spectral expansion to build codes using other operations, and these lead to codes matching (and sometimes beating) codes we’ve constructed previously in terms their rate, distance and alphabet size. The most significant impact of these codes however is deferred to Chapter ?? where we give extremely fast algorithms associated with many of the codes constructed in this chapter.

11.3 Basic Expander Codes

In this section we use strong bipartite expanders (with expansion $\alpha > c/2$) to build asymptotically good error correcting codes.

As we noted in the discussion following Figure 11.1, we can associate a binary code to every bipartite graph and vice versa — by identifying the adjacency matrix of the graph with the parity check matrix of the code. The following definition formalizes this association.

Definition 11.3.1 (Factor graph). *Given any $[n, k]_2$ code C with parity check matrix H , we will call the bipartite graph G_H with $A_{G_H} = H$ to be a factor graph of C . Further, given a bipartite graph $G = (L, R, E)$ with $|L| \geq |R|$, we will denote the corresponding code with parity check matrix A_G to be $C(G)$.*

Note that the factor graph of a code is not unique, since different parity check matrices may lead to different factor graphs. Of particular interest to us will be the case where the factor graph is sparse, i.e., every row of the parity check matrix has a constant number of non-zero elements. Equivalently this corresponds to the fact that the right degree of the factor graph is bounded. Codes that have such a sparse factor graph are called *low density parity check (LDPC)* codes. The efficacy of such algorithms in correcting many errors relies on structural properties, most notably *expansion*, of the underlying graph. This brings us to the definition of expander codes, which are simply codes whose factor graph is an expander.

Definition 11.3.2. *A code $C \subseteq \mathbb{F}_2^n$ is called an expander code if there is a bipartite expander graph G such that $C = C(G)$. In G is d -right bounded then $C(G)$ is said to be a d -LDPC (for Low Density Parity Check) Code.*

Note that the definition above is qualitative in the same sense as the definition of a binary error-correcting code as a subset of $\{0, 1\}^n$ is qualitative. The actual performance of the latter depends on parameters like the rate and distance, and similarly the performance of an expander code will depend on the degree and expansion of the factor graph. In fact a more special form of expansion, that we call *unique expansion* becomes central to the coding theoretic properties. We will take a brief digression to introduce this notion and relate it to our standard notion of expansion. After doing so we will return to the task of analyzing the rate and distance of an expander code.

11.3.1 Unique expansion

Definition 11.3.3 (Unique Neighbor Set). *Given a bipartite graph $G = (L, R, E)$ and a left vertex set $S \subseteq L$, a vertex $u \in R$ is called a unique neighbor of S if it is adjacent to exactly one vertex in S . We let $U(S)$ denote the set of unique neighbors of S .*

For example, in the graph in Figure 11.1.1, if $S = \{\ell_5, \ell_6\}$ (set of gray left vertices), then $U(S) = \{r_2, r_5\}$ (the set of light orange right vertices).

Since $U(S) \subseteq N(S)$ it follows that if $|U(S)|$ grows linearly with $|S|$ for $|S| \leq \gamma|L|$ then so does $|N(S)|$ and so G is a bipartite expander. It turns out that if G is a c -left-regular graph with expansion $\alpha > c/2$ then the converse also holds, namely $|U(S)|$ grows linearly with $|S|$ for $|S| \leq \gamma n$. We prove this below.

Lemma 11.3.4. *Let $G = (L, R, E)$ be an $(n, m, c, \gamma, c(1 - \varepsilon))$ bipartite expander graph with $\varepsilon < 1/2$. Then for every $S \subseteq L$ with $|S| \leq \gamma n$, we have*

$$|U(S)| \geq c(1 - 2\varepsilon)|S|.$$

Proof. Let $U \subseteq N(S)$ be the unique neighbors of S , i.e., those vertices in $N(S)$ with exactly one neighbor in S . We wish to show $|U| \geq c(1 - 2\varepsilon)|S|$. We do so by counting the edges incident to S .

The total number of edges going out of S is exactly $c|S|$ by virtue of G being c -left regular. By the expansion property, $|N(S)| \geq c(1 - \varepsilon)|S|$. For every vertex $v \in N(S)$ designate exactly one edge (u, v) with $u \in S$ as ‘special.’ Note that the number of special edges is exactly $|N(S)| \geq c(1 - \varepsilon)|S|$ and so the number of non-special edges is at most $\varepsilon c|S|$. Now note that if a vertex $v \in N(S)$ is not a unique neighbor of S then v must have a non-special edge incident to it (since only one of the edges incident to it is special). So the total number of vertices in $N(S)$ that are not unique neighbors is upper bounded by the number of non-special edges which in turn is at most $\varepsilon c|S|$. Thus, we have

$$|U(S)| \geq c(1 - \varepsilon)|S| - \varepsilon c|S| = (1 - 2\varepsilon)c|S|,$$

as desired. $\square$

11.3.2 Rate and Distance of Expander Codes

We now show that if G is a left-regular expander with $m/n < 1$ and $\alpha > c/2$, then $C(G)$ is an asymptotically good code. The distance will use the notion of unique neighborhoods and follow easily from Lemma 11.3.4. But before analyzing the distance we make some more basic observations about $C(G)$ (for an arbitrary bipartite graph G).

Proposition 11.3.5. *Let $G = (L, R, E)$ be a bipartite graph with $|L| = n$ and $|R| = n - k$. Then $(c_1, \dots, c_n) \in \{0, 1\}^n$ is in $C(G)$ if and only if the following holds (where $S = \{i \in [n] | c_i \neq 0\}$) for every $r \in R$:*

$$\sum_{\ell \in N(r)} c_\ell = \sum_{\ell \in N(r) \cap S} c_\ell = 0, \quad (11.4)$$

where the sum is over $\mathbb{F}_2$.

Proof. The proof follows from the definition of $C(G)$, a parity check matrix and the fact that c_j for every $j \notin S$, does not contribute to any of the computed parities. $\square$

We record our first result on expander codes.

Theorem 11.3.6. *Let G be an $(n, n - k, c, \gamma, c(1 - \varepsilon))$ bipartite expander with $\varepsilon < \frac{1}{2}$. Then $C(G)$ is a $[n, k', \gamma n + 1]_2$ code for some $k' \geq k$.*

Proof. The claim on the block length and the linearity of $C(G)$ follows from the definition of expander codes. To lower bound the dimension, note that the space of codewords are the space of solutions to a homogenous linear system over $\mathbb{F}_2$ on n variables, namely $c_1, \dots, c_n$, with $n - k$ constraints, namely $\sum_{\ell \in N(r)} c_\ell = 0$ for every $r \in R$. The solution space has dimension $k' \geq n - (n - k) = k$.

We now turn to showing that the distance of the code is at least $\gamma n + 1$. For the sake of contradiction, let us assume that $C(G)$ has distance at most γn . Then, by the linearity of $C(G)$ (in particular Proposition 2.3.6), there exists a non-zero codeword $\mathbf{c} \in C(G)$ such $wt(\mathbf{c}) \leq \gamma n$.

Let S be the set of non-zero coordinates of $\mathbf{c}$. Since G is an $(n, n-k, c, \gamma, c(1-\varepsilon))$ expander, by Lemma 11.3.4 we have,

$$|U(S)| \geq c(1-2\varepsilon)|S| > 0,$$

where the inequality follows from the fact that $\varepsilon < \frac{1}{2}$ and $|S| \geq 1$ (since $\mathbf{c}$ is non-zero). Thus we have that $U(S)$ is non-empty and so there exists $r \in U(S)$. Let $\ell \in S$ be the unique neighbor of r in S implied by the definition of $U(S)$. Now the parity check in (11.4) corresponding to r is given by $\sum_{\ell' \in N(r) \cap S} c_{\ell'} = c_{\ell} = 1$ (where the first equality holds since $N(r) \cap S = \{\ell\}$ and the final equality holds since $c_{\ell} = 1$ for $\ell \in S$). Thus the parity check condition from (11.4) corresponding to r is not satisfied. Thus, Lemma 11.3.5 implies that $\mathbf{c} \notin C(G)$, which leads to a contradiction, as desired. $\square$

Note that Theorem 11.3.6 along with Theorem 11.2.5 answers Question 11.0.1 in the affirmative. In particular we have the following corollary of Theorem 11.3.6:

Corollary 11.3.7. *Suppose there exist $c \in \mathbb{Z}_+$, $\alpha > c/2$ and $\gamma, \rho > 0$ such that $\{G_i\}_{i \in \mathbb{Z}_+}$ is a (c, γ, α) -bipartite expander with G_i having n_i left vertices, m_i right vertices with $m_i/n_i < 1 - \rho$ for every $i \in \mathbb{Z}_+$. Then the family of codes $\{C(G_i)\}_{i \in \mathbb{Z}_+}$ is asymptotically good.*

11.3.3 An Improved Bound on Distance

We now show that $C(G)$ has almost twice the distance as argued in Theorem 11.3.6 (as $\varepsilon \rightarrow 0$).

Theorem 11.3.8. *Let G be an $(n, n-k, c, \gamma, c(1-\varepsilon))$ bipartite expander with $\varepsilon < 1/2$. Then $C(G)$ has distance at least $2\gamma(1-\varepsilon)n$.*

Proof. As in the proof of Theorem 11.3.6, for the sake of contradiction, let us assume that $C(G)$ has distance $< 2\gamma(1-\varepsilon)n$. Then by Proposition 2.3.6, exists a non-zero codeword $\mathbf{c} \in C(G)$ such $wt(\mathbf{c}) < 2\gamma(1-\varepsilon)n$. Let S be the set of non-zero coordinates of $\mathbf{c}$. We will argue that $U(S) \neq \emptyset$ and the rest of the argument is the same as in the proof of Theorem 11.3.6.

If $|S| \leq \gamma n$, then we can just use the proof of Theorem 11.3.6. So let us assume that exists a subset $T \subset S$ such that $|T| = \gamma n$. Then by Lemma 11.3.4, we have

$$|U(T)| \geq c(1-2\varepsilon)\gamma n. \quad (11.5)$$

Now since the total number of edges emanating out of $S \setminus T$ is at most $c|S \setminus T|$, we have

$$|N(S \setminus T)| \leq c|S \setminus T| < c\gamma(1-2\varepsilon)n, \quad (11.6)$$

where the last inequality follows from the facts that $|S| < 2\gamma(1-\varepsilon)n$ and $|T| = \gamma n$.

Now, note that

$$|U(S)| \geq |U(T)| - |N(S \setminus T)| > 0,$$

where the last inequality follows from (11.5) and (11.6). $\square$

We will later study the question of efficiently decoding the above code construction from $\approx \gamma n$ of errors in Chapter ??.

11.4 Codes from weaker expanders

The codes in the previous section required bipartite expander graphs that were c -left-regular with an expansion factor $\alpha = c(1-\varepsilon)$ for some $\varepsilon < 1/2$. In particular the expansion has to be greater than half the degree, which half the maximum possible. (It follows from the definitions that we need to have $\alpha \leq c$. In fact the expansion requirement for efficient error-correction, which we will see in Chapter ??, will be even stronger namely, $\alpha > \frac{3c}{4}$). While constructions of bipartite graphs with such strong expansion are now known (see Theorem 11.2.5), there are few such constructions and they are complicated. So it is desirable to build our codes based on graphs with weaker expansion requirements. This is exactly what we will do in this section.

11.4.1 Tanner codes

One view of the expander codes that we have seen in Section 11.3 is that it ‘combines’ a bipartite expander with small error-correcting codes in the following sense: Each right vertex in the expander can be associated with a small binary code (one whose length equals the size of the neighborhood of the chosen vertex) and require that the neighbors are labelled by a codeword of this code. A word is a labelling of the left vertices which satisfies this property for every right vertex. In Section 11.3, our small codes have been the parity check code of block length equal to the degree of the vertex (see Exercise 11.4). In this section we will extend this notion to allow other (linear) error-correcting codes instead of just the parity check code. (To make sense of this idea, we do have to switch the notion of ‘ordered graphs’ alluded to in Section 11.1 — where the neighborhood of every right vertex is an ordered of left vertices.) The codes of this section are referred to as Tanner codes, so-named after the discoverer of this technique.

The advantage of this extension will be that we can now use codes of larger distance as constraint. Recall that the parity check code only has a distance of two. In our analysis of expander codes thus far, this led to our focus on the unique neighborhood sets, and getting unique neighborhoods to be large requires the expansion factor α to be at least $c/2$. By considering codes of distance $\Delta_0 > 1$ we will see that we can focus on neighborhoods with at most $\Delta_0 - 1$ neighbors in some given set, and this will allow us to work with graphs of expansion greater than c/Δ_0 , a much broader set than unique neighbor expanders. We expand on the details below.

We start by explicitly extending our notion of the neighborhood of a (right) vertex, and introduce some notation. For a d -right regular bipartite graph we extend the definition of the neighborhood $N(u)$ of a right vertex u to be an ordered sequence of vertices of L , i.e., $N(u) = (\ell_1, \dots, \ell_d)$ where $\ell_1, \dots, \ell_d \in L$ are the vertices adjacent to u . Given a d -right regular bipartite graph $G = (L, R, E)$ with $L = [n]$, a vertex $u \in R$ and a vector $\mathbf{c} = (c_1, \dots, c_n) \in \mathbb{F}_2^n$, we let $\mathbf{c}_{N(u)} \in \mathbb{F}_2^d$ be the vector given by $(\mathbf{c}_{N(u)})_i = c_{\ell_i}$ where $N(u) = (\ell_1, \dots, \ell_d)$. We refer to the vector $\mathbf{c}_{N(u)}$ as the vertex u ’s “view” of the word $\mathbf{c}$, where we view $\mathbf{c}$ as an assignment to the vertices of L .

Definition 11.4.1 (Tanner code). *Given a d -right regular bipartite graph $G = (L, R, E)$ with $L = [n]$ and $|R| = m$ and a binary linear code $C_0 \subseteq \mathbb{F}_2^d$, the Tanner code $X(G, C_0)$ is defined as the set*

$$\{\mathbf{c} \in \mathbb{F}_2^n \mid \text{for all } u \in R, \mathbf{c}_{N(u)} \in C_0\}.$$

Remark 11.4.2. We note the following:

1. In our main construction we will work with a Tanner code on a graph G that is the edge vertex incidence graph (see Definition 11.2.14) of another graph H . We use $T(H, C_0)$ to denote this two step-construction, i.e., $T(H, C_0) = X(G, C_0)$ where G is the edge vertex incidence graph of H .
2. Tanner codes are linear codes since C_0 is a linear code (see Exercise 11.3).
3. Tanner Codes are a generalization of expander codes since in expander code in Section 11.3, C_0 was chosen to be the $[d, d-1, 2]_2$ parity check code (see Exercise 11.4).²

The following shows that if C_0 has large rate, then so does $X(G, C_0)$.

Lemma 11.4.3. The dimension of $X(G, C_0)$ is at least $n - m(d - \dim(C_0))$.

Proof. For each $u \in R$, the condition that $\mathbf{c}_{N(u)} \in C_0$ imposes $d - \dim(C_0)$ independent linear constraints on the bits of $\mathbf{c}$. Therefore, the condition for all $u \in R$, $\mathbf{c}_{N(u)} \in C_0$ imposes a total of at most $m(d - \dim(C_0))$ linear constraints on the codeword bits of the code $X(G, C_0)$, leaving $X(G, C_0)$ as space of dimension at least $n - (m(d - \dim(C_0)))$. (Note that some of the $m(d - \dim(C_0))$ constraints may be linearly dependent, but that only increases the dimension.) $\square$

The usefulness of Tanner codes comes from the fact that they require a much lower expansion factor through the choice of an appropriate C_0 . We see this in the following theorem.

Theorem 11.4.4. Let G be an d -right regular $(n, m, c, \gamma, \alpha)$ bipartite expander. Let C_0 be a $[d, \ell, \Delta]_2$ code. Then $X(G, C_0)$ has distance strictly greater than γn provided $\alpha > c/\Delta$.

Remark 11.4.5. Before proving the theorem above we make some remarks. Note that the lower bound above on the distance of $X(G, C_0)$ does not depend on $\ell = \dim(C_0)$, but this dimension needs to be large enough to make the code non-trivial. In particular we need $d - \ell < \frac{n}{m}$ for the dimension given by Lemma 11.4.3 to be positive. Note also that $cn = dm$ since both quantities count the number of edges in the graph G . Thus for the code to be useful, we need $d - \ell < \frac{n}{m} = \frac{d}{c}$, while maximizing Δ . These constraints do constrain the set of codes C_0 that can be used. Corollary 11.3.7 shows that we can choose parameters carefully so as to get asymptotically good codes from this construction.

Proof of Theorem 11.4.4. We mimic the proof of Theorem 11.3.6 with some minor modifications to exploit the fact that C_0 has minimum distance Δ . Let the left vertices of G be the set $[n]$ and let $[m]$ the set of right vertices of G . Let $\mathbf{c} = (c_1, \dots, c_n) \in X(G, C_0)$ be a non-zero codeword and let $S \subseteq [n]$ denote the set of non-zero coordinates of $\mathbf{c}$, i.e., $S = \{i \in [n] | c_i \neq 0\}$. Now let $W = N(S)$ be the neighbors of S in G .

²Note that the parity check code is a symmetric code in that every permutation of a codeword remains a codeword. This allowed us to work with unordered neighborhoods $N(u)$. When C_0 is not symmetric (which is the case for most codes we know) then the ordering within $N(u)$ becomes important.

We first note that every vertex $v \in W$ must have at least Δ neighbors in S . To see this, note that $\mathbf{c}_{N(v)}$ is a codeword of C_0 (by definition of the Tanner code $X(G, C_0)$) and furthermore it is a non-zero codeword since W has a neighbor in S . Thus this codeword must have weight at least Δ (by the distance of C_0) and since each non-zero coordinate of $\mathbf{c}_{N(v)}$ corresponds to a distinct vertex of S we conclude that v has at least Δ neighbors in S .

Let $s = |S|$ and $w = |W|$. From the above we have that the number of edges incident to S equals $cs \geq \Delta w$. If $|S| \leq \gamma n$, then by the expansion of G we would also have $w \geq \alpha s$. Combining the two we would get $cs \geq \alpha \Delta s$ which contradicts the assumption that $\alpha > c/\Delta$. We thus conclude that $|S| > \gamma n$, which in turn implies that $\mathbf{c}$ has weight strictly greater than γn . Since this is true for every non-zero codeword of $X(G, C_0)$ we conclude that the $X(G, C_0)$ has distance greater than γn . $\square$

The following theorem combines Theorem 11.4.4 with Theorem 11.2.15 to show that spectral expanders suffice to get Tanner codes of constant relative distance.

Theorem 11.4.6. Let $C_0 \in \mathbb{F}_2^d$ have distance at least $\delta_0 d$ and rate $\rho > 1/2$. Let G be an (n, d, λ) -spectral expander and let G' be the double cover of G and H be the edge-vertex incidence graph of G' . Then for large enough d , $X(H, C_0)$ has rate at least $2\rho - 1$ and relative distance at least $\delta_0 \left(\delta_0 - \frac{\lambda}{d} \right)$.

Remark 11.4.7. Note that when H is the edge vertex incidence graph of the $d \times d$ complete bipartite graph, the code $X(H, C_0)$ is simply the tensor product of C_0 with itself — see Exercise 11.11. Thus it has relative distance exactly δ_0^2 , but it is only a code of length d^2 . (Recall the definition of a tensor code as well as its distance properties from Exercise 2.20.) The above theorem states that for a good expander with $\lambda = o(d)$, in the limit of large degree d , the relative distance becomes $\approx \delta_0^2$, for codes of arbitrarily large block length. Thus we can obtain distance as good as the product construction, but we can get much longer codes.

Proof of Theorem 11.4.6. Consider any $\beta < \delta_0 d$ and apply Theorem 11.2.15. We get that H is a $(dn, 2n, c, \gamma, \alpha)$ -bipartite expander for $c = 2$, $\alpha = c/\beta$ and $\gamma = \frac{\beta(\beta-\lambda)}{d^2}$. Thus the dimension of the code $X(H, C_0)$ is, by Lemma 11.4.3, at least $dn - 2n(d - d\rho) = (2\rho - 1)dn$. Thus its rate is at least $2\rho - 1$.

Now we turn to the distance of $X(H, C_0)$. Here, our choice of β ensures $\alpha > c/\delta_0 d$ and so we can apply Theorem 11.4.4 to conclude that $X(H, C_0)$ has relative distance strictly greater than γ . Taking limits as $\beta \rightarrow \delta_0 d$ yields the bound on the distance and thus the theorem. $\square$

We are now ready to give another affirmative answer to Question 8.3.2:

Corollary 11.4.8. There exists an explicit (recall Definition 6.3.4) family of asymptotically good Tanner codes. In particular for every $\epsilon > 0$ there exist explicitly constructible Tanner codes of rate $R \geq 1 - \epsilon$ and distance $\delta = \Omega(\epsilon^2 / \log^2(1/\epsilon))$.

Proof. The corollary follows easily from the explicit constructibility of spectral expanders (Theorem 11.2.12) and the existence of (constant-sized) asymptotically good codes, with appropriate setting of parameters, which we do below.

We set $\rho = 1 - \varepsilon/2$ and δ_0 so that $\rho + H_2(\delta_0) \leq 1$. Using the bound $H_2(p) = O\left(p \log \frac{1}{p}\right)$ (see Exercise 3.7) we find that $\delta_0 = \Omega\left(\frac{\varepsilon}{\log(\frac{1}{\varepsilon})}\right)$ satisfies this condition. Now, let d be large enough so that the following two conditions hold:

- (1) There exists a linear code $C_0 \subseteq \mathbb{F}_2^d$ with rate ρ and relative distance δ_0 . (Note such a code exists by the Gilbert-Varshamov bound and can be found in time $2^{O(d)}$ – see Exercise 4.6.)
- (2) For $\lambda \leq \frac{\delta_0}{2} \cdot d$ and for every large enough $n \geq n_0(d)$, there exists an explicitly constructible (n, d, λ) -spectral expander G_n . (Note that such a family exists by Theorem 11.2.12 applied with $\varepsilon = \delta_0/2 > 0$.)

Now fix the code C_0 and let the n th code in our sequence be $C_n = X(H_n, C_0)$ where H_n is the edge vertex incidence graph of the double cover of G_n . Note that C_n is polytime constructible (in n with factors that may depend superpolynomially on d which is a constant independent of n). By Theorem 11.4.6 we have that the rate of C_n is at least $2\rho - 1 = 1 - \varepsilon$ and the distance is at least $\delta_0 \cdot \frac{\delta_0}{2} = \Omega\left(\frac{\varepsilon^2}{\log^2(\frac{1}{\varepsilon})}\right)$. We conclude that $\{C_n\}_{n \geq 1}$ is a polynomial constructible asymptotically good family of Tanner codes. $\square$

11.5 Distance Amplification

Finally in this section we describe new constructions of codes that also use expanders. But in contrast to the previous sections where the graphs were used to design the parity check matrix, in this section the graphs will be used to design the generator matrix. Specifically in this section we will show how to use expanding graphs to convert codes of low (but constant) relative distance into new codes with large minimum distance. In Section 11.5.1 below we start with a simple construction that only achieves codes of low-rate. Later in Section 11.5.2 we show how to use these constructions to design codes approaching the Singleton bound of arbitrary rate over (large) constant sized alphabets. Finally in Section 11.5.3 we show how to get binary codes approaching the Zyablov bound using these methods. In summary the results of this section reproduce many of the results previously obtained by concatenation — however they hold the promise of efficient algorithms and in Chapter ?? we explore the algorithmic power of these codes.

11.5.1 Codes in the low-rate regime

We will begin with a construction that amplifies the distance of the code at the expense of a larger alphabet size (and a corresponding loss in rate). This construction again uses the notion of ordered bipartite graphs.

Definition 11.5.1 (distance amplified code $G(C)$). Let $G = (L, R, E)$ be a c -left-regular and d -right-regular ordered bipartite graph with $L = [n], R = [m]$, and let $(N_1(j), \dots, N_d(j)) \in [n]^d$ denote the ordered neighborhood of $j \in [m]$. Let $\Sigma = \{0, 1\}^d$. For $\mathbf{w} \in \{0, 1\}^n$, define $G(\mathbf{w}) \in \Sigma^m$ by

$$G(\mathbf{w})_j = (\mathbf{w}_{N_1(j)}, \mathbf{w}_{N_2(j)}, \dots, \mathbf{w}_{N_d(j)}).$$

Let C be a binary linear code of block length $n = |L|$. Now define the code $G(C)$ as

$$G(C) = \{G(\mathbf{c}) \mid \mathbf{c} \in C\}.$$

We refer to $G(C)$ as the G -amplification of C .

Note that the codewords of $G(C)$ are in one-to-one correspondence with codewords of C . Each position j of a codeword of $G(C)$ ‘collects together’ the bits of a corresponding codeword of C situated in the positions that are adjacent to j in the graph G .

The alphabet of $G(C)$ is $\Sigma = \{0, 1\}^d$ which can be identified with the extension $\mathbb{F}_{2^d}$. Note though that $G(C)$ is *not* necessarily linear over $\mathbb{F}_{2^d}$. It is, however, linear over $\mathbb{F}_2$, and the sum of two codewords in $G(C)$ also belongs to $G(C)$.

Since each bit of a codeword $\mathbf{c} \in C$ is repeated c times in the associated codeword $G(\mathbf{c}) \in G(C)$, we have

$$\text{Lemma 11.5.2. } R(G(C)) = \frac{1}{c} \cdot R(C).$$

So the G -amplification of C only has worse rate than C . So to make this construction interesting we need to identify graphs that guarantee that the distance of $G(C)$ is better than that of C . In the following we describe such a class of graphs, called dispersers. We first introduce their definition and show how they can be constructed from spectral expanders. We then analyze the distance of $G(C)$ using the dispersal properties of G .

Definition 11.5.3 (dispersers). A bipartite graph $G = (L, R, E)$ is said to be a (γ, ε) -disperser if for all subsets $S \subseteq L$ with $|S| \geq \gamma n$, we have $|N(S)| \geq (1 - \varepsilon)m$.

To contrast the definition of a disperser with that of a bipartite expander, recall that bipartite expanders are required to maintain $|N(S)|/|S| \geq \alpha$ for every small enough set S . In contrast dispersers put an absolute lower bound on $|N(S)|$ (and not on $|N(S)|/|S|$) for every *large enough* set S . As γ and ε get smaller, the stronger our disperser gets. The following lemma shows how to get dispersers from bipartite expanders and from spectral expanders.

Lemma 11.5.4 (Dispersers from Expanders).

1. Let $G = (L, R, E)$ be an $(n, m, c, \gamma, \alpha)$ -bipartite-expander with $\alpha \cdot \gamma \geq (1 - \varepsilon)m/n$. Then G is a (γ, ε) -disperser.
2. Let G be an (n, d, λ) -spectral expander. Then the double cover H of G is a d -regular (γ, ε) -disperser for $\varepsilon = \frac{\lambda^2}{\gamma d^2}$.

Proof. Part (1) follows from the definitions. Let $S \subseteq L$ have size $|S| \geq \gamma n$. Let $T \subseteq S$ be any set of size $|T| = \gamma n$. By the expansion condition we have $|N(T)| \geq \alpha |T| = \alpha \gamma n$. Since $N(S) \supseteq N(T)$, it follows that $|N(S)| \geq \alpha \gamma n \geq (1 - \varepsilon)m$. Thus G satisfies the definition of a (γ, ε) -disperser.

For Part (2) we need to use the expander mixing lemma (Lemma 11.2.11). Let $H = (L, R, E)$ be the double cover of G where G is an (n, d, λ) -spectral expander. We need to prove that for each $S \subseteq L$ with $|S| \geq \gamma n$, we have $|N(S)| \geq (1 - \varepsilon)n$. As in the proof of Part (1) it suffices to consider the case of $|S| = \gamma n$. Fix S , let $T = R \setminus N(S)$, it suffices to prove that $|T| \leq \varepsilon n$. By Expander Mixing Lemma (in particular, (11.2)) and the fact that $E(S, T) = \emptyset$ by definition of T , we have

$$\begin{aligned} 0 &= |E(S, T)| \geq \frac{d|S||T|}{n} - \lambda \sqrt{|S||T|} \\ \Rightarrow d^2|S||T| &\leq \lambda^2 n^2 \\ \Rightarrow d^2|T| &\leq \frac{\lambda^2 n}{\gamma} \quad (\text{Since } |S| = \gamma n) \\ \Rightarrow |T| &\leq \frac{(\lambda/d)^2}{\gamma} n = \varepsilon n, \end{aligned}$$

as desired. $\square$

Note that the expansion condition on the bipartite expander in Part (1) of the lemma above is a very strong one. In particular our existence theorem (Theorem 11.2.3) does not assure us of such strong expansion. But the conditions needed from the spectral expander are easily satisfied by the explicit constructions of Theorem 11.2.12. As a consequence we get the following lemma.

Lemma 11.5.5. *For every $\varepsilon, \gamma > 0$ and for every large enough d that there an explicit (poly-time constructible) family of d -regular (γ, ε) -dispersers on n left and right vertices, for every sufficiently large n . Furthermore one can take $d = \Theta\left(\frac{1}{\gamma\varepsilon}\right)$.*

Proof. Recall that Theorem 11.2.12 gives that for every $\varepsilon' > 0$, for every large enough d there exists an infinite family of explicitly constructive (n, d, λ) -spectral expanders for $\lambda = \varepsilon' d$. We apply this theorem with $\varepsilon' = \sqrt{\varepsilon\gamma}$. Part (2) of Lemma 11.5.4 then implies that the double covers of the resulting family of graphs is an explicit (polynomial time constructible) family of (γ, ε) -dispersers. The furthermore part of the lemma follows from the furthermore part of Theorem 11.2.12. $\square$

We now return to analyzing the distance of G -amplified codes, using the dispersal properties of G . The following lemma follows immediately from the definition of dispersers.

Lemma 11.5.6. *If G is a (γ, ε) -disperser and $\Delta(C) \geq \gamma n$, then $\Delta(G(C)) \geq (1 - \varepsilon)m$, where $n = |L|$ and $m = |R|$.*

Proof. Suppose $\mathbf{c}$ and $\mathbf{c}'$ are distinct codewords in C , and let $G(\mathbf{c})$ and $G(\mathbf{c}')$ be the corresponding codewords in $G(C)$. Let $A \subseteq L$ be the positions where $\mathbf{c}$ and $\mathbf{c}'$ differ. We have that $G(\mathbf{c})$ and $G(\mathbf{c}')$ differ in their j 'th location whenever $j \in N(A)$. Since $|N(A)| \geq (1 - \varepsilon)m$, the Hamming distance between $G(\mathbf{c})$ and $G(\mathbf{c}')$ is at least $(1 - \varepsilon)m$. We can thus conclude that $\Delta(G(C)) \geq (1 - \varepsilon)m$. $\square$

If we instantiate the construction using an explicit binary code C of rate $1/2$ and relative distance, say, $\delta_0 = .001$ (which we can construct by Theorem 10.2.1), then by amplifying with a (δ_0, ε) -disperser d -regular G , we get that $G(C)$ has rate $\frac{1}{d}$ (by Lemma 11.5.2), alphabet size 2^d and relative distance $1 - \varepsilon$ (by Lemma 11.5.6). Further we can use $d = O\left(\frac{1}{\delta_0 \varepsilon}\right) = O\left(\frac{1}{\varepsilon}\right)$ in this construction. This leads to the following corollary.

Corollary 11.5.7. *There are explicit codes of relative distance $(1 - \varepsilon)$ and rate $\Omega(\varepsilon)$, over an alphabet of size $2^{O(1/\varepsilon)}$.*

Note that the best possible rate would be ε by the Singleton bound. So the construction is off by (only) a constant factor. On the other hand, unlike Reed-Solomon codes that needed alphabet size $\Omega(n)$ for block length n codes, here the alphabet size is a constant depending only on ε . Further, the alphabet size matches what we would get with random linear codes (see Exercise 11.13).

11.5.2 Codes almost matching the Singleton bound

Section 11.5.1 gave codes in the large distance regime, with rates optimal within a constant factor (as ε is allowed to approach 0). But the maximum rates that such codes can attain are bounded well away from 1 and certainly the codes did not get close to the rate-vs-distance tradeoff for any given rate $r \in [0, 1]$. In this section we remedy this by reexamining the amplification-technique from the previous section and adding a new ingredient to it, namely another small error correcting code.

To motivate this new construction note the G -amplification of a code C can be viewed locally as follows: Every left vertex in G of degree c sends a bit on each edge it is incident to. Every right vertex collects all the bits that it receives on these edges to form a string in $\mathbb{F}_2^d$. From a left vertex's perspective we could think of the vertex as also having a c -bit string (which happens to be all 0s or all 1s) and it sends one bit each from this string on each of the edges incident to it. This string happened to be a member of the c -bit repetition code of rate $\frac{1}{c}$ and relative distance 1; and the resulting code inherits aspects of its rate and distance from this code. Viewed from this perspective we can ask what would happen if the c -bit strings assigned to the left vertices were codewords of some other code $C_{\text{in}} \subseteq \mathbb{F}_2^c$. This is exactly what we explore in this section. And to give a quick preview of the outcome, we essentially get a final code whose rate and distance approach that of C_{in} (at the price of a somewhat larger alphabet). We give the specific details below. In what follows we work with the encoding functions rather than just the code — this will make the otherwise complex process easier to follow. We also assume the base alphabet is some $\mathbb{F}_q$ rather than $\mathbb{F}_2$.

We start by defining the G -mixing operator for any bipartite graph G .

Definition 11.5.8 (G -Mixing). *Given an ordered c -left regular d -right regular graph $G = (L, R, E)$ with $L = [n]$ and $R = [m]$, the G -mixing operator is a function from $G_{\text{mix}} : \Gamma^n \rightarrow \Sigma^m$ where $\Gamma = \mathbb{F}_q^c$ and $\Sigma = \mathbb{F}_q^d$ given as follows: For $\mathbf{u} = (u_1, \dots, u_n) \in \Gamma^n$ with $u_i = (u_i(1), \dots, u_i(c))$, we have $G_{\text{mix}}(\mathbf{u}) = \mathbf{v} = (v_1, \dots, v_m) \in \Sigma^m$ where $v_j = (v_j(1), \dots, v_j(d))$ is given by $v_j(b) = u_i(a)$ if $(i, j) \in E$*

and j is the a th neighbor of i and i is the b th neighbor of i .³ That is, the a th symbol of u_i is ‘sent’ to the b th symbol of v_j via the edge (i, j) .

We are now ready to describe our more complex amplification technique. The ingredients of this coding will be three elements: (1) An outer code C_{out} given by an encoding map from $\mathbb{F}_q^k$ to $(\mathbb{F}_q^b)^n$, which plays the role of C from the previous section. (2) An inner code C_{in} mapping $\mathbb{F}_q^b$ to $\mathbb{F}_q^B$ where b and B will be constants (independent of n and k). (3) An ordered bipartite B -regular graph $G = (L, R, E)$ with $|L| = |R| = n$. The resulting code, denoted $\mathcal{CA}(C_{\text{out}}, C_{\text{in}}, G)$ and referred to as the C_{in} -coded- G -amplification of C_{out} is defined next.

Definition 11.5.9 (Coded Amplification). *For integers k, n, b and B , codes $C_{\text{out}} : \mathbb{F}_q^k \rightarrow (\mathbb{F}_q^b)^n$ and $C_{\text{in}} : \mathbb{F}_q^b \rightarrow \mathbb{F}_q^B$, and a B -regular ordered bipartite graph $G = (L, R, E)$ with $|L| = n$, the C_{in} -coded- G -amplification of C_{out} is the code $\mathcal{CA}(C_{\text{out}}, C_{\text{in}}, G) : \mathbb{F}_q^k \rightarrow (\mathbb{F}_q^B)^n$ given by $\mathcal{CA}(C_{\text{out}}, C_{\text{in}}, G)(\mathbf{u}) = G_{\text{mix}}((C_{\text{out}} \circ C_{\text{in}})(\mathbf{u}))$, where $C_{\text{out}} \circ C_{\text{in}} : \mathbb{F}_q^k \rightarrow (\mathbb{F}_q^B)^n$ denotes the encoding of the concatenation of the codes C_{out} and C_{in} . Specifically, for $\mathbf{u} \in \mathbb{F}_q^k$ we have $(C_{\text{out}} \circ C_{\text{in}})(\mathbf{u}) = (w_1, \dots, w_n)$ where $w_i = C_{\text{in}}(v_i)$ and $(v_1, \dots, v_n) = C_{\text{out}}(\mathbf{u})$.*

The Coded Amplification is illustrated in Figure 11.3. We note that $G(C) = \mathcal{CA}(C, C_{\text{rep}}, G)$, i.e., the G -amplification of a code C is just the C_{rep} -coded G -amplification of the code C , where C_{rep} is the repetition code that takes a single symbol message and repeats it.

For general C_{in} , the rate of the coded-amplification code is exactly the same as the rate of the underlying concatenated code, which in turn is the product of the rate of C_{out} and the rate of C_{in} . This is so since the final step, namely G_{mix} , is only permuting the q -ary-symbols in the encoding under $C_{\text{out}} \circ C_{\text{in}}$ and not adding any new bits (or deleting them). The advantage of coded-amplification is in the distance where the code ends up having extremely strong distance if G has nice expansion properties. The property needed from G essentially turns out to be very close to that given by the expander mixing lemma (on spectral expanders) so we don’t give this property a new name and instead just prove the distance when G is obtained from spectral expanders.

Lemma 11.5.10. *Let $q = 2^s$ for integer $s \geq 1$. Let $C_{\text{out}} : \mathbb{F}_q^k \rightarrow (\mathbb{F}_q^b)^n$ and $C_{\text{in}} : \mathbb{F}_q^b \rightarrow \mathbb{F}_q^B$ be $\mathbb{F}_2$ -linear codes with of relative distance δ_{out} and δ_{in} respectively. Let G be the double cover of an (n, B, λ) -expander. Then if $\lambda \leq \epsilon \sqrt{\delta_{\text{out}}} B$ then $\delta(\mathcal{CA}(C_{\text{out}}, C_{\text{in}}, G)) \geq \delta_{\text{in}} - \epsilon$.*

Proof. We first note that since C_{out} and C_{in} are $\mathbb{F}_2$ -linear, the C_{in} -coded G -amplification of C_{out} is $\mathbb{F}_2$ -linear. So to analyze the distance it suffices to prove that the Hamming weight of the encoding of a non-zero vector $\mathbf{u} \in \mathbb{F}_q^k$ has large weight. Fix such a non-zero vector $\mathbf{u} \in \mathbb{F}_q^k$. Let $\mathbf{v} = C_{\text{out}}(\mathbf{u})$ and let $\mathbf{w} = (C_{\text{out}} \circ C_{\text{in}})(\mathbf{u})$ and $\mathbf{x} = G_{\text{mix}}(\mathbf{w})$ so that $\mathbf{x} = \mathcal{CA}(C_{\text{out}}, C_{\text{in}}, G)(\mathbf{u})$. Let

$$S = \{i \in [n] \mid v_i \neq 0\}$$

denote the subset of coordinates where $\mathbf{v}$ (or equivalently $\mathbf{w}$) is non-zero. Note $|S| \geq \delta_{\text{out}} n$. Now consider $T = \{j \in [n] \mid x_j \neq 0\}$. Note our goal is to prove $|T| \geq (\delta_{\text{in}} - \epsilon)n$.

³Recall that the neighbors of i are ordered and given by $N_1(i), \dots, N_c(i)$. Node $j \in R$ is the a th neighbor of i if $j = N_a(i)$.

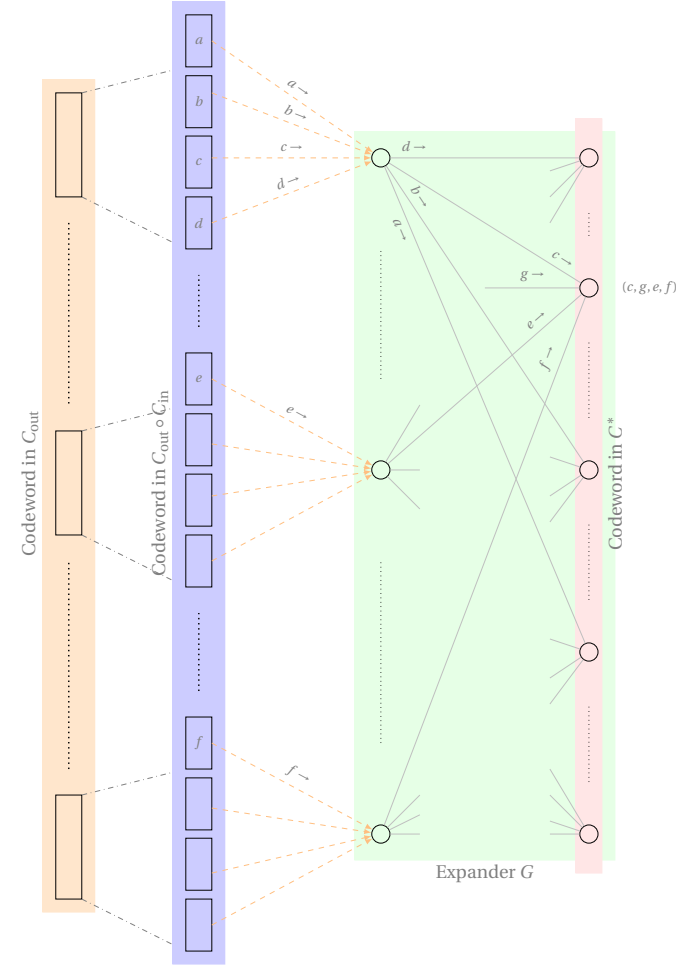


Figure 11.3: The C_{in} -Coded G -Amplification of C_{out}

We view S and T as subsets of the left and right vertices respectively of G . The key observation here is that every vertex $i \in S$ has many edges going into T . To see this note that the

mixing process associates the label w_i with the vertex i and spreads the symbols of w_i out over the edges incident to i . On the one hand, for $i \in S$, w_i is a non-zero codeword of C_{in} and so has at most $(1 - \delta_{\text{in}})B$ coordinates $a \in [B]$ such that $w_i(a) = 0$. On the other hand, every symbol transmitted to vertices outside T is zero, and so if an edge goes from S to T it must transmit the zero symbol. It follows that every vertex $i \in S$ has at most $(1 - \delta_{\text{in}})B$ neighbors outside T and at least $\delta_{\text{in}}B$ neighbors in T . We thus get that $|E(S, T)| \geq \delta_{\text{in}}B|S|$. But by the expander mixing lemma (in particular (11.2)) we also have

$$|E(S, T)| \leq \frac{B|S||T|}{n} + \lambda\sqrt{|S||T|}.$$

Putting the two together, we now have:

$$\delta_{\text{in}}B|S| \leq \frac{B|S||T|}{n} + \lambda\sqrt{|S||T|} \leq \frac{B|S||T|}{n} + \lambda\sqrt{|S|n} \leq \frac{B|S||T|}{n} + \lambda\frac{|S|}{\sqrt{\delta_{\text{out}}}},$$

where the second inequality uses $T \subseteq [n]$ to get $|T| \leq n$, and the third inequality uses $|S| \geq \delta_{\text{out}}n$ (or equivalently $n \leq |S|/\delta_{\text{out}}$). Rearranging terms we get

$$|T| \geq n \cdot \left(\delta_{\text{in}} - \frac{\lambda}{B\sqrt{\delta_{\text{out}}}} \right) \geq n \cdot (\delta_{\text{in}} - \varepsilon),$$

where the second inequality above uses $\lambda \leq \varepsilon\sqrt{\delta_{\text{out}}}B$. We conclude every non-zero codeword of $\mathcal{C}\mathcal{A}(C_{\text{out}}, C_{\text{in}}, G)$ has weight at least $(\delta_{\text{in}} - \varepsilon)n$ and thus $\delta(\mathcal{C}\mathcal{A}(C_{\text{out}}, C_{\text{in}}, G)) \geq \delta_{\text{in}} - \varepsilon$. $\square$

We are now ready to show how to instantiate the codes C_{out} , C_{in} and G (along with the underlying parameters) to construct codes that achieve a rate vs. relative distance trade-off that almost matches the Singleton bound over a constant-sized alphabet. In Chapter ??, we will also give linear time algorithms to decode these codes up to almost half the stated relative distance bound. For concreteness, we constructed the codes over a field of characteristic two, but this is not inherent.

Theorem 11.5.11. *For every $r, 0 < r < 1$, and $\varepsilon > 0$, there exists $q = 2^s$, b, B and an $\mathbb{F}_2$ -linear code $C_{\text{in}} : \mathbb{F}_q^b \rightarrow \mathbb{F}_q^B$ alphabet $\Sigma = \mathbb{F}_q^B$ of size bounded by $\exp(O(\varepsilon^{-4} \log^3(1/\varepsilon)))$, such that there exists an infinite family of C_{in} -coded amplified codes over Σ of rate r and relative distance at least $(1 - r - \varepsilon)$. Further the codes are $\mathbb{F}_2$ -linear.⁴*

Proof. We choose parameters so that the code $\mathcal{C}\mathcal{A}(C_{\text{out}}, C_{\text{in}}, G)$ is a code of rate r and relative distance $1 - r - \varepsilon$ and the alphabet is a power of two of size $\exp(O(\varepsilon^{-4} \log^3(1/\varepsilon)))$.

Fix the target rate r and the proximity (to the Singleton bound) parameter $\varepsilon > 0$. Let $r_{\text{out}} = 1 - \varepsilon/2$. Let δ_{out} be chosen such that there exists an infinite family of explicitly constructible binary linear codes of rate r_{out} and relative distance δ_{out} . In particular we may use the code

⁴This means that treating the alphabet Σ as a vector space over $\mathbb{F}_2$, the sum of any two codewords in the code also belongs to the code.

given by Corollary 11.4.8 to achieve $\delta_{\text{out}} = \Omega(\varepsilon^2 / \log^2(1/\varepsilon))$. Next we choose B and λ so that $\lambda \leq \frac{\varepsilon}{2} \cdot \sqrt{\delta_{\text{out}}B}$ and an infinite family of explicitly constructible (n, B, λ) -spectral expanders exists. By Theorem 11.2.12 it suffices to choose $B = \Theta\left(\frac{1}{\delta_{\text{out}}\varepsilon^2}\right) = \Theta(\varepsilon^{-4} \log^2(1/\varepsilon))$. In particular we choose $B = 2^s$ for integer s . Let $q = B$ and let $r_{\text{in}} = r/r_{\text{out}}$. Let $b = r_{\text{in}} \cdot B$ and let C_{in} be a Reed-Solomon code over $\mathbb{F}_q$ of rate r_{in} and relative distance $\delta_{\text{in}} = 1 - r_{\text{in}}$.

Given a sufficiently large k we let $T : \mathbb{F}_2^{ks} \rightarrow \mathbb{F}_2^{ns}$ be an explicitly constructed linear code of rate r_{out} and distance δ_{out} . Note that by collecting together the symbols of the message as well as encodings into blocks of s symbols and using a linear map from $\mathbb{F}_2^s$ to $\mathbb{F}_q$, we can view T as a map $C_{\text{out}} : \mathbb{F}_q^k \rightarrow \mathbb{F}_q^n$. Note further that this preserves the rate and does not decrease the distance, so C_{out} is an $\mathbb{F}_2$ -linear code of rate r_{out} and distance δ_{out} . Let G be the double cover of an (n, B, λ) -spectral expander. Let $C^* = \mathcal{C}\mathcal{A}(C_{\text{out}}, C_{\text{in}}, G)$. We claim C^* is the desired code for the message space $\mathbb{F}_2^{ks}$.

By construction we have that C^* is from an explicitly constructible family. As observed in the proof of Lemma 11.5.10 the code is $\mathbb{F}_2$ -linear. Its rate is equal to the rate of $C_{\text{out}} \circ C_{\text{in}}$ which in turn equals $r_{\text{out}} \cdot r_{\text{in}} = r$ (by our choice of r_{in}). By Lemma 11.5.10 we have that the relative distance of C^* is at least $\delta_{\text{in}} - \frac{\varepsilon}{2}$ (our choice of λ ensures $\lambda \leq \frac{\varepsilon}{2} \cdot \sqrt{\delta_{\text{out}}B}$). Since

$$\delta_{\text{in}} = 1 - r_{\text{in}} \geq 1 - \frac{r}{1 - \frac{\varepsilon}{2}} \geq 1 - r - \varepsilon/2$$

we get that the relative distance of C^* is at least $\delta_{\text{in}} - \frac{\varepsilon}{2} \geq 1 - r - \varepsilon$. Finally the alphabet size is $q^B = B^B = \exp(B \log B) = \exp\left(\frac{1}{\varepsilon^4} \log^3\left(\frac{1}{\varepsilon}\right)\right)$. This concludes the proof of the theorem. $\square$

Remark 11.5.12. *The near-optimal trade-off (rate r for distance close to $(1 - r)$) that almost matches the optimal Singleton bound comes from using the Reed-Solomon codes. The overall scheme can be viewed as using several independent constant-sized RS codes; the role of the expander is then to “spread out” the errors among the different copies of the RS code, so that most of these copies can be decoded, and the remaining small number of errors can be corrected by the left code C . Since only a small fraction of errors needs to be corrected by C it can have rate close to 1 and there is not much overhead in rate on top of the Reed-Solomon code.*

Remark 11.5.13. *An alternate construction of codes that have a rate vs. distance trade-off close to the Singleton bound are the so-called algebraic geometric codes (see Exercise 5.22). For suitable constructions, these can achieve a trade-off of $\delta \geq 1 - R - \varepsilon$ for rate R and relative distance δ over an alphabet size of $O(1/\varepsilon^2)$. The best known lower bound on alphabet size is $\Omega(1/\varepsilon)$ implied by the Plotkin bound. While the expander based construction requires much larger alphabet size, it is simpler and more elementary.*

11.5.3 Binary codes approaching the Zyablov bound

The codes in Theorem 11.5.11 are defined over a large alphabet Σ (whose size depends exponentially on $1/\varepsilon$). But since this is still a constant for any fixed ε , we can find an inner code of dimension $\log_2 |\Sigma|$ that achieves the Gilbert-Varshamov bound in time that depends only on

ε , and is thus a constant when ε is constant (see Exercises 4.6 and 4.8 for various options to compute such codes in times ranging from $2^{O(kn)}$ to $2^{O(k)}$ for an $[n, k]_2$ inner code). Thus, if the codes in Theorem 11.5.11 are concatenated with constant-sized binary codes that lie on the Gilbert-Varshamov bound (where the inner codes can be constructed using any of the options in Exercises 4.6 and 4.8) to give constructions of binary codes which meet the Zyablov bound (which was described in Section 10.2). That is:

Corollary 11.5.14. *There are explicit binary linear codes that lie within ε of the Zyablov bound and can be constructed in time polynomial in the block length and exponential in $\text{poly}(1/\varepsilon)$.*

We note that the above re-proves Theorem 10.2.1. While it seems like we did not ‘gain’ anything with the above Corollary, in Chapter ?? we show how the codes in Corollary 11.5.14 can be decoded in *linear*-time, something that is not known to be the case for the codes in Theorem 10.2.1.

11.6 Existence of lossless expanders: Proof of Theorem 11.2.3

In this section, we will prove Theorem 11.2.3. As mentioned earlier, we will use the probabilistic method.

We begin by fixing some parameters. Let $\tilde{c} > 0$ be a large enough constant (that will get fixed later in the proof) so that

$$c = \frac{\tilde{c}}{\varepsilon} \cdot \left(\log\left(\frac{1}{\varepsilon}\right) + \log\left(\frac{n}{m}\right) \right),$$

and let

$$\gamma = \frac{\varepsilon m}{2ecn}.$$

Note that these choices satisfy the claim on these parameter in the statement of Theorem 11.2.3. We will pick $G = (L, R, E)$ to be a random bipartite as follows. In particular, we let $|L| = n$ and $|R| = m$ as required and pick the random edges in E as follows. For every vertex $\ell \in L$ be pick c random (with replacement) vertices in R and connect them to ℓ .⁵

Let $1 \leq j \leq \lfloor \gamma n \rfloor$ be an integer and let $S \subseteq L$ be an arbitrary subset of size exactly j . We will argue that with the chosen parameters, the probability that $|N(S)| < c(1 - \varepsilon)j$ is small enough so that even after taking union bound over all choices of j and S , we get that the probability all small enough set expand by a factor of $c(1 - \varepsilon)$ is strictly more than 0, which by the probabilistic method will prove the result.

Let us for now fix j and S as above. Let $r_1, r_2, \dots, r_{jc}$ be the jc random choices of the right neighbor of the j vertices in S as outlined above. We call a choice r_i for $i > 1$ to be a *repeat* if $r_i \in \{r_1, \dots, r_{i-1}\}$. Note that if the total number of repeats is at most εjc ,⁶ then we have $|N(S)| \geq c(1 - \varepsilon)j$. Thus we will show that the probability of $> \varepsilon jc$ repeats is small.

⁵Note that this implies that we can have multi-edges and so technically the vertices in L need not be c -regular. However, this is easy to fix: see Exercise 11.14.

⁶For the rest of the proof, we assume that εjc is an integer. If not, the same proof goes through by changing the constants appropriately.

Towards that end, first note that for any given r_i the probability that r_i is a repeat is at most

$$\frac{i-1}{m} \leq \frac{jc}{m},$$

where the first bound follows from the fact that each of the m choices for r_i in R is picked uniformly at random and at worst all of the previous $i-1$ choices are all distinct while the inequality follows from the fact that $i \leq jc$. This implies that

$$\Pr[\text{number of repetitions} > \varepsilon jc] \leq \binom{jc}{\varepsilon jc} \left(\frac{jc}{m}\right)^{\varepsilon jc} \quad (11.7)$$

$$\leq \left(\frac{e}{\varepsilon}\right)^{\varepsilon jc} \cdot \left(\frac{jc}{m}\right)^{\varepsilon jc} \quad (11.8)$$

$$= \left(\frac{ejc}{\varepsilon m}\right)^{\varepsilon jc} \leq \left(\frac{j}{2\gamma n}\right)^{\varepsilon jc}. \quad (11.9)$$

In the above, (11.7) follows by taking union bound over all possible locations of εjc repetitions (and noting that the choices for each of these repetition are made independently), (11.8) follows from Lemma B.1.3 and finally (11.9) follows from our choice of γ (indeed we have by choice of γ , $2e\gamma n = \frac{\varepsilon m}{c}$).

Now taking union bound over all the $\binom{n}{j}$ choices for S , we see that the probability that exists some set S of size j that does not expand by a factor of $c(1 - \varepsilon)$ is upper bounded by

$$\binom{n}{j} \cdot \left(\frac{j}{2\gamma n}\right)^{\varepsilon jc} \leq \left(\frac{en}{j}\right)^j \cdot \left(\frac{j}{2\gamma n}\right)^{\varepsilon jc} \leq \left(\frac{1}{2}\right)^j, \quad (11.10)$$

where the first inequality follow from Lemma B.1.3 and the second inequality follow from the argument in the next paragraph. Taking union bound over all value of $j \leq \gamma n$, it can be seen that the probability that G is not an $(n, m, c, \gamma, c(1 - \varepsilon))$ bipartite expander is strictly smaller than 1, as desired.

First note that the second inequality in (11.10) is equivalent to proving (for every $1 \leq j \leq \gamma n$):

$$\left(\frac{en}{j}\right) \cdot \left(\frac{j}{2\gamma n}\right)^{\varepsilon c} \leq \frac{1}{2}.$$

Note that since by our choice of c we have $\varepsilon c > 1$, the LHS on the above increasing in j . Hence the above is true if

$$\left(\frac{en}{\gamma n}\right) \cdot \left(\frac{\gamma n}{2\gamma n}\right)^{\varepsilon c} \leq \frac{1}{2},$$

which is satisfied if

$$c \geq \frac{1}{\varepsilon} \cdot \log\left(\frac{2e}{\gamma}\right) = \frac{1}{\varepsilon} \cdot \log\left(\frac{4e^2 cn}{\varepsilon m}\right) = \frac{1}{\varepsilon} \cdot \left(\log\left(\frac{4e^2}{\varepsilon}\right) + \log c + \log\left(\frac{n}{m}\right)\right).$$

We note the above is satisfied for our choice of c for a large enough constant $\tilde{c}$. The proof is complete.

11.7 Exercises

Exercise 11.1. Argue that the graph in Figure 11.1.1 is a $(7, 5, 2, \frac{2}{7}, \frac{3}{2})$ bipartite expander.

Exercise 11.2. For any graph $G = (V, E)$, define the power graph $G^k = (V, E')$ for any $k \in \mathbb{Z}_+$ as follows. Every walk in G of length k , i.e. a sequence $v_1, \dots, v_k$ such that for each $i \in [k-1]$, $(v_i, v_{i+1}) \in E$, there is an edge $(v_1, v_k) \in E'$ (note that this can introduce multi edges, since multiple walks can have the same starting and end point and we keep the multi-edges).

Let G and k be as above. In this problem, we will argue that G^k is an (n, d^k, λ^k) spectral expander.

1. Define A_k to be the $n \times n$ matrix be the adjacency matrix of G^k , i.e. for any $u, v \in V$, we have $A_k[u, v]$ be the number of edges between u and v .⁷ Then if A is the adjacency matrix of G then prove that

$$A_k = A^k$$

2. Prove that if λ_i is an eigenvalue of G then λ_i^k is an eigen value of G^k .
3. Using the above part or otherwise, prove that G^k is an (n, d^k, λ^k) spectral expander.

Exercise 11.3. Show that the Tanner codes defined on linear code C_0 in Definition 11.4.1 are linear codes.

Exercise 11.4. Let G be a bipartite expander graphs that is both left and right regular. Then the expander code corresponding to G is the same as the Tanner code $X(G, C_0)$ where C_0 is the parity code.

Exercise 11.5. Let G be a d -regular graph on n nodes and let H be its double cover. Then for every binary linear code $C_0 \subseteq \mathbb{F}_2^d$, we have that the Tanner code $X(H, C_0)$ has dimension at least $N \cdot \frac{d}{2} - N(d - \dim(C_0))$.

Exercise 11.6. For this problem let G be a d -regular graph and H be its adjacency matrix. Then argue the following:

1. Argue that d is an eigenvalue of H .
2. Argue that the absolute value of all eigenvalues is at most d .

Hint: Consider any eigenvector $\mathbf{v}$ and consider the location of $H\mathbf{v}$ corresponding to largest absolute value in $\mathbf{v}$.

3. Using the above, or otherwise, conclude that $\lambda_1 = d$.

Exercise 11.7. Let $G = (V, E)$ be an (n, d, λ) -graph and $S \subset V$ be a subset with $|S| \leq n/2$. Then prove that

$$|E(S, \bar{S})| \geq \frac{(d - \lambda)}{2} \cdot |S|.$$

Hint: Use the bound (11.1) of Lemma 11.2.11.

⁷Note that in G^k we allow multiple edges between the same two vertices as well as (possibly multiple) self-loops.

Exercise 11.8. Let $G = (V, E)$ be an (n, d, λ) -graph. Then the any independent set⁸ of G has size at most $\frac{\lambda n}{d}$.

Hint: Use the bound (11.2) of Lemma 11.2.11.

Exercise 11.9. In this exercise we will show that in a Ramanujan graph, small enough sets expand by a factor of $\approx \frac{d}{4}$. We do so in a two step process:

1. Let $G = (V, E)$ be an (n, d, λ) -graph. For any $S \subseteq V$, let $N(S) \subseteq V$ be its neighbor set.⁹ Prove that for any $S \subseteq V$ with $|S| = \alpha \cdot n$, we have

$$\frac{|N(S) \cup S|}{|S|} \geq \frac{d^2}{\alpha \cdot d^2 + (1 - \alpha) \cdot \lambda^2}.$$

Hint: Pick $T = V \setminus (N(S) \cup S)$. What can you say about $E(S, T)$? Then apply (11.1) of Lemma 11.2.11.

2. Now consider the case when G is a Ramanujan graph, i.e. $\lambda = 2\sqrt{d-1}$. Using the above part or otherwise argue that for every $\gamma > 0$ and for large enough d , there exists an $\alpha_0 \geq \Omega(\gamma/d)$ such that for every $S \subseteq V$ such that $|S| \leq \alpha_0 \cdot n$, we have

$$|N(S) \cup S| \geq \left(\frac{d}{4} - \gamma\right) \cdot |S|.$$

Exercise 11.10. In this exercise we will argue that an (n, d, λ) -graph must have $\lambda \geq \Omega(\sqrt{d})$ as long as $d \leq n - \Omega(n)$. We will do so in multiple steps.

We first setup some notation. Let $G = (V, E)$ be an (n, d, λ) -graph. Let $\mathbf{M}$ be the adjacency matrix of G (note that $\mathbf{M}$ is symmetric). Let $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ be the eigenvalues of $\mathbf{M}$. Recall that $\lambda = \min\{|\lambda_2|, |\lambda_n|\}$ and (by Exercise 11.6) $\lambda_1 = d$.

1. Argue that the eigenvalues of $\mathbf{M}^2$ are $\lambda_1^2, \lambda_2^2, \dots, \lambda_n^2$.
2. Argue that

$$\lambda \geq \sqrt{d} \cdot \sqrt{\frac{n-d}{n-1}}.$$

Hint: Use the fact that the sum of diagonal elements of a matrix (also known as the trace of the matrix) is equal to the sum of its eigenvalues. The first part might also be useful.

3. Argue that as long as $d \leq n - \Omega(n)$, we have $\lambda \geq \Omega(\sqrt{d})$.

Exercise 11.11. Let $C_0 \subseteq \mathbb{F}_2^d$ be a code of distance at least $\delta_0 \cdot d$. Let H be the $d \times d$ complete bipartite graph. Let H' be the edge-vertex incidence graph of H .

⁸An independent set is a subset $S \subseteq V$ such that there are no edges from E contained in S .

⁹This is the natural generalization of definition of neighbor set for bi-partite graphs from Definition 11.2.1, i.e., $N(S) = \{v | u \in S, (u, v) \in E\}$.

1. Argue that the Tanner code $X(H', C_0)$ is the tensor code $C_0 \times C_0$ (recall the definition of tensor codes from Exercise 2.20).
2. Give a graph G such that H is its double cover. (Note G may be a non-simple graph and may have self-loops.)
3. Prove that $\lambda(G) = 0$.
Hint: Use the fact that for a matrix $\mathbb{M}$ with rank r , $\lambda_{r+1} = \dots = \lambda_n = 0$.
4. Use Theorem 11.4.6 argue that $X(H', C_0)$ has (relative) distance δ_0^2 .

Exercise 11.12. Recall from Definition 11.3.2 that a d -LDPC is a code with a parity check matrix that has at most d ones per row. A theorem due to Gallager shows that d -LDPC codes approach the GV bound with constant d as $d \rightarrow \infty$. In this exercise we prove a weaker form, namely, that $O(\log n)$ ones in each row of the parity check matrix suffices.

Let $0 < \delta < \frac{1}{2}$. Let $H \in \mathbb{F}_2^{m \times n}$ be a random parity check matrix of a code $\mathcal{C}$ where each entry in H is picked to be 1 independently at random with probability

$$p = c \cdot \frac{\ln n}{n},$$

for a large enough constant c (that can depend on δ). We will argue that for large enough n , there exists a code $\mathcal{C}$ with relative distance δ with

$$m = (H(\delta) + \varepsilon) \cdot n,$$

for any constant $\varepsilon > 0$. We do so in the following sequence of steps.

1. Argue that for any $x \in [0, 1]$,

$$(1 - x)^{\frac{1}{x}} \leq \frac{1}{e}.$$

Further, for any natural number $a \geq 0$, we have

$$(1 - x)^a \leq \frac{1}{1 + ax}.$$

Hint: Use Lemma B.2.5 for the first inequality.

2. For any $0 \leq w \leq n$, let N_w denote the expected number of codewords in $\mathcal{C}$ with Hamming weight exactly w . Argue that

$$N_w = \binom{n}{w} \cdot \left(\frac{1 + (1 - 2p)^w}{2} \right)^m.$$

In the next few steps we will show that this quantity is tiny for $0 \leq w < \delta n$.

3. Show that for large enough n (compared to δ, γ) such that for any

$$w \in [\gamma n, \delta n],$$

we have

$$N_w \leq 2^{-\Omega(\varepsilon n)}.$$

4. Show that there exists a $\gamma > 0$ small enough (compared to δ) and a choice of α (in terms of c) such that for any

$$w \in \left[\alpha \cdot \frac{\ln n}{n}, \gamma n \right],$$

we have

$$N_w \leq 2^{-\Omega(\varepsilon n)}.$$

5. Show that there exists a large enough (in terms of $\frac{1}{\delta}$) constant c such that any

$$w \in \left[1, \alpha \cdot \frac{\ln n}{n} \right],$$

we have

$$N_w \leq \frac{1}{n^2}.$$

6. Using the above parts (or otherwise), argue that for large enough n

$$\sum_{w=1}^{\delta n} N_w < 1.$$

Then conclude that there exists a code $\mathcal{C}$ on the GV bound with a parity check matrix where each row has $O(\log n)$ ones in it.

Exercise 11.13. Let $\varepsilon > 0$. Argue that a random linear code over an alphabet of size $2^{O(1/\varepsilon)}$ has relative distance $(1 - \varepsilon)$ and rate $\Omega(\varepsilon)$.

Exercise 11.14. Show that the graph generated in proof of Theorem 11.2.3 can be made to be exactly c -regular with at least as good an expansion property as needed in Theorem 11.2.3.

11.8 Bibliographic notes

This chapter covered a lot of results and here we give sources for the different topics that were mentioned.

Low Density Parity Check Codes Graph-based codes have a long and storied history within coding theory. LDPC codes (Definition 11.3.2) were introduced in a pioneering work by Gallager [54]. Gallager showed that for growing d , there *exist* d -LDPC codes whose rate-distance trade-off approaches the Gilbert-Varshamov bound. Note that this is a stronger result than proven in this chapter or the one we proved in Exercise 11.12. Gallager also showed that these codes admit iterative decoding algorithms for decoding from errors caused by say the Binary Symmetric Channel. LDPC codes have played a rich role in the development and practice of coding theory — we point the reader to the survey by Guruswami [67] or the text by Richardson and Urbanke [132] for further details.

Tanner codes (Definition 11.4.1) were introduced by Tanner [154] who used the *girth* (size of the smallest cycle) of the underlying graph to bound the distance of the resulting code. This bound is typically insufficient to guarantee constant relative distance. Neither the work of Gallager, nor Tanner, explicitly referred to the expansion of the underlying graph. The connection between expansion of the underlying graph and the distance of the code was established in another pioneering work, by Sipser and Spielman [144]. In particular the proofs of Corollaries 11.3.7 and 11.4.8 establishing asymptotically good codes are from [144]. It may be noted that the result of Corollary 11.3.7 was not an explicit construction at that time since explicit constructions of bipartite expanders with $\alpha > c/2$ were not available at the time. Corollary 11.4.8 was the main explicit construction of their work.

Expander Graphs The body of work covering expander graphs is even more rich and extensive, and we can't hope to do justice to this literature here. We refer the reader to the survey by Hoory, Linial and Wigderson [89] for detailed history and references. We give references here only for the specific theorems we allude to. The existence of bipartite expanders has been noted many times in the literature and variations of proofs of Theorem 11.2.3 using the probabilistic method have been discovered many times with [144, Theorem 26] coming closest to our statement. The tightness of the degree c in our construction (see item 3 in Remark 11.2.4) was established by Radhakrishnan and Ta-Shma [127]. The explicit constructions of such expanders achieving $\alpha > c/2$ was a major open challenge till a breakthrough work of Capalbo, Reingold, Vadhan and Wigderson [25] finally resolved it, thereby giving a proof of Theorem 11.2.5. Constructions of Ramanujan graphs and spectral expanders were achieved earlier in celebrated works of Margulis [111] and Lubotzky, Philips and Sarnak [108]. The simple construction of a degree 8 spectral expander mentioned in (11.3) is from [89, Construction 8.1]. The expander mixing lemma is part of a large body of work showing the connection between eigenvalue and expansion. (See Alon [3] and references therein). A proof of Lemma 11.2.11 as stated can be found in [157, Lemma 4.15]. Exercise 11.7 is the 'easy' direction of Cheeger's inequality.

Distance Amplification The use of graph-theoretic methods to amplify the distance of codes as described in Section 11.5 was introduced by Alon, Bruck, Naor, Naor and Roth [4]. Coded-amplification as described and analyzed in Section 11.5.2 was developed by Alon and Luby [5] (see also Guruswami and Indyk [71]).

Part IV

The Algorithms

Chapter 12

Efficient Decoding of Reed-Solomon Codes

In this chapter, we consider the task of decoding Reed-Solomon codes. We know that if the number of errors is less than half the minimum distance of the code, then the received word uniquely determines the codeword. In this chapter we start with an efficient algorithm that computes the codeword, given the corrupted received word. We then generalize this algorithm to a list decoding algorithm that efficiently achieves the Johnson bound (Theorem 7.3.1), i.e. given a received word that was obtained from the transmitted codeword with number of errors being bounded by the Johnson radius for this code, our algorithm efficiently outputs a small list of words that includes the transmitted word.

12.1 Unique decoding of Reed-Solomon codes

Consider the $[n, k, d = n - k + 1]_q$ Reed-Solomon code with evaluation points $(\alpha_1, \dots, \alpha_n)$. (Recall Definition 5.2.1.) In this section we solve the “unique decoding” problem associated with this code. We start by recalling this problem. Here the message is a polynomial $P(X) = \sum_{i=0}^{k-1} c_i X^i$ or equivalently its coefficients $(c_0, \dots, c_{k-1}) \in \mathbb{F}_q^k$. Its encoding is the vector $(P(\alpha_1), \dots, P(\alpha_n)) \in \mathbb{F}_q^n$. The transmission introduces errors and results in some received vector $\mathbf{y} = (y_1, \dots, y_n) \in \mathbb{F}_q^n$. The number of errors is the quantity $e = |\{i \in [n] \mid y_i \neq P(\alpha_i)\}|$. The goal of “unique decoding” is to recover P from $\mathbf{y}$ provided $e < \frac{n-k+1}{2}$, i.e., when the number of errors is less than half the minimum distance. We summarize the resulting problem below:

Problem 12.1.1 (Reed-Solomon Unique Decoding).

- **Input:** Code Parameters: $\mathbb{F}_q, (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n$ and k . Received word: $\mathbf{y} \in \mathbb{F}_q^n$.
- **Output:** $P(X) \in \mathbb{F}_q[X]$ of degree less than k such that $e := |\{i \in [n] \mid y_i \neq P(\alpha_i)\}| < \frac{n-k+1}{2}$ if such a polynomial exists and fail otherwise.

12.1.1 Motivating the decoding algorithm

We will now do a syntactic shift that will help us better visualize the decoding problem and algorithm (in fact all problems and algorithms in this chapter). In this view, we will think of $\mathbf{y}$ as the set of ordered pairs $\{(\alpha_1, y_1), (\alpha_2, y_2), \dots, (\alpha_n, y_n)\}$, that is, as a collection of “points” in “2-D space.” See Figure 12.1 for an illustration. From now on, we will switch back and forth between our usual vector interpretation of $\mathbf{y}$ and this new geometric notation.

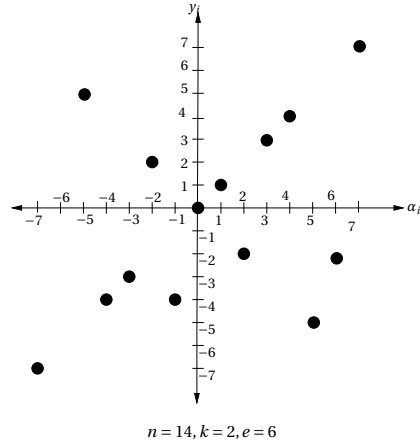


Figure 12.1: An illustration of a received word for a $[14, 2]$ Reed-Solomon code (we have implicitly embedded the field $\mathbb{F}_q$ in the set $\{-7, \dots, 7\}$). The evaluation points are $(-7, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7)$ and the received word is $(-7, 5, -4, -3, 2, -4, 0, 1, -2, 3, 4, -5, -2, 7)$.

Further, let us assume that there exists a polynomial $P(X)$ of degree at most $k - 1$ such that

$$\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e.$$

Alternatively, this means for at least $n - e$ locations $i \in [n]$, we have $P(\alpha_i) = y_i$. (Recall that if such a $P(X)$ exists then it is unique.) See Figure 12.2 for an illustration.

We will use reverse engineering to design a unique decoding algorithm for Reed-Solomon codes. We will assume that we somehow know $P(X)$ and then prove some identities involving the coefficients of $P(X)$. Then, to design the algorithm, we will just use the identities and try to solve for $P(X)$. Towards this end, let us assume that we also magically got our hands on a polynomial $E(X)$ such that

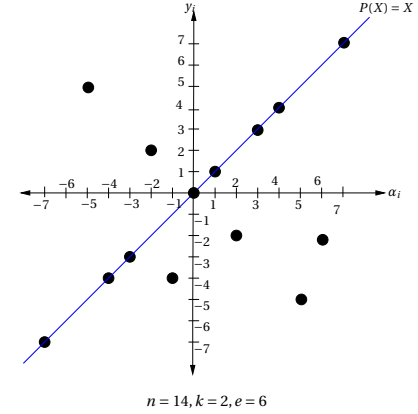


Figure 12.2: An illustration of the closest codeword $P(X) = X$ for the received word from Figure 12.1. Note that we are considering polynomials of degree 1, which are “lines.”

Definition 12.1.2 (Error-Locator Polynomial). $E(X)$ is called an *error-locator polynomial* if for all $i \in [n]$ we have:

$$E(\alpha_i) = 0 \text{ if } y_i \neq P(\alpha_i).$$

In other words, the roots of the error location polynomials include the locations where P and $\mathbf{y}$ disagree (i.e. there is an error). We remark that there exists such a polynomial of degree at most e . In particular, consider the polynomial:

$$E(X) = \prod_{i: y_i \neq P(\alpha_i)} (X - \alpha_i).$$

Now we claim that for every $1 \leq i \leq n$,

$$y_i E(\alpha_i) = P(\alpha_i) E(\alpha_i). \quad (12.1)$$

To see why (12.1) is true, we consider the following two cases:

1. If $y_i \neq P(\alpha_i)$, then both sides of (12.1) are 0 (as $E(\alpha_i) = 0$).
2. On the other hand, if $y_i = P(\alpha_i)$, then multiplying both sides by $E(\alpha_i)$ preserves the equality and so (12.1) holds in this case also.

All the discussion above does not seem to have made any progress as both $E(X)$ and $P(X)$ are unknown. Indeed, the task of the decoding algorithm is to find $P(X)$. Further, if $E(X)$ is known then one can easily compute $P(X)$ from $\mathbf{y}$ (the proof is left as an exercise). However, note that we

can now try and do reverse engineering. If we think of coefficients of $P(X)$ (of which there are k) and the coefficients of $E(X)$ (of which there are $e + 1$) as variables, then we have n equations from (12.1) in $e + k + 1$ variables. From our bound on e , this implies we have more equations than variables. Thus, if we could solve for these unknowns, we would be done. However, there is a catch—these n equations are quadratic equations, which in general are NP-hard to solve (see Definition C.5.7 for more on NP-hardness). However, note that for our choice of e , we have $e + k - 1 \ll n$. Next, we will exploit this with a trick that is sometimes referred to as *linearization*. The idea is to introduce new variables so that we can convert the quadratic equations into linear equations. Care must be taken so that the number of variables after this linearization step does not exceed the (now linear) n equations. If we can do this, we will be in familiar territory as we know how to solve linear equations over a field (e.g. by Gaussian elimination). (See Section 12.5 for some more discussion on the hardness of solving quadratic equations and the linearization technique.)

To perform linearization, define $N(X) \stackrel{\text{def}}{=} P(X) \cdot E(X)$. Note that $N(X)$ is a polynomial of degree less than or equal to $e + k - 1$. Further, if we can find $N(X)$ and $E(X)$, then we are done. This is because we can compute $P(X)$ as follows:

$$P(X) = \frac{N(X)}{E(X)}.$$

In particular, the definitions above require $N(X)$ to be a multiple of $E(X)$ and finding $N(X)$ and $E(X)$ such that $N(X)$ is a multiple of $E(X)$ retains all the computational hardness of the problem at hand. The main idea in the Welch-Berlekamp algorithm is to “forget” the constraint that $E(X)$ should divide $N(X)$ and focus on the remaining constraints: So $N(X)$ should still have degree at most $e + k - 1$, $E(X)$ should still have degree at most e , and for every i , we should have $N(\alpha_i) = y_i \cdot E(\alpha_i)$. This suggests a simple algorithm which we present below and then present the arguably subtle analysis of its correctness. (Note that switching from $P(X) \cdot E(X)$ to $N(X)$ is the so-called linearization. It simplifies the search, but may have completely changed the problem being solved. The analysis will aim to show that this switch still solves our target problem!)

12.1.2 Welch-Berlekamp Algorithm

Algorithm 12.1.1 formally states the Welch-Berlekamp algorithm.

12.1.3 Analysis of the Welch-Berlekamp Algorithm

As stated, apart from *Step 1*, all other steps of the algorithm are clearly efficiently implementable. *Step 1* is also efficiently implementable but we defer the proof of this for now. Instead we assume it can be solved efficiently as specified, and turn to analyzing the correctness of Algorithm 12.1.1.

Algorithm 12.1.1 Welch-Berlekamp Algorithm

INPUT: $n \geq k \geq 1$, $0 < e < \frac{n-k+1}{2}$ and n pairs $\{(\alpha_i, y_i)\}_{i=1}^n$ with α_i distinct

OUTPUT: Polynomial $P(X)$ of degree at most $k - 1$ or fail.

- 1: Compute a non-zero polynomial $E(X)$ of degree exactly e , and a polynomial $N(X)$ of degree at most $e + k - 1$ such that

$$y_i E(\alpha_i) = N(\alpha_i) \quad 1 \leq i \leq n. \quad (12.2)$$

- 2: IF $E(X)$ and $N(X)$ as above do not exist or $E(X)$ does not divide $N(X)$ THEN
 - 3: RETURN fail
 - 4: $P(X) \leftarrow \frac{N(X)}{E(X)}$.
 - 5: IF $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) > e$ THEN
 - 6: RETURN fail
 - 7: ELSE
 - 8: RETURN $P(X)$
-

Correctness of Algorithm 12.1.1. Note that if Algorithm 12.1.1 does not output fail, then the algorithm produces a correct output. Thus, to prove the correctness of Algorithm 12.1.1, we just need the following result.

Theorem 12.1.3. *If $(P(\alpha_i))_{i=1}^n$ is transmitted (where $P(X)$ is a polynomial of degree at most $k - 1$) and at most $e < \frac{n-k+1}{2}$ errors occur (i.e. $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$), then the Welch-Berlekamp algorithm outputs $P(X)$.*

Note that the above implies that the Welch-Berlekamp algorithm can correct RS codes of rate R for up to $\frac{1-R}{2}$ fraction of errors.

The proof of the theorem above follows from the following two claims.

Claim 12.1.4. *There exist a pair of polynomials $E^*(X)$ and $N^*(X)$ that satisfy Step 1 such that $\frac{N^*(X)}{E^*(X)} = P(X)$.*

Proof. We just take $E^*(X)$ to be an error-locating polynomial for $P(X)$ and let

$$N^*(X) = P(X)E^*(X),$$

where $\deg(N^*(X)) \leq \deg(P(X)) + \deg(E^*(X)) \leq e + k - 1$. In particular, define $E^*(X)$ as the following polynomial of degree exactly e :

$$E^*(X) = X^{e-\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n)} \prod_{1 \leq i \leq n | y_i \neq P(\alpha_i)} (X - \alpha_i). \quad (12.3)$$

By definition, $E^*(X)$ is a non-zero polynomial of degree exactly¹ e with the following property:

$$E^*(\alpha_i) = 0 \quad \text{iff} \quad y_i \neq P(\alpha_i).$$

¹The term $X^{e-\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n)}$ was introduced to make sure the degree is exactly e .

We now argue that $E^*(X)$ and $N^*(X)$ satisfy (12.2). Note that if $E^*(\alpha_i) = 0$, then $N^*(\alpha_i) = P(\alpha_i)E^*(\alpha_i) = y_i E^*(\alpha_i) = 0$. When $E^*(\alpha_i) \neq 0$, we know $P(\alpha_i) = y_i$ and so we still have $P(\alpha_i)E^*(\alpha_i) = y_i E^*(\alpha_i)$, as desired. $\square$

Given the claim above, we now have that for the condition in Step 2, a pair of polynomials satisfying (12.2) does exist. So the only way our algorithm can output “sf fail” is if it finds a pair of polynomials (E', N') such that $N'/E' \neq P$. To show this can not happen, we now claim that for any pair of solutions $((N_1(X), E_1(X))$ and $(N_2(X), E_2(X))$ that satisfy Step 1, we have $\frac{N_1(X)}{E_1(X)} = \frac{N_2(X)}{E_2(X)}$. Combined with Claim 12.1.4 this implies that the ratio $N'/E' = P$ for any solution produced in Step 1.

Claim 12.1.5. *If any two distinct solutions $(E_1(X), N_1(X)) \neq (E_2(X), N_2(X))$ satisfy Step 1, then they will satisfy*

$$\frac{N_1(X)}{E_1(X)} = \frac{N_2(X)}{E_2(X)}.$$

Proof. Let us define polynomial $R(X)$ with degree at most $2e + k - 1$ as follows:

$$R(X) = N_1(X)E_2(X) - N_2(X)E_1(X). \quad (12.4)$$

Note that the degrees of each of the polynomials $N_1(X)E_2(X)$ and $N_2(X)E_1(X)$ is at most $2e + k - 1$. Thus $R(X)$ is also a polynomial of degree at most $2e + k - 1$. Furthermore, from Step 1 we have, for every $i \in [n]$,

$$N_1(\alpha_i) = y_i E_1(\alpha_i) \quad \text{and} \quad N_2(\alpha_i) = y_i E_2(\alpha_i). \quad (12.5)$$

We use this to show that $R(\alpha_i) = 0$ for every $1 \leq i \leq n$. Specifically we have

$$\begin{aligned} R(\alpha_i) &= N_1(\alpha_i)E_2(\alpha_i) - N_2(\alpha_i)E_1(\alpha_i) \\ &= (y_i E_1(\alpha_i))E_2(\alpha_i) - (y_i E_2(\alpha_i))E_1(\alpha_i) \\ &= 0, \end{aligned}$$

where the first equality is by the definition of $R(X)$ (12.4), and the second one is obtained by substituting $N_1(\alpha_i)$ and $N_2(\alpha_i)$ using (12.5). Thus $R(X)$ is a polynomial of degree at most $2e_k - 1$ with n zeroes. By our choice of $e < \frac{n-k+1}{2}$ we have $2e + k - 1 < n$, thereby proving R is identically zero by the degree mantra (Proposition 5.1.5).

Using the definition of R again, we thus have $N_1(X)E_2(X) \equiv N_2(X)E_1(X)$. Note that as $E_1(X) \neq 0$ and $E_2(X) \neq 0$, this implies that $\frac{N_1(X)}{E_1(X)} = \frac{N_2(X)}{E_2(X)}$, as desired. $\square$

Proof of Theorem 12.1.3. From Claim 12.1.4 it follows that there exists a pair of polynomials (N_1, E_1) satisfying (12.2), with the further property that $N_1(X)/E_1(X) = P(X)$. Thus Step 1 must produce some pair (N_2, E_2) satisfying (12.2). By Claim 12.1.5 it follows that $N_2(X)/E_2(X) = N_1(X)/E_1(X) = P(X)$. Thus Algorithm 12.1.1 correctly outputs $P(X)$ in Step 8. $\square$

Implementation of Algorithm 12.1.1. We now argue that Algorithm 12.1.1 can be solved in polynomial time, specifically using at most $O(n^3)$ steps, where a single step may involve some field operations over $\mathbb{F}_q$.

Inspecting the algorithm, we see that other than Steps 1, 2 and 4, all other steps are simple bookkeeping tasks that can easily in $O(n)$ steps. Steps 2 and 4 require the division of $N(X)$ by $E(X)$ which can also be done (using long division) with $O(n^2)$ steps. The only remaining Step to analyze is Step 1. Here, the idea is to note that this step essentially involves solving a linear system, as we elaborate below.

Denote $E(X) = \sum_{j=0}^e E_j X^j$ and $N(X) = \sum_{j=0}^{e+k-1} N_j X^j$. The task of finding $E(X)$ and $N(X)$ is the same as finding the coefficients $E_0, \dots, E_e$ and $N_0, \dots, N_{e+k-1}$, which we treat hereon as variables. Now note that each of the constraints $y_i E(\alpha_i) = N(\alpha_i)$ is a linear equation in the variables. So (12.2) is essentially a linear system of equations in some variables. But we are not done: One more constraint needs to be enforced, namely that $E(X)$ has degree exactly e . This amounts to saying $E_e \neq 0$, but this is not a linear constraint. However if we replace this with the constraint $E_e = 1$, then we get a linear system that can be solved in $O(n^3)$ time. We claim this solves the task in Step 1. To see this note that every solution to our linear system is also a solution to the task defined in Step 1 — in particular $E_e = 1$ forces the degree of $E(X)$ to be exactly e as required. Conversely suppose Step 1 has a solution $E(X)$ and $N(X)$ with $E_e \neq 0$. Then note that $E'(X) = E_e^{-1} \cdot E(X)$ and $N'(X) = E_e^{-1} \cdot N(X)$ also form a solution to Step 1, with the coefficient of X^e in $E'(X)$ being exactly one. So if Step 1 has a solution, then so does our linear system. We conclude that Step 1 can be correctly solved in $O(n^3)$ time by solving this linear system. Thus the entire algorithm described above runs in $O(n^3)$ time.

Thus, we have proved that Algorithm 12.1.1 runs in polynomial time. We have thus proven the following theorem.

Theorem 12.1.6. *The Reed-Solomon Unique Decoding can be solved in $O(n^3)$ time.*

Recall that the above is a restatement of the error decoding part of Theorem 14.2.1. Thus, this fills in the final missing piece from the proofs of Theorem 14.3.3 (decoding certain concatenated codes up to half of their design distance) and Theorem 15.4.1 (efficiently achieving the BSC_p capacity).

12.2 List Decoding Reed-Solomon Codes

Recall Question 7.4.3, which asks if there is an efficient list decoding algorithm for a code of rate $R > 0$ that can correct $1 - \sqrt{R}$ fraction of errors, i.e., up to the Johnson bound (Theorem 7.3.1) for the code, assuming the code meets the Singleton bound and achieves $\delta = 1 - R$. In the rest of this chapter we answer this question affirmatively by showing that Reed-Solomon codes of rate R can be efficiently list-decoded up to $1 - \sqrt{R}$ fraction of errors. Note that this gives us an explicit code answering Question 7.4.3. To this end, we will present a sequence of algorithms for (list) decoding Reed-Solomon codes that will list-decode from an increasing fraction of errors, till we ultimately answer Question 7.4.3.

Before we talk about the algorithms, we restate the (list) decoding problem for Reed-Solomon codes.

Problem 12.2.1 (Reed-Solomon List Decoding).

- **Input:** Code Parameters: $\mathbb{F}_q, (\alpha_1, \dots, \alpha_n) \in \mathbb{F}_q^n$ and k and e . Received word: $\mathbf{y} = (y_1, \dots, y_n) \in \mathbb{F}_q^n$.
- **Output:** A list (set) of all polynomials $P(X) \in \mathbb{F}_q[X]$ of degree less than k such that $t := |\{i \in [n] \mid y_i \neq P(\alpha_i)\}| \geq n - e$.

Our main goal of course is to make t as small as possible or e as large as possible. The Johnson bound in this setting would allow t to be as small as $\sqrt{nk}$. In contrast the unique decoding setting corresponds to the $t > \frac{n+k}{2}$. (Note that by the AM-GM inequality we always have $\sqrt{nk} \leq \frac{n+k}{2}$.)

12.2.1 Structure of the (list-)decoding algorithms

We start by reviewing the Welch-Berlekamp algorithm in Algorithm 12.1.1, which we restate below in a slightly different form (that will be useful in developing the subsequent list decoding algorithms).

- **Step 1:** Find polynomials $N(X)$ of degree $k + e - 1$, and $E(X)$ of degree e such that

$$N(\alpha_i) = y_i E(\alpha_i), \text{ for every } 1 \leq i \leq n$$

- **Step 2:** If $Y - P(X)$ divides $Q(X, Y) := YE(X) - N(X)$, then output $P(X)$ (assuming $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$).

Note that $Y - P(X)$ divides $Q(X, Y)$ in **Step 2** above if and only if² $P(X) = \frac{N(X)}{E(X)}$, which is exactly what Step 4 does in Algorithm 12.1.1.

Rewriting the Welch-Berlekamp Algorithm yet another time, note that we can interpret the algorithm as having an “interpolation” step and a “root-finding” step as follows:

- **Step 1:** (Interpolation Step) Find non-zero $Q(X, Y)$ such that $Q(\alpha_i, y_i) = 0, 1 \leq i \leq n$.
- **Step 2:** (Root Finding Step) If $Y - P(X)$ is a factor of $Q(X, Y)$, then output $P(X)$ (assuming it is close enough to the received word).

In particular, in the Welch-Berlekamp algorithm we require that $Q(X, Y) = YE(X) - N(X)$ so that the condition $Q(\alpha_i, y_i) = 0$ is equivalent to the condition $N(\alpha_i) = y_i E(\alpha_i)$. The terminology “interpolation” alludes to the fact that Step 1 aims to find an algebraic explanation of the data,

²Indeed $Q(X, Y) = E(X) \left(Y - \frac{N(X)}{E(X)} \right)$, which means $Y - \frac{N(X)}{E(X)}$ is the only linear (in Y) factor of $Q(X, Y)$.

which is essentially what polynomial interpolation does. The term “root-finding” comes from viewing $Q(X, Y) \in \mathbb{F}_q[X, Y]$ as a polynomial $Q_X(Y) \in (\mathbb{F}_q[X])[Y]$, i.e., a polynomial in Y with coefficients from the ring $\mathbb{F}_q[X]$. In this view, a root of Q_X would be some element $P(X) \in \mathbb{F}_q[X]$ such that $Q_X(P(X)) = 0$ or equivalently $Y - P(X)$ divides $Q(X, Y)$.

The efficient implementation of the algorithm above relies on the fact that Interpolation Step can be solved by solving a linear system (possibly with some minor variations), and the Root-Finding step is easy in the special case where $Q(X, Y)$ is linear in Y , i.e., $Q(X, Y) = E(X)Y - N(X)$ and the root must be $N(X)/E(X)$ if one exists.

All the list decoding algorithms that we will consider in this chapter will have the same two-step structure. The algorithms will differ in how exactly **Step 1** is implemented, but in all cases we will aim to set the problem up so that it can be solved by solving some linear system. We also make a key observation that will effectively ‘take care of’ **Step 2** for us. Note that **Step 2** can be implemented if one can factorize the bivariate polynomial $Q(X, Y)$ (and then only retain the linear factors of the form $Y - P(X)$). Fortunately, it is known that factoring bivariate polynomials can be done in polynomial time. We will need in particular the ability to find roots of bivariate polynomials in polynomial time, as captured in Theorem D.7.9. (See also the proof of this theorem for the algorithmic ideas behind polynomial factorization.)

Armed with these tools we are now ready to present our algorithms for list-decoding Reed-Solomon codes. We will present three instantiations of the algorithm, which we call the “Basic List-Decoder”, the “Weighted-Degree List-Decoder” and the “Multiplicity List-Decoder”. They offer the following tradeoffs (for an $[n, k]$ RS code with rate $R = \frac{k}{n}$):

	Basic LD	Weighted-Deg. LD	Multiplicity LD
Frac. of errors	$1 - 2\sqrt{R}$	$1 - \sqrt{2R}$	$1 - \sqrt{R}$
t	$2\sqrt{nk}$	$\sqrt{2nk}$	$\sqrt{nk}$

We start with the Basic List-Decoder.

12.2.2 Basic List-Decoder

The main insight in the list decoding algorithm that if we allow the degree of the posynomial $Q(X, Y)$ to be sufficiently large then its existence is easy to establish (and so **Step 1** will succeed. However using too high a degree makes Q useless. The key is to control its degree carefully so that the existence proof goes through, while it remains useful in **Step 2**. Here we will use the degree restrictions, along with the degree mantra (Proposition 5.1.5), to show that **Step 2** will succeed too (and find all polynomials that agree with the received word often).

In the Basic version of the list-decoder we restrict the degree of Q is a very simple way - by separately considering its degrees in X and Y and restricting each separately. We recall the definition of maximum degree of a variable.

Definition 12.2.2. $\deg_X(Q)$ is the maximum degree of X in any monomial of $Q(X, Y)$. Similarly, $\deg_Y(Q)$ is the maximum degree of Y in any monomial of $Q(X, Y)$.

For example, for $Q(X, Y) = X^2Y^3 + X^4Y^2$, $\deg_X(Q) = 4$ and $\deg_Y(Q) = 3$. Given $\deg_X(Q) = a$ and $\deg_Y(Q) = b$, we can write

$$Q(X, Y) = \sum_{\substack{0 \leq i \leq a, \\ 0 \leq j \leq b}} c_{ij} X^i Y^j,$$

where the coefficients $c_{ij} \in \mathbb{F}_q$. Note that the number of coefficients is equal to $(a+1)(b+1)$.

The main idea in the first list decoding algorithm for Reed-Solomon code is to place bounds on $\deg_X(Q)$ and $\deg_Y(Q)$ for **Step 1**. The bounds are chosen so that there are enough variables to guarantee the existence of a $Q(X, Y)$ with the required properties. We will then use these bounds along with the degree mantra (Proposition 5.1.5) to argue that **Step 2** works. Algorithm 12.2.1 presents the details. Note that while this algorithm generalizes the spirit of the Welch-Berlekamp algorithm, our analysis will show that its performance is incomparable. (I.e., there exist choices of (n, k, t) where the Basic List-Decoder recovers P while the Welch-Berlekamp algorithm does not, and vice versa.)

Algorithm 12.2.1 The **Basic List-Decoder** for Reed-Solomon Codes

INPUT: $n \geq k \geq 1$, $\ell \geq 1$, $e = n - t$ and n pairs $\{(\alpha_i, y_i)\}_{i=1}^n$

OUTPUT: (Possibly empty) list of polynomials $P(X)$ of degree at most $k-1$

1: **Step 1** Find a non-zero $Q(X, Y)$ with $\deg_X(Q) \leq \ell$, $\deg_Y(Q) \leq \frac{n}{\ell}$ such that

$$Q(\alpha_i, y_i) = 0, 1 \leq i \leq n. \quad (12.6)$$

2: **Step 2:** Factor $Q(X, Y)$ into irreducible factors $Q_1(X, Y), \dots, Q_m(X, Y)$.

3: $\mathbb{L} \leftarrow \emptyset$

4: FOR every factor $Q_j(X, Y) = Y - P_j(X)$ of $Q(X, Y)$ DO

5: IF $\Delta(\mathbf{y}, (P_j(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P_j) \leq k-1$ THEN

6: Add $P_j(X)$ to $\mathbb{L}$.

7: RETURN $\mathbb{L}$

Note that **Step 1** and **Step 2** are really the main steps in the algorithm, with remaining steps being some simple post-processing to prune the output list. We now analyze the runtime and correctness.

Run time analysis. The run time of the algorithm is clearly polynomial conditioned on Steps 1 and 2 being solvable in polynomial time. Step 1 needs to find the coefficients of the polynomial $Q(X, Y)$, i.e., find assignments to the variables $\{Q_{ij}\}_{i \in \{0, \dots, \ell\}; j \in \{0, \dots, \frac{n}{\ell}\}}$ in $\mathbb{F}_q$ such that not all Q_{ij} 's are zero and satisfying (12.6). Since each constraint from (12.6) is a homogenous linear constraint on the variables, this amounts to finding a non-trivial (non-zero) solution to a homogenous linear system, which can be done in polynomial time (using say Gaussian Elimination). Turning to Step 2, this amounts to finding the root of a bivariate polynomial which can be done in polynomial time using Theorem D.7.9. We conclude that the Basic List-Decoder can be implemented in polynomial time.

Correctness of Algorithm 12.2.1. We claim first that Step 1 always returns a non-zero polynomial Q satisfying (12.6). This is obviously true if and only if there exists a polynomial satisfying this condition and we argue this below.

Lemma 12.2.3. *For every input sequence ...*

To ensure the correctness of Step 1, we will need to ensure that the number of coefficients for $Q(X, Y)$ (which is $(\ell+1)(n/\ell+1)$) is larger than the number of constraints in (12.6) (which is n). Indeed, note that we have

$$(\ell+1) \cdot \left(\frac{n}{\ell} + 1\right) > \ell \cdot \frac{n}{\ell} = n.$$

We need to argue that the final $\mathbb{L}$ in Step 6 contains all the polynomials $P(X)$ that need to be output. In other words, we need to show that if $P(X)$ of degree $\leq k-1$ agrees with Y in at least t positions, then $Y - P(X)$ divides $Q(X, Y)$. Towards this end, we define

$$R(X) \stackrel{\text{def}}{=} Q(X, P(X)).$$

Note that $Y - P(X)$ divides $Q(X, Y)$ if and only if $R(X) \equiv 0$. Thus, we need to show $R(X) \equiv 0$. For the sake of contradiction, assume that $R(X) \not\equiv 0$. Note that

$$\begin{aligned} \deg(R) &\leq \deg_X(Q) + \deg(P) \cdot \deg_Y(Q) \\ &\leq \ell + \frac{n(k-1)}{\ell}. \end{aligned}$$

In the above the first inequality follows from the definition of $R(X)$ while the second inequality follows from our assumptions on $\deg_X(Q)$ and $\deg_Y(Q)$. On the other hand, if $P(\alpha_i) = y_i$ then (12.6) implies that

$$Q(\alpha_i, y_i) = Q(\alpha_i, P(\alpha_i)) = 0.$$

Thus, α_i is a root of $R(X)$. In other words, R has at least t roots. Note that the degree mantra (Proposition 5.1.5) this will lead to a contradiction if $t > \deg(R)$, which will be true if

$$t > \ell + \frac{n(k-1)}{\ell}.$$

If we pick $\ell = \sqrt{n(k-1)}$, we will have $t > 2\sqrt{n(k-1)}$. Thus, we have shown that:

Theorem 12.2.4. *Algorithm 12.2.1 can list decode Reed-Solomon codes of rate R from $1 - 2\sqrt{R}$ fraction of errors. Further, the algorithm can be implemented in polynomial time.*

The claim on the efficient run time follows as Step 1 can be implemented by Gaussian elimination and for Step 3, all the factors of $Q(X, Y)$ (and in particular all linear factors of the form $Y - P(X)$) can be computed using e.g. the algorithm from [97].

The bound $1 - 2\sqrt{R}$ is better than the unique decoding bound of $\frac{1-R}{2}$ for $R < 0.07$. This is still far from the $1 - \sqrt{R}$ fraction of errors guaranteed by the Johnson bound. See Figure 12.2.2 for an illustration.

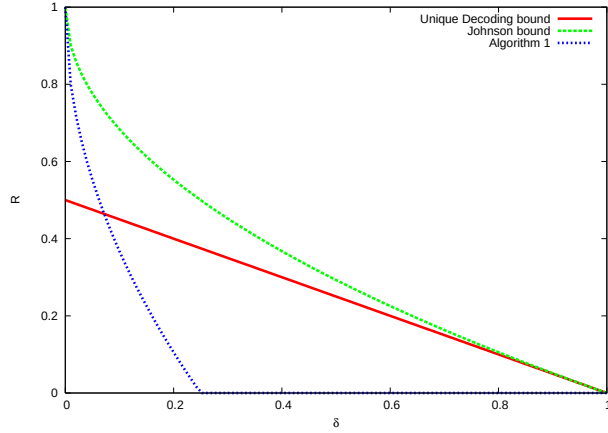


Figure 12.3: The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 12.2.1.

12.2.3 Algorithm 2

To motivate the next algorithm, recall that in Algorithm 12.2.1, in order to prove that the root finding step (Steps 3-6 in Algorithm 12.2.1) works, we defined a polynomial $R(X) \stackrel{\text{def}}{=} Q(X, P(X))$. In particular, this implied that $\deg(R) \leq \deg_X(Q) + (k-1) \cdot \deg_Y(Q)$ (and we had to select $t > \deg_X(Q) + (k-1) \cdot \deg_Y(Q)$). One shortcoming of this approach is that the maximum degree of X and Y might not occur in the same term. For example, in the polynomial $X^2Y^3 + X^4Y^2$, the maximum X and Y degrees do not occur in the same monomial. The main insight in the new algorithm is to use a more “balanced” notion of degree of $Q(X, Y)$:

Definition 12.2.5. The $(1, w)$ weighted degree of the monomial X^iY^j is $i + wj$. Further, the $(1, w)$ -weighted degree of $Q(X, Y)$ (or just its $(1, w)$ degree) is the maximum $(1, w)$ weighted degree of its monomials.

For example, the $(1, 2)$ -degree of the polynomial $XY^3 + X^4Y$ is $\max(1 + 3 \cdot 2, 4 + 2 \cdot 1) = 7$. Also note that the $(1, 1)$ -degree of a bivariate polynomial $Q(X, Y)$ is its total degree (or the “usual” definition of degree of a bivariate polynomial). Finally, we will use the following simple lemma (whose proof we leave as an exercise):

Lemma 12.2.6. Let $Q(X, Y)$ be a bivariate polynomial of $(1, w)$ degree D . Let $P(X)$ be a polynomial such that $\deg(P) \leq w$. Then we have

$$\deg(Q(X, P(X))) \leq D.$$

Note that a bivariate polynomial $Q(X, Y)$ of $(1, w)$ degree at most D can be represented as follows:

$$Q(X, Y) \stackrel{\text{def}}{=} \sum_{\substack{i+wj \leq D \\ i,j \geq 0}} c_{i,j} X^i Y^j,$$

where $c_{i,j} \in \mathbb{F}_q$.

The new algorithm is basically the same as Algorithm 12.2.1, except in the interpolation step, where we compute a bivariate polynomial of bounded $(1, k-1)$ degree. Before we state the precise algorithm, we will present the algorithm via an example. Consider the received word in Figure 12.4.

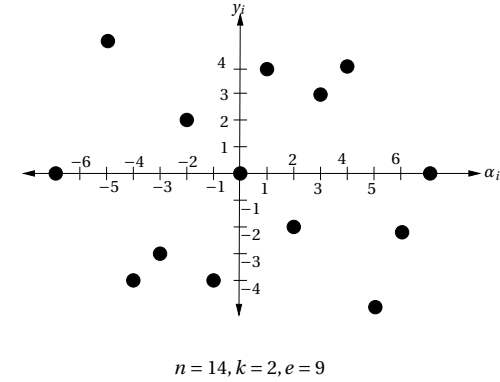


Figure 12.4: An illustration of a received word for the $[14, 2]$ Reed-Solomon code from Figure 12.1 (where again we have implicitly embedded the field $\mathbb{F}_q$ in the set $\{-7, \dots, 7\}$). Here we have considered $e = 9$ errors which is more than what Algorithm 12.1.1 can handle. In this case, we are looking for lines that pass through at least 5 points. By comparison for unique decoding, we would need $t \geq \frac{n+k}{2} = \frac{14+2}{2} = 8$, which is higher than the agreement of 5 we want to handle with list decoding.

Now we want to interpolate a bivariate polynomial $Q(X, Y)$ with a $(1, 1)$ degree of 4 that “passes” through all the 2-D points corresponding to the received word from Figure 12.4. Figure 12.5 shows such an example.

Finally, we want to factorize all the linear factors $Y - P(X)$ of the $Q(X, Y)$ from Figure 12.5. Figure 12.6 shows the two polynomials X and $-X$ such that $Y - X$ and $Y + X$ are factors of $Q(X, Y)$ from Figure 12.5.

We now precisely state the new list decoding algorithm in Algorithm 12.2.2.

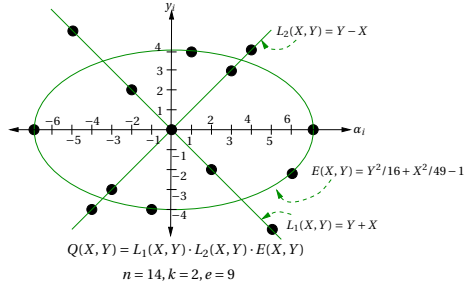


Figure 12.5: An interpolating polynomial $Q(X, Y)$ for the received word in Figure 12.4.

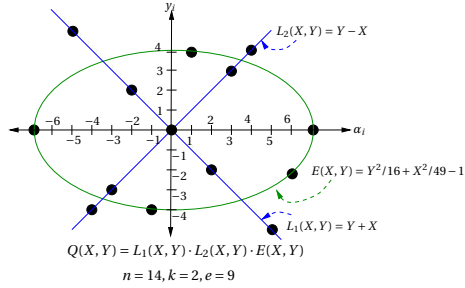


Figure 12.6: The two polynomials that need to be output are shown in blue.

Proof of Correctness of Algorithm 12.2.2. As in the case of Algorithm 12.2.1, to prove the correctness of Algorithm 12.2.2, we need to do the following:

- (Interpolation Step) Ensure that the number of coefficients of $Q(X, Y)$ is strictly greater than n .
- (Root Finding Step) Let $R(X) \stackrel{\text{def}}{=} Q(X, P(X))$. We want to show that if $P(\alpha_i) \geq y_i$ for at least t values of i , then $R(X) \equiv 0$.

To begin with, we argue why we can prove the correctness of the root finding step. Note that since $Q(X, Y)$ has $(1, k-1)$ degree at most D , Lemma 12.2.6 implies that

$$\deg(R) \leq D.$$

Then using the same argument as we used for the correctness of the root finding step of Algo-

Algorithm 12.2.2 The Second List Decoding Algorithm for Reed-Solomon Codes

INPUT: $n \geq k \geq 1$, $D \geq 1$, $e = n - t$ and n pairs $\{(\alpha_i, y_i)\}_{i=1}^n$

OUTPUT: (Possibly empty) list of polynomials $P(X)$ of degree at most $k-1$

- 1: Find a non-zero $Q(X, Y)$ with $(1, k-1)$ degree at most D , such that

$$Q(\alpha_i, y_i) = 0, 1 \leq i \leq n. \quad (12.7)$$

- 2: $\mathbb{L} \leftarrow \emptyset$
- 3: FOR every factor $Y - P(X)$ of $Q(X, Y)$ DO
- 4: IF $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P) \leq k-1$ THEN
- 5: Add $P(X)$ to $\mathbb{L}$.
- 6: RETURN $\mathbb{L}$

rithm 12.2.1, we can ensure $R(X) \equiv 0$ if we pick

$$t > D.$$

Thus, we would like to pick D to be as small as possible. On the other hand, Step 1 will need D to be large enough (so that the number of variables is more than the number of constraints in (12.7)). Towards that end, let the number of coefficients of $Q(X, Y)$ be

$$\mathcal{N} = |\{(i, j) | i + (k-1)j \leq D, i, j \in \mathbb{Z}^+\}|$$

To bound $\mathcal{N}$, we first note that in the definition above, $j \leq \lfloor \frac{D}{k-1} \rfloor$. Define $L = \lfloor \frac{D}{k-1} \rfloor$, which will be the list-size output by the algorithm. Consider the following sequence of relationships:

$$\begin{aligned} \mathcal{N} &= \sum_{j=1}^L \sum_{i=0}^{D-(k-1)j} 1 \\ &= \sum_{j=0}^L (D - (k-1)j + 1) \\ &= \sum_{j=0}^L (D+1) - (k-1) \sum_{j=0}^L j \\ &= (D+1)(L+1) - \frac{(k-1)L(L+1)}{2} \\ &= \frac{L+1}{2} (2D+2 - (k-1)L) \\ &\geq \left(\frac{L+1}{2} \right) (D+2) \end{aligned} \quad (12.8)$$

$$\geq \frac{D(D+2)}{2(k-1)}. \quad (12.9)$$

In the above, (12.8) follows from the fact that $L \leq \frac{D}{k-1}$ and (12.9) follows from the fact that $\frac{D}{k-1} - 1 \leq L$.

Thus, the interpolation step succeeds (i.e. there exists a non-zero $Q(X, Y)$ with the required properties) if

$$\frac{D(D+2)}{2(k-1)} > n.$$

The choice

$$D = \lceil \sqrt{2(k-1)n} \rceil$$

suffices by the following argument:

$$\frac{D(D+2)}{2(k-1)} > \frac{D^2}{2(k-1)} \geq \frac{2(k-1)n}{2(k-1)} = n.$$

Thus for the root finding step to work, we need $t > \lceil \sqrt{2(k-1)n} \rceil$, which implies the following result:

Theorem 12.2.7. *Algorithm 2 can list decode Reed-Solomon codes of rate R from up to $1 - \sqrt{2R}$ fraction of errors. Further, the algorithm runs in polynomial time, and outputs a list of size at most $O(1/\sqrt{R})$.*

Algorithm 2 runs in polynomial time as Step 1 can be implemented using Gaussian elimination (and the fact that the number of coefficients is $O(n)$), while the root finding step can be implemented by any polynomial time algorithm to factorize bivariate polynomials. Further, we note that $1 - \sqrt{2R}$ beats the unique decoding bound of $(1 - R)/2$ for $R < 1/3$. See Figure 12.2.3 for an illustration.

12.2.4 Algorithm 3

Finally, we present the list decoding algorithm for Reed-Solomon codes, which can correct $1 - \sqrt{R}$ fraction of errors. The main idea is to add more restrictions on $Q(X, Y)$ (in addition to its $(1, k-1)$ -degree being at most D). In particular, the restriction is as follows: for some integer parameter $r \geq 1$, we will insist on $Q(X, Y)$ having r roots at (α_i, y_i) , $1 \leq i \leq n$ (we will come to the formal definition of this shortly).

This change will have the following implications:

1. The number of equations (on the coefficients of Q) will increase but the number of coefficients will remain the same. This seems to be bad, as this results in an increase in D (which in turn would result in an increase in t).
2. However, this change also increases the number of roots of $R(X)$ and this gain in the number of roots more than compensates for the increase in D .

To motivate the definition of multiplicity of a root of a bivariate polynomial, let us consider the following simplified examples.

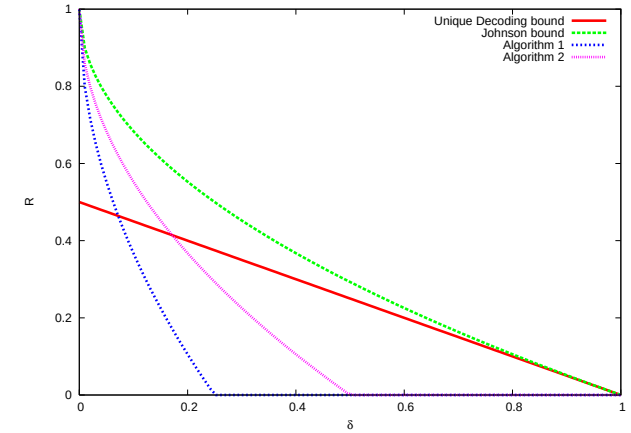


Figure 12.7: The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 12.2.1 and Algorithm 12.2.2.

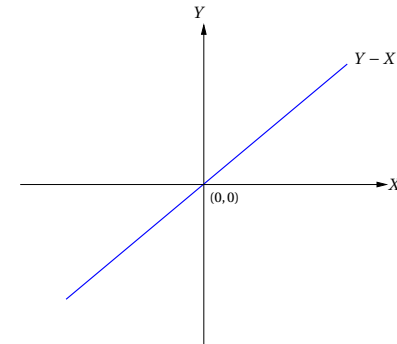


Figure 12.8: Multiplicity of 1

In Figure 12.8 the curve $Q(X, Y) = Y - X$ passes through the origin once and has no term of degree 0.

In Figure 12.9, the curve $Q(X, Y) = (Y - X)(Y + X)$ passes through the origin twice and has no term with degree at most 1.

In Figure 12.10, the curve $Q(X, Y) = (Y - X)(Y + X)(Y - 2X)$ passes through the origin thrice

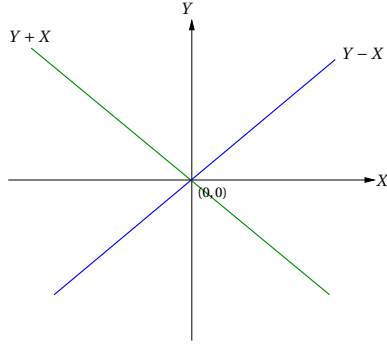


Figure 12.9: Multiplicity of 2

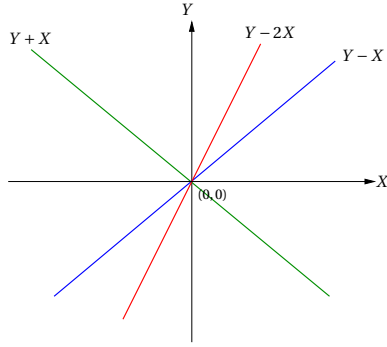


Figure 12.10: Multiplicity of 3

and has no term with degree at most 2. More generally, if r lines pass through the origin, then note that the curve corresponding to their product has no term with degree at most $r - 1$. This leads to the following more general definition:

Definition 12.2.8. $Q(X, Y)$ has r roots at $(0, 0)$ if $Q(X, Y)$ doesn't have any monomial with degree at most $r - 1$.

The definition of a root with multiplicity r at a more general point follows from a simple translation:

Definition 12.2.9. $Q(X, Y)$ has r roots at (α, β) if $Q_{\alpha, \beta}(X, Y) \stackrel{\text{def}}{=} Q(x + \alpha, y + \beta)$ has r roots at $(0, 0)$.

Before we state the precise algorithm, we will present the algorithm with an example. Consider the received word in Figure 12.11.

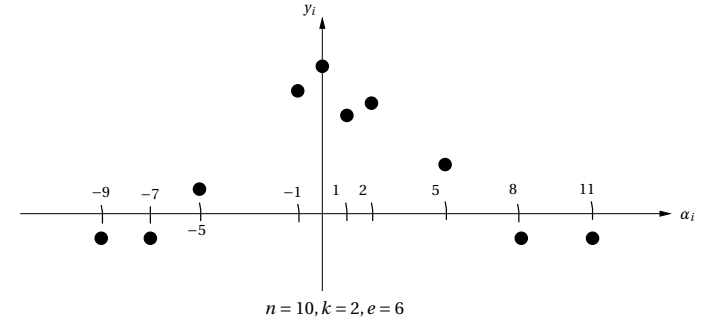


Figure 12.11: An illustration of a received word for the $[10, 2]$ Reed-Solomon code (where we have implicitly embedded the field $\mathbb{F}_q$ in the set $\{-9, \dots, 11\}$). Here we have considered $e = 6$ errors, which is more than what Algorithm 12.2.2 can decode. In this case, we are looking for lines that pass through at least 4 points.

Now we want to interpolate a bivariate polynomial $Q(X, Y)$ with $(1, 1)$ degree 5 that “passes twice” through all the 2-D points corresponding to the received word from Figure 12.11. Figure 12.12 shows such an example.

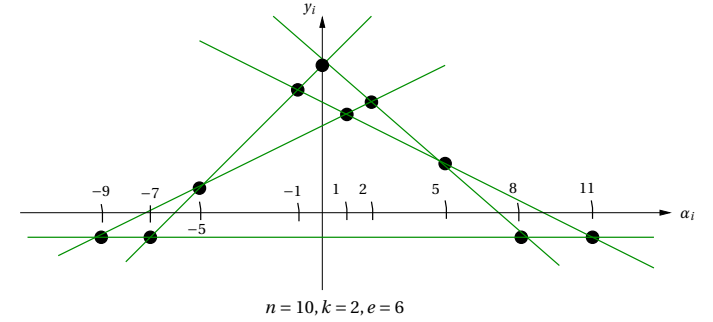


Figure 12.12: An interpolating polynomial $Q(X, Y)$ for the received word in Figure 12.11.

Finally, we want to factorize all the linear factors $Y - P(X)$ of the $Q(X, Y)$ from Figure 12.12. Figure 12.13 shows the five polynomials of degree one are factors of $Q(X, Y)$ from Figure 12.12. (In fact, $Q(X, Y)$ exactly decomposes into the five lines.) Algorithm 12.2.3 formally states the algorithm.

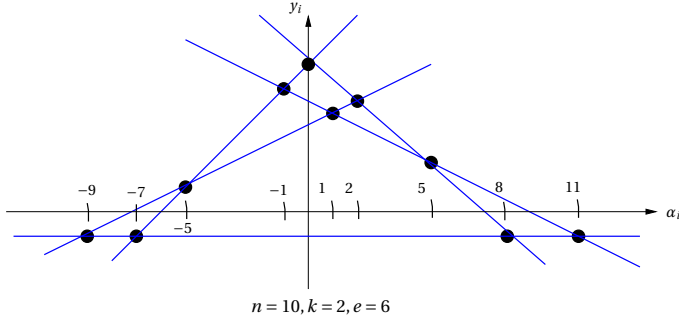


Figure 12.13: The five polynomials that need to be output are shown in blue.

Algorithm 12.2.3 The Third List Decoding Algorithm for Reed-Solomon Codes

INPUT: $n \geq k \geq 1$, $D \geq 1$, $r \geq 1$, $e = n - t$ and n pairs $\{(\alpha_i, y_i)\}_{i=1}^n$

OUTPUT: (Possibly empty) list of polynomials $P(X)$ of degree at most $k - 1$

1: Find a non-zero $Q(X, Y)$ with $(1, k - 1)$ degree at most D , such that

$$Q(\alpha_i, y_i) = 0, \text{ with multiplicity } r \text{ for every } 1 \leq i \leq n. \quad (12.10)$$

2: $\mathbb{L} \leftarrow \emptyset$

3: FOR every factor $Y - P(X)$ of $Q(X, Y)$ DO

4: IF $\Delta(\mathbf{y}, (P(\alpha_i))_{i=1}^n) \leq e$ and $\deg(P) \leq k - 1$ THEN

5: Add $P(X)$ to $\mathbb{L}$.

6: RETURN $\mathbb{L}$

Correctness of Algorithm 12.2.3. To prove the correctness of Algorithm 12.2.3, we will need the following two lemmas (we defer the proofs of the lemmas above to Section 12.2.4):

Lemma 12.2.10. The constraints in (12.10) imply $\binom{r+1}{2}$ constraints for each i on the coefficients of $Q(X, Y)$.

Lemma 12.2.11. $R(X) \stackrel{\text{def}}{=} Q(X, P(X))$ has r roots for every i such that $P(\alpha_i) = y_i$. In other words, $(X - \alpha_i)^r$ divides $R(X)$.

Using arguments similar to those used for proving the correctness of Algorithm 12.2.2, to argue the correctness of the interpolations step we will need

$$\frac{D(D+2)}{2(k-1)} > n \binom{r+1}{2},$$

where the LHS is an upper bound on the number of coefficients of $Q(X, Y)$ as before from (12.9) and the RHS follows from Lemma 12.2.10. Note that the above is equivalent to

$$\frac{D(D+2)}{(k-1)} > n(r+1)r,$$

which in turn is equivalent to

$$D^2 + 2D > n(k-1)(r+1)r.$$

We note that for the above to hold, the choice

$$D = \left\lceil \sqrt{(k-1)nr(r+1)} \right\rceil$$

works. Thus, we have shown the correctness of Step 1.

For the correctness of the root finding step, we need to show that the number of roots of $R(X)$ (which by Lemma 12.2.11 is at least rt) is strictly bigger than the degree of $R(X)$, which from Lemma 12.2.6 is D . That is we would be fine if we have,

$$tr > D,$$

which is the same as

$$t > \frac{D}{r},$$

which in turn will follow if we pick

$$t = \left\lceil \sqrt{(k-1)n \left(1 + \frac{1}{r}\right)} \right\rceil.$$

If we pick $r = 2(k-1)n$, then we will need

$$t > \left\lceil \sqrt{(k-1)n + \frac{1}{2}} \right\rceil.$$

The above is satisfied if we have $t \geq \sqrt{kn}$ (since we have for any $n \geq 1$, $-n + 1/2 < 0$). Thus, we have shown

Theorem 12.2.12. Algorithm 12.2.3 can list decode Reed-Solomon codes of rate R from up to $1 - \sqrt{R}$ fraction of errors. Further, the algorithm runs in polynomial time.

The claim on the run time follows from the same argument that was used to argue the polynomial running time of Algorithm 12.2.2. Thus, Theorem 12.2.12 shows that Reed-Solomon codes can be efficiently decoded up to the Johnson bound. For an illustration of fraction of errors correctable by the three list decoding algorithms we have seen, see Figure 12.2.3.

A natural question to ask is if Reed-Solomon codes of rate R can be list decoded beyond $1 - \sqrt{R}$ fraction of errors. The answer is still not known:

Open Question 12.2.1. Given a Reed-Solomon code of rate R , can it be efficiently list decoded beyond $1 - \sqrt{R}$ fraction of errors?

Recall that to complete the proof of Theorem 12.2.12, we still need to prove Lemmas 12.2.10 and 12.2.11, which we do next.

Proof of key lemmas

Proof of Lemma 12.2.10. Let

$$Q(X, Y) = \sum_{i+j \leq D} c_{i,j} X^i Y^j$$

and

$$Q_{\alpha,\beta}(X, Y) = Q(X + \alpha, Y + \beta) = \sum_{i,j} c_{i,j}^{\alpha,\beta} X^i Y^j.$$

We will show that

- (i) $c_{i,j}^{\alpha,\beta}$ are homogeneous linear combinations of $c_{i,j}$'s.
- (ii) If $Q_{\alpha,\beta}(X, Y)$ has no monomial with degree $< r$, then that implies $\binom{r+1}{2}$ constraints on $c_{i,j}^{\alpha,\beta}$'s.

Note that (i) and (ii) prove the lemma. To prove (i), note that by the definition:

$$Q_{\alpha,\beta}(X, Y) = \sum_{i,j} c_{i,j}^{\alpha,\beta} X^i Y^j \quad (12.11)$$

$$= \sum_{\substack{i',j' \\ i'+(k-1)j' \leq D}} c_{i',j'} (X + \alpha)^{i'} (Y + \beta)^{j'} \quad (12.12)$$

Note that, if $i > i'$ or $j > j'$, then $c_{i,j}^{\alpha,\beta}$ doesn't depend on $c_{i',j'}$. By comparing coefficients of $X^i Y^j$ from (12.11) and (12.12), we obtain

$$c_{i,j}^{\alpha,\beta} = \sum_{\substack{i' > i \\ j' > j}} c_{i',j'} \binom{i'}{i} \binom{j'}{j} \alpha^i \beta^j,$$

which proves (i). To prove (ii), recall that by definition $Q_{\alpha,\beta}(X, Y)$ has no monomial of degree $< r$. In other words, we need to have constraints $c_{i,j}^{\alpha,\beta} = 0$ if $i + j \leq r - 1$. The number of such constraints is

$$|\{(i, j) | i + j \leq r - 1, i, j \in \mathbb{Z}^{\geq 0}\}| = \binom{r+1}{2},$$

where the equality follows from the following argument. Note that for every fixed value of $0 \leq j \leq r - 1$, i can take $r - j$ values. Thus, we have that the number of constraints is

$$\sum_{j=0}^{r-1} r - j = \sum_{\ell=1}^r \ell = \binom{r+1}{2},$$

as desired. $\square$

We now re-state Lemma 12.2.11 more precisely and then prove it.

Lemma 12.2.13. Let $Q(X, Y)$ be computed by Step 1 in Algorithm 12.2.3. Let $P(X)$ be a polynomial of degree $\leq k - 1$, such that $P(\alpha_i) = y_i$ for at least $t > \frac{D}{r}$ many values of i , then $Y - P(X)$ divides $Q(X, Y)$.

Proof. Define

$$R(X) \stackrel{\text{def}}{=} Q(X, P(X)).$$

As usual, to prove the lemma, we will show that $R(X) \equiv 0$. To do this, we will use the following claim.

Claim 12.2.14. If $P(\alpha_i) = y_i$, then $(X - \alpha_i)^r$ divides $R(X)$, that is α_i is a root of $R(X)$ with multiplicity r .

Note that by definition of $Q(X, Y)$ and $P(X)$, $R(X)$ has degree $\leq D$. Assuming the above claim is correct, $R(X)$ has at least $t \cdot r$ roots. Therefore, by the degree mantra (Proposition 5.1.5), $R(X)$ is a zero polynomial as $t \cdot r > D$. We will now prove Claim 12.2.14. Define

$$P_{\alpha_i, y_i}(X) \stackrel{\text{def}}{=} P(X + \alpha_i) - y_i, \quad (12.13)$$

and

$$R_{\alpha_i, y_i}(X) \stackrel{\text{def}}{=} R(X + \alpha_i) \quad (12.14)$$

$$= Q(X + \alpha_i, P(X + \alpha_i)) \quad (12.15)$$

$$= Q(X + \alpha_i, P_{\alpha_i, y_i}(X) + y_i) \quad (12.16)$$

$$= Q_{\alpha_i, y_i}(X, P_{\alpha_i, y_i}(X)), \quad (12.17)$$

where (12.15), (12.16) and (12.17) follow from the definitions of $R(X)$, $P_{\alpha_i, y_i}(X)$ and $Q_{\alpha_i, y_i}(X, Y)$ respectively.

By (12.14) if $R_{\alpha_i, y_i}(0) = 0$, then $R(\alpha_i) = 0$. So, if X divides $R_{\alpha_i, y_i}(X)$, then $X - \alpha_i$ divides $R(X)$. (This follows from a similar argument that we used to prove Proposition 5.1.5.) Similarly, if X^r divides $R_{\alpha_i, y_i}(X)$, then $(X - \alpha_i)^r$ divides $R(X)$. Thus, to prove the lemma, we will show that X^r divides $R_{\alpha_i, y_i}(X)$. Since $P(\alpha_i) = y_i$ when α_i agrees with y_i , we have $P_{\alpha_i, y_i}(0) = 0$. Therefore, X is a root of $P_{\alpha_i, y_i}(X)$, that is, $P_{\alpha_i, y_i}(X) = X \cdot g(X)$ for some polynomial $g(X)$ of degree at most $k - 1$. We can rewrite

$$R_{\alpha_i, y_i}(X) = \sum_{i', j'} c_{i', j'}^{\alpha_i, y_i} X^{i'} (P_{\alpha_i, y_i}(X))^{j'} = \sum_{i', j'} c_{i', j'}^{\alpha_i, y_i} X^{i'} (X g(X))^{j'}.$$

Now for every i', j' such that $c_{i', j'}^{\alpha_i, y_i} \neq 0$, we have $i' + j' \geq r$ as $Q_{\alpha_i, y_i}(X, Y)$ has no monomial of degree $< r$. Thus X^r divides $R_{\alpha_i, y_i}(X)$, since $R_{\alpha_i, y_i}(x)$ has no non-zero monomial X^ℓ for any $\ell < r$. $\square$

12.3 Extensions

We now make some observations about Algorithm 12.2.3. In particular, the list decoding algorithm is general enough to solve more general problems than just list decoding. In this section, we present an overview of these extensions.

Recall that the constraint (12.10) states that $Q(X, Y)$ has $r \geq 0$ roots at (α_i, y_i) , $1 \leq i \leq n$. However, our analysis did not explicitly use the fact that the multiplicity is same for every i . In particular, given non-zero integer multiplicities $w_i \geq 0$, $1 \leq i \leq n$, Algorithm 12.2.3 can be generalized to output all polynomials $P(X)$ of degree at most $k-1$, such that

$$\sum_{i: P(\alpha_i) = y_i} w_i > \sqrt{(k-1)n \sum_{i=0}^n \binom{w_i+1}{2}}.$$

(We leave the proof as an exercise.) Note that till now we have seen the special case $w_i = r$, $1 \leq i \leq n$.

Further, we claim that the α_i 's need not be distinct for all of the previous arguments to go through. In particular, one can generalize Algorithm 12.2.3 even further to prove the following (the proof is left as an exercise):

Theorem 12.3.1. *Given integer weights $w_{i,\alpha}$ for every $1 \leq i \leq n$ and $\alpha \in \mathbb{F}$, in polynomial time one can output all $P(X)$ of degree at most $k-1$ such that*

$$\sum_i w_{i, P(\alpha_i)} > \sqrt{(k-1)n \sum_{i=0}^n \sum_{\alpha \in \mathbb{F}} \binom{w_{i,\alpha}+1}{2}}.$$

This will be useful to solve the following generalization of list decoding called soft decoding.

Definition 12.3.2. *Under soft decoding problem, the decoder is given as input a set of non-negative weights $w_{i,\alpha}$ ($1 \leq i \leq n, \alpha \in \mathbb{F}_q$) and a threshold $W \geq 0$. The soft decoder needs to output all codewords $(c_1, c_2, \dots, c_n)$ in q -ary code of block length n that satisfy:*

$$\sum_{i=1}^n w_{i, c_i} \geq W.$$

Note that Theorem 12.3.1 solve the soft decoding problem with

$$W = \sqrt{(k-1)n \sum_{i=0}^n \sum_{\alpha \in \mathbb{F}} \binom{w_{i,\alpha}+1}{2}}.$$

Consider the following special case of soft decoding where $w_{i, y_i} = 1$ and $w_{i, \alpha} = 0$ for $\alpha \in \mathbb{F} \setminus \{y_i\}$ ($1 \leq i \leq n$). Note that this is exactly the list decoding problem with the received word $(y_1, \dots, y_n)$. Thus, list decoding is indeed a special case of soft decoding. Soft decoding has practical applications in settings where the channel is analog. In such a situation, the “quantizer” might not be able to pinpoint a received symbol y_i with 100% accuracy. Instead, it can use the weight $w_{i, \alpha}$ to denote its confidence level that i th received symbol was α .

Finally, we consider a special case of soft decoding called list recovery, which has applications in designing list decoding algorithms for concatenated codes.

Definition 12.3.3 (List Recovery). *Given $S_i \subseteq \mathbb{F}_q$, $1 \leq i \leq n$ where $|S_i| \leq \ell$, output all $[n, k]_q$ codewords $(c_1, \dots, c_n)$ such that $c_i \in S_i$ for at least t values of i . If for every valid input the number of such codewords is at most L , then the corresponding code is called $(1-t/n, \ell, L)$ -list recoverable.*

We leave the proof that list recovery is a special case of soft decoding as an exercise. Finally, we claim that Theorem 12.3.1 implies the following result for list recovery (the proof is left as an exercise):

Theorem 12.3.4. *Given $t > \sqrt{(k-1)\ell n}$, the list recovery problem with agreement parameter t for $[n, k]_q$ Reed-Solomon codes can be solved in polynomial time.*

12.4 Exercises

Exercise 12.1. *Peterson syndrome based unique decoding of RS codes.*

Exercise 12.2. *For a finite field $\mathbb{F}_{q^m}$, define the Trace map Tr as follows: for $x \in \mathbb{F}_{q^m}$*

$$\text{Tr}(x) = x + x^q + x^{q^2} + \dots + x^{q^{m-1}}.$$

1. *Show that $\text{Tr}(\alpha) \in \mathbb{F}_q$ for every $\alpha \in \mathbb{F}_{q^m}$.*
2. *Prove that $\text{Tr}: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ is a surjective $\mathbb{F}_q$ -linear map.*
3. *Let $C \subseteq \mathbb{F}_{q^m}^n$ be a linear code, and $C^\perp \subseteq \mathbb{F}_{q^m}^n$ its dual. Define $C|_{\mathbb{F}_q} = C \cap \mathbb{F}_q^n$ to be the subfield subcode of C .*

Prove that

$$(C|_{\mathbb{F}_q})^\perp = \text{Tr}(C^\perp)$$

where $\text{Tr}(C^\perp) = \{\text{Tr}(c) \mid c \in C^\perp\}$.

Suggestion: *Prove both the set inclusions, starting with the easier inclusion $\text{Tr}(C^\perp) \subseteq (C|_{\mathbb{F}_q})^\perp$.*

4. *Show that*

$$\dim(C) \leq \dim(\text{Tr}(C)) \leq m \cdot \dim(C),$$

and

$$\dim(C) - (m-1)(n - \dim(C)) \leq \dim(C|_{\mathbb{F}_q}) \leq \dim(C),$$

where for a linear space $X \subseteq \mathbb{F}^n$, $\dim(X)$ stands for its dimension as a $\mathbb{F}$ -vector space.

Exercise 12.3. In this problem, you will prove that a certain “ultimate” form of Reed-Solomon decoding is NP-hard.

You may assume that the following problem is NP-hard.

Instance: A set $S = \{\alpha_1, \dots, \alpha_n\} \subseteq \mathbb{F}_{2^m}$, an element $\beta \in \mathbb{F}_{2^m}$, and an integer $1 \leq k < n$.

Question: Is there a nonempty subset $T \subseteq \{1, 2, \dots, n\}$ with $|T| = k + 1$ such that $\sum_{i \in T} \alpha_i = \beta$?

Consider the $[n, k]$ Reed-Solomon code C_{RS} over $\mathbb{F}_{2^m}$ obtained by evaluating polynomials of degree at most $k - 1$ at points in S . Define $y \in (\mathbb{F}_{2^m})^n$ as follows: $y_i = \alpha_i^{k+1} - \beta \alpha_i^k$ for $i = 1, 2, \dots, n$.

Prove that there is a codeword of C_{RS} at Hamming distance at most $n - k - 1$ from y if and only if there is a set T as above of size $k + 1$ satisfying $\sum_{i \in T} \alpha_i = \beta$.

Conclude that finding the nearest codeword in a Reed-Solomon code over exponentially large fields is NP-hard.

Exercise 12.4. Adapt the algorithm that algorithm Reed-Solomon codes up to the Johnson bound to show that Reed-Solomon codes encoding polynomials in $\mathbb{F}_q[X]_{\leq k}$ by their evaluations at n distinct elements $a_1, a_2, \dots, a_n \in \mathbb{F}_q$ can be “list recovered” in the following sense:

Let $\ell < \frac{n}{k}$.

Given sets $S_i \subseteq \mathbb{F}_q$, $|S_i| \leq \ell$, for each $i \in [n]$, there are at most $O(n^2)$ polynomials $f \in \mathbb{F}_q[X]_{\leq k}$ such that $f(a_i) \in S_i$ for every $i \in [n]$, and the list of such polynomials can be found in polynomial time.

Exercise 12.5. In this exercise, you will show that the list recovery guarantee achieved by the above algorithm is tight, in the sense that when $\ell = \lceil \frac{n}{k} \rceil$, there are settings where there are super-polynomially many (i.e., $n^{\omega(1)}$) polynomials.

Let r be a fixed prime power. Let $n = q = r^m$ and $k = \frac{r^m - 1}{r - 1}$. Prove that there are at least r^{2^m} polynomials $f \in \mathbb{F}_q[X]_{\leq k}$ such that $f(a) \in \mathbb{F}_r$ for every $a \in \mathbb{F}_q$. Deduce that the Reed-Solomon list recovery algorithm cannot be improved to work for $\ell = \lceil \frac{n}{k} \rceil$ in general.

Hint: For $x \in \mathbb{F}_{r^m}$, $x^{\frac{r^m - 1}{r - 1}}$ always belongs to the subfield $\mathbb{F}_r$ (why?). So the polynomials $f_\beta(X) := (X + \beta)^{\frac{r^m - 1}{r - 1}}$ for $\beta \in \mathbb{F}_{r^m}$ take values in $\mathbb{F}_r$ on evaluation points in $\mathbb{F}_{r^m}$. Find 2^m of these polynomials that are linearly independent over $\mathbb{F}_r$.

Exercise 12.6. In this exercise, we will explore some “bad list-decoding configurations” for Reed-Solomon codes, namely a center that has many RS codewords in a Hamming ball of bounded radius (in fact, close to the Johnson radius).

For this, we first need to develop some machinery related to linearized and subspace polynomials.

Let $K := \mathbb{F}_{q^m}$ be an extension field of $\mathbb{F}_q$. A polynomial $L \in K[X]$ with coefficients in K is called a linearized polynomial if all its nonzero terms have exponents which are a power of q , i.e., it is of the form $L(X) = \sum_{i=0}^{m-1} a_i X^{q^i}$ with $a_i \in K$.

1. Let L be a linearized polynomial as above. Prove that the function mapping $\alpha \in K$ to $L(\alpha) \in K$ is $\mathbb{F}_q$ -linear, i.e., $L(a\alpha + b\beta) = aL(\alpha) + bL(\beta)$ for all $\alpha, \beta \in K$ and $a, b \in \mathbb{F}_q$.

2. Prove that there is one-to-one correspondence between $\mathbb{F}_q$ -linear functions mapping K to K and linearized polynomials in $K[X]$.

3. Let W be a $\mathbb{F}_q$ -linear subspace of $\mathbb{F}_{q^m}$. This means that whenever $\alpha, \beta \in W$, we have $a\alpha + b\beta \in W$ for all $a, b \in \mathbb{F}_q$. Define the “subspace polynomial” $S_W(X) \in \mathbb{F}_{q^m}(X)$ as $S_W(X) := \prod_{\alpha \in W} (X - \alpha)$.

Exercise 12.7. A *Kekeya set* in $\mathbb{F}_q^n$ is a subset $K \subseteq \mathbb{F}_q^n$ such that K contains a line in every direction. Formally, K is a Kekeya set if for every $y \in \mathbb{F}_q^n$ there exists a point $x \in \mathbb{F}_q^n$ such that the line $\{x + a \cdot y \mid a \in \mathbb{F}_q\} \subseteq K$. Use the polynomial method to prove that size of any Kekeya set K must satisfy $|K| \geq \binom{q+n-2}{n-1}$.

Hint: Argue that if K were smaller than $\binom{q+n-2}{n-1}$, then one can interpolate a homogenous nonzero polynomial f of total degree $(q-1)$ that vanishes on K . Then use the fact that a nonzero $f \in \mathbb{F}_q[X_1, \dots, X_n]$ of total degree d is 0 on at most a fraction d/q of the space $\mathbb{F}_q^n$.

Exercise 12.8. Cheng-Wan reduction of discrete log to RS list decoding?

Exercise 12.9. One can define a number-theoretic counterpart of Reed-Solomon codes, called Chinese Remainder codes, as follows. Let $1 \leq k < n$ be integers and let $p_1 < p_2 < \dots < p_n$ be n distinct primes. Denote $K = \prod_{i=1}^k p_i$ and $N = \prod_{i=1}^n p_i$. The notation $\mathbb{Z}_M$ stands for integers modulo M , i.e., the set $\{0, 1, \dots, M-1\}$. Consider the Chinese Remainder code defined by the encoding map $E: \mathbb{Z}_K \rightarrow \mathbb{Z}_{p_1} \times \mathbb{Z}_{p_2} \times \dots \times \mathbb{Z}_{p_n}$ defined by:

$$E(m) = (m \bmod p_1, m \bmod p_2, \dots, m \bmod p_n).$$

(Note that this is not a code in the usual sense we have been studying since the symbols at different positions belong to different alphabets. Still notions such as distance of this code make sense and are studied in the questions below.)

1. Suppose that $m_1 \neq m_2$. For $1 \leq i \leq n$, define the indicator variable $b_i = 1$ if $E(m_1)_i \neq E(m_2)_i$ and $b_i = 0$ otherwise. Prove that $\prod_{i=1}^n p_i^{b_i} > N/K$.

Use the above to deduce that when $m_1 \neq m_2$, the encodings $E(m_1)$ and $E(m_2)$ differ in at least $n - k + 1$ locations.

2. This exercise examines how the idea behind the Welch-Berlekamp decoder can be used to decode these codes.

Suppose $\mathbf{r} = (r_1, r_2, \dots, r_n)$ is the received word where $r_i \in \mathbb{Z}_{p_i}$. By Part (a), we know there can be at most one $m \in \mathbb{Z}_K$ such that

$$\prod_{i: E(m)_i \neq r_i} p_i^{b_i} \leq \sqrt{N/K}. \quad (12.18)$$

(Be sure you see why this is the case.) The exercises below develop a method to find the unique such m , assuming one exists.

In what follows, let r be the unique integer in $\mathbb{Z}_N$ such that $r \bmod p_i = r_i$ for every $i = 1, 2, \dots, n$ (note that the Chinese Remainder theorem guarantees that there is a unique such r).

- (a) Assuming an m satisfying (12.18) exists, prove that there exist integers y, z with $0 \leq y < \sqrt{N/K}$ and $1 \leq z \leq \sqrt{N/K}$ such that $y \equiv rz \pmod{N}$.
- (b) Prove also that if y, z are any integers satisfying the above conditions, then in fact $m = y/z$.

(Remark: A pair of integers (y, z) satisfying above can be found by solving the integer linear program with integer variables y, z, t and linear constraints: $0 < z \leq \sqrt{N/K}$; and $0 \leq z \cdot r - t \cdot N < \sqrt{N/K}$. This is an integer program in a fixed number of dimensions and can be solved in polynomial time. Faster, easier methods are also known for this special problem.)

3. Instead of condition (12.18) what if we want to decode under the more natural condition for Hamming metric, that is $|\{i : E(m)_i \neq r_i\}| \leq \frac{n-k}{2}$? Using ideas similar to GMD decoding, show how this can be done by calling the above decoder many times, by erasing the last i symbols for each choice of $1 \leq i \leq n$.

Exercise 12.10. In this problem, we develop a more abstract view of the Reed-Solomon decoding algorithm. This enables extending the approach to other Reed-Solomon-like codes, such as algebraic-geometric codes (and also encompasses the algorithm for the Chinese Remainder codes described above).

First we give some definitions. Let $\mathbb{F}$ be a field. For $u, v \in \mathbb{F}^n$, define $u * v = (u_1 v_1, u_2 v_2, \dots, u_n v_n) \in \mathbb{F}^n$ be the component-wise product. For $U, V \subseteq \mathbb{F}^n$, define $U * V = \{u * v \mid u \in U, v \in V\}$.

The idea of the abstract decoding procedure is that given a code C capable of correcting e errors (i.e., its distance exceeds $2e$) that we want to decode, we construct an error-locator code E , such that $E * C$ is contained in another linear code N that has large distance. Specifically, we want codes E and N to have the following properties:

- $\dim(E) > e$.
- $E * C \subseteq N$.
- $\text{dist}(N) > e$.
- $\text{dist}(C) > n - \text{dist}(E)$

Consider the following decoding algorithm for C . Given as input $r \in \mathbb{F}^n$ with Hamming distance at most e from some codeword $c \in C$, the goal of the algorithm is to find c .

Step 1: Find $a \in E$ and $b \in N$, $a \neq 0$, such that $a * r = b$.

Step 2: For each i , if $a_i = 0$, set $s_i = ?$, and otherwise set $s_i = r_i$. Perform erasure decoding (for the code C) on the resulting vector s , to find a $c \in C$ such that $c_i = s_i$ whenever $s_i \neq ?$.

Output c .

The exercises below justify the algorithm, proving its efficiency and correctness. Again, we assume that the input $r \in \mathbb{F}^n$ satisfies the property that there is a $c \in C$ with $\Delta(r, c) \leq e$ (such a c is then unique, due to the assumed e -error correction property of C).

1. Prove that a, b as in Step 1 exist.
2. Prove that the algorithm can be implemented in polynomial time, given generator matrices of C, N, E .
3. Prove that for every (a, b) satisfying the condition of Step 1, $a * c = b$.
4. Prove that if $a * c' = b$ for some $c' \in C$, then $c' = c$.
5. Conclude the correctness of the algorithm.
6. If C is an $[n, n - 2e]$ Reed-Solomon code, what are E and N in the above abstraction that correspond to the Welch-Berlekamp algorithm covered in lecture?

Exercise 12.11. 1. Recall the definition of “tensor product” of codes. If C_1 is an $[n_1, k_1, d_1]_2$ binary linear code, and C_2 an $[n_2, k_2, d_2]$ binary linear code, then $C = C_1 \otimes C_2 \subseteq \mathbb{F}_2^{n_1 \times n_2}$ is defined to subspace of $n_2 \times n_1$ matrices whose rows belong to C_1 and whose columns belong to C_2 .

Suppose C_2 has an efficient algorithm to correct $< d_2/2$ errors and C_1 has an efficient errors-and-erasures decoding algorithm to correct any combination of e errors and s erasures provided $2e + s < d_1$. Show how one can efficiently decode C up to $< d_1 d_2/2$ errors using these algorithms as subroutines.

2. Consider the bivariate version of the Reed-Solomon code, which encodes a polynomial $f \in \mathbb{F}_q[X, Y]$ with degree less than k in both X and Y by its evaluations at all q^2 points $(\alpha, \beta) \in \mathbb{F}_q \times \mathbb{F}_q$.

- (a) What are the block length, dimension, and minimum distance of this code?
- (b) Describe how one can efficiently decode this code up to (almost) half its minimum distance.

((The natural) hint: Relate to Part (a) of this question.)

Exercise 12.12. Basic list decoding of concatenated codes based on list recovery of outer RS code.

Exercise 12.13. List decoding of RS concatenated with Hadamard based on soft decoding of outer RS codes with Fourier weights.

Exercise 12.14. In the game of 20 questions, an oracle has an arbitrary secret $s \in \{0, 1\}^k$ and the aim is to determine the secret by asking the oracle as few yes/no questions about s as possible. It is easy to see that k questions are necessary and sufficient. Here we consider a variant where the oracle has two secrets $s_1, s_2 \in \{0, 1\}^k$ and can adversarially decide to answer each question

according to either s_1 or s_2 . That is, for a question $f : \{0, 1\}^k \rightarrow \{0, 1\}$, the oracle may answer with either $f(s_1)$ or $f(s_2)$. Here it turns out to be impossible to pin down either of the secrets with certainty, no matter how many questions we ask, but we can hope to compute a small set S of secrets, of a fixed size independent of k , such that $S \cap \{s_1, s_2\} \neq \emptyset$. (In fact, $|S|$ can be made as small as 2.) This variant of twenty questions apparently arose from Internet routing algorithms used by Akamai.

- (i) Let C be a binary code of block length n and 2^k codewords such that (a) every two codewords of C agree in at least a fraction $(1/2 - \epsilon)$ of positions and (b) C can be efficiently list decoded from a fraction $(1/4 + \epsilon)$ of errors with list size ℓ independent of k . Show how to solve the above problem in polynomial time by asking questions based on the code C .
- (ii) Briefly describe how to construct a code with the properties spelled out in (i) above, and deduce that $n \leq \text{poly}(k)$ questions suffice for the above variant of 20 questions. (In fact, $n = O(k)$ questions suffice, and feel free to show this stronger bound.)

12.5 Bibliographic Notes

In 1960, before polynomial time complexity was regarded as an acceptable notion of efficiency, Peterson designed an $O(N^3)$ time algorithm for the unique decoding of Reed-Solomon codes [125]. This algorithm was the first efficient algorithm for unique decoding of Reed-Solomon codes. The Berlekamp-Massey algorithm, which used shift registers for multiplication, was even more efficient, achieving a computational complexity of $O(N^2)$. Currently, an even more efficient algorithm, with a computational complexity of $O(N \text{poly}(\log N))$, is known [129].

The Welch-Berlekamp algorithm, covered under US Patent [162], has a running time complexity of $O(N^3)$. We will follow a description of the Welch-Berlekamp algorithm provided by Gemmell and Sudan in [59].

Håstad, Philips and Safra showed that solving a system of quadratic equations (even those without any square terms like we have in (12.1)) over any field $\mathbb{F}_q$ is NP-hard [86]. (In fact, it is even hard to approximately solve this problem: i.e. where one tries to compute an assignment that satisfies as many equations as possible.) Linearization is a trick that has been used many times in theoretical computer science and cryptography. See [this blog post by Dick Lipton](#) for more on this.

Algorithm 12.2.2 is due to Sudan [149] and Algorithm 12.2.3 is due to Guruswami and Sudan [78]. Near-linear time implementations of these list decoding algorithms are also known [2].

It is natural to ask whether Theorem 12.3.4 is tight for list recovery, i.e. generalize Open Question 12.2.1 to list recovery. It was shown by Guruswami and Rudra that Theorem 12.3.4 is indeed the best possible list recovery result for Reed-Solomon codes [73]. Thus, any algorithm that answers Open Question 12.2.1 in some sense has to exploit the fact that in the list decoding problem, the α_i 's are distinct. Recently it was shown by Rudra and Wootters that at least combinatorially, Reed-Solomon codes (with random evaluation points) are list decodable beyond the Johnson bound [135]. On the flip side, there are limits known on list decoding Reed-Solomon codes (both unconditional ones due to Ben-Sasson et al. [17] as well as those based on hardness

of computing discrete log in the worst-case due to Cheng and Wan [30]) but none of them are close to the Johnson bound (especially for constant rate Reed-Solomon codes).

Chapter 13

Decoding Reed-Muller Codes

In this chapter we describe decoding algorithms for the Reed-Muller codes, introduced in Chapter 9. Recall that these are the codes obtained by evaluations of multivariate polynomials over all possible assignments to the variables. We will see several decoding algorithms for these codes, ranging from simplistic ones that correct a constant fraction of the minimum distance (with the constant depending on q), to algorithms based on more sophisticated concepts that correct up to half the minimum distance.

To elaborate on the above, recall that the Reed-Muller code with parameters q, m, r is the set of functions

$$\text{RM}(q, m, r) \stackrel{\text{def}}{=} \left\{ f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q \mid \deg(f) \leq r \right\}.$$

The minimum distance of the code is

$$\Delta_{q,m,r} \stackrel{\text{def}}{=} (q - t) \cdot q^{m-s-1},$$

where s, t satisfy $r = s(q - 1) + t$ and $0 \leq t \leq q - 2$ (recall Lemma 9.4.1). We will first describe an algorithm to correct $\varepsilon \cdot \Delta_{q,m,r}$ for some constant $\varepsilon > 0$ that depends only on q . Later we will give algorithms that correct $\left\lfloor \frac{\Delta_{q,m,r}-1}{2} \right\rfloor$ errors.

13.1 A natural decoding algorithm

The main insight behind our first decoding algorithm is the simple fact that the degree of polynomials does not increase on *affine substitutions*. Let us introduce this notion and then explain why this might be useful in building decoding algorithms.

Definition 13.1.1. A one-dimensional s -variate affine form $a \in \mathbb{F}_q[Z_1, \dots, Z_s]$ is a polynomial of the form $a(Z_1, \dots, Z_s) = \sum_{i=1}^s \alpha_i Z_i + \alpha_0$. In other words an affine form is a polynomial of degree at most 1. An m -dimensional s -variate affine form $A = \langle a_1, \dots, a_m \rangle$ is simply an m -tuple of one-dimensional affine forms.

For example $A_0 = \langle Z_1 + Z_2, Z_1, Z_2 \rangle$ is a 3-dimensional 2-variate affine form over $\mathbb{F}_2$.

Definition 13.1.2. Given an m -variate polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$ and an m -dimensional s -variate affine form $A = \langle a_1, \dots, a_m \rangle \in (\mathbb{F}_q[Z])^m$ where $Z = (Z_1, \dots, Z_s)$, the affine substitution of A into P is given by the polynomial $P \circ A \in \mathbb{F}_q[Z]$ given by $(P \circ A)(Z) = P(a_1(Z), \dots, a_m(Z))$.

Let A_0 be the affine form as above and let $P_0(X_1, X_2, X_3) = X_1 X_2 + X_1 X_2 X_3 + X_3$ over $\mathbb{F}_2$. Then we have

$$(P_0 \circ A_0)(Z_1, Z_2) = (Z_1 + Z_2)Z_1 + (Z_1 + Z_2)Z_1 Z_2 + Z_2 = Z_1^2 + Z_1 Z_2 + Z_1^2 Z_2 + Z_1 Z_2^2 + Z_2 = Z_1 + Z_1 Z_2 + Z_2,$$

where the last equality follows since we are working over $\mathbb{F}_2$.

Notice that the notion of affine substitutions extends to functions naturally, viewing both f and A as functions (given by the evaluations of corresponding polynomials) in the definition above.

Affine substitutions have nice algebraic, geometric, and probabilistic properties and these combine to give us the decoding algorithm of this section. We introduce these properties in order.

Proposition 13.1.3 (Degree of Affine Substitutions). *Affine substitutions do not increase the degree of a polynomial. Specifically, if A is an affine form, then for every polynomial P , we have $\deg(P \circ A) \leq \deg(P)$.*

Proof. The proof is straightforward. First note that for any single monomial $M = \prod_{i=1}^m X_i^{r_i}$ the affine substitution $M \circ A = \prod_{i=1}^m a_i(Z)^{r_i}$ has degree at most $\deg(M)$. Next note that if we write a general polynomial as a sum of monomials, say $P = \sum_M c_M \cdot M$, then the affine substitution is additive and so $P \circ A = \sum_M c_M (M \circ A)$. The proposition now follows from the fact that

$$\deg(P \circ A) = \deg\left(\sum_M c_M (M \circ A)\right) \leq \max_M \{\deg(M \circ A)\} \leq \max_M \deg(M) = \deg(P).$$

□

We remark that the bound above can be tight (see Exercise 13.1) and that the result above generalizes to the case when we replace each term by a degree d -polynomial instead of a degree 1-polynomial (see Exercise 13.2).

Next we turn to the geometric aspects of affine substitutions. These aspects will be essential for some intuition, though we will rarely invoke them formally.

One way to view affine substitutions into functions is that we are viewing the restriction of a function on a small subset of the domain. For example, when $s = 1$, then an affine substitution A into a function f , restricts the domain of the function to the set $\{A(z) | z \in \mathbb{F}_q\}$ where $A(z)$ is of the form $\mathbf{a}z + \mathbf{b}$ for some $\mathbf{a}, \mathbf{b} \in \mathbb{F}_q^m$. This set forms a *line* in $\mathbb{F}_q^m$ with slope $\mathbf{a}$ and passing through the point $\mathbf{b}$. When s becomes larger, we look at higher dimensional (affine) subspaces such as planes ($s = 2$) and cubes ($s = 3$). While lines, planes and cubes are not exactly the same as in the Euclidean space they satisfy many similar properties and this will be used to drive some of the probabilistic thinking below.

In what follows we will be looking restrictions of two functions f and g on small-dimensional affine subspaces. On these subspaces we would like to argue that f and g disagree roughly as

often as they do on $\mathbb{F}_q^m$. To do so, we use the fact that “random” affine substitutions sample uniformly from $\mathbb{F}_q^m$. We formalize this below.

Consider a uniform choice of an affine form $A(\mathbf{z}) = \mathbf{M}\mathbf{z} + \mathbf{b}$, i.e., where $\mathbf{M} \in \mathbb{F}_q^{m \times s}$ and $\mathbf{b} \in \mathbb{F}_q^m$ are chosen uniformly and independently from their respective domains. (Note that this allows M to be of less than full rank with positive probability, and we will allow this to keep calculations simple and clean. We do warn the reader that this can lead to degenerate lines and subspaces - e.g., when M is the zero matrix then these subspaces contain only one point.)

Proposition 13.1.4. (1) Fix $\mathbf{z} \in \mathbb{F}_q^s$. Then, for a uniformly random A , the point $A(\mathbf{z})$ is distributed uniformly in $\mathbb{F}_q^m$.

(2) Fix $\mathbf{z} \in \mathbb{F}_q^s \setminus \{0\}$ and $\mathbf{x} \in \mathbb{F}_q^m$, and let A be chosen uniformly subject to the condition $A(\mathbf{0}) = \mathbf{x}$. Then the point $A(\mathbf{z})$ is distributed uniformly in $\mathbb{F}_q^m$. Consequently, for every pair of functions $f, g : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$, we have

$$\Pr_A [f \circ A(\mathbf{z}) \neq g \circ A(\mathbf{z})] = \delta(f, g).$$

Proof. Let $A(\mathbf{z}) = \mathbf{M}\mathbf{z} + \mathbf{b}$ where $\mathbf{M} \in \mathbb{F}_q^{m \times s}$ and $\mathbf{b} \in \mathbb{F}_q^m$ are chosen uniformly and independently.

For part (1), we use the fact that for every fixed $\mathbf{M}$ and $\mathbf{z}$, $\mathbf{M}\mathbf{z} + \mathbf{b}$ is uniform over $\mathbb{F}_q^m$ when $\mathbf{b}$ is uniform. In particular, for every $\mathbf{y} \in \mathbb{F}_q^m$ we have

$$\Pr_{\mathbf{b}} [\mathbf{M}\mathbf{z} + \mathbf{b} = \mathbf{y}] = \Pr_{\mathbf{b}} [\mathbf{b} = \mathbf{y} - \mathbf{M}\mathbf{z}] = q^{-m}.$$

Since this holds for every $\mathbf{M}$, it follows that

$$\Pr_{\mathbf{M}, \mathbf{b}} [\mathbf{M}\mathbf{z} + \mathbf{b} = \mathbf{y}] = q^{-m}$$

and so we conclude that $A(\mathbf{z}) = \mathbf{M}\mathbf{z} + \mathbf{y}$ is uniformly distributed over $\mathbb{F}_q^m$.

For part (2), note that the condition $A(\mathbf{0}) = \mathbf{x}$ implies $\mathbf{b} = \mathbf{x}$. So, for fixed $\mathbf{y} \in \mathbb{F}_q^m$, we have

$$\Pr_{\mathbf{M}, \mathbf{b}} [A(\mathbf{z}) = \mathbf{y} \mid A(\mathbf{0}) = \mathbf{x}] = \Pr_{\mathbf{M}} [\mathbf{M}\mathbf{z} + \mathbf{x} = \mathbf{y}].$$

Now let $\mathbf{z} = (z_1, \dots, z_s)$ and denote the columns of $\mathbf{M}$ by $M_1, \dots, M_s$ so that

$$\mathbf{M}\mathbf{z} = z_1 M_1 + \dots + z_s M_s.$$

Since $\mathbf{z} \neq \mathbf{0}$ we must have some $z_i \neq 0$ and let i be the largest such index. We note that for every choice of $M_1, \dots, M_{i-1}, M_{i+1}, \dots, M_s$, the probability, over the choice of M_i that $\mathbf{M}\mathbf{z} + \mathbf{x} = \mathbf{y}$ is q^{-m} , since this happens if and only if $M_i = z_i^{-1}(\mathbf{y} - (z_1 M_1 + \dots + z_{i-1} M_{i-1} + \mathbf{x}))$, and this event happens with probability q^{-m} . Averaging over the choices of the remaining columns of $\mathbf{M}$, we still have

$$\Pr_{\mathbf{M}, \mathbf{b}} [A(\mathbf{z}) = \mathbf{y} \mid A(\mathbf{0}) = \mathbf{x}] = \Pr_{\mathbf{M}} [\mathbf{M}\mathbf{z} + \mathbf{x} = \mathbf{y}] = q^{-m},$$

thus establishing that $A(\mathbf{z})$ is distributed uniformly over $\mathbb{F}_q^m$ even when conditioned on $A(\mathbf{0}) = \mathbf{x}$.

Finally to see the final implication of part (2), fix functions $f, g : \mathbb{F}_q^m$ and let

$$E = \left\{ \mathbf{y} \in \mathbb{F}_q^m \mid f(\mathbf{y}) \neq g(\mathbf{y}) \right\},$$

so that $\delta(f, g) = \Pr_{\mathbf{y}}[\mathbf{y} \in E]$. We have

$$\Pr_A[f \circ A(\mathbf{z}) \neq g \circ A(\mathbf{z})] = \Pr_A[A(\mathbf{z}) \in E],$$

but since $A(\mathbf{z})$ is uniform in $\mathbb{F}_q^m$ even given $A(\mathbf{0}) = \mathbf{x}$, we have

$$\Pr_A[A(\mathbf{z}) \in E] = \Pr_{\mathbf{y}}[\mathbf{y} \in E] = \delta(f, g),$$

as claimed. $\square$

13.1.1 The Actual Algorithm

Now we explain why affine substitutions might help in decoding the Reed-Muller code. Recall that the decoding problem for the Reed-Muller codes is the following:

- **Input:** Parameters q, m, r and e (bound on number of errors) and a function $f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$.
- **Output:** Polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$ with $\deg(P) \leq r$ such that $|\{\mathbf{x} \in \mathbb{F}_q^m \mid f(\mathbf{x}) \neq P(\mathbf{x})\}| \leq e$.

One way to recover the desired polynomial P is to output its value at every given point $\mathbf{x} \in \mathbb{F}_q^m$. (This works provided the polynomial P to be output is uniquely determined by f and the number of errors, and that is the setting we will be working with.) In what follows we will do exactly this. The main idea behind the algorithm of this section is the following: We will pick an affine form A such that $A(\mathbf{0}) = \mathbf{x}$ and attempt to recover the polynomial $P \circ A$. Evaluating $P \circ A(\mathbf{0})$ gives us $P(\mathbf{x})$ and so this suffices, but why is the task of computing $P \circ A$ any easier? Suppose we use an s -variate form A for small s . Then the function $P \circ A$ is given by q^s values with $s < m$ this can be a much smaller sized function and so brute force methods would work faster. But an even more useful observation is that if A is chosen at random such that $A(\mathbf{0}) = \mathbf{x}$, then most of the q^s points (in fact all but one) are random points and so unlikely to be erroneous. In particular for any fixed non-zero $\mathbf{z}$, we would have with high probability $f \circ A(\mathbf{z}) = P \circ A(\mathbf{z})$, where the probability is over the choice of A , assuming the number of errors is small. Since $q^s < q^m$ one can apply a union bound over the roughly q^s choices of $\mathbf{z}$ to (hopefully) establish that all the points $\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}$ are not errors, and if this happens a further hope would be that $P \circ A$ is uniquely determined by its values on $\mathbb{F}_q^s \setminus \{\mathbf{0}\}$. The two hopes are in tension with each other — the former needs small values of s and the latter needs s to be large; and so we pick an intermediate s , specifically $s = \left\lceil \frac{r+1}{q-1} \right\rceil$, where both conditions are realized and this yields the algorithm below. We describe the algorithm first and then explain this choice of parameters later.

Algorithm 13.1.1 SIMPLE REED-MULLER DECODER

INPUT: $r < m$, $0 < e \leq \frac{1}{3} \cdot q^{m - \lceil (r+1)/(q-1) \rceil}$, and function $f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$.

OUTPUT: Polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$ with $\deg(P) \leq r$ such that $|\{\mathbf{x} \in \mathbb{F}_q^m \mid f(\mathbf{x}) \neq P(\mathbf{x})\}| \leq e$, if such a polynomial exists and NULL otherwise

```

FOR  $\mathbf{x} \in \mathbb{F}_q^m$  DO
   $g(\mathbf{x}) \leftarrow \text{LOCAL-DECODE-RM-SIMPLE}(\mathbf{x}, f)$ .
RETURN INTERPOLATE( $q, m, g, r, \mathbb{F}_q^m$ )

```

procedure LOCAL-DECODE-RM-SIMPLE($\mathbf{x}, f$)

Repeat LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) $O(m \log q)$ times and return most frequent answer.

procedure LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$)

```

Let  $s \leftarrow \left\lceil \frac{r+1}{q-1} \right\rceil$ 
Select an  $m$ -dimensional  $s$ -variate affine form  $A$  uniformly conditioned on  $A(\mathbf{0}) = \mathbf{x}$ .
 $g \leftarrow \text{INTERPOLATE}\left(q, s, f \circ A, r, \mathbb{F}_q^s \setminus \{\mathbf{0}\}\right)$ 
IF  $g$  is NULL THEN
   $g \leftarrow 0$ 
RETURN  $g(\mathbf{0})$ .

```

procedure INTERPOLATE(q, m, f, r, S) $\triangleright$ Returns a polynomial $P \in \mathbb{F}_q(Z_1, \dots, Z_s)$ such that $\deg(P) \leq r$ and $P(\mathbf{x}) = f(\mathbf{x})$ for every $\mathbf{x} \in S$ and return NULL is no such P exists. $\triangleright$ See comments in Section 13.1.2 for more on how this algorithm can be implemented.

The detailed algorithm is given as Algorithm 13.1.1. Roughly the algorithm contains two loops. The outer loop enumerates $\mathbf{x} \in \mathbb{F}_q^n$ and invokes a subroutine LOCAL-DECODE-RM-SIMPLE that determines $P(\mathbf{x})$ correctly with very high probability. This subroutine creates an inner loop which invokes a less accurate subroutine LOCAL-DECODE-RM-SIMPLE-ITER, which computes $P(\mathbf{x})$ correctly with probability $\frac{2}{3}$, many times and reports the most commonly occurring answer. The crux of the algorithm is thus LOCAL-DECODE-RM-SIMPLE-ITER. This algorithm picks a random affine form A such that $A(\mathbf{0}) = \mathbf{x}$ and assumes that $f \circ A(\mathbf{z}) = P \circ A(\mathbf{z})$ for every non-zero $\mathbf{z}$. Based on this assumption it interpolates the polynomial $P \circ A$ and returns $P \circ A(\mathbf{0}) = P(A(\mathbf{0})) = P(\mathbf{x})$.

The crux of the analysis is to show that the assumption holds with probability at least $\frac{2}{3}$ over the random choice of A provided the number of errors is small. We will undertake this analysis next.

13.1.2 Analysis of the simple decoding algorithm

We first show that each invocation of LOCAL-DECODE-RM-SIMPLE-ITER succeeds with high probability:

Lemma 13.1.5. *Let $P \in \mathbb{F}_q[X_1, \dots, X_m]$ be a polynomial of degree at most r and let $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ be such that*

$$e = |\{\mathbf{x} \in \mathbb{F}_q^m \mid f(\mathbf{x}) \neq P(\mathbf{x})\}| \leq \frac{1}{3} \cdot q^{m - \lceil (r+1)/(q-1) \rceil}.$$

Then, for every $\mathbf{x} \in \mathbb{F}_q^m$, the probability that LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) returns $P(\mathbf{x})$ is at least $2/3$.

Proof. Recall $s = \left\lceil \frac{r+1}{q-1} \right\rceil$ and so $e \leq \frac{q^{m-s}}{3}$. We use this condition in the analysis below.

Fix $\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}$. Since A was picked conditioned on $A(\mathbf{0}) = \mathbf{x}$, by part (2) of Proposition 13.1.4 we have that $A(\mathbf{z})$ is a uniformly random element of $\mathbb{F}_q^m$ (and in particular this is independent of $\mathbf{x}$). So the probability that $f(A(\mathbf{z})) \neq P(A(\mathbf{z}))$ is exactly $\frac{e}{q^m}$. Taking the union bound over all $\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}$ we get that

$$\Pr_A \left[\exists \mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\} \text{ s.t. } f(A(\mathbf{z})) \neq P(A(\mathbf{z})) \right] \leq (q^s - 1) \cdot \frac{e}{q^m} \leq \frac{e}{q^{m-s}} \leq \frac{1}{3}. \quad (13.1)$$

So, with probability at least $\frac{2}{3}$, we have that $f \circ A(\mathbf{z}) = P \circ A(\mathbf{z})$ for every $\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}$. We argue below that if this holds, then LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) returns $P(\mathbf{x})$ and this proves the lemma.

Since $P \circ A$ is a polynomial in $\mathbb{F}_q[Z_1, \dots, Z_s]$ of degree at most r that agrees with $f \circ A$ on every $\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}$, we have that there exists at least one polynomial satisfying the condition of the final interpolation step in LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$). It suffices to show that this polynomial is unique, but this follows from Exercise 13.4, which asserts that $\delta(P \circ A, h) \geq \frac{2}{q^s}$ for every polynomial $h \in \mathbb{F}_q[Z_1, \dots, Z_s]$ of degree at most r , provided $r < (q-1)s$. (Note that our choice of $s = \left\lceil \frac{r+1}{q-1} \right\rceil$ ensures this.) In particular this implies that every pair of polynomials disagree on at least two points in $\mathbb{F}_q^s$ and so on at least one point in $\mathbb{F}_q^s \setminus \{\mathbf{0}\}$. Thus $P \circ A$ is

the unique polynomial that fits the condition of the interpolation step in LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) and so this subroutine returns $P(\mathbf{x})$ with probability at least $\frac{2}{3}$. $\square$

We note that one can push the $\frac{1}{3}$ fraction of errors to $\frac{1}{2} - \gamma$ for any $0 < \gamma < 1/2$ with a success probability of $\frac{1}{2} + \gamma$ (see Exercise 13.5).

With Lemma 13.1.5 in hand, some routine analysis suffices to show the correctness of the Simple Reed-Muller Decoder (Algorithm 13.1.1) and we do so in the theorem below.

Theorem 13.1.6. *The Simple Reed-Muller Decoder (Algorithm 13.1.1) is a correct (randomized) polynomial in n time algorithm decoding the Reed-Muller code $\text{RM}(q, m, r)$ from $e \leq \frac{1}{3} \cdot q^{m - \lceil (r+1)/(q-1) \rceil}$ errors.*

Proof. Fix $P \in \mathbb{F}_q[X_1, \dots, X_m]$ be a polynomial of degree at most r and $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ such that

$$e = |\{\mathbf{x} \in \mathbb{F}_q^m \mid f(\mathbf{x}) \neq P(\mathbf{x})\}| \leq \frac{1}{3} \cdot q^{m - \lceil (r+1)/(q-1) \rceil}.$$

Further fix $x \in q^m$. Lemma 13.1.5 asserts that a call to LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) returns $P(\mathbf{x})$ with probability at least $\frac{2}{3}$. By an application of the Chernoff bounds (in particular, see Exercise 3.3), the majority of the $O(m \log q)$ calls to LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) is $P(\mathbf{x})$ except with probability $\exp(-m \log q)$ and by setting the constant in the $O(\cdot)$ appropriately, we can ensure this probability is at most $\frac{q^{-m}}{3}$. We thus conclude that for every fixed $\mathbf{x} \in \mathbb{F}_q^m$ the probability that LOCAL-DECODE-RM-SIMPLE($f, \mathbf{x}$) does not return $P(\mathbf{x})$ is at most $\frac{q^{-m}}{3}$. By the union bound, we could that the probability that there exists $\mathbf{x} \in \mathbb{F}_q^m$ such that LOCAL-DECODE-RM-SIMPLE($f, \mathbf{x}$) $\neq P(\mathbf{x})$ is at most $\frac{1}{3}$. Thus with probability at least $\frac{2}{3}$ the algorithm computes $P(\mathbf{x})$ correctly for every $\mathbf{x}$ and thus the interpolation returns P with probability at least $\frac{2}{3}$.

The running time of the algorithm is easy to establish. Let $T_{\text{int}}(n)$ denote the time it takes to interpolate to find the coefficients of a polynomial P given its n evaluations. It is well-known that T_{int} is a polynomial with near linear running time. (See Remarks on Interpolation below.) We have that the LOCAL-DECODE-RM-SIMPLE-ITER takes time at most $T_{\text{int}}(q^s)$ per invocation, and thus LOCAL-DECODE-RM-SIMPLE takes $O(m \cdot T_{\text{int}}(q^s) \cdot \log q)$ steps per invocation. Since LOCAL-DECODE-RM-SIMPLE is executed q^m times by the overall algorithm, the overall running time is bounded by $T_{\text{int}}(q^m) + O(m \cdot q^m \cdot T_{\text{int}}(q^s) \log q)$. Expressed in terms of $n = q^m$ and $e_{\max} = q^{m-s}/3$ and crudely bounding interpolation cost by a cubic function, this translates into a running time of $O(n^3) + O\left(\frac{n^4}{e_{\max}^3} \log n\right)$. $\square$

Remarks on Interpolation. As mentioned in the proof above, the running time of the algorithm depends on the running time of the two interpolation steps in the algorithm DECODE-RM-SIMPLE. To get polynomial time algorithms for either step, it suffices to note that interpolation is just solving a system of linear equations and thus can always be solved in cubic time by Gaussian elimination (see Exercise 13.6). To make the steps more efficient, one can use the structure of polynomial interpolation to get some speedups for the first interpolation step (see Section 13.5). For the second, since we are only interested in evaluating $P \circ A(0)$, interpolation

is a bit of overkill. It turns out one can explicitly determine the exact linear form which describes $P \circ A(0)$ in terms of $\{P \circ A(\mathbf{z}) | \mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}\}$ and this turns out to be extremely simple: In fact $P \circ A(\mathbf{0}) = -\sum_{\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}} P \circ A(\mathbf{z})$ (see Exercise 13.7).

Remark on Fraction of Errors corrected. The number of errors corrected by the Simple Reed-Muller Decoder, $\frac{1}{3} \cdot q^{m - \lceil (r+1)/(q-1) \rceil}$, is complex and requires some explanation. It turns out that this quantity is closely related to the distance of the code. For $s = \lceil \frac{r+1}{q-1} \rceil$ if we now let t be such that $r = s(q-1) - t$ (note that this is different from the way we did this splitting in Lemma 9.4.1), then from Lemma 9.4.1 we have that the distance of the code $\text{RM}(q, m, r)$ is $(t+1)q^{m-s}$ where $1 \leq t \leq q-1$ (see Exercise 13.8). So in particular the distance of the code is between q^{m-s} and q^{m-s+1} . In contrast, our algorithm corrects $\frac{q^{m-s}}{3}$ errors, which is at least a $\frac{1}{3q}$ -fraction of the distance of the code. Ideally we would like algorithms correcting up to $\frac{1}{2}$ as many errors as the distance, and this algorithm falls short by a “constant” factor, if q is a constant. In the rest of the chapter we will try to achieve this factor of $\frac{1}{2}$.

13.2 Majority Logic Decoding

The algorithm of the previous section corrects errors up to a constant fraction of the distance (with the constant depending on q , but not on m or r) but is not the best one could hope for. In this section we develop an algorithm that corrects the optimal number of errors over $\mathbb{F}_2$. The main idea is to continue to explore the function over “affine subspaces” but now the substitutions will be much simpler. Specifically they will be of the form $x_i = b_i$ for many different choices of i where $b_i \in \mathbb{F}_2$. This will leave us with a function on the unset variables and while we won't be able to determine the function completely on the remaining variables, we will be able to determine some coefficients and this will allow us to make progress.

The main idea driving this algorithm is the following proposition about degree r polynomials.

Proposition 13.2.1. *Let $P \in \mathbb{F}_2[X_1, \dots, X_m]$ be of degree r and let $C \in \mathbb{F}_2$ be the coefficient of the monomial $\prod_{i=1}^r X_i$ in P . Then, for every $\mathbf{b} \in \mathbb{F}_2^{m-r}$, it is the case that $\sum_{\mathbf{a} \in \mathbb{F}_2^r} P(\mathbf{a}, \mathbf{b}) = C$.*

Proof. Let $P_{\mathbf{b}}(X_1, \dots, X_r) = P(X_1, \dots, X_r, \mathbf{b})$, i.e., $P_{\mathbf{b}}$ is P restricted to the subspace $X_i = b_i$ for $r < i \leq m$. Note that the coefficient of the monomial $X_1 \cdots X_r$ in $P_{\mathbf{b}}$ remains C , since all other monomial now have degree strictly less than r after the substitutions $X_i = b_i$. (Note that we used the fact that P has degree at most r to make this assertion.) So we can write $P_{\mathbf{b}} = C \cdot X_1 \cdots X_r + g$ where $\deg(g) < r$. We wish to show that

$$\sum_{\mathbf{a} \in \mathbb{F}_2^r} P_{\mathbf{b}}(\mathbf{a}) = \sum_{\mathbf{a} \in \mathbb{F}_2^r} C \cdot \left(\prod_{j=1}^r a_j \right) + \sum_{\mathbf{a} \in \mathbb{F}_2^r} g(\mathbf{a}) = C.$$

We first note that the first summation is trivially C since all terms except when $a_1 = \dots = a_r = 1$ evaluate to zero and the term corresponding to $a_1 = \dots = a_r = 1$ evaluates to C . The proposition

now follows from Exercise 13.7 which asserts that for every polynomial g of degree less than r , the summation $\sum_{\mathbf{a} \in \mathbb{F}_2^r} g(\mathbf{a}) = 0$. $\square$

As such the proposition above only seems useful in the error-free setting — after all, it assumes P is given correctly everywhere. But it extends to the setting of a small number of errors immediately. Note that if a function f disagrees with polynomial P on at most e points in $\mathbb{F}_2^m$ then there are at most e choices of $\mathbf{b} \in \mathbb{F}_2^{m-r}$ for which $\sum_{\mathbf{a} \in \mathbb{F}_2^r} f(\mathbf{a}, \mathbf{b})$ does not equal C . In particular, if $e < 2^{m-r}/2$ then the majority of choices of $\mathbf{b}$ lead to the correct value of C . (And remarkably, for the class of degree r polynomials, this is exactly one less than half the distance of the associated code.) Of course, the monomial $\prod_{i=1}^r X_i$ is not (very) special. The same reasoning allows us to compute the coefficient of any monomial of degree r . For example majority $\sum_{\mathbf{b} \in \mathbb{F}_2^{m-r}} \{ \sum_{\mathbf{a} \in \mathbb{F}_2^r} f(\mathbf{a}, \mathbf{b}) \}$ gives the coefficient of $X_1 \cdots X_r$, and majority $\sum_{\mathbf{b} \in \mathbb{F}_2^{m-r}} \{ \sum_{\mathbf{a} \in \mathbb{F}_2^r} f(\mathbf{b}, \mathbf{a}) \}$ (note the exchange of $\mathbf{a}$ and $\mathbf{b}$) gives the coefficient of $X_{m-r+1} \cdots X_m$. (See Exercise 13.9.) With appropriate notation for substituting $\mathbf{a}$ and $\mathbf{b}$ into the right places, we can calculate any other monomial of degree r as well. And then downward induction on r allows us to compute coefficients of lower degree monomials. This leads us to the algorithm described next. For the sake of completeness we also give the full analysis afterwards.

13.2.1 The Majority Logic Decoding Algorithm

We start with some notation that will help us describe the algorithm more precisely.

Definition 13.2.2. *For $S \subseteq [m]$ we let X_S denote the monomial $\prod_{i \in S} X_i$.*

Definition 13.2.3. *For $S \subseteq [m]$ with $|S| = t$ and vectors $\mathbf{a} \in \mathbb{F}_2^t$ and $\mathbf{b} \in \mathbb{F}_2^{m-t}$, let $(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b})$ denote the vector whose coordinates in S are given by $\mathbf{a}$ and coordinates in $\tilde{S}$ are given by $\mathbf{b}$.*

Definition 13.2.4. *For $S \subseteq [m]$ with $|S| = t$ and vectors $\mathbf{a} \in \mathbb{F}_2^t$ and $\mathbf{b} \in \mathbb{F}_2^{m-t}$, let $f(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b})$ denote the evaluation of f on $(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b})$. In other words, let $S = \{i_1, \dots, i_t\}$ with $i_k < i_{k+1}$ and let $\tilde{S} = \{j_1, \dots, j_{m-t}\}$ with $j_k < j_{k+1}$. Then $f(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b}) = f(\mathbf{c})$ where $\mathbf{c} \in \mathbb{F}_2^m$ is the vector such that $c_{i_k} = a_k$ and $c_{j_\ell} = b_\ell$.*

The majority logic decoder details are presented in Algorithm 13.2.1.

13.2.2 The analysis

We next argue that the algorithm MAJORITY LOGIC DECODER (Algorithm 13.2.1) correct up to half the errors for the $\text{RM}(2, m, r)$ code.

Lemma 13.2.5. *On input $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ that disagrees with a polynomial $Q \in \mathbb{F}_2[X_1, \dots, X_m]$ of degree at most r on at most $e < \frac{1}{2} \cdot 2^{m-r}$ points, the algorithm MAJORITY LOGIC DECODER(f) correctly outputs Q .*

Proof. Let $Q(X) = \sum_{S \subseteq [m]} C'_S X_S$ and let $Q_t(X) = \sum_{S \subseteq [m]: |S| \geq t} C'_S X_S$. We argue by downward induction on t (from $r+1$ down to 0) that $Q_t = P_t$ where P_t 's are the polynomials computed by

Algorithm 13.2.1 Majority Logic Decoder

INPUT: $r < m$, $0 \leq e < \frac{1}{2}2^{m-r}$, and function $f: \mathbb{F}_2^m \rightarrow \mathbb{F}_2$.

OUTPUT: Output P such $\deg(P) \leq r$ and $|\{x \in \mathbb{F}_2^m \mid P(x) \neq f(x)\}| \leq e$.

```
 $P_{r+1} \leftarrow 0$ 
FOR  $t = r$  DOWNTO 0 DO
   $f_t \leftarrow f - P_{t+1}$ 
  FOR every  $S \subseteq [m]$  WITH  $|S| = t$  DO
    FOR every  $\mathbf{b} \in \mathbb{F}_2^{m-t}$  DO
       $C_{S,\mathbf{b}} \leftarrow \sum_{\mathbf{a} \in \mathbb{F}_2^t} f_t(S \leftarrow \mathbf{a}, \bar{S} \leftarrow \mathbf{b})$ .
     $C_S \leftarrow \text{majority}_{\mathbf{b}} \{C_{S,\mathbf{b}}\}$ 
   $P_t \leftarrow P_{t+1} + \sum_{S \subseteq [m], |S|=t} C_S X_S$ 
RETURN  $P_0$ 
```

our algorithm. The base case is obvious since $P_{r+1} = Q_{r+1} = 0$. Assume now that $P_{t+1} = Q_{t+1}$ and so we have that $f_t = f - P_{t+1}$ disagrees with $Q - Q_{t+1}$ on at most e points. (See Exercise 13.10.) We now argue that for every subset $S \subseteq [m]$ with $|S| = t$, $C_S = C'_S$. Fix such a set S . For $\mathbf{b} \in \mathbb{F}_2^{m-t}$, we refer to the partial assignment $\bar{S} \leftarrow \mathbf{b}$ as a “subcube” corresponding to the points $\{(S \leftarrow \mathbf{a}, \bar{S} \leftarrow \mathbf{b}) \mid \mathbf{a} \in \mathbb{F}_2^t\}$. We say that a subcube $\bar{S} \leftarrow \mathbf{b}$ is in error if there exists $\mathbf{a}$ such that $f_t(S \leftarrow \mathbf{a}, \bar{S} \leftarrow \mathbf{b}) \neq (Q - Q_{t+1})(S \leftarrow \mathbf{a}, \bar{S} \leftarrow \mathbf{b})$. By Proposition 13.2.1, we have that if a subcube is not in error then $C_{S,\mathbf{b}} = C'_S$, since C'_S is the coefficient of X_S in the polynomial $(Q - Q_{t+1})$ (whose degree is at most t). Furthermore at most e subcubes can be in error (see Exercise 13.11). Finally, since the total number of subcubes is $2^{m-t} \geq 2^{m-r} > 2e$ we thus have that a majority of subcubes are not in error and so $C_S = \text{majority}_{\mathbf{b}} \{C_{S,\mathbf{b}}\} = C'_S$.

Thus we have for every S with $|S| = t$ that $C_S = C'_S$ and so $Q_t = P_t$, giving the inductive step. So we have for every t , $P_t = Q_t$ and in particular $P_0 = Q_0 = Q$ as desired. $\square$

The running time of the algorithm is easy to see as being at most $n = 2^m$ times the number of coefficients of a degree r polynomial, which is $\sum_{i=0}^r \binom{m}{i} \leq n$ in the binary case. Thus $O(n^2)$ is a crude upper bound on the running time of this algorithm. Note that this algorithm corrects up to exactly $\left\lfloor \frac{d-1}{2} \right\rfloor$ errors where $d = 2^{m-r}$ is the minimum distance of $\text{RM}(2, m, r)$. (Since d is even, this quantity equals $\frac{d}{2} - 1$.) We thus have the following theorem.

Theorem 13.2.6. *For every $0 \leq r < m$, The Majority Logic Decoder (Algorithm 13.2.1), corrects up to $\frac{d}{2} - 1$ errors in the Reed-Muller code, $\text{RM}(2, m, r)$, in $O(n^2)$ time, where $n = 2^m$ is the block length of the code and $d = 2^{m-r}$ is its minimum distance.*

13.3 Decoding by reduction to Reed-Solomon decoding

The algorithms described so far were based on very basic ideas, but they have their limitations. The SIMPLE REED-MULLER DECODER (Algorithm 13.1.1) fails to correct errors up to half the

minimum distance. And the majority logic algorithm (Algorithm 13.2.1) seems to work only over $\mathbb{F}_2$ (where the monomial structure is especially simple). The final algorithm we give uses a slightly more sophisticated algebraic idea, but then ends up yielding an almost ‘trivial’ reduction to Reed-Solomon decoding. (It is trivial in the sense that the reduction algorithm almost does no work.) The resulting reduction can use any algorithm for Reed-Solomon decoding including any of the ones from Chapter 12.

The crux of the reduction is a natural bijection between the vector space $\mathbb{F}_q^m$ and the field $\mathbb{F}_{q^m}$. This bijection converts the space of functions $\{f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q\}$ to the space of functions $\{f: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q\} \subseteq \{f: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_{q^m}\}$. Algorithmically, it is important that the bijection only acts on the domain and so almost no work is needed to convert a function $g \in \{f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q\}$ to its image $G \in \{f: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_{q^m}\}$ under the bijection. Thus Reed-Muller codes get transformed to a subcode of some Reed-Solomon code, and corrupted Reed-Muller codewords get mapped to corrupted Reed-Solomon codewords. Now comes the algebraic part: namely, analyzing how good is the distance of the so-obtained Reed-Solomon code, or equivalently upper bounding the degree of the polynomials G obtained by applying the bijection to $g \in \text{RM}(q, m, r)$. It turns out the bijection preserves the distance exactly and so algorithms correcting the Reed-Solomon code up to half its distance does the same for the Reed-Muller code.

In the rest of this section we first describe the ‘nice’ bijection Φ from $\mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ and introduce a parameter called the *extension degree* that captures how good the bijection is. Then, we analyze the extension degree of the bijection map and show that it ends up mapping Reed-Muller codes to Reed-Solomon codes of the same distance, and thus an algorithm to decode Reed-Solomon codes with errors up to half the distance of the code also yield algorithms to decode Reed-Muller code with errors up to half the distance of the code.

13.3.1 A bijection from $\mathbb{F}_q^m$ vs. $\mathbb{F}_{q^m}$

The bijections we will eventually work with in this section will be *linear-bijections*. We introduce this concept first.

Definition 13.3.1. *A function $\Phi: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ is said to be an $\mathbb{F}_q$ -linear bijection if*

1. Φ is a bijection, i.e., $\Phi(u) = \Phi(v) \Rightarrow u = v$ for every $u, v \in \mathbb{F}_{q^m}$.
2. Φ is $\mathbb{F}_q$ -linear; i.e., for every $\alpha, \beta \in \mathbb{F}_q$ and $u, v \in \mathbb{F}_{q^m}$ it is the case that $\Phi(\alpha u + \beta v) = \alpha \Phi(u) + \beta \Phi(v)$.

(Above and throughout this section it will be useful to remember that $\mathbb{F}_q \subseteq \mathbb{F}_{q^m}$ and so operations such as αu for $\alpha \in \mathbb{F}_q$ and $u \in \mathbb{F}_{q^m}$ are well-defined.)

Note that a linear bijection $\Phi: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ can be viewed as m functions $(\Phi_1, \dots, \Phi_m)$ with $\Phi_i: \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ so that $\Phi(u) = (\Phi_1(u), \dots, \Phi_m(u))$. Furthermore each Φ_i is a linear function from $\mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ and since $\mathbb{F}_q \subseteq \mathbb{F}_{q^m}$, Φ_i can be viewed as a polynomial in $\mathbb{F}_{q^m}[Z]$ (see Exercise 13.12). Our proposition below recalls the basic properties of such *linearized polynomials*.

Proposition 13.3.2. *There exists an $\mathbb{F}_q$ -linear bijection from $\mathbb{F}_{q^m}$ to $\mathbb{F}_q^m$. If $\Phi = (\Phi_1, \dots, \Phi_m): \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ is an $\mathbb{F}_q$ -linear bijection then each $\Phi_i(Z)$ is a trace function. Specifically there exist*

$\lambda_1, \dots, \lambda_m \in \mathbb{F}_{q^m}$, which are linearly independent over $\mathbb{F}_q$, such that $\Phi_i(Z) = \text{Tr}(\lambda_i Z) = \sum_{j=0}^{m-1} \lambda_i^{q^j} Z^{q^j}$. In particular $\deg(\Phi_i) = q^{m-1}$ and Φ is a linearized polynomial (i.e., only non-zero coefficients are for monomials of the form Z^{q^j}).

Proof. Given a bijection Φ the fact that it is a Trace function follows from Proposition D.5.18, which implies its degree and linearized nature (see Exercise 13.13). All that remains to show is that a linear bijection exists. We claim that if $\lambda_1, \dots, \lambda_m \in \mathbb{F}_{q^m}$ are $\mathbb{F}_q$ -linearly independent then $\Phi = (\Phi_1, \dots, \Phi_m)$, with $\Phi_i(Z) = \text{Tr}(\lambda_i Z)$, is a $\mathbb{F}_q$ linear bijection. Linearity follows from the linearity of Trace so all we need to verify is that this is a bijection. And since the domain and range of Φ have the same cardinality, it suffices to show that Φ is surjective. Consider the set

$$S = \{(\Phi(u) | u \in \mathbb{F}_{q^m}) \subseteq \mathbb{F}_q^m.$$

By the linearity of Φ , S is a subspace of $\mathbb{F}_q^m$. If $S \neq \mathbb{F}_q^m$ (i.e., if Φ is not surjective) we must have that elements of S satisfy some non-trivial constraint, i.e., there exists $(\alpha_1, \dots, \alpha_m) \in \mathbb{F}_q^m \setminus \{0\}$ such that $\sum_{i=1}^m \alpha_i \beta_i = 0$ for every $(\beta_1, \dots, \beta_m) \in S$ (see Exercise 13.14). But now consider

$$\sum_i \alpha_i \Phi_i(Z) = \sum_i \alpha_i \text{Tr}(\lambda_i Z) = \text{Tr}\left(\left(\sum_i \alpha_i \lambda_i\right) \cdot Z\right), \quad (13.2)$$

where the last equality follows from the fact that Tr is a linear map (see Proposition D.5.18). On the one hand (see Exercise 13.15) this is a non-zero polynomial in Z of degree at most q^{m-1} , while on the other hand it evaluates to zero on every $u \in \mathbb{F}_{q^m}$, which contradicts the degree mantra. We conclude Φ is surjective, and hence it is an $\mathbb{F}_q$ -linear bijection. $\square$

Our goal is to use a linear bijection from $\Phi : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ (any such bijection will do for us) to convert functions whose domain is $\mathbb{F}_{q^m}$ (which is the case for codewords of the Reed-Muller code) to functions whose domain is $\mathbb{F}_q^m$. Specifically, given $f : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ and $\Phi : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$, let $f \circ \Phi : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ be the function $(f \circ \Phi)(u) = f(\Phi(u))$.

Key to the utility of this transformation is the analysis of how this blows up the degree of the underlying polynomials. Recall that for a function $F : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$, its degree is defined to be the degree of the (unique) polynomial $P \in \mathbb{F}_{q^m}[Z]$ with $\deg(P) < q^m$ such that $P(u) = F(u)$ for every $u \in \mathbb{F}_{q^m}$. Our main challenge henceforth is to understand the question: If $f : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ is a degree r polynomial, how large can the degree of $f \circ \Phi$ be? We do not answer this question right away, but define the parameter quantifying this effect next, and then design and analyze a Reed-Muller decoding algorithm in terms of this parameter.

Definition 13.3.3. For prime power q and non-negative integers m and r , let the extension degree of r over $\mathbb{F}_{q^m}$, denoted $R_{q,m}(r)$, be the maximum degree of $p \circ \Phi$ over all choices of $p \in \mathbb{F}_q[X_1, \dots, X_m]$ (or the associated function $p : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$) of degree at most r and over all $\mathbb{F}_q$ -linear bijections $\Phi : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$.

Our algorithm and its analysis are quite natural modulo the analysis of $R_{q,m}(r)$ and we describe them below.

Algorithm 13.3.1 REED-SOLOMON-BASED DECODER

INPUT: $q, r < m(q-1)$, $0 \leq e < (q^m - R_{q,m}(r))/2$, and function $f : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$.

OUTPUT: Output p such $\deg(p) \leq r$ and $|\{x \in \mathbb{F}_q^m | P(x) \neq f(x)\}| \leq e$.

Let $\Phi : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q^m$ be an $\mathbb{F}_q$ -linear bijection

FOR $u \in \mathbb{F}_{q^m}$ DO

$F(u) \leftarrow f(\Phi(u))$

Let P be the output of decoding F using Reed-Solomon codes by Algorithm 12.1.1 ▷ With inputs $k = R_{q,m}(r) + 1$, $n = q^m$ and n pairs $(u, F(u))$ for every $u \in \mathbb{F}_{q^m}$.

FOR $u \in \mathbb{F}_{q^m}$ DO

$p(u) \leftarrow P(\Phi^{-1}(u))$

RETURN p

We note that the decoder here outputs a polynomial $p \in \mathbb{F}_q[X_1, \dots, X_m]$ in terms of its function representation. If a coefficient representation is desired one can use some interpolation algorithm to convert it. Other than such interpolation, most of the transformation above is mostly syntactic, since a normal representation of $\mathbb{F}_{q^m}$ is already in the form of vectors in $\mathbb{F}_q^m$ via some $\mathbb{F}_q$ -linear bijection. The only real work is in the call to the Reed-Solomon decoder.

Below we show that the algorithm above is correct for $e < (q^m - R_{q,m}(r))/2$. The crux of the analysis is in showing that this quantity is actually half the distance of the Reed-Muller code, and we defer that analysis to the next section.

Proposition 13.3.4. Let $f : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ be any function and let $g : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ be a degree r polynomial such that $|\{u \in \mathbb{F}_{q^m} | f(u) \neq g(u)\}| \leq e < (q^m - R_{q,m}(r))/2$. Then REED-SOLOMON-BASED DECODER(f) returns g

Proof. Let $G = g \circ \Phi$. Then we have that $\deg(G) \leq R_{q,m}(r)$ and we have that $|\{v \in \mathbb{F}_{q^m} | F(v) \neq G(v)\}| \leq e < (q^m - R_{q,m}(r))/2$. Since the distance of the Reed-Solomon code with $N = q^m$ and $K = R_{q,m}(r) + 1$ is $N - K + 1 = q^m - R_{q,m}(r)$ and e is less than half the distance, we have that G is the unique polynomial with this property, and so the Reed-Solomon decoder must return $P = G$. We conclude that $p = P \circ \Phi^{-1} = G \circ \Phi^{-1} = g$, as desired. $\square$

13.3.2 Analysis of Extension Degree

We start with a simple warmup result that already leads to an optimal algorithm for decoding when $r < q$.

Proposition 13.3.5. $R_{q,m}(r) = r \cdot q^{m-1}$.

Proof. We will prove $R_{q,m}(r) \leq r \cdot q^{m-1}$ and leave the other direction to Exercise 13.19. The proof following immediately from the definition and the fact that linear functions are polynomials of degree q^{m-1} . Specifically, let $p \in \mathbb{F}_q[X_1, \dots, X_m]$ be of degree at most r and let $\Phi = (\Phi_1, \dots, \Phi_m)$ be an $\mathbb{F}_q$ -linear bijection. Then since each Φ_i is a polynomial of degree q^{m-1} we have that

$p(\Phi_1(Z), \dots, \Phi_m(Z))$ is a polynomial of degree at most $r \cdot q^{m-1}$. Finally since the reduction modulo $(Z^{q^m} - Z)$ does not increase the degree we have $p \circ \Phi$ is a polynomial of degree at most $r \cdot q^{m-1}$, as desired. $\square$

Corollary 13.3.6. *If $r < q$ then REED-SOLOMON-BASED DECODER decodes $\text{RM}(q, m, r)$ up to half its minimum distance.*

Proof. By Lemma 9.2.2 we have that the distance of the Reed-Muller code $\text{RM}(q, m, r)$ is $(q - r) \cdot q^{m-1}$. From Proposition 13.3.5 we have that REED-SOLOMON-BASED DECODER decodes provided $e < (q^m - R_{q,m}(r))/2 = (q^m - r \cdot q^{m-1})/2 = (q - r) \cdot q^{m-1}/2$, i.e., up to half the minimum distance. $\square$

Finally we turn to the general case (i.e. $r \geq q$). For this part, the crude bound that the degree of $f \circ \Phi$ is at most $q^{m-1} \cdot \deg(f)$ is no longer good enough. This bound is larger than q^m , whereas every function has degree at most $q^m - 1$. To get the ‘right’ degree bound on the degree of $f \circ \Phi$, we now need to use the fact that we can reduce any polynomial modulo $(Z^{q^m} - Z)$ and this leaves the underlying function on $\mathbb{F}_{q^m}$ unchanged. Thus from this point on we will try to understand the degree of $f \circ \Phi \pmod{Z^{q^m} - Z}$. Using this reduction properly we will eventually be able to get the correct bound on the degree of $f \circ \Phi$. We state the bound next and then work our way towards proving it.

Lemma 13.3.7. *Let $r = s(q - 1) + t$ where $0 \leq t < q - 1$. Then $R_{q,m}(r) = q^m - (q - t)q^{m-s-1}$.*

We first state the immediate consequence of Lemma 13.3.7 to the error-correction bound of the Reed-Solomon-based decoder.

Theorem 13.3.8. *For every prime power q , integers $m \geq 1$ and $0 \leq r < m(q - 1)$, the REED-SOLOMON-BASED DECODER decodes $\text{RM}(q, m, r)$ up to half its minimum distance.*

Proof. Let $r = s(q - 1) + t$ with $0 \leq t < q - 1$. By the polynomial distance lemma (Lemma 9.4.1) we have that the distance of the Reed-Muller code $\text{RM}(q, m, r)$ is $(q - t) \cdot q^{m-s-1}$. Combining Proposition 13.3.4 with Lemma 13.3.7 we have that REED-SOLOMON-BASED DECODER decodes provided $e < (q^m - R_{q,m}(r))/2 = (q - t) \cdot q^{m-s-1}/2$, i.e., up to half the minimum distance of $\text{RM}(q, r, m)$. $\square$

The proof of Lemma 13.3.7 is somewhat involved and needs some new notions. We introduce these notions next.

Definition 13.3.9 (q -degree). *For integer d , let $d_0, d_1, d_2, \dots$ denote its expansion in base q , i.e., $d = \sum_{i=0}^{\infty} d_i q^i$ and $0 \leq d_i < q$ for every i . For a monomial Z^d , define its q -degree, denoted $\deg_q(Z^d)$, to be the quantity $\sum_{i=0}^{\infty} d_i$. For a polynomial $p(Z) = \sum_d c_d Z^d$, define its q -degree, denoted $\deg_q(p)$, to be $\max_{d | c_d \neq 0} \{\deg_q(Z^d)\}$.*

For example $\deg_2(X^8 + X^3 + X + 1) = \deg_2(X^3) = 2$.

We describe some basic properties of q -degree below. Informally, the proposition below proves (in parts (1)-(3)) that the q -degree behaves just like the regular degree when it comes

to addition, multiplication and reduction modulo $Z^{q^m} - Z$. Note that while parts (1) and (2) are natural, part (3) is already special in that it only holds for reduction modulo some special polynomials. Finally part (4) of the proposition allows us to related the q -degree of a polynomial with its actual degree and this will come in useful when we try to bound the degree of $f \circ \Phi \pmod{Z^{q^m} - Z}$.

Proposition 13.3.10. *For every $\alpha, \beta \in \mathbb{F}_{q^m}$ and $P, P_1, P_2 \in \mathbb{F}_{q^m}[Z]$ we have:*

- (1) $\deg_q(\alpha P_1 + \beta P_2) \leq \max\{\deg_q(P_1), \deg_q(P_2)\}$.
- (2) $\deg_q(P_1 \cdot P_2) \leq \deg_q(P_1) + \deg_q(P_2)$.
- (3) $\deg_q(P \pmod{Z^{q^m} - Z}) \leq \deg_q(P)$. (Note that here the total degree $\deg(P)$ can be strictly greater than q^m .)
- (4) $\deg(P) < q^m$ and $\deg_q(P) = s(q - 1) + t$ for $0 \leq t < q - 1$ implies

$$\deg(P) \leq q^m - (q - t)q^{m-s-1}.$$

Proof. Part (1) is immediate from the definition since the monomials of $\alpha P_1 + \beta P_2$ is in the union of the monomials of P_1 and P_2 . Next, note that due to part (1), it suffices to prove parts (2)-(4) for monomials.

For part (2) for monomials, we wish to show that $\deg_q(Z^d \cdot Z^e) \leq \deg_q(Z^d) + \deg_q(Z^e)$. Let $d = \sum_i d_i q^i$, $e = \sum_i e_i q^i$ and let $f = d + e = \sum_i f_i q^i$. Then it can be verified that for every i we have $\sum_{j=0}^i f_j \leq \sum_{j=0}^i (d_j + e_j)$ (see Exercise 13.16) and this eventually implies

$$\deg_q(Z^{d+e}) = \sum_j f_j \leq \sum_j (d_j + e_j) = \deg_q(Z^d) + \deg_q(Z^e). \quad (13.3)$$

For part (3), note that since $Z^{q^i} = \left(\left(Z^{q^{\lfloor i/m \rfloor}} \right)^m \right)^i \pmod{Z^{q^m} - Z}$,

$$Z^{q^i} \pmod{Z^{q^m} - Z} = Z^{q^{(i \bmod m)}}.$$

So

$$Z^{d_i q^i} \pmod{Z^{q^m} - Z} = Z^{d_i q^{(i \bmod m)}}. \quad (13.4)$$

We conclude that for $d = \sum_i d_i q^i$ with $0 \leq d_i < q$ we have:

$$\begin{aligned} \deg_q \left(Z^d \pmod{Z^{q^m} - Z} \right) &= \deg_q \left(Z^{\sum_i d_i q^i} \pmod{Z^{q^m} - Z} \right) \\ &\leq \deg_q \left(Z^{\sum_i d_i q^{(i \bmod m)}} \right) \\ &\leq \sum_i \deg_q \left(Z^{d_i q^{(i \bmod m)}} \right) \\ &= \sum_i d_i \\ &= \deg_q(Z^d), \end{aligned}$$

as desired. In the above, the first inequality follows from Exercise 13.17 and the second inequality follows from part (2) while the final two equalities follow from definition of $\deg_q(\cdot)$.

Finally we turn to part (4), which compares the (actual) degree of a polynomial to its q -degree. Again it suffices to prove for monomials. Let $d < q^m$ be given by $d = \sum_{i=0}^{m-1} d_i q^i$. Subject to the condition $\sum_i d_i \leq s(q-1) + t$, we note that d is maximized when $d_{m-1} = \dots = d_{m-s} = (q-1)$ and $d_{m-s-1} = t$ (see Exercise 13.18) in which case we get $d + (q-t)q^{m-s-1} = q^m$, or $d = q^m - (q-t)q^{m-s-1}$. $\square$

Our next lemma relates the degree of a multivariate function $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ to the q -degree of $f \circ \Phi$ for a linear bijection Φ .

Lemma 13.3.11. *For every polynomial $p \in \mathbb{F}_q[X_1, \dots, X_m]$ and every $\mathbb{F}_q$ -linear bijection, we have $\deg_q(p \circ \Phi) \leq \deg(p)$.*

Proof. By Proposition 13.3.10, part (1), it suffices to prove the lemma for the special case of monomials. Fix a monomial $M(X_1, \dots, X_m) = X_1^{r_1} \dots X_m^{r_m}$ with $\sum_j r_j = r$. Also fix an $\mathbb{F}_q$ -linear bijection $\Phi = (\Phi_1, \dots, \Phi_m)$. Let $\tilde{M}$ denote the univariate polynomial $M \circ \Phi \pmod{Z^{q^m} - Z}$. Note that $M \circ \Phi(Z) = \prod_{i=1}^m \Phi_i(Z)^{r_i}$. And note further that $\deg_q(\Phi_i(Z)) = 1$ for every i . By Proposition 13.3.10 part (2), we now conclude that $\deg_q(\prod_{i=1}^m \Phi_i(Z)^{r_i}) \leq \sum_{i=1}^m r_i = r$. Finally by Proposition 13.3.10 part (3) we have that

$$\deg_q(\tilde{M}) = \deg_q(M \circ \Phi \pmod{Z^{q^m} - Z}) \leq \deg_q(M \circ \Phi) \leq r,$$

as desired. $\square$

We are now ready to prove Lemma 13.3.7 which asserts that $R_{q,m}(s(q-1) + t) = q^m - (q-t)q^{m-s-1}$.

Proof of Lemma 13.3.7. We focus on proving $R_{q,m}(s(q-1) + t) \leq q^m - (q-t)q^{m-s-1}$. The other direction follows from Exercise 13.19. Fix a polynomial $p \in \mathbb{F}_q[X_1, \dots, X_m]$ of degree at most $r = s(q-1) + t$ and consider the function $p \circ \Phi: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$. This corresponds to the polynomial $\tilde{p}(Z) = p(\Phi_1(Z), \dots, \Phi_m(Z)) \pmod{Z^{q^m} - Z}$. By Lemma 13.3.11 we have that $\deg_q(\tilde{p}) \leq r$. And by construction $\deg(\tilde{p}) < q^m$. So by Proposition 13.3.10, part (4), we have that $\deg(\tilde{p}) \leq q^m - (q-t)q^{m-s-1}$, yielding the lemma. $\square$

13.4 Exercises

Exercise 13.1. *Show that if $q > \deg(P)$, then for any polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$, there exists an m -dimensional affine form A such that $\deg(P \circ A) = \deg(P)$.*

Exercise 13.2. *An s -variate d -form is a polynomial $a(Z_1, \dots, Z_m)$ of degree at most d . Note that $d = 1$ gives Definition 13.1.1. Similar to Definition 13.1.1 one can define an m -dimensional s -variate d -form $\langle a_1, \dots, a_m \rangle$. Finally, given such a d -form analogous to Definition 13.1.2, for an m -variate polynomial P one can define $P \circ A$.*

Prove that

$$\deg(P \circ A) \leq d \cdot \deg(P).$$

Exercise 13.3. *Prove that for every pair $\mathbf{z}_1 \neq \mathbf{z}_2 \in \mathbb{F}_q^s$, for a random affine form A , $A(\mathbf{z}_1)$ and $A(\mathbf{z}_2)$ are distributed uniformly and independently over $\mathbb{F}_q^m$.*

Exercise 13.4. *Show that any two distinct polynomials in $\mathbb{F}_q[Z_1, \dots, Z_s]$ of degree at most $r < s(q-1)$ differ in at least two positions $\mathbf{x} \in \mathbb{F}_q^s$.*

Hint: Use the polynomial distance lemma (Lemma 9.4.1).

Exercise 13.5. *Let $P \in \mathbb{F}_q[X_1, \dots, X_m]$ be a polynomial of degree at most r , $0 < \gamma < \frac{1}{2}$ and let $f: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ be such that*

$$e = |\{\mathbf{x} \in \mathbb{F}_q^m \mid f(\mathbf{x}) \neq P(\mathbf{x})\}| \leq \left(\frac{1}{2} - \gamma\right) \cdot q^{m - \lceil (r+1)/(q-1) \rceil}.$$

Then, for every $\mathbf{x} \in \mathbb{F}_q^m$, the probability that LOCAL-DECODE-RM-SIMPLE-ITER($\mathbf{x}, f$) returns $P(\mathbf{x})$ is at least $\frac{1}{2} + \gamma$.

Exercise 13.6. *Let $g: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ and integer $0 \leq r < m(q-1) - 1$ be given. Then one can in $O(q^{3m})$ operations compute a polynomial $P \in \mathbb{F}_q[X_1, \dots, X_m]$ of degree at most r such that for every $\mathbf{x} \in \mathbb{F}_q^m$, $P(\mathbf{x}) = g(\mathbf{x})$ (if such a polynomial exists).*

Hint: Use Exercise 2.7.

Exercise 13.7. *If $P: \mathbb{F}_q^s \rightarrow \mathbb{F}_q$ is a polynomial of degree $r < s(q-1)$ then*

$$P(\mathbf{0}) = - \sum_{\mathbf{z} \in \mathbb{F}_q^s \setminus \{\mathbf{0}\}} P(\mathbf{z}).$$

Hint: Use Exercise 9.15.

Exercise 13.8. *Let $r = s(q-1) - t$. Then $\text{RM}(q, s, r)$ has distance at least $(t+1)q^{m-s}$.*

Exercise 13.9. *Let $f: \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ differ from a degree r polynomial P (of degree r) in $< 2^{m-r-1}$ locations. Show that majority $_{\mathbf{b} \in \mathbb{F}_2^{m-r}} \{\sum_{\mathbf{a} \in \mathbb{F}_2^r} f(\mathbf{a}, \mathbf{b})\}$ gives the coefficient of $X_1 \dots X_r$ in $P(X_1, \dots, X_m)$ and majority $_{\mathbf{b} \in \mathbb{F}_2^{m-r}} \{\sum_{\mathbf{a} \in \mathbb{F}_2^r} f(\mathbf{b}, \mathbf{a})\}$ (note the exchange of $\mathbf{a}$ and $\mathbf{b}$) gives the coefficient of $X_{m-r+1} \dots X_m$ in $P(X_1, \dots, X_m)$.*

Exercise 13.10. *Let $f, g: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ disagree in e positions $\mathbf{x} \in \mathbb{F}_q^m$. Then for any function $h: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$, the functions $f - h$ and $g - h$ also disagree in e positions.*

Exercise 13.11. *Let $f, g: \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ disagree in e positions $\mathbf{x} \in \mathbb{F}_q^m$. Fix a subset $S \subseteq [m]$ of size t . Argue that there are at most e values of $\mathbf{b} \in \mathbb{F}_q^{m-t}$ for which there is an $\mathbf{a} \in \mathbb{F}_q^t$ such that $f(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b}) \neq g(S \leftarrow \mathbf{a}, \tilde{S} \leftarrow \mathbf{b})$.*

Exercise 13.12. *Let $\Phi: \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ be a linear bijection. How that*

1. Φ can be viewed as m functions $(\Phi_1, \dots, \Phi_m)$ with $\Phi_i : \mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ so that $\Phi(u) = (\Phi_1(u), \dots, \Phi_m(u))$.
2. Furthermore each Φ_i is a linear function from $\mathbb{F}_{q^m} \rightarrow \mathbb{F}_q$ and each Φ_i can be viewed as a polynomial in $\mathbb{F}_{q^m}[Z]$.

Exercise 13.13. Show that a linear-bijection is a linearized polynomial of degree q^{m-1} .

Exercise 13.14. Argue that if a linear subspace $S \subset \mathbb{F}_q^m$ of dimension $< m$, then there $(\alpha_1, \dots, \alpha_m) \in \mathbb{F}_q^m \setminus \{0\}$ such that $\sum_{i=1}^m \alpha_i \beta_i = 0$ for every $(\beta_1, \dots, \beta_m) \in S$.

Exercise 13.15. Argue that the polynomial $\text{Tr}((\sum_i \alpha_i \lambda_i) \cdot Z)$ from (13.2) is

1. Non-zero and has degree at most q^{m-1} ; and
2. Evaluates to zero on all $u \in \mathbb{F}_{q^m}$.

Exercise 13.16. Let $(d_0, d_1, \dots)$ and $(e_0, e_1, \dots)$ be d and e in base $q \geq 2$. Let $f = d + e = \sum_i f_i q^i$. Then show that for every i we have $\sum_{j=0}^i f_j \leq \sum_{j=0}^i (d_j + e_j)$.

Hint: Use induction.

Exercise 13.17. Show that

$$\deg_q \left(Z^{\sum_i d_i q^i} \mod (Z^{q^m} - Z) \right) \leq \deg_q \left(Z^{\sum_i d_i q^i \mod m} \right).$$

Exercise 13.18. Show that subject to the condition $\sum_{i=0}^{m-1} d_i \leq s(q-1) + t$, that $d = \sum_{i=0}^{m-1} d_i q^i$ is maximized when $d_{m-1} = \dots = d_{m-s} = (q-1)$ and $d_{m-s-1} = t$.

Exercise 13.19. $T \subseteq \mathbb{F}_q$ be a set of size t . Consider the polynomial

$$p(X_1, \dots, X_m) = (X_1^{q-1} - 1) \cdots (X_s^{q-1} - 1) \cdot \prod_{a \in T} (X_{s+1} - a).$$

Note that $\deg(p) = s(q-1) + t$.

1. Prove that for every linear bijection Φ , we have $\deg(p \circ \Phi) \geq q^m - (q-t)q^{m-s-1}$.
Hint: How many zeroes does p have? What does this say about the degree of $p \circ \Phi$?
2. Conclude that $R_{q,m}(r) \geq q^m - (q-t)q^{m-s-1}$, where $r = s(q-1) + t$ where $0 \leq t < q-1$.

13.5 Bibliographic Notes

We start with the history of the Majority Logic Decoder, Algorithm 13.2.1. This algorithm is the first non-trivial algorithm in coding theory, dating back to the paper of Reed [130] from 1954. Indeed the codes were proposed by Muller [118] and the reason Reed's name is associated with the codes is due to the decoding algorithm. Despite their “in hindsight” simplicity, the algorithm is particularly clever and subtle. One proof of the subtlety is in the fact that the algorithm doesn't seem to extend immediately to non-binary settings — indeed even extending to the

ternary case ends up involving some careful reasoning (and is settled in the work of Kim and Kopparty [99]).

See [161, Chapter 10] for details on near-linear time polynomial interpolation algorithm.

The Simple Reed-Muller Decoder, Algorithm 13.1.1, originates from the work of Beaver and Feigenbaum [15] and Lipton [107]. (We note that neither paper mentions the codes by name, and only consider the case $r < q$.) Subsequent works by Gemmell et al. [58] and Gemmell and Sudan [60] extended these algorithms to correct a number of errors *close to* (but not matching) half the distance of the code. These algorithms also played a central role in the theory of “locally decodable codes” that we will describe later.

Finally, the Reed-Solomon-based Decoder, Algorithm 13.3.1, is due to Pellikaan and Wu [123], who gave this algorithm in the context of “list-decoding”.

Chapter 14

Decoding Concatenated Codes

In this chapter, we study Question 10.4.1. Recall that the concatenated code $C_{\text{out}} \circ C_{\text{in}}$ consists of an outer $[N, K, D]_{Q=q^k}$ code C_{out} and an inner $[n, k, d]_q$ code C_{in} , where $Q = O(N)$. (Figure 14.1 illustrates the encoding function.) Then $C_{\text{out}} \circ C_{\text{in}}$ has design distance Dd and Question 10.4.1 asks if we can decode concatenated codes up to half the design distance (say for concatenated codes that we saw in Section 10.2 that lie on the Zyablov bound). In this chapter, we begin with a very natural unique decoding algorithm that can correct up to $Dd/4$ errors. Then we will consider a more sophisticated algorithm that will allow us to answer Question 10.4.1 in the affirmative.

14.1 A Natural Decoding Algorithm

We begin with a natural decoding algorithm for concatenated codes that “reverses” the encoding process (as illustrated in Figure 14.1). In particular, the algorithm first decodes the inner code and then decodes the outer code.

For the time being, let us assume that we have a polynomial time unique decoding algorithm $D_{C_{\text{out}}} : [q^k]^N \rightarrow [q^k]^K$ for the outer code that can correct up to $D/2$ errors.

This leaves us with the task of coming up with a polynomial time decoding algorithm for the inner code. Our task of coming up with such a decoder is made easier by the fact that the running time needs to be polynomial in the *final* block length. This in turn implies that we would be fine if we pick a decoding algorithm that runs in singly exponential time in the inner block length as long as the inner block length is logarithmic in the outer code block length. (Recall that we put this fact to good use in Section 10.2 when we constructed explicit codes on the Zyablov bound.) Note that the latter is what we have assumed so far and thus, we can use the Maximum Likelihood Decoder (or MLD) (recall Algorithm 1.4.1, which we will refer to as $D_{C_{\text{in}}}$). Algorithm 14.1.1 formalizes this algorithm.

It can be shown that each step of Algorithm 14.1.1 can be implemented in polynomial time. In particular,

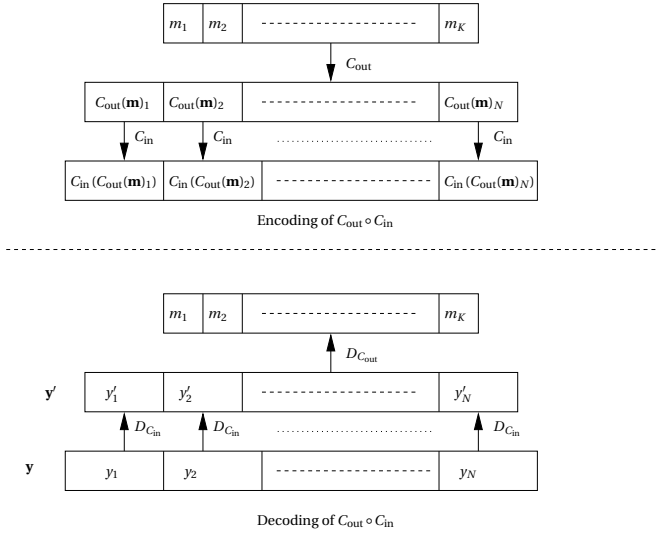


Figure 14.1: Encoding and Decoding of the concatenated code $C_{\text{out}} \circ C_{\text{in}}$. $D_{C_{\text{out}}}$ is a unique decoding algorithm for C_{out} and $D_{C_{\text{in}}}$ is a unique decoding algorithm for the inner code (e.g. MLD).

Algorithm 14.1.1 Natural Decoder for $C_{\text{out}} \circ C_{\text{in}}$

INPUT: Received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$

OUTPUT: Message $\mathbf{m}' \in [q^k]^K$

1: $\mathbf{y}' \leftarrow (y'_1, \dots, y'_N) \in [q^k]^N$ where

$$C_{\text{in}}(y'_i) = D_{C_{\text{in}}}(y_i) \quad 1 \leq i \leq N.$$

2: $\mathbf{m}' \leftarrow D_{C_{\text{out}}}(\mathbf{y}')$

3: RETURN $\mathbf{m}'$

1. The time complexity of Step 1 is $O(nq^k)$, which for our choice of $k = O(\log N)$ (and constant rate) for the inner code, is $(nN)^{O(1)}$ time.
2. Step 2 needs polynomial time by our assumption that the unique decoding algorithm $D_{C_{\text{out}}}$ takes $N^{O(1)}$ time.

Next, we analyze the error-correction capabilities of Algorithm 14.1.1:

Proposition 14.1.1. *Algorithm 14.1.1 can correct $< \frac{Dd}{4}$ many errors.*

Proof. Let $\mathbf{m}$ be the (unique) message such that $\Delta(C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}), \mathbf{y}) < \frac{Dd}{4}$.

We begin the proof by defining a bad event as follows. We say a *bad event* has occurred (at position $1 \leq i \leq N$) if $y_i \neq C_{\text{in}}(C_{\text{out}}(\mathbf{m})_i)$. More precisely, define the set of all bad events to be

$$\mathcal{B} = \{i \mid y_i \neq C_{\text{in}}(C_{\text{out}}(\mathbf{m})_i)\}.$$

Note that if $|\mathcal{B}| < \frac{D}{2}$, then the decoder in Step 2 will output the message $\mathbf{m}$. Thus, to complete the proof, we only need to show that $|\mathcal{B}| < D/2$. To do this, we will define a superset $\mathcal{B}' \supseteq \mathcal{B}$ and then argue that $|\mathcal{B}'| < D/2$, which would complete the proof.

Note that if $\Delta(y_i, C_{\text{in}}(C_{\text{out}}(\mathbf{m})_i)) < \frac{d}{2}$ then $i \notin \mathcal{B}$ (by the proof of Proposition 1.4.2)– though the other direction does not hold. We define $\mathcal{B}'$ to be the set of indices where $i \in \mathcal{B}'$ if and only if

$$\Delta(y_i, C_{\text{in}}(C_{\text{out}}(\mathbf{m})_i)) \geq \frac{d}{2}.$$

Note that $\mathcal{B} \subseteq \mathcal{B}'$.

Now by definition, note that the total number of errors is at least $|\mathcal{B}'| \cdot \frac{d}{2}$. Thus, if $|\mathcal{B}'| \geq \frac{D}{2}$, then the total number of errors is at least $\frac{D}{2} \cdot \frac{d}{2} = \frac{Dd}{4}$, which is a contradiction. Thus, $|\mathcal{B}'| < \frac{D}{2}$, which completes the proof. $\square$

Note that Algorithm 14.1.1 (as well the proof of Proposition 14.1.1) can be easily adapted to work for the case where the inner codes are different, e.g. Justesen codes (Section 10.3).

Thus, Proposition 14.1.1 and Theorem 14.3.3 imply that

Theorem 14.1.2. *There exists an explicit linear code on the Zyablov bound that can be decoded up to a fourth of the Zyablov bound in polynomial time.*

This of course is predicated on the fact that we need a polynomial time unique decoder for the outer code. Note that Theorem 14.1.2 implies the existence of an explicit asymptotically good code that can be decoded from a constant fraction of errors.

We now state an obvious open question and an observation. The first is to get rid of the assumption on the existence of $D_{C_{\text{out}}}$:

Question 14.1.1. *Does there exist a polynomial time unique decoding algorithm for outer codes, e.g. for Reed-Solomon codes?*

Next, note that Proposition 14.1.1 does not quite answer Question 10.4.1. We move to answering this latter question next.

14.2 Decoding From Errors and Erasures

Now we digress a bit from answering Question 10.4.1 and talk about decoding Reed-Solomon codes. For the rest of the chapter, we will assume the following result.

Theorem 14.2.1. *An $[N, K]_q$ Reed-Solomon code can be corrected from e errors (or s erasures) as long as $e < \frac{N-K+1}{2}$ (or $s < N-K+1$) in $O(N^3)$ time.*

We defer the proof of the result on decoding from errors to Chapter 12 and leave the proof of the erasure decoder as an exercise. Next, we show that we can get the best of both worlds by correcting errors and erasures simultaneously:

Theorem 14.2.2. *An $[N, K]_q$ Reed-Solomon code can be corrected from e errors and s erasures in $O(N^3)$ time as long as*

$$2e + s < N - K + 1. \quad (14.1)$$

Proof. Given a received word $\mathbf{y} \in (\mathbb{F}_q \cup \{?\})^N$ with s erasures and e errors, let $\mathbf{y}'$ be the sub-vector with no erasures. This implies that $\mathbf{y}' \in \mathbb{F}_q^{N-s}$ is a valid received word for an $[N-s, K]_q$ Reed-Solomon code. (Note that this new Reed-Solomon code has evaluation points that correspond to evaluation points of the original code, in the positions where an erasure did not occur.) Now run the error decoder algorithm from Theorem 14.2.1 on $\mathbf{y}'$. It can correct $\mathbf{y}'$ as long as

$$e < \frac{(N-s) - K + 1}{2}.$$

This condition is implied by (14.1). Thus, we have proved one can correct e errors under (14.1). Now we have to prove that one can correct the s erasures under (14.1). Let $\mathbf{z}'$ be the output after correcting e errors. Now we extend $\mathbf{z}'$ to $\mathbf{z} \in (\mathbb{F}_q \cup \{?\})^N$ in a natural way. Finally, run the erasure decoding algorithm from Theorem 14.2.1 on $\mathbf{z}$. This works as long as $s < (N-K+1)$, which in turn is true by (14.1).

The time complexity of the above algorithm is $O(N^3)$ as both the algorithms from Theorem 14.2.1 can be implemented in cubic time. $\square$

Next, we will use the above errors and erasure decoding algorithm to design decoding algorithms for certain concatenated codes that can be decoded up to half their design distance (i.e. up to $Dd/2$).

14.3 Generalized Minimum Distance Decoding

Recall the natural decoding algorithm for concatenated codes from Algorithm 14.1.1. In particular, we performed MLD on the inner code and then fed the resulting vector to a unique decoding algorithm for the outer code. A drawback of this algorithm is that it does not take into account the information that MLD provides. For example, it does not distinguish between the situations where a given inner code's received word has a Hamming distance of one vs where the received word has a Hamming distance of (almost) half the inner code distance from the closest codeword. It seems natural to make use of this information. Next, we study an algorithm called the Generalized Minimum Distance (or GMD) decoder, which precisely exploits this extra information.

In the rest of the section, we will assume C_{out} to be an $[N, K, D]_{q^k}$ code that can be decoded (by $D_{C_{\text{out}}}$) from e errors and s erasures in polynomial time as long as $2e + s < D$. Further, let C_{in} be an $[n, k, d]_q$ code with $k = O(\log N)$ which has a unique decoder $D_{C_{\text{in}}}$ (which we will assume is the MLD implementation from Algorithm 1.4.1).

We will in fact look at three versions of the GMD decoding algorithm. The first two will be randomized algorithms while the last will be a deterministic algorithm. We will begin with the first randomized version, which will present most of the ideas in the final algorithm.

14.3.1 GMD algorithm- I

Before we state the algorithm, let us look at two special cases of the problem to build some intuition.

Consider the received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$ with the following special property: for every i such that $1 \leq i \leq N$, either $y_i = y'_i$ or $\Delta(y_i, y'_i) \geq d/2$, where $y'_i = \text{MLD}_{C_{\text{in}}}(y_i)$. Now we claim that if $\Delta(\mathbf{y}, C_{\text{out}} \circ C_{\text{in}}) < dD/2$, then there are $< D$ positions in $\mathbf{y}$ such that $\Delta(y_i, C_{\text{in}}(y'_i)) \geq d/2$ (we call such a position *bad*). This is because, for every bad position i , by the definition of y'_i , $\Delta(y_i, C_{\text{in}}) \geq d/2$. Now if there are $\geq D$ bad positions, this implies that $\Delta(\mathbf{y}, C_{\text{out}} \circ C_{\text{in}}) \geq dD/2$, which is a contradiction. Now note that we can decode $\mathbf{y}$ by just declaring an erasure at every bad position and running the erasure decoding algorithm for C_{out} on the resulting vector.

Now consider the received word $\mathbf{y} = (y_1, \dots, y_N)$ with the special property: for every i such that $i \in [N]$, $y_i \in C_{\text{in}}$. In other words, if there is an error at position $i \in [N]$, then a valid codeword in C_{in} gets mapped to another valid codeword $y_i \in C_{\text{in}}$. Note that this implies that a position with error has at least d errors. By a counting argument similar to the ones used in the previous paragraph, we have that there can be $< D/2$ such error positions. Note that we can now decode $\mathbf{y}$ by essentially running a unique decoder for C_{out} on $\mathbf{y}$ (or more precisely on $(x_1, \dots, x_N)$, where $y_i = C_{\text{in}}(x_i)$).

Algorithm 14.3.1 generalizes these observations to decode arbitrary received words. In particular, it smoothly “interpolates” between the two extreme scenarios considered above.

Note that if $\mathbf{y}$ satisfies one of the two extreme scenarios considered earlier, then Algorithm 14.3.1 works exactly the same as discussed above.

By our choice of $D_{C_{\text{out}}}$ and $D_{C_{\text{in}}}$, it can be shown that Algorithm 14.3.1 runs in polynomial time (in the final block length). More importantly, we will show that the final (deterministic)

Algorithm 14.3.1 Generalized Minimum Decoder (ver 1)

INPUT: Received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$

OUTPUT: Message $\mathbf{m}' \in [q^k]^K$

```

1: FOR  $1 \leq i \leq N$  DO
2:    $y'_i \leftarrow D_{C_{\text{in}}}(y_i)$ .
3:    $w_i \leftarrow \min\left(\Delta(y'_i, y_i), \frac{d}{2}\right)$ .
4:   With probability  $\frac{2w_i}{d}$ , set  $y''_i \leftarrow ?$ , otherwise set  $y''_i \leftarrow x$ , where  $y'_i = C_{\text{in}}(x)$ .
5:  $\mathbf{m}' \leftarrow D_{C_{\text{out}}}(\mathbf{y}'')$ , where  $\mathbf{y}'' = (y''_1, \dots, y''_N)$ .
6: RETURN  $\mathbf{m}'$ 

```

version of Algorithm 14.3.1 can do unique decoding of $C_{\text{out}} \circ C_{\text{in}}$ up to half of its design distance.

As a first step, we will show that in expectation, Algorithm 14.3.1 works.

Lemma 14.3.1. *Let $\mathbf{y}$ be a received word such that there exists a codeword $C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}) = (c_1, \dots, c_N) \in [q^n]^N$ such that $\Delta(C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}), \mathbf{y}) < \frac{Dd}{2}$. Further, if $\mathbf{y}''$ has e' errors and s' erasures (when compared with $C_{\text{out}} \circ C_{\text{in}}(\mathbf{m})$), then*

$$\mathbb{E}[2e' + s'] < D.$$

Note that if $2e' + s' < D$, then by Theorem 14.2.2, Algorithm 14.3.1 will output $\mathbf{m}$. The lemma above says that in expectation, this is indeed the case.

Proof of Lemma 14.3.1. For every $1 \leq i \leq N$, define $e_i = \Delta(y_i, c_i)$. Note that this implies that

$$\sum_{i=1}^N e_i < \frac{Dd}{2}. \quad (14.2)$$

Next for every $1 \leq i \leq N$, we define two indicator variables:

$$X_i^? = \mathbb{1}_{y''_i = ?},$$

and

$$X_i^e = \mathbb{1}_{C_{\text{in}}(y''_i) \neq c_i \text{ and } y''_i \neq ?}.$$

We claim that we are done if we can show that for every $1 \leq i \leq N$:

$$\mathbb{E}[2X_i^e + X_i^?] \leq \frac{2e_i}{d}. \quad (14.3)$$

Indeed, by definition we have: $e' = \sum_i X_i^e$ and $s' = \sum_i X_i^?$. Further, by the linearity of expectation (Proposition 3.1.4), we get

$$\mathbb{E}[2e' + s'] \leq \frac{2}{d} \sum_i e_i < D,$$

where the inequality follows from (14.2).

To complete the proof, we will prove (14.3) by a case analysis. Towards this end, fix an arbitrary $1 \leq i \leq N$.

Case 1: ($c_i = y'_i$) First, we note that if $y''_i \neq ?$ then since $c_i = y'_i$, we have $X_i^e = 0$. This along with the fact that $\Pr[y''_i = ?] = \frac{2w_i}{d}$ implies

$$\mathbb{E}[X_i^?] = \Pr[X_i^? = 1] = \frac{2w_i}{d},$$

and

$$\mathbb{E}[X_i^e] = \Pr[X_i^e = 1] = 0.$$

Further, by definition we have

$$w_i = \min\left(\Delta(y'_i, y_i), \frac{d}{2}\right) \leq \Delta(y'_i, y_i) = \Delta(c_i, y_i) = e_i.$$

The three relations above prove (14.3) for this case.

Case 2: ($c_i \neq y'_i$) As in the previous case, we still have

$$\mathbb{E}[X_i^?] = \frac{2w_i}{d}.$$

Now in this case, if an erasure is not declared at position i , then $X_i^e = 1$. Thus, we have

$$\mathbb{E}[X_i^e] = \Pr[X_i^e = 1] = 1 - \frac{2w_i}{d}.$$

Next, we claim that as $c_i \neq y'_i$,

$$e_i + w_i \geq d, \quad (14.4)$$

which implies

$$\mathbb{E}[2X_i^e + X_i^?] = 2 - \frac{2w_i}{d} \leq \frac{2e_i}{d},$$

as desired.

To complete the proof, we show (14.4) via yet another case analysis.

Case 2.1: ($w_i = \Delta(y'_i, y_i) < d/2$) By definition of e_i , we have

$$e_i + w_i = \Delta(y_i, c_i) + \Delta(y'_i, y_i) \geq \Delta(c_i, y'_i) \geq d,$$

where the first inequality follows from the triangle inequality and the second inequality follows from the fact that C_{in} has distance d .

Case 2.2: ($w_i = \frac{d}{2} \leq \Delta(y'_i, y_i)$) As y'_i is obtained from MLD, we have

$$\Delta(y'_i, y_i) \leq \Delta(c_i, y_i).$$

This along with the assumption on $\Delta(y'_i, y_i)$, we get

$$e_i = \Delta(c_i, y_i) \geq \Delta(y'_i, y_i) \geq \frac{d}{2}.$$

This in turn implies that

$$e_i + w_i \geq d,$$

as desired. □

14.3.2 GMD Algorithm- II

Note that Step 4 in Algorithm 14.3.1 uses “fresh” randomness for each i . Next we look at another randomized version of the GMD algorithm that uses the *same* randomness for every i . In particular, consider Algorithm 14.3.2.

Algorithm 14.3.2 Generalized Minimum Decoder (ver 2)

INPUT: Received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$

OUTPUT: Message $\mathbf{m}' \in [q^k]^K$

- 1: Pick $\theta \in [0, 1]$ uniformly at random.
 - 2: FOR $1 \leq i \leq N$ DO
 - 3: $y'_i \leftarrow D_{C_{\text{in}}}(y_i)$.
 - 4: $w_i \leftarrow \min\left(\Delta(y'_i, y_i), \frac{d}{2}\right)$.
 - 5: If $\theta < \frac{2w_i}{d}$, set $y''_i \leftarrow ?$, otherwise set $y''_i \leftarrow x$, where $y'_i = C_{\text{in}}(x)$.
 - 6: $\mathbf{m}' \leftarrow D_{C_{\text{out}}}(\mathbf{y}'')$, where $\mathbf{y}'' = (y''_1, \dots, y''_N)$.
 - 7: RETURN $\mathbf{m}'$
-

We note that in the proof of Lemma 14.3.1, we only use the randomness to show that

$$\Pr[y''_i = ?] = \frac{2w_i}{d}.$$

In Algorithm 14.3.2, we note that

$$\Pr[y''_i = ?] = \Pr\left[\theta \in \left[0, \frac{2w_i}{d}\right)\right] = \frac{2w_i}{d},$$

as before (the last equality follows from our choice of θ). One can verify that the proof of Lemma 14.3.1 can be used to show the following lemma:

Lemma 14.3.2. *Let $\mathbf{y}$ be a received word such that there exists a codeword $C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}) = (c_1, \dots, c_N) \in [q^n]^N$ such that $\Delta(C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}), \mathbf{y}) < \frac{Dd}{2}$. Further, if $\mathbf{y}''$ has e' errors and s' erasures (when compared with $C_{\text{out}} \circ C_{\text{in}}(\mathbf{m})$), then*

$$\mathbb{E}_{\theta} [2e' + s'] < D.$$

Next, we will see that Algorithm 14.3.2 can be easily “derandomized.”

14.3.3 Derandomized GMD algorithm

Lemma 14.3.2 along with the probabilistic method shows that there exists a value $\theta^* \in [0, 1]$ such that Algorithm 14.3.2 works correctly even if we fix θ to be θ^* in Step 1. Obviously we can obtain such a θ^* by doing an exhaustive search for θ . Unfortunately, there are uncountable choices of θ because $\theta \in [0, 1]$. However, this problem can be taken care of by the following discretization trick.

Define $Q = \{0, 1\} \cup \{\frac{2w_1}{d}, \dots, \frac{2w_N}{d}\}$. Then because for each i , $w_i = \min(\Delta(y'_i, y_i), d/2)$, we have

$$Q = \{0, 1\} \cup \{q_1, \dots, q_m\}$$

where $q_1 < q_2 < \dots < q_m$ for some $m \leq \lfloor \frac{d}{2} \rfloor$. Notice that for every $\theta \in [q_i, q_{i+1})$, just before Step 6, Algorithm 14.3.2 computes the same $\mathbf{y}''$. (See Figure 14.2 for an illustration as to why this is the case.)

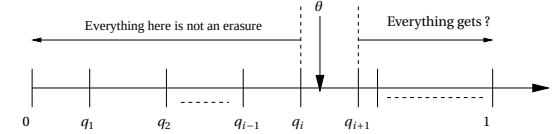


Figure 14.2: All values of $\theta \in [q_i, q_{i+1})$ lead to the same outcome

Thus, we need to cycle through all possible values of $\theta \in Q$, leading to Algorithm 14.3.3.

Algorithm 14.3.3 Deterministic Generalized Minimum Decoder'

INPUT: Received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$

OUTPUT: Message $\mathbf{m}' \in [q^k]^K$

- 1: FOR $1 \leq i \leq N$ DO
 - 2: $y'_i \leftarrow D_{C_{\text{in}}}(y_i)$.
 - 3: $w_i \leftarrow \min\left(\Delta(y'_i, y_i), \frac{d}{2}\right)$.
 - 4: $Q \leftarrow \{\frac{2w_1}{d}, \dots, \frac{2w_N}{d}\} \cup \{0, 1\}$.
 - 5: FOR $\theta \in Q$ DO
 - 6: FOR $1 \leq i \leq N$ DO
 - 7: If $\theta < \frac{2w_i}{d}$, set $y''_i \leftarrow ?$, otherwise set $y''_i \leftarrow x$, where $y'_i = C_{\text{in}}(x)$.
 - 8: $\mathbf{m}'_{\theta} \leftarrow D_{C_{\text{out}}}(\mathbf{y}'')$, where $\mathbf{y}'' = (y''_1, \dots, y''_N)$.
 - 9: RETURN $\mathbf{m}'_{\theta^*}$ for $\theta^* = \arg \min_{\theta \in Q} \Delta(C_{\text{out}} \circ C_{\text{in}}(\mathbf{m}'_{\theta}), \mathbf{y})$
-

Note that Algorithm 14.3.3 is Algorithm 14.3.2 repeated $|Q|$ times. Since $|Q|$ is $O(n)$, this implies that Algorithm 14.3.3 runs in polynomial time. This along with Theorem 10.2.1 implies that

Theorem 14.3.3. *For every constant rate, there exists an explicit linear binary code on the Zyablov bound. Further, the code can be decoded up to half of the Zyablov bound in polynomial time.*

Note that the above answers Question 10.4.1 in the affirmative.

14.4 Bibliographic Notes

Forney in 1966 designed the Generalized Minimum Distance (or GMD) decoding [53].

Chapter 15

Efficiently Achieving the Capacity of the BSC_p

Table 15.1 summarizes the main results we have seen so far for binary codes.

	Shannon	Hamming	
		Unique Decoding	List Decoding
Capacity	$1 - H(p)$ (Thm 6.3.1)	$\geq$ GV (Thm 4.2.1) $\leq$ MRRW (Sec 8.2)	$1 - H(p)$ (Thm 7.4.1)
Explicit Codes	?	Zyablov bound (Thm 10.2.1)	?
Efficient Algorithms	?	$\frac{1}{2}$, Zyablov bound (Thm 14.3.3)	?

Table 15.1: An overview of the results seen so far

In this chapter, we will tackle the open questions in the first column of Table 15.1. Recall that there exist linear codes of rate $1 - H(p) - \epsilon$ such that decoding error probability is not more than $2^{-\delta n}$, $\delta = \Theta(\epsilon^2)$ on the BSC_p (Theorem 6.3.1 and Exercise 6.3). This led to Question 6.3.1, which asks if we can achieve the BSC_p capacity with explicit codes and efficient decoding algorithms.

15.1 Achieving capacity of BSC_p

We will answer Question 6.3.1 in the affirmative by using concatenated codes. The main intuition in using concatenated codes is the following. As in the case of construction of codes on the Zyablov bound, we will pick the inner code to have the property that we are after: i.e. a code that achieves the BSC_p capacity. (We will again exploit the fact that since the block length of the inner code is small, we can construct such a code in a brute-force manner.) However, unlike the case of the Zyablov bound construction, we have not yet seen an explicit *outer* code that is optimal for correcting over a stochastic channel (over say the qSC_p channel). The fact that the BSC_p noise is memory-less can be exploited to pick the outer code that can correct from some small but constant fraction of *worst-case* errors.

Before delving into the details, we present the main ideas. We will use an outer code C_{out} that has rate close to 1 and can correct from some fixed constant (say γ) fraction of worst-case errors. We pick an inner code C_{in} that achieves the BSC_p capacity with parameters as guaranteed by Theorem 6.3.1. Since the outer code has rate almost 1, the concatenated code can be made to have the required rate (since the final rate is the product of the rates of C_{out} and C_{in}). For decoding, we use the natural decoding algorithm for concatenated codes from Algorithm 14.1.1. Assume that each of the inner decoders has a decoding error probability of (about) γ . Then the intermediate received word $\mathbf{y}'$ has an expected γ fraction of errors (with respect to the outer codeword of the transmitted message), though we might not have control over where the errors occur. However, we picked C_{out} so that it can correct up to γ fraction of worst-case errors. This shows that everything works in expectation. To make everything work with high probability (i.e. achieve exponentially small overall decoding error probability), we make use of the fact that since the noise in BSC_p is independent. Therefore, the decoding error probabilities of each of the inner decodings are independent. Thus, by the Chernoff bound (Theorem 3.1.10), with all but an exponentially small probability $\mathbf{y}'$ has $\Theta(\gamma)$ fraction of errors, which we correct with the worst-case error decoder for C_{out} . See Figure 15.1 for an illustration of the main ideas. Next, we present the details.

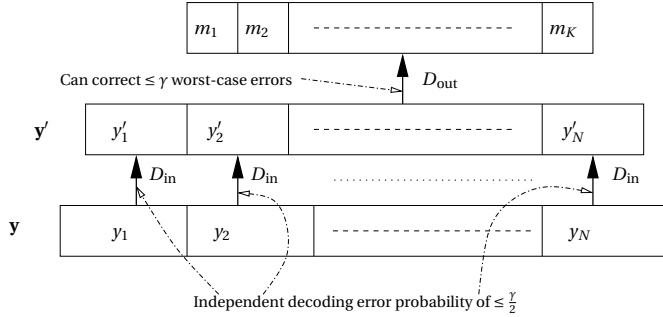


Figure 15.1: Efficiently achieving capacity of BSC_p .

We answer Question 6.3.1 in the affirmative by using a concatenated code $C_{\text{out}} \circ C_{\text{in}}$ with the following properties (where $\gamma > 0$ is a parameter that depends only on ε and will be fixed later on):

- (i) C_{out} : The outer code is a linear $[N, K]_{2^k}$ code with rate $R \geq 1 - \frac{\varepsilon}{2}$, where $k = O(\log N)$. Further, the outer code has a unique decoding algorithm D_{out} that can correct at most γ fraction of worst-case errors in time $T_{\text{out}}(N)$.
- (ii) C_{in} : The inner code is a linear binary $[n, k]_2$ code with a rate of $r \geq 1 - H(p) - \varepsilon/2$. Further, there is a decoding algorithm D_{in} (which returns the transmitted codeword) that runs in time $T_{\text{in}}(k)$ and has decoding error probability no more than $\frac{\gamma}{2}$ over BSC_p .

Table 15.2 summarizes the different parameters of C_{out} and C_{in} .

	Dimension	Block length	q	Rate	Decoder	Decoding time	Decoding guarantee
C_{out}	K	N	2^k	$1 - \frac{\varepsilon}{2}$	D_{out}	$T_{\text{out}}(N)$	$\leq \gamma$ fraction of worst-case errors
C_{in}	$k \leq O(\log N)$	n	2	$1 - H(p) - \frac{\varepsilon}{2}$	D_{in}	$T_{\text{in}}(k)$	$\leq \frac{\gamma}{2}$ decoding error probability over BSC_p

Table 15.2: Summary of properties of C_{out} and C_{in}

Suppose $C^* = C_{\text{out}} \circ C_{\text{in}}$. Then, it can be checked that

$$R(C^*) = R \cdot r \geq \left(1 - \frac{\varepsilon}{2}\right) \cdot \left(1 - H(p) - \frac{\varepsilon}{2}\right) \geq 1 - H(p) - \varepsilon,$$

as desired.

For the rest of the chapter, we will assume that p is an absolute constant. Note that this implies that $k = \Theta(n)$ and thus, we will use k and n interchangeably in our asymptotic bounds. Finally, we will use $\mathcal{N} = nN$ to denote the block length of C^* .

The decoding algorithm for C^* that we will use is Algorithm 14.1.1, which for concreteness we reproduce as Algorithm 15.1.1.

Algorithm 15.1.1 Decoder for efficiently achieving BSC_p capacity

INPUT: Received word $\mathbf{y} = (y_1, \dots, y_N) \in [q^n]^N$

OUTPUT: Message $\mathbf{m}' \in [q^k]^K$

1: $\mathbf{y}' \leftarrow (y'_1, \dots, y'_N) \in [q^k]^N$ where

$$C_{\text{in}}(y'_i) = D_{\text{in}}(y_i) \quad 1 \leq i \leq N.$$

2: $\mathbf{m}' \leftarrow D_{\text{out}}(\mathbf{y}')$

3: RETURN $\mathbf{m}'$

Note that encoding C^* takes time

$$O(N^2 k^2) + O(Nkn) \leq O(N^2 n^2) = O(\mathcal{N}^2),$$

as both the outer and inner codes are linear¹. Further, the decoding by Algorithm 15.1.1 takes time

$$N \cdot T_{\text{in}}(k) + T_{\text{out}}(N) \leq \text{poly}(N),$$

¹Note that encoding the outer code takes $O(N^2)$ operations over $\mathbb{F}_{q^k}$. The term $O(N^2 k^2)$ then follows from the fact that each operation over $\mathbb{F}_{q^k}$ can be implemented with $O(k^2)$ operations over $\mathbb{F}_q$.

where the inequality is true as long as

$$T_{\text{out}}(N) = N^{O(1)} \text{ and } T_{\text{in}}(k) = 2^{O(k)}. \quad (15.1)$$

Next, we will show that decoding via Algorithm 15.1.1 leads to an exponentially small decoding error probability over BSC_p . Further, we will use constructions that we have already seen in this book to instantiate C_{out} and C_{in} with the required properties.

15.2 Decoding Error Probability

We begin by analyzing Algorithm 15.1.1.

By the properties of D_{in} , for any fixed i , there is an error at y'_i with probability $\leq \frac{\gamma}{2}$. Each such error is independent, since errors in BSC_p themselves are independent by definition. Because of this, and by linearity of expectation, the expected number of errors in $\mathbf{y}'$ is $\leq \frac{\gamma N}{2}$.

Considering these two facts we conclude that by the (multiplicative) Chernoff bound (Theorem 3.1.10), the probability that the total number of errors will be more than γN is at most $e^{-\frac{\gamma N}{6}}$. Since the decoder D_{out} fails only when there are more than γN errors, this is also the final decoding error probability. Expressed in asymptotic terms, the error probability is $2^{-\Omega(\frac{\gamma N}{n})}$.

15.3 The Inner Code

We find C_{in} with the required properties by an exhaustive search among linear codes of dimension k with block length n that achieve the BSC_p capacity by Shannon's theorem (Theorem 6.3.1). Recall that for such codes with rate $1 - H(p) - \frac{\epsilon}{2}$, the MLD has a decoding error probability of $2^{-\Theta(\epsilon^2 n)}$ (Exercise 6.3). Thus, if k is at least $\Omega\left(\frac{\log(\frac{1}{\gamma})}{\epsilon^2}\right)$, Exercise 6.3 implies the existence of a linear code with decoding error probability at most $\frac{\gamma}{2}$ (which is what we need). Thus, with the restriction on k from the outer code, we have the following restriction on k :

$$\Omega\left(\frac{\log(\frac{1}{\gamma})}{\epsilon^2}\right) \leq k \leq O(\log N).$$

However, note that since the proof of Theorem 6.3.1 uses MLD on the inner code and Algorithm 1.4.1 is the only known implementation of MLD, we have $T_{\text{in}} = 2^{O(k)}$ (which is what we needed in (15.1)). The construction time is even worse. There are $2^{O(kn)}$ generator matrices; for each of these, we must check the error rate for each of 2^k possible transmitted codewords, and for each codeword, computing the decoding error probability requires time $2^{O(n)}$.² Thus, the construction time for C_{in} is $2^{O(n^2)}$.

²To see why the latter claim is true, note that there are 2^n possible received words and given any one of these received words, one can determine (i) if the MLD produces a decoding error in time $2^{O(k)}$ and (ii) the probability that the received word can be realized, given the transmitted codeword in polynomial time.

15.4 The Outer Code

We need an outer code with the required properties. There are several ways to do this.

One option is to set C_{out} to be a Reed-Solomon code over $\mathbb{F}_{2^k}$ with $k = \Theta(\log N)$ and rate $1 - \frac{\epsilon}{4}$. Then the decoding algorithm D_{out} , could be the error decoding algorithm from Theorem 14.2.2. Note that for this D_{out} we can set $\gamma = \frac{\epsilon}{4}$ and the decoding time is $T_{\text{out}}(N) = O(N^3)$.

Till now everything looks on track. However, the problem is the construction time for C_{in} , which as we saw earlier is $2^{O(n^2)}$. Our choice of n implies that the construction time is $2^{O(\log^2 N)} \leq N^{O(\log N)}$, which of course is not polynomial time. Thus, the trick is to find a C_{out} defined over a smaller alphabet (certainly no larger than $2^{O(\sqrt{\log N})}$). This is what we do next.

15.4.1 Using a binary code as the outer code

The main observation is that we can also use an outer code which is some explicit binary linear code (call it C') that lies on the Zyablov bound and can be corrected from errors up to half its design distance³. We have seen that such a code can be constructed in polynomial time (Theorem 14.3.3).

Note that even though C' is a binary code, we can think of C' as a code over $\mathbb{F}_{2^k}$ in simple way: every k consecutive bits are considered to be an element in $\mathbb{F}_{2^k}$ (say via a linear map). Note that the rate of the code does not change. Further, any decoder for C' that corrects bit errors can be used to correct errors over $\mathbb{F}_{2^k}$. In particular, if the algorithm can correct β fraction of bit errors, then it can correct the same fraction of errors over $\mathbb{F}_{2^k}$. To see this, think of the received word as $\mathbf{y} \in (\mathbb{F}_{2^k})^{N'/k}$, where N' is the block length of C' (as a binary code), which is at a fractional Hamming distance at most ρ away from $\mathbf{c} \in (\mathbb{F}_{2^k})^{N'/k}$. Here, $\mathbf{c}$ is what one gets by "folding" consecutive k bits into one symbol in some codeword $\mathbf{c}' \in C'$. Now consider $\mathbf{y}' \in \mathbb{F}_{2^{N'}}$, which is just "unfolded" version of $\mathbf{y}$. Now note that each symbol in $\mathbf{y}$ that is in error (w.r.t. $\mathbf{c}$) leads to at most k bit errors in $\mathbf{y}'$ (w.r.t. $\mathbf{c}'$). Thus, in the unfolded version, the total number of errors is at most

$$k \cdot \rho \cdot \frac{N'}{k} = \rho \cdot N'.$$

(See Figure 15.2 for an example for the case when $k = 2$.) Thus to decode $\mathbf{y}$, one can just "unfold" $\mathbf{y}$ to $\mathbf{y}'$ and use the decoding algorithm for C' (which can handle ρ fraction of errors) on $\mathbf{y}'$.

We will pick C_{out} to be C' when considered over $\mathbb{F}_{2^k}$, where we choose

$$k = \Theta\left(\frac{\log(\frac{1}{\gamma})}{\epsilon^2}\right).$$

Further, D_{out} is the GMD decoding algorithm (Algorithm 14.3.3) for C' .

Now, to complete the specification of C^* , we relate γ to ϵ . The Zyablov bound gives $\delta_{\text{out}} = (1 - R)H^{-1}(1 - r)$, where R and r are the rates of the outer and inner codes for C' . Now, we can

³Recall that the design distance of a concatenated code (where the outer code has distance D and the inner code has distance d) is dD .

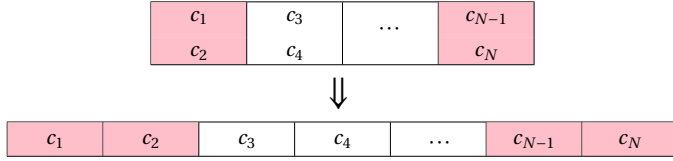


Figure 15.2: Error Correction cannot decrease during “folding.” The example has $k = 2$ and each pink cell implies an error.

set $1 - R = 2\sqrt{\gamma}$ (which implies that $R = 1 - 2\sqrt{\gamma}$) and $H^{-1}(1 - r) = \sqrt{\gamma}$, which implies that r is⁴ $1 - O\left(\sqrt{\gamma} \log \frac{1}{\gamma}\right)$. Since we picked D_{out} to be the GMD decoding algorithm, it can correct $\frac{\delta_{\text{out}}}{2} = \gamma$ fraction of errors in polynomial time, as desired.

The overall rate of C_{out} is simply $R \cdot r = (1 - 2\sqrt{\gamma}) \cdot \left(1 - O\left(\sqrt{\gamma} \log \frac{1}{\gamma}\right)\right)$. This simplifies to $1 - O\left(\sqrt{\gamma} \log \left(\frac{1}{\gamma}\right)\right)$. Recall that we need this to be at least $1 - \frac{\epsilon}{2}$. Thus, we would be done here if we can show that ϵ is $\Omega\left(\sqrt{\gamma} \log \frac{1}{\gamma}\right)$, which in turn follows by setting

$$\gamma = \epsilon^3.$$

15.4.2 Wrapping Up

We now recall the construction, encoding and decoding time complexity for C^* . The construction time for C_{in} is $2^{O(n^2)}$, which substituting for n , is $2^{O\left(\frac{1}{\epsilon^4} \log^2\left(\frac{1}{\epsilon}\right)\right)}$. The construction time for C_{out} , meanwhile, is only $\text{poly}(N)$. Thus, the overall construction time is $\text{poly}(\mathcal{N}) + 2^{O\left(\frac{1}{\epsilon^4} \log^2\left(\frac{1}{\epsilon}\right)\right)}$.

As we have seen in Section 15.1, the encoding time for this code is $O(\mathcal{N}^2)$, and the decoding time is $N^{O(1)} + N \cdot 2^{O(n)} = \text{poly}(\mathcal{N}) + \mathcal{N} \cdot 2^{O\left(\frac{1}{\epsilon^2} \log\left(\frac{1}{\epsilon}\right)\right)}$. Further, we have shown that the decoding error probability is exponentially small: $2^{-\Omega\left(\frac{\gamma \mathcal{N}}{n}\right)} = 2^{-\Omega\left(\frac{\epsilon^5}{\log(1/\epsilon)} \mathcal{N}\right)}$. Thus, we have proved the following result:⁵

Theorem 15.4.1. *For every constant p and $0 < \epsilon < 1 - H(p)$, there exists a linear code C^* of block length $\mathcal{N}$ and rate at least $1 - H(p) - \epsilon$, such that*

- (a) C^* can be constructed in time $\text{poly}(\mathcal{N}) + 2^{O(\epsilon^{-5})}$;
- (b) C^* can be encoded in time $O(\mathcal{N}^2)$; and
- (c) There exists a $\text{poly}(\mathcal{N}) + \mathcal{N} \cdot 2^{O(\epsilon^{-5})}$ time decoding algorithm that has an error probability of at most $2^{-\Omega(\epsilon^6 \mathcal{N})}$ over the BSC $_p$.

⁴Note that $r = 1 - H(\sqrt{\gamma}) = 1 + \sqrt{\gamma} \log \sqrt{\gamma} + (1 - \sqrt{\gamma}) \log(1 - \sqrt{\gamma})$. Noting that $\log(1 - \sqrt{\gamma}) = -\sqrt{\gamma} - \Theta(\gamma)$, we can deduce that $r = 1 - O(\sqrt{\gamma} \log(1/\gamma))$.

⁵Below we have used the fact that for any constant $a > 0$, $\epsilon^{-a} \log^{O(1)}(1/\epsilon)$ is $O(\epsilon^{-a-1})$.

Thus, we have answered in the affirmative Question 6.3.1, which was the central open question from Shannon's work. However, there is a still somewhat unsatisfactory aspect of the result above. In particular, the exponential dependence on $1/\epsilon$ in the decoding time complexity is not nice. This leads to the following question:

Question 15.4.1. *Can we bring the high dependence on ϵ down to $\text{poly}(\frac{1}{\epsilon})$ in the decoding time complexity?*

15.5 Discussion and Bibliographic Notes

Forney answered Question 6.3.1 in the affirmative by using concatenated codes. (As was mentioned earlier, this was Forney's motivation for inventing code concatenation: the implication for the rate vs. distance question was studied by Zyablov later on.)

We now discuss Question 15.4.1. For the binary erasure channel, the decoding time complexity can be brought down to $\mathcal{N} \cdot \text{poly}(\frac{1}{\epsilon})$ using LDPC codes, specifically a class known as Tornado codes developed by Luby et al. [109]. The question for binary symmetric channels, however, is still open. Recently there have been some exciting progress on this front by the construction of the so-called Polar codes.

We conclude by noting an improvement to Theorem 15.4.1. We begin with a theorem due to Spielman:

Theorem 15.5.1 ([146]). *For every small enough $\beta > 0$, there exists an explicit C_{out} of rate $\frac{1}{1+\beta}$ and block length N , which can correct $\Omega\left(\frac{\beta^2}{(\log \frac{1}{\beta})^2}\right)$ errors, and has $O(N)$ encoding and decoding.*

Clearly, in terms of time complexity, this is superior to the previous option in Section 15.4.1. Such codes are called “Expander codes.” One can essentially do the same calculations as in Section 15.4.1 with $\gamma = \Theta\left(\frac{\epsilon^2}{\log^2(1/\epsilon)}\right)$.⁶ However, we obtain an encoding and decoding time of $\mathcal{N} \cdot 2^{\text{poly}(\frac{1}{\epsilon})}$. Thus, even though we obtain an improvement in the time complexities as compared to Theorem 15.4.1, this does not answer Question 15.4.1.

⁶This is because we need $1/(1+\beta) = 1 - \epsilon/2$, which implies that $\beta = \Theta(\epsilon)$.

Chapter 16

Information Theory Strikes Back: Polar Codes

We begin by recalling Question 15.4.1, which we re-produce for the sake of completeness:

Question 16.0.1. *Can we get to within ϵ of capacity for BSC_p (i.e. rate $1 - H(p) - \epsilon$) via codes with block length and decoding times that are $\text{poly}(1/\epsilon)$?*

In this chapter we introduce *Polar codes*, a class of codes developed from purely information-theoretic insights. We then show how these codes lead to a resolution of Question 16.0.1, namely how to get arbitrarily close to capacity on the binary symmetric channel with block length, and decoding complexity growing polynomially in the inverse of the gap to capacity. This answers in the affirmative one of the most crucial questions in the Shannon setting.

This chapter is organized as follows. We define the precise question, after some simplification, in Section 16.1. In the same section, we discuss why the simplified question solves a much more general problem. We then switch the problem from an error-correction question to a *linear-compression* question in Section 16.2. This switch is very straightforward but very useful in providing insight into the working of Polar codes. In Section 16.3 we introduce the idea of polarization, which provides the essential insight to Polar codes. (We remark that this section onwards is based on notions from Information Theory. The reader unfamiliar with the theory should first consult Appendix E to get familiar with the basic concepts.) In Section 16.4 we then give a complete description of the Polar codes, and describe the encoding and decoding algorithms. In Section 16.5 we then describe the analysis of these codes. We remark that the only complex part of this chapter is this analysis and the construction and algorithms themselves are quite simple (and extremely elegant) modulo this analysis.

16.1 Achieving Gap to Capacity

The goal of this section is to present a simple question that formalizes what it means to achieve capacity with polynomial convergence, and to explain why this is the right question (and how answering this positively leads to much more powerful results by standard methods).

Recall that for $p \in [0, 1/2]$ the BSC_p is the channel that takes as input a sequence of bits $\mathbf{X} = (X_1, \dots, X_n)$ and outputs the sequence $\mathbf{Y} = (Y_1, \dots, Y_n)$ where for each i , $X_i = Y_i$ with probability $1 - p$ and $X_i \neq Y_i$ with probability p ; and this happens independently for each $i \in \{1, \dots, n\}$. We use the notation $\mathbf{Z} = (Z_1, \dots, Z_n) \in \text{Bern}(p)^n$ to denote the error pattern, so that $\mathbf{Y} = \mathbf{X} + \mathbf{Z}$.

Our target for this chapter is to prove the following theorem:

Theorem 16.1.1. *For every $p \in [0, 1/2]$ there is a polynomial¹ $n_0(\cdot)$ such that for every $\varepsilon > 0$ there exist k, n and an encoder $E: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ and decoder $D: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$ satisfying the following conditions:*

Length and Rate *The codes are short, and the rate is close to capacity, specifically, $1/\varepsilon \leq n \leq n_0(1/\varepsilon)$ and $k \geq (1 - H(p) - \varepsilon) \cdot n$.*

Running times *The encoder and decoder run in time $O(n \log n)$ (where the $O(\cdot)$ notation hides a universal constant independent of p and ε).*

Failure Probability *The probability of incorrect decoding is at most ε . Specifically, for every $\mathbf{m} \in \mathbb{F}_2^k$,*

$$\Pr_{\mathbf{Z} \in \text{Bern}(p)^n} [\mathbf{m} \neq D(E(\mathbf{m}) + \mathbf{Z})] \leq \varepsilon.$$

Theorem 16.1.1 is not the ultimate theorem we may want for dealing with the binary symmetric channel. For starters, it does not guarantee codes of all lengths, but rather only of one fixed length n for any fixed choice of ε . Next, Theorem 16.1.1 only guarantees a small probability of decoding failure, but not one that say goes to zero exponentially fast in the length of the code. The strength of the theorem is (1) its simplicity - it only takes one parameter ε and delivers a good code with rate ε close to capacity and (2) Algorithmic efficiency: the running time of the encoder and decoder is a polynomial in $1/\varepsilon$. It turns out both the weaknesses can be addressed by applying the idea of concatenation of codes (Chapter 10) while preserving the strength. We present the resulting theorem, leaving the proof of this theorem from Theorem 16.1.1 as an exercise. (See Exercise 16.1.)

Theorem 16.1.2. *There exists polynomially growing functions $n_0: [0, 1] \rightarrow \mathbb{Z}^+$ and $T: \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ such that for all $p \in [0, 1]$, $\varepsilon > 0$ there exists $\delta > 0$, a function $k: \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ and an ensemble of function $E = \{E_n\}_n$ and $D = \{D_n\}_n$ such that for all $n \in \mathbb{Z}^+$ with $n \geq n_0(1/\varepsilon)$ the following hold:*

1. *The codes are ε -close to capacity: Specifically, $k = k(n) \geq (1 - H(p) - \varepsilon)n$, $E_n: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ and $D_n: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$.*

¹I.e. $n_0(x) = x^c$ for some constant c .

2. *The codes correct p -fraction of errors with all but exponentially small probability: Specifically*

$$\Pr_{\mathbf{Z} \sim \text{Bern}(p)^n, \mathbf{X} \sim U(\mathbb{F}_2^k)} [D_n(E_n(\mathbf{X}) + \mathbf{Z}) \neq \mathbf{X}] \leq \exp(-\delta n).$$

3. *Encoding and Decoding are efficient: Specifically E_n and D_n run in time at most $T(n/\varepsilon)$.*

16.2 Reduction to Linear Compression

In this section we change our problem from that of coding for error-correction to compressing a vector of independent Bernoulli random variables i.e., the error-pattern. (Recall that we encountered compression in Exercise 6.10). We show that if the compression is linear and the decompression algorithm is efficient, then this turns into a linear code with efficient decoding (this is the converse of what we saw in Exercise 6.11). By virtue of being linear the code is also polynomial time encodable, given the generator matrix. We explain this simple connection below.

For $n \geq m$, we say that a pair $(\mathbf{H}, D)$ where $\mathbf{H} \in \mathbb{F}_2^{n \times m}$, and $D: \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$ forms an τ -error linear compression scheme for $\text{Bern}(p)^n$ if $\mathbf{H}$ has rank m and

$$\Pr_{\mathbf{Z} \sim \text{Bern}(p)^n} [D(\mathbf{Z} \cdot \mathbf{H}) \neq \mathbf{Z}] \leq \tau.$$

We refer to the ratio $\frac{m}{n}$ as the (compression) rate of the scheme (recall Exercise 6.10).

Proposition 16.2.1. *Let $(\mathbf{H}, D)$ be a τ -error linear compression scheme for $\text{Bern}(p)^n$ with $\mathbf{H} \in \mathbb{F}_2^{n \times m}$. Let $k = n - m$ and let $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ and $\mathbf{G}^* : \mathbb{F}_2^{n \times k}$ be full-rank matrices such that $\mathbf{G} \cdot \mathbf{H} = \mathbf{0}$ and $\mathbf{G} \cdot \mathbf{G}^* = \mathbf{I}_k$ (the $k \times k$ identity matrix). Then the encoder $E: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ given by*

$$E(\mathbf{X}) = \mathbf{X} \cdot \mathbf{G}$$

and the decoder $D': \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$ given by

$$D'(\mathbf{Y}) = (\mathbf{Y} - D(\mathbf{Y} \cdot \mathbf{H})) \cdot \mathbf{G}^*$$

satisfy for every $\mathbf{m} \in \mathbb{F}_2^k$:

$$\Pr_{\mathbf{Z} \in \text{Bern}(p)^n} [\mathbf{m} \neq D'(E(\mathbf{m}) + \mathbf{Z})] \leq \tau.$$

Remark 16.2.2. 1. Recall that straightforward linear algebra implies the existence of matrices $\mathbf{G}$ and $\mathbf{G}^*$ above (see Exercise 16.2).

2. Note that the complexity of encoding is simply the complexity of multiplying a vector by $\mathbf{G}$. The complexity of decoding is bounded by the complexity of decompression plus the complexity of multiplying by $\mathbf{G}^*$. In particular, if all these operations can be carried out in $O(n \log n)$ time then computing E and D' takes $O(n \log n)$ time as well.

3. Note that the above proves the converse of Exercise 6.11. These two results show that (at least for linear codes), channel and source coding are equivalent.

Proof. Suppose $D(\mathbf{Z} \cdot \mathbf{H}) = \mathbf{Z}$. Then we claim that if $\mathbf{Z}$ is the error pattern, then decoding is successful. To see this, note that

$$D'(E(\mathbf{m}) + \mathbf{Z}) = (E(\mathbf{m}) + \mathbf{Z} - D((E(\mathbf{m}) + \mathbf{Z}) \cdot \mathbf{H})) \cdot \mathbf{G}^* \quad (16.1)$$

$$= (E(\mathbf{m}) + \mathbf{Z} - D(\mathbf{Z} \cdot \mathbf{H})) \cdot \mathbf{G}^* \quad (16.2)$$

$$= (E(\mathbf{m}) + \mathbf{Z} - \mathbf{Z}) \cdot \mathbf{G}^* \quad (16.3)$$

$$= E(\mathbf{m}) \cdot \mathbf{G}^* \quad (16.4)$$

In the above (16.1) follows by definition of D' , (16.2) follows from the fact that $E(\mathbf{m}) \cdot \mathbf{H} = \mathbf{m} \cdot \mathbf{G} \cdot \mathbf{H} = \mathbf{0}$, (16.3) follows by assumption that $D(\mathbf{Z} \cdot \mathbf{H}) = \mathbf{Z}$ and (16.4) follows since $E(\mathbf{m}) = \mathbf{m} \cdot \mathbf{G}$ and $\mathbf{G} \cdot \mathbf{G}^* = \mathbf{I}$. Thus, the probability of a decoding failure is at most the probability of decompression failure, which by definition is at most τ , as desired. $\square$

Thus, our updated quest from now on will be to

Question 16.2.1. Design a linear compression scheme for $\text{Bern}(p)^n$ of rate at most $H(p) + \varepsilon$.

See Exercise 16.4 on how one can answer Question 16.2.1 with a non-linear compression scheme.

In what follows we will introduce the polarization phenomenon that will lead us to such a compression scheme.

16.3 The Polarization Phenomenon

16.3.1 Information Theory Review

The only information theoretic notions that we need in this chapter are that of Entropy (see Definition E.1.2) and Conditional Entropy (Definition E.2.2). We use the notations $H(X)$ to denote the entropy of a variable X and $H(X|Y)$ to be the entropy of X conditioned on Y . The main properties of these notions we will use are the chain rule (see Theorem E.2.4):

$$H(X, Y) = H(Y) + H(X|Y),$$

and the fact that conditioning does not increase entropy (see Lemma E.2.6):

$$H(X|Y) \leq H(X).$$

We also use the basic fact that the uniform distribution maximizes entropy (see Lemma E.1.3) and hence,

$$H(X) \leq \log |\Omega|$$

if Ω denotes the support of X . A final fact that will be useful to keep in mind as we develop the polar codes is that variables with low entropy are essentially determined, and variables with low conditional entropy are predictable. We formalize this (with very loose bounds) below.

Proposition 16.3.1. Let $\alpha \geq 0$.

1. Let X be a random variable with $H(X) \leq \alpha$. Then there exists an x such that $\Pr_X[X \neq x] \leq \alpha$.
2. Let (X, Y) be jointly distributed variables with $H(X|Y) \leq \alpha$. Then the function

$$A(y) = \operatorname{argmax}_x \{\Pr[X = x | Y = y]\}$$

satisfies

$$\Pr_{(X,Y)}[X \neq A(Y)] \leq \alpha.$$

We defer the proof to Section 16.6.1.

16.3.2 Polarized matrices and decompression

We now return to the task of designing a matrix $\mathbf{H}$ (and corresponding matrices $\mathbf{G}$ and $\mathbf{G}^*$) such that the map $\mathbf{Z} \mapsto \mathbf{Z} \cdot \mathbf{H}$ is a good compression scheme. While we are seeking an $m \times n$ rectangular matrices with some extra properties, in this section we will convert the question of constructing $\mathbf{H}$ into a question about square $n \times n$ invertible matrices.

For an invertible matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$, we consider its effect on $\mathbf{Z} = (Z_1, \dots, Z_n) \sim \text{Bern}(p)^n$. Let $\mathbf{W} = (W_1, \dots, W_n)$ be given by $\mathbf{W} = \mathbf{Z} \cdot \mathbf{P}$. Now consider the challenge of predicting the W_i 's as they arrive online. Thus, when attempting to predict W_i , we get to see

$$\mathbf{W}_{<i} \triangleq (W_1, \dots, W_{i-1}),$$

which we can use to predict W_i . For arbitrary matrices, e.g. the identity matrix, seeing $\mathbf{W}_{<i}$ gives us no advantage on predicting W_i . A matrix will be considered polarized if, for an appropriately large fraction of i 's, W_i is *highly predictable* from $\mathbf{W}_{<i}$.

To formalize the notion of highly predictable, we turn to information theory and simply require $H(W_i | \mathbf{W}_{<i}) \leq \tau$ for some very small parameter τ of our choice.² With this definition, how many i 's should be very predictable? Let

$$S = S_\tau = \{i \in [n] \mid H(W_i | \mathbf{W}_{<i}) \geq \tau\}$$

denote the set of unpredictable bits. An entropy calculation will tell us how small we can hope S to be. Since $\mathbf{P}$ is invertible, we have (see Exercise 16.5):

$$H(\mathbf{W}) = H(\mathbf{Z}) = n \cdot H(p). \quad (16.5)$$

²Eventually we will set $\tau = o(1/n)$.

However, by the chain rule we have

$$\begin{aligned} H(\mathbf{W}) &= \sum_{i=1}^n H(W_i | \mathbf{W}_{<i}) \\ &= \sum_{i \in S} H(W_i | \mathbf{W}_{<i}) + \sum_{i \notin S} H(W_i | \mathbf{W}_{<i}) \\ &\leq \sum_{i \in S} H(W_i | \mathbf{W}_{<i}) + (n - |S|)\tau \end{aligned} \quad (16.6)$$

$$\begin{aligned} &\leq \sum_{i \in S} H(W_i | \mathbf{W}_{<i}) + n\tau \\ &\leq \sum_{i \in S} H(W_i) + n\tau \end{aligned} \quad (16.7)$$

$$\leq |S| + n\tau. \quad (16.8)$$

In the above, (16.6) follows by using definition of S , (16.7) follows from the fact that conditioning does not increase entropy and (16.8) follows from the fact that uniformity maximizes entropy and so $H(W_i) \leq 1$. We thus conclude that, since τ is small, $|S| \geq H(p) \cdot n - n\tau \approx H(p) \cdot n$. Thus, the smallest that the set S can be is $H(p) \cdot n$. We will allow an εn additive slack to get the following definition:

Definition 16.3.2 (Polarizing matrix, unpredictable columns). *We say that an invertible matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$ is (ε, τ) -polarizing for $\text{Bern}(p)^n$ if for $\mathbf{W} = \mathbf{Z} \cdot \mathbf{P}$ (where $\mathbf{Z} \in \text{Bern}(p)^n$) and*

$$S = S_\tau = \{i \in [n] \mid H(W_i | \mathbf{W}_{<i}) \geq \tau\}$$

we have $|S| \leq (H(p) + \varepsilon)n$. We refer to the set S as the set of unpredictable columns of $\mathbf{P}$ (and $\{W_i\}_{i \in S}$ as the unpredictable bits of $\mathbf{W}$).

We next show how to get a compression scheme from a polarizing matrix (without necessarily having an efficient decompressor). The idea is simple: the compressor simply outputs the “unpredictable” bits of $\mathbf{W}$. Let $\mathbf{W}_S = (W_i)_{i \in S}$, the compression of $\mathbf{Z}$ is simply $(\mathbf{Z} \cdot \mathbf{P})_S$. For the sake of completeness, we record this in Algorithm 16.3.1.

Algorithm 16.3.1 POLAR COMPRESSOR($\mathbf{Z}, S$)

INPUT: String $\mathbf{Z} \in \mathbb{F}_2^n$ and subset $S \subseteq [n]$

OUTPUT: Compressed string $\mathbf{W} \in \mathbb{F}_2^{|S|}$ ▷ Assumes a polarizing matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$

1: RETURN $(\mathbf{Z} \cdot \mathbf{P})_S$

Equivalently, if we let $\mathbf{P}_S$ denote the $n \times |S|$ matrix whose columns correspond to indices in S , then the compression of $\mathbf{Z}$ is $\mathbf{Z} \cdot \mathbf{P}_S$. Thus, $\mathbf{P}_S$ will be the matrix $\mathbf{H}$ we are seeking. Before turning to the decompression, let us also specify $\mathbf{G}$ and $\mathbf{G}^*$ for the linear compression scheme corresponding to Algorithm 16.3.1. Indeed, Exercise 16.3 shows that $\mathbf{G} = (\mathbf{P}^{-1})_{\bar{S}}$ (where $\bar{S}$ is the complement of set S) and $\mathbf{G}^* = \mathbf{P}_{\bar{S}}$. In particular, the complexity of multiplying an arbitrary

vector by $\mathbf{P}$ and $\mathbf{P}^{-1}$ dominate the cost of the matrix multiplications needed for the encoding and decoding.

We finally turn to the task of describing the decompressor corresponding to compression with a polarizing matrix $\mathbf{P}$ with unpredictable columns S . The method is a simple iterative one, based on the predictor from Proposition 16.3.1, and is presented in Algorithm 16.3.2.

Algorithm 16.3.2 Successive Cancellation Decompressor SCD($\mathbf{W}, \mathbf{P}, S$)

INPUT: $S \subseteq [n]$, $\mathbf{W} \in \mathbb{F}_2^S$ and $\mathbf{P} \in \mathbb{F}_2^{n \times n}$

OUTPUT: $\tilde{\mathbf{Z}} \in \mathbb{F}_2^n$ such that $(\tilde{\mathbf{Z}} \cdot \mathbf{P})_S = \mathbf{W}$

PERFORMANCE PARAMETER: ³ $\Pr_{\mathbf{Z} \sim \text{Bern}(p)^n} [\text{SCD}((\mathbf{Z} \cdot \mathbf{P})_S, \mathbf{P}, S) \neq \mathbf{Z}]$ ▷ Smaller is better

```

1: FOR  $i = 1$  TO  $n$  DO
2:   IF  $i \in S$  THEN
3:      $\tilde{W}_i \leftarrow W_i$ 
4:   ELSE
5:      $\tilde{W}_i \leftarrow \text{argmax}_{b \in \mathbb{F}_2} \{\Pr[W_i = b | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}]\}$ 
6: RETURN  $\tilde{\mathbf{Z}} \leftarrow \tilde{\mathbf{W}} \cdot \mathbf{P}^{-1}$ 

```

Next we argue that Algorithm 16.3.2 has low failure probability:

Lemma 16.3.3. *If $\mathbf{P}$ is (ε, τ) -polarizing for $\text{Bern}(p)^n$ with unpredictable columns S , then the successive cancellation decoder has failure probability at most τn , i.e.,*

$$\Pr_{\mathbf{Z} \in \text{Bern}(p)^n} [\mathbf{Z} \neq \text{SCD}((\mathbf{Z} \cdot \mathbf{P})_S, \mathbf{P}, S)] \leq \tau n.$$

Thus, if $\mathbf{P}$ is $(\varepsilon, \varepsilon/n)$ -polarizing for $\text{Bern}(p)^n$ then the failure probability of the successive cancellation decoder is at most ε .

Proof. Let $\mathbf{W} = \mathbf{Z} \cdot \mathbf{P}$. Note that by Step 3 in Algorithm 16.3.2, we have for every $i \in S$, $\tilde{W}_i = W_i$. For any $i \notin S$, by Proposition 16.3.1 applied with $X = W_i$, $Y = \mathbf{W}_{<i}$, $\alpha = \tau$ we get that

$$\Pr_{\mathbf{Z}} [W_i \neq A_i(\tilde{\mathbf{W}}_{<i})] \leq \tau,$$

where $A_i(\cdot)$ is the corresponding function defined for $i \notin S$ (in Proposition 16.3.1). By a union bound, we get that

$$\Pr_{\mathbf{Z}} [\exists i \text{ s.t. } W_i \neq A_i(\tilde{\mathbf{W}}_{<i})] \leq \sum_{i=1}^n \Pr_{\mathbf{Z}} [W_i \neq A_i(\tilde{\mathbf{W}}_{<i})] \leq \tau n.$$

Note that by step 5 in Algorithm 16.3.2 and the definition of $A_i(\cdot)$, we have $\tilde{W}_i = A_i(\tilde{\mathbf{W}}_{<i})$. But if $\mathbf{W} \neq \tilde{\mathbf{W}}$ there must exist a least i such that $\tilde{W}_i \neq W_i$. Thus, we get $\Pr[\mathbf{W} \neq \tilde{\mathbf{W}}] \leq \tau n$ and so probability that $\text{SCD}(\mathbf{W}_S, \mathbf{P}, S) \neq \mathbf{Z}$ is at most τn . $\square$

³Note that the Successive Cancellation Decompressor, and Decompressors in general are not expected to work correctly on every input. Thus, the INPUT/OUTPUT relations don't fully capture the goals of the algorithm. In such cases in the rest of the chapter, we will include a PERFORMANCE PARAMETER, which we wish to minimize that attempts to capture the real goal of the algorithm.

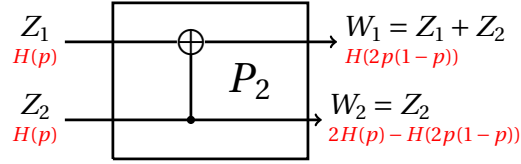


Figure 16.1: The 2×2 Basic Polarizing Transform. Included in red are the conditional entropies of the variables, conditioned on variables directly above them. *Acknowledgement:* Figure by and used with permission from Matt Eichhorn.

To summarize, in this section we have learned that to prove Theorem 16.1.1 it suffices to answer the following question:

Question 16.3.1. Given any $\varepsilon > 0$, does there exist an $(\varepsilon, \varepsilon/n)$ -polarizing matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$ where n is bounded by a polynomial in $1/\varepsilon$; where SCD (potentially a re-implementation of Algorithm 16.3.2) as well as multiplication and inversion by $\mathbf{P}$ takes $O(n \log n)$ time.

Next we will describe the idea behind the construction, which will answer the above question.

16.3.3 A polarizing primitive

Thus far in the chapter, we have essentially only been looking at the problem from different perspectives, but have not yet suggested an idea on how to get the codes or compression schemes that we desire (i.e., to answer Question 16.3.1). In this section we will provide the essence of the idea, which is to start with a simple and basic *polarization* step, and then iterate it appropriately many times to get a highly polarized matrix. Indeed, it is this section that will explain the term polarization (and why we use this term to describe the matrices we seek).

Recall that the ultimate goal of polarization is to start with many bits $Z_1, \dots, Z_n$ that are independent and slightly unpredictable (if p is small), and to produce some linear transform that concentrates all the unpredictability into fewer bits. We will first try to achieve this with two bits. Therefore we have Z_1, Z_2 such that $H(Z_1) = H(Z_2) = p$ and we wish to produce two bits W_1, W_2 such that at least one of these is less predictable than either Z_i . Since there are only 4 possible linear combinations of two bits (over $\mathbb{F}_2$) and three of them are trivial (0, Z_1 , and Z_2) we are left with only one candidate function, namely $Z_1 + Z_2$, so we will set $W_1 = Z_1 + Z_2$. For W_2 we are left with the trivial functions: 0 carries no information and so is ruled out. Without loss of generality, the only remaining choice is $W_2 = Z_2$. Thus, we look at this transformation: $P_2 : (Z_1, Z_2) \mapsto (W_1, W_2) = (Z_1 + Z_2, Z_2)$. (See Figure 16.1.)

This is an invertible linear transformation given by the matrix

$$\mathbf{P}_2 = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

This in turn implies (see Exercise 16.6):

$$H(W_1, W_2) = 2H(p). \quad (16.9)$$

However, some examination shows that $H(W_1) > H(Z_1), H(Z_2)$. In particular, the probability that W_1 is 1 is $2p(1-p) \in (p, 1/2)$, and since $H(\cdot)$ is monotonically increasing in this interval (see Exercise 16.7) it follows that $H(W_1) > H(p) = H(Z_1) = H(Z_2)$. Thus, W_1 is less predictable than either of the input bits, and thanks to the chain rule:

$$H(W_2|W_1) = H(W_1, W_2) - H(W_1) = H(Z_1, Z_2) - H(W_1) = H(Z_1) + H(Z_2) - H(W_1) < H(Z_1), H(Z_2).$$

In other words, multiplying by $\mathbf{P}_2$ has separated the entropy of two equally entropic bits into a more entropic bit and a (conditionally) less entropic one. Of course this may be only slight polarization, and what we are hoping for is many bits that are almost completely determined by preceding ones.

To get more polarization, we apply this 2×2 operation repeatedly. Specifically, let $P_2(Z_1, Z_2) = (Z_1 + Z_2, Z_2)$. Then we let $P_4(Z_1, Z_2, Z_3, Z_4) = (W_1, W_2, W_3, W_4) = (P_2(U_1, U_3), P_2(U_2, U_4))$ where $(U_1, U_2) = P_2(Z_1, Z_3)$ and $(U_3, U_4) = P_2(Z_2, Z_4)$. Thus, the bit $W_1 = Z_1 + Z_2 + Z_3 + Z_4$ has higher entropy than say Z_1 or even $U_1 = Z_1 + Z_3$. On the other hand, $W_4 = Z_4$ conditioned on (W_1, W_2, W_3) can be shown to have much lower entropy than Z_4 (unconditionally) or even $U_4 = Z_4$ conditioned on U_2 .

The composition above can be extended easily to n bit inputs, when n is a power of two, to get a linear transform P_n (See Figure 16.2). We will also give this transformation explicitly in the next section).

It is also clear that some bits will get highly entropic due to these repeated applications of P_2 . What is remarkable is that the polarization is nearly “perfect” - most bits W_i have conditional entropy (conditioned on $\mathbf{W}_{<i}$) close to 1, or close to 0. (We will show this result later on.) This leads to the simple construction of the polarizing matrix we will describe in the next section. A striking benefit of this simple construction is that multiplying a vector by either $\mathbf{P}$ or $\mathbf{P}^{-1}$ only takes $O(n \log n)$ time. Further, a version of SCD (Algorithm 16.3.2) is also computable in time $O(n \log n)$ and this leads to an overall compression and decompression algorithms running in time $O(n \log n)$, which we describe in the next section.

We note that one missing element in our description is the challenge of determining the exact set of indices S that includes all the high-conditional-entropy bits. We will simply skirt the issue and assume that this set is known for a given matrix $\mathbf{P}$ and given to the compression/decompression algorithms. Note that this leads to a non-uniform solution to compression and decompression problem, as well as the task of achieving Shannon capacity on the binary symmetric channel. We stress that this is not an inherent problem with the polarization approach. An explicit algorithm to compute S (or a small superset of S) is actually known and we discuss this in Section 16.7.

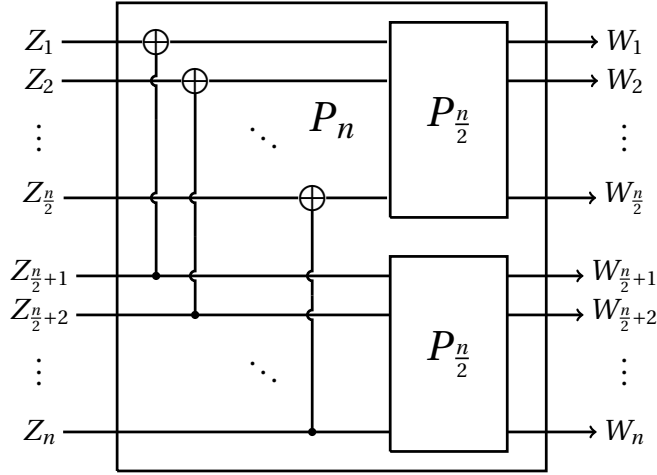


Figure 16.2: The $n \times n$ Basic Polarizing Transform defined as $P_n(\mathbf{Z}) = P_n(\mathbf{U}, \mathbf{V}) = \begin{pmatrix} P_{n/2}(\mathbf{U} + \mathbf{V}) \\ P_{n/2}(\mathbf{V}) \end{pmatrix}$. *Acknowledgement:* Figure by and used with permission from Matt Eichhorn.

16.4 Polar codes, Encoder and Decoder

We begin with the description of polar codes along with the statement of the main results in Section 16.4.1. We explicitly state the encoding algorithm and analyze its runtime in Section 16.4.2. We present the decoder as well as its proof of correctness in Section 16.4.3.

16.4.1 The Code and Polarization Claims

We are now ready to describe the code.

Definition 16.4.1 (Basic polarizing matrix). We define the $n \times n$ polarization matrix $\mathbf{P}_n$ recursively for $n = 2, 4, 8, \dots$, by describing the linear map $P_n : \mathbf{Z} \mapsto \mathbf{Z} \cdot \mathbf{P}_n$. For $n = 2$ and $\mathbf{Z} \in \mathbb{F}_2^2$ we define $P_2(\mathbf{Z}) = (Z_1 + Z_2, Z_2)$.⁴ For $n = 2^t$ and $\mathbf{Z} = (\mathbf{U}, \mathbf{V})$ for $\mathbf{U}, \mathbf{V} \in \mathbb{F}_2^{n/2}$ we define

$$P_n(\mathbf{Z}) = (P_{n/2}(\mathbf{U} + \mathbf{V}), P_{n/2}(\mathbf{V})).$$

Exercise 16.8 talks about an explicit description of $\mathbf{P}_n$ as well as some extra properties.

In Section 16.5 we show that this matrix polarizes quickly as $n \rightarrow \infty$. The main theorem we will prove is the following:

⁴We will be using $\mathbf{P}_n$ to denote the $n \times n$ matrix and P_n to denote the corresponding linear map that acts on $\mathbf{Z}$.

Theorem 16.4.2 (Polynomially) Strong Polarization). Fix $p \in (0, 1/2)$ and constant c . There exists a polynomial function n_0 such that for every $\varepsilon > 0$, there exists $n = 2^t$ with $1/\varepsilon \leq n \leq n_0(1/\varepsilon)$ and a set $E \subseteq [n]$ with $|E| \leq (\varepsilon/2) \cdot n$ such that for every $i \notin E$, the conditional entropy $H(W_i | \mathbf{W}_{<i})$ is either less than n^{-c} , or greater than $1 - n^{-c}$. Furthermore, if we let $S = \{i \in [n] | H(W_i | \mathbf{W}_{<i}) \geq n^{-c}\}$ then $|S| \leq (H(p) + \varepsilon) \cdot n$ and the matrix $\mathbf{P}_n$ is $(\varepsilon, 1/n^c)$ -polarizing for $\text{Bern}(p)^n$ with unpredictable columns S .

This theorem allows us to specify how close to zero the conditional entropy of the polarized bits should be. Notably, this threshold can be the inverse of an arbitrarily-high degree polynomial in n . We will prove Theorem 16.4.2 in Section 16.5, which will be quite technical. But with the theorem in hand, it is quite simple to complete the description of the Basic Polar Code along with the associated encoding and decoding algorithms, and to analyze their performance.

Definition 16.4.3 (Basic Polar (Compressing) Code). Given $0 < p < 1/2$ and $\varepsilon \leq 1/4$, let n and $S \subseteq [n]$ be as given by Theorem 16.4.2 for $c = 2$. Then the Basic Polar Code with parameters p and ε maps $\mathbf{Z} \in \mathbb{F}_2^n$ to $P_n(\mathbf{Z})_S$.

Proposition 16.4.4 (Rate of the Basic Polar Code). For every $p \in (0, 1/2)$ and $\varepsilon > 0$, the rate of the Basic Polar Code with parameters p and ε is at most $H(p) + \varepsilon$.⁵

16.4.2 Encoding

The description of the map $P_n(\mathbf{Z})$ is already explicitly algorithmic, modulo the computation of the set S . For the sake of completeness, we write the algorithm below in Algorithm 16.4.1, assuming S is given as input, and argue its runtime.

Algorithm 16.4.1 BASIC POLAR ENCODER($\mathbf{Z}; n, S$)

INPUT: n power of 2, $\mathbf{Z} \in \mathbb{F}_2^n$ and $S \subseteq [n]$

OUTPUT: Compression $\mathbf{W} \in \mathbb{F}_2^S$ of $\mathbf{Z}$ given by $\mathbf{W} = (P_n(\mathbf{Z}))_S$

```

1: RETURN  $\mathbf{W} = (P(n, \mathbf{Z}))_S$ 

2: FUNCTION  $P(n, \mathbf{Z})$ 
3:   IF  $n = 1$  THEN
4:     RETURN  $\mathbf{Z}$ 
5:   ELSE
6:     Let  $\mathbf{U} = (Z_1, \dots, Z_{n/2})$  and  $\mathbf{V} = (Z_{n/2+1}, \dots, Z_n)$ 
7:     RETURN  $(P(n/2, \mathbf{U} + \mathbf{V}), P(n/2, \mathbf{V}))$ 
```

Proposition 16.4.5. The runtime of the BASIC POLAR ENCODER algorithm is $O(n \log n)$.

Proof. If $T(n)$ denotes the runtime of the algorithm on inputs of length n , then $T(\cdot)$ satisfies the recurrence $T(n) = 2T(n/2) + O(n)$ (since all operations other than the two recursive calls can be done in $O(n)$ time), which implies the claimed runtime. $\square$

⁵Recall that we are trying to solve the compression problem now.

16.4.3 Decoding

Note that a polynomial time algorithm to compute $\Pr[W_i = b | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}]$ given $b \in \mathbb{F}_2$ and $\tilde{\mathbf{W}}_{<i} \in \mathbb{F}_2^{i-1}$ would lead to a polynomial time implementation of the SUCCESSIVE CANCELLATION DECOMPRESSOR (Algorithm 16.3.2). However, by exploiting the nice structure of the Basic Polarizing Matrix, we will get an $O(n \log n)$ algorithm without much extra effort. The key insight into this faster decoder is that the decoder works even if $\mathbf{Z} \sim \text{Bern}(p_1) \times \dots \times \text{Bern}(p_n)$, i.e., even when the bits of $\mathbf{Z}$ are not identically distributed, as long as they are independent. This stronger feature allows for a simple recursive algorithm. Specifically we use the facts that:

1. If $Z_1 \sim \text{Bern}(p_1)$ and $Z_2 \sim \text{Bern}(p_2)$ are independent then $Z_1 + Z_2$ is a Bernoulli random variable. Let $b^+(p_1, p_2)$ denote the bias (i.e., probability of being 1) of $Z_1 + Z_2$ (and so $Z_1 + Z_2 \sim \text{Bern}(b^+(p_1, p_2))$) (see Exercise 16.9).
2. If $Z_1 \sim \text{Bern}(p_1)$ and $Z_2 \sim \text{Bern}(p_2)$ are sampled conditioned on $Z_1 + Z_2 = a$ (for some $a \in \mathbb{F}_2$) then Z_2 is still a Bernoulli random variable. Let $b^l(p_1, p_2, a)$ denote the bias of Z_2 conditioned on $Z_1 + Z_2 = a$. (Note that $b^l(p_1, p_2, 0)$ is not necessarily equal to $b^l(p_1, p_2, 1)$.) See Exercise 16.10 for more.

We now use the functions b^+ and b^l defined above to describe our decoding algorithm. We switch our notation slightly to make for a cleaner description. Rather than being given the vector $\mathbf{W}_S \in \mathbb{F}_2^{|S|}$, we assume that our decoder is given as input a vector $\mathbf{W} \in (\mathbb{F}_2 \cup \{?\})^n$ where $W_i = ?$ if and only if $i \notin S$.

The main idea behind the decoding algorithm is that to complete $\mathbf{W}$ to a vector in $\mathbb{F}_2^n$, we can first focus on computing

$$\mathbf{W}[1, \dots, n/2] \stackrel{\text{def}}{=} (W_1, \dots, W_{n/2}).$$

It will turn out that we can use the fact that $\mathbf{W}[1, \dots, n/2] = P_{n/2}(Z'_1, \dots, Z'_{n/2})$ where $Z'_i \in \text{Bern}(b^+(p, p))$ are independent.⁶ We use recursion to solve this problem, and by computing the Z'_i 's along the way, we can in turn compute $\mathbf{W}[n/2 + 1, \dots, n] = P_{n/2}(Z_{n/2+1}, \dots, Z_n)$. Note that here Z_i is no longer drawn from $\text{Bern}(p)$ but instead we have $Z_i \sim \text{Bern}(b^l(p, p, Z'_{i-n/2}))$. Nevertheless, the Z_i 's are still independent. This stronger condition allows us to solve the problem recursively, as detailed below in Algorithm 16.4.2.

We assume that b^+ and b^l can be computed in constant time, and with this assumption it is straightforward to see that the above algorithm also has a runtime of $O(n \log n)$, by using the same recursive analysis that we used in the proof of Proposition 16.4.5. The correctness is a bit more involved and we argue this in the next lemma.

Lemma 16.4.6. *Let $\mathbf{Z} \sim \text{Bern}(p_1) \times \dots \times \text{Bern}(p_n)$, and $\mathbf{W} = P_n(\mathbf{Z})$. Further let $\mathbf{W}' = (W'_1, \dots, W'_n)$ be given by $W'_i = W_i$ if $i \in S$ and $W'_i = ?$ otherwise. Let $(\tilde{\mathbf{Z}}, \tilde{\mathbf{W}}, \boldsymbol{\rho}) = \text{RPD}(\mathbf{W}'; n, (p_1, \dots, p_n))$. Then, if $H(W_i | \mathbf{W}_{<i}) \leq \tau$ for every $i \notin S$, we have the following:*

- (1) *For every i , we have that $\Pr_{\mathbf{Z}}[W_i = 1 | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] = \rho_i$.*

⁶Note that this follows from definition of P_n .

Algorithm 16.4.2 BASIC POLAR DECODER: $\text{BPD}(\mathbf{W}; n, p)$

INPUT: n (power of 2), $\mathbf{W} \in (\mathbb{F}_2 \cup \{?\})^n$, and $0 \leq p < 1/2$

OUTPUT: $\tilde{\mathbf{Z}} \in \mathbb{F}_2^n$ such that for every i either $\mathbf{W}_i = ?$ or $(\tilde{\mathbf{Z}} \cdot \mathbf{P})_i = \mathbf{W}_i$

PERFORMANCE PARAMETER: $\Pr_{\mathbf{Z} \sim \text{Bern}(p)^n}[\mathbf{Z} \neq \text{BPD}(\mathbf{W}'; n, p)]$ where $\mathbf{W}'_S = (\mathbf{Z} \cdot \mathbf{P})_S$ and $W'_i = ?$ if $i \notin S$

1: $(\tilde{\mathbf{Z}}, \tilde{\mathbf{W}}, \boldsymbol{\rho}) = \text{RPD}(\mathbf{W}; n, (p, \dots, p))$

2: RETURN $\tilde{\mathbf{Z}}$

3: FUNCTION RECURSIVE POLAR DECODER: $\text{RPD}((\mathbf{W}; n, (p_1, \dots, p_n)))$

INPUT: $\mathbf{W} \in (\mathbb{F}_2 \cup \{?\})^n$ and $p_1, \dots, p_n \in [0, 1]$.

OUTPUT: $\tilde{\mathbf{Z}}, \tilde{\mathbf{W}} \in \mathbb{F}_2^n$ with $\tilde{\mathbf{W}} = P_n(\tilde{\mathbf{Z}})$ and $(\tilde{\mathbf{W}})_S = \mathbf{W}_S$. $\boldsymbol{\rho} = (\rho_1, \dots, \rho_n) \in [0, 1]^n$. $\triangleright \tilde{\mathbf{Z}}$ is the main output while $\tilde{\mathbf{W}}$ and $\boldsymbol{\rho}$ will help us reason about correctness.

PERFORMANCE PARAMETER: $\Pr_{\mathbf{Z} \sim \text{Bern}(p_1) \times \dots \times \text{Bern}(p_n)}[\tilde{\mathbf{Z}} \neq \mathbf{Z}]$ where $\tilde{\mathbf{Z}}$ is the first element of the triple output by $\text{RPD}((\mathbf{Z} \cdot \mathbf{P})_S; n, (p_1, \dots, p_n))$

4: IF $n = 1$ and $W_1 \in \mathbb{F}_2$ THEN

5: RETURN (W_1, W_1, p_1)

6: ELSE IF $n = 1$ and $W_1 = ?$ THEN

7: RETURN $(1, 1, p_1)$ if $p_1 \geq 1/2$ and $(0, 0, p_1)$ otherwise

8: ELSE

$\triangleright n \geq 2$

9: $\mathbf{W} = (\mathbf{W}^{(1)}, \mathbf{W}^{(2)})$ where $\mathbf{W}^{(1)} = \mathbf{W}[1, \dots, n/2]$ and $\mathbf{W}^{(2)} = \mathbf{W}[n/2 + 1, \dots, n]$

10: FOR $i = 1$ to $n/2$ DO

11: let $q_i = b^+(p_i, p_{n/2+i})$

12: Let $(\tilde{\mathbf{X}}, \tilde{\mathbf{W}}^1, \boldsymbol{\rho}^1) = \text{RPD}(\mathbf{W}^{(1)}; n/2, (q_1, \dots, q_{n/2}))$

13: FOR $i = 1$ to $n/2$ DO

14: let $r_i = b^l(p_i, p_{n/2+i}, \tilde{X}_i)$

15: Let $(\tilde{\mathbf{Y}}, \tilde{\mathbf{W}}^2, \boldsymbol{\rho}^2) = \text{RPD}(\mathbf{W}^{(2)}; n/2, (r_1, \dots, r_{n/2}))$

16: Let $\tilde{\mathbf{Z}} = (\tilde{\mathbf{X}} \oplus \tilde{\mathbf{Y}}, \tilde{\mathbf{Y}})$, and let $\tilde{\mathbf{W}} = (\tilde{\mathbf{W}}^1, \tilde{\mathbf{W}}^2)$ and $\boldsymbol{\rho} = (\boldsymbol{\rho}^1, \boldsymbol{\rho}^2)$

17: RETURN $(\tilde{\mathbf{Z}}, \tilde{\mathbf{W}}, \boldsymbol{\rho})$

$$(2) \Pr_{\mathbf{Z}}[\mathbf{W} \neq \tilde{\mathbf{W}}] \leq \tau n.$$

$$(3) \Pr_{\mathbf{Z}}[\mathbf{Z} \neq \tilde{\mathbf{Z}}] \leq \tau n.$$

Note that part (3) of Lemma 16.4.6 proves the same decoding error bound that we proved in Lemma 16.3.3 for the SUCCESSIVE CANCELLATION DECOMPRESSOR (Algorithm 16.3.2). The correctness of BASIC POLAR DECODER (Algorithm 16.4.2) follows immediately and we state this explicitly below (see Exercise 16.11).

Corollary 16.4.7. *For every input $n = 2^l$, $p \in [0, 1/2]$, and $\mathbf{W} \in (\mathbb{F}_2 \cup \{?\})^n$, the BASIC POLAR DECODER (Algorithm 16.4.2) runs in time $O(n \log n)$ and computes an output $\tilde{\mathbf{Z}} \in \mathbb{F}_2^n$ such that $P_n(\tilde{\mathbf{Z}})_i = \mathbf{W}_i$ holds for every i for which $\mathbf{W}_i \neq ?$. Furthermore if*

$$(1) \mathbf{Z} \sim \text{Bern}(p)^n,$$

$$(2) \mathbf{W}_S = P_n(\mathbf{Z})_S \text{ and}$$

$$(3) H(W_i | \mathbf{W}_{<i}) \leq \tau \text{ for every } i \notin S$$

then

$$\Pr_{\mathbf{Z}}[\tilde{\mathbf{Z}} \neq \mathbf{Z}] \leq \tau n.$$

Proof of Lemma 16.4.6. The main part of the lemma is part (1). Part (2) follows almost immediately from part (1) and Proposition 16.3.1. And part (3) is straightforward. We prove the parts in turn below.

We begin by first arguing that for every n that is a power of two:

$$\tilde{\mathbf{W}} = P_n(\tilde{\mathbf{Z}}). \quad (16.10)$$

For the base case of $n = 1$, lines 5 and 7 show that $W_1 = Z_1$ as desired.⁷ By induction (and lines 12 and 15) we have that $\tilde{\mathbf{W}}^1 = P_{n/2}(\tilde{\mathbf{X}})$ and $\tilde{\mathbf{W}}^2 = P_{n/2}(\tilde{\mathbf{Y}})$. Finally, line 16 implies that:

$$P_n(\tilde{\mathbf{Z}}) = P_n((\tilde{\mathbf{X}} + \tilde{\mathbf{Y}}, \tilde{\mathbf{Y}})) = (P_{n/2}(\tilde{\mathbf{X}} + \tilde{\mathbf{Y}} + \tilde{\mathbf{Y}}), P_{n/2}(\tilde{\mathbf{Y}})) = (\tilde{\mathbf{W}}^1, \tilde{\mathbf{W}}^2) = \tilde{\mathbf{W}},$$

as desired. In the above, the second equality follows from the definition of P_n .

Part (1) follows by induction on n . If $n = 1$ (where say we call $\text{RPD}((W_1), 1, (p_1''))$), then the claim follows since here we have $\rho_1 = p_1''$, $W_1 = Z_1$ and $Z_1 \sim \text{Bern}(p_1'')$. For larger values of n , we consider two cases (below we have $\mathbf{Z} = (\mathbf{U}, \mathbf{V})$).

If $i \leq n/2$, then we have $W_i = P_{n/2}(\mathbf{U} + \mathbf{V})_i$ (via definition of $\mathbf{P}_n$). Furthermore, $(\mathbf{U} + \mathbf{V}) \sim \text{Bern}(q_1) \times \dots \times \text{Bern}(q_{n/2})$. Thus, by the inductive claim, the recursive call $\text{RPD}(\mathbf{W}'[1, \dots, n/2]; n/2, (q_1, \dots, q_{n/2}))$ satisfies

$$\begin{aligned} \rho_i &= \rho_i^{(1)} \\ &= \Pr_{(\mathbf{U}+\mathbf{V}) \sim \text{Bern}(q_1) \times \dots \times \text{Bern}(q_{n/2})} [W_i = 1 | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \\ &= \Pr_{\mathbf{Z} \sim \text{Bern}(p_1) \times \dots \times \text{Bern}(p_n)} [W_i = 1 | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \end{aligned}$$

⁷Note that when defining P_n , the base case was $n = 2$ but note that if we started with $n = 1$ and define $P_1(\mathbf{Z}) = \mathbf{W}$, then the resulting definition is the same as on the we saw in Definition 16.4.1.

as desired. In the above, the first equality follows from the algorithm definition and the third equality follows from the definition of q_i .

Now if $i > n/2$, note that the condition $\mathbf{W}[1, \dots, n/2] = \tilde{\mathbf{W}}[1, \dots, n/2]$ is equivalent to the condition that $\mathbf{U} + \mathbf{V} = \tilde{\mathbf{X}}$ since $\mathbf{W}[1, \dots, n/2] = P_{n/2}(\mathbf{U} + \mathbf{V})$ and $\tilde{\mathbf{X}} = P_{n/2}^{-1}(\tilde{\mathbf{W}}[1, \dots, n/2])$, where the latter equality follows from (16.10) and the fact that the map $P_{n/2}$ is invertible. And now from definition of $\mathbf{P}_n$, we have $\mathbf{W}[n/2 + 1, \dots, n] = P_{n/2}(\mathbf{V})$. Conditioning on $\mathbf{U} + \mathbf{V} = \tilde{\mathbf{X}}$ implies that $\mathbf{V} \sim \text{Bern}(r_1) \times \dots \times \text{Bern}(r_{n/2})$ — this is exactly how r_i 's were defined. Thus, we have

$$\begin{aligned} &\Pr_{\mathbf{Z} \sim \text{Bern}(p_1) \times \dots \times \text{Bern}(p_n)} [W_i = 1 | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \\ &= \Pr_{\mathbf{V} \sim \text{Bern}(r_1) \times \text{Bern}(r_{n/2})} \left[P_{\frac{n}{2}}(\mathbf{V})_{i - \frac{n}{2}} = 1 | P_{\frac{n}{2}}(\mathbf{V})[1, \dots, i - \frac{n}{2} - 1] = \tilde{\mathbf{W}} \left[\frac{n}{2} + 1, \dots, i - 1 \right] \wedge \mathbf{U} + \mathbf{V} = \tilde{\mathbf{X}} \right]. \end{aligned}$$

By induction again on the recursive call to $\text{RPD}(\mathbf{W}'[n/2 + 1, \dots, n]; n/2, (r_1, \dots, r_{n/2}))$ we have the final quantity above equals $\rho_{i - n/2}^{(2)} = \rho_i$ (where the last equality follows from the algorithm definition). This concludes the proof of part (1).

Part (2) follows from Proposition 16.3.1. We first note that if $i \in S$, then by line 5 of the algorithm we have $W_i = \tilde{W}_i$. Now assume $i \notin S$. We claim that $\tilde{W}_i = 1$ if and only if $\rho_i = \Pr_{\mathbf{Z}}[W_i = 1 | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \geq 1/2$. To see this note that we set $\rho_i = p_i'$ when RPD is called on the input $(W_i', 1, p_i')$ and then the claim follows from line 7 of the algorithm. Thus, $\tilde{W}_i = \arg\max_{b \in \mathbb{F}_2} \Pr[W_i = b | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}]$. By Proposition 16.3.1 applied to the variables $X = W_i$ and $Y = \mathbf{W}_{<i}$ with $\alpha = \tau$, we get

$$\Pr[W_i \neq \tilde{W}_i | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \leq \tau.$$

By a union bound over i , we thus have

$$\Pr[\exists i \in [n] \text{ s.t. } W_i \neq \tilde{W}_i | \mathbf{W}_{<i} = \tilde{\mathbf{W}}_{<i}] \leq \tau n.$$

But if $\mathbf{W} \neq \tilde{\mathbf{W}}$ there must exist an index i such that $W_i \neq \tilde{W}_i$ and so we have $\Pr[\mathbf{W} \neq \tilde{\mathbf{W}}] \leq \tau n$ concluding proof of part (2).

For part (3), note that by (16.10), if $\tilde{\mathbf{W}} = \mathbf{W}$ (which holds with probability at least $1 - \tau n$ from part (2)), we have $\tilde{\mathbf{Z}} = P_n^{-1}(\tilde{\mathbf{W}}) = P_n^{-1}(\mathbf{W}) = \mathbf{Z}$ as desired. $\square$

To summarize the claims of this section, Theorem 16.4.2 guarantees the existence of a polarizing matrix as desired to satisfy the information-theoretic conditions of Question 16.3.1. And Proposition 16.4.5 and Corollary 16.4.7 ensure that the encoding and decoding times are $O(n \log n)$. This allows us to complete the proof of Theorem 16.1.1 (modulo the proof of Theorem 16.4.2 — which will be proved in the next section).

Proof of Theorem 16.1.1. Recall from Proposition 16.2.1 and Question 16.3.1 that it suffices to find, given $p \in [0, 1/2]$ and $\varepsilon > 0$, an $(\varepsilon, \varepsilon/n)$ -polarizing matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$ with n bounded by a polynomial in $1/\varepsilon$; such that multiplication by $\mathbf{P}$ and decompression take $O(n \log n)$ time.

Theorem 16.4.2 applied with parameters p, ε and $c = 2$ yields n and the matrix $\mathbf{P} = \mathbf{P}_n \in \mathbb{F}_2^{n \times n}$ that is $(\varepsilon, 1/n^2)$ -polarizing. Moreover, since Theorem 16.4.2 also guarantees $\varepsilon \geq 1/n$, we have that $\mathbf{P}_n$ is $(\varepsilon, \varepsilon/n)$ -polarizing. Furthermore, there exists a set $S \subseteq [n]$ with $|S| \leq (H(p) + \varepsilon)n$ such

that $H(\mathbf{W}_S | \mathbf{W}_S) \leq \varepsilon$ when $\mathbf{W} = \mathbf{Z} \cdot \mathbf{P}$ and $\mathbf{Z} \sim \text{Bern}(p)^n$. Given such a set S we have, by Proposition 16.4.5, that the time to compress $\mathbf{Z} \in \mathbb{F}_2^n$ to $(\mathbf{Z} \cdot \mathbf{P}_n)_S$ is $O(n \log n)$. Finally Corollary 16.4.7 asserts that a decompression $\tilde{\mathbf{Z}}$ can be computed given $(\mathbf{Z} \cdot \mathbf{P}_n)_S$ in time $O(n \log n)$ and $\tilde{\mathbf{Z}}$ equals $\mathbf{Z}$ with probability at least $1 - \varepsilon$ thereby completing the proof of Theorem 16.1.1. $\square$

16.5 Analysis: Speed of Polarization

We now turn to the most crucial aspect of polarization - the fact that it happens, and it is fast enough to deliver polynomial convergence to capacity. In this section we first give an overview of how we will think about polarization. We will then analyze the convergence by first exploring what happens in a single polarization step (i.e., the action of P_2) and then showing how the local effects aggregate after $\log_2 n$ steps of polarization. This will lead us to the proof of Theorem 16.4.2.

16.5.1 Overview of Analysis

We start by setting up some more notation. Recall $n = 2^t$. In this section it will be convenient for us to give names to intermediate variables $\{Z_i^{(j)}\}_{(i \in [n], 0 \leq j \leq t)}$ that are computed during the computation of $P_n(Z_1, \dots, Z_n)$. Let $Z_i^{(0)} = Z_i$. For $1 \leq j \leq t$, let

$$(Z_i^{(j)}, Z_{i+2^{j-1}}^{(j)}) = P_2(Z_i^{(j-1)}, Z_{i+2^{j-1}}^{(j-1)}) = (Z_i^{(j-1)} + Z_{i+2^{j-1}}^{(j-1)}, Z_{i+2^{j-1}}^{(j-1)}). \quad (16.11)$$

for every $i \in [n]$ such that i and $i + 2^{j-1}$ are in the same “block at the j th” level. i.e., $\lceil i/2^{t-j+1} \rceil = \lceil (i + 2^{j-1})/2^{t-j+1} \rceil$. (Alternatively one could say i and $i + 2^{j-1}$ as in the same *dyadic interval* of size 2^{t-j+1} .) Further,

Definition 16.5.1. We will say that a pair (i, i') are j th-level siblings if they are in the same block at the j th level and $i' = i + 2^{j-1}$.

Note that if one unravels the recursion in (16.11), then if $Z_i, Z_{i'}$ are on the LHS, then (i, i') are siblings at the j th level.

We now claim that (see Exercise 16.12):

$$P_n(\mathbf{Z}) = (Z_1^{(t)}, \dots, Z_n^{(t)}). \quad (16.12)$$

Figure 16.3 illustrates the block structure of P_n and the notion of j th level blocks and siblings at the j th level.

In what follows we will pick a random $i \in [n]$ and analyze the conditional entropy of $Z_i^{(j)} | \mathbf{Z}_{<i}^{(j)}$ as j progresses from 0 to t (we independently pick i for different values of j). Indeed, let

$$X_j = H(Z_i^{(j)} | \mathbf{Z}_{<i}^{(j)}).$$

Clearly $X_0 = H(p)$ since $Z_i^{(0)} = Z_i$ is independent of $\mathbf{Z}_{<i}$ and is distributed according to $\text{Bern}(p)$. In what follows we will show that for every constant c if t is a sufficient large then with high

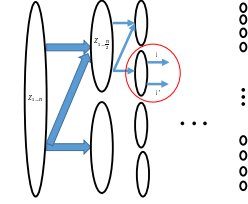


Figure 16.3: Block structure of the Basic Polarizing Transform. Circled are a block at the 2nd level and two 2nd level siblings.

probability over the choice of i , $X_t \notin (n^{-c}, 1 - n^{-c})$. To track the evolution of X_j as j increases, we will first try to analyze *local polarization* which will study how X_j compares with X_{j-1} . Definition 16.5.2 below captures the desired effect of a local step, and the following lemma asserts that the operator P_2 does indeed satisfy the conditions of local polarization.

Definition 16.5.2 (Local Polarization). A sequence of random variables $X_0, \dots, X_j, \dots$, with $X_j \in [0, 1]$ is locally polarizing if the following conditions hold:

- (1) **(Unbiased):** For every j , and $a \in [0, 1]$ we have $\mathbb{E}[X_{j+1} | X_j = a] = a$.
- (2) **(Variance in the middle):** For every $\tau > 0$, there exists $\theta = \theta(\tau) > 0$ such that for all j , we have: If $X_j \in (\tau, 1 - \tau)$ then $|X_{j+1} - X_j| \geq \theta$.
- (3) **(Suction at the ends):** For every $c < \infty$, there exists $\tau = \tau(c) > 0$ such that (i) if $X_j \leq \tau$ then $\Pr[X_{j+1} \leq X_j / c] \geq 1/2$; and similarly (ii) if $1 - X_j \leq \tau$ then $\Pr[(1 - X_{j+1}) \leq (1 - X_j) / c] \geq 1/2$.

We further say a sequence is simple if for every sequence $a_0, \dots, a_j$, conditioned on $X_0 = a_0, \dots, X_j = a_j$, there are two values a^+ and a^- such that X_{j+1} takes value a^+ with probability $1/2$ and a^- with probability $1/2$.

Lemma 16.5.3 (Local Polarization). The sequence $X_0, \dots, X_j, \dots$, with $X_j = H(Z_i^{(j)} | \mathbf{Z}_{<i}^{(j)})$ where i is drawn uniformly from $[n]$ is a simple and locally polarizing sequence.

We will prove Lemma 16.5.3 in Section 16.5.2 but use it below. But first let us see what it does and fails to do. While local polarization prevents the conditional entropies from staying static, it doesn't assert that eventually all conditional entropies will be close to 0 or close to 1, the kind of strong polarization that we desire. The following definition captures our desire from a strong polarizing process, and the lemma afterwards asserts that local polarization does imply strong polarization.

Definition 16.5.4 ((Polynomially) Strong Polarization). A sequence of random variables $X_0, X_1, \dots, X_t, \dots$ with $X_j \in [0, 1]$ strongly polarizes if for all $\gamma > 0$ there exist $\alpha < 1$ and $\beta < \infty$ such that for all t we have

$$\Pr[X_t \in (\gamma^t, 1 - \gamma^t)] \leq \beta \cdot \alpha^t.$$

Lemma 16.5.5 (Local vs. Global Polarization). *If a sequence $X_0, \dots, X_t, \dots$, with $X_t \in [0, 1]$ is simple and locally polarizing, then it is also strongly polarizing.*

Armed with Lemmas 16.5.3 and 16.5.5, proving Theorem 16.4.2 is just a matter of setting parameters.

Proof of Theorem 16.4.2. We assume without loss of generality that $c \geq 1$. (Proving the theorem for larger c implies it also for smaller values of c .) Given p and $c \geq 1$, let $\gamma = 2^{-c}$. Let $\beta < \infty$ and $\alpha < 1$ be the constants given by the definition of strong polarization (Definition 16.5.4) for this choice of γ . We prove the theorem for $n_0(x) = \max\{8x, 2(2\beta x)^{\lceil \log(1/\alpha) \rceil}\}$. Note that n_0 is a polynomial in x . Given $\varepsilon > 0$, let $t = \max\left\{\lfloor \log(8/\varepsilon) \rfloor, \left\lceil \frac{\log(2\beta/\varepsilon)}{\log(1/\alpha)} \right\rceil\right\}$ so that

$$n = 2^t \leq \max\left\{8/\varepsilon, 2 \cdot (2\beta/\varepsilon)^{1/\log(1/\alpha)}\right\} = n_0(1/\varepsilon).$$

Note that the choice of t gives $\beta \cdot \alpha^t \leq \varepsilon/2$ and $\gamma^t = 2^{-ct} = n^{-c}$. We also have $n \geq 4/\varepsilon$ and thus $2n^{-c} \leq 2n^{-1} \leq \varepsilon/2$. We show that for this choice and t and n , the polarizing transform P_n has the desired properties — namely that the set S of variables of noticeably large conditional entropy is of small size.

We first show that the set of variables with intermediate conditional entropies is small. Let us recall some notations from above, specifically (16.11). Let $(Z_i^{(j)})$ denote the intermediate results of the computation $\mathbf{W} = P_n(\mathbf{Z}) = (Z_1^{(t)}, \dots, Z_n^{(t)})$, and let $X_j = H(Z_i^{(j)} | \mathbf{Z}_{< i}^{(j)})$ for a uniformly random choice of $i \in [n]$. By Lemmas 16.5.3 and 16.5.5 we have that the sequence $X_0, \dots, X_t, \dots$ is strongly polarizing. By the definition of strong polarization, we have that

$$\begin{aligned} \Pr_{i \in [n]} [H(W_i | \mathbf{W}_{< i}) \in (n^{-c}, 1 - n^{-c})] &= \Pr_i [H(Z_i^{(t)} | \mathbf{Z}_{< i}^{(t)}) \in (\gamma^t, 1 - \gamma^t)] \\ &= \Pr [X_t \in (\gamma^t, 1 - \gamma^t)] \\ &\leq \beta \alpha^t \\ &\leq \varepsilon/2. \end{aligned}$$

Thus, we have that the set $E = \{i \in [n] | H(W_i | \mathbf{W}_{< i}) \in (n^{-c}, 1 - n^{-c})\}$ satisfies $|E| \leq \varepsilon n/2$.

Finally, we argue the “Further” part. Indeed, we have

$$nH(p) = \sum_{i \in [n]} H(W_i | \mathbf{W}_{< i}) \geq \sum_{i \in S \setminus E} H(W_i | \mathbf{W}_{< i}) \geq (|S| - |E|)(1 - n^{-c}),$$

where the first equality follows from the chain rule and the last inequality follows from definitions of S and E . Re-arranging one gets that

$$|S| \leq \frac{nH(p)}{1 - n^{-c}} + \varepsilon n/2 \leq nH(p)(1 + 2n^{-c}) + \varepsilon n/2 \leq nH(p) + \varepsilon n.$$

□

It remains to prove Lemmas 16.5.3 and 16.5.5 which we prove in the rest of this chapter.

16.5.2 Local Polarization

To understand how X_j compares with X_{j-1} , we start with some basic observations about these variables, or more importantly the variables $Z_i^{(j)}$ and $Z_{i'}^{(j+1)}$ (recall (16.11)). Let i and i' be j th level siblings, so that $(Z_i^{(j)}, Z_{i'}^{(j)}) = P_2(Z_i^{(j-1)}, Z_{i'}^{(j-1)})$. Our goal is to compare the pairs of conditional entropies $(H(Z_i^{(j)} | \mathbf{Z}_{< i}^{(j)}), H(Z_{i'}^{(j)} | \mathbf{Z}_{< i'}^{(j)}))$ with $(H(Z_i^{(j-1)} | \mathbf{Z}_{< i}^{(j-1)}), H(Z_{i'}^{(j-1)} | \mathbf{Z}_{< i'}^{(j-1)}))$. The collection of variables involved and conditioning seem messy, so let us look at the structure of P_n more carefully to simplify the above. We do this by noticing the $Z_i^{(j)}$ is really independent of most $Z_{i'}^{(j)}$ (at least for small values of j) and in particular the set of variables that $Z_i^{(j-1)}$ and $Z_{i'}^{(j-1)}$ depend on are disjoint. Furthermore these two variables, and the sets of variables that they depend on are identically distributed. Next, we present the details.

We begin with a useful notation. Given $i \in [n = 2^t]$ and $0 \leq j \leq t$, let

$$S_{i,j} \stackrel{\text{def}}{=} \{k \in [n] | i \equiv k \pmod{2^{t-j}}\}.$$

Note that the $\ell_1, \ell_2 \in S_{i,j}$ need not be siblings at the j th level.

Proposition 16.5.6. *For every $1 \leq j \leq t$ and j th level siblings i and i' with $i < i'$ the following hold:*

- (1) $S_{i,j} = S_{i',j} = S_{i,j-1} \cup S_{i',j-1}$.
- (2) $\{Z_k^{(j)} | k \in S_{i,j}\}$ is independent of $\{Z_k^{(j)} | k \notin S_{i,j}\}$.
- (3)

$$\begin{aligned} H(Z_i^{(j-1)} | \mathbf{Z}_{< i}^{(j-1)}) &= H\left(Z_i^{(j-1)} | \mathbf{Z}_{\{k \in S_{i,j-1}, k < i\}}^{(j-1)}\right) \\ &= H\left(Z_i^{(j-1)} | \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j-1)}\right). \end{aligned}$$

(4)

$$\begin{aligned} H(Z_{i'}^{(j-1)} | \mathbf{Z}_{< i'}^{(j-1)}) &= H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i',j-1}, k < i'\}}^{(j-1)}\right) \\ &= H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j-1)}\right). \end{aligned}$$

(5)

$$\begin{aligned} H(Z_i^{(j)} | \mathbf{Z}_{< i}^{(j)}) &= H\left(Z_i^{(j)} | \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j)}\right) \\ &= H\left(Z_i^{(j)} | \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j-1)}\right). \end{aligned}$$

(6)

$$\begin{aligned} H(Z_{i'}^{(j)} | \mathbf{Z}_{< i'}^{(j)}) &= H\left(Z_{i'}^{(j)} | \left\{Z_i^{(j)}\right\} \cup \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j)}\right) \\ &= H\left(Z_{i'}^{(j)} | \left\{Z_i^{(j)}\right\} \cup \mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j-1)}\right). \end{aligned}$$

Proof. Part (1) follows from the definition of $S_{i,j}$ and the definition of siblings. Indeed, since $i' = i + 2^{t-j}$, we have $i \equiv i' \pmod{2^{t-j}}$, which implies the first equality. The second equality follows from the observations that $k_1 \equiv k_2 \pmod{2^{t-j+1}}$ implies $k_1 \equiv k_2 \pmod{2^{t-j}}$ (this in turn implies $S_{i,j-1}, S_{i',j-1} \subseteq S_{i,j}$) and that if $k \equiv i \pmod{2^{t-j+1}}$ then $k \not\equiv i' \pmod{2^{t-j}}$ and vice versa (which in turn implies that $S_{i,j-1}$ and $S_{i',j-1}$ are disjoint. Part (2) follows from the fact that (see Exercise 16.13):

Lemma 16.5.7. *For every i , the set $\{Z_k^{(j)} | k \in S_{i,j}\}$ is determined completely by $\{Z_k | k \in S_{i,j}\}$,*

and the Z_k 's are all independent. The first equality in part (3) follows immediately from part (2), and the second uses part (1) and the fact that $Z_i^{(j-1)}$ is independent of $\{Z_k | k \in S_{i',j-1}, k < i\}$ (the latter claim follows from the fact that $S_{i,j-1}$ and $S_{i',j-1}$ are disjoint, as argued in the proof of part (1) above). The first equality in part (4) is similar, whereas the second uses the additional fact that $S_{i',j-1}$ contains no elements between i and i' . Indeed, the latter observation implies that $H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i',j-1}, k < i'\}}^{(j-1)}\right) = H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i',j-1}, k \leq i\}}^{(j-1)}\right)$. But by part (2), $Z_{i'}^{(j-1)}$ is independent of $Z_i^{(j-1)}$ and hence we have $H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i',j-1}, k < i'\}}^{(j-1)}\right) = H\left(Z_{i'}^{(j-1)} | \mathbf{Z}_{\{k \in S_{i',j-1}, k < i\}}^{(j-1)}\right)$. The second equality in (4) then follows from parts (1) and (2). The first equalities in parts (5) and (6) are similar to the first equality in part (3) with part (6) using the fact that $\{k \in S_{i',j} | k < i'\} = \{i\} \cup \{k \in S_{i,j} | k < i\}$. The second equality follows from the fact that (see Exercise 16.14):

Lemma 16.5.8. *There is a one-to-one map from the variables $\mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j-1)}$ to the variables $\mathbf{Z}_{\{k \in S_{i,j}, k < i\}}^{(j)}$,*

and so conditioning on one set is equivalent to conditioning on the other. $\square$

To summarize the effect of Proposition 16.5.6 above, let us name the random variables $U = Z_i^{(j-1)}$ and $V = Z_{i'}^{(j-1)}$ and further let $A = \{Z_k^{(j-1)} | k \in S_{i,j-1}, k < i\}$ and $B = \{Z_k^{(j-1)} | k \in S_{i',j-1}, k < i'\}$. By the proposition above, the conditional entropies of interest (i.e., those of i and i') at the $(j-1)$ th stage are $H(U|A)$ and $H(V|B)$. On the other hand the conditional entropies of interest one stage later (i.e., at the j th stage) are $H(U+V|A, B)$ and $H(V|A, B, U)$. (Here we use that fact that $P_2(U, V) = (U+V, V)$.) By part (2) of Proposition 16.5.6 we also have that (U, A) and (V, B) are independent of each other. Finally, by examination we also have that (see Exercise 16.15)

Lemma 16.5.9. *(U, A) and (V, B) are identically distributed.*

Thus, our concern turns to understanding the local polarization of two independent and identically distributed bits. If one could ignore the conditioning then this is just a problem about two bits (U, V) and their polarization when transformed to $(U+V, V)$.

In the following lemma, we show how in the absence of conditioning these variables show local polarization effects. (For our application it will be useful for us to allow the variables to be not identically distributed, though still independent.) Suppose $H(U) = H(p_1)$ and $H(V) = H(p_2)$, then notice that $H(U+V) = H(p_1 \circ p_2)$ where

$$p_1 \circ p_2 \stackrel{\text{def}}{=} p_1(1-p_2) + p_2(1-p_1).$$

In the following lemma we show how $H(p_1 \circ p_2)$ relates to $H(p_1)$ and $H(p_2)$.

Lemma 16.5.10. *Let $p_1, p_2 \in [0, 1/2]$ with $p_1 < p_2$ and $\tau \in (0, 1/2)$. Then we have:*

- (1) $H(p_1 \circ p_2) \geq H(p_2)$.
- (2) *There exists $\theta = \theta(\tau) > 0$ such that if $H(p_1), H(p_2) \in (\tau, 1-\tau)$ then*

$$H(p_1 \circ p_2) - H(p_2) \geq \theta.$$

- (3) *If $H(p_1), H(p_2) \leq \tau$ then*

$$H(p_1 \circ p_2) \geq (1 - 9/\log(1/\tau))(H(p_1) + H(p_2)).$$

In particular, for every $c < \infty$, if $\tau \leq 2^{-9c}$ then

$$H(p_1) + H(p_2) - H(p_1 \circ p_2) \leq (H(p_1) + H(p_2))/c.$$

- (4) *If $H(p_1), H(p_2) \geq 1-\tau$ and $\tau \leq 1-H(1/4)$ then*

$$H(p_1 \circ p_2) \geq 1 - 20\tau(1 - H(p_2)).$$

In particular, for every $c' < \infty$, if $\tau < 1/(20c')$ then

$$1 - H(p_1 \circ p_2) \leq (1 - H(p_2))/c'.$$

We defer the proof of the above lemma to Section 16.6.2.

The lemma above essentially proves that $H(U+V)$ satisfies the requirements for local polarization relative to $H(U)$ and $H(V)$, but we still need to deal with the conditioning with respect to A and B . We do this below using some careful applications of Markov's inequality.

Lemma 16.5.11. *If (U, A) and (V, B) are identical and independent random variables with U, V being elements of $\mathbb{F}_2$ with $H(U|A) = H(V|B) = H(p)$, then the following hold:*

- (1) *For every $\tau > 0$ there exists $\theta > 0$ such that if $H(p) \in (\tau, 1-\tau)$ then*

$$H(U+V|A, B) \geq H(p) + \theta.$$

(2) For every $c < \infty$ there exists $\tau > 0$ such that if $H(p) \leq \tau$ then

$$H(U + V|A, B) \geq (2 - 1/c)H(p),$$

and if $H(p) \geq 1 - \tau$ then

$$H(U + V|A, B) \geq 1 - 1/c(1 - H(p)).$$

Proof. Let $p_a = \Pr[U = 1|A = a]$ so that $H(p) = H(U|A) = \mathbb{E}_A[H(p_A)]$. Similarly let $q_b = \Pr[V = 1|B = b]$. In what follows we consider what happens when A and B are chosen at random. If $H(p_A)$ and $H(q_B)$ are close to their expectations, then the required polarization comes from Lemma 16.5.10. But if $H(p_A)$ or $H(q_B)$ can deviate significantly from their expectation, then polarization happens simply due to the fact that one of them is much larger than the other and $H(U + V|A = a, B = b) \geq \max\{H(p_a), H(q_b)\}$. The details are worked out below.

We start with part (1). Let $\theta(\cdot)$ be the function from part (2) of Lemma 16.5.10 so that if $H(p_1), H(p_2) \in (\rho, 1 - \rho)$ then $H(p_1 \circ p_2) - H(p_2) \geq \theta(\rho)$. Given $\tau > 0$ let $\theta_1 = \theta(\tau/2)$. We prove this part for $\theta = \min\{\frac{\theta_1}{9}, \frac{\tau^2}{36}\} > 0$.

Let

$$\begin{aligned} r_1 &= \Pr_A[H(p_A) \leq \tau/2], \\ r_2 &= \Pr_A[H(p_A) \in (\tau/2, 1 - \tau/2)], \end{aligned}$$

and

$$r_3 = \Pr_A[H(p_A) \geq 1 - \tau/2].$$

(Note that since (U, A) and (V, B) are identically and independently distributed if one replaces p_A by q_B in the above equalities, then the equalities still remain valid. We will be implicitly using this for the rest of the proof.) Since $r_1 + r_2 + r_3 = 1$, at least one of them must be greater than or equal to $1/3$. Suppose $r_2 \geq 1/3$, then we have with probability at least $1/9$, both $H(p_A) \in (\tau/2, 1 - \tau/2)$ and $H(q_B) \in (\tau/2, 1 - \tau/2)$. Let a, b be such that $H(p_a), H(q_b) \in (\tau/2, 1 - \tau/2)$. Then, since $U + V \sim \text{Bern}(p_a \circ p_b)$, by Lemma 16.5.10 part (2),

$$H(U + V|A = a, B = b) \geq H(p_a) + \theta_1.$$

And by Lemma 16.5.10 part (1), we have for all a, b ,

$$H(U + V|A = a, B = b) \geq H(p_a).$$

Putting it together, we have

$$H(U + V|A, B) \geq \mathbb{E}_A[p_A] + \frac{1}{9} \cdot \theta_1 = H(p) + \frac{\theta_1}{9}.$$

Next we consider the case where $r_3 \geq 1/3$. Now consider the probability that $\Pr_A[H(p_A) \leq 1 - \tau]$. Notice that

$$1 - \tau \geq H(p) \geq (1 - r_3 - \Pr_A[H(p_A) \leq 1 - \tau]) \cdot (1 - \tau) + r_3 \cdot (1 - \tau/2).$$

Rearranging we conclude

$$\Pr_A[H(p_A) \leq (1 - \tau)] \geq \frac{r_3 \tau}{2(1 - \tau)} \geq \frac{\tau}{6}.$$

Thus, with probability at least $\tau/18$ we have A such that $H(p_A) \leq (1 - \tau)$ and B such that $H(q_B) \geq 1 - \tau/2$. Let a, b be such that $H(p_a) \leq 1 - \tau$ and $H(q_b) \geq 1 - \tau/2$. We have (from part (1) of Lemma 16.5.10)

$$H(U + V|A = a, B = b) \geq H(q_b) \geq H(p_a) + \frac{\tau}{2}.$$

We conclude that in this case

$$H(U + V|A, B) \geq \mathbb{E}_A[H(p_A)] + \frac{\tau^2}{36} = H(p) + \frac{\tau^2}{36}.$$

The case $r_1 \geq 1/3$ is similar and also yields

$$H(U + V|A, B) \geq H(p) + \tau^2/36.$$

Thus, in all cases we have

$$H(U + V|A, B) \geq H(p) + \theta,$$

which completes the proof of part (1).

We now turn to part (2). We only prove the case where $H(p) \leq \tau$. The case where $H(p) \geq 1 - \tau$ is similar and we omit that part (see Exercise 16.16). Given $c < \infty$, let $\tau' = \tau(4c)$ be the constant from part (3) of Lemma 16.5.10 for constant $4c$, so that if $H(p_1), H(p_2) \leq \tau'$ then

$$H(p_1 \circ p_2) \geq \left(1 - \frac{1}{4c}\right) \cdot (H(p_1) + H(p_2)).$$

Now let $\tau = \frac{\tau'}{2c}$ and $H(p) \leq \tau$. Define

$$\alpha = \Pr_A[H(p_A) \geq \tau'].$$

By Markov's inequality (Lemma 3.1.6) we have $\alpha \leq 1/(2c)$. Let

$$\gamma = \mathbb{E}_A[H(p_A)|H(p_A) \geq \tau'].$$

and

$$\delta = \mathbb{E}_A[H(p_A)|H(p_A) < \tau'].$$

We have

$$H(p) = \gamma\alpha + \delta(1 - \alpha). \quad (16.13)$$

We divide our analysis into four cases depending on whether $H(p_A) \geq \tau'$ or not, and whether $H(q_B) \geq \tau'$ or not. Let S_{11} denote the event that $H(p_A) \geq \tau'$ and $H(q_B) \geq \tau'$ and S_{00} denotes the event that $H(p_A) < \tau'$ and $H(q_B) < \tau'$. Define S_{10} and S_{01} similarly.

We start with the case of S_{10} . Let a, b be such that $H(p_a) \geq \tau'$ and $H(q_b) < \tau'$. We have by part (1) of Lemma 16.5.10, $H(U + V|A = a, B = b) \geq H(U|A = a) = H(p_a)$ (and similarly $H(U + V|A = a, B = b) \geq H(q_b)$). Thus, taking expectation after conditioning on $(A, B) \in S_{10}$ we have

$$\mathbb{E}_{(A,B)|(A,B) \in S_{10}} [H(U + V|A, B)] \geq \mathbb{E} [H(p_A)|H(p_A) \geq \tau'] = \gamma.$$

Similarly we have

$$\mathbb{E}_{(A,B)|(A,B) \in S_{01}} [H(U + V|A, B)] \geq \gamma$$

as well as

$$\mathbb{E}_{(A,B)|(A,B) \in S_{11}} [H(U + V|A, B)] \geq \gamma.$$

Note that $S_{11} \cup S_{10} \cup S_{01}$ happen with probability $2\alpha - \alpha^2$. Now we turn to S_{00} . Let a, b be such that $H(p_a), H(q_b) < \tau'$. By Lemma 16.5.10 part (3) we have $H(U + V|A = a, B = b) \geq (1 - 1/(4c))(H(p_a) + H(q_b))$. Taking expectations conditioned on $(A, B) \in S_{00}$ we get

$$\begin{aligned} \mathbb{E}_{(A,B)|(A,B) \in S_{00}} [H(U + V|A, B)] &\geq \left(1 - \frac{1}{4c}\right) \cdot (\mathbb{E}_A [H(p_A)|H(p_A) < \tau'] + \mathbb{E}_B [H(q_B)|H(q_B) < \tau']) \\ &= \left(1 - \frac{1}{4c}\right) \cdot 2\delta. \end{aligned}$$

Note finally that S_{00} happens with probability $(1 - \alpha)^2$. Combining the four cases we have

$$\begin{aligned} H(U + A|A, B) &\geq (2\alpha - \alpha^2)\gamma + (1 - \alpha)^2 \left(1 - \frac{1}{4c}\right) (2\delta) \\ &= 2\alpha\gamma + (1 - \alpha)2\delta - \alpha^2\gamma - (\alpha)(1 - \alpha)\delta - \frac{1}{4c} \cdot (1 - \alpha)^2 2\delta \\ &= 2H(p) - \alpha \cdot H(p) - \frac{1}{2c} \cdot (1 - \alpha)((1 - \alpha)\delta). \end{aligned}$$

In the above, the last equality follows from (16.13). Part (2) now follows by using $(1 - \alpha)\delta \leq H(p)$ (which in turn follows from (16.13)) and $\alpha \leq 1/(2c)$. $\square$

We are now ready to prove the local polarization lemma.

Proof of Lemma 16.5.3. Recall that $X_j = \mathbb{E}_{I \sim [n]} [Z_I^{(j)}]$. Let $X_j = H(p)$. Note that conditioned on the value of X_j , for any $(j+1)$ -level siblings $i < i'$, I is equally likely to equal i or i' . Conditioning on $I \in \{i, i'\}$, with probability $1/2$, $I = i$ and with probability $1/2$ $I = i'$. Let $U = Z_i^{(j)}$, $V = Z_{i'}^{(j)}$, $A = \{Z_k^{(j)} | k < i, k \in S_{i,j}\}$ and $B = \{Z_k^{(j)} | k < i', k \in S_{i',j}\}$ then if $I = i$, $X_j = H(U|A)$ (this follows from Lemma 16.5.6 part (3)) and if $I = i'$ then $X_j = H(V|B)$ (this follows from Lemma 16.5.6 part (4)). Furthermore if $I = i$ then $X_{j+1} = H(U + V|A, B)$ (this follows from (16.11) and parts (1) and (5) from Lemma 16.5.6) and if $I = i'$ then $X_{j+1} = H(V|A, B, U)$ (this follows from (16.11), parts (1) and (6) from Lemma 16.5.6 and the fact that $V|U$ and $V|U + V$ have the same distribution). With this setup, we are now ready to prove that the sequence $X_0, X_1, \dots$, satisfy the conditions of local polarization, and furthermore are simple.

We argue the conditions hold for each conditioning $I \in \{i, i'\}$ and so hold without the conditioning (the latter holds because the pairs $\{i, i'\}$ make a disjoint cover of $[n]$ and hence for a random $I \sim [n]$ is equally likely to fall in one of these pairs). The condition $\mathbb{E}[X_{j+1}|X_j = a] = a$ follows from the fact that there is a bijection from (U, V) to $(U + V, V)$, and so

$$H(U + V|A, B) + H(V|A, B, U) = H(U|A) + H(V|B).$$

Indeed, note that $2a$ is the RHS and $2X_{j+1}$ is the LHS of the above equality.

Now note that (see Exercise 16.17):

Lemma 16.5.12. (U, A) and (V, B) are independently and identically distributed.

The variance in the middle condition follows from Lemma 16.5.11 part (1) and the suction at the ends condition follows from Lemma 16.5.11 part (2). Finally simplicity follows from the fact that with probability $1/2$, $X_j = H(U + V|A, B)$ and with probability $1/2$, $X_j = H(V|A, B, U)$. $\square$

16.5.3 Local vs. Global Polarization

Finally we prove Lemma 16.5.5 which shows that simple local polarization implies strong polarization. We prove this part in two phases. First, we show that in the first $t/2$ steps, the sequence shows moderate polarization — namely, with all but exponentially small probability $X_{t/2}$ is an inverse exponential in t , but with a small constant base (so $X_t \notin (\alpha_1^t, 1 - \alpha_1^t)$ for some $\alpha_1 < 1$, but α_1 is close to 1). Next we show that conditioned on this moderate polarization, the sequence gets highly polarized (so $X_t \notin (\gamma^t, 1 - \gamma^t)$ for any $\gamma > 0$ of our choice), again with an exponentially small failure probability. We start with part (1).

In what follows, let $\gamma > 0$ be given and let

$$c = \max \left\{ 4, \frac{\gamma^8}{16} \right\}.$$

Let $\tau = \tau(c)$ be given by condition (3) of the definition of Local Polarization (Definition 16.5.2) and

$$\theta = \min \left\{ 1 - \frac{1}{c}, \theta(\tau) \right\}$$

where $\theta(\tau)$ is the constant given by condition (2) of the same definition.

We start with the first phase. We consider a potential function

$$\phi_j \stackrel{\text{def}}{=} \min \left\{ \sqrt{X_j}, \sqrt{1 - X_j} \right\}.$$

We first notice that ϕ_j is expected to drop by a constant factor in each step of polarization.

Lemma 16.5.13.

$$\mathbb{E} [\phi_{j+1} | \phi_j = a] \leq \left(1 - \frac{\theta^2}{16} \right) \cdot a.$$

Proof. Without loss of generality assume $X_j \leq 1/2$ (see Exercise 16.18) and so $a = \phi_j = \sqrt{X_j}$ and so $X_j = a^2$. Using the simplicity of the sequence $X_0, \dots$ as well as the fact that $\mathbb{E}[X_{j+1}|X_j = a] = a$, we have that there exists δ such that $X_{j+1} = a^2 + \delta$ with probability $1/2$ and $a^2 - \delta$ with probability $1/2$. Furthermore, if $X_j \leq \tau$, by the unbiasedness and suction at the ends conditions, we have $\delta \geq (1 - 1/c)a^2$ and if $X_j > \tau$ by the variance in the middle condition, we have $\delta \geq \theta(\tau) \geq \theta(\tau)a^2$. Thus, in either case we have

$$\delta \geq \theta a^2. \quad (16.14)$$

We now bound $\mathbb{E}[\phi_{j+1}]$ as follows:

$$\begin{aligned} \mathbb{E}[\phi_{j+1}] &\leq \mathbb{E}\left[\sqrt{X_{j+1}}\right] \\ &= \frac{1}{2}\sqrt{a^2 + \delta} + \frac{1}{2}\sqrt{a^2 - \delta} \\ &= \frac{a}{2}\left(\sqrt{1 + \frac{\delta}{a^2}} + \sqrt{1 - \frac{\delta}{a^2}}\right) \\ &\leq \frac{a}{2}\left(1 + \frac{\delta}{2a^2} - \frac{\delta^2}{16a^4} + 1 - \frac{\delta}{2a^2} - \frac{\delta^2}{16a^4}\right) \\ &= a\left(1 - \frac{\delta^2}{16a^4}\right) \\ &\leq a\left(1 - \frac{\theta^2}{16}\right). \end{aligned}$$

In the above, the first inequality follows from Lemma B.1.5 while the second inequality follows from (16.14). $\square$

Lemma 16.5.14 (Weakly Polynomial Polarization). *There exists $\alpha_1 < 1$ such that for all even t , we have*

$$\Pr[X_{t/2} \in (\alpha_1^t, 1 - \alpha_1^t)] \leq \alpha_1^t.$$

Proof. We first prove by induction on j that

$$\mathbb{E}[\phi_j] \leq \left(1 - \frac{\theta^2}{16}\right)^j.$$

This is certainly true for $j = 0$ since $\phi_0 \leq 1$. For higher j , by Lemma 16.5.13 we have

$$\mathbb{E}[\phi_j] \leq \left(1 - \frac{\theta^2}{16}\right)\mathbb{E}[\phi_{j-1}] \leq \left(1 - \frac{\theta^2}{16}\right)^j$$

as claimed. Let

$$\beta = \sqrt{1 - \frac{\theta^2}{16}}$$

and $\alpha_1 = \sqrt{\beta}$ (note that $\alpha_1 < 1$). By our claim, we have $\mathbb{E}[\phi_{t/2}] \leq \beta^t$. By Markov's inequality (Lemma 3.1.6), we now get that

$$\Pr[\phi_{t/2} \geq \alpha_1^t] \leq \frac{\beta^t}{\alpha_1^t} = \alpha_1^t.$$

Finally we note that if $\phi_{t/2} \leq \alpha_1^t$ then $X_{t/2} \notin (\alpha_1^{2t}, 1 - \alpha_1^{2t})$ and so in particular $X_{t/2} \notin (\alpha_1^t, 1 - \alpha_1^t)$. We conclude that the probability that $X_{t/2} \in (\alpha_1^t, 1 - \alpha_1^t)$ is at most α_1^t , yielding the lemma. $\square$

We now turn to phase two of the polarization. Here we use the fact that if $X_{t/2}$ is much smaller than τ , then X_j is very unlikely to become larger than τ for any $t/2 \leq j \leq t$. Furthermore if it does not ever become larger than τ then X_t is very likely to be close to its expected value (which grows like γ^t). The following lemmas provide the details. In the following recall that $\tau = \tau(c)$ where $c \geq 4$.

Lemma 16.5.15. *For all $\lambda > 0$, if $X_0 \leq \lambda$, then the probability there exists $j > 0$ such that $X_j \geq \tau$ is at most λ/τ . Similarly if $X_0 \geq 1 - \lambda$, then the probability there exists $j > 0$ such that $X_j \leq 1 - \tau$ is at most λ/τ .*

The lemma above is a special case of Doob's inequality for martingales. We give the (simple) proof below.

Proof. We give the proof for the case $X_0 \leq \lambda$. The case $X_0 \geq 1 - \lambda$ is symmetrical (see Exercise 16.19). Notice that we wish to show that for every integer $T > 0$

$$\Pr\left[\max_{0 \leq t \leq T} \{X_t\} \geq \tau\right] \leq \lambda/\tau.$$

Let us create a related sequence of variables Y_i as follows. Let $Y_0 = X_0$ and for $i \geq 1$, if $Y_{i-1} < \tau$ then $Y_i = X_i$, else $Y_i = Y_{i-1}$. Note that for every i and a , by the simplicity of X_i 's, we have $\mathbb{E}[Y_i|Y_{i-1} = a] = a$. Note further that $\max_{0 \leq t \leq T} \{X_t\} \geq \tau$ if and only if $Y_T \geq \tau$. Thus,

$$\Pr\left[\max_{0 \leq t \leq T} \{X_t\} \geq \tau\right] = \Pr[Y_T \geq \tau] \leq \frac{\mathbb{E}[Y_T]}{\tau},$$

where the final inequality is Markov's inequality (Lemma 3.1.6). But

$$\mathbb{E}[Y_T] = \mathbb{E}[Y_{T-1}] = \dots = \mathbb{E}[Y_0] = \mathbb{E}[X_0] \leq \lambda$$

and this yields the lemma. $\square$

Lemma 16.5.16 (Weak to Strong Polarization). *There exists $\alpha_2 < 1$ such that for every $\lambda > 0$ if $X_0 \notin (\lambda, 1 - \lambda)$, then the probability that $X_{t/2} \in (\gamma^t, 1 - \gamma^t)$ is at most $\lambda/\tau + \alpha_2^t$.*

Proof. Again we consider the case $X_0 \leq \lambda$ with the other case being symmetrical (see Exercise 16.20).

Let $Z_i = 1$ if $X_i < X_{i-1}$ and 0 otherwise. For simple sequences, notice that Z_i 's are independent and 1 with probability $1/2$. Let $Z = \sum_{i=1}^{t/2} Z_i$. We consider two possible "error" events. $\mathcal{E}_1$ is

the event that there exists $1 \leq j \leq t/2$ such that $X_j \geq \tau$, and $\mathcal{E}_2$ is the event that $Z \leq t/8$. Note that by Lemma 16.5.15, $\mathcal{E}_1$ happens with probability at most λ/τ and (by the Chernoff bounds–Theorem 3.1.10) $\mathcal{E}_2$ happens with probability at most α_2^t for some $\alpha_2 < 1$. Now, if event $\mathcal{E}_1$ does not occur, then

$$X_{t/2} \leq 2^{t/2} \cdot c^{-Z} X_0 \leq 2^{t/2} c^{-Z}.$$

The first inequality follows from the subsequent argument. Using simplicity, we have with probability $1/2$, $X_1 \leq (1/c)X_0 \leq X_0/4$ (because of the suction at the ends condition) and with probability $1/2$ $X_1 \leq 2X_0$ (this follows the bound in the other case and the unbiasedness of the X_i s). Further if $\mathcal{E}_2$ also does not occur we have

$$X_{t/2} \leq 2^{t/2} \cdot c^{-t/8} \leq \gamma^t$$

by the choice of $c = 1/(2/\gamma^2)^4$. $\square$

Proof of Lemma 16.5.5. Recall that we wish to show, given $\gamma > 0$, that there exists $\alpha < 1$ and $\beta < \infty$ such that for all t we have

$$\Pr[X_t \in (\gamma^t, 1 - \gamma^t)] \leq \beta \cdot \alpha^t.$$

Let $\alpha_1 < 1$ be the constant from Lemma 16.5.14. Let $\alpha_2 < 1$ be the constant from Lemma 16.5.16. We prove this for $\alpha = \max\{\alpha_1, \alpha_2\} < 1$ and $\beta = 2 + 1/\tau < \infty$.

Let $\mathcal{E}$ be the event that $X_{t/2} \in (\alpha_1^t, 1 - \alpha_1^t)$. By Lemma 16.5.14 we have that $\Pr[\mathcal{E}] \leq \alpha_1^t$. Now conditioned on $\mathcal{E}$ not occurring, using Lemma 16.5.16 with $\lambda = \alpha_1^t$, we have $\Pr[X_t \in (\gamma^t, 1 - \gamma^t)] \leq \alpha_1^t/\tau + \alpha_2^t$. Thus, putting the two together we get

$$\Pr[X_t \in (\gamma^t, 1 - \gamma^t)] \leq \alpha_1^t + \frac{\alpha_1^t}{\tau} + \alpha_2^t \leq \alpha^t + \frac{\alpha^t}{\tau} + \alpha^t = \beta \cdot \alpha^t,$$

as desired. $\square$

16.6 Entropic Calculations

In this section, we present the omitted proofs on various properties of entropy and probability that mostly need some calculations.

16.6.1 Proof of Proposition 16.3.1

We begin with the first part, which is a straightforward calculation expanding the definition. For any i in the support of X , let p_i denote $\Pr_X[X = i]$ and let $x = \operatorname{argmax}_i\{p_i\}$ be the value maximizing this probability. Let $p_x = 1 - \gamma$. We wish to show that $\gamma \leq \alpha$. We now perform some crude calculations that lead us to this bound.

If $\gamma \leq 1/2$ we have

$$\alpha \geq H(X)$$

$$\begin{aligned} &= \sum_i p_i \log \frac{1}{p_i} \\ &\geq \sum_{i \neq x} p_i \log \frac{1}{p_i} \end{aligned} \quad (16.15)$$

$$\geq \sum_{i \neq x} p_i \log \frac{1}{\sum_{j \neq x} p_j} \quad (16.16)$$

$$\begin{aligned} &= \left(\sum_{i \neq x} p_i \right) \cdot \log \left(\frac{1}{\sum_{j \neq x} p_j} \right) \\ &= \gamma \cdot \log 1/\gamma \\ &\geq \gamma, \end{aligned} \quad (16.17)$$

as desired. In the above, (16.15) follows since all summands are non-negative, (16.16) follows since for every $i \neq x$, $p_i \leq \sum_{j \neq x} p_j$ and (16.17) follows since $\gamma \leq 1/2$ and so $\log 1/\gamma \geq 1$.

Now if $\gamma > 1/2$ we have a much simpler case since now we have

$$\begin{aligned} \alpha &\geq H(X) \\ &= \sum_i p_i \log \frac{1}{p_i} \\ &\geq \sum_i p_i \log \frac{1}{p_x} \end{aligned} \quad (16.18)$$

$$\begin{aligned} &= \log \frac{1}{p_x} \\ &= \log \frac{1}{1 - \gamma} \\ &\geq 1. \end{aligned} \quad (16.19)$$

(In the above, (16.18) follows since $p_i \leq p_x$, (16.19) follows since $\sum_i p_i = 1$ and (16.20) follows from the assumption that $\gamma \geq 1/2$.) But γ is always at most 1 so in this case also we have $\alpha \geq 1 \geq \gamma$ as desired.

We now consider the second part, which follows from the previous part via a simple averaging argument. Given y and i , let $p_{i,y} = \Pr_X[X = i | Y = y]$ and let $x_y = \operatorname{argmax}_i\{p_{i,y}\}$ be the value maximizing this probability. Let $\gamma_y = 1 - p_{x_y,y}$ and note that $\gamma = \mathbb{E}_Y[\gamma_Y]$. Let $\alpha_y = H(X | Y = y)$ and note again we have $\alpha = \mathbb{E}_Y[\alpha_Y]$. By the first part, we have for every y , $\gamma_y \leq \alpha_y$ and so it follows that

$$\gamma = \mathbb{E}_Y[\gamma_Y] \leq \mathbb{E}_Y[\alpha_Y] = \alpha.$$

16.6.2 Proof of Lemma 16.5.10

The lemma follows in a relatively straightforward manner with parts (3) and (4) using Lemma B.2.3.

Part (1) is immediate from the monotonicity of the entropy function in the interval $[0, 1/2]$ (see Exercise 16.7). For $0 \leq p_1, p_2 \leq 1/2$ we have $p_2 \leq p_1 \circ p_2 \leq 1/2$ and so (see Exercise 16.21)

$$H(p_1 \circ p_2) \geq H(p_2). \quad (16.21)$$

Next we turn to part (2). Let $H^{-1}(x) = p$ such that $0 \leq p \leq 1/2$ such that $H(p) = x$. Note H^{-1} is well defined and satisfies $H^{-1}(x) > 0$ if $x > 0$ and $H^{-1}(x) < 1/2$ if $x < 1$. Let

$$\alpha = \alpha(\tau) = H^{-1}(\tau)(1 - 2H^{-1}(1 - \tau))$$

and

$$\beta = \beta(\tau) = 2H^{-1}(1 - \tau)(1 - H^{-1}(1 - \tau))$$

and

$$\gamma = \gamma(\tau) = \log((1 - \beta)/\beta).$$

Note that $\alpha > 0$ and $\beta < 1/2$ and so $\gamma > 0$. We prove that $H(p_1 \circ p_2) - H(p_2) \geq \alpha \cdot \gamma$, and this will yield part (2) for $\theta = \theta(\tau) = \alpha \cdot \gamma > 0$.

First note that since $H(p_1), H(p_2) \in (\tau, 1 - \tau)$, we have $p_1, p_2 \in (H^{-1}(\tau), H^{-1}(1 - \tau))$. Thus,

$$p_1 \circ p_2 - p_2 = p_2(1 - 2p_1) + p_1 - p_2 = p_1(1 - 2p_2) \geq H^{-1}(\tau)(1 - 2H^{-1}(1 - \tau)) = \alpha.$$

Next we consider $\min_{p_2 \leq q \leq p_1 \circ p_2} \{H'(q)\}$. Note that by Exercise 16.22 $H'(q) = \log((1 - q)/q)$ and this is minimized when q is maximum. The maximum value of $q = p_1 \circ p_2$ is in turn maximized by using the maximum values of $p_1, p_2 = H^{-1}(1 - \tau)$. Thus, we have that $\min_{p_2 \leq q \leq p_1 \circ p_2} \{H'(q)\} \geq H'(\beta) = \gamma$. By elementary calculus we now conclude that

$$H(p_1 \circ p_2) - h(p_2) \geq (p_1 \circ p_2 - p_2) \cdot \min_{p_2 \leq q \leq p_1 \circ p_2} \{H'(q)\} \geq \alpha \cdot \gamma = \theta.$$

This concludes the proof of part (2).

Next we move to part (3). For this we first describe some useful bounds on $H(p)$. On the one hand we have $H(p) \geq p \log 1/p$. For $p \leq 1/2$ we also have $-(1 - p) \log(1 - p) \leq (1/\ln 2)(1 - p)(p + p^2) \leq (1/\ln 2)p \leq 2p$. And so we have $H(p) \leq p(2 + \log 1/p)$.

Summarizing, we have for $p \leq 1/2$,

$$p \log(1/p) \leq H(p) \leq p \log(1/p) + 2p. \quad (16.22)$$

We now consider $H(p_1) + H(p_2) - H(p_1 \circ p_2)$. We have

$$\begin{aligned} & H(p_1) + H(p_2) - H(p_1 \circ p_2) \\ & \leq p_1(\log(1/p_1) + 2) + p_2(\log(1/p_2) + 2) - (p_1 \circ p_2) \log(1/(p_1 \circ p_2)) \end{aligned} \quad (16.23)$$

$$\leq p_1(\log(1/p_1) + 2) + p_2(\log(1/p_2) + 2) - (p_1 + p_2 - 2p_1p_2) \log(1/(2p_2)) \quad (16.24)$$

$$\begin{aligned} & = p_1 \log(2p_2/p_1) + p_2 \log(2p_2/p_2) + 2p_1p_2 \log(1/(2p_2)) + 2(p_1 + p_2) \\ & \leq p_1 \log(p_2/p_1) + 2p_1p_2 \log(1/(p_2)) + 6p_2 \end{aligned} \quad (16.25)$$

$$\leq 2p_1 H(p_2) + 7p_2 \quad (16.26)$$

$$\leq 2p_1 H(p_2) + 7H(p_2)/\log(1/p_2) \quad (16.27)$$

$$\leq 9H(p_2)/\log(1/\tau). \quad (16.28)$$

In the above, (16.23) follows from (16.22), (16.24) follows from the fact that $p_1 \circ p_2 \leq p_1 + p_2 \leq 2p_2$, (16.25) follows from the fact that $3(p_1 + p_2) \leq 6p_2$, (16.26) follows from the fact that $p_1 \log(p_2/p_1) \leq$

p_2 , (16.27) follows from (16.22) and (16.28) follows from the subsequent argument. Indeed, by definition of τ and (16.22) we have $p_2 \log(1/p_2) \leq \tau$. Using the fact that $p_2 \leq 1/2$, this implies that $p_2 \leq \tau$, which in turn implies $\log(1/p_2) \geq \log(1/\tau)$. Similarly, we have $p_1 \log(1/p_1) \leq \tau$, which again with $p_1 \leq 1/2$, we have $p_1 \leq \tau \leq 1/\log(1/\tau)$ (where the second equality uses the fact that $\tau < 1/2$). This concludes the proof of part (3).

Finally we turn to part (4). Here we let $H(p_i) = 1 - y_i$ and $p_i = 1/2 - x_i$ for $i \in \{1, 2\}$. Since $\tau \leq 1 - H(1/4)$ and $H(p_i) \geq 1 - \tau$, we have $x_i \leq 1/4$. By Lemma B.2.4, we have $1 - 5x^2 \leq H(1/2 - x) \leq 1 - x^2$ for $x \leq 1/4$. Returning to our setup if $1 - \tau \geq H(1/4)$ and $1 - y_i = H(p_i) \geq 1 - \tau$, and $p_i = 1/2 - x_i$, then $1 - x_i^2 \geq 1 - y_i$, so

$$x_i \leq \sqrt{y_i}. \quad (16.29)$$

Furthermore, $p_1 \circ p_2 = 1/2 - 2x_1x_2$ and

$$\begin{aligned} H(p_1 \circ p_2) &= H(1/2 - 2x_1x_2) \\ &\geq 1 - 20(x_1x_2)^2 \\ &\geq 1 - 20y_1y_2 \\ &= 1 - 20(1 - H(p_1))(1 - H(p_2)) \\ &\geq 1 - 20\tau(1 - H(p_2)), \end{aligned}$$

where the second inequality follows from (16.29). This yields part (4).

16.7 Summary and additional information

In this chapter we showed how a very simple phenomenon leads to a very effective coding and decoding mechanism. Even the idea of reducing error-correction to compression is novel, though perhaps here the novelty is in the realization that this can idea can be put to good use. The idea of using polarization to create a compression scheme, as well as the exact procedure to create polarization are both radically novel, and remarkably effective.

Our description of this compression mechanism is nearly complete. The one omission is that we do not show which columns of the matrix $\mathbf{P}_n$ should be used to produce the compressed output — we only showed that a small subset exists. The reader should know that this aspect can also be achieved effectively, and this was first shown by Tal and Vardy [152], and adapted to the case of strong polarization by Guruswami and Xia. Specifically there is a polynomial time algorithm that given p, ϵ and c outputs $n \leq \text{poly}(1/\epsilon)$, $P_n \in \mathbb{F}_2^{n \times n}$ and a set $S \subseteq [n]$ such that P_n is (ϵ, n^{-c}) -polarizing for $\text{Bern}(p)^n$ with unpredictable columns S , and $|S| \leq (H(p) + \epsilon)n$. The details are not very hard given the work so far, but still out of scope of this chapter.

Our analysis of local polarization differs from the literature in the absence of the use of “Mrs. Gerber’s Lemma” due to Wyner and Ziv, which is a convexity claim that provides a convenient way to deal with conditional entropies (essentially implying that the conditioning can be ignored). In particular, it yields the following statement whose proof can be found as Lemma 2.2 in [36].

Lemma 16.7.1. *If (U, A) and (V, B) are independent and U, V are binary valued random variables with $H(U|A) = H(p)$ and $H(V|B) = H(q)$, then $H(U + V|A, B) \geq H(p(1 - q) + q(1 - p))$.*

The proof of the lemma uses the convexity of the function $H(a \circ H^{-1}(x))$ which turns out to have a short, but delicate and technical proof which led us to omit it here. This lemma would be a much cleaner bridge between the unconditioned polarization statement (Lemma 16.5.10) and its conditional variant (Lemma 16.5.11). Unfortunately Lemma 16.7.1 is known to be true only in the binary case whereas our proof method is applicable to larger alphabets (as shown by Guruswami and Velingker [80]).

16.8 Exercises

Exercise 16.1. *Prove Theorem 16.1.2 (assuming Theorem 16.1.1).*

Exercise 16.2. *Argue that the matrices $\mathbf{G}$ and $\mathbf{G}^*$ in Proposition 16.2.1 exist.*

Exercise 16.3. *Show that for the compressor defined in Algorithm 16.3.1, we have $\mathbf{G} = (P^{-1})_{\bar{S}}$ and $\mathbf{G}^* = \mathbf{P}_{\bar{S}}$.*

Exercise 16.4. *Show that there exists a non-linear compression scheme for $\text{Bern}(p)^n$ of rate at most $H(p) + \varepsilon$.*

Exercise 16.5. *Prove (16.5).*

Exercise 16.6. *Prove (16.9).*

Exercise 16.7. *Show that $H(p)$ is monotonically increasing for $0 \leq p \leq \frac{1}{2}$.*

Exercise 16.8. *Give an explicit description of the polarizing matrix $\mathbf{P}_n$ such that $P_n(\mathbf{Z}) = \mathbf{Z} \cdot \mathbf{P}_n$. Further, prove that $\mathbf{P}_n$ is its own inverse.*

Exercise 16.9. *Show that*

$$b^+(p_1, p_2) = p_1(1 - p_2) + (1 - p_1)p_2.$$

Exercise 16.10. *Show that*

$$b^l(p_1, p_2, 0) = p_1 p_2 / (p_1 p_2 + (1 - p_1)(1 - p_2))$$

and

$$b^l(p_1, p_2, 1) = (1 - p_1)p_2 / ((1 - p_1)p_2 + p_1(1 - p_2)).$$

Exercise 16.11. *Prove Corollary 16.4.7.*

Exercise 16.12. *Prove (16.12).*

Exercise 16.13. *Prove Lemma 16.5.7.*

Exercise 16.14. *Prove Lemma 16.5.8.*

Exercise 16.15. *Prove Lemma 16.5.9.*

Exercise 16.16. *Prove part (2) of Lemma 16.5.11 for the case $H(p) \geq 1 - \tau$.*

Exercise 16.17. *Prove Lemma 16.5.12.*

Exercise 16.18. *Prove Lemma 16.5.13 for the case $X_j > 1/2$.*

Exercise 16.19. *Prove Lemma 16.5.15 when $X_0 \geq 1 - \lambda$.*

Exercise 16.20. *Prove Lemma 16.5.16 when $X_0 > \lambda$.*

Exercise 16.21. *Prove (16.21).*

Exercise 16.22.

$$H'(q) = \log((1 - q)/q).$$

16.9 Bibliographic Notes

Polar codes were invented in the remarkable paper by Arikan [7] where he showed that they achieve capacity in the limit of large block lengths $n \rightarrow \infty$ with $O(n \log n)$ encoding time and $O(n \log n)$ decoding complexity via the successive cancellation decoder. In particular, Arikan proved that the transform $P_2^{\otimes t}$ is polarizing in the limit of $t \rightarrow \infty$, in the sense that for any fixed $\gamma > 0$, the fraction of indices for which $H(\mathbf{W}_i | \mathbf{W}_{<i}) \in (\gamma, 1 - \gamma)$, where $\mathbf{W} = P_2^{\otimes t} \mathbf{Z}$, is vanishing for large t . In fact, Arikan showed that one could take $\gamma = \gamma(t) = 2^{-5t/4}$, which led to an upper bound of $n \cdot \gamma = O(1/n^{1/4})$ (block) decoding error probability for the successive cancellation decoder. Soon afterwards, Arikan and Teletar proved that one can take $\gamma < 2^{-\Omega(2^{\beta t})}$ for any $\beta < 1/2$, which led to improved decoding error probability of $2^{-n^{-\beta}}$ as a function of the block length $n = 2^t$. The fall-off of the parameter γ in n was referred to as the “rate” of (limiting) polarization.

These works considered the basic 2×2 transform P_2 and binary codes. More general transforms, and non-binary codes, were considered later in [136, 102, 116]. These results showed that limiting polarization is universal, as long as some minimal conditions are met by the basic matrix being tensored.

The 2012 survey by Şaşoğlu is an excellent and highly recommended resource for some of the early works on polarization and polar codes [36]. Polar codes were widely described as the first constructive capacity achieving codes. Further, polarization was also found to be a versatile technique to asymptotically resolve several other fundamental problems in information theory such as lossless and lossy source coding problem, coding for broadcast, multiple access, and wiretap channels, etc.

However, none of these works yield effective finite length bounds on the block length n needed to achieve rates within ε of capacity, i.e., a rate at least $1 - h(p) - \varepsilon$ for the binary symmetric channel with crossover probability p . Without this it was not clear in what theoretical sense polar codes are better than say Forney’s construction, which can also get within any desired $\varepsilon > 0$ of capacity, but have complexity growing exponentially in $1/\varepsilon^2$ due to the need for inner codes of length $1/\varepsilon^2$ that are decoded by brute-force.

A finite length analysis of polar codes, and *strong* polarization where the probability of not polarizing falls off exponentially in t , and thus is polynomially small in the block length $n = 2^t$, was established in independent works by Guruswami and Xia [81] and Hassani, Alishahi, and Urbanke [85]. The latter tracked channel “Bhattacharyya parameters” whereas the former tracked conditional entropies (as in the present chapter) which are a bit cleaner to deal with as they form a martingale. This form of fast polarization made polar codes the first, and so far only known, family with block length and complexity scaling polynomially in $1/\varepsilon$ where ε is the gap to capacity,

This analysis of strong polarization in the above works applied only to the 2×2 transform and binary case. The strong polarization of the basic 2×2 transform was also established for all prime alphabets in [80], leading to the first construction of codes achieving the symmetric capacity of all discrete memoryless channels (for prime alphabets) with polynomial complexity in the gap to capacity. However, these analyses relied on rather specific inequalities (which were in particular somewhat painful to establish for the non-binary case) and it was not clear what exactly made them tick.

The recent work of the authors and Błasiok and Nakkiran [11] gave a modular and conceptually clear analysis of strong polarization by abstracting the properties needed from each local step to conclude fast global polarization. This made the demands on the local evolution of the conditional entropies rather minimal and qualitative, and enabled showing strong polarization and polynomially fast convergence to capacity for the *entire* class of polar codes, not just the binary 2×2 case. We followed this approach in this chapter, and in particular borrowed the concepts of variance in the middle and suction at the ends for local polarization from this work. However, we restrict attention to the basic 2×2 transform, and the binary symmetric channel, and gave elementary self-contained proofs of the necessary entropic inequalities needed to establish the properties required of the local polarization step.

Another difference in our presentation is that we described the successive cancellation decoder for the polarizing transform P_2^{*t} , which leads to clean recursive description based on a more general primitive of decoding copies of independent but not necessary identical random variables. In contrast, in many works, including Arkan’s original paper [7], the decoding is described for the transform followed by the bit reversal permutation. The polarization property of the bit reversed transform is, however, notationally simpler to establish. Nevertheless, the transform P_2^{*t} commutes with the bit reversal permutation, so both the transforms, with or without bit reversal, end up having *identical* polarization properties.

Chapter 17

Efficiently Achieving List Decoding Capacity

In the previous chapters, we have seen these results related to list decoding:

- Reed-Solomon codes of rate $R > 0$ can be list-decoded in polynomial time from $1 - \sqrt{R}$ errors (Theorem 12.2.12). This is the best algorithmic list decoding result we have seen so far.
- There exist codes of rate $R > 0$ that are $(1 - R - \varepsilon, O(\frac{1}{\varepsilon}))$ -list decodable for $q \geq 2^{\Omega(\frac{1}{\varepsilon})}$ (and in particular for $q = \text{poly}(n)$) (Theorem 7.4.1 and Proposition 3.3.4). This of course is the best possible combinatorial result.

Note that there is a gap between the algorithmic result and the best possible combinatorial result. This leads to the following natural question:

Question 17.0.1. *Are there explicit codes of rate $R > 0$ that can be list-decoded in polynomial time from $1 - R - \varepsilon$ fraction of errors for $q \leq \text{poly}(n)$?*

In this chapter, we will answer Question 17.0.1 in the affirmative.

17.1 Folded Reed-Solomon Codes

We will now introduce a new family of codes called the Folded Reed-Solomon codes. These codes are constructed by combining every m consecutive symbols of a regular Reed-Solomon code into one symbol from a larger alphabet. Note that we have already seen such a folding trick when we instantiated the outer code in the concatenated code that allowed us to efficiently achieve the BSC_p capacity (Section 15.4.1). For a Reed-Solomon code that maps $\mathbb{F}_q^k \rightarrow \mathbb{F}_q^n$, the corresponding Folded Reed-Solomon code will map $\mathbb{F}_q^k \rightarrow \mathbb{F}_{q^m}^{n/m}$. We will analyze Folded Reed-Solomon codes that are derived from Reed-Solomon codes with evaluation $\{1, \gamma, \gamma^2, \gamma^3, \dots, \gamma^{n-1}\}$, where γ is the generator of $\mathbb{F}_q^*$ and $n \leq q-1$. Note that in the Reed-Solomon code, a message is encoded as in Figure 17.1.

$f(1)$	$f(\gamma)$	$f(\gamma^2)$	$f(\gamma^3)$	...	$f(\gamma^{n-2})$	$f(\gamma^{n-1})$
--------	-------------	---------------	---------------	-----	-------------------	-------------------

Figure 17.1: Encoding $f(X)$ of degree $\leq k-1$ and coefficients in $\mathbb{F}_q$ corresponding to the symbols in the message.

For $m = 2$, the conversion from Reed-Solomon to Folded Reed-Solomon can be visualized as in Figure 17.2 (where we assume n is even).

$f(1)$	$f(\gamma)$	$f(\gamma^2)$	$f(\gamma^3)$	...	$f(\gamma^{n-2})$	$f(\gamma^{n-1})$
--------	-------------	---------------	---------------	-----	-------------------	-------------------

↓

$f(1)$	$f(\gamma^2)$	...	$f(\gamma^{n-2})$
$f(\gamma)$	$f(\gamma^3)$		$f(\gamma^{n-1})$

Figure 17.2: Folded Reed-Solomon code for $m = 2$.

For general $m \geq 1$, this transformation will be as in Figure 17.3 (where we assume that m divides n).

$f(1)$	$f(\gamma)$	$f(\gamma^2)$	$f(\gamma^3)$	...	$f(\gamma^{n-2})$	$f(\gamma^{n-1})$
--------	-------------	---------------	---------------	-----	-------------------	-------------------

↓

$f(1)$	$f(\gamma^m)$	$f(\gamma^{2m})$		$f(\gamma^{n-m})$
$f(\gamma)$	$f(\gamma^{m+1})$	$f(\gamma^{2m+1})$	...	$f(\gamma^{n-m+1})$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
$f(\gamma^{m-1})$	$f(\gamma^{2m-1})$	$f(\gamma^{3m-1})$		$f(\gamma^{n-1})$

Figure 17.3: Folded Reed-Solomon code for general $m \geq 1$.

More formally, here is the definition of Folded Reed-Solomon codes:

Definition 17.1.1 (Folded Reed-Solomon Code). *The m -folded version of an $[n, k]_q$ Reed-Solomon code C (with evaluation points $\{1, \gamma, \dots, \gamma^{n-1}\}$), call it C' , is a code of block length $N = n/m$ over $\mathbb{F}_{q^m}$, where $n \leq q-1$. The encoding of a message $f(X)$, a polynomial over $\mathbb{F}_q$ of degree at most $k-1$, has as its j 'th symbol, for $0 \leq j < n/m$, the m -tuple $(f(\gamma^{jm}), f(\gamma^{jm+1}), \dots, f(\gamma^{jm+m-1}))$. In other words, the codewords of C' are in one-one correspondence with those of the Reed-Solomon code C and are obtained by bundling together consecutive m -tuple of symbols in codewords of C .*

17.1.1 The Intuition Behind Folded Reed-Solomon Codes

We first make the simple observation that the folding trick above cannot *decrease* the list decodability of the code. (We have already seen this argument earlier in Section 15.4.1.)

Claim 17.1.2. *If the Reed-Solomon code can be list-decoded from ρ fraction of errors, then the corresponding Folded Reed-Solomon code with folding parameter m can also be list-decoded from ρ fraction of errors.*

Proof. The idea is simple: If the Reed-Solomon code can be list decoded from ρ fraction of errors (by say an algorithm $\mathcal{A}$), the Folded Reed-Solomon code can be list decoded by “unfolding” the received word and then running $\mathcal{A}$ on the unfolded received word and returning the resulting set of messages. Algorithm 17.1.1 has a more precise statement.

Algorithm 17.1.1 Decoding Folded Reed-Solomon Codes by Unfolding

INPUT: $\mathbf{y} = ((y_{1,1}, \dots, y_{1,m}), \dots, (y_{n/m,1}, \dots, y_{n/m,m})) \in \mathbb{F}_{q^m}^{n/m}$

OUTPUT: A list of messages in $\mathbb{F}_q^k$

1: $\mathbf{y}' \leftarrow (y_{1,1}, \dots, y_{1,m}, \dots, y_{n/m,1}, \dots, y_{n/m,m}) \in \mathbb{F}_q^n$.

2: RETURN $\mathcal{A}(\mathbf{y}')$

The reason why Algorithm 17.1.1 works is simple. Let $\mathbf{m} \in \mathbb{F}_q^k$ be a message. Let $\text{RS}(\mathbf{m})$ and $\text{FRS}(\mathbf{m})$ be the corresponding Reed-Solomon and Folded Reed-Solomon codewords. Now for every $i \in [n/m]$, if $\text{FRS}(\mathbf{m})_i \neq (y_{i,1}, \dots, y_{i,m})$ then in the worst-case for every $j \in [n/m]$, $\text{RS}(\mathbf{m})_{(i-1)n/m+j} \neq y_{i,j}$: i.e. one symbol disagreement over $\mathbb{F}_{q^m}$ can lead to at most m disagreements over $\mathbb{F}_q$. See Figure 17.4 for an illustration.

$f(1)$	$f(\gamma^2)$	...	$f(\gamma^{n-2})$
$f(\gamma)$	$f(\gamma^3)$		$f(\gamma^{n-1})$

↓

$f(1)$	$f(\gamma)$	$f(\gamma^2)$	$f(\gamma^3)$	...	$f(\gamma^{n-2})$	$f(\gamma^{n-1})$
--------	-------------	---------------	---------------	-----	-------------------	-------------------

Figure 17.4: Error pattern after unfolding. A pink cell means an error: for the Reed-Solomon code it is for $\text{RS}(\mathbf{m})$ with $\mathbf{y}'$ and for Folded Reed-Solomon code it is for $\text{FRS}(\mathbf{m})$ with $\mathbf{y}$.

This implies that for any $\mathbf{m} \in \mathbb{F}_q^k$ if $\Delta(\mathbf{y}, \text{FRS}(\mathbf{m})) \leq \rho \cdot \frac{n}{m}$, then $\Delta(\mathbf{y}', \text{RS}(\mathbf{m})) \leq m \cdot \rho \cdot \frac{n}{m} = \rho \cdot n$, which by the properties of algorithm $\mathcal{A}$ implies that Step 2 will output $\mathbf{m}$, as desired. $\square$

The intuition for a *strict* improvement by using Folded Reed-Solomon codes is that if the fraction of errors due to folding increases beyond what it can list-decode from, that error pattern does not need to be handled and can be ignored. For example, suppose a Reed-Solomon

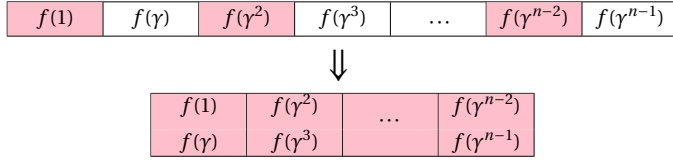


Figure 17.5: An error pattern after folding. The pink cells denotes the location of errors.

code that can be list-decoded from up to $\frac{1}{2}$ fraction of errors is folded into a Folded Reed-Solomon code with $m = 2$. Now consider the error pattern in Figure 17.5.

The error pattern for Reed-Solomon code has $\frac{1}{2}$ fraction of errors, so any list decoding algorithm must be able to list-decode from this error pattern. However, for the Folded Reed-Solomon code the error pattern has 1 fraction of errors which is too high for the code to list-decode from. Thus, this “folded” error pattern case can be discarded from the ones that a list decoding algorithm for Folded Reed-Solomon code needs to consider. This is of course one example— however, it turns out that this folding operation actually rules out *a lot* of error patterns that a list decoding algorithm for Folded Reed-Solomon code needs to handle (even beyond the current best $1 - \sqrt{R}$ bound for Reed-Solomon codes). Put another way, an algorithm for Folded Reed-Solomon codes has to solve the list decoding problem for the Reed-Solomon codes where the error patterns are “bunched” together (technically they’re called *bursty* errors). Of course, converting this intuition into a theorem takes more work and is the subject of this chapter.

Wait a second... The above argument has a potential hole— what if we take the argument to the extreme and “cheat” by setting $m = n$ where *any* error pattern for the Reed-Solomon code will result in an error pattern with 100% errors for the Folded Reed-Solomon code: thus, we will only need to solve the problem of error detection for Reed-Solomon codes (which we can easily solve for any linear code and in particular for Reed-Solomon codes)? It is a valid concern but we will “close the loophole” by only using a constant m as the folding parameter. This will still keep q to be polynomially large in n (recall that the new alphabet size is q^m and let’s assume that q is polynomial in n , which is a common assumption) and thus, we would still be on track to answer Question 17.0.1. Further, if we insist on a smaller list size (e.g. one independent of n), then we can use code concatenation to achieve capacity achieving results for codes over smaller alphabets. (See Section 17.4 for more.)

General Codes. We would like to point out that the folding argument used above is not specific to Reed-Solomon codes. In particular, the argument for the reduction in the number of error patterns holds for any code. In fact, one can prove that for general random codes, with high probability, folding does strictly improve the list decoding capabilities of the original code. (The proof is left as an exercise.)

17.2 List Decoding Folded Reed-Solomon Codes: I

We begin with an algorithm for list decoding Folded Reed-Solomon codes that works with agreement $t \sim mRN$. Note that this is a factor m larger than the RN agreement we ultimately want. In the next section, we will see how to knock off the factor of m .

Before we state the algorithm, we formally (re)state the problem we want to solve (where we want t to be as small as possible):

- **Input:** An agreement parameter $0 \leq t \leq N$ and the received word:

$$\mathbf{y} = \begin{pmatrix} y_0 & y_m & \cdots & y_{n-m} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m-1} & y_{2m-1} & \cdots & y_{n-1} \end{pmatrix} \in \mathbb{F}_q^{m \times N}, \quad N = \frac{n}{m}$$

- **Output:** Return all polynomials $f(X) \in \mathbb{F}_q[X]$ of degree at most $k - 1$ such that for at least t values of $0 \leq i < N$

$$\begin{pmatrix} f(\gamma^{mi}) \\ \vdots \\ f(\gamma^{m(i+1)-1}) \end{pmatrix} = \begin{pmatrix} y_{mi} \\ \vdots \\ y_{m(i+1)-1} \end{pmatrix} \quad (17.1)$$

The algorithm that we will study is a generalization of the Welch-Berlekamp algorithm (Algorithm 12.1.1). However, unlike the previous list decoding algorithms for Reed-Solomon codes (Algorithms 12.2.1, 12.2.2 and 12.2.3), this new algorithm has more similarities with the Welch-Berlekamp algorithm. In particular, for $m = 1$, the new algorithm is *exactly* the Welch-Berlekamp algorithm (since then $Q(X, Y) = A_0(X) + Y A_1(X)$ where A_0 and A_1 correspond to the earlier polynomials $N(X)$ and $E(X)$). Here are the new ideas in the algorithm for the two-step framework that we have seen in the previous chapter:

- **Step 1:** We interpolate using $(m + 1)$ -variate polynomial, $Q(X, Y_1, \dots, Y_m)$, where degree of each variable Y_i is exactly one. In particular, for $m = 1$, this interpolation polynomial is exactly the one used in the Welch-Berlekamp algorithm.
- **Step 2:** As we have done so far, in this step, we output all “roots” of Q . Two remarks are in order. First, unlike Algorithms 12.2.1, 12.2.2 and 12.2.3, the roots $f(X)$ are no longer simpler linear factors $Y - f(X)$, so one cannot use a factorization algorithm to factorize $Q(X, Y_1, \dots, Y_m)$. Second, the new insight in this algorithm, is to show that all the roots form an (affine) subspace,¹ which we can use to compute the roots.

Algorithm 17.2.1 has the details.

¹An affine subspace of $\mathbb{F}_q^k$ is a set $\{\mathbf{v} + \mathbf{u} | \mathbf{u} \in S\}$, where $S \subseteq \mathbb{F}_q^k$ is a linear subspace and $\mathbf{v} \in \mathbb{F}_q^k$.

Algorithm 17.2.1 The First List Decoding Algorithm for Folded Reed-Solomon Codes

INPUT: An agreement parameter $0 \leq t \leq N$, parameter $D \geq 1$ and the received word:

$$\mathbf{y} = \begin{pmatrix} y_0 & y_m & \cdots & y_{n-m} \\ \vdots & \vdots & \ddots & \vdots \\ y_{m-1} & y_{2m-1} & \cdots & y_{n-1} \end{pmatrix} \in \mathbb{F}_q^{m \times N}, \quad N = \frac{n}{m}$$

OUTPUT: All polynomials $f(X) \in \mathbb{F}_q[X]$ of degree at most $k-1$ such that for at least t values of $0 \leq i < N$

$$\begin{pmatrix} f(\gamma^{mi}) \\ \vdots \\ f(\gamma^{m(i+1)-1}) \end{pmatrix} = \begin{pmatrix} y_{mi} \\ \vdots \\ y_{m(i+1)-1} \end{pmatrix} \quad (17.2)$$

1: Compute a non-zero $Q(X, Y_1, \dots, Y_m)$ where

$$Q(X, Y_1, \dots, Y_m) = A_0(X) + A_1(X)Y_1 + A_2(X)Y_2 + \cdots + A_m(X)Y_m$$

with $\deg(A_0) \leq D+k-1$ and $\deg(A_j) \leq D$ for $1 \leq j \leq m$ such that

$$Q(\gamma^{mi}, y_{mi}, \dots, y_{m(i+1)-1}) = 0, \quad \forall 0 \leq i < N \quad (17.3)$$

2: $\mathbb{L} \leftarrow \emptyset$

3: FOR every $f(X) \in \mathbb{F}_q[X]$ such that $Q(X, f(X), f(\gamma X), f(\gamma^2 X), \dots, f(\gamma^{m-1} X)) = 0$ DO

4: IF $\deg(f) \leq k-1$ and $f(X)$ satisfies (17.2) for at least t values of i THEN

5: Add $f(X)$ to $\mathbb{L}$.

6: RETURN $\mathbb{L}$

Correctness of Algorithm 17.2.1. In this section, we will only concentrate on the correctness of the algorithm and analyze its error correction capabilities. We will defer the analysis of the algorithm (and in particular, proving a bound on the number of polynomials that are output by Step 6) till the next section.

We first begin with the claim that there always exists a non-zero choice for Q in Step 1 using the same arguments that we have used to prove the correctness of Algorithms 12.2.2 and 12.2.3:

Claim 17.2.1. *If $(m+1)(D+1)+k-1 > N$, then there exists a non-zero $Q(X, Y_1, \dots, Y_m)$ that satisfies the required properties of Step 1.*

Proof. As in the proof of correctness of Algorithms 12.2.1, 12.2.2 and 12.2.3, we will think of the constraints in (17.3) as linear equations. The variables are the coefficients of $A_i(X)$ for $0 \leq i \leq m$. With the stipulated degree constraints on the $A_i(X)$'s, note that the number of variables participating in (17.3) is

$$D+k+m(D+1) = (m+1)(D+1)+k-1.$$

In the above, we have $D+k$ variables for $A_0(X)$ (since it has degree at most $D+k-1$) and each of the m polynomials $A_j(X)$ for $j \in [m]$ has $D+1$ variables each (since each of them have degree at most D). The number of equations is N . Thus, the condition in the claim implies that we have strictly more variables than equations and thus, there exists a non-zero Q with the required properties. $\square$

Next, we argue that the root finding step works (again using an argument very similar to those that we have seen for Algorithms 12.2.1, 12.2.2 and 12.2.3):

Claim 17.2.2. *If $t > D+k-1$, then every polynomial $f(X) \in \mathbb{F}_q[X]$ of degree at most $k-1$ that agrees with the received word in at least t positions is returned by Step 6.*

Proof. Define the univariate polynomial

$$R(X) = Q(X, f(X), f(\gamma X), \dots, f(\gamma^{m-1} X)).$$

Note that due to the degree constraints on the $A_i(X)$'s and $f(X)$, we have

$$\deg(R) \leq D+k-1,$$

since $\deg(f(\gamma^i X)) = \deg(f(X))$. On the other hand, for every $0 \leq i < N$ where (17.1) is satisfied we have

$$R(\gamma^{mi}) = Q(\gamma^{mi}, y_{mi}, \dots, y_{m(i+1)-1}) = 0,$$

where the first equality follows from (17.1), while the second equality follows from (17.3). Thus, $R(X)$ has at least t roots. Thus, the condition in the claim implies that $R(X)$ has more roots than its degree and thus, by the degree mantra (Proposition 5.1.5) $R(X) \equiv 0$, as desired. $\square$

Note that Claims 17.2.1 and 17.2.2 prove the correctness of the algorithm. Next, we analyze the fraction of errors the algorithm can correct. Note that the condition in Claim 17.2.1 is satisfied if we pick

$$D = \left\lfloor \frac{N-k+1}{m+1} \right\rfloor.$$

This in turn implies that the condition in Claim 17.2.2 is satisfied if

$$t > \frac{N-k+1}{m+1} + k-1 = \frac{N+m(k-1)}{m+1}.$$

Thus, the above would be satisfied if

$$t \geq \frac{N}{m+1} + \frac{mk}{m+1} = N \left(\frac{1}{m+1} + mR \left(\frac{m}{m+1} \right) \right),$$

where the equality follows from the fact that $k = mRN$.

Note that when $m = 1$, the above bound exactly recovers the bound for the Welch-Berlekamp algorithm (Theorem 12.1.6). Thus, we have shown that

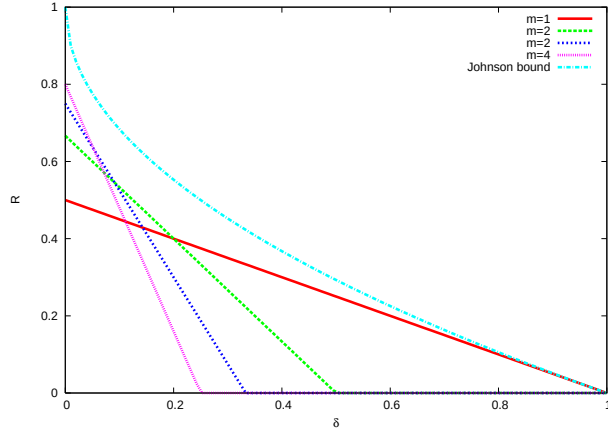


Figure 17.6: The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 17.2.1 for folding parameter $m = 1, 2, 3$ and 4 . The Johnson bound is also plotted for comparison. Also note that the bound for $m = 1$ is the Unique decoding bound achieved by Algorithm 12.1.1.

Theorem 17.2.3. Algorithm 17.2.1 can list decode Folded Reed-Solomon code with folding parameter $m \geq 1$ and rate R up to $\frac{m}{m+1}(1 - mR)$ fraction of errors.

See Figure 17.6 for an illustration of the tradeoff for $m = 1, 2, 3, 4$.

Note that if we can replace the mR factor in the bound from Theorem 17.2.3 by just R then we can approach the list decoding capacity bound of $1 - R$. (In particular, we would be able to correct $1 - R - \epsilon$ fraction of errors if we pick $m = O(1/\epsilon)$.) Further, we need to analyze the number of polynomials output by the root finding step of the algorithm (and then analyze the runtime of the algorithm). In the next section, we show how we can “knock-off” the extra factor m (and we will also bound the list size).

17.3 List Decoding Folded Reed-Solomon Codes: II

In this section, we will present the final version of the algorithm that will allow us to answer Question 17.0.1 in the affirmative. We start off with the new idea that allows us to knock off the factor of m . (It would be helpful to keep the proof of Claim 17.2.2 in mind.)

To illustrate the idea, let us consider the folding parameter to be $m = 3$. Let $f(X)$ be a polynomial of degree at most $k - 1$ that needs to be output and let $0 \leq i < N$ be a position where it agrees with the received word. (See Figure 17.7 for an illustration.)

$$\begin{array}{|c|c|c|} \hline f(\gamma^{3i}) & f(\gamma^{3i+1}) & f(\gamma^{3i+2}) \\ \hline y_{3i} & y_{3i+1} & y_{3i+2} \\ \hline \end{array}$$

Figure 17.7: An agreement in position i .

The idea is to “exploit” this agreement over *one* $\mathbb{F}_q^3$ symbol and convert it into *two* agreements over $\mathbb{F}_{q^2}$. (See Figure 17.8 for an illustration.)

$$\begin{array}{|c|c|} \hline f(\gamma^{3i}) & f(\gamma^{3i+1}) \\ \hline y_{3i} & y_{3i+1} \\ \hline \end{array} \qquad \begin{array}{|c|c|} \hline f(\gamma^{3i+1}) & f(\gamma^{3i+2}) \\ \hline y_{3i+1} & y_{3i+2} \\ \hline \end{array}$$

Figure 17.8: More agreement with a sliding window of size 2.

Thus, in the proof of Claim 17.2.2, for each agreement we can now get *two* roots for the polynomial $R(X)$. In general, for an agreement over one $\mathbb{F}_{q^m}$ symbol translates into $m - s + 1$ agreement over $\mathbb{F}_q^s$ for any $1 \leq s \leq m$ (by ‘sliding a window’ of size s over the m symbols from $\mathbb{F}_q$). Thus, in this new idea the agreement is $m - s + 1$ times more than before which leads to the mR term in Theorem 17.2.3 going down to $\frac{mR}{m-s+1}$. Then making s smaller than m but still large enough we can get down the relative agreement to $R + \epsilon$, as desired. There is another change that needs to be done to make the argument go through: the interpolation polynomial Q now has to be $(s + 1)$ -variate instead of the earlier $(m + 1)$ -variate polynomial. Algorithm 17.3.1 has the details.

Correctness of Algorithm 17.3.1. Next, we analyze the correctness of Algorithm 17.3.1 as well as compute its list decoding error bound. We begin with the result showing that there exists a Q with the required properties for Step 1.

Lemma 17.3.1. If $D \geq \left\lfloor \frac{N(m-s+1)-k+1}{s+1} \right\rfloor$, then there exists a non-zero polynomial $Q(X, Y_1, \dots, Y_s)$ that satisfies Step 1 of the above algorithm.

Proof. Let us consider all coefficients of all polynomials A_i as variables. Then using the same argument (but with s instead of m) as in proof of Claim 17.2.1 the number of variables will be

$$D + k + s(D + 1) = (s + 1)(D + 1) + k - 1.$$

On the other hand, the number of constraints in (17.5), i.e. the number of equations when all coefficients of all polynomials A_i are considered variables) will be $N(m - s + 1)$.

Note that if we have more variables than equations, then there exists a non-zero Q that satisfies the required properties of Step 1. Thus, we would be done if we have:

$$(s + 1)(D + 1) + k - 1 > N(m - s + 1),$$

Algorithm 17.3.1 The Second List Decoding Algorithm for Folded Reed-Solomon Codes

INPUT: An agreement parameter $0 \leq t \leq N$, parameter $D \geq 1$ and the received word:

$$\mathbf{y} = \begin{pmatrix} y_0 & y_m & y_{n-m} \\ \vdots & \vdots & \vdots \\ y_{m-1} & y_{2m-1} & y_{n-1} \end{pmatrix} \in \mathbb{F}_q^{m \times N}, \quad N = \frac{n}{m}$$

OUTPUT: All polynomials $f(X) \in \mathbb{F}_q[X]$ of degree at most $k-1$ such that for at least t values of $0 \leq i < N$

$$\begin{pmatrix} f(\gamma^{mi}) \\ \vdots \\ f(\gamma^{m(i+1)-1}) \end{pmatrix} = \begin{pmatrix} y_{mi} \\ \vdots \\ y_{m(i+1)-1} \end{pmatrix} \quad (17.4)$$

1: Compute non-zero polynomial $Q(X, Y_1, \dots, Y_s)$ as follows:

$$Q(X, Y_1, \dots, Y_s) = A_0(X) + A_1(X)Y_1 + A_2(X)Y_2 + \dots + A_s(X)Y_s,$$

with $\deg[A_0] \leq D+k-1$ and $\deg[A_i] \leq D$ for every $1 \leq i \leq s$ such that for all $0 \leq i < N$ and $0 \leq j \leq m-s$, we have

$$Q(\gamma^{im+j}, y_{im+j}, \dots, y_{im+j+s-1}) = 0. \quad (17.5)$$

2: $\mathbb{L} \leftarrow \emptyset$

3: FOR every $f(X) \in \mathbb{F}_q[X]$ such that

$$Q(X, f(X), f(\gamma X), f(\gamma^2 X), \dots, f(\gamma^{s-1} X)) \equiv 0 \quad (17.6)$$

DO

4: IF $\deg(f) \leq k-1$ and $f(X)$ satisfies (17.2) for at least t values of i THEN

5: Add $f(X)$ to $\mathbb{L}$.

6: RETURN $\mathbb{L}$

which is equivalent to:

$$D > \frac{N(m-s+1) - k + 1}{s+1} - 1.$$

The choice of D in the statement of the claim satisfies the condition above, which complete the proof. $\square$

Next we argue that the root finding step works.

Lemma 17.3.2. If $t > \frac{D+k-1}{m-s+1}$, then every polynomial $f(X)$ that needs to be output satisfies (17.6).

Proof. Consider the polynomial $R(X) = Q(X, f(X), f(\gamma X), \dots, f(\gamma^{s-1} X))$. Because the degree of

$f(\gamma^\ell X)$ (for every $0 \leq \ell \leq s-1$) is at most $k-1$,

$$\deg(R) \leq D + k - 1. \quad (17.7)$$

Let $f(X)$ be one of the polynomials of degree at most $k-1$ that needs to be output, and $f(X)$ agrees with the received word at column i for some $0 \leq i < N$, that is:

$$\begin{pmatrix} f(\gamma^{mi}) \\ f(\gamma^{mi+1}) \\ \vdots \\ f(\gamma^{m(i+1)-1}) \end{pmatrix} = \begin{pmatrix} y_{mi} \\ y_{mi+1} \\ \vdots \\ y_{m(i+1)-1} \end{pmatrix},$$

then for all $0 \leq j \leq m-s$, we have:

$$\begin{aligned} R(\gamma^{mi+j}) &= Q(\gamma^{mi+j}, f(\gamma^{mi+j}), f(\gamma^{mi+1+j}), \dots, f(\gamma^{mi+s-1+j})) \\ &= Q(\gamma^{mi+j}, y_{mi+j}, y_{mi+1+j}, \dots, y_{mi+s-1+j}) = 0. \end{aligned}$$

In the above, the first equality follows as $f(X)$ agrees with $\mathbf{y}$ in column i while the second equality follows from (17.5). Thus, the number of roots of $R(X)$ is at least

$$t(m-s+1) > D + k - 1 \geq \deg(R),$$

where the first inequality follows from the assumption in the claim and the second inequality follows from (17.7). Hence, by the degree mantra $R(X) \equiv 0$, which shows that $f(X)$ satisfies (17.6), as desired. $\square$

17.3.1 Error Correction Capability

Now, we analyze the fraction of errors the algorithm above can handle. (We will come back to the thorny issue of proving a bound on the output list size for the root finding step in Section 17.3.2.)

The argument for the fraction of errors follows the by now standard route. To satisfy the constraint in Lemma 17.3.1, we pick

$$D = \left\lfloor \frac{N(m-s+1) - k + 1}{s+1} \right\rfloor.$$

This along with the constraint in Lemma 17.3.2, implies that the algorithm works as long as

$$t > \left\lfloor \frac{D+k-1}{m-s+1} \right\rfloor.$$

The above is satisfied if we choose

$$t > \frac{\frac{N(m-s+1)-k+1}{s+1} + k - 1}{m - s + 1} = \frac{N(m-s+1) - k + 1 + (k-1)(s+1)}{(m-s+1)(s+1)} = \frac{N(m-s+1) + s(k-1)}{(s+1)(m-s+1)}.$$

Thus, we would be fine if we pick

$$t > \frac{N}{s+1} + \frac{s}{s+1} \cdot \frac{k}{m-s+1} = N \left(\frac{1}{s+1} + \left(\frac{s}{s+1} \right) \left(\frac{m}{m-s+1} \right) \cdot R \right),$$

where the equality follows from the fact that $k = mRN$. This implies the following result:

Theorem 17.3.3. *Algorithm 17.3.1 can list decode Folded Reed-Solomon code with folding parameter $m \geq 1$ and rate R up to $\frac{s}{s+1}(1 - mR/(m-s+1))$ fraction of errors.*

See Figure 17.9 for an illustration of the bound above.

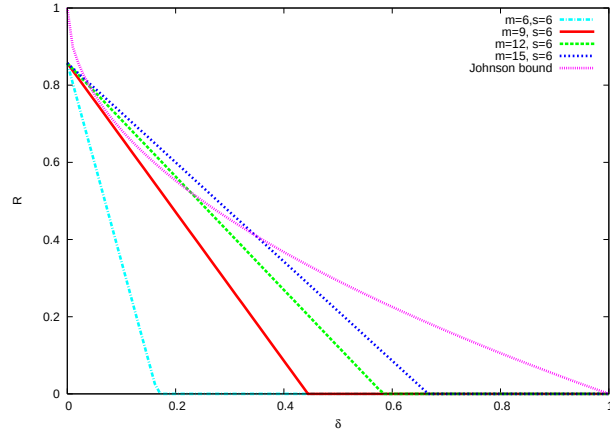


Figure 17.9: The tradeoff between rate R and the fraction of errors that can be corrected by Algorithm 17.3.1 for $s = 6$ and folding parameter $m = 6, 9, 12$ and 15 . The Johnson bound is also plotted for comparison.

17.3.2 Bounding the Output List Size

We finally address the question of bounding the output list size in the root finding step of the algorithm. We will present a proof that will immediately lead to an algorithm to implement the

root finding step. We will show that there are at most q^{s-1} possible solutions for the root finding step.

The main idea is the following: think of the coefficients of the output polynomial $f(X)$ as variables. Then the constraint (17.6) implies $D+k$ linear equations on these k variables. It turns out that if one picks only k out of these $D+k$ constraints, then the corresponding constraint matrix has rank at least $k-s+1$, which leads to the claimed bound. Finally, the claim on the rank of the constraint matrix follows by observing (and this is the crucial insight) that the constraint matrix is upper triangular. Further, the diagonal elements are evaluation of a non-zero polynomial of degree at most $s-1$ in k distinct elements. By the degree mantra (Proposition 5.1.5), this polynomial can have at most $s-1$ roots, which implies that at least $k-s+1$ elements of the diagonal are non-zero, which then implies the claim. See Figure 17.10 for an illustration of the upper triangular system of linear equations.

Figure 17.10: The system of linear equations with the variables $f_0, \dots, f_{k-1}$ forming the coefficients of the polynomial $f(X) = \sum_{i=0}^{k-1} f_i X^i$ that we want to output. The constants $a_{j,0}$ are obtained from the interpolating polynomial from Step 1. $B(X)$ is a non-zero polynomial of degree at most $s-1$.

Next, we present the argument above in full detail. (Note that the constraint on (17.8) is the same as the one in (17.6) because of the constraint on the structure of Q imposed by Step 1.)

Lemma 17.3.4. *There are at most q^{s-1} solutions to $f_0, f_1, \dots, f_{k-1}$ (where $f(X) = f_0 + f_1 X + \dots + f_{k-1} X^{k-1}$) to the equations*

$$A_0(X) + A_1(X)f(X) + A_2(X)f(\gamma X) + \dots + A_s(X)f(\gamma^{s-1}X) \equiv 0 \quad (17.8)$$

Proof. First, we assume that X does not divide all of the polynomials $A_0, A_1, \dots, A_s$. Then it implies that there exists $i^* > 0$ such that the constant term of the polynomial $A_{i^*}(X)$ is not zero. (Otherwise, since $X|A_1(X), \dots, A_s(X)$, by (17.8), we have X divides $A_0(X)$ and hence X divide all the $A_i(X)$ polynomials, which contradicts the assumption.)

To facilitate the proof, we define few auxiliary variables a_{ij} such that

$$A_i(X) = \sum_{j=0}^{D+k-1} a_{ij} X^j \text{ for every } 0 \leq i \leq s,$$

and define the following univariate polynomial:

$$B(X) = a_{1,0} + a_{2,0}X + a_{3,0}X^2 + \dots + a_{s,0}X^{s-1}. \quad (17.9)$$

Notice that $a_{i^*,0} \neq 0$, so $B(X)$ is non-zero polynomial. And because degree of $B(X)$ is at most $s-1$, by the degree mantra (Proposition 5.1.5), $B(X)$ has at most $s-1$ roots. Next, we claim the following:

Claim 17.3.5. For every $0 \leq j \leq k-1$:

- If $B(\gamma^j) \neq 0$, then f_j is uniquely determined by $f_{j-1}, f_{j-2}, \dots, f_0$.
- If $B(\gamma^j) = 0$, then f_j is unconstrained, i.e. f_j can take any of the q values in $\mathbb{F}_q$.

We defer the proof of the claim above for now. Suppose that the above claim is correct. Then as γ is a generator of $\mathbb{F}_q$, $1, \gamma, \gamma^2, \dots, \gamma^{k-1}$ are distinct (since $k-1 \leq q-2$). Further, by the degree mantra (Proposition 5.1.5) at most $s-1$ of these elements are roots of the polynomial $B(X)$. Therefore by Claim 17.3.5, the number of solutions to $f_0, f_1, \dots, f_{k-1}$ is at most q^{s-1} .²

We are almost done except we need to remove our earlier assumption that X does not divide every A_i . Towards this end, we essentially just factor out the largest common power of X from all of the A_i 's, and proceed with the reduced polynomial. Let $l \geq 0$ be the largest l such that $A_i(X) = X^l A'_i(X)$ for $0 \leq i \leq s$; then X does not divide all of $A'_i(X)$ and we have:

$$X^l (A'_0(X) + A'_1(X)f(X) + \dots + A'_s(X)f(\gamma^{s-1}X)) \equiv 0.$$

Thus, we can do the entire argument above by replacing $A_i(X)$ with $A'_i(X)$ since the above constraint implies that $A'_i(X)$'s also satisfy (17.8). $\square$

Next we prove Claim 17.3.5.

Proof of Claim 17.3.5. Recall that we can assume that X does not divide all of $\{A_0(X), \dots, A_s(X)\}$.

²Build a "decision tree" with f_0 as the root and f_j in the j th level: each edge is labeled by the assigned value to the parent node variable. For any internal node in the j th level, if $B(\gamma^j) \neq 0$, then the node has a single child with the edge taking the unique value promised by Claim 17.3.5. Otherwise the node has q children with q different labels from $\mathbb{F}_q$. By Claim 17.3.5, the number of solutions to $f(X)$ is upper bounded by the number of nodes in the k th level in the decision tree, which by the fact that B has at most $s-1$ roots is upper bounded by q^{s-1} .

Let $C(X) = A_0(X) + A_1(X)f(X) + \dots + A_s f(\gamma^{s-1}X)$. Recall that we have $C(X) \equiv 0$. If we expand out each polynomial multiplication, we have:

$$\begin{aligned} C(X) &= a_{0,0} + a_{0,1}X + \dots + a_{0,D+k-1}X^{D+k-1} \\ &+ \left(a_{1,0} + a_{1,1}X + \dots + a_{1,D+k-1}X^{D+k-1} \right) \left(f_0 + f_1X + f_2X^2 + \dots + f_{k-1}X^{k-1} \right) \\ &+ \left(a_{2,0} + a_{2,1}X + \dots + a_{2,D+k-1}X^{D+k-1} \right) \left(f_0 + f_1\gamma X + f_2\gamma^2X^2 + \dots + f_{k-1}\gamma^{k-1}X^{k-1} \right) \\ &\vdots \\ &+ \left(a_{s,0} + a_{s,1}X + \dots + a_{s,D+k-1}X^{D+k-1} \right) \left(f_0 + f_1\gamma^{s-1}X + f_2\gamma^{2(s-1)}X^2 + \dots + f_{k-1}\gamma^{(k-1)(s-1)}X^{k-1} \right) \end{aligned} \quad (17.10)$$

Now if we collect terms of the same degree, we will have a polynomial of the form:

$$C(X) = c_0 + c_1X + c_2X^2 + \dots + c_{D+k-1}X^{D+k-1}.$$

So we have $D+k$ linear equations in variables $f_0, \dots, f_{k-1}$, and we are seeking those solutions such that $c_j = 0$ for every $0 \leq j \leq D+k-1$. We will only consider the $0 \leq j \leq k-1$ equations. We first look at the equation for $j=0$: $c_0 = 0$. This implies the following equalities:

$$0 = a_{0,0} + f_0 a_{1,0} + f_0 a_{2,0} + \dots + f_0 a_{s,0} \quad (17.11)$$

$$0 = a_{0,0} + f_0 (a_{1,0} + a_{2,0} + \dots + a_{s,0}) \quad (17.12)$$

$$0 = a_{0,0} + f_0 B(1). \quad (17.13)$$

In the above (17.11) follows from (17.10), (17.12) follows by simple manipulation while (17.13) follows from the definition of $B(X)$ in (17.9).

Now, we have two possible cases:

- **Case 1:** $B(1) \neq 0$. In this case, (17.13) implies that $f_0 = \frac{-a_{0,0}}{B(1)}$. In particular, f_0 is fixed.
- **Case 2:** $B(1) = 0$. In this case f_0 has no constraint (and hence can take on any of the q values in $\mathbb{F}_q$).

Now consider the equation for $j=1$: $c_1 = 0$. Using the same argument as we did for $j=0$, we obtain the following sequence of equalities:

$$\begin{aligned} 0 &= a_{0,1} + f_1 a_{1,1} + f_0 a_{2,1} + f_1 a_{2,0}\gamma + f_0 a_{2,1} + \dots + f_1 a_{s,0}\gamma^{s-1} + f_0 a_{s,1} \\ 0 &= a_{0,1} + f_1 (a_{1,1} + a_{2,0}\gamma + \dots + a_{s,0}\gamma^{s-1}) + f_0 \left(\sum_{i=1}^s a_{i,1} \right) \\ 0 &= a_{0,1} + f_1 B(\gamma) + f_0 b_0^{(1)} \end{aligned} \quad (17.14)$$

where $b_0^{(1)} = \sum_{i=1}^s a_{i,1}$ is a constant. We have two possible cases:

- **Case 1:** $B(\gamma) \neq 0$. In this case, by (17.14), we have $f_1 = \frac{-a_{0,1} - f_0 b_0^{(1)}}{B(\gamma)}$ and there is a unique choice for f_1 given fixed f_0 .
- **Case 2:** $B(\gamma) = 0$. In this case, f_1 is unconstrained.

Now consider the case of arbitrary j : $c_j = 0$. Again using similar arguments as above, we get:

$$\begin{aligned}
0 &= a_{0,j} + f_j(a_{1,0} + a_{2,0}\gamma^j + a_{3,0}\gamma^{2j} + \dots + a_{s,0}\gamma^{j(s-1)}) \\
&\quad + f_{j-1}(a_{1,1} + a_{2,1}\gamma^{j-1} + a_{3,1}\gamma^{2(j-1)} + \dots + a_{s,1}\gamma^{(j-1)(s-1)}) \\
&\quad \vdots \\
&\quad + f_1(a_{1,j-1} + a_{2,j-1}\gamma + a_{3,j-1}\gamma^2 + \dots + a_{s,j-1}\gamma^{s-1}) \\
&\quad + f_0(a_{1,j} + a_{2,j} + a_{3,j} + \dots + a_{s,j}) \\
0 &= a_{0,j} + f_j B(\gamma^j) + \sum_{l=0}^{j-1} f_l b_l^{(j)}
\end{aligned} \tag{17.15}$$

where $b_l^{(j)} = \sum_{i=1}^s a_{i,j-l} \cdot \gamma^{l(i-1)}$ are constants for $0 \leq j \leq k-1$.

We have two possible cases:

- **Case 1:** $B(\gamma^j) \neq 0$. In this case, by (17.15), we have

$$f_j = \frac{-a_{0,j} - \sum_{l=0}^{j-1} f_l b_l^{(j)}}{B(\gamma^j)} \tag{17.16}$$

and there is a unique choice for f_j given fixed $f_0, \dots, f_{j-1}$.

- **Case 2:** $B(\gamma^j) = 0$. In this case f_j is unconstrained.

This completes the proof. $\square$

We now revisit the proof above and make some algorithmic observations. First, we note that to compute all the tuples $(f_0, \dots, f_{k-1})$ that satisfy (17.8) one needs to solve the linear equations (17.15) for $j = 0, \dots, k-1$. One can state this system of linear equation as (see also Figure 17.10)

$$C \cdot \begin{pmatrix} f_0 \\ \vdots \\ f_{k-1} \end{pmatrix} = \begin{pmatrix} -a_{0,k-1} \\ \vdots \\ -a_{0,0} \end{pmatrix},$$

where C is a $k \times k$ upper triangular matrix. Further each entry in C is either a 0 or $B(\gamma^j)$ or $b_l^{(j)}$ – each of which can be computed in $O(s \log s)$ operations over $\mathbb{F}_q$. Thus, we can setup this system of equations in $O(s \log s k^2)$ operations over $\mathbb{F}_q$.

Next, we make the observation that all the solutions to (17.8) form an affine subspace. Let $0 \leq d \leq s-1$ denote the number of roots of $B(X)$ in $\{1, \gamma, \dots, \gamma^{k-1}\}$. Then since there will be d unconstrained variables among $f_0, \dots, f_{k-1}$ (one of every j such that $B(\gamma^j) = 0$), it is not too

hard to see that all the solutions will be in the set $\{M \cdot \mathbf{x} + \mathbf{z} \mid \mathbf{x} \in \mathbb{F}_q^d\}$, for some $k \times d$ matrix M and some $\mathbf{z} \in \mathbb{F}_q^k$. Indeed every $\mathbf{x} \in \mathbb{F}_q^d$ corresponds to an assignment to the d unconstrained variables among $f_0, \dots, f_j$. The matrix M and the vector $\mathbf{z}$ are determined by the equations in (17.16). Further, since C is upper triangular, both M and $\mathbf{z}$ can be computed with $O(k^2)$ operations over $\mathbb{F}_q$.

The discussion above implies the following:

Corollary 17.3.6. *The set of solutions to (17.8) are contained in an affine subspace $\{M \cdot \mathbf{x} + \mathbf{z} \mid \mathbf{x} \in \mathbb{F}_q^d\}$ for some $0 \leq d \leq s-1$ and $M \in \mathbb{F}_q^{k \times d}$ and $\mathbf{z} \in \mathbb{F}_q^k$. Further, M and $\mathbf{z}$ can be computed from the polynomials $A_0(X), \dots, A_s(X)$ with $O(s \log s k^2)$ operations over $\mathbb{F}_q$.*

17.3.3 Algorithm Implementation and Runtime Analysis

In this sub-section, we discuss how both the interpolation and root finding steps of the algorithm can be implemented and analyze the run time of each step.

Step 1 involves solving Nm linear equation in $O(Nm)$ variables and can e.g. be solved by Gaussian elimination in $O((Nm)^3)$ operations over $\mathbb{F}_q$. This is similar to what we have seen for Algorithms 12.2.1, 12.2.2 and 12.2.3. However, the fact that the interpolation polynomial has total degree of one in the variables $Y_1, \dots, Y_s$ implies a much faster algorithm. In particular, one can perform the interpolation in $O(Nm \log^2(Nm) \log \log(Nm))$ operations over $\mathbb{F}_q$.

The root finding step involves computing all the “roots” of Q . The proof of Lemma 17.3.4 actually suggests Algorithm 17.3.2.

Algorithm 17.3.2 The Root Finding Algorithm for Algorithm 17.3.1

INPUT: $A_0(X), \dots, A_s(X)$

OUTPUT: All polynomials $f(X)$ of degree at most $k-1$ that satisfy (17.8)

- 1: Compute ℓ such that X^ℓ is the largest common power of X among $A_0(X), \dots, A_s(X)$.
 - 2: FOR every $0 \leq i \leq s$ DO
 - 3: $A_i(X) \leftarrow \frac{A_i(X)}{X^\ell}$.
 - 4: Compute $B(X)$ according to (17.9)
 - 5: Compute d , $\mathbf{z}$ and M such that the solutions to the k linear system of equations in (17.15) lie in the set $\{M \cdot \mathbf{x} + \mathbf{z} \mid \mathbf{x} \in \mathbb{F}_q^d\}$.
 - 6: $\mathbb{L} \leftarrow \emptyset$
 - 7: FOR every $\mathbf{x} \in \mathbb{F}_q^d$ DO
 - 8: $(f_0, \dots, f_{k-1}) \leftarrow M \cdot \mathbf{x} + \mathbf{z}$.
 - 9: $f(X) \leftarrow \sum_{i=0}^{k-1} f_i \cdot X^i$.
 - 10: IF $f(X)$ satisfies (17.8) THEN
 - 11: Add $f(X)$ to $\mathbb{L}$.
 - 12: RETURN $\mathbb{L}$
-

Next, we analyze the run time of the algorithm. Throughout, we will assume that all polynomials are represented in their standard coefficient form.

Step 1 just involves figuring out the smallest power of X in each $A_i(X)$ that has a non-zero coefficient from which one can compute the value of ℓ . This can be done with $O(D + k + s(D + 1)) = O(Nm)$ operations over $\mathbb{F}_q$. Further, given the value of ℓ one just needs to “shift” all the coefficients in each of the $A_i(X)$ ’s to the right by ℓ , which again can be done with $O(Nm)$ operations over $\mathbb{F}_q$.

Now we move to the root finding step. The run time actually depends on what it means to “solve” the linear system. If one is happy with a succinct description of a set of possible solution that contains the actual output then one can halt Algorithm 17.3.2 after Step 5 and Corollary 17.3.6 implies that this step can be implemented in $O(s \log s k^2) = O(s \log s (Nm)^2)$ operations over $\mathbb{F}_q$. However, if one wants the actual set of polynomials that need to be output, then the only known option so far is to check all the q^{s-1} potential solutions as in Steps 7–11. (However, we’ll see a twist in Section 17.4.) The latter would imply a total of $O(s \log s (Nm)^2) + O(q^{s-1} \cdot (Nm)^2)$ operations over $\mathbb{F}_q$.

Thus, we have the following result:

Lemma 17.3.7. *With $O(s \log s (Nm)^2)$ operations over $\mathbb{F}_q$, the algorithm above can return an affine subspace of dimension $s - 1$ that contains all the polynomials of degree at most $k - 1$ that need to be output. Further, the exact set of solution can be computed in with additional $O(q^{s-1} \cdot (Nm)^2)$ operations over $\mathbb{F}_q$.*

17.3.4 Wrapping Up

By Theorem 17.3.3, we know that we can list decode a Folded Reed-Solomon code with folding parameter $m \geq 1$ up to

$$\frac{s}{s+1} \cdot \left(1 - \frac{m}{m-s+1} \cdot R\right) \quad (17.17)$$

fraction of errors for any $1 \leq s \leq m$.

To obtain our desired bound $1 - R - \varepsilon$ fraction of errors, we instantiate the parameter s and m such that

$$\frac{s}{s+1} \geq 1 - \varepsilon \text{ and } \frac{m}{m-s+1} \leq 1 + \varepsilon. \quad (17.18)$$

It is easy to check that one can choose

$$s = \Theta(1/\varepsilon) \text{ and } m = \Theta(1/\varepsilon^2)$$

such that the bounds in (17.18) are satisfied. Using the bounds from (17.18) in (17.17) implies that the algorithm can handle at least

$$(1 - \varepsilon)(1 - (1 + \varepsilon)R) = 1 - \varepsilon - R + \varepsilon^2 R > 1 - R - \varepsilon$$

fraction of errors, as desired.

We are almost done since Lemma 17.3.7 shows that the run time of the algorithm is $q^{O(s)}$. The only thing we need to choose is q : for the final result we pick q to be the smallest power

of 2 that is larger than $Nm + 1$. Then the discussion above along with Lemma 17.3.7 implies the following result (the claim on strong explicitness follows from the fact that Reed-Solomon codes are strongly explicit):

Theorem 17.3.8. *There exist strongly explicit Folded Reed-Solomon codes of rate R that for large enough block length N can be list decoded from $1 - R - \varepsilon$ fraction of errors (for any small enough $\varepsilon > 0$) in time $\left(\frac{N}{\varepsilon}\right)^{O(1/\varepsilon)}$. The worst-case list size is $\left(\frac{N}{\varepsilon}\right)^{O(1/\varepsilon)}$ and the alphabet size is $\left(\frac{N}{\varepsilon}\right)^{O(1/\varepsilon^2)}$.*

17.4 Bibliographic Notes and Discussion

There was no improvement to the Guruswami-Sudan result (Theorem 12.2.12) for about seven years till Parvaresh and Vardy showed that “Correlated” Reed-Solomon codes can be list-decoded up to $1 - (mR)^{\frac{1}{m+1}}$ fraction of errors for $m \geq 1$ [122]. Note that for $m = 1$, correlated Reed-Solomon codes are equivalent to Reed-Solomon codes and the result of Parvaresh and Vardy recovers Theorem 12.2.12. Immediately, after that Guruswami and Rudra [74] showed that Folded Reed-Solomon codes can achieve the list decoding capacity of $1 - R - \varepsilon$ and hence, answer Question 17.0.1 in the affirmative. Guruswami [68] reproved this result but with a much simpler proof. In this chapter, we studied the proof due to Guruswami. Guruswami in [68] credits Salil Vadhan for the interpolation step. An algorithm presented in Brander’s thesis [22] shows that for the special interpolation in Algorithm 17.3.1, one can perform the interpolation in $O(Nm \log^2(Nm) \log \log(Nm))$ operations over $\mathbb{F}_q$. The idea of using the “sliding window” for list decoding Folded Reed-Solomon codes is originally due to Guruswami and Rudra [73].

The bound of q^{s-1} on the list size for Folded Reed-Solomon codes was first proven in [73] by roughly the following argument. One reduced the problem of finding roots to finding roots of a *univariate* polynomial related to Q over $\mathbb{F}_{q^k}$. (Note that each polynomial in $\mathbb{F}_q[X]$ of degree at most $k - 1$ has a one to one correspondence with elements of $\mathbb{F}_{q^k}$ —see e.g. Theorem 14.2.1.) The list size bound follows from the fact that this new univariate polynomial had degree q^{s-1} . Thus, implementing the algorithm entails running a root finding algorithm over a big extension field, which in practice has terrible performance.

Discussion. For constant ε , Theorem 17.3.8 answers Question 17.0.1 in the affirmative. However, from a practical point of view, there are three issues with the result: alphabet, list size and run time. Below we tackle each of these issues.

Large Alphabet. Recall that one only needs an alphabet of size $2^{O(1/\varepsilon)}$ to be able to list decode from $1 - R - \varepsilon$ fraction of errors, which is independent of N . It turns out that combining Theorem 17.3.8 along with code concatenation and expanders allows us to construct codes over alphabets of size roughly $2^{O(1/\varepsilon^4)}$ [73]. (The idea of using expanders and code concatenation was not new to [73]: the connection was exploited in earlier work by Guruswami and Indyk [72].)

The above however, does not answer the question of achieving list decoding capacity for fixed q , say e.g. $q = 2$. We know that there exists binary code of rate R that are $(H^{-1}(1 - R -$

ϵ), $O(1/\epsilon)$ -list decodable codes (see Theorem 7.4.1). The best known explicit codes with efficient list decoding algorithms are those achieved by concatenating Folded Reed-Solomon codes with suitable inner codes achieve the so called *Blokh-Zyablov* bound [75]. However, the tradeoff is far from the list decoding capacity. As one sample point, consider the case when we want to list decode from $\frac{1}{2} - \epsilon$ fraction of errors. Then the result of [75] gives codes of rate $\Theta(\epsilon^3)$ while the codes on list decoding capacity has rate $\Omega(\epsilon^2)$. The following fundamental question is still very much wide open:

Open Question 17.4.1. *Do there exist explicit binary codes with rate R that can be list decoded from $H^{-1}(1 - R - \epsilon)$ fraction of errors with polynomial list decoding algorithms?*

The above question is open even if we drop the requirement on efficient list decoding algorithm or we only ask for a code that can list decode from $1/2 - \epsilon$ fraction of errors with rate $\Omega(\epsilon^a)$ for some $a < 3$. It is known (combinatorially) that concatenated codes can achieve the list decoding capacity but the result is via a souped up random coding argument and does not give much information about an efficient decoding algorithm [76].

List Size. It is natural to wonder if the bound on the list size in Lemma 17.3.4 above can be improved as that would show that Folded Reed-Solomon codes can be list decoded up to the list decoding capacity but with a smaller output list size than Theorem 17.3.8. Guruswami showed that in its full generality the bound cannot be improved [68]. In particular, he exhibits explicit polynomials $A_0(X), \dots, A_s(X)$ such that there are at least q^{s-2} solutions for $f(X)$ that satisfy (17.8). However, these $A_i(X)$'s are not known to be the output for an actual interpolation instance. In other words, the following question is still open:

Open Question 17.4.2. *Can Folded Reed-Solomon codes of rate R be list decoded from $1 - R - \epsilon$ fraction of errors with list size $f(1/\epsilon) \cdot N^c$ for some increasing function $f(\cdot)$ and absolute constant c ?*

Even the question above with $N^{(1/\epsilon)^{o(1)}}$ is still open.

However, if one is willing to consider codes other than Folded Reed-Solomon codes in order to answer to achieve list decoding capacity with smaller list size (perhaps with one only dependent on ϵ), then there is good news. Guruswami in the same paper that presented the algorithm in this chapter also present a *randomized* construction of codes of rate R that are $(1 - R - \epsilon, O(1/\epsilon^2))$ -list decodable codes [68]. This is of course worse than what we know from the probabilistic method. However, the good thing about the construction of Guruswami comes with an $O(N/\epsilon)^{O(1/\epsilon)}$ -list decoding algorithm.

Next we briefly mention the key ingredient in the result above. To see the potential for improvement consider Corollary 17.3.6. The main observation is that all the potential solutions lie in an affine subspace of dimension $s - 1$. The key idea in [68] was use the Folded Reed-

Solomon encoding for a special subset of the message space $\mathbb{F}_q^k$. Call a subspace $S \subseteq \mathbb{F}_q^k$ to be a $(q, k, \epsilon, \ell, L)$ -subspace evasive subset if

1. $|S| \geq q^{k(1-\epsilon)}$; and
2. For any (affine) subspace $T \subseteq \mathbb{F}_q^k$ of dimension ℓ , we have $|S \cap T| \leq L$.

The code in [68], applies the Folded Reed-Solomon encoding on a $(q, k, s, O(s^2))$ -subspace evasive subset (such a subset can be shown to exist via the probabilistic method). The reason why this sub-code of Folded Reed-Solomon code works is as follows: Condition (1) ensures that the new code has rate at least $R(1 - \epsilon)$, where R is the rate of the original Folded Reed-Solomon code and condition (2) ensures that the number of output polynomial in the root finding step of the algorithm we considered in the last section is at most L . (This is because by Corollary 17.3.6 the output message space is an affine subspace of dimension $s - 1$ in $\mathbb{F}_Q^k$. However, in the new code by condition 2, there can be at most $O(s^2)$ output solutions.)

The result above however, has two shortcomings: (i) the code is no longer explicit and (ii) even though the worst case list size is $O(\frac{1}{\epsilon^2})$, it was not know how to obtain this output without listing all the q^{s-1} possibilities and pruning them against S . The latter meant that the decoding runtime did not improve over the one achieved in Theorem 17.3.8.

Large Runtime. We finally address the question of the high run time of all the list decoding algorithms so far. Dvir and Lovett [44], presented a construction of an *explicit* $(q, k, \epsilon, s, s^{O(s)})$ -subspace evasive subset S^* . More interestingly, given any affine subspace T of dimension at most s , it can compute $S \cap T$ in time proportional to the output size. Thus, this result along with the discussion above implies the following result:

Theorem 17.4.1. *There exist strongly explicit codes of rate R that for large enough block length N can be list decoded from $1 - R - \epsilon$ fraction of errors (for any small enough $\epsilon > 0$) in time $O\left(\left(\frac{N}{\epsilon^2}\right)^2\right) + \left(\frac{1}{\epsilon}\right)^{O(1/\epsilon)}$. The worst-case list size is $\left(\frac{1}{\epsilon}\right)^{O(1/\epsilon)}$ and the alphabet size is $\left(\frac{N}{\epsilon}\right)^{O(1/\epsilon^2)}$.*

The above answers Question 17.0.1 pretty satisfactorily. However, to obtain a completely satisfactory answer one would have to solve the following open question:

Open Question 17.4.3. *Are there explicit codes of rate $R > 0$ that are $(1 - R - \epsilon, (1/\epsilon)^{O(1)})$ -list decodable that can be list-decoded in time $\text{poly}(N, 1/\epsilon)$ over alphabet of size $q \leq \text{poly}(n)$?*

The above question, without the requirement of explicitness, has been answered by Guruswami and Xing [82].

17.5 Exercises

Exercise 17.1. Ask the algebraic approach for decoding based on $f(\gamma X) \equiv F(X)^q \pmod{X^{q-1} - \gamma}$

Exercise 17.2.

Let $h(X) \in \mathbb{F}_q[X]$ be an irreducible polynomial of degree k . For $f \in \mathbb{F}_q[X]_{<k}$, define $f_1(X) = f(X)^2 \pmod{h(X)}$, and inductively $f_{i+1}(X) = f_i(X)^2 \pmod{h(X)}$ for $i \geq 1$.

Consider a bipartite graph $G = (A, B, E)$ with left side $A := \mathbb{F}_q[X]_{<k}$ and right side $B := \mathbb{F}_q^{m+1}$, where a node $f \in A$ is adjacent to the tuples $(\alpha, f(\alpha), f_1(\alpha), \dots, f_m(\alpha)) \in \mathbb{F}_q^{m+1}$ as α ranges over $\mathbb{F}_q$ (thus the left degree is q).

In this exercise, your goal is to establish the following vertex-expansion property of the bipartite graph G : For $U \subseteq A$ with $|U| = 2^m$, its neighborhood on the right, $N(U) := \{v \in B \mid (u, v) \in E\}$, has size at least $(q - mk)|U|$. (Note that $|N(U)| \leq q|U|$ trivially, since each node on the left has degree q .)

1. Let $D \geq 1$ be any integer. For any $T \subseteq \mathbb{F}_q^{m+1}$ with $|T| < D \cdot 2^m$, prove that there is a nonzero polynomial $Q(X, Y_0, Y_1, \dots, Y_{m-1})$ with $\mathbb{F}_q$ -coefficients that has degree at most $(D - 1)$ in X and degree at most 1 in each Y_i satisfying the interpolation conditions $Q(t_0, t_1, t_2, \dots, t_m) = 0$ for all $(t_0, t_1, \dots, t_m) \in T$.
2. Let T, Q be as above for $D = q - mk$. Suppose all neighbors of $f \in A$ belong to T . Then show that $Q(X, f(X), f_1(X), \dots, f_{m-1}(X)) = 0$.
3. Prove that for any nonzero $Q(X, Y_0, Y_1, \dots, Y_{m-1})$ that is multilinear in Y_i 's and which is not divisible by $h(X)$, there are less than 2^m polynomials $f \in \mathbb{F}_q[X]_{<k}$ satisfying the equation $Q(X, f(X), f_1(X), \dots, f_{m-1}(X)) = 0$.
4. Using the above argue that for any $T \subseteq \mathbb{F}_q^{m+1}$ with $|T| < (q - mk)2^m$, the set $U \subseteq A$ with $N(U) \subseteq T$ has size $|U| < 2^m$. (Note that this is precisely the contrapositive form of the above-mentioned expansion requirement.)

Exercise 17.3. The folded Reed-Solomon decoding algorithm from class relied on a field element γ for folding which had large multiplicative order.

This problem is aimed at explaining why this was needed.

1. Prove that if the order of γ is r , then there exist some choice of polynomials $A_0, A_1, \dots, A_s \in \mathbb{F}_q[X]$, not all zero, $s \geq 2$, such that there are at least $q^{k/r}$ polynomials $f \in \mathbb{F}_q[X]_{\leq k}$ satisfying the condition

$$A_0(X) + A_1(X)f(X) + A_2(X)f(\gamma X) + \dots + A_s(X)f(\gamma^{s-1}X) = 0.$$

(Thus, the number of solutions can be very large when the order of γ is small.)

2. Let C be a s -folded Reed-Solomon code of length $N = (q - 1)/s$ and rate R that is based on folding with a $\gamma \in \mathbb{F}_q^*$ of order s . Prove that if C is efficiently (ρ, L) -list decodable, then there is in fact a Reed-Solomon code (no folding needed!) of length N and rate R that is also efficiently (ρ, L) -list decodable.

(This means that if we get an improvement over the Johnson radius $1 - \sqrt{R}$ for a folded RS code using a small order element for folding, then we will also get an improvement for Reed-Solomon codes themselves.)

Exercise 17.4. Let $\mathbb{F}_q$ be a finite field and let $\Sigma = \mathbb{F}_q^m$ for some positive integer m . Suppose that we have a code $C \subseteq \Sigma^N$ that is $\mathbb{F}_q$ -linear, i.e., closed under $\mathbb{F}_q$ -linear combinations. (Note that folded Reed-Solomon codes are an example of such codes.) Suppose that the relative distance of C is δ and for any $w \in \Sigma^N$, all codewords of C within Hamming distance (over the alphabet Σ) $(\delta - \epsilon)N$ from w are contained in a $\mathbb{F}_q$ -subspace $V_w \subset C$ of dimension at most s , for some positive integer s .

This implies that C is $(\delta - \epsilon, q^s)$ -list-decodable. In this exercise, you will prove that C is in fact $(1 - \delta - \epsilon, L)$ -list-decodable for

$$L := \left(\frac{1}{1 - \delta} \right)^{O\left(\frac{s(1-\delta)}{\epsilon} \log\left(\frac{s}{\epsilon}\right)\right)}, \quad (17.19)$$

so that when s is a constant, we get a list-size bound that is a constant independent of N, q .

In fact, you will show the bound (17.19) algorithmically, by giving a randomized algorithm that, given as input w and a $\mathbb{F}_q$ -basis for V_w , outputs a list of size at most L that will, with probability at least $1/2$, include all codewords of C within Hamming distance $(1 - \delta - \epsilon)N$ of w .

The algorithm is very simple, and repeats the following procedure $O(L \log L)$ times to compute a list $\mathcal{L}$, for a parameter r to be worked out:

- Pick indices $i_1, i_2, \dots, i_r \in [N]$ independently at random.
- If there is a unique $v \in V_w$ such $v_{i_j} = w_{i_j}$ for $j = 1, 2, \dots, r$, and v is within Hamming distance $(\delta - \epsilon)N$ from w , then include v in $\mathcal{L}$.

For a specific run of the algorithm, let $H = \{v \in V_w \mid v_{i_j} = w_{i_j} \text{ for } j = 1, 2, \dots, r\}$.

1. Note that H is an affine $\mathbb{F}_q$ -subspace of V_w . Prove that the probability that $\dim(H) > 0$ is at most

$$\binom{r}{r - s + 1} (1 - \delta)^{r - s + 1}.$$

Hint: Consider the linear subspace $H' = \{v \in V_w \mid v_{i_j} = 0 \text{ for } j = 1, 2, \dots, r\}$. Imagine adding the constraints $v_{i_j} = 0$ one by one to go from V_w to H' . What's the chance that a nonzero vector survives the upcoming constraint? In order for $\dim(H') > 0$, for at least $s - 1$ of the r steps, all current vectors must survive the constraint.

2. Suppose $c \in V_w$ is within Hamming distance $(\delta - \epsilon)N$ from w . Argue that $H = \{c\}$ with probability at least

$$(1 - \delta + \epsilon)^r - (1 - \delta)^r \left(\frac{r}{1 - \delta} \right)^s. \quad (17.20)$$

Hint: $H = \{c\}$ means $c \in H$ and $\dim(H) = 0$.

3. For a suitable choice of $r = \Theta\left(\frac{s(1-\delta)}{\epsilon} \log\left(\frac{s}{\epsilon}\right)\right)$ prove that the quantity (17.20) is lower bounded by $\Omega(1/L)$ for L defined in (17.19).
4. Deduce the claimed list-decoding guarantee of the above algorithm.
5. What does the list-size bound (17.19) translate to for the rate R folded RS codes list-decodable up to radius $1 - R - \epsilon$ from lecture?

Exercise 17.5. Let p be a prime and let $1 \leq k < p$. For prime fields $\mathbb{F}_p$ and m a natural number dividing $(p - 1)$, we can also define m -folded Reed-Solomon codes based on additive folding, namely the map $C : \mathbb{F}_p[X]_{<k} \rightarrow (\mathbb{F}_p^m)^N$ defined by

$$C : f(X) \mapsto \left(\begin{bmatrix} f(1) \\ f(2) \\ \vdots \\ f(m) \end{bmatrix}, \begin{bmatrix} f(m+1) \\ f(m+2) \\ \vdots \\ f(2m) \end{bmatrix}, \dots, \begin{bmatrix} f(p-m) \\ f(p-m+1) \\ \vdots \\ f(p-1) \end{bmatrix} \right).$$

where $N \stackrel{\text{def}}{=} (p-1)/m$. Our goal in this problem is to list-decode additive folded Reed-Solomon codes (additive FRS) up to capacity.

- (a) Consider the map on polynomials $L : \mathbb{F}_p[X] \rightarrow \mathbb{F}_p[X]$ defined as $L(f)(X) \stackrel{\text{def}}{=} X(f(X+1) - f(X))$. For any positive integer $a \leq p - m$, prove that there exists an invertible linear map $M_a : \mathbb{F}_p^m \rightarrow \mathbb{F}_p^m$ satisfying

$$M_a : \begin{bmatrix} f(a) \\ f(a+1) \\ \vdots \\ f(a+m-1) \end{bmatrix} \mapsto \begin{bmatrix} L^0(f)(a) \\ L^1(f)(a) \\ \vdots \\ L^{m-1}(f)(a) \end{bmatrix}$$

For any polynomial $f \in \mathbb{F}_p[X]$. Here, L^ℓ denotes L composed ℓ times with itself.

- (b) For $i \in [N]$, define $a_i \stackrel{\text{def}}{=} 1 + m(i-1)$. Consider the map $\mathcal{M} : (\mathbb{F}_p^m)^N \rightarrow (\mathbb{F}_p^m)^N$ defined as

$$\mathcal{M} : y = (y_1, \dots, y_N) \mapsto (M_{a_1} y_1, M_{a_2} y_2, \dots, M_{a_N} y_N)$$

Show that $\Delta(y, y') = \Delta(\mathcal{M}(y), \mathcal{M}(y'))$ for all $y, y' \in (\mathbb{F}_p^m)^N$.

- (c) Consider the code $C' : \mathbb{F}_p[X]_{<k} \rightarrow (\mathbb{F}_p^m)^N$

$$C' : f(X) \mapsto \left(\begin{bmatrix} L^0(f)(1) \\ L^1(f)(1) \\ \vdots \\ L^{m-1}(f)(1) \end{bmatrix}, \begin{bmatrix} L^0(f)(m+1) \\ L^1(f)(m+1) \\ \vdots \\ L^{m-1}(f)(m+1) \end{bmatrix}, \dots, \begin{bmatrix} L^0(f)(p-m) \\ L^1(f)(p-m) \\ \vdots \\ L^{m-1}(f)(p-m) \end{bmatrix} \right).$$

Show that $C' = \mathcal{M} \circ C$. Thus the codes C and C' are equivalent under a collection of invertible linear maps.

- (d) Fix some natural number $1 \leq s < m$. For any polynomial $f \in \mathbb{F}_p[X]$, consider the vector $\mathcal{L}(f(X)) \in \mathbb{F}_p[X]^{m-s+1}$ defined as

$$\mathcal{L}(f(X)) \stackrel{\text{def}}{=} \begin{bmatrix} L^0(f)(X) \\ L^1(f)(X) \\ \vdots \\ L^{m-s}(f)(X) \end{bmatrix}.$$

Show that there exists a matrix $T(X) \in \mathbb{F}_p[X]^{(m-s+1) \times (m-s+1)}$ such that $\mathcal{L}(Xf(X)) = T(X)\mathcal{L}(f(X))$.

- (e) Consider some natural number D that we will choose later. Consider any received codeword $y \in (\mathbb{F}_p^m)^N$, and define $w \stackrel{\text{def}}{=} \mathcal{M}(y)$. Let $w_{i,j}$ denote the j 'th symbol in the i 'th coordinate of w . Consider a polynomial $Q \in \mathbb{F}_p[X, Y_1, \dots, Y_s]$ of the form

$$Q(X, Y_1, \dots, Y_s) = A_1(X)Y_1 + \dots + A_s(X)Y_s \quad (17.21)$$

where $\deg(A_j) \leq D$ for $1 \leq j \leq s$. Prove that for $D = \lfloor \frac{N(m-s+1)}{s} \rfloor$, there exists a nonzero polynomial Q of the form in Equation 17.21 satisfying

$$\sum_{j=1}^s A_j(T)(a_i) \begin{bmatrix} w_{i,j} \\ w_{i,j+1} \\ \vdots \\ w_{i,j+m-s} \end{bmatrix} = 0$$

for all $i \in [N]$. Here, $A_j(T)(a_i) \in \mathbb{F}_p^{(m-s+1) \times (m-s+1)}$ denotes the matrix obtained by considering the matrix $A_j(T)(X) \in \mathbb{F}_p[X]^{(m-s+1) \times (m-s+1)}$, which is obtained by applying the polynomial A_j on $T(X)$, and then evaluating it at $X = a_i$.

- (e) Given any polynomial $f \in \mathbb{F}_p[X]_{<k}$ such that $C(f)$ and y agree on at least $\left(\frac{1}{s} + \frac{k}{N(m-s+1)}\right)N + 1$ coordinates, show that the polynomial $R(X) \stackrel{\text{def}}{=} Q(X, L^0(f)(X), L^1(f)(X), \dots, L^{s-1}(f)(X))$ is identically zero. (*Hint:* How does the map L change the degree of f ? Thus what can the degree of R at most be? Now, what can you say about the weight of the 'codeword' $\mathcal{L}(R)(X)$ when evaluated on $X = a_1, \dots, a_N$?)

(f) Show that the collection of polynomials $f \in \mathbb{F}_p[X]_{<k}$ satisfying the identity

$$A_1(X)L^0(f)(X) + A_2(X)L^1(f)(X) + \dots + A_s(X)L^{s-1}(f)(X) \equiv 0 \quad (17.22)$$

form a vector space of dimension at most $s - 1$. (*Hint: What can you say about the coefficients of the k highest-degree monomials in Equation 17.22?*)

(g) For any $\varepsilon > 0$, show that by picking s and m appropriately, the code C is efficiently list-decodable up to a fraction of $1 - R - \varepsilon$ errors. (i.e. is list-decodable up to capacity).

Chapter 18

Fast encoding: linear time encodable codes

In the last chapter we saw how *low-density parity check* (LDPC) matrices¹ lead to codes with very fast (linear-time) decoders. Unfortunately these codes are not known to be encodable as efficiently. Indeed if one considers the generator matrix corresponding to a LDPC matrix of a code with good distance, the generator matrix will have high density (see Exercise 1.14), and so the natural encoding $x \mapsto x \cdot G$ will take nearly quadratic time if carried out naively. This motivates the following natural question:

Question 18.0.1. *Do there exist (asymptotically good) binary codes that are linear-time encodable (and linear time decodable)?*

In this chapter we will answer the above question in the affirmative and show how to construct codes that can be encoded as well as decoded in linear time. This construction will rely on ideas from Chapter ?? while involving a host of new ideas as well.

18.1 Overview of the construction

Our construction will yield a *systematic* code, i.e., one in which the first k coordinates of the codeword are the message (see Exercise 2.17), the rest of the codeword are what we will call *check bits*.

The **first idea** behind the construction is to use a *low-density* matrix as a generator. Of course this idea can not possibly work on its own (recall Exercise 1.14). But it turns out that this idea does work well as an ingredient in a more careful construction. Indeed to encode a message $\mathbf{x} \in \mathbb{F}_2^k$, the first set of check bits, denote $\mathbf{y}_1$ will be obtained by multiplying $\mathbf{x}$ with a low-density matrix $\mathbf{G}_1$, i.e., $\mathbf{y}_1 = \mathbf{x} \cdot \mathbf{G}_1$. The insight behind this step is that if $\mathbf{y}_1$ is known (i.e., there are no

¹We saw LDPC codes in Section ??.

errors in the check bit part) then the $\mathbf{x}$ can be recovered from $\hat{\mathbf{x}}$ that is close to $\mathbf{x}$, in the same way that we decoded LDPC codes from errors. (We will elaborate on this in Section 18.2.2.)

So our attention from now turns to protecting $\mathbf{y}_1$ from errors. Here the advantage we will have is that $\mathbf{y}_1$ will be smaller in length than $\mathbf{x}$ and so we can assume that such a code is available by induction. Indeed this is the **second idea**, and this is what we will implement in the rest of the chapter. We thus add some more checkbits $\mathbf{y}_2$ which will correspond to the checkbits when encoding $\mathbf{y}_1$ by a smaller, linear-time encodable code.

Unfortunately we can not stop at this stage. While we can use the fact that $\mathbf{y}_1$ has smaller length than $\mathbf{x}$ to our advantage, the smaller code can only correct a smaller number of errors (proportionate to the length of the code). So we can not hope that $\mathbf{y}_1$ will be recovered completely without protecting it further. Indeed we do so by adding more checkbits, denoted $\mathbf{y}_3$ that will protect the pair $(\mathbf{y}_1, \mathbf{y}_2)$.

This brings us to our final idea which is a strengthening of the idea in the first step. We will compute $\mathbf{y}_3 = [\mathbf{y}_1, \mathbf{y}_2] \cdot G_2$ where G_2 is another low-density matrix (of appropriate dimensions). Earlier we had asserted that if $\mathbf{y}_3$ is known completely and $[\mathbf{y}_1, \mathbf{y}_2]$ are known up to a few errors, then $\mathbf{y}_1$ and $\mathbf{y}_2$ can be recovered completely. However a more robust version of this statement can be proved: If we are given $\mathbf{y}_1, \mathbf{y}_2$ and $\mathbf{y}_3$ each with few errors, then an LDPC-type decoding algorithm can reduce the amount of error in $\mathbf{y}_1$ and $\mathbf{y}_2$ to an even smaller amount. With a careful choice of parameters, we can actually endure that the error in $(\mathbf{y}_1, \mathbf{y}_2)$ is small enough to allow our inductive idea (more precisely the ‘second idea’ above) to work correctly and thus recover $\mathbf{y}_1$ completely. And in turn, our first step will now manage to recover $\mathbf{x}$ completely.

In what follows we give details of the construction sketched above. To keep the number of parameters under control we focus on the case of codes of rate $1/4$. In particular, we will show that

Theorem 18.1.1. *There exists linear-time encodable and decodable codes of rate $\frac{1}{4}$.*

Exercise 18.5 shows how to build codes of higher rate. We start with the *error-reduction* algorithms and show that the codes obtained by computing checkbits by multiplying the message with a low-density matrix have nice error-reduction properties. Armed with this ingredient we describe our codes in Section 18.3. Finally we describe the encoding and decoding algorithms in Section 18.4 and analyze the running times there.

18.2 Low-density Error-Reduction Codes

While we described the ‘first-step’ of the encoding operation as a matrix-vector multiplication, what will be important to us is the graph theoretic view. Recall from Definition 11.1.2 that an $n \times m$ matrix G over $\mathbb{F}_2$ can be viewed as a bipartite graph B with n left vertices, m right vertices with $G_{ij} = 1$ if and only if there is an edge from left vertex i to right vertex j . In these terms the product $\mathbf{x} \cdot G$ corresponds to taking a 0/1 labeling of the left vertices and assigning labels to the right vertices, where the label of a right vertex j is the parity of the labels of its neighbors, i.e., the quantity $\oplus_{i|G_{ij}=1} x_i$ (where as usual we assume $\mathbf{x} = (x_1, \dots, x_n)$).

Let $B = ([n], [m], E)$ be a bipartite graph. For $j \in [m]$, let

$$N(j) = \{i \in [n] | (i, j) \in E\}$$

denote the neighborhoods of the ‘right’ vertices in B . Let $\mathbf{x} \in \mathbb{F}_2^n$ be an arbitrary labeling of the left vertices, and let $\mathbf{y} \in \mathbb{F}_2^m$ be the derived labeling of the right vertices, i.e., $y_j = \oplus_{i \in N(j)} x_i$. Let $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ be vectors that are ‘close’ to $\mathbf{x}$ and $\mathbf{y}$. In what follows we show that if B is a sufficiently good ‘expander’ then a variant of the decoding algorithm from Section ?? produces as output a vector $\hat{\mathbf{x}}$ that is ‘very’ close to $\mathbf{x}$. (We will quantify the many adjectives that we’ve used loosely in Lemma 18.2.1 below.)

18.2.1 The Code

We assume the existence of good bipartite expanders as guaranteed by Theorem 11.2.3. We recall the notion and theorem first. Recall that a $(n, m, D, \gamma, \alpha)$ -expander is a bipartite graph with $[n]$ as the left vertex sets, $[m]$ as the right vertex set, left degree D , and satisfies the feature that every subset $S \subseteq [n]$ with $|S| \leq \gamma n$, we have $|N(S)| \geq \alpha |S|$. We will apply the Theorem 11.2.3 with $n = k = 2^t$ for integer t , $m = k/2$, and $\varepsilon = 1/8$. Let B_k denote the resulting bipartite graphs — so B_k is a $(k, k/2, D, \gamma, (7/8)D)$ -expander for $D = O(1)$ and $\gamma = \Omega(1)$. Finally we will assume that in B_k all right vertices have degree $O(1)$ (see Exercise 18.1).

Given the graphs B_k we now define our Error-reduction codes. These codes have rate $2/3$ and thus map k message bits to $3k/2$ codeword bits, of which the first k are the message bits, and $k/2$ remaining bits are the check bits.

The *error-reduction code* $\tilde{R}_k$ is easy to define: Given $\mathbf{x} \in \mathbb{F}_2^k$ let $\mathbf{y} \in \mathbb{F}_2^{k/2}$ be given by $y_j = \sum_{i|j \in N(j)} x_i$, where $N(j)$ denotes the neighbors of j in B_k . We let $R_k(\mathbf{x}) = \mathbf{y}$ and the encoding $\tilde{R}_k : \mathbf{x} \mapsto (\mathbf{x}, R_k(\mathbf{x}))$.

Throughout this chapter we adopt of the convention of using $\tilde{C}(\cdot)$ to denote some systematic encoding, where $C(\cdot)$ denotes the checkbit part of the encoding.

Thus the code is quite simple. Next we describe an almost equally simple ‘error-reduction’ procedure. The most involved part of the section is describing the error-reduction property (and then proving it!).

18.2.2 GEN-FLIP Algorithm

We formally write down the error-reduction algorithm in Algorithm 18.2.1.

18.2.3 Error-Reduction Guarantee

As motivated earlier, we would like the error-reduction algorithm to have two crucial features:

1. If $(\mathbf{x}, \mathbf{y} = R_k(\mathbf{x}))$ is a codeword, and $\delta(\hat{\mathbf{x}}, \mathbf{x})$ is sufficiently small, then we want GEN-FLIP($\hat{\mathbf{x}}, \mathbf{y}$) to output $\mathbf{x}$.

Algorithm 18.2.1 GEN-FLIPINPUT: bipartite graph $B = ([n], [m], E)$ and vectors $\hat{\mathbf{x}} \in \mathbb{F}_2^n, \hat{\mathbf{y}} \in \mathbb{F}_2^m$ OUTPUT: $\tilde{\mathbf{x}} \in \mathbb{F}_2^n$ 1: $\tilde{\mathbf{x}} \leftarrow \hat{\mathbf{x}}$ ▷ We say $j \in [m]$ is *satisfied* if $\hat{y}_j = \oplus_{i \in N(j)} \tilde{x}_i$ and *unsatisfied* otherwise.2: WHILE there exists $i \in [n]$ with more unsatisfied than satisfied neighbors in $N(i)$ DO3: Flip $\tilde{x}_i$ and update the list of satisfied and unsatisfied vertices in $[m]$.4: RETURN $\tilde{\mathbf{x}}$

2. On the other hand when the exact check bits are not available, we can't hope to get $\mathbf{x}$ exactly. So in this case we would settle for a guarantee of the form: 'if $\delta(\hat{\mathbf{x}}, \mathbf{x})$ and $\delta(\hat{\mathbf{y}}, \mathbf{y})$ are small, then GEN-FLIP($\hat{\mathbf{x}}, \hat{\mathbf{y}}$) outputs $\tilde{\mathbf{x}}$ such that $\delta(\tilde{\mathbf{x}}, \mathbf{x})$ is even smaller'.

The following lemma gives both these properties in a smooth way.

Lemma 18.2.1. *There exists $\beta > 0$ such that for all $k = 2^t$ the following holds for the error-reduction code R_k and the algorithm GEN-FLIP: For every $\mathbf{x}, \hat{\mathbf{x}} \in \mathbb{F}_2^k$ and $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{F}_2^{k/2}$ such that $(\mathbf{x}, \mathbf{y}) = R_k(\mathbf{x})$, $\Delta(\mathbf{x}, \hat{\mathbf{x}}) \leq \beta k$ and $\Delta(\mathbf{y}, \hat{\mathbf{y}}) \leq \beta k$, it is the case that the output $\tilde{\mathbf{x}} = \text{GEN-FLIP}(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ satisfies $\Delta(\mathbf{x}, \tilde{\mathbf{x}}) \leq \Delta(\mathbf{y}, \hat{\mathbf{y}})/2$.*

Proof. Let a denote $\Delta(\mathbf{x}, \hat{\mathbf{x}})$ and let

$$A = \{i | x_i \neq \hat{x}_i\}.$$

Similarly let $b = \Delta(\mathbf{y}, \hat{\mathbf{y}})$ and let

$$B = \{j | y_j \neq \hat{y}_j\}.$$

We have $a, b \leq \beta k$. We will set

$$\beta = \frac{\gamma}{D+2}$$

so that we have $a(D+1) + b \leq \gamma k$ (recall D is the degree of B_k). Assume also that $D \geq 8$ (see Exercise 18.2 on why this assumption is fine for Theorem 11.2.3).

We first claim that the initial number of unsatisfied right nodes is at most $aD + b$. This is so since if $j \notin N(A) \cup B$ then

$$\hat{y}_j = y_j = \oplus_{i \in N(j)} x_i = \oplus_{i \in N(j)} \hat{x}_i$$

and so j is satisfied. We conclude number of initially unsatisfied right nodes is at most

$$|N(A) \cup B| \leq aD + b.$$

Next, we derive from the above that at the end of all iterations $\Delta(\mathbf{x}, \tilde{\mathbf{x}}) \leq a(D+1) + b$. This is so since initially $\Delta(\mathbf{x}, \tilde{\mathbf{x}}) = a$. And in each iteration we change at most one bit of $\tilde{\mathbf{x}}$. And the total number of iterations is upper bounded by the number of initially unsatisfied right nodes, since in each iteration the number of unsatisfied nodes decreases by one (see Exercise 18.3).

Let

$$S = \{i | \tilde{x}_i \neq x_i\}$$

at the end of the algorithm GEN-FLIP, and let T be the set of unsatisfied right vertices at the end. From the previous para we have

$$|S| \leq a(D+1) + b \leq \gamma k.$$

Let $U(S)$ be the unique neighbors of S , i.e.,

$$U(S) = \{j | |N(j) \cap S| = 1\}.$$

(See Definition 11.3.3.) Recall by Lemma 11.3.4 that

$$|U(S)| \geq (1 - 2\varepsilon)D|S| = \frac{3D}{4} \cdot |S|,$$

where we used that $\varepsilon = \frac{1}{8}$. Now every vertex in $U(S) \setminus B$ must be an unsatisfied vertex and so

$$|T| \geq |U(S) \setminus B| \geq \frac{3D}{4} \cdot |S| - b.$$

On the other hand at the end the total number of unsatisfied vertices must be less than $\frac{D}{2} \cdot |S|$ or else some vertex in S has more than $\frac{D}{2}$ unsatisfied neighbors and thus more unsatisfied neighbors than satisfied ones. We thus conclude

$$\frac{3D|S|}{4} - b \leq |T| \leq \frac{D}{2} \cdot |S|.$$

Rearranging and simplifying, we get

$$|S| \leq \frac{4b}{D} \leq \frac{b}{2}$$

as desired. □

It turns out that the above result also holds if we defined B_k based on expanders with fewer right neighbors (as long as it has good enough expansion)– see Exercise 18.4.

18.3 The error-correcting code: Recursive construction

Armed with the error-reduction codes R_k , we are now ready to construct our error-correcting codes C_k for $k = 2^t$ for every integer t . These codes will have rate $1/4$ (so $|C_k(\mathbf{x})| = 3k$ and $|\tilde{C}_k(\mathbf{x})| = 4k$ for $\mathbf{x} \in \mathbb{F}_2^k$).

We define the codes inductively. For small enough constants k (in particular, let k_0 be a constant such that for all $k \leq k_0$, C_k will be chosen to be check bits of some arbitrary systematic code of rate $1/4$).

For the inductive step, assume the code $C_{k/2}$ has been defined. For $\mathbf{x} \in \mathbb{F}_2^k$, we define the encoding

$$C_k(\mathbf{x}) = (\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3) \text{ where } \mathbf{y}_1 = R_k(\mathbf{x}), \mathbf{y}_2 = C_{k/2}(\mathbf{y}_1) \text{ and } \mathbf{y}_3 = R_{2k}(\mathbf{y}_1, \mathbf{y}_2).$$

The final code is defined by

$$\tilde{C}_k(\mathbf{x}) = (\mathbf{x}, C_k(\mathbf{x})).$$

See Figure 18.1 for an illustration of C_k .

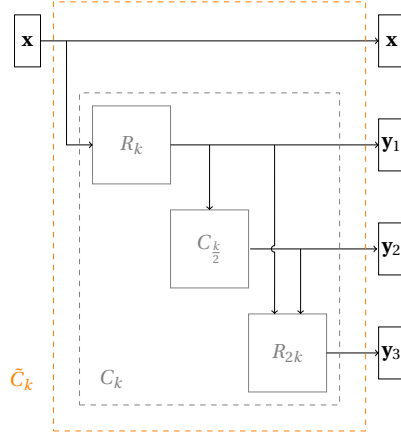


Figure 18.1: The recursive construction of C_k . The final code $\tilde{C}_k$ is also shown.

18.4 Analysis

We analyze the encoding first, and then describe the decoding algorithm and analyze it. The analysis will show that the code corrects $\Omega(1)$ -fraction of error and this will also establish that the code has constant relative distance.

18.4.1 Encoding Analysis

First we note that the error-reduction coding takes linear time. In particular it was argued in the proof of Lemma 18.2.1 that the number of iterations is linear, and we note in Exercise 18.6 each iteration can be implemented in constant time. Let B be the constant such that $R_k(\mathbf{x})$ can be computed in Bk time for $\mathbf{x} \in \mathbb{F}_2^k$.

Let $T_E(k)$ denote the time to encode a vector $\mathbf{x} \in \mathbb{F}_2^k$. We assert that there exists a constant A such that $T_E(k) \leq Ak$. For constant k we ensure this picking A large enough and so we turn to the inductive part. We now have (see Exercise 18.7)

$$T_E(k) \leq Bk + T_E(k/2) + 2Bk. \quad (18.1)$$

By induction, the above implies that

$$T_E(k) \leq 3Bk + Ak/2.$$

So if $3B \leq A/2$ (or equivalently $A \geq 6B$ which we can ensure) then we have $T_E(k) \leq Ak$ as desired.

18.4.2 Decoding Algorithm

The decoding algorithm is straightforward. Following our norm for systematic codes, we will only try to recover the correct message bits given potentially corrupted message and check bits. We describe below the algorithm LINEAR-DECODE whose input is a 4-tuple $(\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3)$ corresponding to a corrupted message $\hat{\mathbf{x}}$ and corrupted checkbits $(\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3)$. Its output will be a vector $\tilde{\mathbf{x}}$ which hopefully equals the original message.

Algorithm 18.4.1 LINEAR-DECODE

INPUT: $(\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3)$ where $\hat{\mathbf{x}} \in \mathbb{F}_2^k$, $\hat{\mathbf{y}}_1 \in \mathbb{F}_2^{k/2}$, $\hat{\mathbf{y}}_2 \in \mathbb{F}_2^{3k/2}$ and $\hat{\mathbf{y}}_3 \in \mathbb{F}_2^k$

OUTPUT: $\tilde{\mathbf{x}} \in \mathbb{F}_2^k$

- 1: IF $k \leq k_0$ THEN
 - 2: RETURN $D_{MLD}(\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3)$ ▷ Run Algorithm 1.4.1 for $\tilde{C}_k$
 - 3: Set $(\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2) \leftarrow \text{GEN-FLIP}(B_{2k}, (\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2), \hat{\mathbf{y}}_3)$.
 - 4: Set $(\mathbf{z}_1, \mathbf{z}_2) \leftarrow \text{LINEAR-DECODE}(\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2)$.
 - 5: Set $\tilde{\mathbf{x}} \leftarrow \text{GEN-FLIP}(B_k, \hat{\mathbf{x}}, \mathbf{z}_1)$.
 - 6: RETURN $\tilde{\mathbf{x}}$
-

In English, the algorithm first reduces the error in the $\mathbf{y}_1, \mathbf{y}_2$ part using the properties of the error-reduction code R_{2k} . Then it recursively decodes $\mathbf{y}_1$ from the information in the error-reduced $\mathbf{y}_1, \mathbf{y}_2$ -parts. Hopefully at this stage $\mathbf{y}_1$ is completely corrected. Finally it uses the error-reduction code R_k to now recover $\mathbf{x}$. We analyze this in the following section.

18.4.3 Analysis of the decoder

We first note that running time analysis is similar to that of the encoder and so we defer it to Exercise 18.8. We turn to the correctness claim. The lemma below asserts that the decoder corrects βk errors where β is the constant from Lemma 18.2.1. (Note that this implies that the code corrects a $\beta/4$ fraction of errors since the length of the codeword is $4k$.)

Lemma 18.4.1. *Let $\beta > 0$ be the constant from Lemma 18.2.1. The Algorithm LINEAR-DECODE corrects βk errors for codes of message length k . Specifically on input $(\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3)$ such that $\Delta((\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3), \tilde{C}_k(\mathbf{x})) \leq \beta k$, LINEAR-DECODE outputs $\tilde{\mathbf{x}}$.*

Proof. We prove the lemma by induction on k . Suppose we have the claim for all $k' < k$ (see Exercise 18.9).

Let $(\mathbf{y}_1, \mathbf{y}_2, \mathbf{y}_3) = C_k(\mathbf{x})$. Since $\Delta((\hat{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3), \tilde{C}_k(\mathbf{x})) \leq \beta k$, we have in particular that $\Delta((\mathbf{y}_1, \mathbf{y}_2), (\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2)) \leq \beta k$ and $\Delta(\mathbf{y}_3, \hat{\mathbf{y}}_3) \leq \beta k$. Thus by Lemma 18.2.1, the output $(\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2)$ of GEN-FLIP($B_{2k}, (\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2), \hat{\mathbf{y}}_3$) in Step 1 satisfies $\Delta((\mathbf{y}_1, \mathbf{y}_2), (\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2)) \leq \Delta(\mathbf{y}_3, \hat{\mathbf{y}}_3)/2 \leq \beta k/2$.

Now we turn to Step 2. By induction we have that LINEAR-DECODE decodes from $\beta k/2$ errors when decoding for messages of length $k/2$. Since $\Delta((\mathbf{y}_1, \mathbf{y}_2), (\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2)) \leq \beta k/2$, we thus have that LINEAR-DECODE($\tilde{\mathbf{y}}_1, \tilde{\mathbf{y}}_2$) outputs $\mathbf{z}_1, \mathbf{z}_2$ and so $\mathbf{z}_1 = \mathbf{y}_1$ and $\mathbf{z}_2 = \mathbf{y}_2$.

Finally, in Step 3, we now have $\Delta(\tilde{\mathbf{x}}, \mathbf{x}) \leq \beta k$ (another consequence of $\Delta(\tilde{\mathbf{x}}, \hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3), \tilde{C}_k(\mathbf{x}) \leq \beta k$) and so the output $\tilde{\mathbf{x}}$ of Step 3 satisfies $\Delta(\tilde{\mathbf{x}}, \mathbf{x}) \leq \Delta(\mathbf{z}_1, \mathbf{y}_1)/2 = 0$, or in other words $\tilde{\mathbf{x}} = \mathbf{x}$. We thus conclude that LINEAR-DECODE corrects decodes from βk errors. $\square$

18.5 Exercises

Exercise 18.1. Argue that B_k as defined in Section 18.2.1 can be assumed to have $O(1)$ maximum right degree.

Hint: Use Lemma 11.2.6.

Exercise 18.2. Argue why in Theorem 11.2.3 one can assume $D \geq 8$ (or more generally we can assume it is larger than any constant).

Exercise 18.3. Argue that in each iteration of GEN-FLIP the number of unsatisfied nodes decreases by one.

Exercise 18.4. Let $\alpha > 0$ be such that $(1/\alpha)^t$ is an integer. Let $k = (1/\alpha)^t$ and let B_k be an $(k, \alpha \cdot k, D, \gamma, 7D/8)$ -expander for $D = O(\log(1/\alpha))$ and $\gamma = \Omega(\alpha/D)$. Then define error-reduction code as we did earlier in Section 18.2.1 but with this updated B_k (note that earlier we used $\alpha = \frac{1}{2}$). Run GEN-FLIP with this updated B_k . Argue the following claim.

There exists $\beta > 0$ such that for all $k = (1/\alpha)^t$ the following holds for the error-reduction code R_k and GEN-FLIP as defined above: For every $\mathbf{x}, \tilde{\mathbf{x}} \in \mathbb{F}_2^k$ and $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{F}_2^{\alpha k}$ such that $(\mathbf{x}, \mathbf{y}) = R_k(\mathbf{x})$, $\Delta(\mathbf{x}, \tilde{\mathbf{x}}) \leq \beta k$ and $\Delta(\mathbf{y}, \hat{\mathbf{y}}) \leq \beta k$, it is the case that the output $\tilde{\mathbf{x}} = \text{GEN-FLIP}(\tilde{\mathbf{x}}, \hat{\mathbf{y}})$ satisfies $\Delta(\mathbf{x}, \tilde{\mathbf{x}}) \leq \Delta(\mathbf{y}, \hat{\mathbf{y}})/2$.

Exercise 18.5. For every rate $R \in (0, 1)$ give a linear-time encodable and decodable code of rate at least R correcting some $\varepsilon(R) > 0$ fraction of errors.

Hint: Exercise 18.4 and Theorem 18.1.1 might be useful here.

Exercise 18.6. Argue that each iteration of GEN-FLIP (i.e. Lines 2 and 3) can be implemented in $O(1)$ time.

Exercise 18.7. Argue (18.1).

Exercise 18.8. Argue why LINEAR-DECODE is a linear time algorithm.

Exercise 18.9. Argue Lemma 18.4.1 for the case when $k \leq k_0$.

18.6 Bibliographic Notes

The construction of this chapter is due to Spielman [146].

Chapter 19

Recovering very locally: Locally Recoverable Codes

19.1 Context

In this chapter, we describe an exciting recent direction in coding theory that emerged due to applications in modern distributed storage systems. Such storage systems store vast amounts of data that need to be maintained in a fault-tolerant manner, resilient to intermittent or permanent failures of servers storing the data. We can imagine the data encoded into a codeword $(c_1, c_2, \dots, c_n) \in \Sigma^n$, where the i 'th symbol is stored on the i 'th server. (Of course, the servers will store multiple codewords, but we will imagine server failures as symbol erasures (recall Proposition 1.4.2) in the codeword.) Erasure codes (i.e. codes capable of recovering from erasures) are thus a natural choice to ensure that the data can be safely recovered even when many servers are unresponsive.

Traditional MDS codes like Reed-Solomon codes appear attractive due to their optimal trade-off between storage overhead and the number of erasures tolerated. MDS codes, and more generally codes with good minimum distance, allow recovery from the worst-case scenario of a large number of erasures (recall Proposition 1.4.2). However, a much more common situation that emerges in the context of large scale distributed storage systems is that a *small number of* servers fail or become unresponsive. This calls for the repair of a single (or few) failed server(s) *quickly*, while at the same time retaining as much of the distance property of the code as possible, thus enabling protection against more rare or catastrophic failures or large-scale maintenance of many servers. This is exactly what *locally recoverable codes* (also called *locally repairable* or *local reconstruction* codes), abbreviated LRCs, are designed to achieve. They enable that any codeword symbol to be quickly recovered, in a *local* fashion, based on few other codeword symbols, and at the same time they have as large a distance as is compatible with the local constraints that the codeword symbols obey. This is a fairly natural trade-off under which to examine classical coding bounds and constructions, and at the same time, LRCs have had a significant practical impact, with their deployment in large scale cloud systems saving billions

of dollars in storage costs!¹

With this backdrop, we now turn to the formal description of locally recoverable codes. We will focus only on the coding-theoretic aspects, and not have a chance to describe further aspects that are related to the use of LRCs for distributed storage. For simplicity we will focus on the local recovery of single erased symbols. Note that compared to locally decodable or locally correctable codes (LDC or LCCs— see Chapter ??), where the goal is to recover from a *constant fraction* of errors, for LRCs the goal is to recover from a single (or small *number* of) erasures. Thus, the noise model is the most benign possible, but the demands on the code are strong — small locality and high rate. In comparison, LDCs and LCCs necessarily have a vanishing rate.

19.2 Definition of Locally Recoverable Codes

We will restrict our attention to linear codes over some field $\mathbb{F}$. This is mainly for simplicity. Our LRC constructions will be linear, and the Singleton type bound we prove on the limitations of LRCs (in Section 19.4) will work for non-linear codes as well.

Definition 19.2.1. Consider a linear code $C \subseteq \mathbb{F}^n$ of dimension k where the first k symbols are information symbols, and the last $n - k$ are check symbols². Such a code is said to be a message symbol (r, d) -locally recoverable code, or (r, d) -mLRC for short, if

- (i) it has minimum distance at least d , and
- (ii) for every $i \in [k]$, there exists $R_i \subseteq [n] \setminus \{i\}$ of size at most r such that the i 'th symbol c_i of any codeword $\mathbf{c} = (c_1, c_2, \dots, c_n) \in C$ can be recovered from $\mathbf{c}_{R_i}$.

One can also demand local recovery of all codeword symbols, including the check symbols, as defined below.

Definition 19.2.2. A linear code $C \subseteq \mathbb{F}^n$ is said to be an (r, d) -locally recoverable code, or (r, d) -LRC for short, if

- (i) it has minimum distance at least d , and
- (ii) for every $i \in [n]$, there exists $R_i \subseteq [n] \setminus \{i\}$ of size at most r such that the i 'th symbol c_i of any codeword $\mathbf{c} = (c_1, c_2, \dots, c_n) \in C$ can be recovered from $\mathbf{c}_{R_i}$.

Property (ii) is so that we can handle say the more benign one erasure case. Property (i) is for the more catastrophic of large number of erasures (in which case we would be forced to do the recovery non-locally).

We make two remarks:

¹A different coding construct, called *regenerating codes*, has also been extensively studied in the context of using codes for efficient repair of single/few erasures. These codes do not optimize for locality, but rather the total amount of information downloaded from other codeword positions, allowing for transmission of partial information about c_j to c_i . We do not discuss these codes in this book.

²I.e. these are systematic codes: recall Exercise 2.17.

1. Note that Definition 19.2.2 does not specify anything about *how* c_i could be recovered from $\mathbf{c}_{R_i}$. However, for linear codes, there is always a canonical ‘linear’ recovery mechanism— see Exercise 19.2.
2. Definition 19.2.2 does not impose any upper bound on r but it turns out that any $[n, k]_q$ code is (r, d) -LRC for $r \leq k$ — see Exercise 19.3.

Given the above definition, the following is the next natural question:

Question 19.2.1. If an $(n, k, d)_q$ code is an (r, d) -LRC, what is the optimal tradeoff between n and k ?

In the remainder of the chapter, we will exactly pin-point the optimal tradeoff between n and k .

19.3 A simple construction for message symbol LRCs

We will allow our codes to be defined over large fields (of size at least $\Omega(n)$).³ It turns out local recovery of only the message symbols is easy to arrange along with good distance (which we will show to be optimal in the next section, which answers Question 19.2.1).

Specifically, we will prove the following.

Theorem 19.3.1. Let $n > k \geq r$ be positive integers and $q \geq n$ a prime power. Then there is an explicit $[n, k]$ linear code over $\mathbb{F}_q$ that is an (r, d) -mLRC where

$$d = n - k - \left\lceil \frac{k}{r} \right\rceil + 2. \quad (19.1)$$

Proof. The construction is quite simple. We begin with a systematic $[n_0, k, d]$ MDS code (recall Definition 5.3.1 and Exercise 2.17) C_0 over $\mathbb{F}_q$ with k message symbols and $n_0 - k = d - 1$ parity symbols. This code encodes a message $\mathbf{x} = (x_1, \dots, x_k) \in \mathbb{F}_q^k$ into $\mathbf{c} \in \mathbb{F}_q^{n_0}$ where $c_i = x_i$ for $1 \leq i \leq k$ is the systematic part, and the check symbols c_{k+j} for $1 \leq j \leq n_0 - k$ are given by $c_{k+j} = \langle \mathbf{p}^{(j)}, \mathbf{x} \rangle$ for some $\mathbf{p}^{(j)} \in \mathbb{F}_q^k$. That is, the encoding map is

$$\mathbf{x} \mapsto \left(\mathbf{x}, \langle \mathbf{p}^{(1)}, \mathbf{x} \rangle, \langle \mathbf{p}^{(2)}, \mathbf{x} \rangle, \dots, \langle \mathbf{p}^{(d-1)}, \mathbf{x} \rangle \right). \quad (19.2)$$

By the MDS property, each vector $\mathbf{p}^{(j)} \in \mathbb{F}_q^k$ has full support (see Exercise 19.4).

We then take one of the check symbols (say c_{n_0}), and split it into $\ell := \left\lceil \frac{k}{r} \right\rceil$ symbols, each of which depends on at most r disjoint message symbols. Formally, partition $[k] = \{1, 2, \dots, k\}$ into

³This assumption is satisfied in practice.

sets $S_1, S_2, \dots, S_\ell$ where $S_1, S_2, \dots, S_{\ell-1}$ have size exactly r , and S_ℓ has the remaining (at most) r elements of $[k]$. More formally, let us take (for $1 \leq j < \ell$):

$$S_j = \{(j-1)r+1, (j-1)r+2, \dots, jr\}$$

and

$$S_\ell = \{(\ell-1)r+1, \dots, k\}.$$

Given these notations, one can represent a generator matrix of the code defined by (19.2) as shown below.

$$\left(\begin{array}{c|ccc} & & & \\ & \uparrow & \uparrow & \\ & \vdots & \vdots & \\ & \downarrow & \downarrow & \\ \mathbf{I} & \mathbf{p}^{(1)} & \mathbf{p}^{(2)} & \dots & \mathbf{p}^{(d-2)} & \begin{matrix} (\mathbf{p}^{(d-1)})_{S_1} \\ (\mathbf{p}^{(d-1)})_{S_2} \\ \vdots \\ (\mathbf{p}^{(d-1)})_{S_j} \\ \vdots \\ (\mathbf{p}^{(d-1)})_{S_\ell} \end{matrix} \end{array} \right)$$

Now consider a variant of the above generator matrix where we ‘split’ the last column as follows:

$$\left(\begin{array}{c|ccccccc} & & & & (\mathbf{p}^{(d-1)})_{S_1} & \mathbf{0} & \dots & \mathbf{0} & \dots & \mathbf{0} \\ & & & & \mathbf{0} & (\mathbf{p}^{(d-1)})_{S_2} & \dots & \mathbf{0} & \dots & \mathbf{0} \\ & & & & \vdots & \vdots & \dots & \vdots & \vdots & \vdots \\ & & & & \mathbf{0} & \mathbf{0} & \dots & (\mathbf{p}^{(d-1)})_{S_j} & \dots & \mathbf{0} \\ & & & & \vdots & \vdots & \dots & \vdots & \vdots & \vdots \\ & & & & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} & \dots & (\mathbf{p}^{(d-1)})_{S_\ell} \\ \mathbf{I} & \mathbf{p}^{(1)} & \mathbf{p}^{(2)} & \dots & \mathbf{p}^{(d-2)} & & & & & \end{array} \right)$$

For $1 \leq j \leq \ell$, define $\mathbf{q}^{(j)}$ to be the vector $\mathbf{p}^{(d-1)}$ that is zeroed out outside S_j . Based on the above generator matrix, we now define a new code C that encodes $\mathbf{x} \in \mathbb{F}_q^k$ into a codeword in $\mathbb{F}_q^n$ for

$$n = k + (d-2) + \ell = k + d - 2 + \left\lceil \frac{k}{r} \right\rceil \quad (19.3)$$

as follows:

$$\mathbf{x} \mapsto \left(\mathbf{x}, \langle \mathbf{p}^{(1)}, \mathbf{x} \rangle, \langle \mathbf{p}^{(2)}, \mathbf{x} \rangle, \dots, \langle \mathbf{p}^{(d-2)}, \mathbf{x} \rangle, \langle \mathbf{q}^{(1)}, \mathbf{x} \rangle, \dots, \langle \mathbf{q}^{(\ell)}, \mathbf{x} \rangle \right). \quad (19.4)$$

Note that by (19.3), the parameters n, k, d, r obey the relation (19.1) claimed in the theorem. We now verify that C is an (r, d) -mLRC, which would finish the proof.

Comparing the encodings (19.2) and (19.4), the sum of the last ℓ check symbols in (19.4) equals the last check symbol in (19.2), and the remaining codeword symbols are the same. It follows that the Hamming weight of the encoding (19.4) of a nonzero $\mathbf{x}$ is at least the Hamming weight under the encoding of $\mathbf{x}$ under (19.2). The latter is at least d , since C_0 has distance d . Thus, the code defined by (19.4) also has distance at least d .

Finally, the message symbol locality of (19.4) is guaranteed by the ℓ ‘split-up’ check symbols. Specifically, each message symbol in S_j can be recovered based on the other message symbols in S_j and the check symbol $\langle \mathbf{q}^{(j)}, \mathbf{x} \rangle$ — this follows because $\mathbf{p}^{(d-1)}$ has full support and therefore the support of $\mathbf{q}^{(j)}$ is precisely S_j . Since $|S_j| \leq r$, every message bit can be recovered based on at most r other bits of the codeword. $\square$

19.4 A Singleton-type bound

We now prove that the distance achieved in (19.1) is the best possible. We will present a general argument that does *not* rely on linearity. Specifically, the proof below works as long as there are k coordinates on which the code projects bijectively (this is the analog of the information set of a linear code), each of which has a local recovery set of size at most r (see Exercise 19.5 for a formal definition).

Theorem 19.4.1. *The distance of an (r, d) -mLRC of length n and dimension k must satisfy*

$$d \leq n - k - \left\lceil \frac{k}{r} \right\rceil + 2. \quad (19.5)$$

Proof. Let C be an (r, d) -mLRC (or to be precise (r, d) -gmLRC as defined in Exercise 19.5) with q^k codewords where q is the alphabet size of the code. Assume, without loss of generality, that C projects onto the first k coordinates bijectively, and that these coordinates all have a local recovery set of size r .

Our first (key) claim will be that we can find a subset $S \subset \{1, 2, \dots, k\}$ of size $\left\lfloor \frac{k-1}{r} \right\rfloor$ and a disjoint set $T \subset [n]$ of size at most $k-1$ such $\forall \mathbf{c} \in C$, $\mathbf{c}_T$ determines $\mathbf{c}_S$, where recall that $\mathbf{c}_T$ (resp. $\mathbf{c}_S$) denotes the projection $\mathbf{c}$ onto the coordinates in T (resp. S).

Indeed, we can construct these sets S and T greedily as follows:

Each of the sets R_ℓ in the above procedure has a size at most r . In fact, since $[k]$ is an information set, each R_ℓ must include at least one index outside $[k]$ (see Exercise 19.6), and thus

$$|R_\ell \cap [k]| \leq r-1.$$

Therefore, at any stage in the above procedure, we have

$$|T \cap [k]| \leq (r-1)|S|.$$

As long as $|S| < \frac{k-1}{r}$, we have

$$|S| + |T \cap [k]| \leq r|S| < (k-1).$$

Algorithm 19.4.1 Computing disjoint S and T

INPUT: Sets R_i for $i \in [k]$ OUTPUT: Set S, T such that $S \cap T = \emptyset$

```

1:  $S, T \leftarrow \emptyset$ 
2: WHILE  $|S| < \left\lfloor \frac{k-1}{r} \right\rfloor$  DO
3:   Let  $t \in [k]$  be the minimum element that does not belong to  $S \cup T$ 
4:    $S \leftarrow S \cup \{t\}$ 
5:    $T \leftarrow T \cup (R_t \setminus S)$ 
6: RETURN  $S, T$ 

```

Therefore an index $t \in [k]$ outside the current $S \cup T$ can be found (and can then be added to S). Thus, the above procedure is well defined.

By construction, it is clear that T is disjoint from S , since at each stage we exclude the current elements in S from R_t before adding them to T . Also it can be argued that $\mathbf{c}_T$ determines $\mathbf{c}_S$ (see Exercise 19.7). Also, the final set T output has size at most

$$r \left\lfloor \frac{k-1}{r} \right\rfloor \leq k-1.$$

Since $\mathbf{c}_S$ is determined by $\mathbf{c}_T$, it is also determined by $\mathbf{c}_{[n] \setminus S}$. This implies that the projection of C onto $[n] \setminus S$ is one-one⁴, and in particular $n - |S| \geq k$. Therefore, we can add $k-1-|T|$ elements outside $S \cup T$ to T to obtain a set T' of size $k-1$. By construction, we have

$$(i) \quad |T'| = k-1,$$

$$(ii) \quad T' \cap S = \emptyset, \text{ and}$$

$$(iii) \quad \mathbf{c}_{T'} \text{ determines } \mathbf{c}_S \text{ (since } T' \supset T \text{ and } \mathbf{c}_T \text{ determines } \mathbf{c}_S).$$

Since $|T'| = k-1$, by the pigeonhole principle⁵, there must exist two distinct codewords $\mathbf{c}, \mathbf{c}' \in C$ such that $\mathbf{c}_{T'} = \mathbf{c}'_{T'}$. Note that (by property (iii) above) this also implies $\mathbf{c}_S = \mathbf{c}'_S$. Thus, $\mathbf{c}$ and $\mathbf{c}'$ agree on at least $k-1 + \left\lfloor \frac{k-1}{r} \right\rfloor$ positions. Therefore the distance of C is at most

$$n - k - \left\lfloor \frac{k-1}{r} \right\rfloor + 1.$$

Noting that $\left\lfloor \frac{k-1}{r} \right\rfloor = \left\lceil \frac{k}{r} \right\rceil - 1$ (see Exercise 19.8), the proof is complete. $\square$

⁴Unless C has distance 0, in which case there is nothing to prove

⁵Recall that the pigeonhole principle states that if $> m$ pigeons are placed in m holes there will be one hole with at least two pigeons in it. We use this with the holes being all the q^{k-1} possible vectors of length q and the pigeons are the q^k codeword projections onto T' .

19.5 An LRC meeting the Singleton type bound

In Section 19.3, we presented a simple construction of a message symbol LRC (recall Definition 19.2.1) whose distance is optimal (meeting the bound (19.5)). We now construct a general (all-symbol) LRC (recall Definition 19.2.2). This will be harder than the message only case. The codes will be a carefully picked sub-code of the Reed-Solomon code that exhibits locality.

In particular, in this section we will argue the following result:

Theorem 19.5.1. *Let $n > k \geq r$ be positive integers with n being a multiple of $(r+1)$ and let $q \geq n+1$ a prime power such that $q-1$ is also divisible by $r+1$.⁶ Then there is an explicit $[n, k]_q$ code, which is in fact a sub-code of a Reed-Solomon code, that is an (r, d) -LRC with the optimal distance*

$$d = n - k - \left\lceil \frac{k}{r} \right\rceil + 2. \quad (19.6)$$

Fix a field $\mathbb{F}_q$ such that $q-1$ is divisible by $r+1$ (we argue in Exercise 19.9 that there are infinitely many such n, q and r that satisfy the divisibility conditions of Theorem 19.5.1). This means that $\mathbb{F}_q$ has primitive $(r+1)$ 'th root of unity (let's call it ω), so that $\omega, \omega^2, \dots, \omega^r$ are all distinct and $\omega^{r+1} = 1$ (see Exercise 19.10).

We will construct an $[n, k]_q$ linear LRC with locality r for $n = q-1$, which will be a sub-code of a certain Reed-Solomon code. Let k' be the smallest integer so that (we will soon see an explicit expression for k' in terms of k and r)

$$k = \left\lceil \frac{k'r}{r+1} \right\rceil. \quad (19.7)$$

Note that this means $k = \frac{k'r+a}{r+1}$ for some $a, 0 < a \leq r$.⁷ Next, inverting the definition of k' in terms of k , expressing k' as a function of k , we have

$$k' = \frac{(r+1)k - a}{r} = k + \frac{k-a}{r} = k + \left\lceil \frac{k}{r} \right\rceil - 1 \quad (19.8)$$

where the last equality holds because $a > 0$ (see Exercise 19.11).

Consider the Reed-Solomon code over $\mathbb{F}_q$ of dimension k' and block length $n := q-1$ which is obtained by evaluating message polynomials $f \in \mathbb{F}_q[X]$ of degree less than k' at $\mathbb{F}_q^*$ (all the nonzero field elements), to give the codeword $\langle f(\alpha) \rangle_{\alpha \in \mathbb{F}_q^*}$ (recall Definition 5.2.1). A degree $d-1$ polynomial can take every possible set of values on any d -tuple of field elements (this e.g. follows from Proposition 5.3.3). Therefore, these Reed-Solomon codes have no non-trivial locality — one needs to know k' other codeword symbols to recover any particular erased codeword symbol. The overall idea in our construction will be to pick polynomials of degree at most $k-1$ with *specific* structure.

Consider the set

$$U = \{1, \omega, \omega^2, \dots, \omega^r\} \quad (19.9)$$

⁶These divisibility requirements can be relaxed, but for simplicity we assume them.

⁷We cannot have $a = 0$ since if $k = \frac{k'r}{r+1}$, then $k = \left\lceil \frac{(k'-1)r}{r+1} \right\rceil$ so k' will not be the minimum choice satisfying (19.7).

of the $(r+1)$ 'th roots of unity in $\mathbb{F}_q^*$. We will now see how to pick a subcode of the above Reed-Solomon code, by restricting which powers are allowed in the message polynomials, so that the evaluations at U will exhibit locality (and similarly for the multiplicative cosets⁸ of U , which partition $\mathbb{F}_q^*$).

We mention the following fact (see Exercise 19.12):

Fact 19.5.2. *We have*

$$\prod_{u \in U} (X - u) = X^{r+1} - 1.$$

If we restrict a polynomial $f \in \mathbb{F}_q[X]$ to U , its restriction f_U agrees with the polynomial $g(X) := f(X) \bmod (X^{r+1} - 1)$ on U (see Exercise 19.13). Now if g has degree less than r , then the evaluations of g on U will satisfy a non-trivial linear constraint, i.e., $\sum_{u \in U} \lambda_u g(u) = 0$ where not all λ_u 's are 0 (see Exercise 19.14).

However, in general the remainder of $f(X)$ modulo $(X^{r+1} - 1)$ is a polynomial of degree r . To ensure that it has degree less than r , we will restrict the message polynomials $f(X)$ in the Reed-Solomon code to not have X^a terms (or equivalently force those coefficients to 0) whenever $a \equiv r \pmod{r+1}$.⁹ Formally, our code will be

$$C^* = \left\{ \langle f(\alpha) \rangle_{\alpha \in \mathbb{F}_q^*} \mid f(X) = \sum_{i=0}^{k'-1} f_i X^i, f_i = 0 \text{ if } i \equiv r \pmod{r+1} \right\}. \quad (19.10)$$

Dimension of the code. We set the coefficients $f_r, f_{2r+1}, f_{3r+2}, \dots$ to 0. Thus, every $(r+1)$ 'th coefficient starting at r is set to 0. The number of such coefficients that are at most $k' - 1$ equals

$$\left\lfloor \frac{k' - 1 - r}{r+1} \right\rfloor + 1 = \left\lfloor \frac{k'}{r+1} \right\rfloor.$$

Therefore, the dimension of C^* equals

$$k' - \left\lfloor \frac{k'}{r+1} \right\rfloor = \left\lceil \frac{k'r}{r+1} \right\rceil.$$

which equals k by our choice of k' in (19.7).

Since C^* is a subcode of the Reed-Solomon code, its distance d is at least that of the original Reed-Solomon code. We thus have

$$d \geq n - k' + 1 = n - k - \left\lceil \frac{k}{r} \right\rceil + 2$$

where the second equality follows from (19.8). Note that this is optimal by Theorem 19.4.1.

⁸See part 2 of Exercise 2.4 for a definition of (multiplicative) coset.

⁹To see why this makes sense, note that $X^a \bmod (X^{r+1} - 1) = X^{a \bmod (r+1)}$. Hence the only monomials X^a that contribute to the coefficient of the monomial X^r in $g(X)$ are those that satisfy $a \equiv r \pmod{r+1}$.

Locality of the code. By construction, the restriction of any polynomial $f(X)$ in (19.10) on U is a polynomial of degree at most $r - 1$. Further, there exists such f for which the restriction has degree equal to $r - 1$ (see Exercise 19.15). The evaluations of a degree $r - 1$ polynomial on $r + 1$ distinct points obey a single non-trivial linear condition that involves all the $r + 1$ evaluations with non-zero coefficients (recall Exercise 19.14). Therefore, the locations U of C^* form a local group of size $r + 1$, where each codeword symbol can be recovered from the remaining r values.

For the other local groups, we use the cosets of U (recall part 2 of Exercise 2.4). Namely define

$$\alpha U = \{\alpha, \alpha\omega, \dots, \alpha\omega^r\}. \quad (19.11)$$

The set $\mathbb{F}_q^*$ can be written as the disjoint union of $\frac{q-1}{r+1}$ cosets αU .¹⁰ The local groups will be in one-one correspondence with these cosets.

Let us focus on any one of these cosets αU and prove the desired locality. Note that (see Exercise 19.16)

$$\prod_{i=0}^r (X - \alpha\omega^i) = (X^{r+1} - \alpha^{r+1}).$$

So the restriction of $f(X)$ on αU is given by $g^{(\alpha)}(X) = f(X) \bmod (X^{r+1} - \alpha^{r+1})$. By the restriction placed on $f(X)$ in (19.10), $g^{(\alpha)}(X)$ has degree less than r . Further, this degree equals $r - 1$ for some $f(X)$. Therefore, by Exercise 19.14, the values of $g^{(\alpha)}(X)$, and hence $f(X)$, at the locations in αU satisfy a linear constraint, $\sum_{i=0}^r \lambda_{\alpha\omega^i} f(\alpha\omega^i) = 0$, with $\lambda_{\alpha\omega^i} \neq 0$ for every i . This proves that the evaluation of $f(X)$ at an arbitrary point in αU can be recovered from its evaluations at the r other points of αU . Thus, all locations in αU have local recovery sets of size r . This completes the proof of Theorem 19.5.1.

19.6 Exercises

Exercise 19.1. In this problem, we will prove a general structural result about linear codes. Let $C \subseteq \mathbb{F}_q^n$ be a linear code. Let $i \in [n]$. Prove that at least one of these two conditions have to hold:

1. There exists $\mathbf{v} \in C^\perp$ such that $i \in \text{supp}(\mathbf{v})$; or
- 2.

$$C = \{(c_1, \dots, c_{i-1}, \alpha, c_{i+1}, \dots, c_n) \mid \alpha \in \mathbb{F}_q, (c_1, \dots, c_{i-1}, c_{i+1}, \dots, c_n) \in C_{[n] \setminus \{i\}}\}.$$

Exercise 19.2. Let C be an (n, d) -LRC that is an $[n, k, d]_q$ code. Then

1. Argue that for every $i \in [n]$, there exists dual codeword $\mathbf{v} \in C^\perp$ with $i \in \text{supp}(\mathbf{v})$ with $\text{supp}(\mathbf{v}) \subseteq R_i \cup \{i\}$.
2. Using the previous part or otherwise argue that any c_i for any codeword $\mathbf{c} = (c_1, \dots, c_n) \in C$ can be recovered as a linear combination of values in $\mathbf{c}_{R_i}$ (where R_i is as defined in Definition 19.2.2).

¹⁰The claim on disjoint union follows from part 2 of Exercise 2.4. The claim on the number of disjoint cosets follows from the fact that each coset has size $r + 1$ and there are $q - 1$ elements in $\mathbb{F}_q^*$.

Hint: Exercise 19.1 could be useful.

Exercise 19.3. Show that any $[n, k, d]$ code is an (r, d) -LRC for some $r \leq k$.

Hint: Exercise 19.1 could be useful.

Exercise 19.4. Let $\mathbf{p}^{(1)}, \dots, \mathbf{p}^{(d-1)}$ be as defined in (19.2). Argue that for all $i \in [d-1]$, $|\text{supp}(\mathbf{p}^{(i)})| = k$.

Exercise 19.5. This exercise will essentially generalize Definition 19.2.1 to work for non-linear codes as well.

Let C be a (not necessarily linear) $(n, k, d)_q$ code with a bijection $\sigma : C_S \rightarrow [q]^k$ for some subset S of size k , i.e. the k codeword symbols indexed by S are the same as set of q^k possible vectors (up to the bijection σ).

Such a code is said to be a general message symbol (r, d) -locally recoverable code, or (r, d) -gmLRC for short, if

- (i) it has minimum distance at least d , and
- (ii) for every $i \in S$, there exists $R_i \subseteq [n] \setminus \{i\}$ of size at most r such that the i 'th symbol c_i of any codeword $\mathbf{c} = (c_1, c_2, \dots, c_n) \in C$ can be recovered from $\mathbf{c}_{R_i}$.

Argue that a linear code C that is an (r, d) -mLRC is also an (r, d) -gmLRC.

Exercise 19.6. Let C be an (r, d) -gmLRC (see Exercise 19.5 for a definition) and S be as defined in Exercise 19.5. Then argue that for every $i \in S$, it has to be the case that $R_i \cap S \neq R_i$ (i.e. R_i has an index outside of S).

Exercise 19.7. Let S and T be as computed by Algorithm 19.4.1 in the proof of Theorem 19.4.1. Argue that $\mathbf{c}_T$ determines $\mathbf{c}_S$.

Hint: Use induction and the definition of the set R_t for $t \in S$.

Exercise 19.8. Argue that for positive integers a and b , we have

$$\left\lfloor \frac{a-1}{b} \right\rfloor = \left\lceil \frac{a}{b} \right\rceil - 1.$$

Exercise 19.9. Argue that there are infinitely many such n, q and r that satisfy the divisibility conditions of Theorem 19.5.1.

Exercise 19.10. Fix a field $\mathbb{F}_q$ such that $q-1$ is divisible by $r+1$. Then argue that there exists an element $\omega \in \mathbb{F}_q^*$ so that $\omega, \omega^2, \dots, \omega^r$ are all distinct and $\omega^{r+1} = 1$.

Hint: Lemma D.5.11 might be useful. Can you define ω in terms of the primitive element of $\mathbb{F}_q^*$?

Exercise 19.11. Let k' be as in (19.7) and let $k = \frac{k'r+a}{r+1}$ for some $a, 0 < a \leq r$. Then argue that

$$\frac{k-a}{r} = \left\lceil \frac{k}{r} \right\rceil - 1.$$

Exercise 19.12. Let $1, \omega, \dots, \omega^{r+1}$ be the $(r+1)$ th roots of unity in $\mathbb{F}_q^*$. Then argue that

$$\prod_{i=0}^r (X - \omega^i) = X^{r+1} - 1.$$

Exercise 19.13. Let U be as in (19.9). Argue If we restrict a polynomial $f \in \mathbb{F}_q[X]$ to U , its restriction f_U agrees with the polynomial $g(X) := f(X) \bmod (X^{r+1} - 1)$ on U .

Exercise 19.14. Let $S \subseteq \mathbb{F}_q$ be a subset and assume a polynomial $g(X)$ has degree at most $|S| - 2$, then there exists λ_α for $\alpha \in S$ such that $\sum_{\alpha \in S} \lambda_\alpha g(\alpha) = 0$ where not all λ_α 's are 0.

Exercise 19.15. Let U be as in (19.9). Then argue there exists an $f(X)$ such that the polynomial corresponding to restriction of f on U has degree exactly $r - 1$.

Exercise 19.16. Let $1, \omega, \dots, \omega^{r+1}$ be the $(r+1)$ th roots of unity in $\mathbb{F}_q^*$. Then argue that

$$\prod_{i=0}^r (X - \alpha \omega^i) = (X^{r+1} - \alpha^{r+1}).$$

19.7 Bibliographic notes

Local Recoverable Codes (LRCs) were invented precisely for meeting the dual objectives of importance in distributed storage: (i) good global erasure resilience and (ii) low latency servicing of user requests or node repair in the wake of single server failures. Locality in distributed storage was first introduced by Huang, Chen, and Li [90], where the construction of message symbol LRCs of Section 19.3, called pyramid codes, also appeared. LRCs were first formally defined and studied by Gopalan, Huang, Simitci and Yekhanin [64] and Papailiopoulos and Dimakis [121].

The Singleton-type bound of Theorem 19.4.1 was first established in [64], and various different proofs and extensions of it have since appeared in the literature. Our treatment is a bit more general as it works also for non-linear codes. The optimal LRC meeting the Singleton-type bound from Section 19.5 is due to Tamo and Barg [153].

LRCs with suitably optimized parameters have been implemented in several large scale systems such as Microsoft Azure [?] and Hadoop [?]. They have led to billions of dollars of savings in storage costs, and thus have had an enormous commercial impact to go along with fundamental theoretical developments.

Another class of codes motivated by the problem of efficient repair of failed nodes in distributed storage are *regenerating codes*. Here the quantity optimized is not the number of other nodes contacted (which is the locality), but rather the total amount of information downloaded from the other nodes. To this end, we imagine that the nodes store vectors of symbols, and one is allowed to download some linear combinations of them.

Part V

The Applications

Chapter 20

Cutting Data Down to Size: Hashing

In this chapter, we will study hashing, which is a method to compute a small digest of data that can be used as a surrogate to later perform quick checks on the data. We begin with brief descriptions of three practical applications where hashing is useful. We then formally state the definition of hash functions that are needed in these applications (the so called “universal” hash functions). Next, we will show how in some sense good hashing functions and good codes are equivalent. Finally, we will see how hashing can solve a problem motivated by outsourced storage in the “cloud.”

20.1 Why Should You Care About Hashing?

Hashing is one of the most widely used objects in computer science. In this section, we outline three practical applications that heavily use hashing. While describing the applications, we will also highlight the properties of hash functions that these applications need.

Before we delve into the applications, let us first formally define a hash function.

Definition 20.1.1 (Hash Function). *Given a domain $\mathbb{D}$ and a range Σ , (typically, with $|\Sigma| < |\mathbb{D}|$), a hash function is a map*

$$h : \mathbb{D} \rightarrow \Sigma.$$

Of course, the definition above is too general and we will later specify properties that will make the definition more interesting.

Integrity Checks on Routers. Routers on the Internet process a lot of packets in a very small amount of time. Among other tasks, router has to perform an “integrity check” on the packet to make sure that the packet it is processing is not corrupted. Since the packet has well defined fields, the router could check if all the field have valid entries. However, it is possible that one of the valid entry could be turned into another valid entry. However, the packet as a whole could still be invalid.

If you have progressed so far in the book, you will recognize that the above is the error detection problem and we know how to do error detection (see e.g., Proposition 2.3.5). However, the

algorithms that we have seen in this book are too slow to implement in routers. Hence, Internet protocols use a hash function on a domain $\mathbb{D}$ that encodes all the information that needs to go into a packet. Thus, given an $\mathbf{x} \in \mathbb{D}$, the packet is the pair $(\mathbf{x}, h(\mathbf{x}))$. The sender sends the packet $(\mathbf{x}, h(\mathbf{x}))$ and the receiver gets $(\mathbf{x}', y)$. In order to check if any errors occurred during transmission, the receiver checks if $h(\mathbf{x}') = y$. If the check fails, the receiver asks for a re-transmission otherwise it assumes there were no errors during transmission. There are two requirements from the hash function: (i) It should be super efficient to compute $h(\mathbf{x})$ given $\mathbf{x}$ and (ii) h should avoid “collisions,” i.e. if $\mathbf{x} \neq \mathbf{x}'$, then $h(\mathbf{x}) \neq h(\mathbf{x}')$.¹

Integrity Checks in Cloud Storage. Say, you (as a client) have data $\mathbf{x} \in \mathbb{D}$ that you want to outsource $\mathbf{x}$ to a cloud storage provider. Of course once you “ship” off $\mathbf{x}$ to the cloud, you do not want to store it locally. However, you do not quite trust the cloud either. If you do not audit the cloud storage server in any way, then nothing stops the storage provider from throwing away $\mathbf{x}$ and send you some other data $\mathbf{x}'$ when you ask for $\mathbf{x}$. The problem of designing an auditing protocol that can verify whether the server has the data $\mathbf{x}$ is called the *data possession* problem.

We consider two scenarios. In the first scenario, you access the data pretty frequently during “normal” operation. In such cases, here is a simple check you can perform. When you ship off $\mathbf{x}$ to the cloud, compute $z = h(\mathbf{x})$ and store it. Later when you access $\mathbf{x}$ and the storage provider send you $\mathbf{x}'$, you compute $h(\mathbf{x}')$ and check if it is the same as the stored $h(\mathbf{x})$. This is exactly the same solution as the one for packet verification mentioned above.

Now consider the scenario, where the cloud is used as an archival storage. In such a case, one needs an “auditing” process to ensure that the server is indeed storing $\mathbf{x}$ (or is storing some massaged version from which it can compute $\mathbf{x}$ —e.g. the storage provider can compress $\mathbf{x}$). One can always ask the storage provider to send back $\mathbf{x}$ and then use the scheme above. However, if $\mathbf{x}$ is meant to be archived in the cloud, it would be better to resolve the following question:

Question 20.1.1. *Is there an auditing protocol with small client-server communication^a, which if the server passes then the client should be able to certain (with some high confidence) that the server is indeed storing $\mathbf{x}$?*

^aIn particular, we rule out solutions where the server sends $\mathbf{x}$ to the client.

We will see later how this problem can be solved using hashing.

Fast Table Lookup. One of the most common operations on databases is the following. Assume there is a table with entries from $\mathbb{D}$. One would like to decide on a data structure to store

¹Note that in the above example, one could have $\mathbf{x} \neq \mathbf{x}'$ and $h(\mathbf{x}) \neq h(\mathbf{x}')$ but it is still possible that $y = h(\mathbf{x}')$ and hence the corrupted packet $(\mathbf{x}', y)$ would pass the check above. Our understanding is that such occurrences are rare.

the table so that later on given an element $\mathbf{x} \in \mathbb{D}$, one would quickly like to decide whether $\mathbf{x}$ is in the table or now.

Let us formalize the problem a bit: assume that the table needs to store N values $a_1, \dots, a_N \in \mathbb{D}$. Then later given $\mathbf{x} \in \mathbb{D}$ one needs to decide if $\mathbf{x} = a_i$ for some i . Here is one simple solution: sort the n elements in an array T and given $\mathbf{x} \in \mathbb{D}$ use binary search to check if $\mathbf{x}$ is in T or not. This solution uses $\Theta(N)$ amounts of storage and searching for $\mathbf{x}$ takes $\Theta(\log N)$ time. Further, the pre-processing time (i.e. time taken to build the array T) is $\Theta(N \log N)$. The space usage of this scheme is of course optimal but one would like the lookup to be faster: ideally we should be able to perform the search in $O(1)$ time. Also it would be nice to get the pre-processing time closer to the optimal $O(N)$. Further, this scheme is very bad for dynamic data: inserting an item to and deleting an item from T takes $\Theta(N)$ time in the worst-case.

Now consider the following solution: build a boolean array B with one entry for each $z \in \mathbb{D}$ and set $B[a_i] = 1$ for every $i \in [N]$ (and every other entry is 0).² Then searching for $\mathbf{x}$ is easy: just lookup $B[\mathbf{x}]$ and check if $B[\mathbf{x}] \neq 0$. Further, this data structure can easily handle addition and deletion of elements (by incrementing and decrementing the corresponding entry of B respectively). However, the amount of storage and pre-processing time are both $\Theta(|\mathbb{D}|)$, which can be much much bigger than the optimal $O(N)$. This is definitely true for tables stored in real life databases. This leads to the following question:

Question 20.1.2. *Is there a data structure that supports searching, insertion and deletion in $O(1)$ time but only needs $O(N)$ space and $O(N)$ pre-processing time?*

We will see later how to solve this problem with hashing.

20.2 Avoiding Hash Collisions

One of the properties that we needed in the applications outlined in the previous section was that the hash function $h : \mathbb{D} \rightarrow \Sigma$ should avoid collisions. That is, given $\mathbf{x} \neq y \in \mathbb{D}$, we want $h(\mathbf{x}) \neq h(y)$. However, since we have assumed that $|\Sigma| < |\mathbb{D}|$, this is clearly impossible. A simple counting argument shows that there will exist an $\mathbf{x} \neq y \in \mathbb{D}$ such that $h(\mathbf{x}) = h(y)$. There are two ways to overcome this hurdle.

The first is to define a cryptographic collision resistant hash function h , i.e. even though there exists collisions for the hash function h , it is computationally hard for an adversary to compute $\mathbf{x} \neq y$ such that $h(\mathbf{x}) = h(y)$.³ This approach is out of the scope of this book and hence, we will not pursue this solution.

²If one wants to handle duplicates, one could store the number of occurrences of y in $B[y]$.

³This is a very informal definition. Typically, an adversary is modeled as a randomized polynomial time algorithm and there are different variants on whether the adversary is given $h(\mathbf{x})$ or $\mathbf{x}$ (or both). Also there are variants where one assumes a distribution on $\mathbf{x}$. Finally, there are no unconditionally collision resistant hash function but there exists provably collision resistant hash function under standard cryptographic assumptions: e.g. factoring is hard.

The second workaround is to define a family of hash functions and then argue that the probability of collision is small for a hash function chosen randomly from the family. More formally, we define a hash family:

Definition 20.2.1 (Hash Family). *Given $\mathbb{D}, \Sigma$ and an integer $m \geq 1$, a hash family $\mathcal{H}$ is a set $\{h_1, \dots, h_m\}$ such that for each $i \in [m]$,*

$$h_i : \mathbb{D} \rightarrow \Sigma.$$

Next we define the notion of (almost) universal hash function (family).

Definition 20.2.2 (Almost Universal Hash Family). *A hash family $\mathcal{H} = \{h_1, \dots, h_m\}$ defined over the domain $\mathbb{D}$ and range Σ is said to be ε -almost universal hash function (family) for some $0 < \varepsilon \leq 1$ if for every $\mathbf{x} \neq y \in \mathbb{D}$,*

$$\Pr_i [h_i(\mathbf{x}) = h_i(y)] \leq \varepsilon,$$

where in the above i is chosen uniformly at random from $[m]$.

We will show in the next section that ε -almost universal hash functions are equivalent to codes with (large enough) distance. In the rest of the section, we outline how these hash families provides satisfactory solutions to the problems considered in the previous section.

Integrity Checks. For the integrity check problem, one pick random $i \in [m]$ and chooses $h_i \in \mathcal{H}$, where $\mathcal{H}$ is an ε -almost universal hash function. Thus, for any $\mathbf{x} \neq y$, we're guaranteed with probability at least $1 - \varepsilon$ (over the random choice of i) that $h_i(\mathbf{x}) \neq h_i(y)$. Thus, this gives a randomized solution to the integrity checking problem in routers and cloud storage (where we consider the first scenario in which the cloud is asked to return the original data in its entirety).

It is not clear whether such hash functions can present a protocol that answers Question 20.1.1. There is a very natural protocol to consider though. When the client ships off data $\mathbf{x}$ to the cloud, it picks a random hash function $h_i \in \mathcal{H}$, where again $\mathcal{H}$ is an ε -universal hash function, and computes $h_i(\mathbf{x})$. Then it stores $h_i(\mathbf{x})$ and ships off $\mathbf{x}$ to the cloud. Later on, when the client wants to audit, it asks the cloud to send $h_i(\mathbf{x})$ back to it. Then if the cloud returns with z , the client checks if $z = h_i(\mathbf{x})$. If so, it assumes that the storage provider is indeed storing $\mathbf{x}$ and otherwise it concludes that the cloud is not storing $\mathbf{x}$.

Note that it is crucial that the hash function be chosen randomly: if the client picks a deterministic hash function h , then the cloud can store $h(\mathbf{x})$ and throw away $\mathbf{x}$ because it knows that the client is only going to ask for $h(\mathbf{x})$. Intuitively, the above protocol might work since the random index $i \in [m]$ is not known to the cloud till the client asks for $h_i(\mathbf{x})$, it seems “unlikely” that the cloud can compute $h_i(\mathbf{x})$ without storing $\mathbf{x}$. We will see later how the coding view of almost universal hash functions can make this intuition rigorous.

Fast Table Lookup. We now return to Question 20.1.2. The basic idea is simple: we will modify the earlier solution that maintained an entry for each element in the domain $\mathbb{D}$. The new solution will be to keep an entry for all possible hash values (instead of all entries in $\mathbb{D}$).

More formally, let $\mathcal{H} = \{h_1, \dots, h_m\}$ be an ε -almost hash family with domain $\mathbb{D}$ and range Σ . Next we build an array of link list with one entry in the array for each value $v \in \Sigma$. We pick a random hash function $h_i \in \mathcal{H}$. Then for each a_j ($j \in [N]$) we add it to the link list corresponding to $h_i(a_j)$. Now to determine whether $\mathbf{x} = a_j$ for some j , we scan the link list corresponding to $h_i(\mathbf{x})$ and check if $\mathbf{x}$ is in the list or not. Before we analyze the space and time complexity of this data structure, we point out that insertion and deletion are fairly easy. For inserting an element $\mathbf{x}$, we compute $h_i(\mathbf{x})$ and add $\mathbf{x}$ to the link list corresponding to $h_i(\mathbf{x})$. For deletion, we first perform the search algorithm and then remove $\mathbf{x}$ from the list corresponding to $h_i(\mathbf{x})$, if it is present. It is easy to check that the algorithms are correct.

Next we analyze the space complexity. Note that for a table with N elements, we will use up $O(N)$ space in the linked lists and the array is of size $O(|\Sigma|)$. That is, the total space usage is $O(N + |\Sigma|)$. Thus, if we can pick $|\Sigma| = O(N)$, then we would match the optimal $O(N)$ bound for space.

Now we analyze the time complexity of the various operations. We first note that insertion is $O(1)$ time (assuming computing the hash value takes $O(1)$ time). Note that this also implies that the pre-processing time is $O(N + |\Sigma|)$, which matches the optimal $O(N)$ bound for $|\Sigma| \leq O(N)$. Second, for deletion, the time taken after performing the search algorithm is $O(1)$, assuming the lists as stored as doubly linked lists. (Recall that deleting an item from a doubly linked list if one has a pointer to the entry can be done in $O(1)$ time.)

Finally, we consider the search algorithm. Again assuming that computing a hash value takes $O(1)$ time, the time complexity of the algorithm to search for $\mathbf{x}$ is dominated by size of the list corresponding to $h_i(\mathbf{x})$. In other words, the time complexity is determined by the number of a_j that collide with $\mathbf{x}$, i.e., $h_i(\mathbf{x}) = h_i(a_j)$. We bound this size by the following simple observation.

Claim 20.2.3. *Let $\mathcal{H} = \{h_1, \dots, h_m\}$ with domain $\mathbb{D}$ and range Σ be an ε -almost universal hash family. Then the following is true for any (distinct) $\mathbf{x}, a_1, a_2, \dots, a_N \in \mathbb{D}$:*

$$\mathbb{E}_i [|\{a_j | h_i(\mathbf{x}) = h_i(a_j)\}|] \leq \varepsilon \cdot N,$$

where the expectation is taken over a uniformly random choice of $i \in [m]$.

Proof. Fix a $j \in [N]$. Then by definition of an ε -almost universal hash family, we have that

$$\Pr_i [h_i(\mathbf{x}) = h_i(a_j)] \leq \varepsilon.$$

Note that we want to bound $\mathbb{E} \left[\sum_{j=1}^N \mathbb{1}_{h_i(a_j)=h_i(\mathbf{x})} \right]$. The probability bound above along with the linearity of expectation (Proposition 3.1.4) and Lemma 3.1.3 completes the proof. $\square$

The above discussion then implies the following:

Proposition 20.2.4. *Given an $O(\frac{1}{N})$ -almost universal hash family with domain $\mathbb{D}$ and range Σ such that $|\Sigma| = O(N)$, there exists a randomized data structure that given N elements $a_1, \dots, a_N \in \mathbb{D}$, supports searching, insertion and deletion in expected $O(1)$ time while using $O(N)$ space in the worst-case.*

Thus, Proposition 20.2.4 answers Question 20.1.2 in the affirmative if we can answer the following question in the affirmative:

Question 20.2.1. Given a domain $\mathbb{D}$ and an integer $N \geq 1$, does there exist an $O(\frac{1}{N})$ -almost universal hash function with domain $\mathbb{D}$ and a range Σ such that $|\Sigma| = O(N)$?

We will answer the question above (spoiler alert!) in the affirmative in the next section.

20.3 Almost Universal Hash Function Families and Codes

In this section, we will present a very strong connection between almost universal hash families and codes with good distance: in fact, we will show that they are in fact *equivalent*.

We first begin by noting that any hash family has a very natural code associated with it and that every code has a very natural hash family associated with it.

Definition 20.3.1. Given a hash family $\mathcal{H} = \{h_1, \dots, h_n\}$ where for each $i \in [n]$, $h_i : \mathbb{D} \rightarrow \Sigma$, consider the following associated code

$$C_{\mathcal{H}} : \mathbb{D} \rightarrow \Sigma^n,$$

where for any $\mathbf{x} \in \mathbb{D}$, we have

$$C_{\mathcal{H}}(\mathbf{x}) = (h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_n(\mathbf{x})).$$

The connection also goes the other way. That is, given an $(n, k)_{\Sigma}$ code C , we call the associated hash family $\mathcal{H}_C = \{h_1, \dots, h_n\}$, where for every $i \in [n]$,

$$h_i : \Sigma^k \rightarrow \Sigma$$

such that for every $\mathbf{x} \in \Sigma^k$ and $i \in [n]$,

$$h_i(\mathbf{x}) = C(\mathbf{x})_i.$$

Next we show that an ε -almost universal hash family is equivalent to a code with good distance.

Proposition 20.3.2. Let $\mathcal{H} = \{h_1, \dots, h_n\}$ be an ε -almost universal hash function, then the code $C_{\mathcal{H}}$ has distance at least $(1 - \varepsilon)n$. On the other hand if C is an $(n, k, \delta n)$ -code, then $\mathcal{H}_C$ is a $(1 - \delta)$ -almost universal hash function.

Proof. We will only prove the first claim. The proof of the second claim is essentially identical and is left as an exercise.

Let $\mathcal{H} = \{h_1, \dots, h_n\}$ be an ε -almost universal hash function. Now fix arbitrary $\mathbf{x} \neq \mathbf{y} \in \mathbb{D}$. Then by definition of $C_{\mathcal{H}}$, we have

$$\{i | h_i(\mathbf{x}) = h_i(\mathbf{y})\} = \{i | C_{\mathcal{H}}(\mathbf{x})_i = C_{\mathcal{H}}(\mathbf{y})_i\}.$$

This implies that

$$\Pr_i [h_i(\mathbf{x}) = h_i(\mathbf{y})] = \frac{|\{i | h_i(\mathbf{x}) = h_i(\mathbf{y})\}|}{n} = \frac{n - \Delta(C_{\mathcal{H}}(\mathbf{x}), C_{\mathcal{H}}(\mathbf{y}))}{n} = 1 - \frac{\Delta(C_{\mathcal{H}}(\mathbf{x}), C_{\mathcal{H}}(\mathbf{y}))}{n},$$

where the second equality follows from the definition of the Hamming distance. By the definition of ε -almost universal hash family the above probability is upper bounded by ε , which implies that

$$\Delta(C_{\mathcal{H}}(\mathbf{x}), C_{\mathcal{H}}(\mathbf{y})) \geq n(1 - \varepsilon).$$

Since the choice of $\mathbf{x}$ and $\mathbf{y}$ was arbitrary, this implies that $C_{\mathcal{H}}$ has distance at least $n(1 - \varepsilon)$ as desired. $\square$

20.3.1 The Polynomial Hash Function

We now consider the hash family corresponding to a Reed-Solomon code. In particular, let C be a $[q, k, q - k + 1]_q$ Reed-Solomon code. By Proposition 20.3.2, the hash family $\mathcal{H}_C$ is an $\frac{k-1}{q}$ -almost universal hash family—this hash family in the literature is called the polynomial hash. Thus, if we pick q to be the smallest power of 2 larger than N and pick $k = O(1)$, then this leads to an $O(1/N)$ -universal hash family that satisfies all the required properties in Question 20.2.1.

Note that the above implies that $|\mathbb{D}| = N^{O(1)}$. One might wonder if we can get an $O(1/N)$ -almost universal hash family with the domain size being $N^{\omega(1)}$. We leave the resolution of this question as an exercise.

20.4 Data Possession Problem

In this section, we return to Question 20.1.1. Next we formalize the protocol for the data possession problem that we outlined in Section 20.2. Algorithm 20.4.1 presents the pre-processing step.

Algorithm 20.4.1 Pre-Processing for Data Possession Verification

INPUT: Data $\mathbf{x} \in \mathbb{D}$, hash family $\mathcal{H} = \{h_1, \dots, h_m\}$ over domain $\mathbb{D}$

- 1: Client computes an index i for $\mathbf{x}$.
 - 2: Client picks a random $j \in [m]$.
 - 3: Client computes $z \leftarrow h_j(\mathbf{x})$ and stores (i, j, z) .
 - 4: Client sends $\mathbf{x}$ to the server.
-

Algorithm 20.4.2 formally states the verification protocol. Note that if the server has stored $\mathbf{x}$ (or is able to re-compute $\mathbf{x}$ from what it had stored), then it can pass the protocol by returning $a \leftarrow h_j(\mathbf{x})$. Thus, for the remainder of the section, we will consider the case when the server tries to cheat. We will show that if the server is able to pass the protocol in Algorithm 20.4.2 with high enough probability, then the server indeed has stored $\mathbf{x}$.

Algorithm 20.4.2 Verification for Data Possession Verification

INPUT: Index i of data $\mathbf{x} \in \mathbb{D}$ OUTPUT: 1 if Server has $\mathbf{x}$ and 0 otherwise

```

1: Client sends a challenge  $(i, j)$  to the server.
2: Client receives an answer  $a$ .
3: IF  $a = z$  THEN
4:   RETURN 1
5: ELSE
6:   RETURN 0

```

Before we formally prove the result, we state our assumptions on what the server can and cannot do. We assume that the server follows the following general protocol. First, when the server receives $\mathbf{x}$, it does performs some computation (that terminates) on $\mathbf{x}$ to produce $\mathbf{y}$ and then it stores $\mathbf{y}$. (E.g., the server could store $\mathbf{y} = \mathbf{x}$ or $\mathbf{y}$ could be a compressed version of $\mathbf{x}$.) Then when it receives the challenge (i, j) for $\mathbf{x}$, it uses another algorithm $\mathcal{A}$ and returns the answers $a \leftarrow \mathcal{A}(\mathbf{y}, j)$. We assume that $\mathcal{A}$ always terminates on any input.⁴ Note that the server is allowed to use arbitrary (but finite) amount of time to compute its answer. Next, we will prove that under these assumptions, the server cannot cheat with too high a probability.

Theorem 20.4.1. *Assume that the hash family $\mathcal{H}$ is an ε -almost universal hash family. Then if the server passes the protocol in Algorithm 20.4.2 with probability $> \frac{1}{2} + \frac{\varepsilon}{2}$, then the server has enough information to recreate $\mathbf{x}$.*

Proof. To prove the claim, we present an algorithm that can compute $\mathbf{x}$ from $\mathbf{y}$. (Note that we do not need this algorithm to be efficient: it just needs to terminate with $\mathbf{x}$.) In particular, consider Algorithm 20.4.3.

Algorithm 20.4.3 Decompression Algorithm

INPUT: $\mathcal{A}, \mathbf{y}$ OUTPUT: $\mathbf{x}'$

```

1:  $\mathbf{z} \leftarrow (\mathcal{A}(\mathbf{y}, j))_{j \in [m]}$ .
2: Run the MLD algorithm (Algorithm 1.4.1) for  $C_{\mathcal{H}}$  on  $\mathbf{z}$  and let  $C_{\mathcal{H}}(\mathbf{x}')$  be its output.
3: RETURN  $\mathbf{x}'$ 

```

To complete the proof, we will show that $\mathbf{x}' = \mathbf{x}$. Towards this end we claim that $\Delta(\mathbf{z}, C_{\mathcal{H}}(\mathbf{x})) < \frac{m}{2} \cdot (1 - \varepsilon)$. Assuming this is true, we complete the proof. Note that Proposition 20.3.2 implies that $C_{\mathcal{H}}$ has distance at least $m(1 - \varepsilon)$. Thus, Proposition 1.4.2 (in particular, its proof) implies that Algorithm 1.4.1 will return $C_{\mathcal{H}}(\mathbf{x})$ and thus, $\mathbf{x}' = \mathbf{x}$, as desired.

⁴We have stated the algorithm to be independent of $\mathbf{y}$ and j but that need not be the case. However later in the section, we will need the assumption that $\mathcal{A}$ is independent of $\mathbf{y}$ and j , so we will keep it that way.

Finally, we argue that $\Delta(\mathbf{z}, C_{\mathcal{H}}(\mathbf{x})) < m(1 - \varepsilon)/2$. To see this note that if the server passes the protocol in Algorithm 20.4.2 (i.e. the client outputs 1), then it has to be the case that $z_j \stackrel{\text{def}}{=} \mathcal{A}(\mathbf{y}, j) = h_j(\mathbf{x})$. Recall that by definition of $C_{\mathcal{H}}$, $h_j(\mathbf{x}) = C_{\mathcal{H}}(\mathbf{x})_j$ and that the server passes the protocol with probability $> 1/2 + \varepsilon/2$. Since j is chosen uniformly from $[m]$, this implies that for $> m(1/2 + \varepsilon/2)$ positions j , $z_j = C_{\mathcal{H}}(\mathbf{x})_j$, which then implies the claim. $\square$

20.4.1 Driving Down the Cheating Probability

One of the unsatisfying aspects of Theorem 20.4.1 is that the probability of catching a “cheating” server is strictly less than 50%.⁵ It is of course desirable to drive this up as close to 100% as possible. One way to obtain this would be to “repeat” the protocol: i.e. the client can choose multiple random hashes and store the corresponding values (in Algorithm 20.4.1) and (in Algorithm 20.4.2) asks the server to send back all the hash values and accepts if and only if all the returned answers match with the stored hash values. This however, comes at a cost: the client has to store more hash values (and also the communication cost between the client and the server goes up accordingly.)

Next we argue using list decoding that the protocol in Algorithm 20.4.1 and 20.4.2 (without any modification) gives a more powerful guarantee than the one in Theorem 20.4.1. To see why list decoding might buy us something, let us look back at Algorithm 20.4.3. In particular, consider Step 2: since we run MLD, we can only guarantee unique decoding up to half the (relative) distance of $C_{\mathcal{H}}$. This in turn leads to the bound of $1/2 + \varepsilon/2$ in Theorem 20.4.1. We have seen that list decoding allows us to go beyond half the distance number of errors. So maybe, running a list decoding algorithm instead of MLD in Step 2 of Algorithms 20.4.3 would help us get better results. There are two potential issues that we’ll need to tackle:

- We will need a general bound that shows that list decoding (arbitrarily) close to 100% is possible for any $C_{\mathcal{H}}$ for an ε -almost universal hash family; and
- Even if the above is possible, what will we do when a list decoding algorithm returns a *list* of possible data?

We will get around the first concern by using the Johnson bound 7.3.1. To get around the second issue we will indirectly use “side information” (like we mentioned in Section 7.2). For the latter, we will need the notion of Kolmogorov complexity, which captures the amount of information content in any given string. For our purposes, the following informal description is enough:

Definition 20.4.2. *Given a string $\mathbf{x}$, its Kolmogorov complexity, denoted by $\mathcal{K}(\mathbf{x})$ is the minimum of $|\mathbf{y}| + |\mathcal{D}|$, where $\mathcal{D}$ is a decompression algorithm that on input $\mathbf{y}$ outputs $\mathbf{x}$ (where $|\mathbf{x}|$ and $|\mathcal{D}|$ are the length of $\mathbf{x}$ and (a description of) $\mathcal{D}$ in bits).*

Informally, $\mathcal{K}(\mathbf{x})$ is the amount of information that can be obtained algorithmically from $\mathbf{x}$. Kolmogorov complexity is a fascinating topic that it outside the scope of this book. Here we will

⁵Note that Theorem 20.4.1 states that a server that cannot recreate $\mathbf{x}$ can pass the test with probability at most $1/2 + \varepsilon/2$. In other words, the probability that such a server is caught is at most $1/2 - \varepsilon/2 < 1/2$.

only need to use the definition of $\mathcal{K}(\mathbf{x})$. We are now ready to prove the following list decoding counterpart of Theorem 20.4.1:

Theorem 20.4.3. *Assume that the hash family $\mathcal{H}$ is an ε -almost universal hash family. Then if the server passes the protocol in Algorithm 20.4.2 with probability $> \sqrt{\varepsilon}$, then the amount of information server has stored for $\mathbf{x}$ is at least $\mathcal{K}(\mathbf{x}) - O(\log|\mathbf{x}|)$.*

We note that the above is not a strict generalization of Theorem 20.4.1, as even though probability of catching a cheating server has gone up our guarantee is weaker. Unlike Theorem 20.4.1, where we can guarantee that the server can re-create $\mathbf{x}$, here we can only guarantee “storage enforceability”—i.e. we can only force a server to store close to $\mathcal{K}(\mathbf{x})$ amounts of memory.

Proof of Theorem 20.4.3. Here is the main idea of the proof. We first assume for the sake of contradiction that $|\mathbf{y}| < \mathcal{K}(\mathbf{x}) - O(\log(|\mathbf{x}|))$. Then using we construct a decompression algorithm $\mathcal{D}$ that on given input $\mathbf{y}$ and $O(\log(|\mathbf{x}|))$ extra information (or “advice”), outputs $\mathbf{x}$. Then we will show this overall contradicts the definition of $\mathcal{K}(\mathbf{x})$ (as this gives an overall smaller description of $\mathbf{x}$).

Before we state the decompression algorithm, we recall some facts. First note that $C_{\mathcal{H}}$ by Proposition 20.3.2 is a q -ary code (with $|\Sigma| = q$) with distance $m(1 - \varepsilon)$. Further, by the Johnson bound (Theorem 7.3.1), $C_{\mathcal{H}}$ is a $(1 - \sqrt{\varepsilon}, L)$ -list decodable, where

$$L \leq qm^2. \quad (20.1)$$

Next, in Algorithm 20.4.4, we present a decompression algorithm that can compute $\mathbf{x}$ from $\mathbf{y}$ and an advice string $a \in [L]$. (As before, we do not need this algorithm to be efficient: it just needs to terminate with $\mathbf{x}$.)

Algorithm 20.4.4 Decompression Algorithm Using List Decoding

INPUT: $\mathcal{A}, \mathbf{y}, a$

OUTPUT: $\mathbf{x}$

```

1:  $\mathbf{z} \leftarrow (\mathcal{A}(\mathbf{y}, j))_{j \in [m]}$ .
2:  $\mathcal{L} \leftarrow \emptyset$ .
3: FOR  $\mathbf{x}' \in \mathbb{D}$  DO
4:   IF  $\Delta(C_{\mathcal{H}}(\mathbf{x}'), \mathbf{z}) \leq (1 - \sqrt{\varepsilon})m$  THEN
5:     Add  $\mathbf{x}'$  to  $\mathcal{L}$ 
6: RETURN The  $a$ th element in  $\mathcal{L}$ 
```

To complete the proof, we claim that there exists a choice of $a \in [L]$ such that Algorithm 20.4.4 outputs $\mathbf{x}$. Note that this implies that $(\mathbf{y}, a)$ along with Algorithm 20.4.4 gives a complete description of $\mathbf{x}$. Now note that Algorithm 20.4.4 can be described in $O(1)$ bits. This implies that the size of this description is $|\mathbf{y}| + \log L + O(1)$, which by Definition 20.4.2 has to be at least $\mathcal{K}(\mathbf{x})$. This implies that

$$|\mathbf{y}| \geq \mathcal{K}(\mathbf{x}) - |a| - O(1) = \mathcal{K}(\mathbf{x}) - \log L - O(1) \geq \mathcal{K}(\mathbf{x}) - O(\log|\mathbf{x}|),$$

where the last inequality follows from (20.1).

Next, we argue the existence of an appropriate $a \in [L]$. Towards this end we claim that $\Delta(\mathbf{z}, C_{\mathcal{H}}(\mathbf{x})) < m(1 - \sqrt{\varepsilon})$. Note that this implies that $\mathbf{x} \in \mathcal{L}$. Since $|\mathcal{L}| \leq L$, then we can just assign a to be the index of $\mathbf{x}$ in $\mathcal{L}$. Finally, we argue that $\Delta(\mathbf{z}, C_{\mathcal{H}}(\mathbf{x})) < m(1 - \sqrt{\varepsilon})$. To see this note that if the server passes the protocol in Algorithm 20.4.2 (i.e. the client outputs 1), then it has to be the case that $z_j \stackrel{\text{def}}{=} \mathcal{A}(\mathbf{y}, j) = h_j(\mathbf{x})$. Recall that by definition of $C_{\mathcal{H}}$, $h_j(\mathbf{x}) = C_{\mathcal{H}}(\mathbf{x})_j$ and that the server passes the protocol with probability $> \sqrt{\varepsilon}$. Since j is chosen uniformly from $[m]$, this implies that for $> m\sqrt{\varepsilon}$ positions j , $z_j = C_{\mathcal{H}}(\mathbf{x})_j$, which then implies the claim. $\square$

20.5 Bibliographic Notes

Universal hash functions were defined in the seminal paper of Carter and Wegman [26]. Almost universal hash function family was defined by Stinson [148].

Kolmogorov complexity was defined by Kolmogorov [101]. For a thorough treatment see the textbook by Li and Vitányi [105].

Chapter 21

Securing Your Fingerprints: Fuzzy Vaults

String-based passwords are the dominant mode of authentication in today's information-reliant world. Indeed, all of us use passwords on a regular basis to authenticate ourselves in almost any online activity. Strings, a primitive data type, have become widespread due to several nice mathematical properties. First, matching two strings (that is, checking if two strings are exactly the same) is computationally very fast (and easy). Second, and more importantly, there exist secure hash functions that map a string x to another string $h(x)$ such that, given $h(x)$, determining x is hard. Furthermore, since $h(x)$ is itself a string, we can check if a claimed password y is the same as the original string x by comparing $h(y)$ and $h(x)$, which (as we just observed) is easy to do. This implies that the server performing the authentication only needs to store the hashes $h(x)$ of the original passwords. Hence, even if the list of hashed passwords were compromised, the passwords themselves would remain secure.

The above scheme is perfect as long as the passwords x are “random enough,” and this can be achieved if the passwords were generated randomly by some automated process. However, in real life passwords are generated by humans and are not really random. (One of the most quoted facts is that the most commonly used password is the string “password” itself.) Further, we tend to forget passwords, which has led to the near ubiquity of “Forgot passwords” links in places where we need to login.

One alternative that has been gaining traction in the last decade or so is to use a user's fingerprint (and more generally their biometrics: e.g. their face) as their password. The big advantage is that it is hard to “forget” one's fingerprint. In this chapter, we will look at the issues in using fingerprints as passwords and see how Reed-Solomon codes can help.

21.1 Some quick background on fingerprints

First we give a very short (and necessarily incomplete) description of how a fingerprint (image/sensor reading) is converted into a string f . The standard way to store a fingerprint (see Figure 21.1 for images of two fingerprints) is as a collection of triples, called minutia. Each minutia point is located where one ridge¹ splits into two, or where one ridge ends. The i^{th}

¹In Figure 21.1, the ridges are the lines.

minutia is the triple (x_i, y_i, θ_i) , where x_i and y_i are the x and y coordinates of a point on the finger, and θ_i indicates the direction of the ridge that created the minutia point relative to the plane.

Given that we now have a string representation of a fingerprint, we have the following naive solution for designing a hash function for the fingerprint f :

Naive Solution. Use any off-the-shelf hash function h for strings and then store $h(f)$ instead of f .

To see the issues with the naive solution, we first need to know a little bit about how fingerprints are stored.

The main issue with our naive solution is that two fingerprint readings will never be exactly the same, even if the same finger is used. For any two fingerprint readings, the following issues may produce errors:

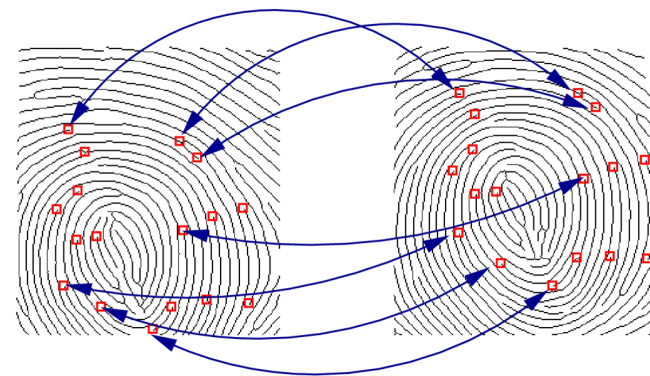
1. Translation and rotation, even when using the same finger.
2. Varying pressure.
3. Each reading may not completely overlap with the ideal fingerprint region (i.e., the finger may be slightly tilted).
4. The minutiae are not ordered, so they form a set instead of a vector. Of course one can sort the set by the lexicographic order to produce a string. But the earlier issue (especially points 1 and 2) imply that the specific values of (x_i, y_i, θ_i) are not that important individually. Furthermore, the fact that any two readings might not have complete overlap means that we are interested in matching readings that have significant overlap, so it turns out that the set notation is ideal to theoretically deal with these issues.

We can now see that the naive solution is inadequate. Even if we could somehow correct the first three issues, existing hash functions for strings require a vector, not a set, so our naive solution will fail.

Remark 21.1.1. *The four problems that came up in our naive solution will come up in any solution we propose. Technology has not yet developed to the point where we can securely eliminate these issues, which is why there are no prevalent secure commercial systems that safeguard secrets using fingerprints. (The reason government agencies, such as the police or FBI, use fingerprinting is because there is an inherent trust that the government will keep your data secure, even when it does not apply a good hash function to it.)*

Thus, we need secure hash functions designed to handle the additional challenges posed by fingerprints. We would like to mention that for fingerprints to replace strings as passwords, the hash function needs to satisfy both of these properties *simultaneously*: (i) we should be able to match hashes from the “same” fingerprint and (ii) an adversary should not be able to “break” the hash function.

Figure 21.1: The minutiae are unordered and form a set, not a vector.



21.2 The Fuzzy Vault Problem

We begin with the fuzzy vault problem, which is slightly different from the one we have been studying so far. Say you have a secret string s , and you want to store it in a secure way. Instead of using a password, you want to use your fingerprint, f , to “lock” the secret s . You want the locked version of your secret to have two properties:

1. You should be able to “unlock” the locked version of s
2. No one else should be able to “unlock” s

We claim that if we can solve the above problem, then we can solve the problem of designing a secure hash function for fingerprints. We leave the details as an exercise. (Hint: pick s at random and then in addition to the locked version of s also store $h(s)$, where h is an off-the-shelf secure hash function for strings.)

We will now formalize the fuzzy vault problem.

21.2.1 Formal Problem Statement

The first thing we need to do is quantize the measurements of the minutiae. We cannot be infinitely precise in our measurements anyways, so let us assume that all quantized minutiae, (x_i, y_i, θ_i) , can be embedded into $\mathbb{F}_q$ for some large enough prime power q . Theoretically, this can also help to correct the first two issues from our naive solution. We could go through all possible values in $\mathbb{F}_q$ to get rid of translation and rotation errors (e.g., for every $(\Delta x, \Delta y, \Delta z) \in \mathbb{F}_q$, we rotate and translate each minutia (x_i, y_i, z_i) to $(x_i + \Delta x, y_i + \Delta y, z_i + \Delta z)$).² We could also

²To be more precise we first perform the translation and rotation over the reals and then quantize and map to the appropriate $\mathbb{F}_q$ value.

do some local error correction to a quantized value to mitigate the effect of varying pressure. Going over all possible shifts is not a practical solution, but theoretically this can still lead to a polynomial-time solution.

We now formally define our problem, which primarily captures issues 3 and 4. (Below for any integers $t \geq 1$, $\binom{\mathbb{F}_q}{t}$ denotes the set of all subsets of $\mathbb{F}_q$ of size exactly t .) The following are the components of the problem:

- Integers $k \geq 1, n \geq t \geq 1$
- Secret $s \in \mathbb{F}_q^k$
- Fingerprint $f \in \binom{\mathbb{F}_q}{t}$
- $\text{LOCK} : \mathbb{F}_q^k \times \binom{\mathbb{F}_q}{t} \rightarrow \binom{\mathbb{F}_q}{n}$
- $\text{UNLOCK} : \binom{\mathbb{F}_q}{t} \times \binom{\mathbb{F}_q}{n} \rightarrow \mathbb{F}_q^k$

The goal of the problem is to define the functions LOCK and UNLOCK such that they satisfy these two properties (for some $c < t$):

1. (c -completeness.) For any $f, f' \in \binom{\mathbb{F}_q}{t}$ such that $|f - f'| \leq c$, the following should hold:

$$\text{UNLOCK}(\text{LOCK}(s, f), f') = s.$$

2. (Soundness.) It should be “hard” for an adversary to get s from $\text{LOCK}(s, f)$. (The notion of “hard” will be more formally defined later.)

Note that the completeness property corresponds to the matching property we need from our hash function, while the soundness property corresponds to the security property of the hash function.

21.2.2 Two Futile Attempts

We begin with two attempts at designing the LOCK and UNLOCK functions, which will not work. However, later we will see how we can combine both to get our final solution.

For this section, unless mentioned otherwise, we will assume that the original fingerprint f is given by the set $\{\alpha_1, \dots, \alpha_t\}$.

Attempt 1. We begin with a scheme that focuses on the soundness property. A very simple idea, which is what we will start off with, would be to just add $n - t$ random values to f to get our vault. The intuition, which can be made precise, is that an adversary just looking at the vault will just see random points and will not be able to recover f from the random set of points. The catch of course is that this scheme has terrible completeness. In particular, if we get a match between a value in the second fingerprint f' and the vault, we have no way to know whether the match is to one of the original values in f or if the match is with one of the random “chaff” points there were added earlier.

Attempt 2. Next, we specify a scheme that has good completeness (but has pretty bad soundness).

We begin with the LOCK function:

$$\text{LOCK}_2(s, f) = \{(\alpha_1, P_s(\alpha_1)), \dots, (\alpha_t, P_s(\alpha_t))\},$$

where $P_s(X) = \sum_{i=0}^{k-1} s \cdot X^i$ and recall $f = \{\alpha_1, \dots, \alpha_t\}$. (Note that we have $n = t$.)

The main intuition behind LOCK_2 is the following. Given another fingerprint $f' = \{\beta_1, \dots, \beta_t\}$ such that it is close enough to f , i.e. $|f \setminus f'| \leq c$, for every value in $f \cap f'$, we will know the corresponding P_s value and thus, we can use the fact that we can decode Reed-Solomon codes from erasures to recover the secret s . We formally present UNLOCK_2 as Algorithm 21.2.1.

Algorithm 21.2.1 UNLOCK_2

INPUT: Vault $\{(\alpha_1, y_1), \dots, (\alpha_t, y_t)\} = \text{LOCK}(s, f)$ and another fingerprint $f' = \{\beta_1, \dots, \beta_t\}$

OUTPUT: s if $|f \setminus f'| \leq c$

```

1: FOR  $i = 1, \dots, t$  DO
2:   IF there exists a  $j \in [t]$  such that  $\alpha_i = \beta_j$  THEN
3:      $z_j \leftarrow y_i$ 
4:   ELSE
5:      $z_j \leftarrow ?$ 
6:  $\mathbf{z} \leftarrow (z_1, \dots, z_t)$ 
7: Run Algorithm from Theorem 14.2.1 to correct  $\mathbf{z}$  from erasures for RS codes with evaluation points  $\{\beta_1, \dots, \beta_t\}$  and output resulting message as  $s$ .
```

The following result is fairly simple to argue.

Lemma 21.2.1. *The pair $(\text{LOCK}_2, \text{UNLOCK}_2)$ of functions is $(t - k)$ -complete. Further, both functions can be implemented in polynomial time.*

Proof. Let us assume $|f \setminus f'| \leq t - k$. Now as both f and f' have exactly t values, this means that $\mathbf{z}$ has at most $t - k$ erasures. Thus, by Theorem 14.2.1, Step 6 will output s and UNLOCK_2 can be implemented in polynomial time. Further, the claim on the polynomial run time of LOCK_2 follows from the fact that one can do encoding of Reed-Solomon code in polynomial time. $\square$

Unfortunately, $(\text{LOCK}_2, \text{UNLOCK}_2)$ pair has terrible soundness. This is because the vault $\{(\alpha_1, y_1), \dots, (\alpha_t, y_t)\}$ has f in the first component in each pair. This an adversary can just read off those values and present $f' = \{\alpha_1, \dots, \alpha_t\}$, which would imply that $\text{UNLOCK}_2(\text{LOCK}_2(s, f), f') = s$, which means that the vault would be “broken.”

21.3 The Final Fuzzy Vault

So far we have seen two attempts: one that (intuitively) has very good soundness but no completeness and another which has good completeness but terrible soundness. It is natural to consider if we can combine both of these attempts and get the best of both worlds. Indeed, it turns we can easily combine both of our previous attempts to get the final fuzzy vault.

Algorithm 21.3.1 presents the new LOCK_3 function.

Algorithm 21.3.1 LOCK_3

INPUT: Fingerprint $f = \{\alpha_1, \dots, \alpha_t\}$ and secret $s = (s_0, \dots, s_{k-1}) \in \mathbb{F}_q^k$
 OUTPUT: Vault with f locking s

```

1:  $R, T \leftarrow \emptyset$ 
2:  $P_s(X) \leftarrow \sum_{i=0}^{k-1} s_i \cdot X^i$ 
3: FOR  $i = 1, \dots, t$  DO
4:    $T \leftarrow T \cup \{\alpha_i\}$ 
5:    $R \leftarrow R \cup \{(\alpha_i, P_s(\alpha_i))\}$ 
6: FOR  $i = t+1, \dots, n$  DO
7:    $\alpha_i$  be a random element from  $\mathbb{F}_q \setminus T$ 
8:    $T \leftarrow T \cup \{\alpha_i\}$ 
9:    $\gamma$  be a random element from  $\mathbb{F}_q \setminus P_s(\alpha)$ 
10:   $R \leftarrow R \cup \{(\alpha, \gamma)\}$ 
11: Randomly permute  $R$ 
12: RETURN  $R$ 
```

Algorithm 21.3.2 presents the new UNLOCK_3 function.

The following result is a simple generalization of Lemma 21.2.1.

Lemma 21.3.1. *The pair $(\text{LOCK}_3, \text{UNLOCK}_3)$ of functions is $(t-k)/2$ -complete. Further, both functions can be implemented in polynomial time.*

Proof. Let us assume $|f \setminus f'| \leq (t-k)/2$. Now as both f and f' have exactly t values, it implies that $|f \cap f'| \geq (t+k)/2$. Further for each $j \in [t]$ such that $\beta_j \in f \cap f'$, we have that $z_j = P_s(\beta_j)$. In other words, this means that $\mathbf{z}$ has at most $(t-k)/2$ errors.³ Thus, by Theorem 14.2.2, Step 6 will output s and UNLOCK_3 can be implemented in polynomial time. Further, the claim on the polynomial run time of LOCK_3 follows from the fact that one can do encoding of Reed-Solomon code in polynomial time. $\square$

³To be more precise if $\mathbf{z}$ has e errors and s erasures w.r.t. the codeword corresponding to s , then $2e + s \leq t - k$.

Algorithm 21.3.2 UNLOCK_3

INPUT: Vault $\{(\alpha_1, y_1), \dots, (\alpha_n, y_n)\} = \text{LOCK}(s, f)$ and another fingerprint $f' = \{\beta_1, \dots, \beta_t\}$
 OUTPUT: s if $|f \setminus f'| \leq c$

```

1: FOR  $i = 1, \dots, t$  DO
2:   IF there exists a  $j \in [n]$  such that  $\alpha_i = \beta_j$  THEN
3:      $z_j \leftarrow y_i$ 
4:   ELSE
5:      $z_j \leftarrow ?$ 
6:  $\mathbf{z} \leftarrow (z_1, \dots, z_t)$ 
7: Run Algorithm from Theorem 14.2.2 to correct  $\mathbf{z}$  from errors and erasures for RS codes with
   evaluation points  $\{\beta_1, \dots, \beta_t\}$  and output resulting message as  $s$ .
```

21.3.1 Soundness

To avoid getting into too much technical details, we will present a high level argument for why the proposed fuzzy vault scheme has good soundness. Given a vault $\{(\alpha_1, y_1), \dots, (\alpha_n, y_n)\} = \text{LOCK}_3(s, f)$, we know that there are exactly t values (i.e. those $\alpha_j \in f$) such that the polynomial $P_s(X)$ agrees with the vault on exactly those t points. Thus, an intuitive way to argue the soundness of the vault would be to argue that there exists a lot other secrets $s' \in \mathbb{F}_q^k$ such that $P_{s'}(X)$ also agrees with the vault in exactly t positions. (One can formalize this intuition and argue that the vault satisfies a more formal definition of soundness but we will skip those details.)

We will formalize the above argument by proving a slightly different result (and we will leave the final proof as an exercise).

Lemma 21.3.2. *Let $V = \{(x_1, y_1), \dots, (x_n, y_n)\}$ be n independent random points from $\mathbb{F}_q \times \mathbb{F}_q$. Then, in expectation, there are at least $\frac{1}{3} \cdot q^k \cdot \left(\frac{n}{qt}\right)^t$ polynomials $P(X)$ of degree at most $k-1$ such that for exactly t values of $j \in [n]$, we have $P(x_j) = y_j$.*

Proof. Consider a fixed polynomial $P(X)$ and a $j \in [n]$. Then for any $x_j \in \mathbb{F}_q$, the probability that for a random $y_j \in \mathbb{F}_q$, $P(x_j) = y_j$ is exactly $1/q$. Further, these probabilities are all independent. This implies that the probability that $P(X)$ agrees with V in exactly t positions is given by

$$\binom{n}{t} \cdot \left(\frac{1}{q}\right)^t \cdot \left(1 - \frac{1}{q}\right)^{n-t} \geq \frac{1}{3} \left(\frac{n}{qt}\right)^t.$$

Since there are q^k such polynomials, the claimed result follows. $\square$

We note that there are two aspects of the above lemma that are not satisfactory. (i) The result above is for a vault V with completely random points whereas we would like to prove a similar result but with $V = \text{LOCK}_3(s, f)$ and (ii) Instead of a bound in expectation, we would like to prove a similar exponential lower bound but with high probability. We leave the proof that these can be done as an exercise. (Hint: Use the “Inverse Markov Inequality.”)

21.4 Bibliographic Notes

The fuzzy vault presented in this chapter is due to Juels and Sudan [95]. The “inverse Markov inequality” first appeared in Dumer et al. [41].

Chapter 22

Finding Defectives: Group Testing

Consider the following situation that arises very frequently in *biological screens*. Say there are N individuals and the objective of the study is to identify the individuals with a certain “biomarker” that could e.g. indicate that the individual has some specific disease or is at risk for a certain health condition. The naive way to do this would be to test each person individually, that is:

1. Draw sample (e.g. a blood or DNA sample) from a given individual,
2. Perform required tests, then
3. Determine presence or absence of the biomarker.

This method of one test per person will give us a total of N tests for a total of N individuals. Say we had more than 70 – 75% of the population infected. At such large numbers, the use of the method of individual testing is reasonable. However, our goal is to achieve effective testing in the more likely scenario where it doesn't make sense to test 100,000 people to get just (say) 10 positives.

The feasibility of a more effective testing scheme hinges on the following property: We can combine blood samples and test a combined sample together to check if at least one individual has the biomarker.

The main question in group testing is the following: If we have a very large number of items to test, and we know that only certain few will turn out positive, what is a nice and efficient way of performing the tests?

22.1 Formalization of the problem

We now formalize the group testing problem. The input to the problem consists of the following:

- The total number of individuals: N .
- An upper bound on the number of infected individuals d .

- The input can be described as a vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ where $x_i = 1$ if individual i has the biomarker, else $x_i = 0$.

Note that $wt(\mathbf{x}) \leq d$. More importantly, notice that the vector $\mathbf{x}$ is an implicit input since we do not know the positions of 1s in the input. The only way to find out is to run the *tests*. Now, we will formalize the notion of a test.

A query/test S is a subset of $[N]$. The answer to the query $S \subseteq [N]$ is defined as follows:

$$A(S) = \begin{cases} 1 & \text{if } \sum_{i \in S} x_i \geq 1; \\ 0 & \text{otherwise.} \end{cases}$$

Note that the answer to the S is the logical-OR of all bits in S , i.e. $A(S) = \bigvee_{i \in S} x_i$.

The goal of the problem is to compute $\mathbf{x}$ and minimize the number of tests required to determine $\mathbf{x}$.

Testing methods. There is another aspect of the problem that we need specify. In particular, we might need to restrict how the tests interact with each other. Below are two common ways to carry out tests:

1. *Adaptive group testing* is where we test a given subset of items, get the answer and base our further tests on the outcome of the previous set of tests and their answers.
2. *Non-Adaptive group testing* on the other hand is when all our tests are set even before we perform our first test. That is, all our tests are decided a priori.

Non-adaptive group testing is crucial for many applications. This is because the individuals could be geographically spread pretty far out. Due to this, adaptive group testing will require a very high degree of co-ordination between the different groups. This might actually increase the cost of the process.

Notation. We will also need notation to denote the minimum number of tests needed in group testing. Towards this end, we present the following two definitions.

Definition 22.1.1 ($t(d, N)$). Given a subset of N items with d defects represented as $\mathbf{x} \in \{0, 1\}^N$, the minimum number of non-adaptive tests that one would have to make is defined as $t(d, N)$.

Definition 22.1.2. $t^a(d, N)$: Given a set of N items with d defects, $t^a(d, N)$ is defined as the number of adaptive tests that one would have to make to detect all the defective items.

The obvious questions are to prove bounds on $t(d, N)$ and $t^a(d, N)$:

Question 22.1.1. Prove asymptotically tight bounds on $t(d, N)$.

Question 22.1.2. Prove asymptotically tight bounds on $t^a(d, N)$.

We begin with some simple bounds:

Proposition 22.1.3. For every $1 \leq d \leq N$, we have

$$1 \leq t^a(d, N) \leq t(d, N) \leq N.$$

Proof. The last inequality follows from the naive scheme of testing all individuals with singleton tests while the first inequality is trivial. The reason for $t^a(d, N) \leq t(d, N)$ is due to the fact that any non-adaptive test can be performed by an adaptive test by running all of the tests in the first step of the adaptive test. Adaptive tests can be faster than non-adaptive tests since the test can be changed after certain things are discovered. $\square$

Representing the set of tests as a matrix. It turns out that is is useful to represent a non-adaptive group testing scheme as a matrix. Next, we outline such a representation. For, $S \subseteq [N]$, define $\chi_S \in \{0, 1\}^N$ such that $i \in S$ if and only if $\chi_S(i) = 1$. Consider a non-adaptive group testing scheme with t test $S_1, \dots, S_t$. The corresponding matrix M is a $t \times N$ matrix where the i th row is χ_{S_i} . (Note that the trivial solution of testing each individual as a singleton set would just be the $N \times N$ identity matrix.) In other words, $M = \{M_{ij}\}_{i \in [t], j \in [N]}$ such that $M_{ij} = 1$ if $j \in S_i$ and $M_{ij} = 0$ otherwise. For example, consider the case of $n = 3$ with $t = 2$ such that $S_1 = \{1, 2\}$ and $S_2 = \{2, 3\}$. Then the corresponding testing matrix will be:

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

If we assume that multiplication is logical AND ($\wedge$) and addition is logical OR ($\vee$), then we have $M \times \mathbf{x} = \mathbf{r}$ where $\mathbf{r} \in \{0, 1\}^t$ is the vector of the answers to the t tests. We will denote this operation as $M \odot \mathbf{x}$. To think of this in terms of testing, it is helpful to visualize the matrix-vector multiplication. Here, $\mathbf{r}$ will have a 1 in position i if and only if there was a 1 in that position in both M and $\mathbf{x}$ i.e. if that person was tested with that particular group and if they tested to be positive.

Thus, our goal is to compute $\mathbf{x}$ from $M \odot \mathbf{x}$ with as small a t as possible.

22.2 Bounds on $t^a(d, N)$

In this section, we will explore lower and upper bounds on $t^a(d, N)$ with the ultimate objective of answering Question 22.1.2.

We begin with a lower bound that follows from a simple counting argument.

Proposition 22.2.1. For every $1 \leq d \leq N$,

$$t^a(d, N) \geq d \log \frac{N}{d}.$$

Proof. Fix any valid adaptive group testing scheme with t tests. Observe that if $\mathbf{x} \neq \mathbf{y} \in \{0, 1\}^N$, with $wt(\mathbf{x}), wt(\mathbf{y}) \leq d$ then $\mathbf{r}(\mathbf{x}) \neq \mathbf{r}(\mathbf{y})$, where $\mathbf{r}(\mathbf{x})$ denotes the result vector for running the tests on $\mathbf{x}$ and similarly for $\mathbf{r}(\mathbf{y})$. The reason for this is because two valid inputs cannot give the same result. If this were the case and the results of the tests gave $\mathbf{r}(\mathbf{x}) = \mathbf{r}(\mathbf{y})$ then it would not be possible to distinguish between $\mathbf{x}$ and $\mathbf{y}$.

The above observation implies that the total number of distinct test results is the number of distinct binary vectors with Hamming weight at most d , i.e. $Vol_2(d, N)$. On the other hand, the number of possible distinct t -bit vectors is at most 2^t , which with the previous argument implies that

$$2^t \geq Vol_2(d, N)$$

and hence, it implies that

$$t \geq \log Vol_2(d, N).$$

Recall that

$$Vol_2(d, N) \geq \binom{N}{d} \geq \left(\frac{N}{d}\right)^d,$$

where the first inequality follows from (3.23) and the second inequality follows from Lemma B.1.1. So $t \geq d \log(N/d)$, as desired. $\square$

It turns out that $t^a(d, N)$ is also $O(d \log(\frac{N}{d}))$. (See Exercise 22.1.) This along with Proposition 22.2.1 implies that $t^a(d, N) = \Theta(d \log(\frac{N}{d}))$, which answers Question 22.1.2. The upper bound on $t^a(d, N)$ follows from an adaptive group testing scheme and hence does not say anything meaningful for Question 22.1.1. (Indeed, we will see later that $t(d, N)$ cannot be $O(d \log(N/d))$.) Next, we switch gears to talk about non-adaptive group testing.

22.3 Bounds on $t(d, N)$

We begin with the simplest case of $d = 1$. In this case, it is instructive to recall our goal. We want to define a matrix M such that given any $\mathbf{x}$ with $wt(\mathbf{x}) \leq 1$, we should be able to compute $\mathbf{x}$ from $M \odot \mathbf{x}$. In particular, let us consider the case when $\mathbf{x} = \mathbf{e}_i$ for some $i \in [N]$. Note that in this case $M \odot \mathbf{x} = M^i$, where M^i is the i th column of M . Hence we should design M such that M^i uniquely defined i . We have already encountered a similar situation before in Section 2.5 when trying to decode the Hamming code. It turns out that it suffices to pick M as the parity check matrix of a Hamming code. In particular, we can prove the following result:

Proposition 22.3.1. $t(1, N) \leq \lceil \log(N+1) \rceil + 1$

Proof. We prove the upper bound by exhibiting a matrix that can handle non adaptive group testing for $d = 1$. The group test matrix M is the parity check matrix for $[2^m - 1, 2^m - m - 1, 3]_2$, i.e. H_m where the i -th column is the binary representation of i (recall Section 2.4). This works because when performing $H_m \odot \mathbf{x} = \mathbf{r}$, if $wt(\mathbf{v}) \leq 1$ then $\mathbf{r}$ will correspond to the binary representation of i . Further, note that if $wt(\mathbf{x}) = 0$, then $\mathbf{r} = \mathbf{0}$, which is exactly $\mathbf{x}$. Hence, $M \odot \mathbf{x}$ uniquely identifies $\mathbf{x}$ when $wt(\mathbf{x}) \leq 1$, as desired.

If $N \neq 2^m - 1$ for any m , the matrix H_m corresponding to the m such that $2^{m-1} - 1 < N < 2^m - 1$ can be used by adding 0s to the end of $\mathbf{x}$. By doing this, decoding is "trivial" for both cases since the binary representation is given for the location. So the number of tests is at most $\lceil \log(N+1) \rceil + 1$, which completes the proof. $\square$

Note that Propositions 22.1.3 and 22.2.1 imply that $t(d, N) \geq \log N$, which with the above result implies that $t(1, N) = \Theta(\log N)$. This answers Question 22.1.1 for the case of $d = 1$. We will see later that such a tight answer is not known for larger d . However, at the very least we should try and extend Proposition 22.3.1 for larger values of d . In particular,

Question 22.3.1. Prove asymptotic upper bounds on $t(d, N)$ that hold for every $1 < d \leq N$.

We would like to point out something that was implicitly used in the proof of Proposition 22.3.1. In particular, we used the implicitly understanding that a non-adaptive group testing matrix M should have the property that given any $\mathbf{x} \in \{0, 1\}^N$ such that $wt(\mathbf{x}) \leq d$, the result vector $M \odot \mathbf{x}$ should uniquely determine $\mathbf{x}$. This notion is formally captured in the following definition of non-adaptive group testing matrix:

Definition 22.3.2. A $t \times N$ matrix M is d -separable if and only if for every $S_1 \neq S_2 \subseteq [N]$ such that $|S_1|, |S_2| \leq d$, we have

$$\bigcup_{j \in S_1} M^j \neq \bigcup_{i \in S_2} M^i. \quad (22.1)$$

In (22.1), we think of a columns $M^i \in \{0, 1\}^t$ as a subset of $[t]$ and for the rest of this chapter we will use both views of the columns of a group testing matrix. Finally, note that the above definition is indeed equivalent to our earlier informal definition since for any $\mathbf{x} \in \{0, 1\}^N$ with $wt(\mathbf{x}) \leq d$, the vector $M \odot \mathbf{x}$ when thought of as its equivalent subset of $[t]$ is exactly the set $\bigcup_{i \in S} M^i$, where S is the support of $\mathbf{x}$, i.e. $S = \{i | x_i = 1\}$.

Like in the coding setting, where we cared about the run time of the decoding algorithm, we also care about the time complexity of the decoding problem (given $M \odot \mathbf{x}$ compute $\mathbf{x}$) for group testing. We will now look at the obvious decoding algorithm for d -separable matrices: just check all possible sets that could form the support of $\mathbf{x}$. Algorithm 22.3.1 has the details.

The correctness of Algorithm 22.3.1 follows from Definition 22.3.2. Further, it is easy to check that this algorithm will run in $N^{\Theta(d)}$ time, which is not efficient for even moderately large d . This naturally leads to the following question:

Question 22.3.2. Do there exist d -separable matrices that can be efficiently decoded?

We would like to remark here that the matrices that we seek in the answer to Question 22.3.2 should have small number of tests (as otherwise the identity matrix answers the question in the affirmative).

Algorithm 22.3.1 Decoder for Separable Matrices

INPUT: Result vector $\mathbf{r}$ and d -separable matrix M OUTPUT: $\mathbf{x}$ if $\mathbf{r} = M \odot \mathbf{x}$ else Fail

```

1:  $R \leftarrow \{i | r_i = 1\}$ .
2: FOR Every  $T \subseteq [N]$  such that  $|T| \leq d$  DO
3:    $S_T \leftarrow \cup_{i \in T} M^i$ 
4:   IF  $R = S_T$  THEN
5:      $\mathbf{x} \leftarrow (x_1, \dots, x_N) \in \{0, 1\}^N$  such that  $x_i = 1$  if and only if  $i \in T$ .
6:     RETURN  $\mathbf{x}$ 
7: RETURN Fail

```

22.3.1 Disjunct Matrices

We now define a stronger notion of group testing matrices that have a more efficient decoding algorithm than d -separable matrices.

Definition 22.3.3. A $t \times N$ matrix M is d -disjunct if and only if for every $S \subset [N]$ with $|S| \leq d$ and for every $j \notin S$, there exist an $i \in [t]$ such that $M_{ij} = 1$ but for all $k \in S$, $M_{ik} = 0$. Or equivalently

$$M^j \not\subseteq \bigcup_{k \in S} M^k.$$

For an illustration of the definition, see Figure 22.3.3.

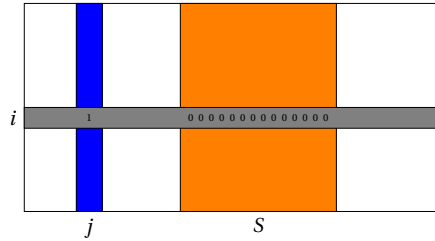


Figure 22.1: Pick a subset S (not necessarily contiguous). Then pick a column j that is not present in S . There will always be i such that row i has a 1 in column j and all zeros in S .

Next we argue that disjunctness is a sufficient condition for separability.

Lemma 22.3.4. Any d -disjunct matrix is also d -separable.

Proof. For contradiction, assume M is d -disjunct but not d -separable. Since M is not d -separable, then union of two subset $S \neq T \subset [N]$ of size at most d each are the same; i.e.

$$\bigcup_{k \in S} M^k = \bigcup_{k \in T} M^k.$$

Since $S \neq T$, there exists $j \in T \setminus S$. But we have

$$M^j \subseteq \bigcup_{k \in T} M^k = \bigcup_{k \in S} M^k,$$

where the last equality follows from the previous equality. However, since by definition $j \notin S$, the above contradicts the fact that M is d -disjunct. $\square$

In fact, it turns out that disjunctness is also almost a necessary condition: see Exercise 22.2.

Next, we show the real gain of moving to the notion of disjunctness from the notion of separability.

Lemma 22.3.5. There exists a $O(tN)$ time decoding algorithm for any $t \times N$ matrix that is d -disjunct.

Proof. The proof follows from the following two observations.

First, say we have a matrix M and a vector $\mathbf{x}$ and $\mathbf{r} = M \odot \mathbf{x}$ such that $r_i = 1$. Then there exists a column j in matrix that made it possible i.e. if $r_i = 1$, then there exists a j such that $M_{ij} = 1$ and $x_j = 1$.

Second, let T be a subset and j be a column not in T where $T = \{\ell \mid x_\ell = 1\}$ and $|T| \leq d$. Consider the i th row such that T has all zeros in the i th row, then $r_i = 0$. Conversely, if $r_i = 0$, then for every $j \in [N]$ such that $M_{ij} = 1$, it has to be the case that $x_j = 0$. This naturally leads to the decoding algorithm in Algorithm 22.3.2.

The correctness of Algorithm 22.3.2 follows from the above observation and it can be checked that the algorithm runs in time $O(tN)$ —see Exercise 22.3. $\square$

Modulo the task of exhibiting the existence of d -disjunct matrices, Lemmas 22.3.5 and 22.3.4 answer Question 22.3.2 in the affirmative. Next, we will tackle the following question:

Question 22.3.3. Design d -disjunct matrices with few rows.

As we will see shortly answering the above question will make connection to coding theory becomes even more explicit.

Algorithm 22.3.2 Naive Decoder for Disjunct Matrices

INPUT: Result vector $\mathbf{r}$ and d -disjunct matrix M OUTPUT: $\mathbf{x}$ if $M \odot \mathbf{x} = \mathbf{r}$ else Fail

```

1: Initialize  $\mathbf{x} \in \{0, 1\}^N$  to be the all ones vector
2: FOR every  $i \in [t]$  DO
3:   IF  $r_i = 0$  THEN
4:     FOR Every  $j \in [N]$  DO
5:       IF  $M_{ij} = 1$  THEN
6:          $x_j \leftarrow 0$ 
7: IF  $M \odot \mathbf{x} = \mathbf{r}$  THEN
8:   RETURN  $\mathbf{x}$ 
9: ELSE
10:  RETURN Fail

```

22.4 Coding Theory and Disjunct Matrices

In this section, we present the connection between coding theory and disjunct matrices with the final goal of answering Question 22.3.3. First, we present a sufficient condition for a matrix to be d -disjunct.

Lemma 22.4.1. *Let $1 \leq d \leq N$ be an integer and M be a $t \times N$ matrix, such that*

- (i) *For every $j \in [N]$, the j th column has at least $w_{\min}$ ones in it, i.e. $|M^j| \geq w_{\min}$ and*
- (ii) *For every $i \neq j \in [N]$, the i and j th columns have at most $a_{\max}$ ones in common, i.e. $|M^i \cap M^j| \leq a_{\max}$*

for some integers $a_{\max} \leq w_{\min} \leq t$. Then M is a $\left\lfloor \frac{w_{\min}-1}{a_{\max}} \right\rfloor$ -disjunct.

Proof. For notational convenience, define $d = \left\lfloor \frac{w_{\min}-1}{a_{\max}} \right\rfloor$. Fix an arbitrary $S \subset [N]$ such that $|S| \leq d$ and a $j \notin S$. Note we have to show that

$$M^j \not\subseteq \cup_{i \in S} M^i,$$

or equivalently

$$M^j \not\subseteq \cup_{i \in S} (M^i \cap M^j).$$

We will prove the above by showing that

$$|M^j \setminus \cup_{i \in S} (M^i \cap M^j)| > 0.$$

Indeed, consider the following sequence of relationships:

$$\begin{aligned} |M^j \setminus \cup_{i \in S} (M^i \cap M^j)| &= |M^j| - |\cup_{i \in S} (M^i \cap M^j)| \\ &\geq |M^j| - \sum_{i \in S} |M^i \cap M^j| \end{aligned} \quad (22.2)$$

$$\geq w_{\min} - |S| \cdot a_{\max} \quad (22.3)$$

$$\geq w_{\min} - d \cdot a_{\max} \quad (22.4)$$

$$\begin{aligned} &\geq w_{\min} - \frac{w_{\min}-1}{a_{\max}} \cdot a_{\max} \\ &= 1. \end{aligned} \quad (22.5)$$

In the above, (22.2) follows from the fact that size of the union of sets is at most the sum of their sizes. (22.3) follows from the definitions of $w_{\min}$ and $a_{\max}$. (22.4) follows from the fact that $|S| \leq d$ while (22.5) follows from the definition of d . The proof is complete. $\square$

Next, we present a simple way to convert a code into a matrix. Let $C \subseteq [q]^t$ be a code such that $C = \{\mathbf{c}_1, \dots, \mathbf{c}_N\}$. Then consider the matrix M_C whose i 'th column is $\mathbf{c}_i$, i.e.

$$M_C = \begin{bmatrix} \uparrow & \uparrow & \cdots & \uparrow \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ \downarrow & \downarrow & \cdots & \downarrow \end{bmatrix}.$$

Thus, by Lemma 22.4.1, to construct an $\left\lfloor \frac{w_{\min}-1}{a_{\max}} \right\rfloor$ -disjunct matrix, it is enough to design a binary code $C^* \subseteq \{0, 1\}^t$ such that (i) for every $\mathbf{c} \in C^*$, $w(\mathbf{c}) \geq w_{\min}$ and (ii) for every $\mathbf{c}^1 \neq \mathbf{c}^2 \in C^*$, we have $|\{i | c_i^1 = c_i^2 = 1\}| \leq a_{\max}$. Next, we look at the construction of such a code.

22.4.1 Kautz-Singleton Construction

In this section, we will prove the following result:

Theorem 22.4.2. *For every integer $d \geq 1$ and large enough $N \geq d$, there exists a $t \times N$ matrix is d -disjunct where $t = O(d^2 (\log_d N)^2)$.*

Note that the above result answers Question 22.3.3. It turns out that one can do a bit better: see Exercise 22.4.

Towards this end, we will now study a construction of C^* as in the previous section due to Kautz and Singleton. As we have already seen in Chapter 10, concatenated codes are a way to design binary codes. For our construction of C^* , we will also use code concatenation. In particular, we will pick $C^* = C_{\text{out}} \circ C_{\text{in}}$, where C_{out} is a $[q, k, q - k + 1]_q$ Reed-Solomon code (see Chapter 5) while the inner code $C_{\text{in}} : \mathbb{F}_q \rightarrow \{0, 1\}^q$ is defined as follows. For any $i \in \mathbb{F}_q$, define $C_{\text{in}}(i) = \mathbf{e}_i$. Note that $M_{C_{\text{in}}}$ is the identity matrix and that $N = q^k$ and $t = q^2$.

Example 22.4.3. Let $k = 1$ and $q = 3$. Note that by our choice of $[3, 1]_3$ Reed-Solomon codes, we have $C_{\text{out}} = \{(0, 0, 0), (1, 1, 1), (2, 2, 2)\}$. In other words,

$$M_{C_{\text{out}}} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix}.$$

Then the construction of M_{C^*} can be visualized as in Figure 22.4.3.

$$\begin{array}{c} \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix} \\ M_{C_{\text{out}}} \end{array} \circ \begin{array}{c} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \\ M_{C_{\text{in}}} \end{array} \rightarrow \begin{array}{c} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \\ M_{C^*} \end{array}$$

Figure 22.2: Construction of the final matrix M_{C^*} from $M_{C_{\text{out}}}$ and $M_{C_{\text{in}}}$ from Example 22.4.3. The rows in M_{C^*} that correspond to the same row in $M_{C_{\text{out}}}$ have the same color.

Next, we instantiate parameters in the Kautz-Singleton construction to prove Theorem 22.4.2.

Proof of Theorem 22.4.2. We first analyze the construction to determine the parameters $w_{\min}$ and $a_{\max}$. Then we pick the parameters q and k in terms of d to complete the proof.

Recall that $N = q^k$ and $t = q^2$. It is easy to check that every column of M_{C^*} has exactly q ones in it. In other words, $w_{\min} = q$. Next, we estimate $a_{\max}$.

Divide the rows into q sized chunks, and index the $t = q^2$ rows by pairs in $[q] \times [q]$. Recall that each column in M_{C^*} corresponds to a codeword in C^* . For notational convenience, we will use M for M_{C^*} . Note that for any row $(i, j) \in [q] \times [q]$ and a column index $\ell \in [N]$, we have $M_{(i,j),\ell} = 1$ if and only if $\mathbf{c}_\ell(j) = i$ (where we use some fixed bijection between $\mathbb{F}_q$ and $[q]$ and $\mathbf{c}_\ell$ is the ℓ 'th codeword in C_{out}). In other words, the number of rows where the ℓ_1 th and ℓ_2 th columns both have a one is exactly the number of positions where $\mathbf{c}_{\ell_1}$ and $\mathbf{c}_{\ell_2}$ agree, which is exactly $q - \Delta(\mathbf{c}_{\ell_1}, \mathbf{c}_{\ell_2})$. Since C_{out} is a $[q, k, q - k + 1]_q$ code, the number of rows where any two columns agree is at most $k - 1$. In other words, $a_{\max} = k - 1$.¹

Lemma 22.4.1 implies that M_{C^*} is d -disjunct if we pick

$$d = \left\lfloor \frac{q-1}{k-1} \right\rfloor.$$

¹The equality is obtained due to columns that corresponds to codewords that agree in exactly $k - 1$ positions.

Thus, we pick q and k such that the above is satisfied. Note that we have $q = O(kd)$. Further, since we have $N = q^k$, we have

$$k = \log_q N.$$

This implies that $q = O(d \cdot \log_q N)$, or $q \log q = O(d \log N)$. In other words we have

$$q = O(d \log_d N).$$

Recalling that $t = q^2$ completes the proof. $\square$

An inspection of the proof of Theorem 22.4.2 shows that we only used the distance of the Reed-Solomon code and in particular, any C_{out} with large enough distance suffices. In particular, if we pick C_{out} to be a random code over an appropriate sized alphabet then one can obtain $t = O(d^2 \log N)$. (See Exercise 22.5 for more on this.) Note that this bound is incomparable to the bound in Theorem 22.4.2. It turns out that these two are the best known upper bounds on $t(d, N)$. In particular,

Open Question 22.4.1. Can we beat the upper bound of $O(d^2 \cdot \min(\log(N/d), \log_d^2 N))$ on $t(d, N)$?

It turns out that the quadratic dependence on d in the upper bounds is tight. In fact it is known that $t(d, N) \geq \Omega(d^2 \log_d N)$. (See Exercises 22.7 and 22.8.)

Next, we present an application of group testing in the field of data stream algorithms, which in turn will bring out another facet of the connection between coding theory and group testing.

22.5 An Application in Data Stream Algorithms

Let us consider the problem of tracking updates on stock trades. Given a set of trades $(i_1, u_1), \dots, (i_m, u_m)$, where i_j is the stock id for the j^{th} trade, u_j is the amount of the stocks in the j^{th} trade. The problem is to keep track of the top d stocks. Such a problem is also called hot items/ heavy hitters problem.

Let N be the total number of stocks in the market. This problem could be solved in $O(m) + O(N \log N) \approx O(m)$ time and $O(N)$ space by setting a $O(N)$ size buffer to record the total number of trading for each stock and then sort the buffer later. However, m could be of the order of millions for one minute's trading, e.g. in the first minute of April 19, 2010, there are 8077600 stocks were traded. Taking the huge amount of trades into consideration, such an algorithm is not practical.

A more practical model of efficient algorithms in this context is one of *data stream algorithm*, which is defined as follows.

Definition 22.5.1. A data stream algorithm has four requirements listed below:

1. The algorithm should make one sequential pass over the input.

2. The algorithm should use poly-log space. (In particular, it cannot store the entire input.)
3. The algorithm should have poly-log update time, i.e. whenever a new item appears, the algorithm should be able to update its internal state in poly-log time.
4. The algorithm should have poly-log reporting time, i.e. at any point of time the algorithm should be able to return the required answers in poly-log time.

Thus, ideally we would like to design a data stream algorithm to solve the hot items problem that we discussed earlier. Next, we formally define the hot items problem. We begin with the definition of frequencies of each stock:

Definition 22.5.2. Let f_ℓ denote the total count for the stock id ℓ . Initially $f_\ell = 0$, given (ℓ, u_ℓ) , $f_\ell \leftarrow f_\ell + u_\ell$.

Next, we define the hot items problem.

Definition 22.5.3 (Hot Items Problem). Given N different items, for m input pairs of data (i_ℓ, u_ℓ) for $1 \leq \ell \leq m$, where $i_\ell \in [N]$ indicates the item index and u_ℓ indicates corresponding count. The problem requires updating the count f_ℓ ($1 \leq \ell \leq m$) for each item, and to output all item indices j such that $f_j > \frac{\sum_{\ell=1}^m u_\ell}{d}$. (Any such item is called a hot item.)

Note that there can be at most d hot items. In this chapter, we will mostly think of d as $O(\log N)$. Hot items problem is also called heavy hitters problems. We state the result below without proof:

Theorem 22.5.4. Computing hot items exactly by a deterministic one pass algorithm needs $\Omega(n)$ space (even with exponential time).

This theorem means that we cannot solve the hot items problem in poly-log space as we want. However, we could try to find solutions for problems around this. The first one is to output an approximate solution, which will output a set that contains all hot items and some non-hot items. For this solution, we want to make sure that the size of the output set is not too large (e.g. outputting $[N]$ is not a sensible solution).

Another solution is to make some assumptions on the input. For example, we can assume Zipf-like distribution of the input data, which means only a few items appear frequently. More specifically, we can assume *heavy-tail distribution* on the input data, i.e.:

$$\sum_{\ell: \text{not hot}} f_\ell \leq \frac{m}{d}. \quad (22.6)$$

This is reasonable for many applications, such as hot stock finding, where only a few of them have large frequency. Next, we will explore the connection between group testing and hot items problem based on this assumption.

22.5.1 Connection to Group Testing

Let us recall the naive solution that does not lead to a data stream algorithm: for each item $j \in [N]$, we maintain the actual count of number of trades for stock j . In other words, at any point of time, if C_j is the count for stock j , we have $C_j = f_j$. Another way to represent this is if M is the $N \times N$ identity matrix, then we maintain the vector of counts via $M \cdot \mathbf{f}$, where $\mathbf{f}$ is the vector of the frequencies of the items. Paralleling the story in group testing where we replace the identity matrix with a matrix with fewer rows, a natural idea here would be to replace M by matrix with fewer rows that utilizes the fact that there can be at most d hot items. Next, we show that this idea works if the heavy tail distribution holds. In particular, we will reduce the hot items problem to the group testing problem.

We now show how we solve the hot items problem from Definition 22.5.3. Let M be an $t \times N$ matrix that is d -disjunct. We maintain counters $C_1, \dots, C_t$, where each C_i is the total count of any item that is present in the i th row. We also maintain the total number of items m seen so far. Algorithm 22.5.1 and Algorithm 22.5.2 present the initialization and update algorithms. The reporting algorithm then needs to solve the following problem: at any point of time, given the counts $C_1, \dots, C_t$ and m output the at most d hot items.

Algorithm 22.5.1 Initialization

OUTPUT: Initialize the counters

```

1:  $m \leftarrow 0$ 
2: FOR every  $j \in [t]$  DO
3:    $C_j \leftarrow 0$ 

```

Algorithm 22.5.2 Update

INPUT: Input pair (i, u) , $i \in [N]$ and $u \in \mathbb{Z}$

OUTPUT: Update the Counters

```

1:  $m \leftarrow m + 1$ ,
2: FOR every  $j \in [t]$  DO
3:   IF  $M_{ij} = 1$  THEN
4:      $C_j \leftarrow C_j + u$ 

```

Next, we reduce the problem of reporting hot items to the decoding problem of group testing. The reduction essentially follows from the following observations.

Observation 22.5.5. If j is a hot item and $M_{ij} = 1$, then $C_i > \frac{m}{d}$.

Proof. Let $i \in [t]$ be such that $M_{ij} = 1$. Then note that at any point of time,

$$C_i = \sum_{k: M_{ik}=1} f_k \geq f_j.^2$$

Since j is a hot item, we have $f_j > \frac{m}{d}$, which completes the proof. $\square$

Observation 22.5.6. For any $1 \leq i \leq t$, if all j with $M_{ij} = 1$ are not hot items, then we have $C_i \leq \frac{m}{d}$.

Proof. Fix an $i \in [t]$ such that every $j \in [N]$ such that $M_{ij} = 1$ is not a hot item. Then by the same argument as in proof of Observation 22.5.5, we have

$$C_i = \sum_{k: M_{ik}=1} f_k.$$

The proof then follows by the choice of i and (22.6). $\square$

Armed with these observations, we now present the reduction. Define $\mathbf{x} = (x_1, x_2, \dots, x_N) \in \{0, 1\}^N$ with $x_j = 1$ if and only if j is a hot item, and $\mathbf{r} = (r_1, r_2, \dots, r_t) \in \{0, 1\}^t$ with $r_i = 1$ if and only if $C_i > \frac{m}{d}$, we will have $r_i = \vee_{j: M_{ij}=1} x_j$. The latter claim follows from Observations 22.5.5 and 22.5.6 above. This means we have:

$$M \odot \mathbf{x} = \mathbf{r}. \quad (22.7)$$

Note that by definition, $wt(\mathbf{x}) < d$. Thus reporting the hot items is the same as decoding to compute $\mathbf{x}$ given M and $\mathbf{r}$, which successfully changes the hot items problem into group testing problem. Algorithm 22.5.3 has the formal specification of this algorithm.

Algorithm 22.5.3 Report Heavy Items

INPUT: Counters m and $C_1, \dots, C_t$

OUTPUT: Output the heavy items

```

1: FOR every  $j \in [t]$  DO
2:   IF  $C_t > \frac{m}{d}$  THEN
3:      $r_j \leftarrow 1$ 
4:   ELSE
5:      $r_j \leftarrow 0$ 
6: Let  $\mathbf{x}$  be the result of decoding (for group testing)  $\mathbf{r}$ 
7: RETURN  $\{i | x_i = 1\}$ 

```

Next, we will design and analyze the algorithm above and check if the conditions in Definition 22.5.1 are met.

²The equality follows e.g. by applying induction on Algorithm 22.5.2.

Analysis of the Algorithm

In this part, we will review the requirements on data stream algorithm one by one and check if the algorithm for the hot items problem based on group testing satisfies them. In particular, we will need to pick M and the decoding algorithm. We will pick M to be the d -disjunct matrix from Theorem 22.4.2.

1. One-pass requirement

If we use non-adaptive group testing, the algorithm for the hot items problem above can be implemented in one pass, which means each input is visited only once. (If adaptive group testing is used, the algorithm is no longer one pass, therefore we choose non-adaptive group testing.) We note that by definition, our choice of M satisfies this condition.

2. Poly-log space requirement

In the algorithm, we have to maintain the counters C_i and m . The maximum value for them is mN , thus we can represent each counter in $O(\log N + \log m)$ bits. This means we need $O((\log N + \log m)t)$ bits to maintain the counters. Theorem 22.4.2 implies that $t = O(d^2 \log_d^2 N)$. Thus, the total space we need to maintain the counters is $O(d^2 \log_d^2 N (\log N + \log m))$.

On the other hand, if we need to store the matrix M , we will need $\Omega(tN)$ space. Therefore, poly-log space requirement can be achieved only if matrix M is not stored directly. (We will tackle this issues in the next point.)

3. Poly-log update time

As mentioned in the previous part, we cannot store the matrix M directly in order to have poly-log space. Since RS code is strongly explicit (see Exercise 6.9), we do not need to explicitly store M (we just need to store the parameters of the code C_{out} and C_{in} , which can be done in poly-log space). In the following, we will argue that the runtime of Algorithm 22.5.2 is $O(t \times \text{polylog } t)$. It is easy to check the claimed time bound is correct as long as we can perform the check in Step 3 in $\text{polylog}(t)$ time. In particular, we would be done if given $j \in [N]$, we can compute the column M^j in $O(t \times \text{polylog } t)$ time. Next, we argue that the latter claim is correct.

Recall that $M = M_{C^*}$, with $C^* = C_{\text{out}} \circ C_{\text{in}}$, where C_{out} is a $[q, k, q - k + 1]_q$ RS code and C_{in} chosen such that $M_{C_{\text{in}}}$ is the $q \times q$ identity matrix. Recall that codewords of C^* are columns of the matrix M , and we have $n = q^k$, $t = q^2$.

Since every column of M corresponds to a codeword of C^* , we can think of j equivalently as a message $\mathbf{m} \in \mathbb{F}_q^k$. In particular, M^j then corresponds to the codeword $C_{\text{out}}(\mathbf{m})$. On the other hand, the column M^j can be partitioned into q chunks, each chunk is of length q . Notice that $(C_{\text{out}}(\mathbf{m}))_{i_1} = i_2$ if and only if the i_1 th chunk has 1 on its i_2 th position and 0 on other positions (recall the definition of C_{in}). Therefore, we can compute M^j by computing $C_{\text{out}}(\mathbf{m})$. Because C_{out} is a linear code, $C_{\text{out}}(\mathbf{m})$ can be computed in

$O(q^2 \times \text{polylog } q)$ time,³ implies that M^j can be computed in $O(q^2 \times \text{polylog } q)$ time. Since we have $t = q^2$, the update process can be finished with $O(t \times \text{polylog } t)$ time. (We do not need C_{out} to be strongly explicit: as long as C_{out} is linear the arguments so far work just as well.)

4. Reporting time

It is easy to check that the run time of Algorithm 22.5.3 is dominated by Step 6. So far, the only decoding algorithm for M that we have seen is Algorithm 22.3.2, which runs in time $\Omega(tN)$, which does not satisfy the required reporting time requirement. In Exercise 22.11, we show that using the fact that C_{out} is the Reed-Solomon code, one can solve the decoding problem in $\text{poly}(t)$.

Thus, we have argued that

Theorem 22.5.7. *There exists a data streaming algorithm that computes d hot items with one pass, $O(t \log N)$ space for $t = O(d^2 \log_d^2 N)$, $O(t \text{polylog } t)$ update time and $\text{poly}(t)$ reporting time.*

22.6 Summary of best known bounds

We conclude the chapter by collecting the best known bounds on both adaptive and non-adaptive group testing. First, we know the correct bound on the best possible number of adaptive tests:

Theorem 22.6.1.

$$t^a(d, N) = \Theta(d \log(N/d)).$$

The upper bound follows from Exercise 22.1 while the lower bound follows from Proposition 22.2.1.

There is a gap between the best known upper and lower bound on the number of non-adaptive tests:

Theorem 22.6.2.

$$\Omega(d^2 \log_d N) \leq t(d, N) \leq O(d^2 \min(\log(N/d), \log_d^2 N)).$$

The upper bounds follow from Theorem 22.4.2 and Exercise 22.5 while the lower bound follows from Exercise 22.8.

Finally, note that Theorem 22.6.1 and 22.6.2 imply that there is a gap between the minimum number of tests needed for adaptive and non-adaptive group testing:

Corollary 22.6.3.

$$\frac{t(d, N)}{t^a(d, N)} \geq \Omega\left(\frac{d}{\log d}\right).$$

³This follows from Proposition 2.3.4 and the fact that C_{out} is strongly explicit

22.7 Exercises

Exercise 22.1 (Upper bound on $t^a(d, N)$). *In this problem we will show that $t^a(d, N) = O(d \log(N/d))$. We begin by trying to prove a weaker bound of $O(d \log N)$:*

- *Show that one can identify at least one i such that $x_i = 1$ (or report none exist) with $O(\log N)$ adaptive tests.*
(Hint: Use binary search.)
- *Using the scheme above argue that one can compute $\mathbf{x}$ with $O(wt(\mathbf{x}) \cdot \log N)$ adaptive tests. Conclude that $t^a(d, N) \leq O(d \log N)$.*

Next we argue that we can tighten the bound to the optimal bound of $O(d \log(N/d))$:

- *Argue that any scheme that computes $\mathbf{x} \in \{0, 1\}^N$ with $O(wt(\mathbf{x}) \cdot \log N)$ adaptive tests can be used to compute $\mathbf{x}$ with $O(d \log(N/d))$ adaptive tests where $wt(\mathbf{x}) \leq d$.*
- *Conclude that $t^a(d, N) \leq O(d \log(N/d))$.*

Exercise 22.2. *Show that every d -separable matrix is also $(d-1)$ -disjunct.*

Exercise 22.3. *Prove that Algorithm 22.3.2 is correct and runs in time $O(tN)$.*

Exercise 22.4. *For every integer $d \geq 1$ and large enough integer $N \geq d$ show that there exists a d -disjunct matrix with $O(d^2 \log(N/d))$ rows.*

(Hint: Use the probabilistic method. It might help to pick each of tN bits in the matrix independently at random with the same probability.)

Exercise 22.5. *We first begin by generalizing the argument of Theorem 22.4.2:*

- *Let C_{out} be an $(n, k, D)_q$ code. Let C_{in} be defined such that $M_{C_{\text{in}}}$ is the $q \times q$ identity matrix. Let $M_{C_{\text{out}} \circ C_{\text{in}}}$ be a $t \times N$ matrix that is d -disjunct. Derive the parameters d, t and N .*

Next argue that it is enough to pick an outer random code to obtain a d -disjunct matrix with the same parameters obtained in Exercise 22.4:

- *Pick $q = \Theta(d)$. Then using the previous part or otherwise show that if C_{out} is a random $[n, k, D]_q$ code, then the resulting $t \times N$ matrix $M_{C_{\text{out}} \circ C_{\text{in}}}$ is d -disjunct with $t = O(d^2 \log N)$ for large enough N .*

(Hint: Use Theorem 4.2.1 and Proposition 3.3.7.)

Exercise 22.6. *For every integer $d \geq 1$ and large enough $N \geq d$, construct a d -disjunct matrix with $O(d^2 \log N)$ rows in (deterministic) time $\text{poly}(N)$.*

Hint: Recall Exercise 4.8.

Exercise 22.7 (Lower Bound on $t(d, N)$ due to Bassalygo). In this problem we will show that $t(d, N) \geq \min \left\{ \binom{d+2}{2}, N \right\}$. In what follows let M be a $t \times N$ matrix that is d -disjunct.

- Argue that if $wt(M^j) < d$ then M^j has a private row i.e. there exists a row $i \in [t]$ such that $M_{ij} = 1$ but $M_{ij'} = 0$ for every $j' \neq j$.
- Using part (a) or otherwise, argue that if all columns of M have Hamming weight at most $d-1$, then $t \geq N$.
- Let M^{-j} for $j \in [N]$ be the matrix M with M^j as well as all rows $i \in [t]$ such that $M_{ij} = 1$ removed. Then argue that M^{-j} is $(d-1)$ -disjunct.
- Argue that $t(1, N) \geq \min \{3, N\}$.
- Using induction with parts (b)-(d) or otherwise, argue that $t \geq \min \left\{ \binom{d+2}{2}, N \right\}$.

Exercise 22.8 (Lower Bound on $t(d, N)$ due to Ruzsinkó and Alon-Asodi). In this problem, we will show that

$$t(d, N) \geq \Omega \left(\min \{d^2 \log_d N, N\} \right). \quad (22.8)$$

In what follows let M be a $t \times N$ matrix that is d -disjunct.

- Let M' be the sub-matrix obtained from M as follows. For any $j \in [N]$ such that $wt(M^j) \geq \frac{2t}{d}$, remove the j th column as well as any rows that has a one in that column. Repeat this process until no such columns exist and let the resulting $t' \times N'$ matrix be denoted by M' . Prove that

$$N' \geq N - \frac{d}{2}.$$

- Argue that any $j \in [N']$ the j th column in M' has a private subset of size at most $\lceil 4t/d^2 \rceil$, i.e. there exists a subset $S \subseteq [t']$ with $|S| \leq \lceil 4t/d^2 \rceil$ such that M'^j has ones in all $i \in S$ but for every $j' \neq j$, $M'^{j'}$ has at least one row $i' \in S$ such that $M'_{i'j'} = 0$.
- Using part 2 or otherwise, prove that for $d \geq 3$ (and $t > 18$):

$$N - \frac{d}{2} \leq \binom{t}{\lceil 4t/d^2 \rceil}.$$

- Using Exercise 22.7 and part 3 or otherwise argue (22.8).

Exercise 22.9. In this exercise and the ones that follow it, we will consider the following equivalent version of the decoding problem: given $\mathbf{r} = M \odot \mathbf{x}$ with $wt(\mathbf{x}) \leq d$, output $\{i | x_i = 1\}$. Now consider the following easier version of the problem. In addition to $\mathbf{r}$ and M assume that one is also given a set S such that $\{i | x_i = 1\} \subseteq S$. Modify Algorithm 22.3.2 to design a decoding algorithm that computes $\{i | x_i = 1\}$ in time $O(t \cdot |S|)$.

Exercise 22.10. A $t \times N$ matrix M is called (d, L) -list disjunct if and only if the following holds for every disjoint subsets $S, T \subseteq [N]$ such that $|S| = d$ and $|T| = L - d$, there is a row in M where all columns in S have a 0 but at least one column in T has a 1.

- What is a $(d, d+1)$ -list disjunct matrix?
- Let C_{out} be an $(n, k)_q$ code that is $(0, d, L)$ -list recoverable code (recall Definition 12.3.3). Let C_{in} be the inner code such that $M_{C_{\text{in}}}$ is the $q \times q$ identity matrix. Argue that $M_{C_{\text{out}} \circ C_{\text{in}}}$ is (d, L) list disjunct.

Exercise 22.11. Using Exercises 22.9 and 22.10 or otherwise prove the following. Let M_{C^*} be the Kautz-Singleton matrix from Section 22.4.1. Then given $M_{C^*} \odot \mathbf{x}$ with $wt(\mathbf{x}) \leq d$, one can compute $\{i | x_i = 1\}$ in $\text{poly}(t)$ time. (Hint: Theorem 12.3.4 could be useful.)

22.8 Bibliographic Notes

Robert Dorfman's paper in 1943 [39] introduced the field of (Combinatorial) Group Testing. It must be noted that though this book covers group testing as an application of coding theory, it took off long before coding theory itself.

The original motivation arose during the Second World War when the United States Public Health service and the Selective Service embarked upon a large scale project. The objective was to weed out all syphilitic men called up for induction. [39].

The connection between group testing and hot items problem considered in Section 22.5 was established by Cormode and Muthukrishnan [34]. More details on data stream algorithms can be bound in the survey by Muthukrishnan [119].

Chapter 23

Complexity of Coding Problems

Throughout this book it should have become clear that the algorithmic complexity of some fundamental tasks play a critical role in the utility of the code. The most basic tasks are encoding, and decoding; though in the latter case we may consider many variants such as decoding up to half the minimum distance, or list-decoding (when the number of errors is more than half the minimum distance), or simply finding the nearest codeword. In most of the book thus far, we considered these tasks for specially designed codes. In this chapter we will revisit the complexity of solving some of these tasks for general (linear) codes.¹

The main goal in this chapter is to point to some algorithmic tasks that are not likely to be solvable in polynomial time. Before launching on our quest let us first eliminate the complexity of encoding from our focus. For all linear codes, once the generator matrix is known, the encoding takes at most quadratic time in the block length which is already polynomial time. Indeed for special codes this running time can be reduced even further. For example, for many algebraic codes this running time can be nearly-linear — formally $O(n \log^c n)$ for some universal constant c , where n denotes the block length. And for the codes in Chapter 18, the running time even became $O(n)$.

The main focus in this chapter is decoding, where some of the most general problems turn out to be too hard. At a high level, we will be considering the following question:

Question 23.0.1. *Given an arbitrary linear code (say via its generator matrix), how easy (or hard) is to perform (various notions of) decoding?*

We will describe some variations which remain hard. Finally we will also talk about the challenge of determining the minimum distance of a linear code, which also turns out to be hard.

¹As we saw in Chapter 2, linear codes have the advantage that they can be represented in size $O(n^2)$ for codes of block length n . For general random codes one needs to use exponential (in dimension k) space to even represent the code. It turns out that in this case all interesting algorithm operations are polynomial time in the input size due to trivial reasons—see Exercise 23.1.

The main point of this chapter is to reinforce the message that codes need to be carefully designed (and presented) to determine their distance and enable efficient decoding.

We will assume for this chapter that the reader is familiar with the notions of NP-completeness (and related notions of intractability). Appendix C has a primer on the notions of computation efficiency and intractability (and in particular, Appendix C.5 has a quick overview of the theory of NP-completeness and lists the background knowledge that we will assume for this chapter).

23.1 Nearest Codeword Problem (NCP)

The Nearest Codeword Problem is the most basic and ambitious goal for decoding of linear codes. Roughly, here the problem is to find the nearest codeword to a given word in the ambient space, given the generator matrix of the code. As in most complexity analyses, we focus on a decision problem (see Definition C.5.1) that captures the complexity of this general task.

Problem 23.1.1 (Nearest Codeword Problem (NCP)).

- **Input:** $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ where $\mathbf{G} \in \mathbb{F}^{k \times n}$, $\mathbf{v} \in \mathbb{F}^n$ and $t \in \mathbb{Z}^{\geq 0}$.
- **Output:** YES if there exists $\mathbf{x} \in \mathbb{F}^k$ such that $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$ and NO otherwise.

We note that the above is the decision problem version of the following (perhaps more natural) algorithmic problem, where is an $\mathbf{x}$ exists with $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$, then we actually output such an $\mathbf{x}$. Exercise 23.2 shows that these two problems are essentially equivalent in the sense that if there exists a polynomial time algorithm for one problem then there exists a polynomial time algorithm for the other problem (and vice-versa).

It turns out the NCP is NP-complete and hence not likely to find a polynomial time solution. We prove this below. As usual we rely on the knowledge of other NP-complete problems. The easiest to use for our goal is the NP-completeness of the MaxCut Problem.

Problem 23.1.2 (Maximum Cut Problem (MaxCut)).

- **Input:** Graph $H = (V, E)$ with vertices V and edges $E \subseteq \binom{V}{2}$; and integer ℓ .
- **Output:** YES if there exists a cut in H of size at least ℓ i.e., a set $S \subseteq V$ such that $\text{Cut}(S) \triangleq \{e \in E \mid |e \cap S| = 1\}$ satisfies $|\text{Cut}(S)| \geq \ell$ and NO otherwise.

The following is a well-known result from the theory of NP-completeness.

Theorem 23.1.3. *MaxCut is NP-complete.*

We now use the theorem above to show the NP-Completeness of NCP.

Theorem 23.1.4. *The Nearest Codeword Problem (NCP) is NP-complete.*

Proof. We first note that NCP is indeed in NP, by noting that there is a polynomial time algorithm that can *verify* Yes instances given an appropriate certificate. Specifically certificate we will use is the vector $\mathbf{x} \in \mathbb{F}^k$ and the verification algorithm simply checks that $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$ and accepts if this holds. Clearly the verification algorithm runs in polynomial time, and satisfies the condition that for every instance $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ for which the answer is YES there is a certificate (namely the $\mathbf{x} \in \mathbb{F}^k$) such that the verification algorithm accepts.

We now turn to the NP-hardness result. We show how to reduce MaxCut to NCP with $\mathbb{F} = \mathbb{F}_2$.² Given a graph H on vertex set V and edges E , let $k = |V|$ and $n = |E|$. Our matrix $\mathbf{G}$ will be the so-called *incidence matrix* of H , i.e., rows of $\mathbf{G}$ are indexed by V and columns by E and $\mathbf{G}_{u,e} = 1$ if $u \in e$ and 0 otherwise. In other words for vertex u and edge e , $\mathbf{G}_{u,e} = 1$ if and only if the edge e touches (or is ‘incident to’) the vertex u . Our target vector is $\mathbf{v} = \mathbf{1}_n$.³ Finally the parameter $t = n - \ell$. This gives us the instance $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ of NCP. We now show that $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ is a YES instance of NCP if and only if (H, ℓ) is a YES instance of MaxCut which will conclude the proof of this theorem.

Assume (for simplicity of notation) that $V = [k]$. We first note that there is a one-to-one correspondence between $\mathbf{x} \in \mathbb{F}_2^k$ and $S \subseteq V$ where $\mathbf{x}_i = 1$ if and only if $i \in S$. For this correspondence, now note that $\mathbf{xG}$ is simply the characteristic vector of $\text{Cut}(S)$, since $e \in \text{Cut}(S)$ and only if $(\mathbf{xG})_e = 1$ (see Exercise 23.5). It follows thus that $\Delta(\mathbf{xG}, \mathbf{1}_n) = n - |\text{Cut}(S)|$. We conclude that there exists $\mathbf{x} \in \mathbb{F}_2^k$ such that $\Delta(\mathbf{xG}, \mathbf{1}_n) \leq n - \ell$ if and only if there exists $S \subseteq V$ such that $|\text{Cut}(S)| \geq \ell$. In other words $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ is a YES instance of NCP if and only if (H, ℓ) is a YES instance of MaxCut, thereby establishing that NCP is NP-hard. $\square$

As pointed out earlier, NCP was an ambitious problem and a hardness result should not be too daunting. NCP asks for the exact distance to the nearest codeword, possibly in codes of small minimum distance and possibly in codes which do have nice structure, but this is not evident in the generator matrix. In what follows we will consider each of these aspects, model them formally and try to assess the complexity of the problem.

23.2 Decoding with Preprocessing

One of the sources of the complexity of the Nearest Codeword Problem might be the fact that the decoding algorithm does not have the time to ‘preprocess’ the code and understand its ‘intricacies’. For example our code may have a low-density parity check matrix, and if it is does surely we should be able to decode it efficiently. However, if the code is presented by an arbitrary generator matrix, then a low-density parity check matrix may not necessarily be algorithmically easy to find. This leads to the informal question: *Can every code be easy to decode, after some preprocessing?*

To formalize this question we fix some family of codes $\{C_n\}_n$ with generator matrices $\{\mathbf{G}_n\}_n$ and consider the nearest codeword problem for this family of codes as follows:

²Note that this is enough to prove that NCP is NP-hard since $\mathbb{F}$ is part of the input for NCP. A similar result can be proven for other fields as well— see Exercise 23.3.

³The choice of $\mathbf{1}_n$ is crucial. Not all choices will work— see e.g. Exercise 23.4.

Problem 23.2.1 (Nearest Codeword Problem with Preprocessing (NCP) $\{\mathbf{G}_n\}_n$).

- **Input:** $(\mathbf{v}, t)$ where $\mathbf{v} \in \mathbb{F}^n$ and $t \in \mathbb{Z}^{\geq 0}$
- **Output:** YES if there exists $\mathbf{x} \in \mathbb{F}^{k(n)}$ such that $\Delta(\mathbf{v}, \mathbf{x}\mathbf{G}_n) \leq t$ and NO otherwise.

The above does not fully capture the effect of preprocessing, in that it does not capture how the understanding gained by preprocessing could be provided to an algorithm aiming to solve (NCP) $\{\mathbf{G}_n\}_n$. It turns out that the right way to capture the effect of preprocessing is to allow the decoding algorithm to be *non-uniform*. I.e., we will be satisfied if there is an ‘efficient’ algorithm $A(\cdot, \cdot; \cdot)$ and a sequence of strings (often referred to as *advice* in the literature) $\{B_n\}_n$ such that $A(\mathbf{v}, t; B_n)$ correctly solves NCP($\{\mathbf{G}_n\}_n$) for every $(\mathbf{v}, t)$ with $\mathbf{v} \in \mathbb{F}^n$. Two aspects to stress here are:

1. A should be efficient, i.e., run in time polynomial in n . In particular, this implies that B_n is a string of length polynomial in n .
2. B_n itself may not be easy to compute given $\mathbf{G}_n$ and it is important that we allow the computation time of B_n to be unboundedly large (or else our definitions do not capture the intuition that our preprocessor takes a long time to study the code). What is important is that B_n be a string of length polynomial in n (both to enable A to run efficiently and to capture the intuition that our understanding can be captured succinctly).

The two definitions above, of the (NCP) $\{\mathbf{G}_n\}_n$ problem, and the non-uniform algorithmic solution concept turn out to capture the effect of preprocessing adequately. The resulting question is the following:

Question 23.2.1. *Is it possible that for every family of codes given by generators $\{\mathbf{G}_n\}_n$, the problem NCP($\{\mathbf{G}_n\}_n$) has an efficient non-uniform solution?*

Unfortunately, the answer to this question also turns out negative as we will see shortly. However, we move to this result it is worthwhile to reflect on the proof of Theorem 23.1.4 fails in this case. Go on, think about it before looking at the answer below.

The ‘issue’ with the proof of Theorem 23.1.4 is that the hardness of MaxCut for the input graph G is encoded into the generator matrix $\mathbf{G}_n$ itself. In other words, since in the non-uniform setting we are allowed to pre-process the generator matrix $\mathbf{G}_n$ —we can simply set $B_n = 1$ if the original graph G has a cut of size at least ℓ and set $B_n = 0$ otherwise. Note that this is a perfectly legitimate thing to do since we have no restriction on the computation tractability of *computing* B_n —just that it should not be too large (and in this case it is just a bit). Thus, intuitively what we want is a reduction where the hardness is ‘baked’ into the received word $\mathbf{v}$ instead of the generator matrix $\mathbf{G}_n$. Next, we show that this is possible to do by reducing MaxCut to NCP:

Theorem 23.2.2. *There exists a family of codes with generators $\{\mathbf{G}_n\}_n$ such that NCP($\{\mathbf{G}_n\}_n$) is NP-hard. Specifically there is a polynomial time reduction from MaxCut to NCP($\{\mathbf{G}_n\}_n$).*

Proof. Roughly the idea here is to build a generator matrix corresponding to the incidence matrix of complete graph on k vertices. Thus the generator matrix is always the same for all graphs on k vertices.⁴ To capture the actual edges of a graph H we will use the target vector $\mathbf{v}$. The target distance t will turn out to depend on the number of vertices and the number of edges in H , as also the size of the cut expected in H . Details below.

We start with a description of the family $\{\mathbf{G}_n\}_n$. We define $\mathbf{G}_n \in \mathbb{F}_2^{k \times n}$ for every positive integer k and for $n = k(k-1)$. Given such a pair k and n , we index the rows of $\mathbf{G}$ by the elements of $[k]$, and the columns by pairs (i, j) where $i \neq j$ and $i, j \in [k]$. Given a row index $r \in [k]$ and column index (i, j) , the entry $\mathbf{G}_n[r, (i, j)] = 1$ if $r = i$ or $r = j$ and 0 otherwise. In effect $\mathbf{G}_n$ has two columns corresponding to every *undirected* edge $\{i, j\}$ in the complete graph on k vertices—one indexed by (i, j) and the other by (j, i) . These two columns are actually identical. The reason for the two copies will become clear shortly. Specifically this implies that for every $\mathbf{x} \in \mathbb{F}_2^k$ and every $\{i, j\}$ we have that $(\mathbf{x}\mathbf{G}_n)_{(i, j)} = (\mathbf{x}\mathbf{G}_n)_{(j, i)}$ (see Exercise 23.6).

We now show how to reduce an instance (H, ℓ) of MaxCut to NCP($\{\mathbf{G}_n\}_n$). Let H be a graph on k vertices, and say the vertex set equals $[k]$ and let E denote the edges of H . We map H to a vector $\mathbf{v} \in \mathbb{F}_2^n$ as follows: If the edge $\{i, j\} \in E$ then we let $\mathbf{v}_{(i, j)} = \mathbf{v}_{(j, i)} = 1$. Else, assuming $i < j$, we set $\mathbf{v}_{(i, j)} = 0$ and $\mathbf{v}_{(j, i)} = 1$. Finally we set

$$t = \frac{n}{2} + |E| - 2\ell.$$

We now explain why this reduction works correctly.

As in the proof of Theorem 23.1.4 we use the fact that there is a correspondence between $\mathbf{x} \in \mathbb{F}_2^k$ and cuts in the graph H (using $\mathbf{x}$ as the characteristic vector of the set S). Fix a vector $\mathbf{x}$ and the corresponding cut S and let $c(S)$ denote the number of edges cut by S , i.e.,

$$c(S) = |\{i, j\} \in E | \mathbf{x}_i + \mathbf{x}_j = 1|.$$

Let $\mathbf{w} = \mathbf{x}\mathbf{G}_n$. Note that for every $i \neq j \in [k]$, Exercise 23.6 implies that $\mathbf{w}_{(i, j)} = \mathbf{w}_{(j, i)}$ (note that this is true whether $\{i, j\} \in E$ or not) and we will use this equality multiple times in the proof.

We claim that the distance $\Delta(\mathbf{w}, \mathbf{v})$ is exactly

$$\frac{n}{2} + |E| - 2c(S).$$

To see this, note that if $\{i, j\} \notin E$ then $\mathbf{w}_{(i, j)} = \mathbf{w}_{(j, i)}$ while by construction $\mathbf{v}_{(i, j)} \neq \mathbf{v}_{(j, i)}$. It follows that the contribution of the coordinates (i, j) corresponding to $\{i, j\} \notin E$ to $\Delta(\mathbf{w}, \mathbf{v})$ is exactly $\frac{n}{2} - |E|$ (i.e., each pair $\{i, j\} \notin E$ contributes 1 to the distance). Now we turn to the coordinates $\{i, j\} \in E$ —and here the analysis is exactly as in the proof of Theorem 23.1.4. If the edge $\{i, j\}$ is cut

⁴Going back to our earlier discussion on non-uniform algorithms, note that the advice that the pre-processing algorithm on $\mathbf{G}_n$ can then compute only depends on n and is independent of the MaxCut instance. This allows us to side-step the issue with the proof of Theorem 23.1.4 in the non-uniform setting.

by S , then we have $\mathbf{w}_{(i,j)} = \mathbf{w}_{(j,i)} = \mathbf{v}_{(i,j)} = \mathbf{v}_{(j,i)} = 1$ and so these coordinates do not contribute to the Hamming distance. (Note that there are $2c(S)$ such coordinates). Finally if $\{i, j\}$ is not cut by S then $\mathbf{w}_{(i,j)} = \mathbf{w}_{(j,i)} = 0$ whereas $\mathbf{v}_{(i,j)} = \mathbf{v}_{(j,i)} = 1$ and so each such coordinate contributes 1 to the Hamming distance $\Delta(\mathbf{w}, \mathbf{v})$ (and now we have $2(|E| - c(S))$ such coordinates). We conclude that

$$\Delta(\mathbf{w}, \mathbf{v}) = \frac{n}{2} + |E| - 2c(S).$$

It follows that the maximum cut will minimize the distance and so a vector at distance at most $n/2 + |E| - 2\ell$ exists if and only if a cut of size at least ℓ exists. Thus $(\mathbf{v}, t)$ is a YES instance of $\text{NCP}(\mathbf{G}_n)_n$ if and only if (H, ℓ) is a YES instance of MaxCut, thus showing that MaxCut reduces (in polynomial time) to $\text{NCP}(\mathbf{G}_n)_n$. $\square$

We stress that while the hardness result for $\text{NCP}(\mathbf{G}_n)_n$ is a straightforward NP-hardness result, the application to preprocessing only asserts that: If $\text{NCP}(\mathbf{G}_n)_n$ has an efficient non-uniform algorithm, then so does all of NP (see Exercise 23.7). This would *not* imply $\text{NP} = \text{P}$. But the conclusion that all of NP has efficient non-uniform algorithms is considered almost as unlikely (see Appendix C.5), thus suggesting that not all codes can be preprocessed effectively to yield efficient decoders.

23.3 Approximate NCP

The results of the previous sections rule out finding the *nearest* codeword to a given word $\mathbf{v}$ in a general linear code generated by $\mathbf{G}$. But what if we are willing to find some other nearby codeword? Of course if the distance t of the target vector $\mathbf{v}$ from the code generated by $\mathbf{G}$ is much smaller than the minimum distance of the code, then there are no other nearby codewords. But this is not the case in the hardness results we have proved (see Exercise 23.8). So it is conceivable that there are other codewords nearby (and not too much further than the nearest one). Could it be easier to find one such codeword? Again after appropriate formalization, we will show that the answer is negative.

To formalize this question we exploit promise problems (Definition C.5.2). The correct promise problems to capture approximations turn out to be a *gap problem*. To define this problem, we define two disjoint subsets of inputs to decoding problems. For a real number $g \geq 1$, let

$$\text{Gap}_g\text{NCP}_{\text{Yes}} = \{(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \mid \text{exists } \mathbf{x} \in \mathbb{F}_2^k \text{ s.t. } \Delta(\mathbf{v}, \mathbf{xG}) \leq t\}$$

$$\text{and } \text{Gap}_g\text{NCP}_{\text{No}} = \{(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \mid \text{for every } \mathbf{x} \in \mathbb{F}_2^k, \Delta(\mathbf{v}, \mathbf{xG}) > g \cdot t\}.$$

The GapNCP problem, defined below is simply the restriction of NCP to inputs from $\text{Gap}_g\text{NCP}_{\text{Yes}} \cup \text{Gap}_g\text{NCP}_{\text{No}}$.

Problem 23.3.1 (Gap Nearest Codeword Problem (GapNCP)) with parameter g .

• **Input:** $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{NCP}_{\text{Yes}} \cup \text{Gap}_g\text{NCP}_{\text{No}}$.

• **Output:** YES if $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{NCP}_{\text{Yes}}$ and NO otherwise.

For a real number $g > 1$, the Gap_gNCP -problem captures the complexity of finding a ' g -approximate nearby codeword' where a codeword $\mathbf{yG}$ is said to be a g -approximate nearby codeword to $\mathbf{v}$ if for every $\mathbf{x} \in \mathbb{F}^k$, we have $\Delta(\mathbf{v}, \mathbf{yG}) \leq g \cdot \Delta(\mathbf{v}, \mathbf{xG})$. Thus a 2-approximate nearby codeword is at distance at most twice the distance of the nearest codeword to $\mathbf{v}$. Exercise 23.9 shows that if Gap_gNCP is NP-hard and $\text{NP} \neq \text{P}$ then no polynomial time algorithm can find a g -approximate nearby codeword.

Having formulated the correct problem to capture 'approximately nearby codewords' it is easy to use known hardness of approximation results to show that Gap_gNCP is NP-hard for some $g > 1$. This result in turn uses the hardness of the Gap version of the MaxCut problem defined below. Let

$$\text{Gap}_g\text{CUT}_{\text{Yes}} = \{(H, \ell) \mid \text{exists a cut in } H \text{ of size at least } \ell\}$$

$$\text{and } \text{Gap}_g\text{CUT}_{\text{No}} = \{(H, \ell) \mid \text{every cut in } H \text{ has size at most } \ell/g\}.$$

We use the following theorem that captures the known hardness of Gap_gCUT .

Theorem 23.3.2. *There exists $g > 1$ such that Gap_gCUT is NP-hard.*

It turns out that our reduction from MaxCut to NCP (proof of Theorem 23.1.4) essentially preserves approximation (see Exercise 23.10) and so we get

Lemma 23.3.3. *There exists $g > 1$ such that Gap_gNCP is NP-hard. Furthermore the hardness holds even when restricted to instances $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ where $\mathbb{F} = \mathbb{F}_2$ and the target vector $\mathbf{v} = \mathbf{1}_n$.*

The more interesting result for NCP is that we can now amplify the hardness to a much stronger one.

Theorem 23.3.4. *For every $g_1 < \infty$, $\text{Gap}_{g_1}\text{NCP}$ is NP-hard.*

Proof. The main ingredient of this proof is a reduction that shows that for every g , Gap_gNCP reduces to $\text{Gap}_{g^2}\text{NCP}$ provided the target vector $\mathbf{v} = \mathbf{1}_n$. Given such a reduction the theorem is straightforward. We use Lemma 23.3.3 to get $\text{Gap}_{g_0}\text{NCP}$ is NP-hard for some $g_0 > 1$ (with $\mathbf{v} = \mathbf{1}_n$). We then compose our reductions to get that $\text{Gap}_{g_0}\text{NCP}$ reduces to $\text{Gap}_{g_0^k}\text{NCP}$ for every k of the form 2^s for positive integer s . Setting k large enough so that $g_0^k > g_1$, we get $\text{Gap}_{g_0}\text{NCP}$ reduces to $\text{Gap}_{g_1}\text{NCP}$ and so the latter is NP-hard, yielding the theorem. We thus turn to the reduction asserted above.

The goal of the reduction is to take a code C of block length n (generated by $\mathbf{G}$) and construct a new code that we will call C_2 of block length n^2 (generated by some matrix $\mathbf{G}_2$) such that C_2 contains a codeword at distance at most t^2 from the all 1's vector if and only if C contains a codeword at distance at most t from the all 1's vector. This code C_2 is in turn the direct sum of two codes C_R and C_I (R for repetition and I for independent). Codewords of both codes should be viewed as $n \times n$ matrices: C_R has as its rows codewords of C and the columns are the all 0

vector or the all 1 vector⁵. C_I has as its columns codewords of C , and the rows are arbitrary (so the columns are totally independent). Some inspection reveals that if $\mathbf{w} \in C$ has distance t to $\mathbf{1}_n$ (i.e., it has t zeroes) then the matrix $M_R + M_I$, where the rows of M_R are all $\mathbf{w}$ and M_I is the zero vector on the columns where M_R is all one, and $\mathbf{w}$ on the columns where M_R is all 0, satisfies: (1) $M_R + M_I$ is a codeword of C_2 and (2) $M_R + M_I$ is zero on t^2 coordinates. Further inspection reveals this is the nearest codeword to the all 1's matrix. We give the formal description and proof below.

Given codes C and D recall that $C \otimes D$ denotes the code whose codewords are matrices such that every row is a codeword of C and every column is a codeword of D (recall Exercise 2.20). Let $R \subseteq \mathbb{F}_2^n$ be the code $R = \{0^n, \mathbf{1}_n\}$ be the n -fold repetition code. Let $I = \mathbb{F}_2^n$ be the identity code (i.e., the code corresponding to the identity matrix as the generator). Let $(\mathbb{F}_2, \mathbf{G}, \mathbf{v}, t)$ be an instance of Gap_gNCP with $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ and $\mathbf{v} = \mathbf{1}_n$, and let C be the code generated by $\mathbf{G}$. Let $C_R = C \otimes R$ and let $C_I = I \otimes C$. (Note that by Exercise 2.20 each is a linear code whose generators $\mathbf{G}_R$ and $\mathbf{G}_I$ can be computed in polynomial time from $\mathbf{G}$.) Let $C_2 = C_R + C_I$, i.e.

$$C_2 = \{\mathbf{c}_R + \mathbf{c}_I \mid \mathbf{c}_R \in C_R, \mathbf{c}_I \in C_I\}.$$

Exercise 23.11 shows that we can compute a generator matrix $\mathbf{G}_2$ of code C_2 in polynomial time. Our reduction outputs $(\mathbb{F}, \mathbf{G}_2, \mathbf{1}_{n^2}, t^2)$. We argue below that this reduction is correct.

First we show that if there exists $\mathbf{w} \in C$ such that $\Delta(\mathbf{w}, \mathbf{1}_n) \leq t$ then there exists $\mathbf{w}_2 \in C_2$ such that $\Delta(\mathbf{w}_2, \mathbf{1}_{n^2}) \leq t^2$. Let $\mathbf{v} = \mathbf{1}_n$. Recall that by convention our vectors are row vectors. For row vector $\mathbf{x}$, let $\mathbf{x}^T$ denote its transpose, namely the column vector $\mathbf{x}$. We claim that $\mathbf{w}_2 = \mathbf{v}^T \cdot \mathbf{w} + \mathbf{w}^T \cdot (\mathbf{v} - \mathbf{w})$ satisfies $\mathbf{w}_2 \in C_2$ and $\Delta(\mathbf{w}_2, \mathbf{1}_{n^2}) \leq t^2$. First note that by construction $\mathbf{v}^T \cdot \mathbf{w} \in C_R$ and $\mathbf{w}^T \cdot (\mathbf{v} - \mathbf{w}) \in C_I$ and so $\mathbf{w}_2 \in C_2$. Next to verify the distance, let $S = \{i \mid \mathbf{w}_i = 0\}$. We claim that $(\mathbf{w}_2)_{i,j} = 0$ if and only if $i, j \in S$. (See Exercise 23.12.) The distance bound follows since $|S| \leq t$.

To see the other direction suppose $\mathbf{w}_2 \in C_2$ satisfies $\Delta(\mathbf{w}_2, \mathbf{1}_{n^2}) \leq g^2 t^2$ (i.e. $\mathbf{w}_2$ has $\leq g^2 t^2$ zeroes). We wish to show that there exists $\mathbf{w} \in C$ such that $\Delta(\mathbf{w}, \mathbf{1}_n) \leq gt$. Let $\mathbf{w}_2 = \mathbf{w}_R + \mathbf{w}_I$ where $\mathbf{w}_R \in C_R$ and $\mathbf{w}_I \in C_I$. By our observation on C_R earlier, we have that $\mathbf{w}_R$ contains n identical rows each of which is some codeword $\mathbf{w}_a \in C$, i.e., $\mathbf{w}_R = \mathbf{v}^T \cdot \mathbf{w}_a$. If $\Delta(\mathbf{w}_a, \mathbf{1}_n) \leq gt$ then we are done (setting $\mathbf{w} = \mathbf{w}_a$). If not, we have that $\mathbf{w}_a$ has $> gt$ zeroes, which implies $> gt$ columns of $\mathbf{w}_R$ is all zero. Let T be the matrix obtained by restricting $\mathbf{w}_2$ to those columns where $\mathbf{w}_R$ is all zero. Note that T has at least gt columns (by our observation on $\mathbf{w}_R$ earlier) and each column is a codeword of C (by definition of C_I). Now let $\mathbf{w}_b$ be the column of maximum weight in T . If $\Delta(\mathbf{w}_b, \mathbf{1}_n) > gt$ then we have that $\Delta(\mathbf{w}', \mathbf{1}_n) > gt$ for every column $\mathbf{w}'$ of T ; and so T , and hence $\mathbf{w}_2$ has strictly more than $g^2 t^2$ zeroes contradicting our assumption. We conclude that $\Delta(\mathbf{w}_b, \mathbf{1}_n) \leq gt$.

This gives the desired reduction from Gap_gNCP reduces to $\text{Gap}_{g^2}\text{NCP}$, which concludes the proof. $\square$

We note that Theorem 23.5.3 (which we will prove later) provides an alternate proof of Theorem 23.3.4 (see Exercise 23.13).

⁵Alternatively, each codeword matrix in C_R has the same codeword from C in all of its rows.

Theorem 23.3.4 thus rules out seemingly very weak approximation algorithms also. In fact the proof rules out even more (e.g. we can fix the received word $\mathbf{v}$ to be the all ones vector), but it is important to note that these are likely being ruled out in codes whose minimum distance is quite small, and so correcting large amount of errors (much more than the distance) is not a particularly useful task. The next section turns to this question.

23.4 Distance bounded decoding

In this section we consider the task of decoding when the number of errors is bounded by the distance of the code. Once again formalizing the problem is non-trivial given that we do not know of an algorithm to compute the minimum distance of a code (see Section 23.5 for the hardness of the problem of computing the distance of a linear code). Once again promise problems come to our rescue in articulating the problem here. We define the problem already with a gap, keeping in mind some future applications.

To define this problem, we again define two disjoint subsets of inputs to decoding problems. For a matrix $\mathbf{G} \in \mathbb{F}^{k \times n}$ let $d(\mathbf{G})$ denote the minimum distance of the code generated by $\mathbf{G}$, i.e. (recall Proposition 2.3.6),

$$d(\mathbf{G}) = \min_{\mathbf{x} \in \mathbb{F}^k \setminus \{0^k\}} \{\text{wt}(\mathbf{xG})\}.$$

For a real number $g \geq 1$, let

$$\text{Gap}_g\text{DBD}_{\text{Yes}} = \{(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \mid d(\mathbf{G}) \geq g \cdot t \text{ and } \exists \mathbf{x} \in \mathbb{F}_2^k \text{ s.t. } d(\mathbf{v}, \mathbf{xG}) \leq t\}$$

and

$$\text{Gap}_g\text{DBD}_{\text{No}} = \{(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \mid d(\mathbf{G}) \geq g \cdot t \text{ and } \forall \mathbf{x} \in \mathbb{F}_2^k, d(\mathbf{v}, \mathbf{xG}) > g \cdot t\}.$$

We now define the Distance Bounded Decoding problem to be the restriction of NCP to the union of the sets above. We discuss the meaning of this problem after the definition.

Problem 23.4.1 (Gap Distance Bounded Decoding (GapDBD) with parameter g).

- **Input:** $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{DBD}_{\text{Yes}} \cup \text{Gap}_g\text{DBD}_{\text{No}}$
- **Output:** YES if $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{DBD}_{\text{Yes}}$ and NO otherwise.

So in other words, GapDBD is the restriction of GapNCP to instances where the code itself has distance greater than $d = g \cdot t$ and the goal is to determine if the target $\mathbf{v}$ is within distance of d/g from the code, or at least d . Thus, if the goal of NCP is to detect instances where the target vector $\mathbf{v}$ is obtained by introducing few errors into a codeword, DBD enhances the problem by ensuring that the number of errors is less than the distance of the code. We remark that we will not be going down to half the distance of the code. As we discuss later (Open Question 23.6.1) decoding up to half the minimum distance of every code is not known to be NP-complete. But in this section we will see that decoding up to the minimum distance is actually NP-complete, at least under randomized reductions.

Theorem 23.4.2. *There exists $g' > 1$, such that $\text{Gap}_{g'}\text{DBD}$ is NP-hard under randomized reductions. Specifically there exists g and a randomized polynomial time reduction R from Gap_gNCP to $\text{Gap}_{g'}\text{DBD}$ such that for every instance $I \in \text{Gap}_g\text{NCP}_{\text{Yes}} \cup \text{Gap}_g\text{NCP}_{\text{No}}$ we have*

- (1) $R(I) \in \text{Gap}_{g'}\text{DBD}_{\text{Yes}} \cup \text{Gap}_{g'}\text{DBD}_{\text{No}}$ with probability $1 - o(1)$, and
- (2) with probability $1 - o(1)$, $R(I) \in \text{Gap}_{g'}\text{DBD}_{\text{Yes}}$ if and only if $I \in \text{Gap}_g\text{NCP}_{\text{Yes}}$.

Note that the above result states that decoding up to $\frac{d(\mathbf{G})}{g'}$ is NP-hard (under randomized reductions). Ideally, we would like to have g' be as close to 2 as possible (but such a result is not known as mentioned earlier). See Exercise 23.14 for more the largest value of g' that we can achieve.

To prove Theorem 23.4.2, we need several ingredients, each one of which is non-trivial. We first motivate their need, then state lemmas asserting their existence, and then use them to prove the theorem.

Our goal is to consider an instance $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ of Gap_gNCP and somehow ‘boost’ the distance of the underlying code. Let $\mathbf{G} \in \mathbb{F}^{k \times n}$. One simple idea would be to take a known code of high distance of dimension k , block length n_0 with generator $\mathbf{G}_0$ and adjoin $\mathbf{G}_0$ to $\mathbf{G}$ to get a new code. So the new code is generated by $\mathbf{G}' = [\mathbf{G}_0 | \mathbf{G}]$. Clearly this code has high distance since $\mathbf{G}_0$ itself has high distance. However it is yet unclear how to extend our target vector $\mathbf{v}$ to some $(n_0 + n)$ -dimensional vector. Here comes the first new ingredient. We will select C_0 , the code generated by $\mathbf{G}_0$ to be a code of large distance, say d_0 , and also find a vector $\mathbf{w} \in \mathbb{F}^{n_0}$ that has many codewords of C_0 at distance at most $(1 - \varepsilon)d_0$ from it (this is done in Lemma 23.4.3). Now we can try to use the vector $\mathbf{v}' = (\mathbf{w}, \mathbf{v})$ as our target.

Indeed we now have a candidate reduction (assuming we are given $\mathbf{G}_0$ and $\mathbf{w}$) which maps $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ to $(\mathbb{F}, \mathbf{G}', \mathbf{v}', t')$ where $\mathbf{G}' = [\mathbf{G}_0 | \mathbf{G}]$, $\mathbf{v}' = (\mathbf{w}, \mathbf{v})$ and $t' = t + (1 - \varepsilon)d_0$. If we select $t = \varepsilon \frac{d_0}{2}$ and $g' = \frac{1}{1 - \varepsilon/2}$, then we get that the resulting code has distance at least d_0 , while the target distance is at most $(1 - \varepsilon/2)d_0$. Furthermore the reduction is ‘sound’ in that if there exists $\mathbf{x}$ such that $\Delta(\mathbf{x}\mathbf{G}', \mathbf{v}') \leq d_0 = g' \cdot t'$ then $\Delta(\mathbf{x}\mathbf{G}, \mathbf{v}) \leq d_0 \leq g t$ (where the last inequality follows by choosing $g = 2/\varepsilon$), and so $(\mathbb{F}, \mathbf{G}', \mathbf{v}', t') \in \text{Gap}_g\text{DBD}_{\text{Yes}}$ implies that $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_{g'}\text{NCP}_{\text{Yes}}$.

Completeness (i.e., the condition $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_{g'}\text{NCP}_{\text{Yes}}$ implies that $(\mathbb{F}, \mathbf{G}', \mathbf{v}', t') \in \text{Gap}_g\text{DBD}_{\text{Yes}}$), unfortunately does not hold for this reduction. In particular if there exists $\mathbf{x}$ such that $\Delta(\mathbf{x}\mathbf{G}, \mathbf{v}) \leq t$, we don’t necessarily have $\Delta(\mathbf{x}\mathbf{G}_0, \mathbf{w}) \leq (1 - \varepsilon)d_0$. (There are many such $\mathbf{x}$ ’s, but not every $\mathbf{x} \in \mathbb{F}^k$ satisfies this condition.) To fix this problem we need a second idea: Roughly, the set $S = \{\mathbf{y} | \Delta(\mathbf{y}\mathbf{G}_0, \mathbf{w}) \leq (1 - \varepsilon)d_0\}$ is too unstructured. We will impose structure on it by compressing it, using a random linear map A . If the parameters are chosen right then the image of S under A will be the entire range (or at least any given element will be hit with high probability) and so in particular there will exist $\mathbf{y} \in S$ such that $\mathbf{y}\mathbf{A} = \mathbf{z}$ for the $\mathbf{z}$ such that $\Delta(\mathbf{z}\mathbf{G}, \mathbf{v}) \leq t$. This ensures the desired completeness. We now state our lemmas which assert the existence of $\mathbf{G}_0, \mathbf{w}$ and A as mentioned above and formally prove the theorem follows from them.

Lemma 23.4.3. *There exists $0 < \varepsilon > \frac{1}{2}$ and a randomized algorithm that on input an integer k , runs in time polynomial in k and outputs, with probability $1 - o(1)$ integers k_0, n_0 , a matrix*

$\mathbf{G}_0 \in \mathbb{F}_2^{k_0 \times n_0}$ generating a code C_0 of minimum distance

$$d_0 = \frac{k}{2} \cdot \left\lfloor (k-1)^{\frac{2}{1-2\varepsilon}} \right\rfloor$$

and a vector $\mathbf{w} \in \mathbb{F}^{n_0}$ such that

$$\left| \left\{ \mathbf{y} \in \mathbb{F}^{k_0} | \Delta(\mathbf{y}\mathbf{G}_0, \mathbf{w}) \leq (1 - \varepsilon)d_0 \right\} \right| \geq 4^k.$$

(The above result does not produce an asymptotically good code but we can prove a stronger version of Lemma 23.4.3 that produces an asymptotically good code— see Exercise 23.18.)

Our next lemma is a standard property of linear maps, used for instance in constructions of small families of hash functions. It says that if $S \subseteq \mathbb{F}^{k_0}$ is a set of size sufficiently larger than $\mathbb{F}^k$ then for every $\mathbf{y} \in \mathbb{F}^k$ the probability that for a random $A \in \mathbb{F}^{k_0 \times k}$ that there exists a $\mathbf{x} \in S$ such that $\mathbf{x}\mathbf{A} = \mathbf{y}$ is close to 1.

Lemma 23.4.4. *For integers k, k_0, N let $S \subseteq \mathbb{F}_2^{k_0}$ be a set of size at least N and let $\mathbf{z} \in \mathbb{F}_2^k$ be a fixed vector. Then for a uniformly random $\mathbf{A} \in \mathbb{F}_2^{k_0 \times k}$, we have*

$$\Pr_{\mathbf{A}}[\text{Exists } \mathbf{y} \in S \text{ s.t. } \mathbf{y}\mathbf{A} = \mathbf{z}] \geq 1 - \frac{2^k}{N}.$$

Given the lemmas above, Theorem 23.4.2 is not too hard to prove along the lines described earlier and we do so below.

Proof of Theorem 23.4.2. Let ε be as given by Lemma 23.4.3. We assume (to avoid some complex roundings) that $\varepsilon = \frac{1}{s}$ for some integer s ; if this is not the case we can reduce ε so that it takes this form while remaining positive. We let

$$g' = \frac{1}{1 - \varepsilon/2}$$

and

$$g = \frac{2}{\varepsilon}.$$

For this choice of parameters, we show that Gap_gNCP reduces to $\text{Gap}_{g'}\text{DBD}$.

We start with the reduction. Let $(\mathbb{F}_2, \mathbf{G}, \mathbf{v}, t)$ be an instance of $\text{Gap}_{g'}\text{NCP}$, with $\mathbf{G} \in \mathbb{F}_2^{k \times n}$. Let $n_0, k_0, \mathbf{G}_0, d_0$ and $\mathbf{w} \in \mathbb{F}_2^{n_0}$ be the output of the randomized algorithm from Lemma 23.4.3 on input k . Let⁶

$$d_0 = t \cdot g = \frac{2t}{\varepsilon}.$$

⁶If $t < d_0 \cdot \frac{\varepsilon}{2}$, then we can take the product of the code generated by $\mathbf{G}$ with a $\left\lceil \frac{d_0 \varepsilon}{2t} \right\rceil$ -fold repetition code and repeat $\mathbf{v}$ also $\left\lceil \frac{d_0 \varepsilon}{2t} \right\rceil$ times — this will multiply n and t by $\left\lceil \frac{d_0 \varepsilon}{2t} \right\rceil$ while leaving membership in $\text{Gap}_g\text{NCP}_{\text{Yes}}$ or $\text{Gap}_g\text{NCP}_{\text{No}}$ unchanged. If on the other hand, $t > d_0 \cdot \frac{\varepsilon}{2}$, then the code C_0 from Lemma 23.4.3 would be the $\left\lceil \frac{2t}{d_0 \varepsilon} \right\rceil$ -fold repetition of the code generated by $\mathbf{G}_0$ (and we repeat $\mathbf{w}$ the same number of times). This would increase the distance of the code to the required amount but the other properties remains preserved. We note that there might be some rounding errors that we are ignoring for the sake of clarity.

Let $\mathbf{A} \in \mathbb{F}_2^{k_0 \times k}$ be a uniformly random matrix. Then let

$$\mathbf{G}' = [\mathbf{G}_0 | \mathbf{A}\mathbf{G}],$$

so that $\mathbf{G}' \in \mathbb{F}_2^{k_0 \times (n_0 + n)}$. Further, let $\mathbf{v}' = (\mathbf{w}, \mathbf{v}) \in \mathbb{F}_2^{n_0 + n}$. Finally let

$$t' = t + (1 - \varepsilon)d_0.$$

Note that for this choice of t' and the choice of d_0 we have

$$t' = \frac{\varepsilon}{2}d_0 + (1 - \varepsilon)d_0 = \left(1 - \frac{\varepsilon}{2}\right) \cdot d_0 = \frac{d_0}{g'},$$

or which yields $d_0 \geq g't'$ as desired⁷. We show in the next two paragraphs that (1) Completeness holds, i.e., $(\mathbb{F}_2, \mathbf{G}', \mathbf{v}', t') \in \text{Gap}_g\text{DBD}_{\text{Yes}}$ if $(\mathbb{F}_2, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{NCP}_{\text{Yes}}$ (with high probability) and (2) Soundness holds, i.e., $(\mathbb{F}_2, \mathbf{G}', \mathbf{v}', t') \in \text{Gap}_g\text{DBD}_{\text{No}}$ if $(\mathbb{F}_2, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{NCP}_{\text{No}}$ (with probability 1), and this will prove the theorem.

We start with the soundness since it is simpler. Since $(\mathbb{F}_2, \mathbf{G}, \mathbf{v}, t) \in \text{Gap}_g\text{NCP}_{\text{No}}$ we have for every $\mathbf{x} \in \mathbb{F}_2^k$ we have $\Delta(\mathbf{v}, \mathbf{x}\mathbf{G}) > gt$. It follows that for every $\mathbf{y} \in \mathbb{F}_2^{k_0}$ we have $\Delta(\mathbf{v}, \mathbf{y}\mathbf{A}\mathbf{G}) > gt$ (since this holds for $\mathbf{x} = \mathbf{y}\mathbf{A}$). Finally since $\Delta((\mathbf{w}, \mathbf{v}), \mathbf{y}[\mathbf{G}_0 | \mathbf{A}\mathbf{G}]) \geq \Delta(\mathbf{v}, \mathbf{y}\mathbf{A}\mathbf{G})$ we conclude

$$\Delta(\mathbf{v}', \mathbf{y}\mathbf{G}') \geq gt = d_0 = g't',$$

as desired (recall we argued above that $d_0 = g't'$).

Finally we need to argue the completeness. Let $\mathbf{z} \in \mathbb{F}_2^k$ be such that $\Delta(\mathbf{v}, \mathbf{z}\mathbf{G}) \leq t$. We now assume that the conditions of Lemmas 23.4.3 and 23.4.4 hold, i.e.,

- (i) $S \triangleq \{\mathbf{y} \in \mathbb{F}_2^{k_0} | \Delta(\mathbf{y}\mathbf{G}, \mathbf{w}) \leq (1 - \varepsilon)d_0\}$ satisfies $|S| \geq 4^k$; and
- (ii) For the $\mathbf{z}$ and S as defined above, there exists $\mathbf{y} \in S$ such that $\mathbf{y}\mathbf{A} = \mathbf{z}$.

Note that the probability that any one of these events does not happen is $o(1)$. (In particular since $|S| \geq 4^k$, by Lemma 23.4.4 the probability that (ii) does not hold is at most $2^k/4^k = o(1)$.) So by the union bound we get the probability that both hold simultaneously is still at least $1 - o(1)$. We now verify that for this choice of $\mathbf{y}$, we have $\Delta(\mathbf{v}', \mathbf{y}\mathbf{G}') \leq t'$ and this will conclude the proof. To verify this note that since $\mathbf{y} \in S$ we have $\Delta(\mathbf{y}\mathbf{G}_0, \mathbf{w}) \leq (1 - \varepsilon)d_0$. And since $\mathbf{y}\mathbf{A} = \mathbf{z}$ we have

$$\Delta(\mathbf{v}, \mathbf{y}\mathbf{A}\mathbf{G}) = \Delta(\mathbf{v}, \mathbf{z}\mathbf{G}) \leq t.$$

We conclude that

$$\Delta(\mathbf{v}', \mathbf{y}\mathbf{G}') = \Delta(\mathbf{w}, \mathbf{y}\mathbf{G}_0) + \Delta(\mathbf{v}, \mathbf{y}\mathbf{A}\mathbf{G}) \leq (1 - \varepsilon)d_0 + t \leq t',$$

as desired. □

⁷Recall that we need $d(\mathbf{G}') \geq g' \cdot t'$.

23.5 Minimum distance problem

Finally we turn to a different computational problem associated with codes — that of determining the minimum distance of a (linear) code.⁸ Specifically we consider the task of determining, or approximating the minimum distance of a code given its generator matrix. We show directly that even the latter is a hard task, specifically that it is NP-hard under randomized reductions. As usual to show such hardness we work with gap problems. We define the *Gap Minimum Distance Problem* next.

As usual, let $\mathbb{F}$ denote a field, $\mathbf{G} \in \mathbb{F}^{k \times n}$ denote a generator matrix of a code of dimension k and block length n , and let $d(\mathbf{G})$ denote the minimum distance of a code. We define our Gap problem by defining as YES instances the codes of small minimum distance, and as NO instances the codes of large minimum distance.⁹ For a real number $g \geq 1$, let

$$\text{Gap}_g\text{MINDIST}_{\text{Yes}} = \{(\mathbb{F}, \mathbf{G}, \bar{d}) | d(\mathbf{G}) \leq \bar{d}\}$$

$$\text{and } \text{Gap}_g\text{MINDIST}_{\text{No}} = \{(\mathbb{F}, \mathbf{G}, \bar{d}) | d(\mathbf{G}) > g \cdot \bar{d}\}.$$

Problem 23.5.1 (Gap Minimum Distance (GapMinDist)) with parameter).

- **Input:** $(\mathbb{F}, \mathbf{G}, \bar{d}) \in \text{Gap}_g\text{MINDIST}_{\text{Yes}} \cup \text{Gap}_g\text{MINDIST}_{\text{No}}$
- **Output:** YES if $(\mathbb{F}, \mathbf{G}, \bar{d}) \in \text{Gap}_g\text{MINDIST}_{\text{Yes}}$ and NO otherwise.

Lemma 23.5.2. *The following are true:*

- (i) For every $g > 1$ there is a deterministic polynomial time reduction from Gap_gDBD to $\text{Gap}_g\text{MINDIST}$.
- (ii) For every $g > 1$ and $g' < \infty$ there is a deterministic polynomial time reduction from $\text{Gap}_g\text{MINDIST}$ to $\text{Gap}_{g'}\text{MINDIST}$.

Proof. Both parts are quite simple.

For part (i) given an instance $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$ of Gap_gDBD we transform it to the instance $(\mathbb{F}, \mathbf{G}', \bar{d})$ for $\bar{d} = t$ and $\mathbf{G}' = \begin{bmatrix} \mathbf{G} \\ \mathbf{v} \end{bmatrix}$, i.e., $\mathbf{G}'$ is a $(k + 1) \times n$ matrix with $\mathbf{v}$ being the added row. One can show that if for some $\mathbf{y} \in \mathbb{F}^k$, $\Delta(\mathbf{v}, \mathbf{y}\mathbf{G}) \leq t$ then $d(\mathbf{G}') \leq t$, while if for every $\mathbf{y} \in \mathbb{F}^k$, we have $\Delta(\mathbf{v}, \mathbf{y}\mathbf{G}) > gt$ and $d(\mathbf{G}) > gt$ then $d(\mathbf{G}') > gt$ —see Exercise 23.21. This proves the completeness and soundness of the reduction.

For part (ii) we reduce $(\mathbb{F}, \mathbf{G}, \bar{d})$ to $(\mathbb{F}, \mathbf{G}^{\otimes \ell}, \bar{d}^\ell)$, where $\mathbf{G}_1 \otimes \mathbf{G}_2$ denotes the tensor product of matrices (recall Exercise 2.20), and $\mathbf{G}^{\otimes \ell}$ denotes the product of $\mathbf{G}$ with itself ℓ times. Recall from

⁸We note that the linearity condition is required since the problem of computing the distance of a completely arbitrary code is polynomial time—see Exercise 23.20

⁹Note that this inversion is necessitated by the fact that it is possible to prove that a code has small minimum distance by exhibiting two nearby codewords. In particular, this gives the definition of a witness to show that the problem is in NP (see Definition C.5.4).

Exercise 2.20 that the tensor product of two codes of minimum distance d_1 and d_2 respectively yields a code of minimum distance $d_1 d_2$. This implies that $d(\mathbf{G}^{\otimes \ell}) = d(\mathbf{G})^\ell$ and so if $d(\mathbf{G}) \leq \bar{d}$ then $d(\mathbf{G}^{\otimes \ell}) \leq \bar{d}^\ell$ and if $d(\mathbf{G}) > g\bar{d}$ then $d(\mathbf{G}^{\otimes \ell}) > (g\bar{d})^\ell = g^\ell \bar{d}^\ell$. By picking ℓ large enough we get $g^\ell \geq g'$ and this yields the desired reduction from $\text{Gap}_g \text{MINDIST}$ to $\text{Gap}_{g'} \text{MINDIST}$. $\square$

Combining Lemma 23.5.2 with Theorem 23.4.2 we conclude that approximating the minimum distance to within any constant factor is hard. Specifically it is NP-hard under randomized reductions.

Theorem 23.5.3. *For every constant $g < \infty$, the problem $\text{Gap}_g \text{MINDIST}$ is NP-hard under randomized reductions. Consequently, unless all of NP has randomized polynomial time algorithms, there are no (randomized) polynomial time algorithms to approximate the minimum distance of a linear code given its generator to within a multiplicative factor of g .*

We note that the NP-hardness result above is not necessarily for an asymptotically good code but see Exercise 23.22 on how to extend it to work for asymptotically good codes as well.

23.6 Conclusions

To summarize the results of this chapter, we see that many coding theoretic problems can become very hard (NP-hard, or NP-hard under randomized reductions) if the code is not carefully designed. In particular, if we are just given the generator matrix of a code, it may be hard to determine the minimum distance of the code (Theorem 23.5.3), or decode the nearest codeword (Theorem 23.1.4), or even find a nearby codeword (not necessarily the nearest— see Theorem 23.3.4). Furthermore the decoding may remain hard even if one is given arbitrary amounts of time to preprocess the code (i.e., to design an efficient decoder)— see Theorem 23.2.2.

One way to avoid the hardness results, is to design the codes carefully — which is exactly what we have been doing for much of this book. But there is another glimmer of hope (yet). All the hardness results work beyond the list-decoding setting, i.e., when the number of errors is so large that a full list of codewords within the target ball may be exponentially large. What happens if the list sizes are guaranteed to be small? Or even unique — i.e., when the goal is to decode only up to the error-correction bound of the code (i.e., half its minimum distance). Specifically we note that

Open Question 23.6.1. *The status of the $\text{Gap}_{1/2} \text{DBD}$ problem (whether it is in P or whether it is NP-hard) is still open.*

We note that hardness of coding problems has been one of the sources of ‘hard’ problems for cryptography as well. Examples of such proposals include the McEliece cryptosystem and Alekhnovich’s cryptosystem. More broadly an entire array of cryptographic primitives have now been proposed based on the *Learning Parity with Noise* (LPN) and *Learning With Noise* (LWN)

problems, both of which are essentially problems based on decoding linear codes from error. Any attempts to prove the security of these schemes, or to firm them up further, will surely require an improved understanding of the problems discussed in this chapter.

23.7 Exercises

Exercise 23.1. *Argue that for an arbitrary (not necessarily linear) code, one can perform encoding and decoding in time polynomial in the description size of the code. (Recall that in the worst-case one has to explicitly list all the codewords.)*

Exercise 23.2. *Show that if there is a polynomial time algorithm for NCP (Problem 23.1.1, then there is a polynomial time algorithm to find an $\mathbf{x}$ such that $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$. Show the converse (i.e. if there exists a polynomial time algorithm for the latter problem then there exists one for NCP as well).*

Exercise 23.3. *In this exercise, we will see how one can extend the proof of Theorem 23.1.4 to make it work for any field $\mathbb{F}$.*

1. (Warmup) Consider the variant of NCP where instead of looking for any $\mathbf{x} \in \mathbb{F}^k$ such that $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$, we only consider binary vectors. I.e. given $(\mathbb{F}, \mathbf{G}, \mathbf{v}, t)$, output YES if there exists an $\mathbf{x} \in \{0, 1\}^k$ such that $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$ and NO otherwise. Argue that this problem is NP-complete.

Hint: The proof of Theorem 23.1.4 with very little modification should do the trick.

2. Argue that the NCP problem is NP-complete for any field $\mathbb{F}$.

Hint: Modify all of $\mathbf{G}$, $\mathbf{v}$ and t from the proof of Theorem 23.1.4 so that the only $\mathbf{x} \in \mathbb{F}^k$ that can have $\Delta(\mathbf{v}, \mathbf{xG}) \leq t$ are binary vectors. Then use the previous part.

Hint: To modify $\mathbf{G}$ consider adding appropriate number of copies of the identity matrix $\mathbf{I}_{k \times k}$ to the $\mathbf{G}$ from the proof of Theorem 23.1.4.

Exercise 23.4. *Argue why the reduction in the proof of Theorem 23.1.4 fails if we pick $\mathbf{v}$ to be the all zeroes vector.*

Exercise 23.5. *Argue the following. Let $\mathbf{x} \in \mathbb{F}_2^n$ and $S = \text{supp}(\mathbf{x})$. Then we have $(\mathbf{xG})_e = 1$ (where $\mathbf{G}$ is as defined in proof of Theorem 23.1.4) if and only if $e \in \text{Cut}(S)$*

Exercise 23.6. *Let $\mathbf{G}_n$ be as defined in the proof of Theorem 23.2.2. Argue that for every $\mathbf{x} \in \mathbb{F}_2^k$ and every $\{i, j\}$ we have that $(\mathbf{xG}_n)_{(i,j)} = (\mathbf{xG}_n)_{(j,i)}$.*

Exercise 23.7. *Prove that if $\text{NCP}\{\mathbf{G}_n\}_n$ has an efficient non-uniform algorithm, then so does all of NP.*

Exercise 23.8. *In this problem we look into the distance of the code generated in the proof of Theorem 23.2.2. For the rest of the problem let $\mathbf{G}_n$ be as defined in the proof of Theorem 23.2.2.*

1. Argue that the distance d_k of the code generated by $\mathbf{G}_n$ is $2(k-1)$ (recall that $n = k(k-1)$).

2. Argue that the threshold quantity t in the proof of Theorem 23.2.2 satisfies $t \geq \frac{k}{4} \cdot d_k$. Conclude that for large enough k , the distance threshold t is larger than half the distance of the code and hence there could be more than one possible codeword close enough to the received word.

Exercise 23.9. Show that if A is a poly time algorithm such that $A(\mathbb{F}, \mathbf{G}, \mathbf{v})$ always outputs a g -approximate nearby codeword to $\mathbf{v}$, then Gap_gNCP can be decided in polynomial time.

Exercise 23.10. In this problem, we will reduce Gap_gCUT to $\text{Gap}_{g'}\text{NCP}$. We will do this in two parts:

1. Argue that any graph $G = (V, E)$ has a cut of size at least $\frac{|E|}{2}$.
Hint: Use the probabilistic method.
2. Reduce Gap_gCUT to Gap_gNCP with $\mathbf{v} = \mathbf{1}$. Further argue that the reduction will satisfy $g' > 1$ if $g > 1$. Conclude that Lemma 23.3.3 holds.

Hint: Use the first part and the proof of Theorem 23.1.4.

Exercise 23.11. Let C_1 and C_2 be linear codes with generator matrices $\mathbf{G}_1$ and $\mathbf{G}_2$ respectively. Let C_3 be their direct sum, i.e.

$$C_3 = \{\mathbf{c}_1 + \mathbf{c}_2 \mid \mathbf{c}_1 \in C_1, \mathbf{c}_2 \in C_2\}.$$

Show how to compute a generator matrix for C_3 from $\mathbf{G}_1, \mathbf{G}_2$ in polynomial time.

Exercise 23.12. We first recall some notation used in the proof of Theorem 23.3.4. Let $\mathbf{w}_2 = \mathbf{v}^T \cdot \mathbf{w} + \mathbf{w}^T \cdot (\mathbf{v} - \mathbf{w})$, where $\mathbf{w} \in C$ and $\mathbf{v} = \mathbf{1}_n$. Further define

$$S = \{\mathbf{w}_i = 0\}.$$

Then argue that $(\mathbf{w}_2)_{i,j} = 0$ iff $i, j \in S$.

Exercise 23.13. In this problem we show the following for any $g > 1$. If $\text{Gap}_g\text{NCP} \in \text{promise-P}$, then $\text{Gap}_g\text{MINDIST} \in \text{promise-P}$. In particular, we will prove a Cook/Turing reduction from $\text{Gap}_g\text{MINDIST}$ to Gap_gNCP (i.e. a reduction that preserves the hardness of approximation as well). Note that assuming Theorem 23.5.3, this gives an alternate proof for Theorem 23.3.4. We will do so in a sequence of steps.

Assume we are given as an input a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ for a code C whose distance d we want to compute/approximate (i.e. $\mathbf{G}$ will be an input for the $\text{Gap}_g\text{MINDIST}$ problem). Define $\mathbf{G}^i$ to be $\mathbf{G}$ without its i th row and let C^i be the corresponding code. We consider the following k instances of the Gap_gNCP problem: $(\mathbb{F}_q, \mathbf{G}^i, \mathbf{c}^i, d)$ where $\mathbf{c}^i$ is the i th row of $\mathbf{G}$ (for $i \in [k]$).

1. Argue that for every $i \in [k]$, all codewords in C^i are at distance at least d from $\mathbf{c}^i$. The next couple of problems tries to prove that in some sense the converse holds as well.
2. Let $\mathbf{u} \in C$ be a non-zero codeword. Argue that there exists at least one $i \in [k]$ such that $\mathbf{c}^i$ is at distance at most $wt(\mathbf{u})$ from some codeword in C^i .

Hint: Let $\mathbf{u} = \sum_{j=1}^k \alpha_j \cdot \mathbf{c}^j$. Let $i \in [k]$ be such that $\alpha_i \neq 0$ (which should such an i exist?). Think of an appropriate $\beta \neq 0$ (that is related to α_i) such that $\Delta(\sum_{j \neq i \in [k]} \beta \cdot \alpha_j \cdot \mathbf{c}^j, \mathbf{c}^i) = wt(\mathbf{u})$.

3. Using the above (or otherwise), argue that there exists an $i \in [k]$ such that $\Delta(\mathbf{c}^i, C^i) = d$.
4. Using the above sub-problems or otherwise argue that if $\text{Gap}_g\text{NCP} \in \text{promise-P}$, then $\text{Gap}_g\text{MINDIST} \in \text{promise-P}$.

Exercise 23.14. What is the largest value of g' for which you can argue Theorem 23.4.2?

Exercise 23.15. In this exercise we will argue that for any code with distance d there are lots of Hamming balls of radius $d(1 - \epsilon)$ with 'lots' of codewords in it. In particular, we will quantify what 'lots' means for any code.

Let C be an $(n, k, d)_q$ code (note that the code need not be linear). Now construct a bipartite graph $G = (C, [q]^n, E)$, where E is defined as follows:

$$E = \{(\mathbf{c}, \mathbf{y}) \mid \mathbf{c} \in C, \mathbf{y} \in [q]^n, \Delta(\mathbf{c}, \mathbf{y}) \leq (1 - \epsilon)d\}.$$

Also for notational convenience define $r = (1 - \epsilon)d$. We talk through a sequence of problems to prove our final bound:

1. Argue that G is left regular: i.e. every $\mathbf{c} \in C$ has degree D_L for some positive integer D_L .
2. Argue that $D_L = \text{Vol}_q(r, n)$
3. Argue the average degree of the right vertices in G satisfy

$$\overline{D_R} = q^{k-n} \cdot \text{Vol}_q(r, n).$$

4. Argue that at most γ fraction of edges $e = (u, v) \in E$ such that v has degree at most $\gamma \cdot \overline{D_R}$.
5. Argue that pick $\mathbf{c} \in C$ uniformly at random and then pick a random right neighbor of $\mathbf{c}$ is the same as picking an edge in E uniformly at random.
6. Using the above parts or otherwise argue that

$$\Pr_{\mathbf{c} \in C, \mathbf{y} \in B(\mathbf{c}, r)} \left[|C \cap B(\mathbf{y}, r)| \leq \gamma \cdot q^{k-n} \cdot \text{Vol}_q(r, n) \right] \leq \gamma.$$

7. Argue that any code that lies beyond the GV bound (i.e. has relative distance δ and rate at least $1 - H_q(\delta) + \epsilon_0$) satisfies the following property with all but an exponentially small probability— a random Hamming ball of relative radius $(1 - \epsilon)\delta$ has exponentially many codewords (where ϵ is some constant that depends on ϵ_0 and δ).

Exercise 23.16. In this exercise we will prove Lemma 23.4.3 but for large enough alphabet size. In Exercise 23.17, we will see how to get the result for binary codes.

Let $0 < \delta < \frac{1}{2}$ be arbitrary and define

$$\epsilon = \frac{1}{2} - \delta.$$

Assume $\ell \geq 33$ and let q be a power of 2 such that

$$q^\delta = \ell - 1.$$

(We note that the equality might not hold for all ℓ but we make the above simplifying assumptions to make some of the subsequent calculations easier.)

Let C be an $[q, q - q^\delta, q^\delta + 1]_q$ Reed-Solomon code. We will now prove Lemma 23.4.3 (with slightly different parameters) using the Reed-Solomon code C and Exercise 23.15.

1. Define

$$\gamma = \frac{4^\ell \cdot q^{\ell-1}}{\text{Vol}_q((1-\varepsilon)\ell, q)}. \quad (23.1)$$

Then argue that for a random $\mathbf{w} \in \mathbb{F}_q^\ell$ with probability at least $1 - \gamma$, we have that

$$|\{\mathbf{c} \in C \mid \Delta(\mathbf{w}, \mathbf{c}) \leq (1-\varepsilon)\ell\}| \geq 4^\ell.$$

2. Argue that γ as defined in (23.1) satisfies $\gamma \leq 2^{-\ell}$.

3. Using the above parts or otherwise, argue that the statement of Lemma 23.4.3 holds (except that $d_0 = \ell$ and the result is for a q -ary code instead of a binary code).

Exercise 23.17. Prove Lemma 23.4.3.

Hint: Use Exercise 23.1 along with a Hadamard code. The proof of Proposition 2.6.3 might be useful.

Exercise 23.18. Prove Lemma 23.4.3 but for an asymptotically good code.

Hint: Use the fact that there exists q -ary codes with $q \geq 49$ with relative distance δ and rate at least $1 - \delta - \frac{1}{\sqrt{q}-1}$ (and this is known to be strictly better than the GV bound).

Exercise 23.19. In this exercise we will prove Lemma 23.4.4 via a sequence of steps. Let $k, k_0, N, S, \mathbf{z}, \mathbf{A}$ be as defined in statement of Lemma 23.4.4. Then consider the following steps:

1. Re-define S to subset of size exactly $N - 1$ such that all vectors in S are non-zero.¹⁰ Define $N_{\mathbf{z}}$ to be (the random variable that denotes) the number of vectors $\mathbf{y}$ such that $\mathbf{y}\mathbf{A} = \mathbf{z}$. Argue that

$$\mathbb{E}_{\mathbf{A}}[N_{\mathbf{z}}] = (N - 1) \cdot 2^{-k}.$$

2. Let $N_{\mathbf{z}}$ be as defined in the item above. Argue that

$$\mathbb{E}_{\mathbf{A}}[(N_{\mathbf{z}})^2] = (N - 1) \cdot 2^{-k} + (N - 1)(N - 2) \cdot 2^{-2k}.$$

Hint: Argue that for any $\mathbf{y}_1 \neq \mathbf{y}_2 \in S$, $\mathbf{y}_1\mathbf{A}$ and $\mathbf{y}_2\mathbf{A}$ are independent and then use this fact.

¹⁰Convince yourself that this is valid.

3. We will take a bit digression to prove a general result about random variables. Let X be a non-negative integer random variable. Argue that

$$\Pr[X > 0] \geq \frac{(\mathbb{E}[X])^2}{\mathbb{E}[X^2]}.$$

Hint: Write down the expression for $(\mathbb{E}[X])^2$ and use Cauchy-Schwartz inequality (Lemma B.1.6).

4. Using the above parts (or otherwise), complete the proof of Lemma 23.4.4.

Hint: The lemma can also be proved using Chebyshev's inequality without using the previous part.

Exercise 23.20. Argue that given a completely arbitrary code C , one can compute its distance in polynomial time (in the size of the representation of C).

Hint: Recall that an arbitrary $(n, k)_q$ code needs $\Theta(q^k \cdot n)$ space to represent it.

Exercise 23.21. Let $\mathbf{v}, \mathbf{G}$ and $\mathbf{G}'$ be as defined in the proof of Lemma 23.5.2. Argue the following:

1. If for some $\mathbf{y} \in \mathbb{F}^k$, $\Delta(\mathbf{v}, \mathbf{y}\mathbf{G}) \leq t$ then $d(\mathbf{G}') \leq t$,

2. If for every $\mathbf{y} \in \mathbb{F}^k$, we have $\Delta(\mathbf{v}, \mathbf{y}\mathbf{G}) > gt$ and $d(\mathbf{G}) > gt$ then $d(\mathbf{G}') > gt$.

Exercise 23.22. Prove Theorem 23.5.3 for the special case of the code being asymptotically good as well.

23.8 Bibliographic Notes

Theorem 23.1.4 is due to Berlekamp, McEliece and van Tilborg [19]. This work seems to be the first to apply the lens of computational complexity, and in particular NP-hardness, to coding theoretic problems. Theorem 23.1.3 showing the hardness of MaxCut is due to Garey, Johnson and Stockmeyer [56]. For more on classical NP-completeness results the reader is pointed to the text by Garey and Johnson [57]. Theorem 23.2.2 on the hardness of decoding after preprocessing is due to Bruck and Naor [23]. The hardness of approximating MaxCut, Theorem 23.3.2, is based on the PCP theorem due to Arora et al. [9, 8]. The hardness of approximating the distance of the nearest codeword, Theorem 23.3.4, is due to Stern [147]. The hardness of decoding up to the minimum distance, Theorem 23.4.2, is due to Dumer, Micciancio and Sudan [42]. We note that the literature includes many significant variations of this result. In particular Guruswami and Vardy [79] (see also Gandikotta, Ghazi, Grigorescu [55]) have shown that it is NP-hard to decode Generalized Reed-Solomon codes (recall Exercise 5.10). Since these are MDS codes, any decoding hardness is automatically a hardness for a 'Distance Bounded Decoding' problem. The NP-hardness of computing the minimum distance of a linear code is due to Vardy [159]. We stress that this is NP-hardness with a deterministic reduction unlike the results proven in this chapter. The NP-hardness, under randomized reductions, of approximating the minimum distance, Theorem 23.5.3, is from Dumer et al. [42].

Chapter 24

Giving Computational Complexity a helping hand

In this chapter we cover a few settings where coding theory turns out to be a useful tool in computational complexity, the field that studies the limits of algorithms, and in particular the inability of algorithms with limited resources to solve some computational tasks. A central theme in computational complexity is the comparison of resources. For example, if the time taken by an algorithm is viewed as one resource, and a second resource is access to random numbers, then what kind of effect does limiting the second resource have on the first? For several such questions, tools from coding theory lead to interesting answers (and not always in a predictable way). For instance in *communication complexity*, coding theory helps establish that randomness is a powerful resource. Whereas for other algorithmic tasks, coding theory helps establish that randomness is not so powerful. We will see some of these examples below.

24.1 Communication Complexity

Communication complexity studies a very basic task in distributed computing. Some data is partitioned among two entities, say Alice and Bob who wish to compute some function of the combined data, but they wish to minimize the communication between them while doing so. One simple, but communicationally expensive solution is for one of the players to send their entire share of the data to the other player. But such an expensive solution may not always be needed. For instance if the function to be computed is that average of some $2n$ numbers where Alice and Bob each have n of the numbers, then Alice only needs to send the sum of their numbers and this may be much smaller number of bits of communication than sending each of the numbers separately. So the communication complexity may depend on the task being performed (aka the function being computed). And as we will see shortly it also depends on whether Alice and Bob have access to randomness, either private or public. Below we model these problems formally and show how error-correcting codes are helpful (and inherently so) for giving an upper bound on the randomized communication complexity of a simple task.

24.1.1 (Deterministic) Communication Complexity

In what follows, we let $\mathcal{X}$ denote the space of inputs to Alice and $\mathcal{Y}$ denote the space of inputs to Bob. Our examples will use $\mathcal{X} = \mathcal{Y} = \{0, 1\}^n$ but we use the more elaborate notation for clarity. The ‘data’ is an element

$$(x, y) \in \mathcal{X} \times \mathcal{Y}$$

and we assume Alice knows x and Bob knows y . Their task is now modelled by a function

$$f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{O}.$$

Again we will use $\mathcal{O} = \{0, 1\}$ in this section, but the extra notation will hopefully help us explain the problem.

Communication proceeds in rounds, where in each round a designated player (either Alice or Bob) send a bit to the other player. The communicated bit should be a function of the knowledge of the communicating player—namely their initial input and bits already communicated in previous rounds. After a predetermined number of rounds of communication, one of the players (we will set this to be Bob) outputs the value of the function being computed. How many rounds of communication there are, who speaks in a given round, and what information the communication and output may depend on, are all modeled by a *protocol*, formally defined below.

Definition 24.1.1 (Deterministic protocol). A (deterministic) protocol Π is given by a tuple

$$(c; P_1, \dots, P_c; \pi_1, \dots, \pi_c; w)$$

where:

- c is a non-negative integer denoting the number of bits being communicated.
- $\{P_i \in \{\mathcal{X}, \mathcal{Y}\} \mid 1 \leq i \leq c\}$ determines who speaks in round i . That is, if $P_i = \mathcal{X}$ then Alice speaks and if $P_i = \mathcal{Y}$, Bob speaks.
- $\pi_i : P_i \times \{0, 1\}^{i-1} \rightarrow \{0, 1\}$ describes how the next bit to be communicated depends on the player's input and the past bits of communication.
- $w : \mathcal{Y} \times \{0, 1\}^c \rightarrow \mathcal{O}$ is Bob's output at the end of the communication.

Next, we define what it means for a protocol to ‘compute’ a function:

Definition 24.1.2 (Protocol computation). We say that a (deterministic) protocol Π computes a function f if for every $(x, y) \in \mathcal{X} \times \mathcal{Y}$ we have

$$f(x, y) = w(y, b_1, \dots, b_c)$$

where for every i we let

$$b_i = \pi_i(z_i, b_1, \dots, b_{i-1})$$

where in turn we have $z_i = x$ if $P_i = \mathcal{X}$ and $z_i = y$ otherwise.

In other words, if the player (corresponding to) P_i sends the bit b_i obtained by applying the function π_i to their input and the bits exchanged so far, then Bob's output equals the value of the function f .

Finally, we are ready to define the notion of *communication complexity*:

Definition 24.1.3 ((Deterministic) communication complexity). We define the *communication complexity of a protocol* $\Pi = (c, \dots)$, denoted $\text{CC}(\Pi)$ to be c . We define the *communication complexity of a function* f , denoted $\text{CC}(f)$, to be the minimum, over all protocols Π that compute f , of $\text{CC}(\Pi)$.

We make some simple observations about deterministic protocols in Exercise 24.1 (WLOG we can assume that Alice speaks last) and Exercise 24.2 (if $\mathcal{X} = \{0, 1\}^n$, then for any f we have $\text{CC}(f) \leq n$).

24.1.2 Randomized Communication Complexity

We now turn to *randomized* protocols. While the literature includes multiple versions of this setting, we only consider one, namely the case of shared randomness. Here we consider a setting where Alice and Bob share some element r chosen uniformly at random from some finite set Ω , and the bits that they communicate, as well as the output computed by Bob, can additionally depend on r . We say that Π computes f if for every x, y , we have that for a random choice of r the protocol computes $f(x, y)$ correctly. Formally, a randomized protocol is defined as follows.

Definition 24.1.4 (Randomized protocol). A randomized protocol Π is given by a tuple

$$(c; P_1, \dots, \pi_1, \dots; w; \Omega)$$

where:

- $c : \Omega \rightarrow \mathbb{Z}_+$, such that for each $r \in \Omega$, $c(r)$ denotes the number of bits being communicated with r being the random bits.
- $\{P_i \in \{\mathcal{X}, \mathcal{Y}\} \mid 1 \leq i \leq c(r)\}$ determines who speaks in round i for randomness choice $r \in \Omega$. (Againn, if $P_i = \mathcal{X}$ then Alice speaks and if $P_i = \mathcal{Y}$, Bob speaks.)
- $\pi_i : \Omega \times P_i \times \{0, 1\}^{i-1} \rightarrow \{0, 1\}$ describes how the next bit to be communicated depends on the randomness, the player's input and the past bits of communication.
- $w : \Omega \times \mathcal{Y} \times \{0, 1\}^c \rightarrow \mathcal{O}$ is Bob's output at the end of the communication.

We now have the obvious generalization of Definition 24.1.2 to randomized protocols:

Definition 24.1.5 (Randomized protocol computation). We say a randomized protocol Π computes a function f if for every $x \in \mathcal{X}, y \in \mathcal{Y}$, we have

$$\Pr_{r \in \Omega} [f(x, y) = w(r, y, b_1, \dots, b_{c(r)})] \geq \frac{2}{3},$$

where $b_i = \pi_i(r, z_i, b_1, \dots, b_{i-1})$ and z_i 's are as in Definition 24.1.2.

Finally, we can define the notion of randomized communication complexity.

Definition 24.1.6 ((Randomized) communication complexity). *The communication complexity of a randomized protocol $\Pi = (c; \dots; \Omega)$, denoted $\text{CC}^R(\Pi)$, is defined as before $\max_{r \in \Omega} c(r)$. The randomized communication complexity of a function f , denoted $\text{CC}^R(f)$, is the minimum, over all randomized protocols Π that compute f , of $\text{CC}^R(\Pi)$.*

We note that the choice of ‘success probability’ as $\frac{2}{3}$ in Definition 24.1.5 can be amplified to a larger constant incurring only a constant factor increase in the communication complexity of the function (see Exercise 24.3).

24.1.3 Separation between $\text{CC}(f)$ and $\text{CC}^R(f)$

A very basic question in communication complexity (raised and settled early in the study of this topic) is:

Question 24.1.1. *What is the largest separation between $\text{CC}(f)$ and $\text{CC}^R(f)$, for a function f ?*

This was settled by looking at a very simple function:

Definition 24.1.7 (Equality function). *For every $n \geq 1$, define*

$$\text{EQ}_n : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$$

satisfying for any $\mathbf{x}, \mathbf{y} \in \{0, 1\}^n$:

$$\text{EQ}_n(\mathbf{x}, \mathbf{y}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{y} \\ 0 & \text{otherwise} \end{cases}.$$

It is not too hard to show that $\text{CC}(\text{EQ}_n) = n$ for every positive integer n and we leave this as a (guided) exercise for the reader: see Exercise 24.4.

Theorem 24.1.8. *For every positive integer n , we have $\text{CC}(\text{EQ}_n) = n$.*

Turning to Question 24.1.1, or more precisely– what is $\text{CC}^R(\text{EQ}_n)$, we find the following tight connection between error correcting codes and randomized communication complexity. We restrict our attention to ‘one-way’ communication, defined below, as this already suffices to achieve the optimal randomized communication complexity for the Equality function.

Definition 24.1.9 (One-way communication). *A protocol Π is a one-way protocol if in every round Alice is the speaker (or $P_i = \mathcal{X}$ for every i). We let $\text{OW-CC}(f)$ and $\text{OW-CC}^R(f)$ denote the one-way communication complexities of a function f .*

Lemma 24.1.10. *Let $n \geq c \geq 1$ be integers. If there is a 2^c -ary code with 2^n codewords of relative distance $\frac{2}{3}$ then*

$$\text{OW-CC}^R(\text{EQ}_n) \leq c.$$

Conversely, if $\text{OW-CC}^R(\text{EQ}_n) \leq c$ then there is a 2^c -ary code of relative distance at least $\frac{1}{3}$.

We remark that while many proofs of the forward direction are known that do not seem to involve coding theory, the converse makes it clear that each of these results is in fact a coding-theory result, possibly in disguise.

Proof of Lemma 24.1.10. For the forward direction, let $E : \{0, 1\}^n \rightarrow \Sigma^N$ be the encoding function of an error-correcting code with 2^n codewords with $|\Sigma| = 2^c$. Furthermore let the relative distance of the code be $\frac{2}{3}$, i.e., for every $\mathbf{x} \neq \mathbf{y} \in \{0, 1\}^n$ we have

$$\Delta(E(\mathbf{x}), E(\mathbf{y})) \geq \frac{2}{3} \cdot N.$$

A shared randomness protocol for communication between Alice and Bob can be designed around E as follows:

1. Alice and Bob share a random variable i where i is distributed uniformly over $[N]$.
2. On input $\mathbf{x} \in \{0, 1\}^n$, Alice sends $E(\mathbf{x})_i$ to Bob.
3. Bob outputs 1 if $E(\mathbf{x})_i = E(\mathbf{y})_i$ and 0 otherwise.

It is clear that that protocol above involves c bits of one-way communication from Alice to Bob. It is also clear that if $\mathbf{x} = \mathbf{y}$ then Bob always outputs 1 = $\text{EQ}_n(\mathbf{x}, \mathbf{y})$. Now if $\mathbf{x} \neq \mathbf{y}$, then with probability at least $\frac{\Delta(E(\mathbf{x}), E(\mathbf{y}))}{N}$ over the choice of i we have $E(\mathbf{x})_i \neq E(\mathbf{y})_i$. (Indeed this immediately follows from the definition of $\Delta(\cdot, \cdot)$.) Thus, if $\mathbf{x} \neq \mathbf{y}$ then Bob outputs 0 = $\text{EQ}_n(\mathbf{x}, \mathbf{y})$ with probability at least $\frac{\Delta(E(\mathbf{x}), E(\mathbf{y}))}{N} \geq \frac{2}{3}$, thus establishing that the protocol above outputs the correct answer with probability $\frac{2}{3}$ in all cases.

For the reverse direction, let Π be a one-way communication protocol for EQ_n . Let

$$m : \{0, 1\}^n \times \Omega \rightarrow \{0, 1\}^c$$

be the function that determines Alice’s message to Bob. Specifically on input $\mathbf{x} \in \{0, 1\}^n$ and randomness $r \in \Omega$ let $m(\mathbf{x}, r)$ denote Alice’s message to Bob; and further let $w(\mathbf{y}, r, \sigma)$ denote Bob’s output on input $\mathbf{y}$, randomness r and message $\sigma \in \{0, 1\}^c$ from Alice. Our error correcting code will be given by the image of the encoding function $E : \{0, 1\}^n \rightarrow \Sigma^N$, where $\Sigma = \{0, 1\}^c$ and $N = |\Omega|$, with

$$E(\mathbf{x})_r = m(\mathbf{x}, r)$$

for $r \in \Omega$ (and we associate Ω with $[N] = \{1, 2, \dots, N\}$).

To see that

$$\Delta(E(\mathbf{x}), E(\mathbf{y})) \geq \frac{N}{3},$$

fix $\mathbf{x} \neq \mathbf{y}$ and consider two possible scenarios: the first where Alice gets $\mathbf{x}$ and Bob gets $\mathbf{y}$; and a second scenario where both Alice and Bob get $\mathbf{y}$ as their input. If r is such that $E(\mathbf{x})_r = E(\mathbf{y})_r$ then Bob outputs the same output in both scenarios. On the other hand we know that in the first scenario Bob outputs 1 with probability at most $\frac{1}{3}$ while in the second scenario he outputs one with probability at least $\frac{2}{3}$. Thus the probability that Bob has a different output in the two scenarios is at least $\frac{2}{3} - \frac{1}{3} = \frac{1}{3}$. We conclude that the probability over r that $E(\mathbf{x})_r \neq E(\mathbf{y})_r$ is at least $\frac{1}{3}$, or in other words $\frac{\delta(E(\mathbf{x}), E(\mathbf{y}))}{N} \geq \frac{1}{3}$, thus establishing that the image of E is a code of relative distance at least $\frac{1}{3}$.¹ $\square$

We note that a easy consequence of our proof above shows that Alice and Bob can solve EQ_n by sharing $O(\log n)$ bits— see Exercise 24.5. Next, we note that the proof of the forward direction of Lemma 24.1.10 is generally proved using hash functions. As we noted in Proposition 20.3.2 (almost universal) hash function and codes are equivalent, which explains why our proof is stated in terms of codes.

24.2 Derandomization and Pseudorandomness

One of the richest connections between computational complexity and error-correcting codes occurs in the area of *pseudorandomness*. This is a broad area, which includes t -wise independence (see Exercise 5.13) and ε -bias (see Exercise 2.15) as special cases. In this section we see the broader context from which these examples arise, while also seeing the impact of these concepts on the complexity of some simple tasks.

To motivate this broader context, note that randomness is considered a valuable resource for computation: For many tasks, the fastest and simplest algorithms that are known are randomized. And while these are desirable features, deterministic algorithms have the advantage of repeatability — two runs of a deterministic algorithm on the same input will always produce the same output. The broad goal of the field of ‘derandomization’ is to convert randomized algorithms into deterministic ones, possibly with a small loss in the running time. (In particular we may wish to preserve polynomial running times.)

For us, the term ‘pseudorandomness’ refers to a specific approach to ‘derandomization’. The central object of attention here is a *pseudorandom generator*, a deterministic (and hopefully efficiently computable) function $G: \{0, 1\}^s \rightarrow \{0, 1\}^m$ where $s \ll m$, which takes s bits of randomness and outputs m pseudorandom bits. The idea of pseudorandomness is to use the output of a pseudorandom generator instead of m genuinely random (independent, unbiased) bits as inputs to a randomized algorithm, and hope that the pseudorandom bits fool the algorithm.

We now turn to formalizing the notion of *fooling* being alluded to above. Recall that a randomized algorithm A running on input x can be viewed as a deterministic running on two inputs, the actual input x and a random string y . Recall further (see Appendix C) that a randomized algorithm A is said to compute a function f correctly if $\Pr_y[A(x, y) = f(x)] \geq 2/3$, where

¹If the protocol has *one-sided error*, and always outputs 1 when $\mathbf{x} = \mathbf{y}$, then one can conclude that the encoding map E defines a code with relative distance at least $\frac{2}{3}$.

$A(x, y)$ denotes the output of algorithm A on input x and randomness y . We are now ready to define pseudorandomly fooling the algorithm A .

Definition 24.2.1. Given an algorithm A , input x and parameter $\varepsilon \in [0, 1]$, we say that a generator G ε -fools the pair (A, x) if

$$\left| \Pr_{y \in \{0, 1\}^m} [A(x, y) = f(x)] - \Pr_{z \in \{0, 1\}^s} [A(G(z)) = f(x)] \right| \leq \varepsilon.$$

In other words the probability with which A outputs the correct answer with pseudorandom strings is ε -close to the probability it outputs the correct answer with pure randomness. And since the correct answer is reported with probability at least $2/3$ with pure randomness, if $\varepsilon < 1/6$ then even with pseudorandom strings, the correct answer is output with probability strictly greater than $1/2$. And to revert to ‘derandomization’: note that the probability with which $A(G(z))$ computes any value, can be computed deterministically by 2^s runs of A (just run A with every deterministic choice of $z \in \{0, 1\}^s$). In particular if A is a polynomial time algorithm and $s = O(\log m)$ then running A a total of $2^{O(\log m)}$ times only takes polynomial time.

The above discussion thus motivates pseudorandomness and the challenge of finding generators that fool our algorithm of choice. Now the most ambitious studies try to find generators that fool all polynomial time algorithms, but this study is out of scope for this chapter. In this chapter we will focus on generators that fool some very simple algorithms. We will use one algorithm as our example, though the approaches to fooling this algorithm will lead us to fairly general pseudorandom generators.

24.2.1 Max t -SAT and a randomized algorithm

The principal problem we use to capture the effect of pseudorandomness is MAX t -SAT, a problem involving finding a good assignment to n Boolean variables $x_1, \dots, x_n$. Recall that a Boolean *variable* is one that can be assigned one of two values 0 (representing false) and 1 (representing true). A *literal* is either a variable or its negation (where the negation of a variable x , denoted $\neg x$ is true if and only if x is false). For positive integer t , a t -*clause* is the disjunction (logical OR) of t distinct literals, where the disjunction of t literals $\ell_1, \dots, \ell_t$, denoted $\ell_1 \vee \dots \vee \ell_t$, is true if and only if at least one of the literals ℓ_i is true. Finally an assignment $x_1 \leftarrow a_1, \dots, x_n \leftarrow a_n$ where $a_1, \dots, a_n \in \{0, 1\}$ is said to satisfy a clause $C = \ell_1 \vee \dots \vee \ell_t$ if at least one of the literals is true (i.e., assigned the value 1 under the assignment). Given a collection $\Phi = (C_1, \dots, C_m)$ of m t -clauses over variables $x_1, \dots, x_n$, and an assignment $\mathbf{a} \in \{0, 1\}^n$ to the variables $x_1, \dots, x_n$, we let $\text{val}(\Phi, \mathbf{a})$ denote the fraction of clauses of Φ satisfied by $\mathbf{a}$, and we let

$$\text{opt}(\Phi) = \max_{\mathbf{a}} \{\text{val}(\Phi, \mathbf{a})\}.$$

The goal of the MAX t -SAT problem is to determine, given such a collection Φ , an assignment $\mathbf{a}$ that maximizes $\text{val}(\Phi, \mathbf{a})$.

Problem 24.2.2. (Max t -SAT)

GIVEN: m t -clauses $C_1, \dots, C_m$ over n variables $x_1, \dots, x_n$.

OUTPUT: An assignment $\mathbf{a} \in \{0, 1\}^n$ such that $\text{val}(\Phi, \mathbf{a}) = \text{opt}(\Phi)$.

MAX t -SAT is known to be NP-hard for all $t \geq 2$ (see Exercise 24.6). So we often look for approximation algorithms, i.e., a polynomial time algorithm that finds an assignment satisfying a large number of clauses though not necessarily the assignment that maximizes the number of satisfied clauses. Formally,

Definition 24.2.3 (Approximation Algorithm). *For $\alpha \in [0, 1]$ an algorithm A that takes as input a MAX t -SAT input Φ and outputs a value $A(\Phi) \in \mathbb{R}$ is said to be an α -approximation algorithm if for all Φ we have*

$$\alpha \cdot \text{opt}(\Phi) \leq A(\Phi) \leq \text{opt}(\Phi).$$

In other words the algorithm outputs a number close to $\text{opt}(\Phi)$ and this approximation gets better as $\alpha \rightarrow 1$. As we will see next, it is easy to find non-trivial α -approximation algorithms with polynomial running time. The following gives the simplest such algorithm, though it is randomized.

Algorithm 24.2.1 RANDOM t -SAT

INPUT: t -SAT formula Φ on n variables

OUTPUT: A rational number in $[0, 1]$

- 1: Pick $\mathbf{a} \in \{0, 1\}^n$ uniformly at random
 - 2: RETURN $\text{val}(\Phi, \mathbf{a})$
-

In other words, the algorithm simply ignores its input and picks a random assignment and outputs the number of clauses satisfied by this random assignment. And yet this turns out to be a non-trivial approximation algorithm, or at least leads to one. We start to analyze this next. First note that the number of clauses of Φ satisfied by a random assignment is a random variable and so the output of RANDOM t -SAT is a random variable that can vary between 0 and m . What we see below is that the expectation of this random variable is at least

$$(1 - 2^{-t}) \cdot 1 \geq (1 - 2^{-t}) \cdot \text{opt}(\Phi),$$

since clearly $\text{opt}(\Phi) \leq 1$.

Lemma 24.2.4. *For every input Φ to MAX t -SAT with m t -clauses, we have*

$$\mathbb{E}[\text{RANDOM } t\text{-SAT}(\Phi)] \geq 1 - 2^{-t}$$

Proof. Let $\mathbf{a}$ denote the assignment chosen by RANDOM t -SAT(Φ) and let Z_i denote the indicator variable that is 1 if the i -th clause C_i is satisfied by $\mathbf{a}$. Then we have $\mathbb{E}[Z_i] \geq 1 - 2^{-t}$ since the only way that $C_i = \ell_1 \vee \dots \vee \ell_t$ is not satisfied is if $\ell_1 = \dots = \ell_t = 0$, an event that happens with probability most 2^{-t} . Specifically if two of the literals are complementary (i.e., are of the form x_j and $\neg x_j$) then the clause is always satisfied, else the literals use distinct variables and in which case the probability that $\ell_1 = \dots = \ell_t = 0$ is exactly 2^{-t} . (Note that we are crucially using the fact that a t -clause has t distinct literals.) We now note that

$$\text{val}(\Phi, \mathbf{a}) = \frac{1}{m} \sum_{i=1}^m Z_i$$

and use the linearity of expectations to conclude that

$$\mathbb{E}[\text{val}(\Phi, \mathbf{a})] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}[Z_i] \geq 1 - 2^{-t},$$

as desired. □

Unfortunately the bound above is only on the expected value of the output of the algorithm and so does not immediately lead to a $1 - 2^{-t}$ -approximation algorithm. It turns out that we can replace the expectation bound with a high probability bound with a slight loss in approximation factor—more precisely there exists a randomized $(1 - 2^{-t} - \epsilon)$ -approximation algorithm that runs in time $O(m/\epsilon)$ for any $\epsilon > 0$ (see Exercise 24.7). This leads to the following natural question:

Question 24.2.1. *Does there exist a deterministic polynomial time (ideally close to linear runtime) $(1 - 2^{-t})$ -approximation algorithm for the MAX t -SAT problem?*

Note that in the above we want to get rid of the randomness as well as the loss of ϵ in the approximation ratio. We answer this question in the next couple of sections.

24.2.2 Limited independence

To convert our algorithm into a deterministic one, we use the concept of limited independence. We already saw this concept in Exercise 2.14 but now we define this concept as a special case of pseudo-randomness.

Given a generator $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ and set $T \subseteq [n]$ let $G|_T(\cdot)$ denote the projection of $G(\cdot)$ to the coordinates of T .

Definition 24.2.5. *We say that a generator $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ is a t -wise independent generator if it satisfies the following condition: For every subset $T \subseteq [n]$ with $|T| = t$, we have $G|_T(z)$ is uniformly distributed over $\{0, 1\}^t$ if z is distributed uniformly over $\{0, 1\}^s$.*

Exercises 2.14 and 5.14 imply the following result:

Lemma 24.2.6. 1. *Let C be a linear $[n, k, d]_2$ code with encoding function $E : \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ with the property that its dual $C^\perp$ is a code of distance at least $t + 1$. Then E is a t -wise independent generator.*

2. *For infinitely many n and t there is a linear t -wise independent generator $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ with seed length $s \leq (\frac{n}{t}) \log n$.*

One can show that the bound on the seed length in the second item above is essentially tight (even without the condition of linear generator)— see Exercise 24.8.

We now turn to applying this notion of pseudorandomness to ‘fooling’ algorithms. Specifically we apply t -wise independent generators to our algorithm RANDOM t -SAT. Unfortunately, for this application, we will resort to a weaker definition of fooling than the one above. Roughly, our definition (Definition 24.2.7) expects that the distribution of outputs of a randomized algorithm is nearly preserved when uniform randomness is replaced by the output of a pseudo-random generator. The following definition weakens this requirement and only says that the expected output is nearly preserved. As we will see even this weak definition turns out to be quite useful.

Definition 24.2.7. *Given an algorithm A whose output is in the range $[0, 1]$, input $\mathbf{x}$ and parameter $\varepsilon \in [0, 1]$, we say that a generator G ε -weakly fools the pair $(A, \mathbf{x})$ if*

$$\left| \mathbb{E}_{\mathbf{y} \in \{0,1\}^m} [A(\mathbf{x}, \mathbf{y})] - \mathbb{E}_{\mathbf{z} \in \{0,1\}^s} [A(\mathbf{x}, G(\mathbf{z}))] \right| \leq \varepsilon.$$

Our next lemma shows that the RANDOM t -SAT is weakly fooled by a t -wise independent generator.

Lemma 24.2.8. *Let $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ be a t -wise independent generator. Then G 0-weakly fools RANDOM t -SAT.*

Proof. We simply go through the proof of Lemma 24.2.4. We note that for a fixed t -clause C , the probability that it is satisfied by a uniformly random assignment a is the same as the probability that it is satisfied by the output of G , since the satisfaction only depends on the t variables participating in C and G is t -wise independent. Thus the expected output of RANDOM t -SAT on input Φ , whether its randomness is uniform or the output of the generator G , is simply the probability that a single clause is satisfied by a uniform assignment. It follows that RANDOM t -SAT is 0-weakly fooled by G . $\square$

Corollary 24.2.9. *There is a deterministic $1 - 2^{-t}$ -approximation algorithm for MAX t -SAT with running time $O_t(n^{\frac{t}{2}+1})$ time on inputs of length n .*

Proof. Let Φ be a formula of length n , and so it has at most n variables and n clauses. Our deterministic algorithm on input Φ acts as follows: (1) Picks a t -wise independent generator $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ for $s \leq t/2 \lceil \log_2(n+1) \rceil$ (as guaranteed by Lemma 24.2.6). (2) Runs RANDOM t -SAT on every random input in image of G , thus running it on 2^s random assignments. (3) Output the maximum number of clauses satisfied by these 2^s random assignments. The formal algorithm is present in Algorithm 24.2.2.

We now analyze the performance of Algorithm 24.2.2. Note that Step 4 takes time $O(n)$ since one can compute the number of clauses satisfied by a fixed assignment in the same time. Running it on $2^s = n^{\frac{t}{2}}$ assignments takes time $O(n^{1+t/2})$. Since the expected value of the output of RANDOM t -SAT on random output of G is $(1 - 2^{-t})$, by Lemma 24.2.8, it follows that the maximum v at the end is also at least $(1 - 2^{-t}) \geq (1 - 2^{-t}) \text{opt}(\Phi)$ and so this algorithm is a $1 - 2^{-t}$ -approximation algorithm. $\square$

Algorithm 24.2.2 De-randomized RANDOM t -SAT

INPUT: t – SAT formula Φ on n variables

OUTPUT: A rational number in $[0, 1]$

```

1:  $s \leftarrow \left(\frac{t}{2}\right) \log n$ 
2: Pick  $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$  to be a  $t$ -wise independent generator
3:  $v \leftarrow 0$ 
4: FOR  $\mathbf{z} \in \{0, 1\}^s$  DO
5:    $v \leftarrow \max(v, \text{val}(\Phi, G(\mathbf{z})))$ 
6: RETURN  $v$ 
```

The approximation factor given by the algorithm above turns out to be best possible for $t \geq 3$ for a polynomial time algorithm, assuming $\text{NP} \neq \text{P}$. But the running time is not the best possible and we improve it in the next section.

24.2.3 ε -bias and almost limited independence

We now turn to an alternate notion of pseudorandomness that at first glance seems to be unrelated to the notion of t -wise independence, but which we will later show is also closely related. The notion we now introduce is that of fooling ‘linear’ functions. Recall that a function $\ell : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ is linear if there exists coefficient $\alpha_1, \dots, \alpha_n \in \mathbb{F}_2$ such that $\ell(y_1, \dots, y_n) = \sum_{i=1}^n \alpha_i y_i$, where all arithmetic is in the field $\mathbb{F}_2$.

Definition 24.2.10. *We say that a generator $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ is ε -biased if for non-zero every linear function $\ell : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ we have*

$$\left| \Pr_{z \in \mathbb{F}_2^s} [\ell(G(z)) = 1] - 1/2 \right| \leq \varepsilon.$$

Note that over the uniform distribution, the probability that $\ell(\cdot)$ takes on the value 1 is exactly $\frac{1}{2}$ for linear functions ℓ (recall Lemma 3.1.11). So the condition above is requiring that ℓ is preserving its output distribution to within ε when its inputs are pseudorandom (i.e., the output of G) as compared to the case when they are uniformly random. In other words G ε -fools every linear function.

We now recall the notion of ‘balanced’ codes (mentioned in Exercise 2.15) and show how they lead to ε -biased generators quite immediately.

Definition 24.2.11. *We say that a linear code $C \subseteq \mathbb{F}_2^N$ is ε -balanced if for every pair of distinct codewords $\mathbf{x}, \mathbf{y} \in C$ we have*

$$\left(\frac{1}{2} - \varepsilon\right)N \leq \Delta(\mathbf{x}, \mathbf{y}) \leq \left(\frac{1}{2} + \varepsilon\right)N.$$

We now state the simple equivalence between ε -biased generators and ε -balanced codes (see Exercise 24.9).

Proposition 24.2.12. 1. Let $A \in \mathbb{F}_2^{K \times N}$ be the generator matrix of an ε -balanced code. Let $A_i \in \mathbb{F}_2^K$ denote the i th column of A . Then the function $G : [N] \rightarrow \mathbb{F}_2^K$ given by $G(i) = A_i$ is an ε -biased generator.

2. Let $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ be an ε -biased generator. Let $A : \mathbb{F}_2^{n \times 2^s}$ be the matrix whose $\mathbf{z}$ th column (for $\mathbf{s} \in \{0, 1\}^s$) is given by $A_{\mathbf{z}} = G(\mathbf{z})$. Then A generates an ε -balanced code.

As a corollary, using the best known constructions of binary linear error-correcting codes, we get the following:

Lemma 24.2.13. For every $\varepsilon > 0$ and every sufficiently large n , there exists a ε -biased generator $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$, where $s = \log(n/\varepsilon^2) + O(1)$. Furthermore if we allow $s = \log(n^2/\varepsilon^2) + O(1)$, or $s = \log(n/\varepsilon^3) + O(1)$ then this generator runs in time $\text{poly}(n/\varepsilon)$.

While fooling linear functions may not sound all that exciting, it turns out that ε -bias gives us a lot more. In particular, if ε is exponentially small in n , then the output distribution is almost uniform, and even when ε is larger, the output ‘almost’ fools small tests. In turn this gives a way to speed up the MAX t -SAT approximation algorithm from the previous section, for a tiny loss in the approximation guarantee.

We start by showing that the output distribution is close to uniform if ε is really small. As a quick aside we recall the following definition:

Definition 24.2.14. The variation distance between two distributions $\mathcal{D}_1, \mathcal{D}_2$ with the same support set S is given by

$$\frac{1}{2} \sum_{x \in S} \left| \Pr_{X \sim \mathcal{D}_1} [X = x] - \Pr_{Y \sim \mathcal{D}_2} [Y = x] \right|.$$

We present some implications of the total variational distance in Exercise 24.10.

Lemma 24.2.15. For $\varepsilon \geq 0$ if $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ is an ε -biased generator, then the output of G is $2^n \cdot \varepsilon$ close to uniform in total variation distance.

Proof. We prove an even stronger statement. Specifically for every $\mathbf{x} \in \mathbb{F}_2^n$ we prove that $|\Pr_{\mathbf{z}}[G(\mathbf{z}) = \mathbf{x}] - 2^{-n}| \leq 2\varepsilon$. The lemma follows from the fact that the total variation distance of the distribution $G(\mathbf{z})$ from the uniform distribution over $\mathbb{F}_2^n$ equals $\frac{1}{2} \sum_{\mathbf{x}} |\Pr_{\mathbf{z}}[G(\mathbf{z}) = \mathbf{x}] - 2^{-n}| \leq 2^n \varepsilon$.

Fix $\mathbf{x} \in \mathbb{F}_2^n$. Let $\mathbb{1}_{\mathbf{x}}(\mathbf{y})$ denote the function that is 1 when $\mathbf{y} = \mathbf{x}$ and 0 otherwise and note that our goal is to get a close estimate of $\mathbb{E}_{\mathbf{z}}[\mathbb{1}_{\mathbf{x}}(G(\mathbf{z}))]$. For $\alpha \in \mathbb{F}_2^n$, let $L_{\alpha}(\mathbf{y}) = \sum_{i=1}^n \alpha_i y_i$ and let $\chi_{\alpha}(\mathbf{y}) = (-1)^{L_{\alpha}(\mathbf{y})}$. Note that χ_{α} is a real-valued function. On the one hand we have, by the ε -biased property of G , that $|\mathbb{E}_{\mathbf{z}}[\chi_{\alpha}(G(\mathbf{z}))]| \leq 2\varepsilon$, for every $\alpha \in \mathbb{F}_2^n - \{0\}$. Now note that we can write

$$\mathbb{1}_{\mathbf{x}}(\mathbf{y}) = \frac{1}{2^n} \left(\sum_{\alpha \in \mathbb{F}_2^n : \chi_{\alpha}(\mathbf{x})=1} \chi_{\alpha}(\mathbf{y}) - \sum_{\alpha \in \mathbb{F}_2^n : \chi_{\alpha}(\mathbf{x})=-1} \chi_{\alpha}(\mathbf{y}) \right).$$

We can now get our estimate on $\mathbb{E}_{\mathbf{z}}[\mathbb{1}_{\mathbf{x}}(G(\mathbf{z}))]$ as follows. We have $\mathbb{E}_{\mathbf{z}}[\chi_0(G(\mathbf{z}))] = 1$ since χ_0 is the constant function. Thus

$$\begin{aligned} \mathbb{E}_{\mathbf{z}}[|\mathbb{1}_{\mathbf{x}}(G(\mathbf{z})) - 2^{-n}|] &= \mathbb{E}_{\mathbf{z}} \left[\left| \frac{1}{2^n} \left(\sum_{\alpha \in \mathbb{F}_2^n - \{0\} : \chi_{\alpha}(\mathbf{x})=1} \chi_{\alpha}(G(\mathbf{z})) - \sum_{\alpha \in \mathbb{F}_2^n : \chi_{\alpha}(\mathbf{x})=-1} \chi_{\alpha}(G(\mathbf{z})) \right) \right| \right] \\ &\leq \frac{1}{2^n} \sum_{\alpha \in \mathbb{F}_2^n - \{0\}} |\chi_{\alpha}(G(\mathbf{z}))| \\ &\leq \frac{2^n - 1}{2^n} (2\varepsilon) \\ &\leq 2\varepsilon. \end{aligned}$$

□

We remark that the bound above is not tight. It is known that ε -biased distributions are $\varepsilon \cdot 2^{n/2}$ -close to uniform, but proving that requires some additional ingredients from Discrete Fourier Analysis. Exercise 24.13 shows how to get this improvement.

We now define the notion of ‘almost-limited independence’.

Definition 24.2.16. For positive integer t and real $\delta \geq 0$ we say that a generator $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ is a δ -almost t -wise independent generator if it satisfies the following condition: For every subset $T \subseteq [n]$ with $|T| = t$, we have that $G|_T(z)$ is δ -close (in total variation distance) to being uniformly distributed over $\{0, 1\}^t$ when z is distributed uniformly over $\{0, 1\}^s$.

The following lemma connects small-biased generators to almost-limited-independence. The key here is to note that the projection of the output of an ε -biased generator to any subset of t output bits retains the ε -bias property. The lemma then follows immediately from Lemma 24.2.15.

Lemma 24.2.17. For every positive integer t and $\varepsilon \geq 0$ if $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ is an ε -biased generator, then G is also a δ -almost t -wise independent generator, for $\delta = \varepsilon \cdot 2^t$.

Proof. Let $G|_T^s : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^t$ be an ε -biased generator and let $T \subset [n]$ be a set of cardinality $|T| = t$. Let $G|_T : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^t$ be the projection of the output of G to the coordinates of T , i.e.,

$$G|_T(z) = (x_{i_1}, \dots, x_{i_t})$$

where $G(z) = (x_1, \dots, x_n)$ and $T = \{i_1, \dots, i_t\}$.

It follows immediately from the definition of ε -bias that $G|_T$ is also ε -biased. (G fools all linear functions, and in particular linear functions supported on T also.) Thus, by applying Lemma 24.2.15 to $G|_T$, we get that the output of G , restricted to the set T , is $\varepsilon \cdot 2^t$ close in total variation distance to the uniform distribution on $\{0, 1\}^t$.

Since this holds for every subset $T \subseteq [n]$ of cardinality t , it follows that G is $\varepsilon 2^t$ -almost- t -wise independent. □

We now show how this leads to an improved algorithms for MAX- t -SAT.

The following lemma is quite similar to Lemma 24.2.8 and we defer the proof to Exercise 24.11.

Lemma 24.2.18. *Let $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ be a δ -almost t -wise independent generator. Then G (2δ)-weakly fools RANDOM t -SAT.*

Using the lemma above we easily get the following application to MAX t -SAT.

Corollary 24.2.19. *There is a deterministic $1 - 2^{-t}$ -approximation algorithm for MAX t -SAT with running time $O(4^t \cdot n^5)$ time on inputs of length n .*

Proof. We prove this by setting parameters appropriately. Let ϕ be a t -CNF formula of length n . In particular note that ϕ has at most n variables and $m \leq n$ clauses. We set

$$\delta = \frac{1}{2^{t+2}m}$$

and let

$$\varepsilon = 2^{-t}\delta.$$

Let $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ be a ε -biased distribution, and hence (by Lemma 24.2.17) G is δ -almost t -wise independent. It follows from Lemma 24.2.18 that G 2δ -weakly fools RANDOM t -SAT. In other words

$$\mathbb{E}_Z[\text{val}(\Phi, G(\mathbf{z}))] \geq 1 - 2^{-t} - 2\delta.$$

In particular, there exists a $\mathbf{z}$ such that

$$\text{val}(\Phi, G(\mathbf{z})) \geq 1 - 2^{-t} - 2\delta = 1 - 2^{-t} - 2^{-(t+1)}/m.$$

But $\text{val}(\phi, \mathbf{x})$ is an integral multiple of $1/m$ and so if

$$\text{val}(\Phi, G(\mathbf{z})) \geq 1 - 2^{-t} - 2^{-(t+1)}/m,$$

then

$$\text{val}(\Phi, G(\mathbf{z})) \geq 1 - 2^{-t}.$$

Thus, we only need to search over a space of size 2^s to find such a $\mathbf{z}$ and by Lemma 24.2.13 we can take $s = \log(n^2/\varepsilon^2) + O(1)$ so that $2^s = O(n^2/\varepsilon^2) = O(4^t n^4)$. Computing $\text{val}(\phi, \mathbf{x})$ for any given $\mathbf{x}$ takes $O(n)$ time, leading to the claimed running time. $\square$

The result above is a significant improvement on the previous bound in Lemma 24.2.9, but we are not done yet. It turns out one can combine the notions of ε -bias and t -wise independence to get an even better construction of almost- t -wise independent distributions leading us to our final result.

Lemma 24.2.20. *Let $G_1 : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^\ell$ be an ε -biased generator. Let $G_2 : \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2^n$ be a linear t -wise independent generator. Then $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$ given by $G(\mathbf{z}) = G_2(G_1(\mathbf{z}))$ is an $(\varepsilon \cdot 2^t)$ -almost t -wise independent generator.*

Proof. Fix a set $T \subseteq [n]$ with $|T| = t$ and let $G|_T$ and $G_2|_T$ denote the projection of G and G_2 (resp.) to the coordinates in T , so that $G|_T(\mathbf{z}) = G_2|_T(G_1(\mathbf{z}))$. We claim that $G|_T$ is ε -biased and so by Lemma 24.2.17 its output is $\varepsilon 2^t$ close to the uniform distribution. Since this holds for every $T \subseteq [n]$ with $|T| = t$, we get that G is an $(\varepsilon \cdot 2^t)$ -almost t -wise independent generator.

We now prove the claim above. Let $L : \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2$ be a non-zero linear function. Since $G_2 : \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2^n$ is also linear, we have that $G_2|_T$ is also a linear function and so

$$L' = L \circ G_2|_T : \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2$$

is also a linear function. We claim that L' is not the zero-function. Once we show this we are done, since by the ε -bias of G_1 we have:

$$\left| \Pr_{\mathbf{z} \in \mathbb{F}_2^s} [L(G|_T(\mathbf{z})) = 1] - 1/2 \right| = \left| \Pr_{\mathbf{z} \in \mathbb{F}_2^s} [L'(G_1(\mathbf{z})) = 1] - 1/2 \right| \leq \varepsilon.$$

We thus turn to showing L' is non-zero. Since L is non-zero, there must be an $\mathbf{a} \in \mathbb{F}_2^t$ such that $L(\mathbf{a}) \neq 0$. Furthermore since G_2 is t -wise independent, we must have $G_2|_T(\mathbf{y})$ is uniformly distributed for a uniformly random $\mathbf{y} \in \mathbb{F}_2^\ell$ and in particular, there must exist a $\mathbf{b} \in \mathbb{F}_2^\ell$ such that $G_2|_T(\mathbf{b}) = \mathbf{a}$. But then we have

$$L'(\mathbf{b}) = L(G_2|_T(\mathbf{b})) = L(\mathbf{a}) \neq 0.$$

This concludes the proof. $\square$

One natural question to ask is whether the G as constructed above is also an ε -biased source: see Exercise 24.14.

Lemma 24.2.21. *For every $\delta > 0$, integer t and every sufficiently large n , there exists a polynomial time computable δ -almost t -wise independent generator $G : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^n$, where*

$$s = 2(t + \log 1/\delta + \log \log n) + O(1).$$

Proof. We let

$$\ell = \frac{t}{2} \cdot \log n,$$

and let $G_2 : \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2^n$ be the map given by Lemma 24.2.6, so G_2 is t -wise independent and linear. We next set

$$\varepsilon = 2^{-t}\delta$$

and

$$s = \log(\ell^2/\varepsilon^2) = 2t + \log(\ell^2/\delta^2) + O(1) = 2t + \log(1/\delta^2) + 2\log \log n$$

and let $G_1 : \mathbb{F}_2^s \rightarrow \mathbb{F}_2^\ell$ be an ε -biased map given by Lemma 24.2.13. Letting $G = G_2 \circ G_1$ and applying Lemma 24.2.20 yields our lemma. $\square$

As with Corollaries 24.2.9 and 24.2.19, we get the following corollary with the above construction (see Exercise 24.15).

Corollary 24.2.22. *For every $\delta > 0$, there is a deterministic $1 - 2^{-t} - \delta$ -approximation algorithm for MAX t -SAT with running time $O(4^t/\delta^2 \cdot n(\log n)^2)$ time on inputs of length n .*

24.3 Hardcore Predicates

We now turn to some applications of coding theory to the foundations of cryptography. In this section we consider the task of constructing a ‘hardcore predicate’ for a ‘one-way permutation’. We start by defining these two notions. We start with the second notion which is a very basic and ubiquitous notion in cryptography.

Recall that $\{0, 1\}^* = \cup_{n \in \mathbb{Z}_{\geq 0}} \{0, 1\}^n$.

Definition 24.3.1. We say that $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a permutation if

- (1) f is length-preserving, i.e., for every n and $\mathbf{x} \in \{0, 1\}^n$ we have $f(\mathbf{x}) \in \{0, 1\}^n$ and
- (2) f is invertible, i.e., there exists a function $f^{-1} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that for every $\mathbf{x} \in \{0, 1\}^*$ we have $f^{-1}(f(\mathbf{x})) = \mathbf{x}$.

The essence of cryptography are functions that are ‘very hard’ to compute. To this end, we define the following notions of a function being hard:

Definition 24.3.2. (1) A function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ to be easy if there exists a polynomial time algorithm A that computes f , i.e., for every $\mathbf{x} \in \{0, 1\}^*$, $A(\mathbf{x}) = f(\mathbf{x})$.

- (2) A function f is said to be (merely) hard if it is not easy.
‘Very hard’-ness is a quantified strengthening of the last notion above:

- (3) For a function $\alpha : \mathbb{Z}_{\geq 0} \rightarrow [0, 1]$, we say that f is α -hard if for every polynomial time algorithm A and for every sufficiently large $n \in \mathbb{Z}_{\geq 0}$ we have

$$\Pr_{\mathbf{x} \in \{0, 1\}^n} [A(\mathbf{x}) = f(\mathbf{x})] \leq \alpha(n).$$

- (4) Finally we say that f is very hard if for every constant $c \geq 0$ we have f is $\frac{1}{n^c}$ -hard.

We are finally ready to define one-way permutations:

Definition 24.3.3. A permutation $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is said to be a one-way permutation if (1) f is easy and (2) f^{-1} is very hard.

In what follows we will switch from functions on arbitrarily long inputs to functions on finite length inputs by fixing an input length, denoted k , for f (or b). When we talk about ‘polynomial time’ computability, it will be good to keep in mind that f is defined on all inputs and the algorithm must run in polynomial time for all input lengths.

While a one-way permutation is in itself a very powerful object, if a one-way function comes augmented with a hardcore bit, or ‘hardcore predicate’, it becomes even more useful. We explain this notion next.

Definition 24.3.4. For $\varepsilon > 0$, we say that a Boolean function (or predicate) $b : \{0, 1\}^k \rightarrow \{0, 1\}$ is said to be an ε -hardcore predicate for a permutation $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$ if

- (1) b is easy and

- (2) $b \circ f^{-1}$ is $(1/2 + \varepsilon)$ -hard.

We say that b is a hardcore predicate for f if b is k^{-c} -hardcore for f for every $c \geq 0$.

In other words $b(\mathbf{x})$ is easy to compute given $\mathbf{x}$, but very hard to compute given $f(\mathbf{x})$. Note that if b is a hardcore predicate for f , then f^{-1} must be very hard (see Exercise 24.16).

More importantly b captures the essence of f^{-1} being very hard, in that it gives a ‘single bit’ of information about $\mathbf{x}$, namely $b(\mathbf{x})$, that is very hard to guess (better than random) given $f(\mathbf{x})$. Exercise 24.17 illustrates the utility of one-way permutations in building very general pseudo-random generators.

If the function f is crafted carefully, then some very natural predicates turn out to be hardcore. (For the reader familiar with the RSA function we mention that there a hardcore predicate is $b(\mathbf{x}) = x_1$, the most significant bit of the input.) However for such simple predicates b it is also possible to construct one way functions f for which b is not a hardcore predicate. (See Exercise 24.18.)

In what follows we will consider one-way functions with some ‘padding’.

Definition 24.3.5. We say that a function $F : \{0, 1\}^{k+\ell} \rightarrow \{0, 1\}^{k+\ell}$ is an ℓ -padding (or simply padding) of $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$, if for every $\mathbf{x} \in \{0, 1\}^k$ and $\mathbf{i} \in \{0, 1\}^\ell$ we have $F(\mathbf{x}, \mathbf{i}) = (f(\mathbf{x}), \mathbf{i})$.

Exercise 24.18 shows how paddings of one-way permutations pose obstacles to some natural candidates for hardcore predicates.

In view of this difficulty of constructing ‘generic hardcore predicates’, (i.e., ones that work for every one-way function) the next theorem will likely come as a pleasant surprise: it says that for every one-way function, there is a padded version that admits a simple hardcore predicate. The key ingredient in our construction is an efficiently list-decodable binary code. We recall the notion below.

The following is a simple (algorithmic) generalization of Definition 7.2.1:

Definition 24.3.6. A function $E : \{0, 1\}^k \rightarrow \{0, 1\}^n$ is (ρ, L) -efficiently-list decodable if there is an efficient algorithm L such that

- (1) For every $\mathbf{y} \in \{0, 1\}^n$, $D(\mathbf{y}) \subseteq \{0, 1\}^k$ satisfies $|D(\mathbf{y})| \leq L$ and
- (2) For every $\mathbf{x} \in \{0, 1\}^k$ such that $\Delta(E(\mathbf{x}), \mathbf{y}) \leq \rho n$, we have $\mathbf{x} \in D(\mathbf{y})$.

In what follows we will show that a random bit of the encoding of $\mathbf{x}$, under a nicely list decodable code E , is a hardcore predicate for every padded one-way function.

Lemma 24.3.7. Let $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$ be a one-way permutation and let $E : \{0, 1\}^k \rightarrow \{0, 1\}^n$ be a $(1/2 - \varepsilon, L)$ -efficiently list decodable code with $n = 2^t$ for some integer t . Let $F : \{0, 1\}^{k+t} \rightarrow \{0, 1\}^{k+t}$ be the padding of f given by $F(\mathbf{x}, i) = (f(\mathbf{x}), i)$ for $i \in \{0, 1\}^t$. Finally let $b : \{0, 1\}^{k+t} \rightarrow \{0, 1\}$ be given by $b(\mathbf{x}, i) = E(\mathbf{x})_i$ where $i \in \{0, 1\}^t$ is viewed as an element of $[n]$ and $E(\mathbf{x})_i$ denotes the i th coordinate of $E(\mathbf{x})$. Then b is (2ε) -hardcore for F .

Proof. The proof follows mostly from definitions. Assume for contradiction that b is not 2ε -hardcore for F and let $A : \{0, 1\}^{k+t} \rightarrow \{0, 1\}$ be a polynomial time algorithm showing this. Note that this implies

$$\Pr_{x,i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i] \geq \frac{1}{2} + 2\varepsilon.$$

We show how to use A to invert f with high probability.

Our algorithm for inverting f proceeds as follows:

Algorithm 24.3.1 INVERT f

INPUT: $\mathbf{w} = f(\mathbf{x}) \in \{0, 1\}^k$ for unknown $\mathbf{x}$

OUTPUT: $\mathbf{x} \in \{0, 1\}^k$ or ERROR

```

1: Compute  $\mathbf{y} = (y_1, \dots, y_n) \in \{0, 1\}^n$  given by  $y_i = A(\mathbf{w}, i)$ .
2:  $S \leftarrow D(\mathbf{y})$ . ▷ Run the list decoder  $D$  for  $E$ 
3: FOR every  $\mathbf{x}' \in S$  DO
4:   IF  $f(\mathbf{x}') = \mathbf{y}$  THEN
5:     RETURN  $\mathbf{x}'$ 
6: RETURN ERROR
```

The running time of Algorithm 24.3.1 is clearly bounded by a polynomial in n .

We now analyze the probability that it successfully outputs $\mathbf{x}$ given $\mathbf{w} = f(\mathbf{x})$ as input. Let

$$G = \left\{ \mathbf{x} \mid \Pr_i [A(f(\mathbf{x}), i) = E(\mathbf{x})_i] \geq \frac{1}{2} + \varepsilon \right\}.$$

We note that

$$\frac{1}{2} + 2\varepsilon \leq \Pr_{\mathbf{x},i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i] \leq \Pr_{\mathbf{x}} [\mathbf{x} \in G] + \frac{1}{2} + \varepsilon, \quad (24.1)$$

where the upper bound follows from the following argument. Note that

$$\begin{aligned} \Pr_{\mathbf{x},i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i] &= \Pr_{\mathbf{x}} [\mathbf{x} \in G] \cdot \Pr_{\mathbf{x},i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i \mid \mathbf{x} \in G] \\ &\quad + \Pr_{\mathbf{x}} [\mathbf{x} \notin G] \cdot \Pr_{\mathbf{x},i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i \mid \mathbf{x} \notin G] \\ &\leq \Pr_{\mathbf{x}} [\mathbf{x} \in G] + \Pr_{\mathbf{x},i} [A(f(\mathbf{x}), i) = E(\mathbf{x})_i \mid \mathbf{x} \notin G], \end{aligned}$$

which from definition of G proves the upper bound in (24.1). Further note that from (24.1), we can conclude that:

$$\Pr_{\mathbf{x}} [\mathbf{x} \in G] \geq \varepsilon.$$

We now argue that if $\mathbf{x} \in G$ then our inversion algorithm above outputs $\mathbf{x}$. To see this note that by definition of G we have that

$$\Delta(E(\mathbf{x}), \mathbf{y}) \leq \left(\frac{1}{2} - \varepsilon \right) n.$$

Now by the property of our list-decoder we have that $\mathbf{x} \in S \triangleq D(\mathbf{y})$. In turn this implies that our algorithm will report $\mathbf{x}$ as output (because of line 4), as desired. $\square$

To apply the lemma above to get a theorem showing the existence of a hardcore predicate (one that works for every $\varepsilon(k) = k^{-c}$) for every padded one-way permutation one needs a few additional steps. Specifically one would need to know more about the hardness of f , and use this to choose the function $\varepsilon(k)$ which in turn will determine $n(k)$ and in turn this determines the running time of the inverter of f . We omit these details in this book (see Section 24.6 for some pointers).

24.4 Average case complexity

Next we turn to another application of error-correcting codes in complexity theory — this time to relate the average case complexity of problems to the worst-case complexity of related problems. Most results in computational complexity refer only to the worst-case complexity. For instance if we assume $P \neq NP$ and we know that some problem Π is NP-hard this implies that for every polynomial time algorithm A there exists an instance x for which $A(\mathbf{x}) \neq \Pi(\mathbf{x})$. But such $\mathbf{x}$'s may be extremely rare. Average-case complexity seeks much stronger notions of hardness one that may imply that for every algorithm A and sufficiently large n ,

$$\Pr[A(\mathbf{x}) \neq \Pi(\mathbf{x})] \leq \frac{1}{10}.$$

(I.e., at least 10% of the inputs are hard, when the inputs are chosen uniformly at random.) While in general such problems that are hard on uniformly chosen random inputs seems quite hard, coding theory seems to lead to such hardness quite easily (though for problem which are much harder than NP-complete ones). In this section we describe such a hardness result for the 'permanent' problem, which relies on the local decodability of Reed-Muller codes. Later we abstract this reduction into a generic one based on locally decodable codes (see Chapter ??).

Definition 24.4.1. For a field $\mathbb{F}$ and matrix $M \in \mathbb{F}^{n \times n}$ with $M = [m_{i,j}]_{i,j \in [n]}$ the Permanent of M , denoted $\text{perm}(M)$ is the quantity $\text{perm}(M) = \sum_{\pi \in S_n} \prod_{i=1}^n m_{i,\pi(i)}$ where $S_n = \{\pi : [n] \rightarrow [n] : \pi \text{ invertible}\}$ is the set of permutations over $[n]$.

Note that the permanent of M is very similar to the notion of the determinant of a matrix. Specifically, the determinant can be defined as $\sum_{\pi \in S_n} \text{sign}(\pi) \prod_{i=1}^n m_{i,\pi(i)}$ where $\text{sign} : S_n \rightarrow \{-1, +1\}$ is an easy to compute (though somewhat long to define) function. Indeed the permanent is arguably simpler in that it is simpler to define than the determinant. Remarkably while the more 'complex' determinant is easy to compute, the permanent turns out to be quite hard. The following theorem hints at its complexity.

Theorem 24.4.2. The permanent is NP-hard. Specifically there are polynomial time algorithms A and B such that for every instance ϕ of satisfiability, we have $A(\phi) = (\mathbb{F}_q, n, M)$ where $M \in \mathbb{F}_q^{n \times n}$ and $B(\text{perm}(M)) = 1$ if and only if ϕ is satisfiable.

We remark that the theorem above merely bounds the complexity of the permanent from below. It is not known to be reducible to satisfiability (and indeed not believed to be in NP).

Turning to the theme of this section, the result above indicates that the permanent is hard in the usual ‘worst-case’ sense. But as we will see below this problem is as hard on random instances as on worst case

For this section we use the following definition of easy on the average. (This definition is similar in spirit to the definition of ‘hard on average’ in Definitions 24.3.2.)

Definition 24.4.3. *Given a function f and family of distributions $D = \{D_n\}_n$ where D_n is a distribution on inputs of length n , we say that the pair (f, D) is in average polynomial time if there exists a polynomial time algorithm A such that for every n , we have*

$$\Pr_{\mathbf{x} \sim D_n} [f(\mathbf{x}) \neq A(\mathbf{x})] \leq \frac{1}{10}.$$

Remark 24.4.4. *The threshold of being incorrect with probability at most $\frac{1}{10}$ is an arbitrary choice. Unfortunately, unlike the case of randomized polynomial time, the notion of average polynomial time is not robust with respect to this threshold. So it is conceivable (likely?) that some pairs (f, D) would be in average polynomial time under the above definition, but not so if the threshold were set to something smaller such as $\frac{1}{20}$.*

Theorem 24.4.5. *Let $D = \{D_n\}_n$ be a family of distributions given as follows: Given n , let $p = p(n) \in [30n, 60n]$ be a fixed prime. Let D_n be a uniformly random matrix $M \in \mathbb{F}_p^{n \times n}$. Then if the pair (perm, D) is in average polynomial time, then $\text{perm}(M)$ is computable in randomized polynomial time for every matrix in the support of D .*

Proof. By assumption on (perm, D) being in average polynomial time, let A be a polynomial time algorithm such that

$$\Pr_{M \in \mathbb{F}_p^{n \times n}} [A(M) \neq \text{perm}(M)] \leq \frac{1}{10}. \quad (24.2)$$

Define the polynomial over the ‘matrix’ of variables $\mathbf{X} = \{X_{i,j}\}_{i,j \in [n]}$:

$$P(\mathbf{X}) = \sum_{\pi \in S_n} \prod_{i=1}^n X_{i,\pi(i)},$$

where S_n is as in Definition 24.4.1. Note that $P(\mathbf{X})$ is a multivariate polynomial of degree n in n^2 variables. Further for any $M \in \mathbb{F}_p^{n \times n}$, we have

$$\text{perm}(M) = P(M).$$

The above implies that the vector

$$\mathbf{c} = (P(M))_{M \in \mathbb{F}_p^{n \times n}}$$

is a valid codeword in $\text{RM}(p, n^2, n)$ (recall Definition 9.1.3). Further define

$$\mathbf{y} = (A(M))_{M \in \mathbb{F}_p^{n \times n}}.$$

Then by (24.2), we have

$$\delta(\mathbf{c}, \mathbf{y}) \leq \frac{1}{10}. \quad (24.3)$$

It can be verified that the above fraction of errors is less than half the distance of $\text{RM}(p, n^2, n)$ and hence for example we could use Theorem 13.3.8. However, the issue with this is that the block length of $\text{RM}(p, n^2, n)$ is p^{n^2} and so a runtime that is polynomial in p^{n^2} (which is what we will get from Theorem 13.3.8) is too expensive for our purposes.

However, the point to note is that we do not need to decode $\mathbf{y}$ in its entirety. I.e. we are not interested in computing the entire codeword $\mathbf{c}$ but given any $M \in \mathbb{F}_p^{n \times n}$, we are interested in computing $\mathbf{c}_M$ only. In other words, we are interested in *locally* decoding $\mathbf{y}$ for which we can use results from Section ?? . Specifically, the algorithm is to return the output of the following call:

$$\text{RM-Local-Corrector}^{\mathbf{y}}(M; p, n, n^2).$$

There is one ‘catch’ in that the setting of locally correctable codes we assume oracle access to $\mathbf{y}$, which we do not have directly. However, note that since A is polynomial time algorithm, we can compute $A(M)$ for any given $M \in \mathbb{F}_p^{n \times n}$ in polynomial time (so as long as the locally correctable code that runs in $\text{poly}(n)$ time (which means we will need to call A at most those many times), our overall algorithm will run in $\text{poly}(n)$ time as well.

Specifically, Proposition ?? implies that we need to make at most $p-1$ calls to A and the runtime is $\text{poly}(p, n^2)$. Thus, the overall runtime (by our choice of p) is $\text{poly}(n)$, as desired. Finally, for the correctness of our reduction, by Lemma ?? our reduction works as long as

$$\delta(\mathbf{c}, \mathbf{y}) \leq \frac{1}{6} \left(1 - \frac{n+1}{p-1}\right).$$

Note that the above and (24.3) imply that we would be done if can ensure

$$\frac{1}{6} \left(1 - \frac{n+1}{p-1}\right) > \frac{1}{10},$$

which in turn is satisfied if

$$\frac{n+1}{p-1} < \frac{1}{15},$$

which in turn is satisfied if

$$p > 15(n+1) + 1.$$

The above is satisfied by our choice of p (the existence of such a p follows from known results) and the proof is complete. $\square$

Next, we generalize the essence of the above proof to show a general worst-case hardness to average case hardness result.

Lemma 24.4.6. *Let $N = 2^n$ and $K = 2^k$ and*

$$C : \{0, 1\}^K \rightarrow \{0, 1\}^N$$

be a $(\text{polylog } N, \frac{1}{10})$ -locally decodable code. Given a function

$$f : \{0, 1\}^k \rightarrow \{0, 1\}$$

let

$$F : \{0, 1\}^n \rightarrow \{0, 1\}$$

be the function

$$F = f \circ C.$$

If F is in average polynomial time, then f is computable in randomized polynomial time (on worst-case instances).

Proof. This proofs turns out to follow essentially by combining the relevant definitions.

Let A be a polynomial time algorithm that shows that F is in average polynomial time. So,

$$\Pr_{\mathbf{x} \in \{0,1\}^n} [A(\mathbf{x}) \neq F(\mathbf{x})] \leq \frac{1}{10}.$$

Let $T(A) \in \{0, 1\}^N$ denote the vector $T(A)_{\mathbf{x}} = A(\mathbf{x})$. Changing notations, and using $F = f \circ C$, we have

$$\delta(T(A), C(T(f))) \leq \frac{1}{10}.$$

Let D be the local decoding algorithm for C . We then have for every $\mathbf{y} \in \{0, 1\}^k$, with probability at least $\frac{2}{3}$ (over the coin tosses of D), $D^{T(A)}(\mathbf{y}) = f(\mathbf{y})$.

But note that $D^{T(A)}$ can be simulated in $\text{polylog } K$ time since D runs in $\text{polylog } N = \text{polylog } K$ time given oracle access to A and oracle access to A amounts to simply computing A which takes polynomial time in k . Since $\log N = O(\log K) = O(k)$ we have that f can be computed on every input in time polynomial in the input (by a probabilistic algorithm which is correct with probability at least $2/3$ on every input). $\square$

While the lemma above is quite general, we warn the reader that the resulting function F is not necessarily simple to compute even if f is. Note that the natural way to compute F on an input of length n would be to compute f on all inputs of length k , encode the resulting 2^k -bit long string, and then read the specific bit of the encoding. Assuming $k = \Omega(n)$, this takes time $2^{\Omega(n)}$. So in particular this does not necessarily lead to functions in F that are in NP that are hard on average (assuming for example that f is in NP but not in P). However the reduction can be applied to functions f of higher complexity, say exponential time computable, to get new functions in the same class that are also hard on average. We omit the details in this section.

24.5 Exercises

Exercise 24.1. Prove that in a protocol Π minimizing $\text{CC}(f)$, the last person to speak is always Alice.

Exercise 24.2. Prove that if $\mathcal{X} = \{0, 1\}^n$, then for every $\mathcal{Y}$, every $\mathcal{O}$ and every function $f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{O}$, we have $\text{CC}(f) \leq n$.

Exercise 24.3. For any $\delta \geq \frac{1}{3}$, define $\text{CC}_{\delta}^R(f)$ denote the randomized communication complexity where we replace the success probability of $\frac{2}{3}$ by $1 - \delta$ in Definition 24.1.5. Note in our usual definition, we have $\text{CC}^R(f) = \text{CC}_{\frac{1}{3}}^R(f)$. Prove that for any $\delta \geq \frac{1}{3}$,

$$\text{CC}^R(f) \leq O\left(\log\left(\frac{1}{\delta}\right) \cdot \text{CC}^R(f)\right).$$

Exercise 24.4. We start off with the easier version: show that $\text{CC}(\text{EQ}) \leq n$.

For the rest of the exercise, we will argue that $\text{CC}(\text{EQ}) \geq n$.

Fix a (deterministic) protocol Π that solves EQ with c bits of communication and let $t_{\mathbf{x}, \mathbf{y}}$ denote the transcript of the interaction between the players on inputs $\mathbf{x}$ and $\mathbf{y}$. So $t_{\mathbf{x}, \mathbf{y}} = b_1, \dots, b_c$ where c is the number of bits communicated.

1. Show that if

$$t_{\mathbf{x}, \mathbf{y}} = t_{\mathbf{x}', \mathbf{y}'} = t$$

then we also have

$$t_{\mathbf{x}, \mathbf{y}'} = t_{\mathbf{x}', \mathbf{y}} = t.$$

2. Show that $t_{\mathbf{x}, \mathbf{x}} \neq t_{\mathbf{y}, \mathbf{y}}$ for every $\mathbf{x} \neq \mathbf{y}$.

3. Prove that

$$2^c \geq 2^n.$$

4. Conclude that $\text{CC}(\text{EQ}) \geq n$.

Exercise 24.5. Show that the protocol in the proof of Lemma 24.1.10 can be implemented with $O(\log n)$ bits.

Exercise 24.6. Show that for any $t \geq 2$, MAX t -SAT is NP-hard for all $t \geq 2$.

Hint: Reduce from MAXCUT (see Definition C.4.2).

Exercise 24.7. Design a randomized $(1 - 2^{-t} - \epsilon)$ -approximation algorithm that runs in time $O(mt/\epsilon)$ for any $\epsilon > 0$ on any t -SAT formula on m clauses (and the algorithm succeeds with constant probability).

Hint: Use Lemma 24.2.4 and the Markov's inequality.

Exercise 24.8. In this problem, we will prove a lower bound on the size of a t -wise independent generator. Let $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ be a t -wise independent generator. Our goal in this exercise is to argue that

$$2^s \geq \sum_{i=0}^{\ell} \binom{n}{i}, \quad (24.4)$$

where we define

$$\ell = \left\lceil \frac{t}{2} \right\rceil.$$

Towards proving (24.4), for any $T \subseteq [n]$ of size at most ℓ , define the function $\chi_T : \{0, 1\}^s \rightarrow \{-1, 1\}$, such that for any $\mathbf{x} = (x_1, \dots, x_s) \in \{0, 1\}^s$, we have

$$\chi_T(\mathbf{x}) = \prod_{i \in T} (-1)^{G(\mathbf{x})_i}.$$

Now, argue the following:

1. Let $S, T \subseteq [n]$ such that $|S|, |T| \leq \ell$, argue that

$$\langle \chi_S, \chi_T \rangle \stackrel{\text{def}}{=} \sum_{\mathbf{x} \in \{0, 1\}^s} \chi_S(\mathbf{x}) \cdot \chi_T(\mathbf{x}) = \begin{cases} 2^s & \text{if } S = T \\ 0 & \text{otherwise} \end{cases}.$$

2. Using the above part or otherwise, argue that the set of $\{\chi_T\}_{T \subseteq [n], |T| \leq \ell}$, when thought of as a subset of $\mathbb{R}^{2^s}$ (think of each function χ_T by its ‘truth table’), are all linearly independent vectors in $\mathbb{R}^{2^s}$.
3. Using the above part or otherwise, argue (24.4).
4. Using (24.4) or otherwise, argue that any t -wise independent generator needs to have seed-length at least $\ell \cdot (\log_2 n - O(1))$. Then conclude that the seed length achieved in Lemma 24.2.6 is essentially optimal.

Exercise 24.9. Prove Proposition 24.2.12

Exercise 24.10. Consider two distributions $\mathcal{D}_1, \mathcal{D}_2$ with the same (finite) support set S . Then prove the following:

1. The total variational distance between $\mathcal{D}_1, \mathcal{D}_2$ is exactly

$$\max_{T \subseteq S} \left| \Pr_{X \sim \mathcal{D}_1} [X \in T] - \Pr_{X \sim \mathcal{D}_2} [X \in T] \right|.$$

2. Let $f : S \rightarrow [0, 1]$ be any function and let δ be the total variational distance between $\mathcal{D}_1$ and $\mathcal{D}_2$, then

$$|\mathbb{E}_{X \sim \mathcal{D}_1} [f(X)] - \mathbb{E}_{X \sim \mathcal{D}_2} [f(X)]| \leq 2\delta.$$

Exercise 24.11. Prove Lemma 24.2.18.

Hint: Use Lemma 24.2.8 and Exercise 24.10.

Exercise 24.12. In this exercise, we will define and consider properties of orthonormal basis. In addition to being of interest in its own right, it’ll help us improve the bound in Lemma 24.2.15 (see Exercise 24.13 below).

Let $n \geq 1$ be an integer and we will consider the vector space $\mathbb{R}^{2^n}$. We note that for every vector $\mathbf{v} \in \mathbb{R}^{2^n}$ can be equivalently be defined by a function $f_v : \{0, 1\}^n \rightarrow \mathbb{R}$ such that $f_v(\mathbf{x}) = v_{\mathbf{x}}$ where we identify the positions in $\mathbf{v}$ by elements $\mathbf{x} \in \{0, 1\}^n$. In this exercise we will switch back and forth between the vector and function representations.

We say a subset $B \subseteq \mathbb{R}^{2^n}$ is an orthonormal basis if

- $|B| = 2^n$
- For every $\mathbf{u} \in B$, we have $\|\mathbf{u}\|_2 \stackrel{\text{def}}{=} \sqrt{\sum_{i=1}^{2^n} u_i^2} = 1$
- For every $\mathbf{u} \neq \mathbf{v} \in B$, we have $\langle \mathbf{u}, \mathbf{v} \rangle \stackrel{\text{def}}{=} \sum_{i=1}^{2^n} u_i \cdot v_i = 0$.

We first begin with some example orthonormal basis and then prove some properties of such basis.

1. Prove that the set $\{\mathbf{e}_{\mathbf{x}}\}_{\mathbf{x} \in \{0, 1\}^n}$ (i.e. the function $f_{\mathbf{e}_{\mathbf{x}}}(\mathbf{y}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{y} \\ 0 & \text{otherwise} \end{cases}$) is an orthonormal basis. (This is the so called standard basis.)
2. For any subset $S \subseteq [n]$, define the function

$$\chi_S(\mathbf{y}) = \frac{1}{\sqrt{2^n}} \cdot (-1)^{\sum_{i \in S} y_i}.$$

Prove that the set of vectors $\{\mathbf{v}_S\}_{S \subseteq [n]}$, where for each S $\mathbf{v}_S$ is the function corresponding to χ_S , is an orthonormal basis. (This is the so called Fourier basis and we have seen this basis already in Exercise 24.8.)

3. Prove that the vectors in an orthonormal basis are linearly independent (hence the term basis).
4. Let $\mathbf{u} \in \mathbb{R}^{2^n}$ (in other words $\mathbf{u} = (u_1, \dots, u_{2^n})$ is its representation in the standard basis). Then the following is true for any orthonormal basis B :

$$\mathbf{u} = \sum_{\mathbf{v} \in B} \langle \mathbf{u}, \mathbf{v} \rangle \cdot \mathbf{v}.$$

The vector $(\langle \mathbf{u}, \mathbf{v} \rangle)_{\mathbf{v} \in B}$ is the representation of $\mathbf{u}$ in B . The term $\langle \mathbf{u}, \mathbf{v} \rangle$, also sometimes denoted as $\hat{\mathbf{u}}_{\mathbf{v}}$ is the coefficient of $\mathbf{v}$ corresponding to $\mathbf{v}$ in B .

5. The above result means we can talk about the norm of a vector $\mathbf{u} \in \mathbb{R}^{2^n}$ under any orthonormal basis B . Define for any integer p :

$$\|\mathbf{u}\|_p^{(B)} = \sqrt[p]{\sum_{\mathbf{v} \in B} |\langle \mathbf{u}, \mathbf{v} \rangle|^p}.$$

Prove that

$$\|\mathbf{u}\|_{\infty}^{(B)} = \max_{\mathbf{v} \in B} |\langle \mathbf{u}, \mathbf{v} \rangle|.$$

6. Prove that for any vector $\mathbf{u} \in \mathbb{R}^{2^n}$ and any two orthonormal basis A and B we have

$$\|\mathbf{u}\|_2^{(A)} = \|\mathbf{u}\|_2^{(B)}.$$

The above is called the Parseval's identity.

7. Prove that the following holds for any vector $\mathbf{u} \in \mathbb{R}^{2^n}$ and any orthonormal basis B :

$$\|\mathbf{u}\|_1^{(B)} \leq \sqrt{2^n} \cdot \|\mathbf{u}\|_2^{(B)}.$$

8. Prove that the following holds for any vector $\mathbf{u} \in \mathbb{R}^{2^n}$ and any orthonormal basis B :

$$\|\mathbf{u}\|_2^{(B)} \leq \sqrt{2^n} \cdot \|\mathbf{u}\|_{\infty}^{(B)}.$$

9. Prove that for any vector $\mathbf{u} \in \mathbb{R}^{2^n}$ and any two orthonormal basis A and B we have

$$\|\mathbf{u}\|_1^{(A)} \leq 2^n \cdot \|\mathbf{u}\|_{\infty}^{(B)}.$$

Exercise 24.13. In this exercise, we will improve on the bound in Lemma 24.2.15: specifically we want to replace the $2^n \cdot \epsilon$ bound by $\sqrt{2^n} \cdot \epsilon$.

Towards that end, let π be an arbitrary probability distribution over $\{0, 1\}^n$. We will overload notation and use π to denote the vector in $\mathbb{R}^{2^n}$ where $\pi(\mathbf{u})$ for any $\mathbf{u} \in \{0, 1\}^n$ is the probability assigned to $\mathbf{u}$ by π . Also define μ to be the uniform distribution on $\{0, 1\}^n$, i.e. for every $\mathbf{u} \in \{0, 1\}^n$, we have $\mu(\mathbf{u}) = \frac{1}{2^n}$.

Define the 'maximum bias' of the distribution π as follows:

$$\text{maxbias}(\pi) = \max_{\emptyset \neq S \subseteq [n]} \left| \Pr_{\mathbf{u} \sim \pi} \left[\sum_{i \in S} \mathbf{u}[i] = 0 \right] - \Pr_{\mathbf{u} \sim \pi} \left[\sum_{i \in S} \mathbf{u}[i] = 1 \right] \right|$$

Now consider the following steps:

1. Prove that for any distribution π , we have (where the max is over all non-zero linear functions $\ell : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$):

$$\text{maxbias}(\pi) = 2 \cdot \max_{\ell} \left| \Pr_{\mathbf{u} \sim \pi} [\ell(\mathbf{u}) = 1] - \frac{1}{2} \right|.$$

2. Prove that

$$\text{maxbias}(\pi) = \sqrt{2^n} \cdot \|\pi - \mu\|_{\infty}^{(F)},$$

where F denotes the Fourier basis from part 2 of Exercise 24.12.

3. Prove that the total variational distance between μ and π is $\frac{1}{2} \cdot \|\pi - \mu\|_1^{(S)}$, where S is the standard basis as defined in part 1 of Exercise 24.12.

4. Using the above parts or otherwise, argue that the bound of $2^n \cdot \epsilon$ in Lemma 24.2.15 can be replaced by $\sqrt{2^n} \cdot \epsilon$.

Hint: Part 9 of Exercise 24.12 might be useful.

Exercise 24.14. Is G from Lemma 24.2.20 ϵ -biased? If so prove it. If not give a counterexample.

Exercise 24.15. Prove Corollary 24.2.22.

Exercise 24.16. Let f be a permutation that is easy to compute and let b be a hardcore predicate for f . Prove that f^{-1} is very hard.

Hint: Utilize the fact that f is easy to compute.

Exercise 24.17. Let $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$ be a one-way permutation and let $b : \{0, 1\}^k \rightarrow \{0, 1\}$ be a hardcore predicate for f . For $m \leq \text{poly}(k)$, let $G_m : \{0, 1\}^k \rightarrow \{0, 1\}^{m+k}$ be the function given by $G_m(\mathbf{x}) = (b(\mathbf{x}), G_{m-1}(f(\mathbf{x})))$ with $G_0(\mathbf{x}) = \mathbf{x}$. We will solve the following sub-problems to ultimately prove that G_m $o(1)$ -fools every polynomial time algorithm A :

1. Prove that if $b \circ f^{-1}$ is $(\frac{1}{2} + \epsilon)$ -hard, then $G_1 = (b(\mathbf{x}), f(\mathbf{x}))$ ϵ -fools all polynomial time algorithms. Towards this end:

(a) Let A be any algorithm $A : \{0, 1\}^{k+1} \rightarrow \{0, 1\}$. Define

$$S_A = \left\{ \mathbf{y} \in \{0, 1\}^k \mid A(0, \mathbf{y}) \neq A(1, \mathbf{y}) \right\}.$$

Then prove that (where all the expectations are over the uniform distribution over the domain):

$$\begin{aligned} & \left| \mathbb{E}_{(B, \mathbf{y}) \in \{0, 1\}^{k+1}} [A(B, \mathbf{y})] - \mathbb{E}_{\mathbf{y} \in \{0, 1\}^k} [A(b \circ f^{-1}(\mathbf{y}), \mathbf{y})] \right| \\ &= \Pr_{\mathbf{y} \in \{0, 1\}^k} [\mathbf{y} \in S_A] \cdot \left(\max_{B \in \{0, 1\}} \mathbb{E}_{\mathbf{y} \in S_A} [\mathbb{1}_{A(b \circ f^{-1}(\mathbf{y}), \mathbf{y})=B}] - \frac{1}{2} \right). \end{aligned} \quad (24.5)$$

(b) Let A^* be an algorithm that is not ϵ -fooled by G_1 . Then consider the the following two algorithms (Algorithm 24.5.1 parameterized by $b \in \{0, 1\}$).

Algorithm 24.5.1 $B_b^{A^*}$

INPUT: $\mathbf{y} \in \{0, 1\}^k$

OUTPUT: A bit in $\{0, 1\}$

```

1: IF  $A^*(0, \mathbf{y}) = A^*(1, \mathbf{y})$  THEN
2:   RETURN random bit in  $\{0, 1\}$ 
3: ELSE
4:   RETURN  $b$ 

```

Prove that the following holds:

$$\max_{c \in \{0, 1\}} \left\{ \Pr_{\mathbf{y} \in \{0, 1\}^k} [B_c^{A^*}(\mathbf{y}) = b \circ f^{-1}(\mathbf{y})] \right\} \geq \frac{1}{2} + \epsilon.$$

(c) Using the above sub-parts or otherwise prove part 1 for all polynomial time algorithms $A : \{0, 1\}^{k+1} \rightarrow \{0, 1\}$ (note that the algorithm does not have a binary output anymore).

2. Prove that if $b \circ f^{-1}$ is $(\frac{1}{2} + \varepsilon)$ -hard, then $G_m = (b(\mathbf{x}), G_{m-1}(\mathbf{x}))$ ($m \cdot \varepsilon$)-fools all polynomial time algorithms. Towards this end:

(a) Let $i \in [m]$ and Let $A^i : \{0, 1\}^{m+k-i+1} \rightarrow \{0, 1\}$ be a polynomial time algorithm such that

$$\left| \mathbb{E}_{(B, \mathbf{y}) \in \{0, 1\}^{k+1}} \left[A^i(B, G_{m-i}(\mathbf{y})) \right] - \mathbb{E}_{\mathbf{y} \in \{0, 1\}^k} \left[A^i(b \circ f^{-1}(\mathbf{y}), G_{m-i}(\mathbf{y})) \right] \right| \geq \varepsilon.$$

The prove that there exists an algorithm B' such that

$$\Pr_{\mathbf{y} \in \{0, 1\}^k} [B'(\mathbf{y}) = b \circ f^{-1}(\mathbf{y})] \geq \frac{1}{2} + \varepsilon.$$

Hint: Try and generalize the arguments for part 1.

(b) Prove that every polynomial time algorithm $A : \{0, 1\}^{m+k} \rightarrow \{0, 1\}$ satisfies the following for every $i \in [m]$:

$$\left| \mathbb{E}_{B^{(i-1)} \in \{0, 1\}^{i-1}} \left[\mathbb{E}_{(B, \mathbf{y}) \in \{0, 1\}^{k+1}} \left[A(B^{(i-1)}, B, G_{m-i}(\mathbf{y})) \right] \right] - \mathbb{E}_{\mathbf{y} \in \{0, 1\}^k} \left[A(B^{(i-1)}, b \circ f^{-1}(\mathbf{y}), G_{m-i}(\mathbf{y})) \right] \right| \leq \varepsilon.$$

Hint: Use part 2a.

(c) Prove that every polynomial time algorithm $A : \{0, 1\}^{m+k} \rightarrow \{0, 1\}$ is $m \cdot \varepsilon$ -fooled by G_m .

Hint: Use part 2b and induction.

(d) Using the above sub-parts or otherwise prove part 2 for all polynomial time algorithms $A : \{0, 1\}^{k+m} \rightarrow \{0, 1\}$ (note that the algorithm does not have a binary output anymore).

Exercise 24.18. We consider the following results related to a padding of a permutation (recall Definition 24.3.5):

1. Prove that if $f : \{0, 1\}^k \rightarrow \{0, 1\}^k$ is a one-way permutation, then the k -padding of f is also a one-way permutation.
2. Let $b : \{0, 1\}^{2k} \rightarrow \{0, 1\}$ be a predicate that only depends on the second half of its input bits. Then prove that b is not a hardcore predicate for some one-way permutation.

24.6 Bibliographic Notes

Communication complexity is due to Yao [?]. The separation of deterministic and probabilistic communication complexity of the Equality function is from [?]. Our proof here is by now folklore. More details on communication complexity can be found in the text by Kushilevitz and Nisan [?].

The notion of limited independence go back to Carter-Wegman. The connection between limited independence and linear codes is due to Alon, Babai and Itai [?]. The notion of ε -bias and almost limited-independence, and the relationship between the two, are from Naor and Naor [?]. Some of the specific connections between codes and ε -biased spaces are from Alon, Goldreich, Håstad, and Peralta [?]. Vadhan's book on Pseudorandomness [?] is an excellent source for much more on this topic.

The notion of hardcore predicates and connections to pseudo-randomness is from the works of Blum and Micali [?] and Yao [?]. The connections to list-decoding of codes, and to coding theory in general, is from the seminal work of Goldreich and Levin [?].

The hardness of permanent on the average is from the work of Lipton [?] building on the work of Beaver and Feigenbaum [15]. Quantitative strengthenings of this result leading to our Theorem ?? and Exercise?? are from Gemmell et al. [58] and Gemmell and Sudan [61]. The general connection between local decoding and worst-case to average-case reductions is from Sudan et al. [151].

Bibliography

- [1] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES Is in P. *Annals of Mathematics*, 160(2):781–793, 2004.
- [2] M. Alekhnovich. Linear diophantine equations over polynomials and soft decoding of reed-solomon codes. In *The 43rd Annual IEEE Symposium on Foundations of Computer Science, 2002. Proceedings.*, pages 439–448, Nov 2002.
- [3] N. Alon. Eigenvalues and expanders. *Combinatorica*, 6(2):83—96, 1986.
- [4] Noga Alon, Jehoshua Bruck, Joseph Naor, Moni Naor, and Ron M. Roth. Construction of asymptotically good low-rate error-correcting codes through pseudo-random graphs. *IEEE Trans. Inf. Theory*, 38(2):509–516, 1992.
- [5] Noga Alon and Michael Luby. A linear time erasure-resilient code with nearly optimal recovery. *IEEE Transactions on Information Theory*, 42(6):1732–1736, 1996.
- [6] Noga Alon and Joel Spencer. *The Probabilistic Method*. John Wiley, 1992.
- [7] Erdal Arkan. Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels. *IEEE Transactions on Information Theory*, pages 3051–3073, July 2009.
- [8] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, May 1998.
- [9] Sanjeev Arora and Shmuel Safra. Probabilistic checking of proofs: A new characterization of NP. *Journal of the ACM*, 45(1):70–122, January 1998.
- [10] M. Artin. *Algebra*. Prentice-Hall Of India Pvt. Limited, 1996.
- [11] Jarosław Błasiok, Venkatesan Guruswami, Preetum Nakkiran, Atri Rudra, and Madhu Sudan. General strong polarization. In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing*, pages 485–492, 2018.
- [12] P.G.H. Bachmann. *Die analytische Zahlentheorie*. Number v. 2 in Zahlentheorie. Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. 2. th. Teubner, 1894.

- [13] L.A. Bassalygo. New upper boundes for error-correcting codes. *Problems of Information Transmission*, 1(1):32–35, 1965.
- [14] John Bather. A conversation with herman chernoff. *Statistical Science*, 11(4):335–350, 1996.
- [15] Donald Beaver and Joan Feigenbaum. Hiding instances in multioracle queries. In Christian Choffrut and Thomas Lengauer, editors, *STACS 90, 7th Annual Symposium on Theoretical Aspects of Computer Science, Rouen, France, February 22-24, 1990, Proceedings*, volume 415 of *Lecture Notes in Computer Science*, pages 37–48. Springer, 1990.
- [16] Mihir Bellare, Oded Goldreich, and Madhu Sudan. Free bits, pcps, and nonapproximability-towards tight results. *SIAM J. Comput.*, 27(3):804–915, 1998.
- [17] Eli Ben-Sasson, Swastik Kopparty, and Jaikumar Radhakrishnan. Subspace polynomials and limits to list decoding of reed-solomon codes. *IEEE Trans. Information Theory*, 56(1):113–120, 2010.
- [18] E. R. Berlekamp. Factoring polynomials over large finite fields. *Mathematics of Computation*, 24:713–735, 1970.
- [19] Elwyn R. Berlekamp, Robert J. McEliece, and Henk C. A. van Tilborg. On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory*, 24(3):384–386, May 1978.
- [20] E. L. Blokh and V. V. Zyablov. Coding of generalized concatenated codes. *Probl. Peredachi Inf.*, 10(3):45–50, 1974. English Translation in *Problems of Information Transmission*, 10:3(218–222), 1974.
- [21] R. C. Bose and D. K. Ray-Chaudhuri. On a class of error correcting binary group codes. *Information and Control*, 3:68–79, 1960.
- [22] Kristian Brander. *Interpolation and list decoding of algebraic codes*. PhD thesis, Technical University of Denmark, 2010.
- [23] Jehoshua Bruck and Moni Naor. The hardness of decoding linear codes with preprocessing. *IEEE Transactions on Information Theory*, 36(2), March 1990.
- [24] P.S. Bullen. *Handbook of Means and Their Inequalities*. Mathematics and Its Applications. Springer Netherlands, 2010.
- [25] Michael R. Capalbo, Omer Reingold, Salil P. Vadhan, and Avi Wigderson. Randomness conductors and constant-degree lossless expanders. In John H. Reif, editor, *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, pages 659–668. ACM, 2002.

- [26] Larry Carter and Mark N. Wegman. Universal classes of hash functions. *J. Comput. Syst. Sci.*, 18(2):143–154, 1979.
- [27] Donald G. Chandler, Eric P. Batterman, and Govind Shah. Hexagonal, information encoding article, process and system. *US Patent Number 4,874,936*, October 1989.
- [28] C. L. Chen and M. Y. Hsiao. Error-correcting codes for semiconductor memory applications: A state-of-the-art review. *IBM Journal of Research and Development*, 28(2):124–134, 1984.
- [29] Peter M. Chen, Edward K. Lee, Garth A. Gibson, Randy H. Katz, and David A. Patterson. RAID: High-performance, reliable secondary storage. *ACM Computing Surveys*, 26(2):145–185, 1994.
- [30] Q. Cheng and D. Wan. On the list and bounded distance decodability of reed-solomon codes. *SIAM Journal on Computing*, 37(1):195–209, 2007.
- [31] Herman Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, December 1952.
- [32] Alan Cobham. The Intrinsic Computational Difficulty of Functions. In Y. Bar-Hillel, editor, *Logic, Methodology and Philosophy of Science, proceedings of the second International Congress, held in Jerusalem, 1964*, Amsterdam, 1965. North-Holland.
- [33] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158, 1971.
- [34] Graham Cormode and S. Muthukrishnan. What’s hot and what’s not: tracking most frequent items dynamically. *ACM Trans. Database Syst.*, 30(1):249–278, 2005.
- [35] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. John Wiley & Sons, Inc., second edition edition, 2005.
- [36] Eren Şaçoğlu. Polarization and polar codes. *Foundations and Trends in Communications and Information Theory*, 8(4):259–381, 2012.
- [37] Phillipe Delsarte. An algebraic approach to the association schemes of coding theory. *Philips Research Reports*, Suppl. 10, 1973.
- [38] Richard A. DeMillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Inform. Process. Lett.*, 7(4):193–195, 1978.
- [39] Robert Dorfman. The detection of defective members of large populations. *The Annals of Mathematical Statistics*, 14(4):436–440, December 1943.
- [40] Devdatt P. Dubhashi and Alessandro Panconesi. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, 2009.

- [41] Ilya Dumer, Daniele Micciancio, and Madhu Sudan. Hardness of approximating the minimum distance of a linear code. *IEEE Transactions on Information Theory*, 49(1):22–37, 2003.
- [42] Ilya Dumer, Daniele Micciancio, and Madhu Sudan. Hardness of approximating the minimum distance of a linear code. *IEEE Trans. Information Theory*, 49(1):22–37, 2003.
- [43] Ilya I. Dumer. Concatenated codes and their multilevel generalizations. In V. S. Pless and W. C. Huffman, editors, *Handbook of Coding Theory*, volume 2, pages 1911–1988. North Holland, 1998.
- [44] Zeev Dvir and Shachar Lovett. Subspace evasive sets. *Electronic Colloquium on Computational Complexity (ECCC)*, 18:139, 2011.
- [45] Todd Ebert, Wolfgang Merkle, and Heribert Vollmer. On the autoreducibility of random sequences. *SIAM J. Comput.*, 32(6):1542–1569, 2003.
- [46] Jack Edmonds. Paths, trees, and flowers. In Ira Gessel and Gian-Carlo Rota, editors, *Classic Papers in Combinatorics*, Modern Birkhäuser Classics, pages 361–379. Birkhäuser Boston, 1987.
- [47] Peter Elias. Error-free coding. *IEEE Transactions on Information Theory*, 4(4):29–37, 1954.
- [48] Peter Elias. Coding for two noisy channels. In Cherry, editor, *Information Theory*, pages 61–74. Butterworth, 1956.
- [49] Peter Elias. List decoding for noisy channels. *Technical Report 335, Research Laboratory of Electronics, MIT*, 1957.
- [50] P. Erdős. On extremal problems of graphs and generalized graphs. *Israel Journal of Mathematics*, 2(3):183–190, 1964.
- [51] Paul Erdős. Some remarks on the theory of graphs. *Bulletin of the American Mathematical Society*, 53:292–294, 1947.
- [52] G. David Forney. *Concatenated Codes*. MIT Press, Cambridge, MA, 1966.
- [53] G. David Forney. Generalized Minimum Distance decoding. *IEEE Transactions on Information Theory*, 12:125–131, 1966.
- [54] Robert G. Gallager. *Low-Density Parity-Check Codes*. MIT Press, 1963.
- [55] Venkata Gandikota, Badih Ghazi, and Elena Grigorescu. Np-hardness of reed-solomon decoding, and the prouhet-tarry-escott problem. *SIAM J. Comput.*, 47(4):1547–1584, 2018.
- [56] M. R. Garey, D. S. Johnson, and L. J. Stockmeyer. Some simplified np-complete graph problems. *Theoretical Computer Science*, 1(3):237–267, 1976.

- [57] Michael R. Garey and David S. Johnson. *Computers and Intractability*. Freeman, 1979.
- [58] Peter Gemmell, Richard J. Lipton, Ronitt Rubinfeld, Madhu Sudan, and Avi Wigderson. Self-testing/correcting for polynomials and for approximate functions. In Cris Koutsougeras and Jeffrey Scott Vitter, editors, *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, May 5-8, 1991, New Orleans, Louisiana, USA*, pages 32–42. ACM, 1991.
- [59] Peter Gemmell and Madhu Sudan. Highly resilient correctors for multivariate polynomials. *Information Processing Letters*, 43(4):169–174, 1992.
- [60] Peter Gemmell and Madhu Sudan. Highly resilient correctors for polynomials. *Inf. Process. Lett.*, 43(4):169–174, 1992.
- [61] Peter Gemmell and Madhu Sudan. Highly resilient correctors for polynomials. *Inf. Process. Lett.*, 43(4):169–174, 1992.
- [62] E. N. Gilbert. A comparison of signalling alphabets. *Bell System Technical Journal*, 31:504–522, 1952.
- [63] M. J. E. Golay. Notes on digital coding. *Proceedings of the IRE*, 37:657, 1949.
- [64] Parikshit Gopalan, Cheng Huang, Huseyin Simitci, and Sergey Yekhanin. On the locality of codeword symbols. *IEEE Trans. Inf. Theory*, 58(11):6925–6934, 2012.
- [65] Venkatesan Guruswami. Limits to list decodability of linear codes. In *Proceedings of the 34th ACM Symposium on Theory of Computing (STOC)*, pages 802–811, 2002.
- [66] Venkatesan Guruswami. *List decoding of error-correcting codes*. Number 3282 in Lecture Notes in Computer Science. Springer, 2004. (Winning Thesis of the 2002 ACM Doctoral Dissertation Competition).
- [67] Venkatesan Guruswami. Iterative decoding of low-density parity check codes. *Bull. EATCS*, 90:53–88, 2006.
- [68] Venkatesan Guruswami. Linear-algebraic list decoding of folded reed-solomon codes. In *Proceedings of the 26th Annual IEEE Conference on Computational Complexity (CCC)*, pages 77–85, 2011.
- [69] Venkatesan Guruswami, Johan Håstad, and Swastik Kopparty. On the list-decodability of random linear codes. *IEEE Transactions on Information Theory*, 57(2):718–725, 2011.
- [70] Venkatesan Guruswami, Johan Håstad, Madhu Sudan, and David Zuckerman. Combinatorial bounds for list decoding. *IEEE Transactions on Information Theory*, 48(5):1021–1035, 2002.
- [71] Venkatesan Guruswami and Piotr Indyk. Linear-time encodable/decodable codes with near-optimal rate. *IEEE Trans. Inf. Theory*, 51(10):3393–3400, 2005.

- [72] Venkatesan Guruswami and Piotr Indyk. Linear-time encodable/decodable codes with near-optimal rate. *IEEE Transactions on Information Theory*, 51(10):3393–3400, 2005.
- [73] Venkatesan Guruswami and Atri Rudra. Limits to list decoding reed-solomon codes. *IEEE Transactions on Information Theory*, 52(8):3642–3649, August 2006.
- [74] Venkatesan Guruswami and Atri Rudra. Explicit codes achieving list decoding capacity: Error-correction with optimal redundancy. *IEEE Transactions on Information Theory*, 54(1):135–150, 2008.
- [75] Venkatesan Guruswami and Atri Rudra. Better binary list decodable codes via multilevel concatenation. *IEEE Transactions on Information Theory*, 55(1):19–26, 2009.
- [76] Venkatesan Guruswami and Atri Rudra. The existence of concatenated codes list-decodable up to the hamming bound. *IEEE Transactions on Information Theory*, 56(10):5195–5206, 2010.
- [77] Venkatesan Guruswami and Igor Shparlinski. Unconditional proof of tightness of johnson bound. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 754–755, 2003.
- [78] Venkatesan Guruswami and Madhu Sudan. Improved decoding of Reed-Solomon and algebraic-geometry codes. *IEEE Transactions on Information Theory*, 45(6):1757–1767, 1999.
- [79] Venkatesan Guruswami and Alexander Vardy. Maximum-likelihood decoding of reed-solomon codes is np-hard. *IEEE Trans. Information Theory*, 51(7):2249–2256, 2005.
- [80] Venkatesan Guruswami and Ameya Velingker. An entropy sumset inequality and polynomially fast convergence to Shannon capacity over all alphabets. In *Proceedings of 30th Conference on Computational Complexity*, pages 42–57, 2015.
- [81] Venkatesan Guruswami and Patrick Xia. Polar codes: Speed of polarization and polynomial gap to capacity. *IEEE Trans. Information Theory*, 61(1):3–16, 2015. Preliminary version in Proc. of FOCS 2013.
- [82] Venkatesan Guruswami and Chaoping Xing. Folded codes from function field towers and improved optimal rate list decoding. *Electronic Colloquium on Computational Complexity (ECCC)*, 19:36, 2012.
- [83] Richard W. Hamming. Error Detecting and Error Correcting Codes. *Bell System Technical Journal*, 29:147–160, April 1950.
- [84] G.H. Hardy and J.E. Littlewood. Some problems of diophantine approximation. *Acta Mathematica*, 37(1):193–239, 1914.

- [85] Seyed Hamed Hassani, Kasra Alishahi, and Rüdiger L. Urbanke. Finite-length scaling for polar codes. *IEEE Trans. Information Theory*, 60(10):5875–5898, 2014.
- [86] Johan Håstad, Steven Phillips, and Shmuel Safra. A well-characterized approximation problem. *Inf. Process. Lett.*, 47(6):301–305, 1993.
- [87] A. Hocquenghem. Codes correcteurs d’erreurs. *Chiffres (Paris)*, 2:147–156, 1959.
- [88] Tom Høholdt, J. H. van Lint, and Ruud Pellikaan. Algebraic geometry codes. In W. C. Huffman V. S. Pless and R. A. Brualdi, editors, *Handbook of Coding Theory*. North Holland, 1998.
- [89] Shlomo Hoory, Nathan Linial, and Avi Wigderson. Expander graphs and their applications. *Bull. Amer. Math. Soc. (N.S.)*, 43(4):439–561, 2006.
- [90] Cheng Huang, Minghua Chen, and Jin Li. Pyramid codes: Flexible schemes to trade space for access efficiency in reliable data storage systems. *ACM Trans. Storage*, 9(1), March 2013.
- [91] C. Hyltén-Cavallius. On a combinatorial problem. *Colloquium Mathematicum*, 6:61–65, 1958.
- [92] Tao Jiang and Alexander Vardy. Asymptotic improvement of the gilbert-varshamov bound on the size of binary codes. *IEEE Trans. Inf. Theory*, 50(8):1655–1664, 2004.
- [93] D. D. Joshi. A note on upper bounds for minimum distance codes. *Inf. Control.*, 1(3):289–295, 1958.
- [94] E. J. Weldon Jr. Justesen’s construction-the low-rate case (corresp.). *IEEE Trans. Inf. Theory*, 19(5):711–713, 1973.
- [95] Ari Juels and Madhu Sudan. A fuzzy vault scheme. *Des. Codes Cryptography*, 38(2):237–257, 2006.
- [96] J. Justesen. Class of constructive asymptotically good algebraic codes. *IEEE Trans. Inform. Theory*, pages 652–656, Sep 1972.
- [97] Erich Kaltofen. Polynomial-time reductions from multivariate to bi- and univariate integral polynomial factorization. *SIAM J. Comput.*, 14(2):469–489, 1985.
- [98] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103, 1972.
- [99] John Y. Kim and Swastik Kopparty. Decoding reed-muller codes over product sets. *Theory of Computing*, 13(1):1–38, 2017.
- [100] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–24, April 1976.

- [101] Andrei N. Kolmogorov. Three Approaches to the Quantitative Definition of Information. *Problems of Information Transmission*, 1(1):1–7, 1965.
- [102] Satish Babu Korada, Eren Sasoglu, and Rüdiger L. Urbanke. Polar codes: Characterization of exponent, bounds, and constructions. *IEEE Transactions on Information Theory*, 56(12):6253–6264, 2010.
- [103] E. Landau. *Handbuch der lehre von der verteilung der primzahlen*. Number v. 1 in *Handbuch der lehre von der verteilung der primzahlen*. B. G. Teubner, 1909.
- [104] Leonid A Levin. Universal sequential search problems. *Problemy Peredachi Informatsii*, 9(3):115–116, 1973.
- [105] Ming Li and Paul M. B. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Graduate Texts in Computer Science. Springer, New York, NY, USA, third edition, 2008.
- [106] Rudolf Lidl and Harald Niederreiter. *Introduction to Finite Fields and their applications*. Cambridge University Press, Cambridge, MA, 1986.
- [107] Richard J. Lipton. Efficient checking of computations. In Christian Choffrut and Thomas Lengauer, editors, *STACS 90, 7th Annual Symposium on Theoretical Aspects of Computer Science, Rouen, France, February 22-24, 1990, Proceedings*, volume 415 of *Lecture Notes in Computer Science*, pages 207–215. Springer, 1990.
- [108] A. Lubotzky, R. Phillips, and P. Sarnak. Ramanujan graphs. *Combinatorica*, 8(3):261–277, 1988.
- [109] Michael Luby, Michael Mitzenmacher, Mohammad Amin Shokrollahi, and Daniel A. Spielman. Efficient erasure correcting codes. *IEEE Transactions on Information Theory*, 47(2):569–584, 2001.
- [110] Florence Jessie MacWilliams. A theorem on the distribution of weights in a systematic code. *Bell Systems Technical Journal*, 42:79–94, 1963.
- [111] G. A. Margulis. Explicit constructions of expanders. *Problemy Peredači Informacii*, 9(4):71–80, 1973.
- [112] James L. Massey. *Threshold Decoding*. MIT Press, Cambridge, MA, USA, 1963. 129 pages.
- [113] Robert J. McEliece. On the average list size for the Guruswami-Sudan decoder. In *7th International Symposium on Communications Theory and Applications (ISCTA)*, July 2003.
- [114] Robert J. McEliece, Eugene R. Rodemich, Howard Rumsey Jr., and Lloyd R. Welch. New upper bounds on the rate of a code via the Delsarte-MacWilliams inequalities. *IEEE Transactions on Information Theory*, 23:157–166, 1977.
- [115] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [116] Ryuhei Mori and Toshiyuki Tanaka. Source and channel polarization over finite fields and Reed-Solomon matrices. *IEEE Trans. Information Theory*, 60(5):2720–2736, 2014.
- [117] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.
- [118] David E. Muller. Application of boolean algebra to switching circuit design and to error detection. *Trans. I.R.E. Prof. Group on Electronic Computers*, 3(3):6–12, 1954.
- [119] S. Muthukrishnan. Data streams: Algorithms and applications. *Foundations and Trends in Theoretical Computer Science*, 1(2), 2005.
- [120] Øystein Ore. Über höhere kongruenzen (German) [About higher congruences]. *Norsk Mat. Forenings Skrifter*, 1(7):15, 1922. (see [?, Theorem 6.13]).
- [121] Dimitris S. Papailiopoulos and Alexandros G. Dimakis. Locally repairable codes. *IEEE Trans. Inf. Theory*, 60(10):5843–5855, 2014.
- [122] Farzad Parvaresh and Alexander Vardy. Correcting errors beyond the guruswami-sudan radius in polynomial time. In *Proceedings of the 46th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 285–294, 2005.
- [123] Ruud Pellikaan and Xin-Wen Wu. List decoding of q-ary reed-muller codes. *IEEE Trans. Information Theory*, 50(4):679–682, 2004.
- [124] Larry L. Peterson and Bruce S. Davis. *Computer Networks: A Systems Approach*. Morgan Kaufmann Publishers, San Francisco, 1996.
- [125] W. Wesley Peterson. Encoding and error-correction procedures for Bose-Chaudhuri codes. *IEEE Transactions on Information Theory*, 6:459–470, 1960.
- [126] Michael O. Rabin. Probabilistic algorithms. In J. F. Traub, editor, *Algorithms and Complexity, Recent Results and New Directions*, pages 21–39, 1976.
- [127] Jaikumar Radhakrishnan and Amnon Ta-Shma. Bounds for dispersers, extractors, and depth-two superconcentrators. *SIAM J. Discret. Math.*, 13(1):2–24, 2000.
- [128] I. Reed. A class of multiple-error-correcting codes and the decoding scheme. *Transactions of the IRE Professional Group on Information Theory*, 4(4):38–49, 1954.
- [129] I. Reed, R. Scholtz, , and L. Welch. The fast decoding of reed-solomon codes using fermat theoretic transforms and continued fractions. *IEEE Transactions on Information Theory*, 24(1):100–106, January 1978.

- [130] Irving S. Reed. A class of multiple-error-correcting codes and the decoding scheme. *Trans. of the IRE Professional Group on Information Theory (TIT)*, 4:38–49, 1954.
- [131] Irving S. Reed and Gustav Solomon. Polynomial codes over certain finite fields. *SIAM Journal on Applied Mathematics*, 8(2):300–304, 1960.
- [132] T. Richardson and R. Urbanke. The capacity of low-density parity check codes under message-passing decoding. *IEEE Trans. Inform. Theory*, 47:599–618, February 2001.
- [133] Herbert Robbins. A remark on Stirling’s formula. *Amer. Math. Monthly*, 62:26–29, 1955.
- [134] Atri Rudra and Steve Uurtamo. Two theorems on list decoding. In *Proceedings of the 14th Intl. Workshop on Randomization and Computation (RANDOM)*, pages 696–709, 2010.
- [135] Atri Rudra and Mary Wootters. Every list-decodable code for high noise has abundant near-optimal rate puncturings. In *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 764–773, 2014.
- [136] E. Sasoglu, E. Telatar, and E. Arikan. Polarization for arbitrary discrete memoryless channels. In *2009 IEEE Information Theory Workshop*, pages 144–148, Oct 2009.
- [137] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, October 1980.
- [138] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.
- [139] Claude E. Shannon, Robert G. Gallager, and Elwyn R. Berlekamp. Lower bounds to error probability for coding on discrete memoryless channels. II. *Information and Control*, 10:65–103 (Part I), 522–552 (Part II), 1967.
- [140] Victor Shoup. New algorithms for finding irreducible polynomials over finite fields. *Math. Comp.*, 54:435–447, 1990.
- [141] Victor Shoup. *A computational introduction to number theory and algebra*. Cambridge University Press, 2006.
- [142] R. Singleton. Maximum distance q -nary codes. *Information Theory, IEEE Transactions on*, 10(2):116 – 118, apr 1964.
- [143] Michael Sipser. The history and status of the p versus np question. In *Proceedings of the Twenty-fourth Annual ACM Symposium on Theory of Computing*, STOC ’92, pages 603–618, New York, NY, USA, 1992. ACM.
- [144] Michael Sipser and Daniel Spielman. Expander codes. *IEEE Transactions on Information Theory*, 42(6):1710–1722, 1996.

- [145] David Slepian. A class of binary signaling alphabets. *The Bell System Technical Journal*, 35(1):203–234, 1956.
- [146] Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. *IEEE Transactions on Information Theory*, 42(6):1723–1731, 1996.
- [147] Jacques Stern. Approximating the number of error locations within a constant ratio is np -complete. In Gérard D. Cohen, Teo Mora, and Oscar Moreno, editors, *Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, 10th International Symposium, AAECC-10, San Juan de Puerto Rico, Puerto Rico, May 10-14, 1993, Proceedings*, volume 673 of *Lecture Notes in Computer Science*, pages 325–331. Springer, 1993.
- [148] Douglas R. Stinson. Universal hashing and authentication codes. *Des. Codes Cryptography*, 4(4):369–380, 1994.
- [149] Madhu Sudan. Decoding of Reed Solomon codes beyond the error-correction bound. *J. Complexity*, 13(1):180–193, 1997.
- [150] Madhu Sudan. List decoding: Algorithms and applications. *SIGACT News*, 31:16–27, 2000.
- [151] Madhu Sudan, Luca Trevisan, and Salil P. Vadhan. Pseudorandom generators without the XOR lemma. *J. Comput. Syst. Sci.*, 62(2):236–266, 2001.
- [152] Ido Tal and Alexander Vardy. How to construct polar codes. *IEEE Trans. Information Theory*, 59(10):6562–6582, 2013.
- [153] Itzhak Tamo and Alexander Barg. A family of optimal locally recoverable codes. *IEEE Trans. Inf. Theory*, 60(8):4661–4676, 2014.
- [154] R. M. Tanner. A recursive approach to low complexity codes. *IEEE Transactions on Information Theory*, 27(5):533–547, 1981.
- [155] Robert Endre Tarjan. Algorithmic design. *Commun. ACM*, 30(3):204–212, 1987.
- [156] Aimo Tietavainen. On the nonexistence theorems for perfect error-correcting codes. *SIAM Journal of Applied Mathematics*, 24(1):88–96, 1973.
- [157] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends® in Theoretical Computer Science*, 7(1–3):1–336, 2012.
- [158] Jacobus H. van Lint. Nonexistence theorems for perfect error-correcting codes. In *Proceedings of the Symposium on Computers in Algebra and Number Theory*, pages 89–95, 1970.
- [159] Alexander Vardy. The intractability of computing the minimum distance of a code. *IEEE Trans. Information Theory*, 43(6):1757–1766, 1997.

- [160] R. R. Varshamov. Estimate of the number of signals in error correcting codes. *Doklady Akadamii Nauk*, 117:739–741, 1957.
- [161] Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, 3 edition, 2013.
- [162] Lloyd R. Welch and Elwyn R. Berlekamp. Error correction of algebraic block codes. *US Patent Number 4,633,470*, December 1986.
- [163] John M. Wozencraft. List Decoding. *Quarterly Progress Report, Research Laboratory of Electronics, MIT*, 48:90–95, 1958.
- [164] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Proc. International Symp. of Symbolic and Algebraic Computation (EUROSAM)*, volume 72 of *LNCS*, pages 216–226. Springer, 1979.
- [165] V. V. Zyablov. An estimate of the complexity of constructing binary linear cascade codes. *Probl. Peredachi Inf.*, 7(1):5–13, 1971. English Translation in *Problems of Information Transmission*, 7:1(3–10), 1971.

Appendix A

Notation Table

$\mathbb{R}$	The set of real numbers	
$\mathbb{Z}$	The set of integers	
$\mathbb{Z}_+$	The set of positive integers	
$\mathbb{Z}_{\geq 0}$	The set of non-negative integers	
$\neg E$	Negation of the event E	
$\log x$	Logarithm to the base 2	
Σ^m	Vectors of length m with symbols from Σ	
$\mathbf{v}$	A row vector	
$\mathbf{v}^T$	Transpose of a row vector (i.e. a column vector)	
M^T	Transpose of a matrix M	
$\mathbf{0}$	The all zero vector	
$\mathbf{e}_i$	The i th standard vector, i.e. 1 in position i and 0 everywhere else	
$\mathbf{v}_S$	Vector $\mathbf{v}$ projected down to indices in S	
$\langle \mathbf{u}, \mathbf{v} \rangle$	Inner-product of vectors $\mathbf{u}$ and $\mathbf{v}$	
$[a, b]$	$\{x \in \mathbb{R} a \leq x \leq b\}$	
$[x]$	The set $\{1, \dots, x\}$	Section 1.2
n	Block length of a code	Definition 1.2.1
Σ	Alphabet of a code	Definition 1.2.1
q	$q = \Sigma $	Definition 1.2.1
k	Dimension of a code	Definition 1.2.3
R	Rate of a code	Definition 1.2.4
$\Delta(\mathbf{u}, \mathbf{v})$	Hamming distance between $\mathbf{u}$ and $\mathbf{v}$	Definition 1.3.3
$\delta(\mathbf{u}, \mathbf{v})$	Relative Hamming distance between $\mathbf{u}$ and $\mathbf{v}$	Definition 1.3.3
d	Minimum distance of a code	Definition 1.4.1
$wt(\mathbf{v})$	Hamming weight of $\mathbf{v}$	Definition 1.5.1
$B(\mathbf{x}, r)$	Hamming ball of radius r centered on $\mathbf{x}$	Definition 1.6.1
$Vol_q(r, n)$	Volume Hamming ball of radius r	Definition 3.3.2
$(n, k, d)_\Sigma$	A code with block length n , dimension k , distance d and alphabet Σ	Definition 1.7.1

$(n, k, d)_q$	A code with block length n , dimension k , distance d and alphabet size q	Definition 1.7.1
$[n, k, d]_q$	A linear $(n, k, d)_q$ code	Definition 2.3.1
$\mathbb{F}_q$	The finite field with q elements (q is a prime power)	Section 2.1
$\mathbb{F}^*$	The set of non-zero elements in the field $\mathbb{F}$	
$\mathbb{F}_q^{m \times N}$	The set of all $m \times N$ matrices where each entry is from $\mathbb{F}_q$	
$\mathbb{F}_q[X_1, \dots, X_m]$	The set of all m -variate polynomials with coefficients from $\mathbb{F}_q$	
$R(C)$	Rate of a code family C	Definition 1.8.1
$\delta(C)$	Relative distance of a code family C	Definition 1.8.1
$\mathcal{U}$	The uniform distribution	Definition 3.1.1
$\mathbb{E}[V]$	Expectation of a random variable V	Definition 3.1.2
$\mathbb{1}_E$	Indicator variable for event E	Section 3.1
$H_q(x)$	$x \log_q(q-1) - x \log_q x - (1-x) \log_q(1-x)$	Definition 3.3.1
$H_q^{-1}(y)$	Unique $x \in [0, 1-1/q]$ such that $H_q(x) = y$	Section 3.3.2
$\deg(P)$	Degree of polynomial $P(X)$	Definition 5.1.1
$\mathbb{F}_q[X]$	The set of all univariate polynomials in X over $\mathbb{F}_q$	Section 5.1
$J_q(x)$	$(1-1/q)(1-\sqrt{1-qx/(q-1)})$	Theorem 7.3.1
$\binom{S}{t}$	$\{T \subseteq S \mid T = t\}$	
$\bar{S}$	The complement set of S	
$M \odot \mathbf{x}$	Binary matrix-vector multiplication where multiplication is AND and addition is OR	
$\text{supp}(X)$	The support of a random variable X	Definition E.1.2

Appendix B

Some Useful Facts

B.1 Some Useful Inequalities

Recall that the binomial coefficient for integers $a \leq b$, defined as

$$\binom{b}{a} = \frac{b!}{a!(b-a)!}.$$

We begin with a simple lower bound on the binomial coefficient:

Lemma B.1.1. *For all integers $1 \leq a \leq b$, we have*

$$\binom{b}{a} \geq \left(\frac{b}{a}\right)^a.$$

Proof. The following sequence of relations completes the proof:

$$\binom{b}{a} = \prod_{i=0}^{a-1} \frac{b-i}{a-i} \geq \prod_{i=0}^{a-1} \frac{b}{a} = \left(\frac{b}{a}\right)^a.$$

In the above, the first equality follows from definition and the inequality is true since $b \geq a$ and $i \geq 0$. $\square$

We state the next set of inequalities without proof (see [133] for a proof):

Lemma B.1.2 (Stirling's Approximation). *For every integer $n \geq 1$, we have*

$$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\lambda_1(n)} < n! < \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\lambda_2(n)},$$

where

$$\lambda_1(n) = \frac{1}{12n+1} \text{ and } \lambda_2(n) = \frac{1}{12n}.$$

We prove another inequality involving Binomial coefficient.

Lemma B.1.3. For all integers $1 \leq a \leq b$, we have

$$\binom{b}{a} \leq \left(\frac{eb}{a}\right)^a.$$

Proof. First note that

$$\binom{b}{a} = \frac{b(b-1)\cdots(b-a+1)}{a!} \leq \frac{b^a}{a!}.$$

The final bound follows from the fact that

$$a! > \left(\frac{a}{e}\right)^a,$$

which in turns follows from the following relationships:

$$\frac{a^a}{a!} < \sum_{i=0}^{\infty} \frac{a^i}{i!} = e^a.$$

□

We next state Bernoulli's inequality:

Lemma B.1.4 (Bernoulli's Inequality). For every real numbers $k \geq 1$ and $x \geq -1$, we have

$$(1+x)^k \geq 1+kx.$$

Proof Sketch. We only present the proof for integer k . For the full proof see e.g. [24].

For the base case of $k = 1$, the inequality holds trivially. Assume that the inequality holds for some integer $k \geq 1$ and to complete the proof, we will prove it for $k+1$. Now consider the following inequalities:

$$\begin{aligned} (1+x)^{k+1} &= (1+x) \cdot (1+x)^k \\ &\geq (1+x) \cdot (1+kx) \\ &= 1 + (k+1)x + kx^2 \\ &\geq 1 + (k+1)x, \end{aligned}$$

as desired. In the above, the first inequality follows from the inductive hypothesis and the second inequality follows from the fact that $k \geq 1$. □

Lemma B.1.5. For $|X| \leq 1$,

$$\sqrt{1+x} \leq 1 + \frac{x}{2} - \frac{x^2}{16}.$$

Proof. Squaring the RHS we get

$$\left(1 + \frac{x}{2} - \frac{x^2}{16}\right)^2 = 1 + \frac{x^2}{4} + \frac{x^4}{256} + x - \frac{x^2}{16} - \frac{x^3}{32} = 1 + x + \frac{3x^2}{16} - \frac{x^3}{32} + \frac{x^4}{256} \geq 1+x,$$

as desired. □

We will also use the Cauchy-Schwarz inequality:

Lemma B.1.6. For any vector $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, we have

$$|\langle \mathbf{x}, \mathbf{z} \rangle| \leq \|\mathbf{x}\|_2 \cdot \|\mathbf{z}\|_2.$$

B.2 Some Useful Identities and Bounds

We start off with an equivalence between two inequalities.

Lemma B.2.1. Let $a, b, c, d > 0$. Then $\frac{a}{b} \leq \frac{c}{d}$ if and only if $\frac{a}{a+b} \leq \frac{c}{c+d}$.

Proof. Note that $\frac{a}{b} \leq \frac{c}{d}$ if and only if

$$\frac{b}{a} \geq \frac{d}{c}.$$

The above is true if and only if

$$\frac{b}{a} + 1 \geq \frac{d}{c} + 1,$$

which is same as $\frac{a}{a+b} \leq \frac{c}{c+d}$. □

Next, we state some infinite sums that are identical to certain logarithms (the proofs are standard and are omitted).

Lemma B.2.2. For $|x| < 1$,

$$\ln(1+x) = x - \frac{x^2}{2!} + \frac{x^3}{3!} - \cdots.$$

We can use the above to prove some bounds on $\ln(1+x)$ (we omit the proof):

Lemma B.2.3. For $0 \leq x < 1$, we have

$$x - x^2/2 \leq \ln(1+x) \leq x,$$

and for $0 \leq x \leq 1/2$, we have

$$-x - x^2 \leq \ln(1-x) \leq -x.$$

We can use the above bounds to further prove boounds on the (binary) entropy function:

Lemma B.2.4. For $x \leq 1/4$, we have

$$1 - 5x^2 \leq H(1/2 - x) \leq 1 - x^2.$$

Proof. By definition $H(1/2 - x) = 1 - 1/2 \log(1 - 4x^2) + x \log(1 - 2x)/(1 + 2x)$, and using the approximations for $\ln(1+x)$ from Lemma B.2.3, we have, for $x < 1/4$,

$$\begin{aligned} H(1/2 - x) &\leq 1 + \frac{1}{2\ln 2} \cdot (4x^2 + 16x^4) + \frac{1}{\ln 2} \cdot (-2x^2) - \frac{1}{\ln 2} \cdot (2x^2 - 2x^3) \\ &= 1 - \frac{2}{\ln 2} \cdot x^2 + \frac{2}{\ln 2} \cdot x^3 + \frac{8}{\ln 2} \cdot x^4 \\ &\leq 1 - \frac{x^2}{\ln 2} \\ &\leq 1 - x^2. \end{aligned} \tag{B.1}$$

In the above, (B.1) follows by using our assumption that $x \leq 1/4$.

Using the other sides of the approximations we also have:

$$\begin{aligned} H(1/2 - x) &\geq 1 + \frac{1}{2\ln 2} \cdot (4x^2) + \frac{1}{\ln 2} \cdot (-2x^2 - 4x^3) - \frac{1}{\ln 2} \cdot (2x^2) \\ &\geq 1 - \frac{3x^2}{\ln 2} \\ &\geq 1 - 5x^2, \end{aligned}$$

where the second inequality uses our assumption that $x \leq 1/4$. □

The following fact follows from the well-known fact that $\lim_{x \rightarrow \infty} (1 + 1/x)^x = e$:

Lemma B.2.5. *For every real $x > 0$,*

$$\left(1 + \frac{1}{x}\right)^x \leq e.$$

Appendix C

Background on Asymptotic notation, Algorithms and Complexity

In this chapter, we collect relevant background on algorithms and their analysis (as well as their limitations). We begin with notation that we will use to bound various quantities when we do not pay close attention to the constants.

C.1 Asymptotic Notation

Throughout the book, we will encounter situations where we would be interested in how a function $f(N)$ grows as the input parameter N grows. (We will assume that the real valued function f is monotone.) The most common such situation is when we would like to bound the runtime of an algorithm we are analyzing— we will consider this situation in some detail shortly. In particular, we will be interested in bounds on $f(N)$ that are “oblivious” to constants. E.g. given that an algorithm takes $24N^2 + 100N$ steps to terminate, we would be interested in the fact that the dominating term is the N^2 (for large enough N). Technically, speaking we are interested in the *asymptotic* growth of $f(N)$. Throughout this chapter, we will assume that all functions are monotone.

The first definition is when we are interested in an upper bound on the function $f(N)$. When talking about numbers, we say b is an upper bound on a if $a \leq b$. We will consider a similar definition for functions that in some sense ignores constants.

Definition C.1.1. *We say $f(N)$ is $O(g(N))$ (to be read as $f(N)$ is “Big-Oh” of $g(N)$) if there exists constants $c, N_0 \geq 0$ that are independent of N such that for every large enough $N \geq N_0$:*

$$f(N) \leq c \cdot g(N).$$

Alternatively $f(N)$ is $O(g(N))$ if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} \leq C,$$

for some absolute constant C . (See Exercise C.1.) So for example both $24N^2 + 100N$ and $N^2/2 - N$ are $O(N^2)$ as well as $O(N^3)$. However, neither of them are $O(N)$ or $O(N^{3/2})$.

The second definition is when we are interested in a lower bound on the function $f(N)$. When talking about numbers, we say b is a lower bound on a if $a \geq b$. We will consider a similar definition for functions that in some sense ignores constants.

Definition C.1.2. We say $f(N)$ is $\Omega(g(N))$ (to be read as $f(N)$ is “Big-Omega” of $g(N)$) if there exists constants $\epsilon, N_0 \geq 0$ that are independent of n such that for every large enough $N \geq N_0$:

$$f(N) \geq \epsilon \cdot g(N).$$

Alternatively $f(N)$ is $\Omega(g(N))$ if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} \geq C,$$

for some absolute constant C . (See Exercise C.2.) So for example both $24N^2 + 100N$ and $N^2/2 - N$ are $\Omega(N^2)$ as well as $\Omega(N^{3/2})$. However, neither of them are $\Omega(N^3)$ or $\Omega(N^{5/2})$.

The third definition is when we are interested in a tight bound on the function $f(N)$. When talking about numbers, we say b is same as a if $a = b$. We will consider a similar definition for functions that in some sense ignores constants.

Definition C.1.3. We say $f(N)$ is $\Theta(g(N))$ (to be read as $f(N)$ is “Theta” of $g(N)$) if and only if $f(N)$ is $O(g(N))$ and is also $\Omega(g(N))$.

Alternatively $f(N)$ is $\Theta(g(N))$ if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = C,$$

for some absolute constant C . (See Exercise C.3.) So for example both $24N^2 + 100N$ and $N^2/2 - N$ are $\Theta(N^2)$. However, neither of them are $\Theta(N^3)$ or $\Theta(N)$.

The fourth definition is when we are interested in a strict upper bound on the function $f(N)$. When talking about numbers, we say b is a strict upper bound on a if $a < b$. We will consider a similar definition for functions that in some sense ignores constants.

Definition C.1.4. We say $f(N)$ is $o(g(N))$ (to be read as $f(N)$ is “little-oh” of $g(N)$) if $f(N)$ is $O(g(N))$ but $f(N)$ is not $\Omega(g(N))$.

Alternatively $f(N)$ is $o(g(N))$ if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = 0.$$

(See Exercise C.4.) So for example both $24N^2 + 100N$ and $N^2/2 - N$ are $o(N^3)$ as well as $o(N^{5/2})$. However, neither of them are $o(N^2)$ or $o(N^{3/2})$.

The final definition is when we are interested in a strict lower bound on the function $f(N)$. When talking about numbers, we say b is a strict lower bound on a if $a > b$. We will consider a similar definition for functions that in some sense ignores constants.

Definition C.1.5. We say $f(N)$ is $\omega(g(N))$ (to be read as $f(N)$ is “little-omega” of $g(N)$) if $f(N)$ is $\Omega(g(N))$ but $f(N)$ is not $O(g(N))$.

Alternatively $f(N)$ is $\omega(g(N))$ if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = \infty.$$

(See Exercise C.5.) So for example both $24N^2 + 100N$ and $N^2/2 - N$ are $\omega(N)$ as well as $\omega(N^{3/2})$. However, neither of them are $\omega(N^2)$ or $\omega(N^{5/2})$.

C.1.1 Some Properties

We now collect some properties of asymptotic notation that we will be useful in this book.

First all the notations are transitive:

Lemma C.1.6. Let $\alpha \in \{O, \Omega, \Theta, o, \omega\}$. Then if $f(N)$ is $\alpha(g(N))$ and $g(N)$ is $\alpha(h(N))$, then $f(N)$ is $\alpha(h(N))$.

Second, all the notations are additive:

Lemma C.1.7. Let $\alpha \in \{O, \Omega, \Theta, o, \omega\}$. Then if $f(N)$ is $\alpha(h(N))$ and $g(N)$ is $\alpha(h(N))$, then $f(N) + g(N)$ is $\alpha(h(N))$.

Finally, all the notations are multiplicative:

Lemma C.1.8. Let $\alpha \in \{O, \Omega, \Theta, o, \omega\}$. Then if $f(N)$ is $\alpha(h_1(N))$ and $g(N)$ is $\alpha(h_2(N))$, then $f(N) \cdot g(N)$ is $\alpha(h_1(N) \cdot h_2(N))$.

The proofs of the above properties are left as an exercise (see Exercise C.6).

C.2 Bounding Algorithm run time

Let $\mathcal{A}$ be the algorithm we are trying to analyze. Then we will define $T(N)$ to be the worst-case run-time of $\mathcal{A}$ over all inputs of size N . Slightly more formally, let $t_{\mathcal{A}}(\mathbf{x})$ be the number of steps taken by the algorithm $\mathcal{A}$ on input $\mathbf{x}$. Then

$$T(N) = \max_{\mathbf{x}: \mathbf{x} \text{ is of size } N} t_{\mathcal{A}}(\mathbf{x}). \quad (\text{C.1})$$

In this section, we present two useful strategies to prove statements like $T(N)$ is $O(g(N))$ or $T(N)$ is $\Omega(h(N))$. Then we will analyze the run time of a very simple algorithm. However, before that we digress to clarify the following: (i) For most of the book, we will be interested in deterministic algorithms (i.e. algorithm whose execution is fixed given the input). However, we will consider randomized algorithms (see Section C.3 for more on this). (ii) One needs to clarify what constitutes a “step” in the definition of $T(N)$ above. We do so next.

C.2.1 RAM model

In this book, unless specified otherwise we will assume that the algorithms run on the RAM model. Informally, this computation model is defined as follows. For an input with n items, the memory consists of registers with $O(\log n)$ bits. For simplicity, we can assume that the input and output have separate dedicated registers. Note that the input will have n dedicated registers.

Any (atomic) step an algorithm can take are essentially any basic operations on constant such registers which can be implemented in $O(\log n)$ bit operations. In particular, the following operations are considered to take one step: loading $O(\log n)$ from a register or storing $O(\log n)$ bits in a register, initializing the contents of a register, bit-wise operations among registers, e.g. taking bit-wise XOR of the bits of two registers, adding numbers stored in two registers, incrementing the value stored in a register, comparing the values stored in two registers. Some examples of operations that are *not* single step operations: multiplying numbers or exponentiation (where the operands fit into one register each).

C.2.2 Proving $T(N)$ is $O(f(N))$

We start off with an analogy. Say you wanted prove that given m numbers $a_1, \dots, a_m$, $\max_i a_i \leq U$. Then how would you go about doing so? One way is to argue that the maximum value is attained at i^* and then show that $a_{i^*} \leq U$. Now this is a perfectly valid way to prove the inequality we are after but note that you will *also* have to prove that the maximum value is attained at i^* . Generally, this is a non-trivial task. However, consider the following strategy:

Show that for *every* $1 \leq i \leq m$, $a_i \leq U$. Then conclude that $\max_i a_i \leq U$.

Let us consider an example to illustrate the two strategies above. Let us say for whatever reason we are interested in showing that the age of the oldest person in your coding theory lectures is at most 100. Assume there are 98 students registered and the instructor is always present in class. This implies that there are at most $m = 99$ folks in the class. Let us order them somehow and let a_i denote the age of the i 'th person. Then we want to show that $\max\{a_1, \dots, a_{99}\} \leq 100$ (i.e. $U = 100$). The first strategy above would be to first figure out who is the oldest person in room: say that is the i^* 'th person (where $1 \leq i^* \leq 99$) and then check if $a_{i^*} \leq 100$. However, this strategy is somewhat invasive: e.g. the oldest person might not want to reveal that he or she is the oldest person in the room. This is where the second strategy works better: we ask every person in the room if their age is ≤ 100 : i.e. we check if for every $1 \leq i \leq 99$, $a_i \leq 100$. If everyone says yes, then we have proved that $\max_i a_i \leq 100$ (without necessarily revealing the identity of the oldest person).

Mathematically the above two strategies are the same. However, in "practice," using the second strategy turns out to be much easier. (E.g. this was true in the age example above.) Thus, here is the strategy to prove that $T(N)$ is $O(f(N))$:

For every large enough N , show that for **every** input $\mathbf{x}$ of size N , $t_{\mathcal{A}}(\mathbf{x})$ is $O(f(N))$. Then conclude that $T(N)$ is $O(f(N))$.

C.2.3 Proving $T(N)$ is $\Omega(f(N))$

We start off with the same analogy as in the previous section. Say you wanted prove that given m numbers $a_1, \dots, a_m$, $\max_i a_i \geq L$. Then how would you go about doing so? Again, one way is to argue that the maximum value is attained at i^* and then show that $a_{i^*} \geq L$. Now this is a perfectly valid way to prove the inequality we are after but note that you will *also* have to prove that the maximum value is attained at i^* . Generally, this is a non-trivial task. However, consider the following strategy:

Show that there *exists* an $1 \leq i \leq m$, such that $a_i \geq L$. Then conclude that $\max_i a_i \geq L$.

Let us go back to the class room example. Now let us say we are interesting in proving that the oldest person in the room is at least 25 years old. (So $a_1, \dots, a_m$ is as in Section C.2.2 but now $L = 25$.) Again, the first strategy would be to first figure out the oldest person, say i^* and check if $a_{i^*} \geq 25$. However, as we saw in Section C.2.2, this strategy is somewhat invasive. However, consider the the following implementation of the second strategy above. Say for the sake of mathematics, the instructor comes forward and volunteers the information that her age is at least 25. Since the oldest person's age has to be at least the instructor's age, this proves that $\max_i a_i \geq 25$, as desired.

Mathematically the above two strategies are the same. However, in "practice," using the strategy second turns out to be much easier. (E.g., this was true in the age example above.) Thus, here is the strategy to prove that $T(N)$ is $\Omega(f(N))$:

For every large enough N , show that there **exists** an input $\mathbf{x}$ of size N , $t_{\mathcal{A}}(\mathbf{x})$ is $\Omega(f(N))$. Then conclude that $T(N)$ is $\Omega(f(N))$.

C.2.4 An Example

Now let us use all the strategies from Section C.2.2 and Section C.2.3 to asymptotically bound the run-time of a simple algorithm. Consider the following simple problem: given $n+1$ numbers $a_1, \dots, a_n; v$, we should output $1 \leq i \leq n$ if $a_i = v$ (if there are multiple such i 's then output any one of them) else output -1 . Below is a simple algorithm to solve this problem.

Algorithm C.2.1 Simple Search

INPUT: $a_1, \dots, a_n; v$

OUTPUT: i if $a_i = v$; -1 otherwise

```

1: FOR every  $1 \leq i \leq n$  DO
2:   IF  $a_i = v$  THEN RETURN  $i$ 
3: RETURN  $-1$ 
```

We will show the following:

Theorem C.2.1. *The Simple Search algorithm C.2.1 has a run time of $\Theta(n)$.*

We will prove Theorem C.2.1 by proving Lemmas C.2.2 and C.2.3.

Lemma C.2.2. $T(n)$ for Algorithm C.2.1 is $O(n)$.

Proof. We will use the strategy outlined in Section C.2.2. Let $a_1, \dots, a_n; v$ be an arbitrary input. Then first note that there are at most n iterations of the for loop in Step 1. Further, each iteration of the for loop (i.e. Step 2) can be implemented in $O(1)$ time (since it involves one comparison and a potential return of the output value). Thus, by Lemma C.1.8, the total times taken overall in Steps 1 and 2 is given by

$$T_{12} \leq O(n \cdot 1) = O(n).$$

Further, since Step 3 is a simple return statement, it takes time $T_3 = O(1)$ time. Thus, we have that

$$t_{\text{Algorithm C.2.1}}(a_1, \dots, a_n; v) = T_{12} + T_3 \leq O(n) + O(1) \leq O(n),$$

where the last inequality follows from Lemma C.1.7 and the fact that $O(1)$ is also $O(n)$. Since the choice of $a_1, \dots, a_n; v$ was arbitrary, the proof is complete. $\square$

Lemma C.2.3. $T(n)$ for Algorithm C.2.1 is $\Omega(n)$.

Proof. We will follow the strategy laid out in Section C.2.3. For every $n \geq 1$, consider the specific input $a'_i = n+1-i$ (for every $1 \leq i \leq n$) and $v' = 1$. For this specific input, it can be easily checked that the condition in Step 2 is only satisfied when $i = n$. In other words, the for loop runs at least (actually exactly) n times. Further, each iteration of this loop (i.e. Step 2) has to perform at least one comparison, which means that this step takes $\Omega(1)$ time. Since n is $\Omega(n)$, by Lemma C.1.8 (using notation from the proof of Lemma C.2.2), we have

$$T_{12} \geq \Omega(n \cdot 1) = \Omega(n).$$

Thus, we have

$$t_{\text{Algorithm C.2.1}}(a'_1, \dots, a'_n; v') \geq T_{12} \geq \Omega(n).$$

Since we have shown the existence of one input for each $n \geq 1$ for which the run-time is $\Omega(n)$, the proof is complete. $\square$

A quick remark on the proof of Lemma C.2.3. Since by Section C.2.3, we only need to exhibit only *one* input with runtime $\Omega(n)$, the input instance in the proof of Lemma C.2.3 is only one possibility. One can choose other instances: e.g. we can choose an instance where the output has to be -1 (as a specific instance consider $a_i = i$ and $v = 0$). For this instance one can make a similar argument as in the proof of Lemma C.2.3 to show that $T(n) \geq \Omega(n)$.

C.2.5 The Best-Case Input “Trap”

We now briefly talk about a common mistake that is made when one starts trying to prove $\Omega(\cdot)$ on $T(N)$. Note that in Section C.2.3, it says that one can prove that $T(N)$ to be $\Omega(f(N))$ for every large enough N , one only needs to pick *one* input of size N for which the algorithm takes $\Omega(f(N))$ steps.

The confusing part about the strategy in Section C.2.3 is how does one get a hand on that special input that will prove the $\Omega(f(N))$ bound. There is no mechanical way of finding this input. Generally, speaking you have to look at the algorithm and get a feel for what input might force the algorithm to spend a lot of time. Sometimes, the analysis of the $O(\cdot)$ bound itself gives us a clue.

However, one way of picking the “special” input that **almost always never works in practice** is to consider (for every large enough N), the “best-case input,” i.e. an input of size N on which the algorithm runs very fast. Now such an input will give you a valid lower bound but it would almost never give you a *tight* lower bound.

So for example, let us try to prove Lemma C.2.3 using the best case input. Here is one best case input: $a_i = i$ for every $i \in [n]$ and $v = 1$. Note that in this case the algorithm finds a match in the first iteration and this terminates in constant many steps. Thus, this will prove an $\Omega(1)$ lower bound but that is not tight/good enough.

Another common mistake is to make an argument for a fixed value of N (say $N = 1$). However, note that in this case one can never prove a bound better than $\Omega(1)$ and again, this trick never works in proving any meaningful lower bound.

C.3 Randomized Algorithms

So far the algorithms we have considered are deterministic, i.e. these are algorithm whose behavior is completely determined once the input is fixed. We now consider a generalization of such algorithms to algorithms that have access to random bits. In particular, even when the input is fixed, the behavior of the algorithm might change depending on the actual value of the random bits.¹ For the machine model, it is easy to modify the RAM model from Section C.2.1 to handle randomized algorithms: we can always load a register with independent and uniform random bits in one step.

Typically one considers randomized algorithms due to the following reasons:

- For some problems, it is easier to think of a randomized algorithm. Once one has designed a randomized algorithm, one could then attempt to “derandomize” the randomized algorithm to construct deterministic algorithms.
- In addition to conceptual simplicity, a randomized algorithm might run faster than all corresponding known deterministic algorithms.
- For certain problems, it might be provably impossible to design deterministic algorithms with certain guarantees but it is possible to design randomized algorithms with such guarantees. This is a common situation when we might be interested in algorithms that run in sub-linear time.

¹There are many fundamental and interesting questions regarding how truly random these random bits are and how many such bits can an algorithm access. We will consider the ideal case, where an algorithm has access to as many uniform and independent random bits as it wants.

In this section, we will consider a problem where the third scenario above is applicable. For examples of the first and second scenarios, see Sections 14.3 and D.6 respectively.

Before delving into an example problem, we would like to clarify how we determine the run time and correctness of a randomized algorithm. There are multiple natural definitions but we will consider the following ones. The run time of a randomized algorithm will again be the worst-case run time as we defined for deterministic algorithms in Section C.2 for every possible choice of internal random bits that the algorithm might use (with the modification to the RAM model as discussed above). For correctness, the definition for deterministic algorithm was obvious so we did not explicitly state it: for every input, a deterministic algorithm must return the correct output. We call a randomized algorithm correct if on all its inputs, it returns the correct answer with probability bounded away from a 1/2– to be precise let us say it has to return the correct output with probability at least 2/3.²

We would like to remark on a subtle point in the definition above. In the definition of the correctness of a randomized algorithm above, the probability is taken over the random coin tosses that the algorithm might make. However, note that the guarantee is of the worst-case flavor in the sense that the algorithm has to be correct with high probability for *every* input. This should be contrasted with a scenario where the input might itself be random in which case we might be happy with an average case guarantee where the algorithm is supposed to return the correct output with high probability (over the distribution over the input). In particular, the algorithm is allowed to err on certain inputs as long as the total probability mass on the inputs on which it is incorrect is small: see Chapter 6 where this definition of correctness makes perfect sense. In such situations one can always assume that the algorithm itself is deterministic (see Exercise C.9).

C.3.1 An example problem

In the rest of the section, we will consider the following problem and will attempt to design (deterministic and randomized) algorithms with an eye to illustrate various points that were raised when we defined randomized algorithms.

Given a vector $\mathbf{x} \in \{0, 1\}^n$ determine whether $wt(\mathbf{x}) \leq \frac{n}{3}$ or $wt(\mathbf{x}) \geq \frac{2n}{3}$. For the cases where $wt(\mathbf{x}) \in (n/2, 2n/3)$ the algorithm can have arbitrary behavior.³

We will refer to the above as the GAPHAMMING problem.

It is easy to design an $O(n)$ time deterministic algorithm to solve GAPHAMMING: In $O(n)$ one can compute $wt(\mathbf{x})$ and then in $O(1)$ time one can verify if $wt(\mathbf{x}) \leq n/3$ or $wt(\mathbf{x}) \geq 2n/3$. In addition one can show that *any* correct deterministic algorithm will need a run time of $\Omega(n)$: see Exercise C.10.

We will now design a randomized algorithm that solves GAPHAMMING problem. Recall that we only need to determine if $wt(\mathbf{x}) \leq n/3$ or $wt(\mathbf{x}) \geq 2n/3$ (note that we assumed we do not get

²The choice of 2/3 was arbitrary: see Exercise C.8.

³Or equivalently one can assume that the algorithm is given the *promise* that it will never encounter an input $\mathbf{x}$ with $wt(\mathbf{x}) \in (n/3, 2n/3)$.

inputs with Hamming weight in $(n/3, 2n/3)$ with high probability. We will present what is called a *sampling* algorithm for this task. To gain intuition, pick a random index $i \in [n]$. Note that then x_i is a random bit. Further, if $wt(\mathbf{x}) \leq n/3$, then $\Pr_i[x_i = 1] \leq 1/3$. On the other hand, if $wt(\mathbf{x}) \geq 2n/3$, then the probability is at least 2/3. Thus, if we take s samples, with high probability in the first case we expect to see less than $s/3$ ones and in the second case we expect to see at least $2s/3$ ones. To get a constant probability of success we will invoke Chernoff bound to bound the probability of seeing more ones in the first case than the second case. Algorithm C.3.1 for the details.

Algorithm C.3.1 Sampling algorithm for GAPHAMMING

INPUT: $\mathbf{x} \in \{0, 1\}^n$

OUTPUT: 0 if $wt(\mathbf{x}) \leq n/3$ and 1 if $wt(\mathbf{x}) \geq 2n/3$ with probability at least $1 - \epsilon$

```

1:  $s \leftarrow 98 \cdot \ln(1/\epsilon)$ 
2:  $C \leftarrow 0$ 
3: FOR  $j \in [s]$  DO
4:   Pick  $i$  to be a random index from  $[n]$            ▷ The choice of  $i$  is independent for each  $j$ 
5:    $C \leftarrow C + x_i$ 
6: IF  $C < s/2$  THEN
7:   RETURN 0
8: RETURN 1
```

It can be checked that Algorithm C.3.1 runs in time $O(\log(1/\epsilon))$: see Exercise C.11. Next we argue that the algorithm is correct with probability at least $1 - \epsilon$.

Lemma C.3.1. *Algorithm C.3.1 outputs the correct answer with probability at least $1 - \epsilon$ for every $\mathbf{x}$ (such that $wt(\mathbf{x}) \notin (n/3, 2n/3)$).*

Proof. We will prove the lemma for the case when $wt(\mathbf{x}) \leq n/3$ and leave the other case to Exercise C.12.

Fix an arbitrary input $\mathbf{x}$ such that $wt(\mathbf{x}) \leq n/3$. We will argue that at Step 6, we have

$$\Pr \left[C \geq \frac{s}{3} + \frac{s}{7} \right] \leq \epsilon. \quad (\text{C.2})$$

Note that the above is enough to prove that the algorithm will output 0, as desired.

Towards that end for every $j \in [s]$, let Y_j be the random bit x_i that is picked. Note that $C = \sum_{j=1}^s Y_j$. Since each of the Y_j 's are independent binary random variables, the additive Chernoff bound (Theorem 3.1.10) implies that

$$\Pr \left[C > \mathbb{E}[C] + \frac{s}{7} \right] \leq e^{-\frac{s}{7^2 \cdot 2}} \leq \epsilon,$$

where the last inequality follows from our choice of s . As observed earlier for any j , $\Pr[Y_j = 1] \leq 1/3$, which implies that $\mathbb{E}[C] \leq s/3$, which with the above bound implies (C.2), as desired. $\square$

Finally, we consider the average-case version of the GAPHAMMING problem. Our goal is to illustrate the difference between randomized algorithms and average-case algorithms that was alluded to earlier in this section. Recall that in the GAPHAMMING problem, we are trying to distinguish between two classes of inputs: one with Hamming weight at most $n/3$ and the other with Hamming weight at least $2n/3$. We now consider the following natural version where the inputs themselves comes from two distributions and our goal is to distinguish between the two cases.

Let $\mathbb{D}_p$ denote the distribution on $\{0, 1\}^n$, where each bit is picked independently with probability $0 \leq p \leq 1$. Given an $\mathbf{x} \in \{0, 1\}^n$ sampled from either $\mathbb{D}_{\frac{1}{3}}$ or $\mathbb{D}_{\frac{2}{3}}$, we need to figure out which distribution $\mathbf{x}$ is sampled from.

The intuition for an algorithm that is correct with high probability (over the corresponding distributions) is same as Algorithm C.3.1, so we directly present the the algorithm for the new version of the problem above.

Algorithm C.3.2 An average-case algorithm for GAPHAMMING

INPUT: $\mathbf{x} \in \{0, 1\}^n$ sampled from either $\mathbb{D}_{\frac{1}{3}}$ or $\mathbb{D}_{\frac{2}{3}}$

OUTPUT: 0 if $\mathbf{x}$ was sampled from $\mathbb{D}_{\frac{1}{3}}$ and 1 otherwise with probability at least $1 - \varepsilon$

```

1:  $s \leftarrow 98 \cdot \ln(1/\varepsilon)$ 
2:  $C \leftarrow 0$ 
3: FOR  $j \in [s]$  DO
4:    $C \leftarrow C + x_j$ 
5: IF  $C < s/2$  THEN
6:   RETURN 0
7: RETURN 1
```

Note that unlike Algorithm C.3.1, Algorithm C.3.2 is a *deterministic* algorithm and the algorithm might make an incorrect decision on certain specific inputs $\mathbf{x}$ that it receives.

Using pretty much the same analysis as in the proof of Lemma C.3.1, one can argue that:

Lemma C.3.2. *Let $\mathbf{x}$ be a random sample from $\mathbb{D}_{\frac{1}{3}}$ ($\mathbb{D}_{\frac{2}{3}}$ resp.). Then with probability at least $1 - \varepsilon$ (over the choice of $\mathbf{x}$), Algorithm C.3.2 outputs 0 (1 resp.)*

(See Exercise C.13 for a proof.)

C.4 Efficient Algorithms

A major focus of this book is to design algorithms that are efficient. A somewhat smaller focus is to argue that for certain problems efficient algorithms do not exist (maybe with a well accepted assumption that certain computational tasks are hard to accomplish efficiently). In this section, we first begin with the notion of efficient algorithms that will be standard for this book and then

present a peek into how one might argue that a computational task is hard. To illustrate various concepts we will focus on the following problem:

Definition C.4.1. *Given n linear equations over k variables (all over $\mathbb{F}_2$: i.e. all the variables are in $\{0, 1\}$ and all arithmetic operations are over the binary field⁴ $\mathbb{F}_2$) and an integer $0 \leq s \leq n$, we want to find a solution to the systems of equations that satisfies at least s out of the n equations. We will denote this problem as MAXLINEAREQ(k, n, s). We will drop the arguments when we want to talk about the problem in general.*

We choose the non-standard notation of k for number of variables and n for number of equations as they correspond better to problems in coding theory that we would be interested in.

An overwhelming majority of the algorithmic problems considered in this book will have the following property: there are exponentially many possible solutions and we are interested in a solution (or solutions) that satisfy a certain objective. For example, in the MAXLINEAREQ problem, there are 2^k possible solutions and we are interested in a solution that satisfies at least s many linear equations. Note that such problems have a very natural exponential time algorithm: generate all (the exponentially many) potential solutions and check if the current potential solution satisfy the objective. If it does, then the algorithm stops. Otherwise the algorithm continues to the next solution. For example, Algorithm C.4.1 instantiates this general algorithm for the MAXLINEAREQ problem.

Algorithm C.4.1 Exponential time algorithm for MAXLINEAREQ

INPUT: n linear equations over k variables and an integer $0 \leq s \leq n$

OUTPUT: A solution in $\{0, 1\}^k$ that satisfies at least s of the equations or fail if none exists

```

1: FOR every  $\mathbf{x} \in \{0, 1\}^k$  DO
2:   Let  $t$  be the number of equations the solution  $\mathbf{x}$  satisfies
3:   IF  $t \geq s$  THEN
4:     RETURN  $\mathbf{x}$ 
5: RETURN fail
```

It is not too hard to argue that Algorithm C.4.1 runs in time $O(kn2^k)$ (see Exercise C.14). We point out two things that will expand into more detailed discussion on what is an efficient algorithm (and what is not):

1. A run time of $\Omega(2^k)$ is not efficient for even moderate values of k : indeed for $k = 100$, the number of steps of the algorithm exceeds the number of particles in the universe.
2. In the generic exponential time algorithm mentioned earlier, we made the implicit assumption that given a potential solution we can “quickly” verify if the potential solution satisfies the objective or not.

⁴In other words, addition is XOR and multiplication is AND.

Notwithstanding the fact that an exponential run time can become infeasible for moderate input size, one might think that one cannot do better than Algorithm C.4.1 to solve the MAXLINEAREQ problem. In particular, the lack of any extra information other than the fact that we have a system of linear equations on our hands seems to make the possibility of coming up with a faster algorithm slim. However, looks can sometimes be deceptive, as we will see shortly.

Consider the special case of the MAXLINEAREQ problem: MAXLINEAREQ(k, n, n). In other words, we want to see if there exists a solution $\mathbf{x} \in \{0, 1\}^n$ that satisfies all the equations. Not only is this an interesting special case but it is also relevant to this book since this setting corresponds to the error detection problem (Definition 1.3.6) for linear codes (Chapter 2)– see Exercise C.15. It turns out this special setting of parameters makes the problem easy: one can use Gaussian elimination to solve this problem in time $O(kn^2)$ (see Exercise C.16). For the case of $k = \Theta(n)$ (which would be the most important parameter regime for this book), this cubic run time is much faster than the exponential time Algorithm C.4.1. In particular, any run time of the form $O(n^c)$ for some fixed constant c would be much faster than the $2^{\Omega(n)}$ run time of Algorithm C.4.1 (for large enough n). Note that the appealing aspect of a run time of the form $O(n^c)$ is that when the input size doubles, the run time only increases by a constant (though clearly we might be pushing the boundary of a practical definition of a constant for moderately large values of c) as opposed to the exponential run time, where the run time on the new input size is quadratic in the old run time.

In theoretical computer science, the notion of an efficient algorithm is one that runs in time $O(N^c)$ on inputs of size N for some fixed constant $c \geq 0$: such algorithms are said to have *polynomial run time*. In particular, a problem is said to be in the *complexity class* P if it admits a polynomial time algorithm⁵. While one might debate the definition of P as capturing algorithms that are efficient in practice, it clearly seems to capture the difference between problems that “need” exponential time algorithms and problems that have some inherent structure that allows much faster algorithmic solutions (in this case polynomial time algorithm).

C.4.1 Computational Intractability

So far we have talked mainly about problems that admit efficient algorithms. We now consider the issue of how we talk about a problem being hard: e.g. can we somehow formally argue that a certain problem cannot admit efficient solutions? In particular, are there problems where the generic exponential time algorithm discussed earlier is the best possible? To be more precise, let us consider problems where given a potential solution one can in polynomial time determine whether the solution satisfies the objective or not. We call the class of such problems as NP.⁶ For example, MAXLINEAREQ(k, n, s) is such a problem because given a potential solution $\mathbf{x}$, one can in time $O(kn)$ verify whether it satisfies at least s out of the n solutions– see Exercise C.17 and hence MAXLINEAREQ $\in$ NP. Like the earlier special case of MAXLINEAREQ(k, n, n),

⁵The technical definition of P is a bit more nuanced: in particular it only considers problems with a binary output but we will ignore this technical issue in this book.

⁶Again for the technical definition we need to only consider problems with binary output but we will ignore this technicality.

the more general problem MAXLINEAREQ(k, n, s) for $s < n$ is also interesting from a coding theory perspective: see Exercise C.18.

Thus, the question of whether there exists a problem where the earlier exponential time algorithm is the best possible is essentially the same as showing $P \neq NP$ (see Exercise C.19). While we are nowhere close to answer this fundamental question, we do know a way to identify the “core” of hard problems in NP. Such problems (called NP-complete problems) have the property that if any of them do not have a polynomial time algorithms then none of them do. (Conversely if any of them do have a polynomial time algorithm then $P = NP$.) Note that this implies if one assumes that $P \neq NP$, then these NP-complete problems are hard problems since they are not in P (which we consider to be the class of “easy” problems).

At first blush, one might wonder how one would go about proving that such problems exist. Proving the existence of such a problem is out of the scope of the book. However, we do want to give an overview of how given one such specific problem that is NP-complete one might argue that another problem is also NP-complete. The way to show such a result is to *reduce* the known NP-complete problem (let us call this problem P_1) to the other problem (let us call this problem P_2). Without going into the technical definition of a reduction, we present an informal definition, which would be sufficient for our purposes. A reduction is a polynomial time algorithm (let us call it $\mathcal{A}_1$) that given an *arbitrary* instance $\mathbf{x}_1$ of P_1 can produce in polynomial time another instance $\mathbf{x}_2$ but this time for the problem P_2 such that given the answer for problem P_2 on $\mathbf{x}_2$, one can in polynomial time exactly determine the answer of P_1 on $\mathbf{x}_1$ (by another algorithm, which let us call $\mathcal{A}_2$). There are two (equivalent) ways to think about such a reduction:

1. A reduction implies that to solve P_1 in polynomial time, it is enough to “only” solve some subset of instances for problem P_2 (in particular, those inputs for P_2 that are generated by $\mathcal{A}_1$ on all possible input instances of P_1). In other words, P_2 is “harder” to solve than P_1 . Since the problem P_1 is a hard problem, P_2 is also a hard problem.
2. Let us for the sake of contradiction assume that there exists a polynomial time algorithm $\mathcal{A}_3$ that solves P_2 on all instances (i.e. P_2 is easy). Then one can construct a polynomial time algorithm to solve P_1 as follows. Given an arbitrary input $\mathbf{x}_1$ for P_1 , first use $\mathcal{A}_1$ to generate an input $\mathbf{x}_2$ for P_2 . Then use $\mathcal{A}_3$ to solve P_2 on $\mathbf{x}_2$ and then convert the answer of P_2 on $\mathbf{x}_2$ to the answer of P_1 on $\mathbf{x}_1$ by using $\mathcal{A}_2$. Note that this is a polynomial time algorithm and is a correct algorithm. Thus, we have proved that P_1 is easy, which contradicts our assumption that P_1 is hard.

To make the concept of reduction a bit less abstract we outline a reduction from a known NP-complete problem to our MAXLINEAREQ problem.⁷ In particular, the following problem is known to be NP-complete

Definition C.4.2. Given a graph $G = (V, E)$ with $|V| = k$ and $|E| = n$ and an integer $0 \leq s \leq n$, does there exist a cut of size at least s . In other words, does there exist a subset $S \subset V$ such that the number of edges with one end point in S and the other in $V \setminus S$ is at least s ? We will call this the MAXCUT(k, n, s) problem.

⁷We assume that the reader is familiar with the mathematical concept of graphs, where we do *not* mean graphs in the sense of plots.

Algorithm C.4.2 is the algorithm $\mathcal{A}_1$ of the reduction from MAXCUT(k, n, s) to MAXLINEAREQ(k, n, s).

Algorithm C.4.2 Reduction from MAXCUT to MAXLINEAREQ

INPUT: An instance for MAXCUT(k, n, s): a graph G and an integer s
 OUTPUT: An instance of MAXLINEAREQ(k', n', s')

- 1: $k' \leftarrow k, n' \leftarrow n, s' \leftarrow s$
 - 2: FOR every vertex $i \in V$ DO
 - 3: Add a variable x_i to the set of variables
 - 4: FOR every edge $(i, j) \in E$ DO
 - 5: Add a linear equation $x_i + x_j = 1$ to the system of equation
-

Further, the algorithm $\mathcal{A}_2$ is simple: given a solution $(x_1, \dots, x_k)$ to the instance for MAXLINEAREQ(k, n, s) problem, consider the cut $S = \{i \in V | x_i = 1\}$. It can be checked that this algorithm and Algorithm C.4.2 forms a valid reduction. (See Exercise C.20.)

C.5 More on intractability

In this section, we formally define some of the notions we defined informally in Section C.4.1. This section is by design terse and is not meant as a substitute for a more formal exposition of computational complexity.

We begin with the specific kind of problem that we need to work with when arguing that certain computational tasks as hard. The most basic notion of a problem is one with a binary output:

Definition C.5.1 (Decision Problem). *Without loss of generality, the input to a decision problem $\mathbf{x} \in \Sigma^*$, where Σ^* refers to the set of all strings (including the empty string) over the alphabet Σ . Then a problem is defined by a subset $L \subseteq \Sigma^*$. If $\mathbf{x} \in L$, then we say that $\mathbf{x}$ is an YES instance/input; otherwise we say it is a NO instance/input. We will sometimes use L to denote the problem as well.*

We say an algorithm $\mathcal{A}$ solves the problem L if for every input $\mathbf{x} \in \Sigma^$, we have that*

$$\mathcal{A}(\mathbf{x}) = \begin{cases} \text{YES} & \text{if } \mathbf{x} \in L \\ \text{NO} & \text{if } \mathbf{x} \notin L \end{cases}.$$

Note that the MAXCUT problem from Definition C.4.2 is a decision problem. We will define our ‘hard’ problems to be decision problems.

We will also need to handle a variant of decision problems.

Definition C.5.2 (Promise Problem). *A promise problem is defined by a pair of two disjoint non-empty subsets $\mathcal{Y}, \mathcal{N} \subset \Sigma^*$ such that all $\mathbf{x} \in \mathcal{Y}$ are the YES instances and all $\mathbf{x} \in \mathcal{N}$ are the NO instances. We will refer to the problem by the pair $(\mathcal{Y}, \mathcal{N})$.*

We say an algorithm $\mathcal{A}$ solves the promise problem (defined by $\mathcal{Y}$ and $\mathcal{N}$) if for every input $\mathbf{x} \in \mathcal{Y} \cup \mathcal{N}$, we have that

$$\mathcal{A}(\mathbf{x}) = \begin{cases} \text{YES} & \text{if } \mathbf{x} \in \mathcal{Y} \\ \text{NO} & \text{if } \mathbf{x} \in \mathcal{N} \end{cases}.$$

Note that there are no constraints on how $\mathcal{A}$ behaves on inputs in $\Sigma^ \setminus (\mathcal{Y} \cup \mathcal{N})$*

It is easy to see that a decision problem (as defined in Definition C.5.1) is a special case of a promise problem (see Exercise C.21). Promise problems will be used to define ‘hard’ problems when we need to argue that a computational problem cannot be solved even *approximately*.

We are now ready to define the classes P and NP (as well as their promise versions promise – P and promise – NP).

Definition C.5.3 (P and promise – P). *We say that a problem L is in P (a promise problem $(\mathcal{Y}, \mathcal{N})$ is in the class promise – P respectively) if there exists a polynomial time algorithm that solves the problem L (the problem $(\mathcal{Y}, \mathcal{N})$ respectively).*

Definition C.5.4 (NP and promise – NP). *We say that a problem L is in NP (a promise problem $(\mathcal{Y}, \mathcal{N})$ is in the class promise – NP respectively) if there exists a polynomial time verification algorithm R with the following properties:*

- *If $\mathbf{x} \in L$ ($\mathbf{x} \in \mathcal{Y}$ resp.), then there exists another string $\mathbf{y}$ (that can be polynomially larger than $\mathbf{x}$) such that $R(\mathbf{x}, \mathbf{y})$ returns YES;*
- *If $\mathbf{x} \notin L$ ($\mathbf{x} \notin \mathcal{N}$ resp.), then for every string $\mathbf{y}$ (that is polynomially larger than $\mathbf{x}$), we have that $R(\mathbf{x}, \mathbf{y})$ returns NO*

We first note the following inclusion (see Exercise C.22):

Proposition C.5.5. *Prove that $P \subseteq NP$ and promise – P $\subseteq$ promise – NP.*

Next, we formally define a notion of a reduction.

Definition C.5.6 (Polynomial time reduction). *We say a decision problem L_1 is polynomial time reducible to another decision problem L_2 (denoted by $L_1 \leq_P L_2$) if there exists a polynomial time algorithm $\mathcal{A}$ such that given a potential input $\mathbf{x}$ for L_1 , $\mathcal{A}(\mathbf{x})$ is a potential output to L_2 such that $\mathbf{x} \in L_1$ if and only if $\mathcal{A}(\mathbf{x}) \in L_2$.⁸*

We say a promise problem $(\mathcal{Y}_1, \mathcal{N}_1)$ is polynomial time reducible to another promise problem $(\mathcal{Y}_2, \mathcal{N}_2)$ if there exists a polynomial time algorithm $\mathcal{A}$ such that given a potential input $\mathbf{x}$ for $(\mathcal{Y}_1, \mathcal{N}_1)$, $\mathcal{A}(\mathbf{x})$ is a potential output to $(\mathcal{Y}_2, \mathcal{N}_2)$ such that $\mathbf{x} \in \mathcal{Y}_1$ if and only if $\mathcal{A}(\mathbf{x}) \in \mathcal{Y}_2$ and $\mathbf{x} \in \mathcal{N}_1$ if and only if $\mathcal{A}(\mathbf{x}) \in \mathcal{N}_2$.⁹

Finally, sometimes we will allow for the reductions to be randomized polynomial time algorithms in which case the condition $\mathbf{x} \in L_1$ if and only if $\mathcal{A}(\mathbf{x}) \in L_2$ holds with say probability at least $2/3$.

⁸Technically, this is called a *Karp reduction*. There is also the notion of a *Cook/Turing reduction*, which is a polynomial time algorithm that decides whether $\mathbf{x} \in L_1$ with the “extra power” of being able to query in constant time (technically this is called *oracle access*) for any $\mathbf{y}$ such that $|\mathbf{y}| = \text{poly}(|\mathbf{x}|)$ whether $\mathbf{y} \in L_2$.

⁹Again this is a Karp reduction. The notion of a Cook/Turing reduction is similar to that in the decision version.

We can use the notion of reductions algorithmically (and indeed this forms the basis of a lot of algorithm design) by noting that if $L_1 \leq_P L_2$ and $L_2 \in P$, then $L_1 \in P$ (see Exercise C.23). In other words, assume $L_1 \leq_P L_2$. Then if L_2 is "easy" (i.e. $L_2 \in P$) then so is L_1 . However, we can use the logically equivalent negation of this implication to argue that if L_1 is "hard" then so is L_2 . The question then becomes what is the notion of a "hard" problem, which we define next.

Definition C.5.7. We say a decision problem L is NP-complete if the following two hold:

- (1) $L \in NP$
- (2) Every decision problem $L' \in NP$, is polynomial time reducible to L , i.e., $L' \leq_P L$.

If L only satisfies the second property above, then it is called an NP-hard problem.

We note that NP-complete problems are the hardest problems in NP because of the following result (and Proposition C.5.5):

Proposition C.5.8. If there exists an NP-complete problem L such that $L \in P$. Then $P = NP$.

(See Exercise C.24.)

It is a non-trivial fact to prove that there do exist NP-complete problems. Once we have this fact, one can "port" the hardness of NP-complete problems to other problems as follows (see Exercise C.25):

Proposition C.5.9. Let L be an NP-complete problem. Then if $L \leq_P L'$, then L' is an NP-hard problem. If further, we have $L' \in NP$, then L' is also NP-complete.

The above proposition gives a roadmap for us to prove that the problem we are considering is NP-complete/hard: we just need to reduce an NP-complete problem to our problem.

This finally, brings us to the hardness assumption that we will be primarily using to prove lower bounds in this book:

We assume that $P \neq NP$.

We will also sometimes assume that NP is not the same as class of problems that have a randomized polynomial time algorithms as well not being the same as problems that have polynomial sized circuits.

C.6 Exercises

Exercise C.1. Prove that $f(N)$ is $O(g(N))$ (as per Definition C.1.1) if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} \leq C,$$

for some absolute constant C .

Exercise C.2. Prove that $f(N)$ is $\Omega(g(N))$ (as per Definition C.1.2) if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} \geq C,$$

for some absolute constant C .

Exercise C.3. Prove that $f(N)$ is $\Theta(g(N))$ (as per Definition C.1.3) if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = C,$$

for some absolute constant C .

Exercise C.4. Prove that $f(N)$ is $o(g(N))$ (as per Definition C.1.4) if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = 0.$$

Exercise C.5. Prove that $f(N)$ is $\omega(g(N))$ (as per Definition C.1.5) if and only if

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = \infty.$$

Exercise C.6. Prove Lemmas C.1.6, C.1.7 and C.1.8.

Exercise C.7. Prove or disprove the following for every $\alpha \in \{O, \Omega, \Theta, o, \omega\}$:

Let $f(N)$ be $\alpha(h(N))$ and $g(N)$ be $\alpha(h(N))$. Then $f(N) \cdot g(N)$ is $\alpha(h(N))$.

Exercise C.8. Say there exists a randomized algorithm $\mathcal{A}$ that is correct with probability $\frac{1}{2} + \delta$ for some $\delta > 0$ with runtime $T(N)$. Then show that for every $\epsilon > 0$, there exists another randomized algorithm that is correct with probability $1 - \epsilon$ with runtime $O\left(\frac{\log(1/\epsilon)}{\delta} \cdot T(N)\right)$.

Hint: Repeat $\mathcal{A}$ multiple times and pick one among the multiple outputs. For analysis use the Chernoff bound (Theorem 3.1.10).

Exercise C.9. Assume that there is a randomized algorithm $\mathcal{A}$, which one when provided with an input from a distribution $\mathbb{D}$, is correct with probability p (where the probability is taken over both $\mathbb{D}$ and internal randomness of $\mathcal{A}$). Then show that there exists a deterministic algorithm that is correct with probability at least p (where the probability is now only taken over $\mathbb{D}$) with the same run time as $\mathcal{A}$.

Exercise C.10. Argue that any correct deterministic algorithm that solves the GAPHAMMING problem needs a run time of $\Omega(n)$.

Hint: Argue that any correct deterministic algorithm needs to read $\Omega(n)$ bits of the input.

Exercise C.11. Argue that Algorithm C.3.1 runs in time $O(\log(1/\epsilon))$.

Exercise C.12. Prove that for every $\mathbf{x} \in \{0, 1\}^n$ such that $wt(\mathbf{x}) \geq 2n/3$, Algorithm C.3.1 outputs 1 with probability at least $1 - \epsilon$.

Exercise C.13. Prove Lemma C.3.2 and that Algorithm C.3.2 runs in time $O(\log(1/\epsilon))$.

Exercise C.14. Argue that Algorithm C.4.1 runs in time $O(kn^2)$. Conclude that the algorithm runs in time $2^{O(n)}$.

Exercise C.15. Show that if any $\text{MAXLINEAREQ}(k, n, n)$ problem can be solved in time $T(k, n)$, then the error detection for any $[n, k]_2$ code can be solved in $T(k, n)$ time.

Hint: The two problems are in fact equivalent.

Exercise C.16. Argue that the problem $\text{MAXLINEAREQ}(k, n, n)$ can be solved in time $O(kn^2)$.

Exercise C.17. Show that given a system of n linear equation on k variables over $\mathbb{F}_2$, there exists a $O(kn)$ time algorithm that given a vector $\mathbf{x} \in \{0, 1\}^n$ can compute the exact number of equations $\mathbf{x}$ satisfies.

Exercise C.18. Consider the following problem called the $\text{BOUNDED DISTANCE DECODING}$ problem. Given a code $C \subseteq \{0, 1\}^n$, a vector $\mathbf{y} \in \{0, 1\}^n$ and an integer $0 \leq e \leq n$ (called the error radius), output any codeword $\mathbf{c} \in C$ such that $\Delta(\mathbf{c}, \mathbf{y}) \leq e$ (or state that no such codeword exists).

Prove that if any $\text{MAXLINEAREQ}(k, n, s)$ problem can be solved in time $T(k, n, s)$, then one can solve the $\text{BOUNDED DISTANCE DECODING}$ problem for any $[k, n]_2$ linear code with error radius $n - s$.

Exercise C.19. Argue that showing $P \neq NP$ is equivalent to showing that $NP \setminus P \neq \emptyset$.

Exercise C.20. Argue that Algorithm C.4.2 and the algorithm $\mathcal{A}_2$ defined just below it are correct and run in polynomial time.

Exercise C.21. Let L be a decision problem (as defined in Definition C.5.1). Then argue that L is also a promise problem (as defined in Definition ??).

Exercise C.22. Argue Proposition C.5.5.

Exercise C.23. Argue that if $L_1 \leq_P L_2$ and $L_2 \in P$, then $L_1 \in P$.

Exercise C.24. Prove Proposition C.5.8.

Exercise C.25. Prove Proposition C.5.9.

C.7 Bibliographic Notes

The full suite of asymptotic notation in Section C.1 was advocated for analysis of algorithms by Knuth [100]. The Big-Oh notation is credited to Bachmann [12] from a work in 1894 and the little-oh notation was first used by Landau [103] in 1909. A variant of the Big-Omega notation was defined by Hardy and Littlewood [84] in 1914 though the exact definition in Section C.1 seems to be from [100]. The Theta and little omega notation seem to have been defined by Knuth [100] in 1976: Knuth credits Tarjan and Paterson for suggesting the Theta notation to him.

The choice to use worst-case run time as measure of computational efficiency in the RAM model as well as only considering the asymptotic run time (as opposed to more fine grained analysis as advocated by Knuth) seem to have been advocated by Hopcroft and Tarjan: see Tarjan's Turing award lecture for more on this [155].

Cobham [32] and Edmonds [46] are generally credited with making the first forceful case for using P as the notion of efficiently solvable problems. Somewhat interestingly, Peterson's paper on decoding of Reed-Solomon codes [125] that predates these two work explicitly talks about why a polynomial time algorithm is better than an exponential time algorithm (though it does not explicitly define the class P). The notion of NP (along with a proof of the existence of an NP -complete problem) was defined independently by Cook [33] and Levin [104]. This notion really took off when Karp showed that 21 natural problems were NP -complete (including the MAXCUT problem) [98]. For more historical context on P and NP including some relevant historical comments, see the survey by Sipser [143].

The first randomized algorithms is generally credited to Rabin [126]. However, an earlier work of Berlekamp on factoring polynomials presents a randomized algorithm (though it is not stated explicitly as such) [18].

This chapter gave a very brief overview of topics that generally span multiple classes. For further readings, please consult standard textbooks on the subjects of (introductory) algorithms and computational complexity as well as randomized algorithms.

Appendix D

Basic Algebraic Algorithms

D.1 Executive Summary

In this appendix we include some basic facts about abstract algebra that were used throughout the book. Readers who are comfortable with their background in algebra should feel free to skip it entirely. However, this background should include both aspects introduced by *finiteness* — most fields we work with are finite, and so are the vector spaces defined over them — and *computation* — the mere existence of a nice algebraic structure is not good enough for us, we need to know how to carry out basic, and not so basic operations over these structures efficiently. If you are not very comfortable with these settings you will find the appropriate sections of this appendix more useful. The opening paragraph of each section summarizes the main aspects covered in the section and the reader may use them to decide if they wish to read further.

Some of the material in this appendix appears earlier in the book (e.g. Sections 2.1, 2.2 and 5.1). Finally, this coverage of algebra in this appendix is not exhaustive and the reader is referred to the book by Lidl and Niederreiter [106] for more material (and proofs) on finite fields and the book by Shoup for more details on the basic algebraic algorithms [141].

D.2 Groups, Rings, Fields

The title of this section says it all. We cover, very tersely, the definition of a group, a ring, and a field.

We begin with some terminology. We consider binary operations over some set of elements. Given a set X such a binary operator would be a function $\circ : X \times X \rightarrow X$, and we usually use $a \circ b$ to denote $\circ(a, b)$, for $a, b \in X$. We say the operator $\circ$ is *associative* if $a \circ (b \circ c) = (a \circ b) \circ c$, for every $a, b, c \in X$. For associative operations it is customary to drop the parenthesis. We say the operator $\circ$ is *commutative* if $a \circ b = b \circ a$ for every $a, b \in X$. We say an element $e \in X$ is an identity for $\circ$ if $a \circ e = e \circ a = a$ for every $a \in X$. Identities are, by definition, unique if they exist, since if $e_1, e_2 \in X$ were identities, we would have $e_1 = e_1 \circ e_2 = e_2$. Given $a \in X$ and operator $\circ$ with inverse e we say that a is *invertible* with respect to $\circ$ if there exists an element $a^{-1} \in X$ such that $a \circ a^{-1} = a^{-1} \circ a = e$. Often $\circ$ will be clear from context in which case we will refer to a

as simply invertible.

Definition D.2.1 (Group). Given a set G and a binary operation $\circ$ over G , we say that $(G, \circ)$ is a group if $\circ$ is associative, has an identity, and every element of G is invertible. A group $(G, \circ)$ is said to be an abelian group if $\circ$ is also commutative.

Examples of groups include the integers with addition, the non-zero rationals with multiplication and the set of permutations (one-to-one functions) on any finite set under the composition operation.

Definition D.2.2 (Ring). A finite set R with two binary operations $+$ and $\cdot$ are said to form a ring if (1) $(R, +)$ form an abelian group, (2) $\cdot$ is associative and has an identity and (3) $\cdot$ distributes over $+$, i.e., for every $a, b, c \in R$ we have $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ and have $(b + c) \cdot a = (b \cdot a) + (c \cdot a)$. The ring $(R, +, \cdot)$ is said to be a commutative ring if $\cdot$ is commutative.

Examples include the integers over addition and multiplication (a commutative ring) and the set of $k \times k$ integer matrices (for any positive integer k) under matrix addition and matrix multiplication (which forms a non-commutative ring for $k \geq 2$).

Definition D.2.3 (Field). A set $\mathbb{F}$ with operations $+$ and $\cdot$ forms a field if $(\mathbb{F}, +, \cdot)$ is a commutative ring, and $(\mathbb{F} \setminus \{0\}, \cdot)$ is a group where 0 denotes the identity for $+$.

Examples of fields include the rationals, the reals, the complexes (all under addition and multiplication) and (more interestingly to us) the integers modulo any prime number p (see Lemma 2.1.4 for the latter).

It is customary in rings and fields to let 0 denote the additive identity, 1 the multiplicative identity and to let $-a$ denote the additive inverse of a and a^{-1} the multiplicative inverse of a . It is also customary to abbreviate $a + (-b)$ to $a - b$.

D.3 Polynomials

In this section we will introduce polynomial rings, mention when they satisfy the unique factorization property, describe the ‘remainder algorithm’, and describe the evaluation map and the polynomial distance property (where the latter is a re-statement of the degree mantra (Proposition 5.1.5)).

Definition D.3.1 (Formal Polynomials). Let $(R, +, \cdot)$ be a commutative ring with identity 0 . The set of formal polynomials over R in indeterminate X , denoted $R[X]$, is given by finite formal sums $R[X] = \{\sum_{i=0}^d f_i X^i \mid f_0, \dots, f_d \in R; d \in \mathbb{Z}^{\geq 0}\}$, under the equivalence $\sum_{i=0}^d f_i X^i = \sum_{i=0}^{d-1} f_i X^i$ if $f_d = 0$. (The term formal refers to the fact that the summation, and the terms X^i are just formal symbols and do not have operational meaning, yet. So really polynomials are just finite sequences of elements from R under the equivalence $(f_0, \dots, f_d, 0) \cong (f_0, \dots, f_d)$.)

Basic terminology The elements f_i are referred to as the coefficients of f , the symbols X^i as the monomials of f and the product $f_i X^i$ as the terms of f . For $f = \sum_{i=0}^d f_i X^i$, its degree, denoted $\deg_X(f)$ or simply $\deg(f)$, is the largest integer e such that $f_e \neq 0$.

Addition The sum of two polynomials $f = \sum_{i=0}^d f_i X^i$ and $g = \sum_{i=0}^d g_i X^i$, denoted $f + g$, is the polynomial $\sum_{i=0}^d (f_i + g_i) X^i$. (Note that by padding the coefficients of f and g with zeroes we can always arrange it so that they have the same number of terms.)

Multiplication Finally, the product of $f = \sum_{i=0}^d f_i X^i$ $g = \sum_{i=0}^e g_i X^i$, denoted $f \cdot g$ (or sometimes simply fg), is given by $\sum_{i=0}^{d+e} \left(\sum_{j=0}^e f_{i-j} \cdot g_j \right) X^i$.

The following proposition follows immediately from the definitions above.

Proposition D.3.2. For every commutative ring R , $R[X]$ is a commutative ring under the sum and product of polynomials.

In fact R inherits many properties of R and in particular the notion of “unique factorization” which we describe next.

Definition D.3.3 (Unique Factorization Domains). Let R be a commutative ring. An element $u \in R$ is said to be a unit if it has a multiplicative inverse in R . Elements a and b are said to be associates if there exists a unit u such that $a = b \cdot u$. (Note that being associates is an equivalence relationship.) Element $a \in R$ is said to be irreducible if $a = b \cdot c$ implies either b or c is a unit. A factorization of $a \in R$ is a sequence $b_1, \dots, b_k$ such that $a = b_1 \cdot b_2 \cdots b_k$ and none of the b_i ’s are units. The b_i are referred to as the factors of a in this factorization. Ring R is a factorization domain if for non-zero every $a \in R$ that is not a unit, there is a finite bound k_a such that every factorization of a has at most k_a factors. A factorization domain R is a unique factorization domain (UFD) if every non-zero, non-unit element has a unique irreducible factorization, upto associates. I.e., if $a = b_1 \cdots b_k = c_1 \cdots c_\ell$ and the b_i ’s and c_j ’s are irreducible, then $k = \ell$ and there exists a bijection $\pi: [k] \rightarrow [\ell]$ such that b_i and $c_{\pi(i)}$ are associates, for every $i \in [k]$.

Since every non-zero element of a field is a unit, every field is a UFD.

Proposition D.3.4. Every field is a UFD.

A central result in basic commutative algebra is the following lemma of Gauss.

Lemma D.3.5 (Gauss). If R is a UFD, then so is $R[X]$.

We omit the proof of the above lemma here, but point out its implications. It allows us to build many interesting rings from a simple base case, namely a field. Given a field $\mathbb{F}$, $\mathbb{F}[X]$ is a UFD. So is $(\mathbb{F}[X])[Y]$. Now we could have gone in the other direction and created the ring $(\mathbb{F}[Y])[X]$ and this would be a UFD too. However if X and Y commute (so $XY = YX$) then the rings $(\mathbb{F}[X])[Y]$ and $(\mathbb{F}[Y])[X]$ are isomorphic under the isomorphism that preserves $\mathbb{F}$ and sends $X \rightarrow X$ and $Y \rightarrow Y$. So we tend to compress the notation and refer to this ring as $\mathbb{F}[X, Y]$, the ring of “bivariate” polynomials over $\mathbb{F}$. Rings of univariate and multivariate polynomials play a central role in algebraic coding theory.

We now turn to the notion of polynomial division with remainder that lead us to some important notions associated with polynomials.

Let $f \in R[X]$ and let $f = \sum_{i=0}^d f_i X^i$ with $f_d \neq 0$. f is said to be monic if f_d is a unit in R .

Proposition D.3.6. *Given a monic polynomial f , and general polynomial p there exists a unique pair of polynomials q (for quotient) and r (for remainder) such that $p = q \cdot f + r$ and $\deg(r) < \deg(f)$.*

See Exercise D.1 for a proof.

The function $p \mapsto_f (q, r)$ described is often referred to as the ‘division algorithm’ (since it is the outcome of long division). A special case that is of great interest to us is when $f = X - \alpha$ for $\alpha \in R$. In this case the remainder is polynomial of degree at most 0, and so can be associated with an element of R . Denote this element $p(\alpha)$ (since it depends only on p and α) and we get the “evaluation” map which maps elements of $\mathbb{R}[X] \times R$ to R . The remainder $p(\alpha)$ can be worked out explicitly and is given by the simple form below (where the uniqueness follows from Proposition D.3.6).

Proposition D.3.7. *Given $p = \sum_{i=0}^d p_i X^i \in R[X]$ and $\alpha \in R$, let $p(\alpha) = \sum_{i=0}^d p_i \alpha^i$. Then there exists a unique $q \in R[X]$ such that $p = q \cdot (X - \alpha) + p(\alpha)$. It follows that $p(\alpha) = 0$ if and only if $X - \alpha$ divides $p(X)$.*

Finally using Proposition D.3.7 and the fact that $\mathbb{F}[X]$ is a UFD, we get the following central fact about univariate polynomials.

Lemma D.3.8 (Polynomial Distance Lemma). *Let $f \neq g \in \mathbb{F}[X]$ be polynomials of degree at most d . Then there exist at most d elements $\alpha \in \mathbb{F}$ such that $f(\alpha) = g(\alpha)$.*

Proof. Let $h = f - g$. We have h is non-zero and of degree at most d . Let $S = \{\alpha \mid f(\alpha) = g(\alpha)\}$. Then we have $(X - \alpha)$ divides h for every $\alpha \in S$. Furthermore, by the unique factorization property we have $\tilde{h} = \prod_{\alpha \in S} (X - \alpha)$ divides h . But if $\tilde{h}$ divides h , then $\deg(\tilde{h}) \leq \deg(h)$ and $\deg(\tilde{h}) = |S|$. We conclude $|S| \leq d$. $\square$

D.4 Vector Spaces

In this section we introduce vector spaces over fields and describe two basic views of describing a finite dimensional vector space: first via its generators (and the generator matrix) and next via constraints on the vector space (and its parity check matrix). We first start with a quick overview of matrices and the corresponding notation and then move on to vector spaces.

D.4.1 Matrices and Vectors

In this book, a vector of length n over the field $\mathbb{F}$ (i.e. $\mathbf{x} \in \mathbb{F}^n$) is a row vector¹. E.g., we have $\mathbf{x} = (0 \ 1 \ 3 \ 4 \ 0) \in \mathbb{F}_5^4$. Given two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$, their *inner product* is defined as

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{i=1}^n u_i \cdot v_i,$$

¹We acknowledge that this is different from the usual assumption in linear algebra that all vectors are column vectors. We are assuming row vectors to be consistent with how message vectors are assumed to be row vectors in coding theory.

where the multiplication and addition are over $\mathbb{F}$.

A matrix $M \in \mathbb{F}^{k \times n}$ is a two-dimensional array/vector, where we refer to the (i, j) 'th entry (for $(i, j) \in [k] \times [n]$) as $M_{i,j}$ (or M_{ij} if the two indices are clear without being separated by a comma). We will use $M_{i,\cdot}$ as the i 'th row and $M_{\cdot,j}$ as the j 'th column of M respectively.

So e.g. consider $G \in \mathbb{F}_3^{2 \times 3}$ as follows:

$$G = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \end{pmatrix}.$$

In the above $G_{1,2} = 2$, $G_{1,\cdot} = (1 \ 0 \ 1)$ and $G_{\cdot,2} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$.

The *transpose* of a matrix $M \in \mathbb{F}^{k \times n}$, denoted by M^T is an $n \times k$ matrix over $\mathbb{F}$ such that for any $(j, i) \in [n] \times [k]$, we have

$$M_{j,i}^T = M_{ij}.$$

Note that if $k = 1$, then the above says that for a row vector $\mathbf{x} \in \mathbb{F}^n$, its transpose $\mathbf{x}^T$ is a column vector.

The product of two matrices $A \in \mathbb{F}^{k \times n}$ and $B \in \mathbb{F}^{n \times m}$ is a matrix $C \in \mathbb{F}^{k \times m}$ such that for any $(i, j) \in [k] \times [m]$, we have

$$C_{i,j} = \langle A_{i,\cdot}, B_{\cdot,j} \rangle.$$

D.4.2 Definition and Properties of Vector Spaces

This section will repeat some of the material from Section 2.2. We begin with the definition of a vector space:

Definition D.4.1 (Vector Space). *Over a field $\mathbb{F}$, a vector space is given by a triple $(V, +, \cdot)$ where $(V, +)$ is a commutative group and $\cdot : \mathbb{F} \times V \rightarrow V$ distributes over addition, so that $\alpha \cdot (\mathbf{u} + \mathbf{v}) = \alpha \cdot \mathbf{u} + \alpha \cdot \mathbf{v}$ for every $\alpha \in \mathbb{F}$ and $\mathbf{u}, \mathbf{v} \in V$. It is customary to denote the identity of the group $(V, +)$ by $\mathbf{0}$ and to refer to V as an $\mathbb{F}$ -vector space.*

The simplest example of an $\mathbb{F}$ -vector space is $\mathbb{F}^n$, whose elements are sequences of n elements of $\mathbb{F}$. The sum is coordinate-wise summation and product is “scalar” product, so if $\mathbf{u} = (u_1, \dots, u_n)$, $\mathbf{v} = (v_1, \dots, v_n)$ and $\alpha \in \mathbb{F}$ then $\mathbf{u} + \mathbf{v} = (u_1 + v_1, \dots, u_n + v_n)$ and $\alpha \cdot \mathbf{u} = (\alpha \cdot u_1, \dots, \alpha \cdot u_n)$. Essentially these are the only vector spaces (as we will make precise soon), but representations of the vectors is important to us, and will make a difference.

Definition D.4.2 (Dimension of a vector space). *A sequence of vectors $\mathbf{v}_1, \dots, \mathbf{v}_k \in V$ are said to be linearly independent if $\sum_{i=1}^k \beta_i \cdot \mathbf{v}_i = \mathbf{0}$ implies that $\beta_1 = \dots = \beta_k = 0$. $\mathbf{v}_1, \dots, \mathbf{v}_k \in V$ are said to be linearly dependent otherwise.*

V is said to be finite dimensional of dimension k if every sequence of $k+1$ vectors from V is linearly dependent and there exists a sequence of length k that is linearly independent.

A linearly independent set $\mathbf{v}_1, \dots, \mathbf{v}_k \in V$ is said to form a basis for V if V has dimension k .

Every $\mathbb{F}$ -vector space of dimension k is isomorphic to $\mathbb{F}^k$ as described by the following proposition.

Proposition D.4.3. If $\mathbf{v}_1, \dots, \mathbf{v}_k$ for a basis for an $\mathbb{F}$ -vector space V , then $V = \{\sum_{i=1}^k \beta_i \cdot \mathbf{v}_i \mid \beta_1, \dots, \beta_k \in \mathbb{F}\}$ and the map $(\beta_1, \dots, \beta_k) \mapsto \sum_{i=1}^k \beta_i \cdot \mathbf{v}_i$ is an isomorphism from $\mathbb{F}^k$ to V .

The point we wish to stress now is that even though all vector spaces are isomorphic, different spaces do lead to different codes with different error-correction properties, and these properties are not preserved by such isomorphisms. So not all k -dimensional vector spaces are identical for our purposes. We will specially be interested in k -dimensional vector spaces contained in $\mathbb{F}^n$, and how these can be represented succinctly in matrix form.

Definition D.4.4 (Generator Matrix, Parity Check Matrix). A matrix $G \in \mathbb{F}^{k \times n}$ is said to be a generator matrix of an $\mathbb{F}$ -vector space $V \subseteq \mathbb{F}^n$ if the rows of G are linearly independent in $\mathbb{F}^n$ and $V = \{\mathbf{x} \cdot G \mid \mathbf{x} \in \mathbb{F}^k\}$. The rows of G form a basis of V . A matrix $H \in \mathbb{F}^{n \times (n-k)}$ is said to be a parity check matrix of an $\mathbb{F}$ -vector space $V \subseteq \mathbb{F}^n$ if the columns of H are linearly independent and $V = \{\mathbf{y} \in \mathbb{F}^n \mid H \cdot \mathbf{y}^T = \mathbf{0}\}$. Given a vector space V with parity check matrix H , its dual space, denoted $V^\perp$, is the vector space generated by the transpose of H , i.e., $V^\perp = \{\mathbf{x} \cdot H^T \mid \mathbf{x} \in \mathbb{F}^{n-k}\}$.

Our goal below is to show that every space has a generator matrix and a parity check matrix. The former is obvious from definitions. If $V \subseteq \mathbb{F}^n$ is a k -dimensional vector space, then it has a basis $\mathbf{v}_1, \dots, \mathbf{v}_k$ and if we build a matrix G with these vectors as its rows, then G satisfies the conditions of the generator matrix.

We sketch the idea for construction of a parity check matrix. We say that a $k \times k$ matrix R forms a row operation if $R_{ii} = 1$, and $R_{ij} = 0$ for all must at most one pair $i \neq j \in [k]$. We say that $\tilde{G}$ is obtained from G by row operations, denoted $G \rightsquigarrow \tilde{G}$, if $\tilde{G} = R_m \cdot R_{m-1} \cdots R_1 \cdot G$ where the R_i 's are row operations. Note that if G is a generator matrix for V then so is $\tilde{G}$. Gaussian elimination allows us to “simplify” G till its columns are special, and in particular after permuting the columns $\tilde{G}$ would look like $[I_k \mid A]$ where I_k denotes the $k \times k$ identity matrix. Assume for simplicity that $\tilde{G} = [I_k \mid A]$ (without permuting columns). Now let H be given by $H^T = [-A^T \mid I_{n-k}]$. It can be verified that $\tilde{G} \cdot H = 0$ and so $G \cdot H = 0$. Furthermore all columns of H are linearly independent and so H satisfies the conditions of the parity check matrix of V . We conclude with the following.

Proposition D.4.5. If $V \subseteq \mathbb{F}^n$ is a k -dimensional vector space then it has a generator matrix $G \in \mathbb{F}^{k \times n}$ and a parity check matrix $H \in \mathbb{F}^{n \times (n-k)}$. Furthermore its dual $V^\perp$ is generated by H^T , has dimension $n - k$, and has G^T as its parity check matrix. Finally $(V^\perp)^\perp = V$.

Before concluding we mention one important difference from the case of *orthogonality* of real vectors. For vector spaces over finite fields it is possible that there are non-zero vectors in $V \cap V^\perp$ and indeed even have $V = V^\perp$. (See Exercise 2.31 for more on this.)

D.5 Finite Fields

In this section we describe the existence and uniqueness of finite fields. We also describe the basic maps going from prime fields to extensions and vice versa. Parts of this section will repeat material from Section 2.1 and 5.1.

D.5.1 Prime Fields

We start by describing a field of size p , for any prime number p . Let $\mathbb{Z}_p$ be the set of integers $\{0, \dots, p-1\}$. For integer a and positive integer b , let $a \bmod b$ denote the unique integer c in $\mathbb{Z}_p$ such that b divides $a - c$. Let $+_p$ be the binary operation on $\mathbb{Z}_p$ that maps a and b to $(a + b) \bmod p$. Let $\cdot_p$ map a and b to $(ab) \bmod p$. We have the following (see Section 2.1 for a proof).

Proposition D.5.1. $(\mathbb{Z}_p, +_p, \cdot_p)$ form a field of cardinality p .

Given a finite field $\mathbb{F}$, its characteristic, denoted $\text{char}(\mathbb{F})$, is the smallest positive integer p such that $p \cdot 1 = 1 + 1 + \dots + 1 = 0$. (See Exercise D.2 for why such a finite characteristic exists.)

Proposition D.5.2. For every finite field $\mathbb{F}$, $\text{char}(\mathbb{F})$ is a prime. Furthermore, $\mathbb{F}$ is a $\mathbb{Z}_p$ -vector space, where $p = \text{char}(\mathbb{F})$. Thus $\mathbb{F}$ has cardinality p^n for prime p and integer n .

Proof. Let $p = \text{char}(\mathbb{F})$. We first note that p is the smallest integer such that $p \cdot a = 0$ for any non-zero element of $\mathbb{F}$. This is so since $p \cdot a = p \cdot 1 \cdot a = 0$, and if $p \cdot a = 0$ then so is $p \cdot a \cdot a^{-1} = p \cdot 1$. Next we note that if $p = qr$ then for the element $w = q \cdot 1 \in \mathbb{F}$, we have $w \cdot r = 0$ which contradicts the minimality of p .

Next we note that $(\mathbb{F}, +, \circ)$ satisfy the conditions of a $\mathbb{Z}_p$ -vector space where $i \circ a = (a + \dots + a)$ (i times), for $i \in \mathbb{Z}_p$ and $a \in \mathbb{F}$. We conclude that $|\mathbb{F}| = p^n$ where n is the dimension of the vector space $(\mathbb{F}, +, \circ)$ and $p = \text{char}(\mathbb{F})$. $\square$

We conclude now by claiming that $\mathbb{Z}_p$ is the unique field of cardinality p .

Proposition D.5.3. For any prime p , there is a unique field of cardinality p upto isomorphism.

Proof. Let $\mathbb{F}$ be a field of cardinality p . By Proposition D.5.2 we have that $\text{char}(\mathbb{F}) = p$. It can be verified that the map $1_{\mathbb{F}} \rightarrow 1$ extends to an isomorphism (see Exercise D.3). $\square$

The uniqueness of the field of cardinality p allows us to call it $\mathbb{F}_p$ in the future.

D.5.2 Extension fields and subfields

We now move towards determining when non-prime fields exists. While the answer is simple (they exist for every number of the form p^n for prime p and positive integer n), proving when they exist requires some structural understanding of how fields behave.

We will first present a basic property of all finite fields that is crucial when working with fields.

We recall a basic result about finite groups (see Exercise D.4 for a proof for the abelian case).

Proposition D.5.4. If $(G, \cdot)$ is a finite group with identity 1, then for every $a \in G$, we have $a^{|G|} = 1$.

Proposition D.5.5. Let $\mathbb{F}$ be a field of cardinality q . The every element $\alpha \in \mathbb{F}$ is a root of the polynomial $X^q - X$ and so $X^q - X = \prod_{\alpha \in \mathbb{F}} (X - \alpha)$.

Proof. If $\alpha = 0$, then it is trivial to see that is a root of $X^q - X$. If $\alpha \neq 0$, then it is a member of a group $(\mathbb{F} \setminus \{0\}, \cdot)$ and so by Proposition D.5.4, satisfies $\alpha^{|\mathbb{F} \setminus \{0\}|} = 1$. Thus, $\alpha^{q-1} = 1$, and finally $\alpha^q = \alpha$, as desired. $\square$

Let $\mathcal{K}$ be a field and $\mathbb{F} \subseteq \mathcal{K}$ be a set that is closed under addition and multiplication. Then $\mathbb{F}$ is itself a field and we denote if $\mathbb{F} \triangleleft \mathcal{K}$ to denote that it is a subfield of $\mathcal{K}$. We say $\mathcal{K} \triangleright \mathbb{F}$ to denote that $\mathcal{K}$ extends $\mathbb{F}$.

Proposition D.5.6. *If $\mathcal{K} \triangleright \mathbb{F}$ then $\mathcal{K}$ is an $\mathbb{F}$ -vector space and so $|\mathcal{K}| = |\mathbb{F}|^n$ where n is the dimension of $\mathcal{K}$ as an $\mathbb{F}$ -vector space. Furthermore there is a unique copy of $\mathbb{F}$ in $\mathcal{K}$.*

Proof. The fact that $\mathcal{K}$ is a vector space follows from the definitions, and thus the claim about its cardinality. The fact that there is a unique copy of $\mathbb{F}$ follows from the fact that the elements of $\mathbb{F}$ satisfy $X^q - X = 0$, where $q = |\mathbb{F}|$ and there can be at most q roots of this polynomial. $\square$

D.5.3 Existence of Finite Fields

In what follows we will rely heavily on the modular reduction of polynomials. Following the notation of previous sections, for field $\mathbb{F}$ and $f, g \in \mathbb{F}[X]$, we let $f \bmod g$ be the remainder when f is divided by g - so $\deg(f \bmod g) < \deg(g)$ and g divides $f - (f \bmod g)$. Let $f +_g h = (f + h) \bmod g$ and let $f \cdot_g h = (f h) \bmod g$. Recall that an irreducible polynomial in $\mathbb{F}_q[X]$ is one that does not have any non-trivial factor (recall Definition 5.1.6).

Proposition D.5.7. *Let $\mathbb{F}$ be a finite field of cardinality q and let $g \in \mathbb{F}[X]$ be an irreducible polynomial of degree n . Then $(\mathbb{F}[X]/g, +_g, \cdot_g)$ form a field of cardinality q^n .*

Essentially all fields can be obtained in the above manner, but to prove this fact, we need to prove that there is an irreducible polynomial of degree n over $\mathbb{F}_p$ for every p and unfortunately this proof is not much simpler than proving the existence of a field of cardinality p^n . So we prove the existence directly, or rather sketch a proof of this fact.

The rough idea of the proof is as follows: First we establish that every polynomial $f \in \mathbb{F}[X]$ splits completely (into linear factors) over some extension $\mathcal{K}$ of $\mathbb{F}$. To do this we work slowly, working away at one irreducible factor of f at a time. If g is such an irreducible factor, we consider the field² $\mathbb{L} = \mathbb{F}[Z]/g(Z)$ and note that Z is a root of g ,³ and hence of f , in $\mathbb{L}$ and so f splits more in $\mathbb{L}$. We continue this process till f splits completely in some field $\mathcal{K}$.

Now we work with a very special polynomial f , namely $f(X) = X^{p^n} - X$ in the ring $\mathbb{F}_p[X]$ and let $\mathcal{K}$ be a field in which f splits completely. Now let $S \subseteq \mathcal{K}$ be the set $S = \{\alpha \in \mathcal{K} \mid f(\alpha) = 0\}$. We note that this set, miraculously, is closed under addition and multiplication. The latter is easy: $f(\alpha) = 0$ if and only if $\alpha^{p^n} = \alpha$. So if $f(\alpha) = f(\beta) = 0$ then $\alpha^{p^n} = \alpha$ and $\beta^{p^n} = \beta$ and so $(\alpha\beta)^{p^n} = \alpha^{p^n} \beta^{p^n} = \alpha\beta$ and so $\alpha\beta \in S$. For the former we explicitly highlight another crucial fact in finite fields.

²Recall Theorem 5.1.7.

³This is because $g(Z) \equiv 0$ in $\mathbb{L}$.

Proposition D.5.8. *Let $\mathcal{K}$ be a field of characteristic p and let $A, B \in \mathcal{K}[X, Y]$. Then for all positive integers n we have $(A + B)^{p^n} = A^{p^n} + B^{p^n}$.*

The proof of the lemma above follows immediately from the fact that $\binom{p}{i} \bmod p$ is 0 unless p divides i (see Exercise D.5). And while the lemma is stated for very general A and B , we only need it for $A, B \in \mathcal{K}$ itself. However we state it generally since it is fundamental to working over extension fields and indeed we will see a few applications later.

Returning to our quest to prove that S is closed under addition, let us apply the above proposition to $\alpha, \beta \in S$. We get that $(\alpha + \beta)^{p^n} = \alpha^{p^n} + \beta^{p^n} = \alpha + \beta$ and so S is closed under addition as well. What we will show next is that S has exactly p^n elements and so is a field of size p^n (it is closed under addition and multiplication and the rest of the properties follow from the fact that S is a subseq of a field $\mathcal{K}$).

First note that S has all roots of f . We note further that f has no multiple roots. In general this is proved by looking at derivatives etc., but in this case we can do it by inspection. We wish to show that $(X - \alpha)^2$ does not divide $X^{p^n} - X$, but this is the same as showing that Z^2 does not divide $(Z + \alpha)^{p^n} - (Z + \alpha) = Z^{p^n} - Z + \alpha^{p^n} - \alpha$, but the latter polynomial has a coefficient of $-1 \neq 0$ for Z and so is not divisible by Z^2 . We conclude that since S has all roots of $X^{p^n} - X$ and this polynomial has p^n distinct roots, and so $|S| \geq p^n$. On the other hand since every element of S is a root of $X^{p^n} - X$ and this polynomial has at most p^n roots, we conclude that $|S| = p^n$ and so there exists a field of cardinality p^n . Thus we get the following theorem (the first part follows from Proposition D.5.2 and the second part follows from Proposition D.5.7).

Theorem D.5.9. *If $\mathbb{F}$ is a finite field, then it has characteristic p for some prime p and its cardinality is p^n for positive integer n . Conversely, for every prime p and positive integer n , there is a field of cardinality p^n .*

D.5.4 Uniqueness of finite fields

We start by proving that every finite field has a multiplicative generator. To do so we need to understand cyclic groups a bit better.

The cyclic group of order n is the group $\mathbb{Z}_n = \{0, \dots, n-1\}$ with addition modulo n being the binary operation. This group clearly has an element of order n (namely the number 1). Let $N^=(G, m)$ denote the number of elements of order exactly m in G and let $N(G, m)$ denote the number of elements of order dividing m in G . We have $N(G, m) = \sum_{k|m} N^=(G, k)$. For the cyclic group, we have for every $k|n$, $N(\mathbb{Z}_n, k) = k$ and $N^=(\mathbb{Z}_n, k) \geq 0$. (The latter is trivial and for the former see Exercise D.6.)

We now turn to understanding $(\mathbb{F}^*, \cdot)$ the group of non-zero elements of $\mathbb{F}$ under multiplication.

Lemma D.5.10. *Let $q = |\mathbb{F}|$ and $n = q - 1$. We claim that for every k dividing n , $N(\mathbb{F}^*, k) = N(\mathbb{Z}_n, k)$ and $N^=(\mathbb{F}^*, k) = N^=(\mathbb{Z}_n, k)$.*

Proof. The claim is straightforward for $N(\mathbb{F}^*, k)$. We have that every $\alpha \in \mathbb{F}^*$ is a root of the polynomial $X^n - 1$ and since $X^k - 1$ divides⁴ $X^n - 1$, k elements of $\mathbb{F}^*$ must be roots of this polynomial

⁴See Exercise D.7.

also. We thus have $N(\mathbb{F}^*, k) = k = N(\mathbb{Z}_n, k)$.

For the claim about $N^-(\mathbb{F}^*, k)$, we use induction and the inductive formula. We have $\sum_{\ell|k} N^-(\mathbb{F}^*, \ell) = N(\mathbb{F}^*, k) = k = N(\mathbb{Z}_n, k) = \sum_{\ell|k} N^-(\mathbb{Z}_n, \ell)$. But since by induction we have $N^-(\mathbb{F}^*, \ell) = N^-(\mathbb{Z}_n, \ell)$ for $\ell < k$, we may conclude that the remaining term $N^-(\mathbb{F}^*, k) = N^-(\mathbb{Z}_n, k)$. $\square$

We say that an element $\omega \in \mathbb{F}$ is primitive if $\omega^i \neq 1$ for $i < |\mathbb{F}| - 1$ and $\omega^{|\mathbb{F}|-1} = 1$. Since $N^-(\mathbb{F}^*, n)$ counts the number of primitive elements, Lemma D.5.10 implies that the number of primitive elements is at least one. Indeed, if p is the smallest prime divisor of n , then we have that $N^-(\mathbb{F}^*, n) = N(\mathbb{F}^*, n) - N(\mathbb{F}^*, n/p) - N(\mathbb{F}^*, p) = n - n/p - p > 0$, assuming $p < n/p$. Otherwise if $n = p^2$, then we have $N^-(\mathbb{F}^*, n) = N(\mathbb{F}^*, n) - N(\mathbb{F}^*, p) = n - p > 0$. If n itself is a prime then we have $N^-(\mathbb{F}^*, n) = N(\mathbb{F}^*, n) = n > 0$.

Proposition D.5.11. *Every finite field $\mathbb{F}$ has a primitive element. Consequently the multiplicative group is cyclic.*

We now describe a weaker form of special element in $\mathbb{F}$. Let $\mathcal{K}$ extend $\mathbb{F}$. We say that $\alpha \in \mathcal{K}$ is an $\mathbb{F}$ -generator for $\mathcal{K}$ if for every element $\beta \in \mathcal{K}$ there is a polynomial $p \in \mathbb{F}[X]$ such that $\beta = p(\alpha)$.

Proposition D.5.12. *Let $\mathcal{K}$ be a finite field and let ω be a primitive element in $\mathcal{K}$. Then for every subfield $\mathbb{F} \triangleleft \mathcal{K}$ we have that ω is an $\mathbb{F}$ -generator of $\mathcal{K}$. As a consequence, for every $\mathcal{K} \supset \mathbb{F}$ there is an $\mathbb{F}$ -generator in $\mathcal{K}$.*

Proof. Consider the lowest degree polynomial $p \in \mathbb{F}[X]$ such that $p(\omega) = 0$. Let $|\mathbb{F}| = q$ and $|\mathcal{K}| = q^n$.

We claim that $\deg(p) = n$. If $\deg(p) > n$, we have that $1, \omega, \omega^2, \dots, \omega^n$ are linearly independent over $\mathbb{F}$ and so $\mathcal{K}$ has size strictly larger than q^n . Now if $\deg(p) < n$, then consider the polynomials $X, X^2, X^3, \dots, X^{q^n-1}$ modulo $p \in \mathbb{F}[X]$. Since we have only $q^{\deg(p)}$ options for the residues, two of these must be equal modulo p and so there exist $i \neq j$ and $f \in \mathbb{F}[X]$ such that $X^i = X^j + p \cdot f$. Substituting $X = \omega$ yields $\omega^i = \omega^j + p(\omega)f(\omega) = \omega^j$. But this contradicts the assumption that ω is a primitive element.

Finally, note that every non-zero element $\beta \in \mathcal{K}$ can be written as the polynomial $q^j \pmod{p(X)}$ evaluated at $X = \omega$ for some $0 \leq j < q^n$. $\square$

Generators are useful in that they show that the only way to construct field extensions is via irreducible polynomials.

Proposition D.5.13. *Let $\mathcal{K} \supset \mathbb{F}$ and let α be an $\mathbb{F}$ -generator of $\mathcal{K}$. Then, if p is the minimal polynomial in $\mathbb{F}[X]$ such that $p(\alpha) = 0$, we have p is irreducible and $\mathcal{K}$ is isomorphic to $\mathbb{F}[X]/p$.*

Proof. Irreducibility of p follows from its minimality (see Exercise D.8). The isomorphism is obtained by fixing $\mathbb{F} \triangleleft \mathcal{K}$ and letting $\alpha \mapsto X$. We leave it to the reader to verify that this extends to an isomorphism (uniquely)– see Exercise D.9. $\square$

We are almost ready to prove uniqueness of finite fields. We need one more fact about irreducible polynomials to do so.

Proposition D.5.14. *If $f \in \mathbb{F}_p[X]$ is irreducible of degree n , then f divides $X^{p^n} - X$.*

Proof. Consider the field $\mathcal{K} = \mathbb{F}_p[X]/(f)$. This is a field of cardinality p^n and so every element $\alpha \in \mathcal{K}$ satisfies $\alpha^{p^n} = \alpha$. In particular $X \in \mathcal{K}$ also satisfies this, implying that $X^{p^n} - X = 0 \pmod{f}$ and so f divides $X^{p^n} - X$. $\square$

We now turn to proving uniqueness of finite fields.

Theorem D.5.15. *For every prime p and integer n , there is a unique field of cardinality p^n up to isomorphism.*

Proof. Suppose $\mathcal{K}, \mathbb{L}$ are both fields of cardinality p^n . Both fields contain a unique copy of $\mathbb{F}_p$, and by mapping $1_{\mathcal{K}}$ to $1_{\mathbb{L}}$ and extending additively, we get a partial isomorphism between these copies of $\mathbb{F}_p$. Now we show how to extend it. Let $\alpha \in \mathcal{K}$ be a $\mathbb{F}_p$ -generator and let $f \in \mathbb{F}_p[X]$ be its minimal polynomial. Since f is irreducible of degree n , we have that $f(X)$ divides the polynomial $X^{p^n} - X$ (see Proposition D.5.14).

Using the fact that $X^{p^n} - X = \prod_{\beta \in \mathbb{L}} (X - \beta)$, we conclude that $\mathbb{L}$ contains a root β of f . We assert (see Exercise D.10) that the map that sends $\alpha \mapsto \beta$ is an isomorphism from $\mathcal{K}$ to $\mathbb{L}$. $\square$

D.5.5 The Trace and Norm maps

We conclude this section with two basic polynomials that have some very nice regularity property when dealing with finite fields and their extensions.

Definition D.5.16. *Let $\mathbb{F} = \mathbb{F}_q$ and let $\mathcal{K} = \mathbb{F}_{q^n}$. Then the Trace function $\text{Tr} = \text{Tr}_{\mathcal{K} \rightarrow \mathbb{F}}$ is the function obtained by the evaluation of the polynomial $\text{Tr}(X) = X + X^q + X^{q^2} + \dots + X^{q^{n-1}}$. The Norm function is obtained by evaluation of $N(X) = X^{1+q+q^2+\dots+q^{n-1}}$.*

The Norm and Trace functions are important because they map elements of $\mathcal{K}$ to the subfield $\mathbb{F}$ and they do so in a nice uniform way. We mention some properties below.

Proposition D.5.17. *1. Trace is a $\mathbb{F}$ -linear map, i.e., for every $\alpha \in \mathbb{F}$ and $\beta, \gamma \in \mathcal{K}$, we have $\text{Tr}(\alpha \cdot \beta + \gamma) = \alpha \cdot \text{Tr}(\beta) + \text{Tr}(\gamma)$.*

2. Norm is multiplicative, i.e., $N(\beta \cdot \gamma) = N(\beta)N(\gamma)$

3. Trace is a q^{n-1} -to-one map from $\mathcal{K}$ to $\mathbb{F}$.

4. Norm is a $(q^n - 1)/(q - 1)$ -to-one map from $\mathcal{K}^$ to $\mathbb{F}^*$.*

Proof. 1. The $\mathbb{F}$ -linearity follows from the facts that $(\alpha\beta + \gamma)^{q^i} = \alpha^{q^i}\beta^{q^i} + \gamma^{q^i}$ and $\alpha^{q^i} = \alpha$.

2. The multiplicativity is obvious from definition.

3. For $\beta \in \mathcal{K}$ we have $\text{Tr}(\beta)^q = \beta^q + \dots + \beta^{q^n} = \beta^q + \dots + \beta^{q^{n-1}} + 1 = \text{Tr}(\beta)$ and so the range of Tr is $\mathbb{F}$. Since Tr is a polynomial of degree q^{n-1} it can take on any value in the range at most q^{n-1} times. But it has a domain of size q^n and range of size q , so it must take on every value exactly q^{n-1} times.

4. The is similar to Part (3) above. By Exercise D.11, we have that $N(\beta)^q = N(\beta)$ and furthermore note that by definition, $N(\beta)$ is non-zero iff $\beta \neq 0$. We then use the degree of N and counting to determine that it is a regular function on non-zero values. $\square$

The Trace function from $\mathcal{K} \rightarrow \mathbb{F}$ is especially important since it captures all $\mathbb{F}$ -linear maps from $\mathcal{K} \rightarrow \mathbb{F}$, as explained below.

Proposition D.5.18. *A function $L: \mathcal{K} \rightarrow \mathbb{F}$ is $\mathbb{F}$ -linear if and only if there exists $\lambda \in \mathcal{K}$ such that $L(\beta) = \text{Tr}(\lambda\beta)$ for every $\beta \in \mathcal{K}$.*

Proof. First note that $f(\beta) = \text{Tr}(\lambda\beta)$ is obviously $\mathbb{F}$ -linear since

$$f(\alpha\beta + \gamma) = \text{Tr}(\lambda(\alpha\beta + \gamma)) = \text{Tr}(\lambda\alpha\beta) + \text{Tr}(\lambda\gamma) = \alpha \text{Tr}(\lambda\beta) + \text{Tr}(\lambda\gamma) = \alpha f(\beta) + f(\gamma)$$

for every $\alpha \in \mathbb{F}$ and $\beta, \gamma \in \mathcal{K}$ (where in the above we used Proposition D.5.17). This concludes one direction of the proposition.

To see the converse we employ a *counting argument*. First note that if $\lambda \neq 0$ then the function $f_\lambda(\beta) = \text{Tr}(\lambda\beta)$ is not identically zero. (To see this, note that $f_\lambda(Z)$, viewed as a polynomial in Z has degree $|\mathcal{K}|/|\mathbb{F}|$ and it is a non-zero polynomial since the coefficient of Z is non-zero.) By linearity this implies that $f_\lambda \neq f_\tau$ if $\lambda \neq \tau$ since $f_\lambda - f_\tau = f_{\lambda-\tau} \neq 0$. So, including $\lambda = 0$, we have at least $|\mathcal{K}|$ distinct linear functions of the form $f_\lambda(\cdot)$. We now note there are also at most $|\mathcal{K}|$ such functions. To see this let $\beta_1, \dots, \beta_n \in \mathcal{K}$ be $\mathbb{F}$ -linearly independent elements of $\mathcal{K}$ (i.e., $\sum_{i=1}^n \alpha_i \beta_i \neq 0$ if $(\alpha_1, \dots, \alpha_n) \in \mathbb{F}^n \setminus \{0\}$). Since $\mathcal{K}$ is a degree n extension of $\mathbb{F}$ we know such a sequence exists and furthermore the β_i 's generate $\mathcal{K}$ in that for every $\beta \in \mathcal{K}$ there exist $\alpha_1, \dots, \alpha_n \in \mathbb{F}$ such that $\beta = \sum_i \alpha_i \beta_i$. We note that a linear function $L: \mathcal{K} \rightarrow \mathbb{F}$ is completely determined by its values at $\beta_1, \dots, \beta_n$, since for every $\beta = \sum_i \alpha_i \beta_i \in \mathcal{K}$ with $\alpha_1, \dots, \alpha_n \in \mathbb{F}$, we have $L(\beta) = L(\sum_i \alpha_i \beta_i) = \sum_i \alpha_i L(\beta_i)$. Thus the number of linear functions is upper bounded by $|\{(L(\beta_1), \dots, L(\beta_n)) \in \mathbb{F}^n\}| \leq |\mathbb{F}|^n = |\mathcal{K}|$. We conclude that these are exactly $|\mathcal{K}|$ functions that are $\mathbb{F}$ -linear from $\mathcal{K} \rightarrow \mathbb{F}$, and these are exactly the Trace functions. $\square$

D.6 Algorithmic aspects of Finite Fields

In this section we show how finite fields may be represented and field operations computed efficiently.

Let $q = p^t$ for prime p and positive integer t . We consider how to work with $\mathbb{F}_q$ — the field on q elements.

We start by noticing that if $O(q^2)$ space is not imposing, then four tables — one for addition, and one for multiplication, and one each for additive and multiplicative inverses would suffice for working with fields, with each field operation now requiring a single table look up. In what follows we give more succinct descriptions that still allow moderately fast (some polynomial in $\log q$) operations.

D.6.1 Prime Fields

We start with the case of $t = 1$. Here there is not much to do. The most natural representation of the field is by specifying the prime p which takes $\log_2 p + 1 = \log q + 1$ bits to specify. The most complex part of addition and multiplication is the computation of the remainder of the operation modulo p , and this takes at most $O((\log p)^2)$ steps by the naive method. More sophisticated algorithms can bring this complexity down to $O((\log p)(\log \log p)^2)$.

D.6.2 General fields as vectors

For general fields, we can adopt one of two approaches. The first of these uses less of the knowledge that we have about finite fields, but helps abstract away many issues. In this view we use the isomorphism between $\mathbb{F}_{p^t}$ and $\mathbb{F}_p^t$ to represent elements of the former as vectors in $\mathbb{F}_p^t$. This, thus represents elements of $\mathbb{F}_q$ by $O(\log q)$ bits which is nice. This also tells us how to add in $\mathbb{F}_q$ since it is simply coordinatewise $\mathbb{F}_p$ -addition. However this representation by itself is not sufficient to do $\mathbb{F}_q$ -multiplication. To multiply elements we enhance this representation by maintaining t^2 vectors $\mathbf{w}_{ij} \in \mathbb{F}_p^t$ with $\mathbf{w}_{ij} = \mathbf{e}_i \cdot \mathbf{e}_j$ where the $\mathbf{e}_i$'s are the unit vectors (so $\mathbf{e}_i = (0, \dots, 0, 1, 0, \dots, 0)$ is 1 in the i th coordinate and zero elsewhere). Now given $\mathbf{u} = (u_1, \dots, u_t)$ and $\mathbf{v} = (v_1, \dots, v_t)$ we can compute $\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^t \sum_{j=1}^t u_i v_j \mathbf{w}_{ij}$. This leads to a roughly $O(t^3 (\log p)^2) = O((\log q)^3)$ time algorithm for multiplying in $\mathbb{F}_q$ while requiring $O(t^3 \log p)$ bits to store p and the vectors $\mathbf{w}_{ij}$. While not the most efficient representation, this may afford a clean representation that may be sufficient in several settings.

D.6.3 General fields as polynomial rings

Our final representation uses the fact that the field $\mathbb{F}_{p^t}$ is isomorphic to $\mathbb{F}_p[X]/(g)$ for any irreducible polynomial g of degree t . Here, a field element is simply a polynomial in $\mathbb{F}_p[X]$ of degree strictly less than t which is maintained as a vector of coefficients. Addition is just coordinate-wise addition, whereas multiplication is polynomial multiplication followed by a remainder computation modulo g . Thus addition takes time $O(t(\log p)^2)$ while multiplication naively takes time $O(t^2(\log p)^2)$. The only ingredients that need to be remembered to do field operations are the integer p and the polynomial $g \in \mathbb{F}_p[X]$, all of which take $O(t \log p)$ bits. So this representation definitely outperforms the generic representation via vector spaces in almost all senses (though we might find the vector space view helpful when discussing certain operations with codes).

D.6.4 Finding primes and irreducible polynomials

The final question that remains to be discussed is how hard is to find the ingredients that describe a field. Of course, this depends on how the field is described, and the most natural one may be by giving the cardinality q of the field.

Given $q = p^t$, it is straightforward to enumerate all candidate (p, t) pairs such that $q = p^t$ — there are only $\log q$ possible values of t and thus $\log q$ such integers. Only one of these, the

one with the largest t could correspond to prime p . Testing if an integer is prime can be done efficiently with randomization, and thanks to a recent breakthrough [1] even deterministically in time polynomial in $\log q$.

When $t = 1$ no further work needs to be done. If $t > 1$ one needs to find an irreducible polynomial g of degree t and this can be a challenge. There are several possible solutions here:

Randomized It is known (and can actually be proved with a little effort, given the ingredients of this chapter) that a random polynomial $g \in \mathbb{F}_p[X]$ of degree t is irreducible with probability at least $1/t$. Furthermore, irreducibility can be tested (see next section) in time $\text{poly}(\log q)$. Thus repeatedly sampling random polynomials till an irreducible polynomial is found takes expected time $\text{poly}(\log q)$. (See Algorithm 5.1.1.)

Deterministic Shoup [140] gave an algorithm to deterministically find an irreducible polynomial of degree t in $\mathbb{F}_p[X]$ in time $\text{poly}(t, p)$. Notice that this dependence is slower than one may hope for in terms of p , but works well when p is small (say, smaller than t).

Explicit In some rare cases, i.e., a few choices of p and t , explicit polynomials are known that are irreducible. These may be used when the field size seems appropriate. One such family of irreducible polynomials is given in the following proposition.

Proposition D.6.1 ([106]). *Let $p = 2$ and $t = 2 \cdot 3^\ell$ for any non-negative integer ℓ . Then the polynomial $X^t + X^{t/2} + 1$ is irreducible in $\mathbb{F}_2[X]$.*

D.7 Algorithmic aspects of Polynomials

In this section we review basic facts about algorithmic aspects of manipulating polynomials. We start with basic tasks and move to more complex tasks ending with factoring and root-finding.

D.7.1 Adding, Multiplying, Dividing

Given two polynomials $f, g \in \mathbb{F}_q[X]$ of degree at most n , they can be added with $O(n)$ operations in $\mathbb{F}_q$ and no more needs to be said. f and g can also be multiplied with $O(n^2)$ operations by the standard *long multiplication*. Similarly the quotient and remainder obtained when dividing f by g can be computed with $O(n^2)$ operations using the *long division* algorithm. More efficient algorithms do exist for both these tasks making $O(n(\log n)^c)$ field operations, for some constant c . (See [161] for this and other references for this section.)

D.7.2 Greatest Common Divisor

Perhaps the most surprising algorithm in algebra is that of finding greatest common divisors (of integers or polynomials), and would be even more so, if it were not for over 2000 years of exposure. To explain, let us look at the definition of the problem.

Definition D.7.1 (Greatest Common Divisor). *Given polynomials $f, g \in \mathbb{F}[X]$, their greatest common divisor, denoted $\gcd(f, g)$, is the maximal degree polynomial $h(X)$ with leading coefficient being 1 such that h divides f and g .*

The natural algorithm for finding $\gcd(f, g)$ would be to factor f and g into irreducible factors, and then to take all common factors (with multiplicity) and take their product to get h . Unfortunately this reduces gcd to factoring which goes in the wrong direction. (As we will see below, factoring can also be solved efficiently for polynomials, but by reduction to gcd computation.)

But fortunately, we can employ Euclid's algorithm which uses the following algorithmic reduction: If $\deg(g) < \deg(f)$ and g does not divide f , then $\gcd(f, g) = \gcd(g, r)$ where $f = q \cdot g + r$ with $\deg(r) < \deg(g)$ is as given by the division algorithm. This simple fact turns out to be algorithmically effective reducing the (sum of the) degree of the polynomials in a single step of polynomial division, and thus leading to a polynomial time algorithm for finding the greatest common divisor.

Once again the steps of this algorithm can be combined in clever ways to get an implementation in $O(n(\log n)^c)$ time.

D.7.3 Factoring and Root-Finding

Finally, one of the most striking tasks related to polynomials that turns out to have a polynomial time algorithm is the factorization of polynomials. Polynomials, even multivariate ones, can be factored extremely efficiently with randomization and this is a consequence of many years of research in algebraic computing. We won't give the strongest results here, since even stating the result is non-trivial. For our purposes it will suffice to know that polynomials in $\mathbb{F}_q[X]$ of degree n can be factored in time $\text{poly}(n, \log q)$. We state this general result, and prove a very special case of it.

Theorem D.7.2. *There exists a constant c and a randomized algorithm running in expected time $O((n \log q)^c)$ that factors polynomials of degree n in $\mathbb{F}_q[X]$. Furthermore, if $q = p^t$ for prime t , then there is a deterministic algorithm with running time $O((npt)^c)$ for factoring.*

To give an idea behind this powerful algorithm, we consider a simple special case of root-finding.

Definition D.7.3 (Root-Finding Problem). *The input to the root finding problem is a polynomial $f \in \mathbb{F}_q[X]$ of degree at most n (given as a list of coefficients $f_0, \dots, f_n \in \mathbb{F}_q$). The task is to find all $\alpha \in \mathbb{F}_q$ that are roots of f , i.e., to output the set $\{\alpha \in \mathbb{F}_q \mid f(\alpha) = 0\}$.*

We now turn towards the root-finding algorithm. The algorithm relies crucially on the algorithm for computing greatest common divisors (mentioned in the previous section) and two additional facts. First we use the fact $X^q - X = \prod_{\alpha \in \mathbb{F}_q} (X - \alpha)$ to algorithmic advantage as follows.

Lemma D.7.4. *A polynomial $f \in \mathbb{F}_q[X]$ has a root in $\mathbb{F}_q$ if and only if $\gcd(f, X^q - X) \neq 1$.*

Proof. The proof is immediate. If f has a root α , then $X - \alpha$ divides $\gcd(f, X^q - X)$ and so their gcd can't be trivial. Conversely a factor of $X^q - X$ is of the form $\prod_{\alpha \in S} (X - \alpha)$ for some $S \subseteq \mathbb{F}_q$, and so $\gcd(f, X^q - X)$ must be of this form. If the gcd is non-trivial, then S must be non-empty implying that for every $\alpha \in S$ we have $X - \alpha$ divides f and thus f has a root in $S \subseteq \mathbb{F}_q$. $\square$

The step above is almost algorithmic, but to verify this, we need to stress that the gcd of f and $X^q - X$ can be computed in time polynomial in $\deg(f)$ and $\log q$. We explain how this can be done in the next few paragraphs, taking a detour on sparse polynomials. But assuming this can be done, this provides a natural starting point for a root finding algorithm. Given f we compute $g = \gcd(f, X^q - X)$. If $g \neq 1$, then we take the set S_1 of roots of g and the set S_2 of roots of f/g and output $S_1 \cup S_2$. The set S_2 can be computed recursively (since f/g has smaller degree than f), but for S_1 we need some new ideas to determine how to compute the roots of g , when g splits into linear and distinct factors over $\mathbb{F}_q$. To get to this point we will use the fact that $X^q - X$ splits into some high-degree sparse factors and this will turn out to be crucial to finding S . Indeed the sparsity of $X^q - X$ and its factors are heavily used concepts and we now take a detour to explain these effects.

Sparse high degree polynomials

We start with some terminology. We say that a polynomial $h \in \mathbb{F}[X]$ is t -sparse if at most t of its coefficients are non-zero. Every polynomial h is thus $(\deg(h) + 1)$ -sparse, but often the sparsity can be smaller, and this will be useful. One sparse polynomial that is already motivated by the earlier discussion is $X^q - X$, which is 2-sparse. We will see a few more below.

Lemma D.7.5. *Let $f \in \mathbb{F}[X]$ be a polynomial of degree n and let $h \in \mathbb{F}[X]$ be a t -sparse polynomial of degree D . Then $h \bmod f$ and $\gcd(f, h)$ can be computed in time $\text{poly}(n, t, \log D)$.*

Proof. It obviously suffices to compute $h \bmod f$ in time $\text{poly}(n, t, \log D)$ and then one can use Euclid's algorithm to compute $\gcd(f, h) = \gcd(f, h \bmod f)$ in time $\text{poly}(n)$. It turns out that if $h = \sum_{i=1}^t h_i X^{d_i}$ and we can compute $h_i X^{d_i} \bmod f$ in time $\text{poly}(n, \log d_i)$ for every i , then we can add the results in time $\text{poly}(n, t)$ to get $h \bmod f$. Finally, we note that $X^d \bmod f$ can be computed by repeated squaring. Let $d = \sum_{j=0}^{\log_2 d} d_j 2^j$. We can first compute the sequence of polynomials $g_j = X^{2^j} \bmod f = g_{j-1}^2 \bmod f$ by repeatedly squaring the output of the previous step. Then we can compute $X^d \bmod f = \prod_{j=0}^{\log_2 d} (g_j)^{d_j}$ by using $\log d$ more multiplications, yielding the desired result. $\square$

The lemma above shows that sparse polynomials can be used effectively. The following lemma shows that the central sparse polynomial also has sparse factorizations, and this will be useful later.

Proposition D.7.6. *1. Let $\mathbb{F}_q$ be a field of odd characteristic (and so q is odd). Then $X^q - X = X \cdot (X^{(q-1)/2} - 1) \cdot (X^{(q-1)/2} + 1)$. In particular $X^q - X$ factors into three 2-sparse polynomials of degree at most $q/2$.*

2. Let $q = 2^t$ for integer $t \geq 2$. Then $(X^q - X) = \text{Tr}(X) \cdot (\text{Tr}(X) - 1)$ where $\text{Tr}(X) = \text{Tr}_{\mathbb{F}_q \rightarrow \mathbb{F}_2}(X) = X + X^2 + X^4 + \dots + X^{2^{t-1}}$ is the Trace map from $\mathbb{F}_q$ to $\mathbb{F}_2$. In particular $X^q - X$ factors into two $(2 + \log_2 q)$ -sparse polynomials of degree $q/2$.

Proof. The case of odd q is obvious by inspection. Only aspect to be stressed is that $(q-1)/2$ is an integer.

For the case of even q , we use the fact that the trace map is a map from $\mathbb{F}_q$ to $\mathbb{F}_2$. So every $\alpha \in \mathbb{F}_q$ satisfies $\text{Tr}(\alpha) = 0$ or $\text{Tr}(\alpha) = 1$. It follows that $X - \alpha$ divides $\text{Tr}(X) \cdot (\text{Tr}(X) - 1)$ for every α . Consequently $X^q - X$ divides $\text{Tr}(X) \cdot (\text{Tr}(X) - 1)$. The identity $X^q - X = \text{Tr}(X) \cdot (\text{Tr}(X) - 1)$ now follows from the fact that both polynomials have the same degree and have leading coefficient 1. $\square$

The existence of such sparse polynomials with many roots is one of the remarkable aspects of finite fields and leads to many algorithmic effects. We demonstrate this by showing how this is utilized in root-finding.

Univariate Root finding algorithm

We now complete the root-finding algorithm. Recall that by letting $g = \gcd(f, X^q - X)$ we can reduce to the case of polynomials that split into distinct linear factors in $\mathbb{F}_q$. We now focus on this case. We will also focus on the case of odd q for simplicity, though all we will use is the fact that $X^q - X$ splits into sparse factors of degree at most $q/2$.

If we were lucky, then g would have two roots α and β with $X - \alpha$ dividing $(X^{(q-1)/2} - 1)$ and $X - \beta$ not dividing it. Then we would have that $g_1 = \gcd(g, X^{(q-1)/2} - 1)$ would be a non-trivial factor of g and we could recurse on g_1 and $g_2 = g/g_1$. The key to the randomized root-finding is that by an appropriate affine change of variables, we can try to arrange to be "lucky".

Specifically, fix $a \in \mathbb{F}_q^*$ and $b \in \mathbb{F}_q$ and let $g_{a,b}(X) = g((X - b)/a)$. We have the following proposition.

Proposition D.7.7. *Let $g \in \mathbb{F}_q[X]$ have $\alpha \neq \beta$ as its roots. Then we have:*

1. *The coefficients of $g_{a,b}$ can be computed efficiently given a , b and the coefficients of g .*
2. *$g_{a,b}$ has $a\alpha + b$ and $a\beta + b$ as its roots.*
3. *If $a \in \mathbb{F}_q^*$ and $b \in \mathbb{F}_q$ are chosen uniformly at random independently, then the probability that exactly one of $a\alpha + b$ and $a\beta + b$ is a root of $X^{(q-1)/2-1}$ is at least $1/2$.*

Proof. Parts (1) and (2) are straightforward to verify. For part (3) we note that for any pair of distinct elements $\gamma, \delta \in \mathbb{F}_q$ there is exactly one pair $a \in \mathbb{F}_q^*$ and $b \in \mathbb{F}_q$ such that $a\alpha + b = \gamma$ and $a\beta + b = \delta$. Since the fraction of distinct pairs $\gamma, \delta \in \mathbb{F}_q$ such that exactly one of them comes from a set of size $(q-1)/2$ (the set of roots of $X^{(q-1)/2} - 1$) is at least $1/2$ (the exact formula is $1/2 + 1/(2q)$) we have that the probability that exactly one of $a\alpha + b$ and $a\beta + b$ is a root of $X^{(q-1)/2-1}$ is at least $1/2$. $\square$

Algorithm D.7.1 ROOT-FIND($\mathbb{F}_q, f$)

INPUT: $\mathbb{F}_q, f(X) \in \mathbb{F}_q[X]$ OUTPUT: $\mathbb{F}_q$ roots of $f(X)$

```
1:  $g \leftarrow \gcd(f, X^q - X)$ 
2: IF  $g = 1$  THEN
3:   RETURN  $\emptyset$ 
4: RETURN LINEAR-ROOT-FIND( $\mathbb{F}_q, g$ )  $\cup$  ROOT-FIND( $\mathbb{F}_q, (f/g)$ )
```

Algorithm D.7.2 LINEAR-ROOT-FIND($\mathbb{F}_q, g$)

INPUT: $\mathbb{F}_q, g(X) \in \mathbb{F}_q[X]$ OUTPUT: $\mathbb{F}_q$ roots of $g(X)$ if $g(X)$ divides $X^q - X$

```
1: IF  $\deg(g) = 1$  THEN
2:   RETURN  $\{\alpha\}$  where  $g = X - \alpha$ 
3: REPEAT
4:   Pick  $a \in \mathbb{F}_q^*$  and  $b \in \mathbb{F}_q$  uniformly independently
5:    $g_{a,b} \leftarrow g((X - b)/a)$ 
6:    $h_1 \leftarrow \gcd(g_{a,b}, X^{(q-1)/2-1})$ 
7:    $g_1 \leftarrow h_1(aX + b)$ 
8: UNTIL  $0 < \deg(g_1) < \deg(g)$ 
9: RETURN LINEAR-ROOT-FIND( $\mathbb{F}_q, g_1$ )  $\cup$  LINEAR-ROOT-FIND( $\mathbb{F}_q, (g/g_1)$ )
```

We conclude by giving the full root-finding algorithm and summary of analysis of its runtime.

Lemma D.7.8. ROOT-FIND($\mathbb{F}_q, f$) outputs the multiset of roots of f in expected time $\text{poly}(n, \log q)$.

Proof. Let $n = \deg(f)$. It is straightforward to see that ROOT-FIND makes at most n calls to LINEAR-ROOT-FIND. (This is a very weak estimate, but we leave out optimizations here, in favor of simplicity.) By Proposition D.7.7, Part (3), we have that the loop in LINEAR-ROOT-FIND will be executed an expected constant number of times before a non-trivial split is found. Finally, the degrees of the polynomials in the two recursive calls add up to $\deg(g)$ and so this leads to a tree of recursive calls of size at most n with each internal node and leaf performing $\text{poly}(n, \log q)$ work (to compute the various gcds, and the transformation of variables). Thus the overall expected running time is $\text{poly}(n, \log q)$. $\square$

For the curious reader we also give a brief sketch of the deterministic algorithm running in time $\text{poly}(n, p, t)$ where $q = p^t$ for prime p . Given $g(X) \in \mathbb{F}_q[X]$ this algorithm first find a polynomial $f(X)$ with $0 < \deg(f) < \deg(g)$ such that $f(X)^p - f(X) \equiv 0 \pmod{g(X)}$. The reason this can be found efficiently is that the search for f is a linear system over $\mathbb{F}_p$ (we omit the details of this step). And the reason such a polynomial even exists is that if $g(X) = g_1(X) \cdot g_2(X)$ where g_1 and g_2 are relatively prime then choosing any $a \neq b \in \mathbb{F}_p$ and letting $f(X) = a \pmod{g_1(X)}$ and $f(X) = b \pmod{g_2(X)}$ yields such a polynomial. By the Chinese Remainder Theorem such a polynomial exists and has degree less than that of $g_1(X) \cdot g_2(X)$. So the linear system we wish to solve has a solution and in such a case a solution can be found efficiently. Having found such an f , we write $f(X)^p - f(X) = \prod_a (f(X) - a)$ and note that $\gcd(g(X), f(X) - a)$ must be nontrivial for some $a \in \mathbb{F}_p$, since $\gcd(g(X), \prod_a (f(X) - a)) = g(X)$, but for every a , $g(X)$ does not divide $f(X) - a$. Thus enumerating over all $a \in \mathbb{F}_p$ and computing gcd's gives a non-trivial factorization of g . (This settles the problem when g has a non-trivial factorization into relatively prime elements g_1 and g_2 , which is true in our case when we are only interested in finding all the roots and work with $\gcd(g(X), X^q - X)$.) This leads to a deterministic algorithm running in time $\text{poly}(n, p, t)$.

Bivariate Root Finding

Given a bivariate polynomial $R(X, Y) \in \mathbb{F}_q[X, Y]$, we say that a polynomial $P(X)$ is a root of $R(X, Y)$ if $Y - P(X)$ divides $R(X, Y)$. We show below how to solve the root-finding problem for bivariate polynomials in polynomial time.

Theorem D.7.9. *There exists a randomized algorithm that, given a bivariate polynomial $R(X, Y) \in \mathbb{F}_q[X, Y]$ of degree at most D (say, as a list of coefficients), outputs a list of all its roots in expected time polynomial in D and $\log q$. If $q = p^t$, there also exists a deterministic algorithm to output all roots in time polynomial in p, t and D .*

Proof. The algorithm uses a simple reduction to the univariate case. We find a monic irreducible polynomial of $F(X)$ of degree N where $D < N \leq O(D)$. We then consider the field $\mathbb{F}_Q = \mathbb{F}_q[X] \pmod{F(X)}$ and view $R(X, Y)$ as a polynomial $R_X(Y) \in \mathbb{F}_Q[Y]$. We find the roots of $R_X(Y)$

(using the univariate root-finding algorithm from Lemma D.7.8 or the deterministic root-finder implied by Theorem D.7.2). Let $\alpha_1, \dots, \alpha_s \in \mathbb{F}_Q$ be the roots of R_X . Using $\mathbb{F}_Q = \mathbb{F}_q[X] \bmod F(X)$ we interpret $\alpha_1, \dots, \alpha_s$ as polynomials $A_1(X), \dots, A_s(X) \in \mathbb{F}_q[X]$. We report all $A_i(X)$ such that $Y - A_i(X)$ divides $R(X, Y)$. This completes the description of the algorithm and we argue correctness and run time below.

Since the algorithm checks if $A_i(X)$ is a root before outputting it, it is clear that it outputs a subset of the roots. To prove correctness we only need to show every root is included in the output. In particular we need to show that if $Y - P(X)$ divides $R(X, Y)$ and $\alpha = P(X) \bmod F(X)$ represents the corresponding element of $\mathbb{F}_Q$ then $Y - \alpha$ also divides $R_X(Y)$. Let $h(X, Y) \in \mathbb{F}_q[X, Y]$ be such that $R(X, Y) = h(X, Y)(Y - P(X))$. Let $h_X(Y) = h(X, Y) \bmod F(X)$ be an element of $\mathbb{F}_Q[Y]$. We now have $h_X(Y) \cdot (Y - \alpha) = R(X, Y) \bmod F(X) = R_X(Y)$, as desired. This proves correctness.

To argue the run time, note that the algorithm $F(X)$ can be found in expected time polynomial in N and $\log q$, or deterministically in time poly in N , p and t . Once F is found the univariate root-finding takes a number of steps that is polynomial in N and $\log Q = N \log q$, where each step is a field operation in $\mathbb{F}_Q$ which may take time polynomial in $N \log q$. Using $N = D + 1$, we get that the overall expected run time remains a polynomial in D and $\log q$. Similarly the deterministic run time is also polynomial in D , t and p , where we use the deterministic root-finder implied by Theorem D.7.2. $\square$

D.8 Exercises

Exercise D.1. Let R be a commutative ring. Then prove the following:

1. If a is a unit in R , then $b \cdot a = 0$ if and only if $b = 0$.
2. Using the previous part or otherwise, prove Proposition D.3.6.

Exercise D.2. Argue that every finite field $\mathbb{F}$ has a finite characteristic $\text{char}(\mathbb{F})$.

Exercise D.3. Let $\mathbb{F}$ be a field with p elements for prime p . Argue that the map $1_{\mathbb{F}} \rightarrow 1$ can be extended to an isomorphism between $\mathbb{F}$ and $\mathbb{Z}_p$.

Exercise D.4. Let G be an abelian group with identity 1 and let $a \in G$.

1. Argue that the map $x \rightarrow a \cdot x$ for $x \in G$ is a bijection.
2. Argue that

$$\prod_{x \in G} x = a^n \cdot \prod_{x \in G} x.$$

3. Using the previous part or otherwise prove Proposition D.5.4 for abelian groups.

Exercise D.5. Let p be a prime and let $0 \leq i \leq p$. Then show that

$$\binom{p}{i} \bmod p = \begin{cases} 1 & \text{if } i = p \text{ or } i = 0 \\ 0 & \text{otherwise} \end{cases}.$$

Exercise D.6. In this exercise we will argue that for every k that divides n , we have $N(\mathbb{Z}_n, k) = k$, i.e. show that the number of elements of $\mathbb{Z}_n$ that have an order that divides k is exactly k . Consider the following:

1. Prove that

$$S_k = \left\{ a \cdot \frac{n}{k} \mid 0 \leq a < k \right\}$$

is a sub-group of $\mathbb{Z}_n$.

2. Argue that any $b \in \mathbb{Z}_n$ that has an order that divides k satisfies, $k \cdot b \bmod n = 0$.
3. Argue that any $b \in \mathbb{Z}_n \setminus S_k$ it must be the case that $k \cdot b \bmod n \neq 0$.
4. Argue that any $b \in S_k$ has an order that divides k .
5. Using the above parts or otherwise, argue that S_k contains all elements of $\mathbb{Z}_n$ with an order that divides k . Conclude that $N(\mathbb{Z}_n, k) = k$.

Exercise D.7. If k divides n , then show that $X^k - 1$ divides $X^n - 1$.

Exercise D.8. Let $\mathcal{K} \supset \mathbb{F}$ and let α be an $\mathbb{F}$ -generator of $\mathcal{K}$. Let p be the minimal polynomial in $\mathbb{F}[X]$ such that $p(\alpha) = 0$. Argue that p is irreducible.

Exercise D.9. Let $\mathcal{K} \supset \mathbb{F}$ and let α be an $\mathbb{F}$ -generator of $\mathcal{K}$. Let p be the minimal polynomial in $\mathbb{F}[X]$ such that $p(\alpha) = 0$. Argue that there is an isomorphism between $\mathcal{K}$ and $\mathbb{F}[x]/p$ obtained by fixing $\mathbb{F} \triangleleft \mathcal{K}$ and letting $\alpha \mapsto x$, which can be extended to all other elements.

Exercise D.10. Using notation in proof of Theorem D.5.15, prove that the map $\alpha \mapsto \beta$ can be extended to an isomorphism between $\mathcal{K}$ and $\mathbb{L}$.

Exercise D.11. Argue that for any $\beta \in \mathbb{F}_{q^n}$, the norm function satisfies $N(\beta)^q = N(\beta)$.

Appendix E

Some Information Theory Essentials

We used the notions of Shannon entropy and conditional entropy in Chapter 6. This appendix collects some basic facts relating to entropy and mutual information as a quick refresher. It is very elementary and can be skipped by readers with basic familiarity with information theory.

E.1 Entropy

Let X be a discrete random variable, say,

$$X \sim \begin{cases} a & \frac{2}{3} \\ b & \frac{1}{6} \\ c & \frac{1}{6} \end{cases}$$

and define $H(X)$ to be the entropy of the random variable X . We'd like the entropy to measure the *amount of information conveyed* by the value of the random variable X , where the amount of information is, roughly speaking, the amount of surprise we get from this random variable.

Suppose we had such a function $S : [0, 1] \rightarrow \mathbb{R}^+$ that takes a probability and maps it to a non-negative real. Let's think about some natural properties of S .

Suppose we told you that X is a , b , or c . Are you surprised? No—because this occurs with probability 1 and we already know this. So therefore we'd like $S(1) = 0$: flipping a two-headed coin doesn't give us any surprise.

Now suppose we told you that $X = a$. This is a little bit surprising, but $X = b$ is more surprising because this is a rarer event. So we'd like S to satisfy $S(p) > S(q)$ if $p < q$.

We'd also like $S(x)$ to be a continuous function of x , because we'd like the surprise to not exhibit jumps (if we had instead

$$X \sim \begin{cases} a & \frac{2}{3} + 10^{-6} \\ b & \frac{1}{6} - 10^{-6} \\ c & \frac{1}{6} \end{cases}$$

our impression of the “surprise” of a should not be much different than the original X).

Now suppose X_1 and X_2 are two independent instantiations of X . It's natural for our surprise from $X_1 = a$ and $X_2 = b$ to be additive (as each event “adds” to our surprise):

$$S\left(\frac{2}{3} \cdot \frac{1}{3}\right) = S\left(\frac{2}{3}\right) + S\left(\frac{1}{3}\right)$$

which means we must have $S(pq) = S(p) + S(q)$.

Exercise E.1.1. The only S that satisfies the above requirements is $S(p) = c \log_2\left(\frac{1}{p}\right)$, $c > 0$. A convenient normalization constant is that we are unit surprised by a fair coin flip ($S(1/2) = 1$), which sets $c = 1$.

Definition E.1.2 (Entropy). The entropy of a discrete random variable X , denoted $H(X)$, is the average surprise for an outcome of X :

$$\mathbb{E}_x \left(\log_2 \frac{1}{\Pr(X=x)} \right) = \sum_{x \in \text{supp}(X)} p(x) \log_2 \frac{1}{p(x)},$$

where we define $0 \log_2 \frac{1}{0} = 0$ and $\text{supp}(X)$ is the support of X (i.e. the values x such that $\Pr[X=x] \neq 0$).

We have the following obvious facts:

1. $H(X) \geq 0$; $H(X) = 0$ if and only if X is deterministic.
2. Let $X \in \{0, 1\}$ be such that $X = 1$ with probability p and 0 with probability $1 - p$. In this case

$$H(X) = p \log_2\left(\frac{1}{p}\right) + (1-p) \log_2\left(\frac{1}{1-p}\right) = H_2(X),$$

which we have seen earlier in the book.

3. Let X be uniform on $\{a_1, \dots, a_n\}$. Then, note that

$$H(X) = \sum_{i=1}^n \frac{1}{n} \cdot \log_2\left(\frac{1}{1/n}\right) = \log_2 n.$$

Lemma E.1.3. $|\text{supp}(X)| = n$ implies that $H(X) \leq \log_2 n$.

Proof. We use Jensen's inequality: for convex functions f , we have $\mathbb{E}[f(x)] \leq f(\mathbb{E}[x])$ (this implies $\mathbb{E}[X^2] \geq \mathbb{E}[X]^2$ and $\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$). Since we have

$$\begin{aligned} H(X) &= \mathbb{E}_{x \leftarrow X} \left[\log \frac{1}{p(x)} \right] \\ &\leq \log \left(\mathbb{E}_{x \leftarrow X} \frac{1}{p(x)} \right) = \log n \end{aligned}$$

where we have applied Jensen's inequality to the expression with $f(x) = \log x$ while using the random variable $Z \sim 1/p(x)$ w.p. $p(x)$:

$$H(X) = \mathbb{E}_Z [\log Z].$$

□

Communication. Let's see if we can derive the binary entropy function in a different setting. Suppose $X \leftarrow \text{Bernoulli}(1/2)$, then $H(X) = 1$. And if $X \leftarrow \text{Bernoulli}(1)$, then $H(X) = 0$. Now suppose $X \leftarrow \text{Bernoulli}(p)$, and we have $X_1, X_2, \dots, X_n$ iid samples from this distribution, represented as a bitstring $\{0, 1\}^n$. A way to communicate this in an efficient way is that we first send k , the number of 1 bits in the string, and then use $\lceil \log_2 \binom{n}{k} \rceil$ bits to reveal which subset those 1s go in. The expected number of bits communicated with this scheme is

$$\mathbb{E}(\text{number of bits}) = \underbrace{\lceil \log_2 n \rceil}_{\text{sending } k} + \underbrace{\sum_{k=0}^n \binom{n}{k} p^k (1-p)^{n-k} \left\lceil \log_2 \binom{n}{k} \right\rceil}_{\text{number of bits required for each } k}.$$

What we are interested is the average fraction of bits we need as the number of bits as n grows large: $\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E}(\text{number of bits})$. Dividing our expression by n , we can omit the first term ($\log n/n \rightarrow 0$ as $n \rightarrow \infty$) and we can use Stirling's approximation (or the intuitive concept that if k is far from pn the term in the sum is close to zero) to obtain that

$$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E}(\text{number of bits}) = h(p) = p \log_2 \frac{1}{p} + (1-p) \log_2 \frac{1}{1-p},$$

which tells us that we can take $H(\text{Bernoulli}(p))$ bits, on average, to communicate a $\text{Bernoulli}(p)$ random variable.

Non-binary random variable. We can extend this definition of entropy in the communications context to non-binary random variables. Suppose $X \leftarrow \{a_1, \dots, a_n\}$. Suppose we have related variables

$$Z_1 = \begin{cases} 1 & X = a_1 \\ 0 & \text{else} \end{cases}$$

and

$$Z_2 = \begin{cases} a_2 & \frac{p_2}{1-p_1} \\ \vdots & \\ a_n & \frac{p_n}{1-p_1} \end{cases}.$$

We can break

$$H(X) = H(Z_1) + \Pr(Z_1 = 0) H(Z_2),$$

as we need to communicate Z_1 and then, if Z_1 is 0, we need to communicate Z_2 . Since we know how to compute $H(Z_1)$ from the binary case, we can recursively use this to obtain $H(X) = \sum_{i=1}^n p_i \log 1/p_i$, which is the same expression as we got before.

E.2 Joint and conditional entropy

If our variable Z is actually a pair of random variables, i.e. $Z = (X, Y)$, then we already have defined $H(Z)$. However, as it'll become clear shortly it is beneficial to explicitly (re)define the *joint entropy* $H(X, Y)$:

Definition E.2.1 (Joint entropy). Let X and Y be two possibly correlated random variables. The joint entropy of X and Y , denoted $H(X, Y)$, is

$$H(X, Y) = \sum_{x,y} p(x, y) \log \frac{1}{p(x, y)},$$

where $p(x, y)$ is defined to be $\Pr(X = x \wedge Y = y)$.

If X and Y are independent, $p(x, y) = p(x)p(y)$ and

$$H(X, Y) = \sum_{x,y} p(x)p(y) \left(\log \frac{1}{p(x)} + \log \frac{1}{p(y)} \right) = H(X) + H(Y).$$

In general,

$$H(X, Y) = \sum_{x,y} p(x, y) \log \frac{1}{p(x)p(y|x)},$$

where $p(y|x) = \Pr(Y = y|X = x)$.

We can then do the following calculation:

$$\begin{aligned} H(X, Y) &= \sum_{x,y} p(x, y) \log \frac{1}{p(x)p(y|x)} \\ &= \sum_{x,y} p(x, y) \log \frac{1}{p(x)} + \sum_{x,y} p(x, y) \log \frac{1}{p(y|x)} \\ &= \sum_x p(x) \log \frac{1}{p(x)} + \sum_x p(x) \sum_y p(y|x) \log \frac{1}{p(y|x)} \\ &= H(X) + \sum_x p(x) H(Y|X = x) \\ &= H(X) + \mathbb{E}_x[H(Y|X = x)]. \end{aligned}$$

This motivates the definition of conditional entropy:

Definition E.2.2 (Conditional entropy). The conditional entropy of Y given X is

$$H(Y|X) = \mathbb{E}_x[H(Y|X = x)].$$

Our calculation then shows this lemma:

Lemma E.2.3. $H(X, Y) = H(X) + H(Y|X)$.

Intuitively, this says that how surprised we are by drawing from the joint distribution of X and Y is how surprised we are by X plus how surprised we are by Y given that we know X already.

Note that if X and Y are independent, $H(Y|X) = H(Y)$ and $H(X, Y) = H(X) + H(Y)$.

Recall the chain rule for probability: $p(x, y) = p(x)p(y|x)$, or, more generally,

$$p(x_1, \dots, x_n) = p(x_1)p(x_2|x_1) \dots p(x_n|x_1, \dots, x_{n-1}).$$

There is a similar chain rule for entropy:

Theorem E.2.4 (Chain rule). For random variables X , Y , and Z ,

$$H(X, Y, Z) = H(X) + H(Y|X) + H(Z|X, Y).$$

For n random variables $X_1, \dots, X_n$,

$$H(X_1, X_2, \dots, X_n) = H(X_1) + H(X_2|X_1) + \dots + H(X_n|X_1, X_2, \dots, X_{n-1}).$$

The log in the definition of entropy changes the multiplication in the probability chain rule to addition. Also, the order of the random variables does not matter. For example, it also holds that

$$H(X, Y) = H(Y) + H(X|Y).$$

Note that $H(X|X) = 0$.

Example E.2.5. Let X be a random variable that is uniform on $\{0, 1, 2, 3\}$. Let $Y = X \bmod 2$.

Clearly, $H(X) = 2$.

$H(Y) = 1$ since Y is uniform on $\{0, 1\}$.

$H(X|Y) = 1$ because knowing Y tells us if X is odd or even.

$H(Y|X) = 0$ since knowing X tells us the exact value of Y .

$H(X, Y) = 2$ because X tells us everything about X and Y .

Intuitively, it seems like conditioning should never increase entropy: knowing more should never increase our surprise. This is indeed the case:

Lemma E.2.6 (Conditioning cannot increase entropy). $H(Y|X) \leq H(Y)$.

Proof. First, we have that

$$\begin{aligned} H(Y|X) - H(Y) &= H(X, Y) - H(X) - H(Y) \\ &= \sum_{x,y} p(x, y) \log \frac{1}{p(x, y)} - \sum_x p(x) \log \frac{1}{p(x)} - \sum_y p(y) \log \frac{1}{p(y)}. \end{aligned}$$

Since $p(x) = \sum_y p(x, y)$ and $p(y) = \sum_x p(x, y)$,

$$H(Y|X) - H(Y) = \sum_{x,y} p(x, y) \log \frac{p(x)p(y)}{p(x, y)}.$$

We now define Z to be a random variable taking value $\frac{p(x)p(y)}{p(x, y)}$ with probability $p(x, y)$, so

$$\begin{aligned} H(Y|X) - H(Y) &= \mathbb{E}_{x,y}[\log Z] \\ &\leq \log \mathbb{E}[Z] \quad \text{by Jensen's Inequality} \\ &= \log \left(\sum_{x,y} p(x, y) \frac{p(x)p(y)}{p(x, y)} \right) \\ &= \log \left(\left(\sum_x p(x) \right) \left(\sum_y p(y) \right) \right) \\ &= \log 1 \\ &= 0. \end{aligned}$$

□

As a corollary, we have a statement similar to the union bound:

Corollary E.2.7. For random variables X and Y ,

$$H(X, Y) \leq H(X) + H(Y).$$

More generally, for random variables $X_1, \dots, X_n$,

$$H(X_1, \dots, X_n) \leq \sum_{i=1}^n H(X_i).$$

Exercise E.2.8. For a random variable X with support size n , we can think of entropy as a function from $[0, 1]^n$ to $\mathbb{R}_{\geq 0}$. If X takes on n different values with probabilities $p_1, \dots, p_n$, then for $\mathbf{p} = (p_1, \dots, p_n)$, $H(\mathbf{p}) = \sum_{i=1}^n p_i \log \frac{1}{p_i}$. Show that $H(\mathbf{p})$ is a concave function, i.e., show

$$H(\lambda \mathbf{p} + (1 - \lambda) \mathbf{q}) \geq \lambda H(\mathbf{p}) + (1 - \lambda) H(\mathbf{q})$$

for all $\lambda \in [0, 1]$, $\mathbf{p}, \mathbf{q} \in [0, 1]^n$.

E.3 Mutual information

Definition E.3.1 (Mutual information). The mutual information between random variables X and Y , denoted $I(X; Y)$, is

$$I(X; Y) = H(X) - H(X|Y).$$

Intuitively, mutual information is the reduction in the uncertainty of X that comes from knowing Y .

We can write $I(X; Y)$ in several other equivalent ways:

$$\begin{aligned} I(X; Y) &= H(X) - H(X|Y) \\ &= H(X) - (H(X, Y) - H(Y)) \\ &= H(X) + H(Y) - H(X, Y) \\ &= H(Y) - H(Y|X) \\ &= I(X; Y) \end{aligned}$$

Note that $I(X; Y) = I(Y; X)$.

The next lemma follows from the fact that conditioning cannot increase entropy.

Lemma E.3.2. $I(X; Y) \geq 0$.

Also, if X and Y are independent, $I(X; Y) = 0$.

Example E.3.3. Consider X and Y as defined in Example E.2.5. Then

$$I(X; Y) = H(X) - H(X|Y) = 2 - 1 = 1.$$

Figure E.1: Relationship between entropy, joint entropy, conditional entropy, and mutual information for two random variables.

Example E.3.4. Let the Z_i 's be i.i.d. random variables that are uniform over $\{0, 1\}$. Let $X = Z_1 Z_2 Z_3 Z_4 Z_5$ and $Y = Z_4 Z_5 Z_6 Z_7$. Then $I(X; Y) = 2$ since X and Y have 2 bits in common.

We show the relationship between entropy, joint entropy, conditional entropy, and mutual information for two random variables X and Y in Figure 5.1.

We can also define a conditional version of mutual information.

Definition E.3.5 (Conditional mutual information). The conditional mutual information between X and Y given Z is

$$\begin{aligned} I(X; Y|Z) &= H(X|Z) - H(X|Y, Z) \\ &= H(Y|Z) - H(Y|X, Z). \end{aligned}$$

Exercise E.3.6. Prove the chain rule for mutual information:

$$I(X_1, X_2, \dots, X_n; Y) = \sum_{i=1}^n I(X_i; Y|X_1, X_2, \dots, X_{i-1}).$$

Note that the order of the X_i 's does not matter.

Exercise E.3.7. It is not true that conditioning never increases mutual information. Give an example of random variables X, Y, Z where $I(X; Y|Z) > I(X; Y)$.